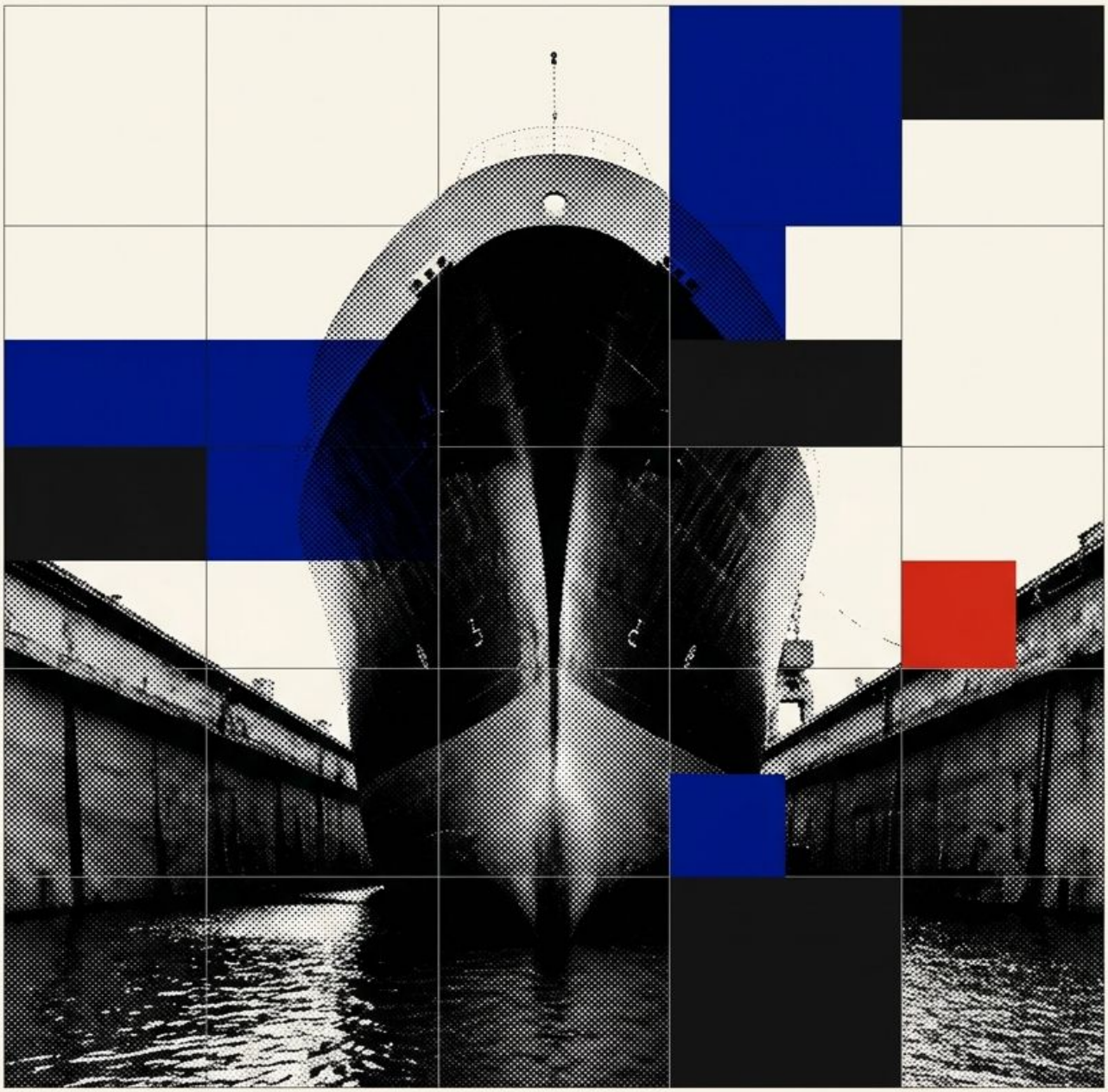
Governing AI Agents: The Harbor, the Person, and the Economy

A Textbook of Concurrency, Identity, and Accountable Autonomous Work

Erich Owens · Curiositech · Textbook Edition



Governing AI Agents: The Harbor, the Person, and the Economy

A Textbook of Concurrency, Identity, and Accountable Autonomous Work

For autonomous work to remain accountable, each action needs an authority, a record, and someone responsible for its consequences.

This book follows that claim from one machine and one operator to work shared across principals, economic interests, and mutually distrustful systems. Its eight chapters are ordered by what they stand on; each proof chapter follows the chapter whose promises it keeps.

Erich Owens · Curiositech LLC · engineering@portdaddy.dev
Curiositech · Version 4.0 (Textbook Edition) · September 2026 · https://portdaddy.dev/whitepaper

"Nature (the art whereby God hath made and governes the world) is by the art of man, as in many other things, so in this also imitated, that it can make an Artificial Animal... For by art is created that great Leviathan *called a* Common-Wealth*, or* State *(in latine* Civitas*) which is but an Artificiall Man; though of greater stature and strength than the Naturall, for whose protection and defence it was intended."*

— **Thomas Hobbes**, *Leviathan* (1651), Introduction

"Error is viewed, therefore, not as an extraneous and misdirected or misdirecting accident, but as an essential part of the process under consideration—its importance in the synthesis of automata being fully comparable to that of the factor which is normally considered, the intended and correct logical structure."

— **John von Neumann**, *Probabilistic Logics and the Synthesis of Reliable Organisms from Unreliable Components* (1956)

Abstract

Two workers can agree on a task and still overwrite each other's work. A passing test can concern the wrong revision. A process can disappear while its obligations remain. This book studies the mechanisms needed to distinguish these cases and make coordinated work inspectable.

The argument starts with a local transaction boundary and follows authority through delegation, confined execution, human supervision, process succession, exchange and federation. Worked examples establish the problem before the formal statements. Theorems name their assumptions; finite checks name their models; proposed institutions remain separate from implemented paths.

The central construction is a durable coordination kernel with explicitly mediated effects. It is one way to preserve the stated invariants, not a proof that every system needs the same architecture. Cryptographic evidence can bind records to keys without making their descriptions true. Economic deterrence requires credible detection and enforcement. Summaries help readers navigate retained evidence without reconstructing information they omit. These limits are part of the argument, because later mechanisms depend on them.

Reader	**Part I: Ground Truth** Chapters 1–3	**Part II: The Cost of Seeing** Chapter 4
Practitioner	Skim the mechanisms	**Close read:** consent and digest with zoom
Systems engineer	**Close read:** admission, delegation, confinement	**Close read:** evidence and inspection
Security reviewer	**Close read:** threat model and trust boundaries	Skim the supervision interface
Mechanism designer	Skim the local guarantees	Read attention economics
Institutional theorist	Skim the enforcement boundary	Read consent and legibility

Use an express-lane paragraph to locate a claim, then read its assumptions and evidence label. If the notation is unfamiliar, start with the worked example and return to the formal statement. When resuming after a break, the chapter question and the last example provide a useful starting point. Worked numbers name their evidence. [verified] refers to the stated check or reproducible calculation; [verified arithmetic] certifies only the calculation. [internal] denotes a chapter derivation or a project-reported run. Read the accompanying source and assumptions: none of these tags alone establishes deployed behavior. Exercises come in three kinds, Check, Trace, and Open, rated one to three; their solutions are collected at the back of the book, and each exercise and solution links to the other.

Cross-references are live: chapter titles, numbered results, figures, tables, and contents entries lead to their destinations. Full source entries appear in the margin at first use; later short citations link back to those entries.

Reader	**Part III: What Survives the Restart** Chapter 5	**Part IV: Trade Between Strangers** Chapters 6–8
Practitioner	**Close read:** role and person	Read exchange and remedies
Systems engineer	Read continuity and recovery	**Close read:** settlement and federation
Security reviewer	Skim the identity argument	**Close read:** verification scope and failure cases
Mechanism designer	Read reputation and reset costs	**Close read:** markets, bonds and conservation
Institutional theorist	**Close read:** personhood and continuity	Read adjudication and local sovereignty

Figure 1.
Five first-reading routes through the four parts. Every cell names its emphasis; close reading, ordinary reading and skimming are editorial suggestions, not prerequisites or measured learning outcomes.

Contents

The complete argument, in 8 chapters.

Every chapter, section and subsection is listed here, followed by solutions, the result atlas, reference material and image credits. Titles and page numbers lead directly to their place in the book.



0 Foundations and Prerequisites **2**

Pre-LLM multi-agent theory, transformer KV-cache asymptotics, and why generative models cannot act as their own kernel.

0.1 Introduction: The Unit of Account, Agency, and the Kernel Gap 5
0.1.1 The Unit of Account is the Work 5
0.1.2 Defining the Agent and the Kernel 5
0.1.3 The Dual Lineage of Autonomous Agency 6
0.1.4 Proposals and checked effects 6
0.1.5 Eight Propositions of Accountable Coordination 6
0.1.6 What Counts as Knowing: Epistemic Bounds and Evidence Channels 16

0.2 Classical Multi-Agent Systems Foundations 17
0.2.1 Intentional Agency and Practical Rationality 17
0.2.2 The Rao–Georgeff BDI Temporal Logic (BDI_{CTL^*}) 18
0.2.3 Formal Commitment Strategies 19
0.2.4 Operational Semantics of AgentSpeak(L) and Normative Extensions 20
0.2.5 Concurrency Foundations: Hewitt–Agha Actors vs. Hoare's Communicating Sequential Processes 23
0.2.6 From Asynchronous Isolation to Synchronous Rendezvous 24
0.2.7 Communicative Action and Speech Acts (FIPA-00037) 26
0.2.8 Task Allocation: Smith's Contract Net Protocol (1980) 28
0.2.9 Dynamic Epistemic Logic and Big Brother Logic 31
0.2.10 Game-Theoretic Foundations and Mechanism Design 34
0.2.11 Social Choice Theory and Arrow's Impossibility Theorem 35
0.2.12 The Ostrom Institutional Commons 38

0.3 The Physics of Modern LLM Execution: What an Operating Kernel Must Know 39
0.3.1 Transformer Mechanics: Self-Attention, Quadratic Complexity, and the KV-Cache 39
0.3.2 Training Regimes and the Alignment Illusion 42
0.3.3 Sampling and Conditional Failure Hazards 42
0.3.4 The Context Window as Volatile, Lossy RAM 43

0.4 Modern Coding Agent Architectures, Planning, and Search 45
0.4.1 The Contemporary Landscape: Systems Comparison 45
0.4.2 Closed-Loop Deliberation and Partial Observation 45
0.4.3 Deliberation, Planning, and Search Frameworks 46
0.4.4 Cognitive Control, Test-Time Compute, and Process Reward Models 47
0.4.5 Multi-Agent Formal Verification and Benchmarking 47

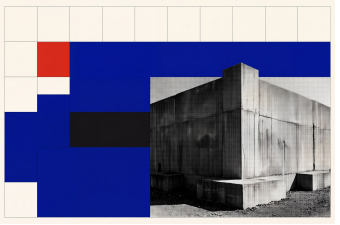
0.5 Tools, Protocols, Skills, and Hooks 48
0.5.1 Model Context Protocol (MCP) as the Substrate 48
0.5.2 Context Economics and Progressive Skill Disclosure 49
0.5.3 Lifecycle Hooks and Supervisory Control 50

0.6 Multi-Agent Swarm Topologies, Coordination, and Multi-Agent Economics 54
0.6.1 Stigmergic Communication: Environmental Coordination 54
0.6.2 Conway's Law in Multi-Agent Swarms and Role-Based SOPs 55
0.6.3 Six Sigma Defect Minimization in Agent Swarms 55
0.6.4 Runtime Task Overlap Detection, Solely Responsible Agent Patterns, and Sandboxing 56
0.6.5 Zombie Detection, Heartbeats, and State Salvage 57
0.6.6 Closed-Loop Attestation and Execution Verification 58

0.7 The Kernel Gap: Effects and Recovery 60

0.8 Chapter Map and Architectural Codas 61
0.8.1 Structure of the Volume 61

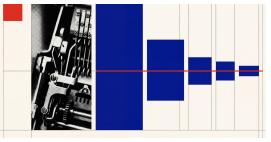
0.9 Review of the key ideas 62
0.10 Exercises 62
0.11 History and references 63



Part I Ground Truth

The first three chapters examine a local writer, delegated permissions, and a confined execution environment. They distinguish what the software prevents from what it can only detect, and what remains a design proposal.

Part opens on page 66



1 The Single-Writer Kernel 68

One writer orders changes to a local database; the chapter specifies which failures that record can survive.

1.1 Where this chapter sits, and what it promises 69
1.1.1 The one-sentence thesis 71
1.1.2 Why “reference monitor,” and why *not* consensus 73
1.1.3 A research-maturity scale, and why the grades matter 74
1.2 The kernel as seven subsystems 74
1.3 The substrate subsystem: one writer, one file 75
1.3.1 The single-writer discipline 76
1.3.2 The configuration *is* the durability claim 77
1.3.3 Schema evolution: an ordered boot ledger, not yet a sequencer (OP-7) 78
1.3.4 Path and ownership invariants 79
1.3.5 The second writer: institutional truth above the local harbor 79
1.4 Durability, split by fault class 80
1.4.1 Separate process crashes from power loss 81
1.4.2 Why the trade is defensible — and where it stops being so 83
1.4.3 A recovery story, not just a durability story 83
1.5 The resource subsystem: claims, locks, and fair exclusion 85
1.5.1 Claim granularity is richer than “claims” 85
1.5.2 The claim lifecycle as a finite-state machine 85
1.5.3 Fairness without a scheduler: the Stigmergic Ticket-Lock (OP-1) 91

1.6 The communication subsystem: the bus, stigmergy, and two delegation chains . . . 94
1.6.1 The bus: a durable carrier envelope . . . 95
1.6.2 Stigmergic markers: decay as a provided guarantee . . . 96
1.6.3 Two delegation chains, one dangerous name . . . 98
1.6.4 A second transport . . . 99

1.7 The obligation & enforcement subsystem: the sovereign's two levers 100
1.7.1 The deontic split . . . 100
1.7.2 The policy monitor is a detector, not a regimenter . . . 100
1.7.3 The general boundary: regimentation is controllability . . . 104
1.7.4 Commitments: obligation with oracle-bound closure . . . 118
1.7.5 Extending the oracle vocabulary: State-Machine Assertions (OP-5) . . . 120

1.8 The continuity subsystem: memory, checkpoint, and the foundation of the economy . . . 121
1.8.1 Three subsystems, one weak link . . . 122
1.8.2 The operation log and tamper-evidence . . . 124

1.9 The self-attestation subsystem: honest green . . . 125

1.10 The invariants, stated as theorems . . . 126
1.10.1 The work unit, model-checked . . . 126
1.10.2 The consistency-model theorem . . . 133
1.10.3 The dual-runtime hazard and runtime conformance . . . 135

1.11 Cross-subsystem atomicity: a buildable defect, not a frontier . . . 136

1.12 Threat model and the limits of mediation . . . 137
1.12.1 Advisory claims are not sandboxing . . . 137
1.12.2 The enforcement gate proves; it does not yet compel . . . 137
1.12.3 Partial actor-soul identity bounds every other invariant . . . 142
1.12.4 Hash-chain tamper-evidence, and the hardening OP-9 would need . . . 143
1.12.5 Where information-flow control goes, and why it is not here . . . 144

1.13 Adjacency contract: what the kernel assumes and provides . . . 145
1.13.1 What the substrate assumes from the machine . . . 146
1.13.2 What the substrate provides to the coordination layer . . . 146
1.13.3 The explicit non-provisions . . . 147

Review of the key ideas . . . 148

1.14 Exercises . . . 149

1.15 Related work . . . 154
1.15.1 Reference monitors and the trusted computing base . . . 154
1.15.2 Durable storage and transaction processing . . . 155
1.15.3 Coordination, stigmergy, and deontic control . . . 156
1.15.4 Capability security and tamper-evident logs . . . 156

1.16 Open problems . . . 157

1.17 Conclusion: solid where local, provisional where it must grow . . . 161

1. Chapter Appendices . . . 163

1.A Implementation & status . . . 163

1.A.1 The reference implementation . . . 163

1.A.2 Mechanism-to-artifact map . . . 163

1.A.3 Status and the limits that matter . . . 163

1.A.4 Nomenclature . . . 165



2 The Anchor Protocol **166**

Delegated capability only narrows: every child is a subset of its parent, at every hop, and the verifier checks the whole chain.

Examines assumptions used in The Single-Writer Kernel

2.1 What problem this solves . . . 167

2.2 The Anchor Protocol Architecture . . . 169

2.2.1 Phase 1: Symmetric Pinning . . . 172

2.2.2 Phase 2: Asymmetric Identity (Ed25519) . . . 174

2.2.3 Phase 3: Offline Delegation . . . 175

2.2.4 Withdrawing a card before it expires . . . 178

2.3 How we know it is correct . . . 185

2.3.1 Symbolic Analysis with ProVerif . . . 188

2.3.2 Runtime Enforcement: The Arbiter . . . 189

2.3.3 Implementation Verification . . . 190

2.3.4 The procedures, precisely . . . 191

2.4 One source crate, different verification scopes . . . 194

2.4.1 Core Verification Logic . . . 195

2.4.2 FFI Bridge to the Node.js Daemon . . . 196

2.4.3 Kani Verification Harnesses . . . 197

2.5 What the adversary can do, and what it cannot . . . 199

2.5.1 Threats this takes seriously . . . 199

2.5.2 Security Properties . . . 200

2.5.3 Where this stops . . . 201

2.6 What the verification covers . . . 204

2. Chapter Appendices . . . 206

2.A Formal Verification Status . . . 206

2.B Full ProVerif Models . . . 209

2.B.1 Phase 1: Symmetric HS256 . . . 210

2.C Phase 2: Asymmetric Ed25519 . . . 211

2.D Phase 3: Multi-hop Delegation . . . 212

2.D.1 Soundness of the Attenuation Proof (v5) . . . 214

2.E Limitations & Boundaries . . . 218

2.F Further reading and prior art . . . 219

2.F.1 Credential Formats in Agentic Commerce . . . 219

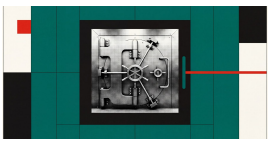
2.F.2 Formal Verification of Security Protocols . . . 220

2.F.3 Capability-Based Authorization . . . 221

2.F.4 JWT Security . . . 221

Review of the key ideas . . . 221

2.G Exercises . . . 223



3 The Sealed Harbor **226**

A confined work order has explicit release gates and a shared leakage budget, under the model's stated channel assumptions.

Examines assumptions used in The Single-Writer Kernel

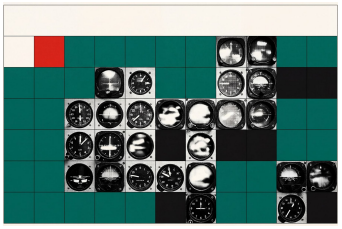
3.1 Two owners, one answer . . . 227

3.2 One work order, two fences, two gates . . . 231



Plate: The controlled opening 244

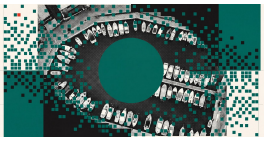
3.3 Silence except through the slot 245
3.4 What the mediated gate can enforce 250
3.5 The budget is a ledger, and the ledger conserves 254
3.6 Canaries with a power curve and a clock 260
3.7 The leakage budget: $q \cdot b$ bits, before timing 265
3.8 Limitations and boundaries 268
Review of the key ideas 269
3.9 Exercises 270
3.10 History and references 272



Part II The Cost of Seeing

An operator cannot read every action a fleet produces. This part studies what a summary must retain, when the operator needs the underlying evidence, and how information theory bounds the cost of supervision.

Part opens on page 276



4 The Legible Swarm 278

Useful supervision requires summaries that preserve the operator's decisions and link back to inspectable evidence.

4.1 Introduction: the swarm is a state of nature 280
4.1.1 The failure taxonomy 280
4.1.2 Why "local" 282
4.1.3 A worked afternoon: Mira and the six agents 283

4.2 Consent to the Leviathan (the normative claim) 285
4.2.1 The legible-sovereign amendment 286
4.2.2 Consent as a first-class, revocable primitive 287

4.3 The mechanism: legibility-with-zoom 292



Plate: What a summary leaves unseen 293

4.3.1 What to flatten, what to preserve 295
4.3.2 The zoom target must be verifiable, not self-reported 296
4.3.3 The four design questions (a decision procedure) 297

4.4 The Leviathan rules, it does not just watch (the authority half) 299
4.4.1 The roadmap as a versioned statement of intent 299
4.4.2 Adversarial review: the quality organ 301
4.4.3 Sole-responsibility roles and the specialization boundary 301
4.4.4 Completionist obligation: completion as a verifier, not a vibe 308
4.4.5 Thoughtful landing and the regimentation/enforcement split 309
4.4.6 Human-in-the-loop escalation as a typed, prioritized interrupt 310

4.5 The operator as a modeled entity 313
4.5.1 The attention objective, spined on Signal Detection Theory 314
4.5.2 Trust calibration is multi-dimensional, and the vigilance decrement is real . . . 318

4.5.3 Situation awareness needs projection, not just perception . . . 320
4.5.4 The Bainbridge–Scott convergence and the abdication gradient . . . 321
4.5.5 The operator's instrument . . . 322
4.5.6 Modes of cooperation . . . 324

4.6 Read-poverty: when inspection falls behind . . . 325
4.6.1 Value at different swarm sizes . . . 328
4.6.2 A legibility lower bound . . . 328

4.7 Discovery: the directory substrate and the split ranker . . . 339
4.7.1 Existence: the yellow pages (a solved shape) . . . 341
4.7.2 Relevance: the split ranker (the headline result) . . . 341
4.7.3 The quality gates: refuse-to-route and the costly escalation . . . 350
4.7.4 Staleness and decay on every read-surface . . . 351
4.7.5 Ambient legibility: a proposed alert channel . . . 351

4.8 Tokens: the swarm's COGS *and* its legibility engine . . . 352
4.8.1 Tokens as COGS, and the effective-context trap . . . 352
4.8.2 Compaction is the legibility engine; the digest IS the compaction . . . 353
4.8.3 Context paging: virtual memory for the context window . . . 358
4.8.4 The context-degradation cascade (the shared failure surface) . . . 361
4.8.5 Single-operator: account, don't auction . . . 363

4.9 Reconciling the canon: roles, consent, and local knowledge . . . 363
4.9.1 Who is the multitude, who is the sovereign? . . . 363
4.9.2 Express consent, not tacit; voice *and* exit . . . 365
4.9.3 Mêtis is Hayek, and some of it cannot be made legible . . . 366

Review of the key ideas . . . 366

4.10 Exercises . . . 368

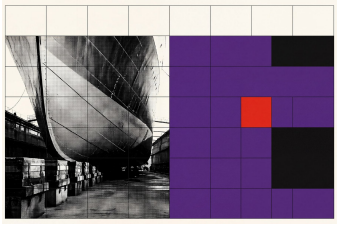
4.11 Related work and prior art . . . 375

4.12 The time axis and the bridge to the economy . . . 377
4.12.1 Legibility over time, not just the present . . . 377
4.12.2 What this layer hands upward (the through-line) . . . 378
4.12.3 The open problems, as a map . . . 380

4. Chapter Appendices . . . 382

4.A Implementation and status . . . 382

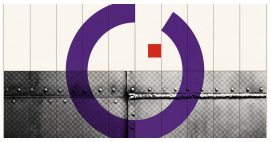
4.B Notation and the nomenclature key . . . 385



Part III What Survives the Restart

If an agent can discard its history at restart, a sanction may not follow it. This part asks which identity, outcome, and obligation records must persist for reputation to mean anything.

Part opens on page 386



5 From Spawn to Person 388

Continuity turns an anonymous spawn into a ledger position with a witnessed record; reputation is what that record is worth.

5.1 Introduction: the spawn that cannot be paid . . . 389
5.1.1 S1 — the spawn that cannot be paid . . . 389
5.1.2 What this chapter claims, and what it refuses to claim . . . 390

5.2 Prior art, and where this chapter departs from it . . . 392
5.2.1 Personal identity: what makes a later thing the same person . . . 392
5.2.2 Reputation and skill-rating estimators . . . 393
5.2.3 Reputation economics: cheap pseudonyms and limited records . . . 394
5.2.4 LLM-as-judge, and why it cannot be the whole story . . . 396

5.3 The role / person distinction, made precise . . . 396
5.3.1 Capability is not permission (a nomenclature the series holds) . . . 398

5.4 Continuity is a personal-identity problem . . . 400



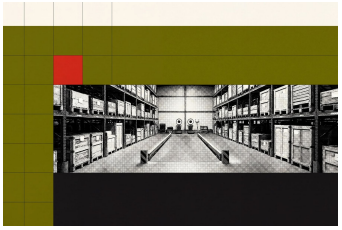
Plate: What survives the worker 401

5.4.1 Locke's memory criterion, and its famous bug . . . 402
5.4.2 Parfit's repair: continuity, not connectedness . . . 402

5.5 The three subsystems of continuity . . . 410
5.5.1 Subsystem 1 — Memory: the episodic record **implemented** . . . 410
5.5.2 Subsystem 2 — Checkpoint: restorable state partial . . . 411
5.5.3 Subsystem 3 — Outcome ledger: the witnessed record of delivery partial . . . 412
5.5.4 The canonical seam sentence (stated identically in every paper) . . . 413

5.6 Identity: the root the whole chain hangs from . . . 416
5.6.1 S2 — whitewashing in action . . . 419
5.6.2 The classic attacks this forecloses . . . 419
5.6.3 The principal above the actor . . . 429

5.6.4 The body behind the name: engine swaps and resurrection 431
5.7 The keystone split: local identity vs. cross-operator attestation 439
5.8 Reputation as a substrate, not a score 441
5.8.1 Choosing an estimator for the signal 441
5.8.2 Score as telemetry; gate on predicates 443
5.9 Routing policies and public reputation answer different questions . . 443
5.10 The multi-dimensional reputation protocol 445
5.10.1 Multi-dimensional quality (different axes, different judges) 445
5.10.2 The neutral-judge market (the harbor's universities and rating agencies) 447
5.10.3 S3 — a graded job, end to end 449
5.10.4 Skill → agent helpfulness attribution 450
5.11 The grading oracle's incentive-compatibility 451
5.12 Revocation, tombstones, and bounded memory 461
5.12.1 Append-only evidence, individually revocable outcomes 461
5.12.2 S4 — the fraudulent outcome retracted 462
5.12.3 Bounded memory is a design parameter, not a virtue to assert away 464
5.13 What this chapter hands the market 465
Review of the key ideas 467
5.14 Exercises 468
5.15 Reproducing the chapter results 474
5.16 Open problems (the starred items, collected) 474
5.17 Conclusion 476
5. Chapter Appendices 478
5.A Implementation status of the reference system 478



Part IV Trade Between Strangers

Operators who trade work need evidence of performance, rules for disputes, and limits on loss. These chapters develop the market and its settlement rules, then examine how evidence and grants pass between independently governed machines.

Part opens on page 480



6 The Harbor Economy 482

Labor, rented assets, and licensed skills expose different risks but settle through a shared ledger that conserves each native unit.

6.0.1 The through-line: attributing a record of past work . . . 485

6.1 Three purchases, three information problems . . . 485

6.1.1 What the design still requires . . . 493

One ledger, three different reputation keys . . . 493

6.2 The float plan: the built floor . . . 494

6.2.1 The bond ledger conserves value (this is built) . . . 496

6.2.2 Settlement: four terminal states bound to an oracle . . . 497

6.3 Three sides on one escrow . . . 498

6.3.1 One trade, all three sides, one settlement . . . 509

6.4 The keystone the market rests on — and does not yet have . . . 510

6.4.1 Local non-forgeable identity is the keystone — for one operator . . . 510

6.4.2 But its threat model does not reach the market . . . 511

6.4.3 The principal above the actor . . . 512

6.5 Conservation under composition . . . 512

6.5.1 The Myerson–Satterthwaite corner . . . 514

6.6 The Anchor Protocol proper: cross-harbor capability transfer . . . 516

6.7 Federation: trading across machines you don't own . . . 518

6.7.1 No consensus, by design . . . 518

6.7.2 Settlement and the cross-chain-deals lineage . . . 519

6.7.3 Revocation gossip and the equivocation race . . . 519

6.8 Reputation: monotone, but revocable . . . 521

6.8.1 "Never ends" is not an anti-revocation property . . . 522

6.8.2 Why Sagas-style compensation does not restore reputation . . . 522

6.8.3 The grading-oracle keystone . . . 523

6.9 Hosted trust is the product . . . 524
6.10 Cold start, and the auction question . . . 526
6.10.1 Static escrow vs. competitive underwriting . . . 527
6.10.2 Cartels and wash-trading . . . 527
6.11 A gluing analogy for the open federation problem . . . 528
6.12 Gaps the design must still close . . . 529
6.13 One further market: adversarial testing and purchased assurance . . . 530
6.13.1 Putting a parameter on the echo . . . 535
Review of the key ideas . . . 538
6.14 Exercises . . . 539
6.15 Related work . . . 544
6.16 Conclusion . . . 547
6. Chapter Appendices . . . 550
6.A Implementation & status . . . 550
6.B Proof sketches . . . 553



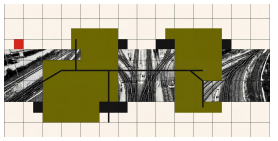
7 The Bonded Commons **556**

Bonds, witnesses, audits and appeals turn residual uncertainty into an institution; the ledger's conservation law is mechanically checked.

Examines assumptions used in The Harbor Economy

7.0.1 Three Risks of Ad Hoc Pairwise Trust . . . 557
7.0.2 Contributions . . . 558
7.0.3 Related Work in Crypto-Economic Bonding . . . 559
7.1 The Commons Authority . . . 560
7.1.1 The Daemon as Correlating Device . . . 563
7.1.2 Why Agents Consent . . . 565
7.2 Layer 1: Structural Prevention . . . 566
7.2.1 Capability Attenuation . . . 566
7.2.2 Isolation Domains . . . 568
7.2.3 Runtime Invariant Enforcement . . . 569
7.3 Layer 2: Immutable Attribution . . . 570
7.3.1 The Evidence Chain . . . 570
7.3.2 The Merkle Forest: Cross-Daemon Inclusion Proofs . . . 571
7.3.3 Mutable-Signal Ledger (Pheromone Lineage) . . . 574

7.3.4 Attribution Survives Identity Disposal . . . 576
7.4 The Limits of Decisive Allocation . . . 576
7.4.1 A Sen-Inspired Design Warning . . . 576
7.4.2 Advisory Claims as Resolution . . . 578
7.4.3 Governance, Disputes, and Appeals . . . 579
7.5 Crash Recovery as Social Continuation . . . 582
7.5.1 What Morality Means When Death Is Cheap . . . 583
7.5.2 The Limits of Reputation . . . 584
7.6 Layer 3: Economic Alignment . . . 585
7.6.1 The Float Plan . . . 585
7.6.2 Settlement . . . 585
7.6.3 Why This Defeats the Three Adversaries . . . 586
7.6.4 Truthful Claim Signaling as Nash Equilibrium . . . 587
7.6.5 Pricing the Bond . . . 595
7.7 Coordination as Substrate: An Expressive-Act Taxonomy . . . 609
7.7.1 Five Expressive Classes . . . 609
7.7.2 Per-Class Bond Profiles . . . 609
7.8 Evidence-Growth Monitoring and Replay . . . 612
7.9 Formal Model . . . 614
7.9.1 State and Actions . . . 614
7.9.2 Properties . . . 617
7.9.3 The Conservation Theorem . . . 618
7.10 Related Work . . . 622
7.11 Discussion and Limitations . . . 625
Review of the key ideas . . . 629
7.12 Exercises . . . 630
7.13 Conclusion . . . 634
7. Chapter Appendices . . . 636
7.A Formal Verification Status . . . 636
7.B Worked Example: One Agent, All Layers . . . 638
7.C Scope Boundary Checklist . . . 642



8 The Federated Harbor 644

Mutually distrustful machines relay evidence, not sovereignty: what may gossip, what needs an authority point, and how a grant crosses a boundary.

Examines assumptions used in The Harbor Economy

8.0.1 Harbor sovereignty as a property characterization . . . 645
8.0.2 Inter-harbor scope . . . 647
8.0.3 The witness log . . . 647
8.0.4 What the federation is not . . . 648

8.1 Threat Model . . . 650
8.1.1 In-scope attacks . . . 651
8.1.2 Out-of-scope attacks . . . 652
8.1.3 Threat-band visualization . . . 653



Plate: A commitment between independent shores 655

8.2 Cross-Harbor Capability Transfer . . . 656
8.2.1 The four-message ceremony . . . 656
8.2.2 Why this composes cleanly with Anchor . . . 657
8.2.3 Attenuation across the boundary . . . 658
8.2.4 Threats foreclosed and threats deferred . . . 659
8.2.5 Mechanization status . . . 659

8.3 Federated Revocation . . . 660
8.3.1 Convergence bound . . . 661
8.3.2 Conflict resolution across harbors . . . 663
8.3.3 Event Log & Ephemeral Cuckoo Discipline . . . 664
8.3.4 Visible discrepancies and their certificates . . . 665
8.3.5 From observations to a checkable refusal . . . 679

8.4 Cross-Harbor Settlement . . . 681
8.4.1 The bounded escrow . . . 682
8.4.2 Cross-harbor conservation . . . 684
8.4.3 Settlement protocol . . . 685
8.4.4 Fee structure and economic discipline . . . 687

8.5 Federation Admission . . . 687
8.5.1 The bonded-sponsor pattern . . . 687
8.5.2 Cold-start failure modes . . . 689

8.6 Worked Example . . . 690
8.6.1 Setup . . . 690

8.6.2 The agent task . . . 691
8.6.3 Step 1: Cross-harbor transfer . . . 692
8.6.4 Step 2: Damage detection . . . 692
8.6.5 Step 3: Settlement . . . 693
8.6.6 Step 4: Revocation propagation . . . 693
8.6.7 What this example shows . . . 694

8.7 Formal Model . . . 694
8.7.1 ProVerif extensions to Anchor . . . 695
8.7.2 TLA+ extension for cross-harbor conservation . . . 696
8.7.3 What is mechanized and what is structural . . . 696

8.8 Failure Modes . . . 697
8.8.1 Centralization gravity . . . 697
8.8.2 Equivocation by a malicious witness . . . 698
8.8.3 Partition tolerance . . . 698

8.9 Related Work . . . 699

8.10 Limitations and Open Questions . . . 702
8.10.1 The multi-principal correlated-equilibrium extension . . . 702
8.10.2 Trustless settlement for non-fungible bonds . . . 702
8.10.3 Cartel formation across federations . . . 703
8.10.4 Cold-start admission and the invitation-only failure mode . . . 703
8.10.5 Equivocation propagation speed . . . 703
8.10.6 Cost of per-write durability . . . 704
8.10.7 What this means for the reader . . . 704

Review of the key ideas . . . 704

8.11 Exercises . . . 706

8.12 Conclusion . . . 709

8. Chapter Appendices . . . 710

8.A Verification Status . . . 710

8.B ProVerif Artifacts and Proof Obligations . . . 712

8.C TLA+ Artifact and Cross-Harbor Scope . . . 712

Solutions to the exercises 714

Chapter 0: Foundations and Prerequisites 714
Chapter 1: The Single-Writer Kernel 715
Chapter 2: The Anchor Protocol 724
Chapter 3: The Sealed Harbor 730
Chapter 4: The Legible Swarm 733
Chapter 5: From Spawn to Person 747
Chapter 6: The Harbor Economy 757
Chapter 7: The Bonded Commons 768
Chapter 8: The Federated Harbor 777

Appendices 787

A Implementation boundary 787
A.1 Five assurance modes 787
A.2 What exists now 788
B Mechanized claims 789
C Result atlas 812
C.1 The eight propositions and their evidence 812
C.2 A result is admitted only with its limit 814
D Notation and cross-chapter concordance 815
Which chapter owns which primitive 816
Routes to the open problems 816
Image credits 818

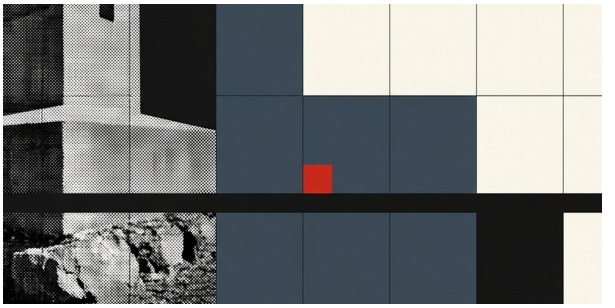
0

Foundations and Prerequisites

The question this chapter answers

What physical and theoretical ground must an agent operating kernel stand on?

Classical multi-agent theory, contemporary coding agent harnesses, transformer KV-cache limits, and the kernel gap.



> *I conclude that there are two ways of constructing a software design: One way is to make it so simple that there are obviously no deficiencies and the other way is to make it so complicated that there are no obvious deficiencies. The first method is far more difficult.*
>
> C. A. R. Hoare, *The Emperor's Old Clothes* (1980 Turing Award Lecture)

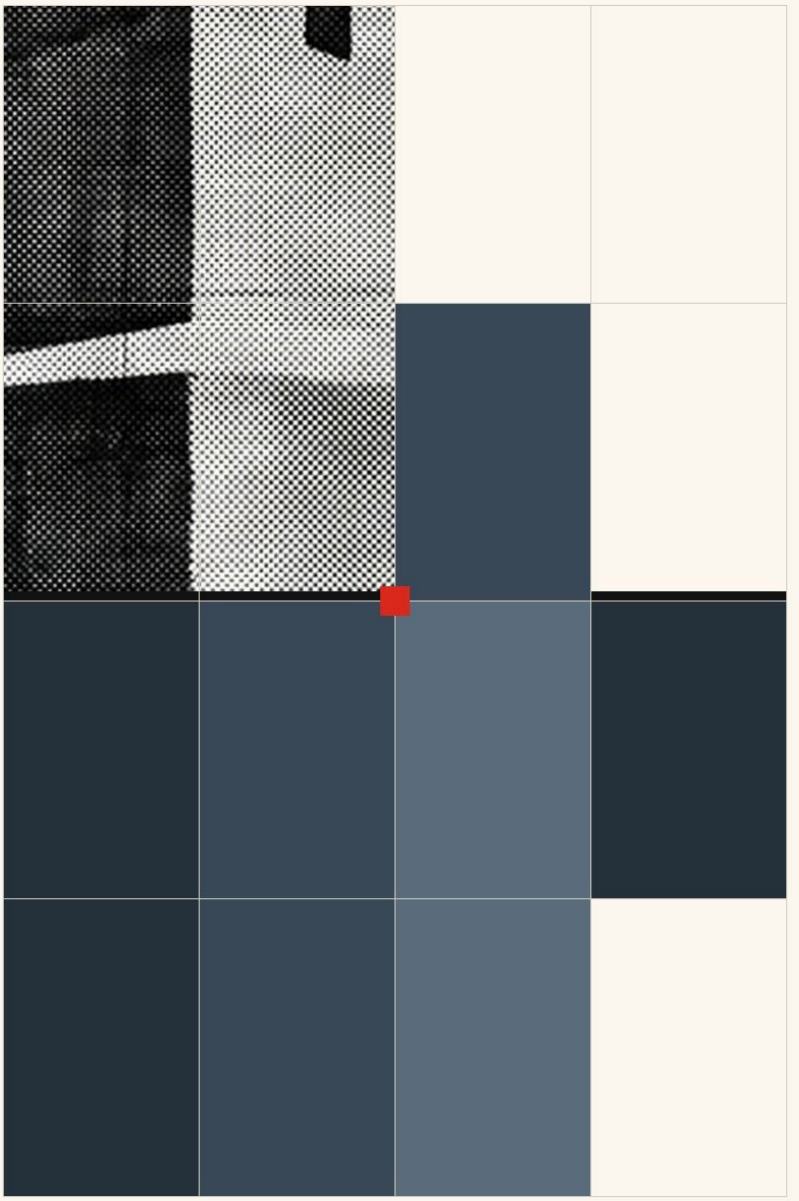
Theoretical Foundations Before constructing an operating kernel for autonomous systems, we must establish the physical and theoretical ground upon which modern AI agents stand: classical intentional agency, the actor model, transformer KV-cache memory scaling, the closed-loop harness, and which guarantees require mediated effects and retained evidence.

Start with one interrupted job. Alice asks two workers to fix the same checkout failure. Both read revision 7. The first publishes a repair; the second overwrites it with a patch based on revision 7. Then both processes exit. Alice needs three answers: which change was permitted, which change took effect, and which work is still owed?

Numbers by Hand 0.0.1 (Two increments, one missing update).

Start a counter at 0. Each worker reads 0, computes $0 + 1$, and writes 1. The final counter is 1. If each entire read–increment–write operation runs before the other begins, the states are $0 \to 1 \to 2$. Merely putting the two final writes in a queue still gives $0 \to 1 \to 1$. We will return to this stale-observation problem when we define admission and recovery.

The route. First distinguish a worker's proposal, an admitted effect and a retained record. Then examine commitments, messages and incentives; account for the cost of keeping information in a prompt; and follow a request through a refusal, a timeout and a restart. The final result assembles these cases. Read the examples before the formalism. The literature comparisons identify obligations that return in the later chapters, not a vocabulary to memorize.



Landmark 01 · The Substrate Bedrock & The Kernel Gap

A proposed action is not evidence that it occurred. A state-dependent admission rule needs adequate evidence of the state on which it depends. When recovery depends on a past event, an adequate record must survive the failure being considered.

0.1 Introduction: The Unit of Account, Agency, and the Kernel Gap

0.1.1 The Unit of Account is the Work

A *work unit* is the case file for a job: who requested it, what actions are allowed, what has been observed, and what remains to be done. An *obligation* is a recorded requirement, such as supplying a test result before the job may be called complete. Alice's repair remains the same job when a worker exits. Chapters 1 and 5 develop its record and its succession.

0.1.2 Defining the Agent and the Kernel

The counter example needs a proposer, an actuator and a rule separating them. An *actuator* actually changes a resource, for example by writing a file. A proposal may describe a write without yet performing it.

Definition 0.1.1 (Autonomous Agent)**.**

An **agent** chooses a proposed action $a_t \in \mathcal{A}$ after an observation history $h_t \in \mathcal{H}$. Its policy $\pi_\theta(a_t \mid h_t)$ specifies the probability of that choice; θ denotes its parameters or rules. A deterministic policy assigns probability one to its chosen action. The surrounding harness runs the policy, invokes tools and returns observations.

For Alice's worker, a patch is a proposal and a test result is an observation. A passive log that records requests but makes no choices is not an agent in this model. Neither a proposal nor a reported belief certifies the host state.

Definition 0.1.2 (Operating Kernel)**.**

The **coordination kernel** is the trusted application component that admits changes to specified resources and retains coordination state. Its contract names the operations it mediates, the invariants it checks, the evidence it retains and the failures its storage survives. A state invariant is a property required before and after each admitted transition, such as at most one current holder of a particular lease.

Here “kernel” does not mean the operating-system kernel. A daemon (a background service) and its database implement Chapter 1’s construction. Host isolation and external actuators need additional controls. A log written after an unrestricted file write is evidence, not an admission gate; a coordination-database transaction does not make that file write atomic.

0.1.3 The Dual Lineage of Autonomous Agency

Symbolic multi-agent research supplies explicit representations of beliefs, goals, plans and messages. Generative models supply another way to propose plans and interpret unfamiliar observations. Neither representation settles who may overwrite revision 7. That depends on the surrounding protocol.

0.1.4 Proposals and checked effects

Table 0.1.
The distinction applies to symbolic and neural proposers alike.

Question	Proposal supplies	System checks
What should change?	A patch	Authorized resource and revision
What is still owed?	A remembered plan	Durable open obligations
Did it work?	A reported outcome	Specified evidence
May another worker act?	A delegation request	Current scope and expiry

The last section proves an elementary preservation claim for the checked boundary. Chapter 2 returns to the last row: communicating a task description and granting permission to act are different operations.

0.1.5 Eight Propositions of Accountable Coordination

These previews introduce the questions that recur in the eight chapters. They are not eight proofs or claims that the complete system exists. Each owning chapter supplies its definitions, examples and limits.

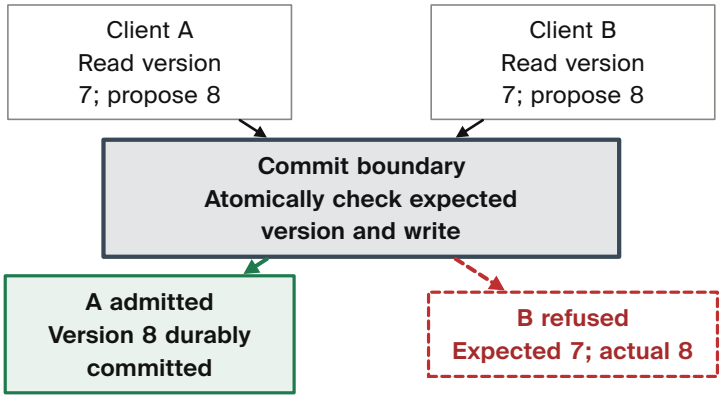


Figure 0.1.
Serial admission with version validation. Two proposals based on version 7 cannot both replace it. The gate commits one and rejects the stale request. The guarantee requires validation and effect to share one indivisible boundary.

A serialized log records the accepted order. Preventing lost updates also requires a transaction or version check covering each complete operation.

Proposition 1: Prevention Begins at the Effect Boundary (*The Single-Writer Kernel*)

Preventing a forbidden effect requires control over every path that can produce it. Advisory instructions alone do not provide that control. For example, if two agents can read the same value $x = 0$ and later write their independently computed value $x + 1$, the schedule read–read–write–write ends at $x = 1$, losing one increment. This is a permitted counterexample, not an inevitable outcome of concurrency: disjoint updates and correctly implemented transactions can be safe.

The Single-Writer Kernel checks a request's authorization and state preconditions at the same boundary that commits its ledger transition. A complete read–modify–write transaction, or an atomic version check followed by the write, rejects the stale update. Serializing only the final writes is insufficient. SQLite's transaction isolation covers its database operations; making a filesystem or network effect atomic with a receipt requires a separate actuator and crash-recovery protocol. Strict serializability additionally requires that the operation order respect real-time completion.

Formally, let e_i and e_j be concurrent events under Lamport's happened-before relation. Their incomparability alone does not imply an error. A conflict arises when their resource accesses interfere and at least one writes. An admission gate assigns admitted operations a commit order $\sigma(e)$, checks each operation against

Leslie Lamport. Time, Clocks, and the Ordering of Events in a Distributed System. *Communications of the ACM*, 21(7):558–565, 1978.

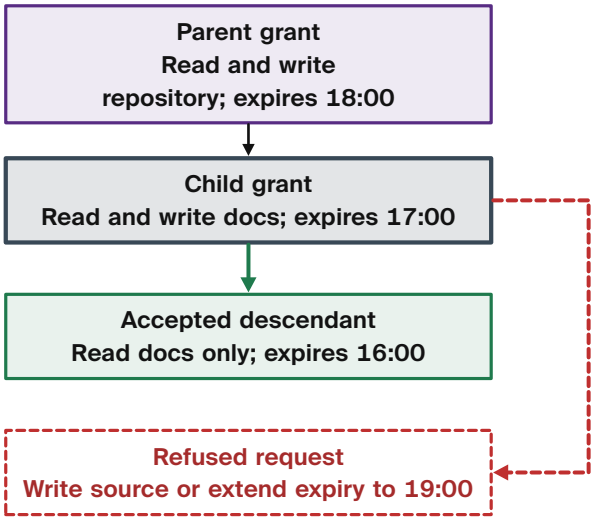


Figure 0.2. Capability attenuation. Child scope and expiry do not exceed the parent's. Chain verification and conjunctive caveat checks enforce this relation; they do not revoke a broader ancestor token already held by a delegate.

the current version, and makes the check and state transition indivisible. Under complete mediation and correct recovery, this preserves any invariant maintained by each admitted transition. Locks, compare-and-swap and transactional validation are alternative ways to implement that boundary.

Jerome H. Saltzer and Michael D. Schroeder. The Protection of Information in Computer Systems. *Proceedings of the IEEE*, 63(9):1278–1308, 1975.

Arnar Birgisson, Joe Gibbs Politz, Úlfar Erlingsson, Ankur Taly, Michael Vrable, and Mark Lentczner. Macaroons: Cookies with Contextual Caveats for Decentralized Authorization in the Cloud. In *Network and Distributed System Security Symposium (NDSS)*, 2014. Online source

Proposition 2: Delegation Only Narrows (*The Anchor Protocol*)

In classical security engineering, authority must obey Saltzer and Schroeder's fundamental Principle of Least Privilege: every process must operate using the least set of privileges necessary to complete its assigned task. In naive multi-agent architectures where agents delegate subtasks by synthesizing unconstrained prompts, authority frequently inflates due to broad, ambiguous system instructions, leading to catastrophic privilege escalation.

Under the Anchor Protocol, every child capability has authority and expiry no greater than its parent's ($C_0 \supseteq C_1 \supseteq C_2$ and $T_0 \geq T_1 \geq T_2$). Authentication proves who signed the credential; attenuation specifies what the signature is permitted to authorize. Chained cryptographic capability tokens (such as macaroons and Harbor Cards) enforce monotonic restriction at every delegation hop: an intermediate delegate cannot widen permissions, extend deadlines, or remove sandbox constraints when dispatching subordinate workers.

Mathematically, a capability token M_k at delegation depth k is constructed as an HMAC chain:

$$s_k = \text{HMAC}(s_{k-1}, c_k) \tag{0.1}$$

Here s_0 denotes the authenticated root tag derived with the issuer's secret, and c_k is an added caveat. Under HMAC unforgeability, a holder of the final tag cannot manufacture a valid tag for a less restricted chain it was never issued. The verifier must authenticate the complete chain and conjoin every caveat. A holder that retained a broader ancestor token can still use that token until it expires or is revoked; attenuation does not revoke previously issued authority.

Proposition 3: Confinement Is a Property of the Room, Not of the Tenant (*The Sealed Harbor*)

In multi-tenant, multi-principal systems, mutual distrust is the necessary default. Two owners who trust neither each other nor the hosting venue can still take a verified computation out of a sealed room, provided the execution chamber guarantees non-bypassable physical isolation. Confinement cannot be promised by the tenant model: an LLM process is vulnerable to prompt injection, memory corruption, or adversarial jailbreaking.

Confinement is supplied by a specified host policy and its trusted computing base. The intended room closes unauthorized network and filesystem paths and meters the admitted egress channel. Its guarantee covers the modeled channels only. Timing, termination, shared hardware and other side channels need separate analysis. Canary tokens provide evidence of particular monitored disclosures; they cannot establish that no other secret or encoding escaped.

The confinement policy is expressed as a non-bypassable sandbox profile P_{room} compiled into the kernel:

$$\forall \sigma \in \Sigma_{\text{syscall}}, \quad \text{Permit}(\sigma \mid P_{\text{room}}) \iff \sigma \in \Sigma_{\text{confined}} \tag{0.2}$$

Here Σ_{confined} is shorthand for the allowed syscall-and-argument cases in a stated sandbox policy; executable identity, paths and inherited descriptors matter as well as syscall names. A canary $\kappa \sim \mathcal{U}(\{0,1\}^{256})$ may be registered with an egress matcher. Observing its registered representation is a disclosure witness under the matcher's attribution assumptions. An adversary that leaks a

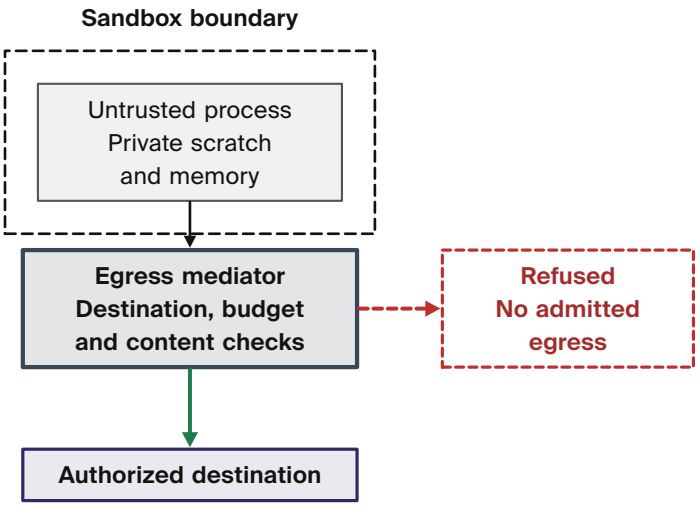


Figure 0.3. Mediated egress. The room closes unauthorized modeled paths and admits traffic through a checked channel. Canary matching supplies evidence of particular disclosures. Timing and other unmodeled side channels require separate analysis.

Canary matching detects matching content. Timing channels, unmediated interfaces and arbitrary secrets require separate analysis.

different secret or an unrecognized encoding may evade it. Revocation and any financial penalty therefore follow the declared evidence and contestation rules.

Proposition 4: Supervision Has an Information Cost (*Attention and Supervision*)

Human oversight is constrained by fundamental cognitive and information-theoretic limits. As Bainbridge observed in *Ironies of Automation*, automating routine operations does not eliminate human responsibility; it concentrates attention on rare, high-consequence failure modes. A human operator cannot parse a raw firehose of millions of agent telemetry tokens without suffering cognitive saturation and supervisory fatigue.

Lisanne Bainbridge. Ironies of Automation. *Automatica*, 19(6):775–779, 1983.

Yet naive LLM-generated summaries are actively dangerous: they hallucinate away subtle test failures, sanitize errors, and declare false victories. A digest cannot guarantee that a supervisor

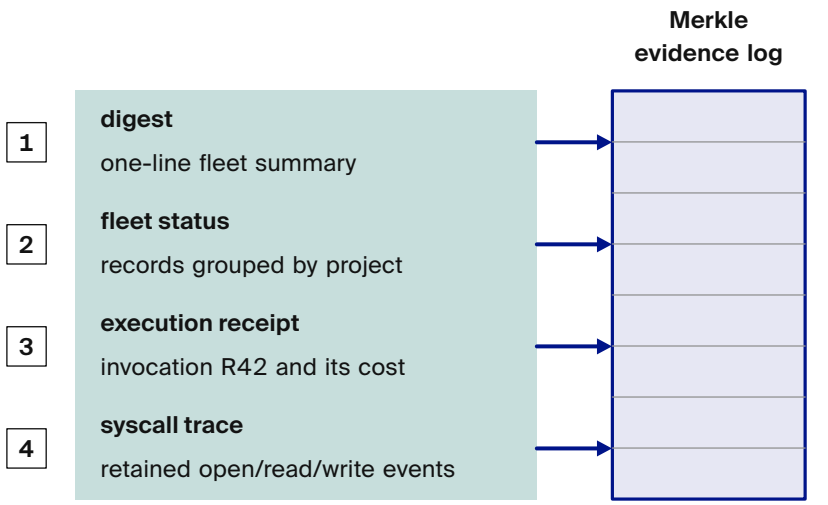


Figure 0.4.
An index into retained evidence. A status claim links to receipt R42 and its retained artifacts. The digest does not reconstruct omitted data; the evidence store must retain it.

Resolution	Retained support
1 Digest	References to the evidence log as a whole.
2 Fleet status	The same records, grouped by project.
3 Receipt R42	A committed leaf identifying one invocation.
4 Syscall trace	Trace bytes checked against the commitment.

References navigate; commitments check retrieved bytes. Neither reconstructs missing artifacts or proves their semantic correctness.

will notice any one of many relevant events unless it expends sufficient bits and attention to distinguish them. In an accountable architecture, a summary is therefore an index into evidence, not a substitute for it. The supervisory dashboard presents low-entropy summaries backed by cryptographic pointers directly into the underlying Merkle event log, allowing instant drilling down from high-level status to raw syscall traces.

Let X be a finite discrete execution history and $D = f(X)$ its digest. Then $I(X;D) = H(D) \leq H(X)$. If $H(D) < H(X)$, the digest alone cannot reconstruct the history: $H(X \mid D) = H(X) - H(D) > 0$. Retained receipts are the additional information that makes drill-down possible. A balanced binary Merkle tree over N retained receipts supplies an inclusion path with $O(\log N)$ hashes. That bound concerns proof size and verification work under a trusted root and collision-resistant hashing. It gives no logarithmic bound on discovering every relevant event or on the human effort needed to understand it.

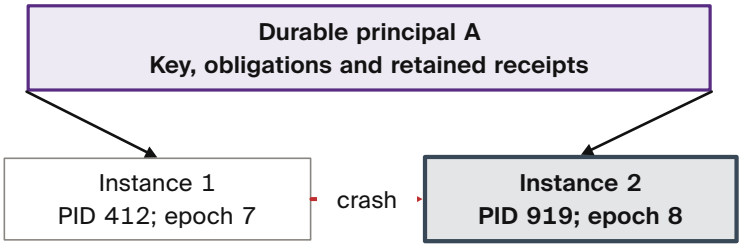


Figure 0.5. Identity across process replacement. A stable principal links successive execution instances. Epochs distinguish current authority from stale credentials, while committed obligations remain in durable storage.

The new instance proves a fresh binding and reacquires valid authority. Principal continuity does not revive expired leases or recover unrecorded state.

Proposition 5: Accountability Must Outlive the Process (*From Spawn to Person*)

Operating system processes are inherently mortal: they can be terminated by out-of-memory killers, abort on unhandled exceptions, or be restarted intentionally. If an agent's identity, reputation, debt ledger, and liabilities exist only in the volatile RAM of a running Python or Node process, a defecting agent can escape consequences simply by crashing and rebooting under a new process ID.

An Agent Operating Kernel decouples ephemeral execution bodies from durable agent personas. A task acquires a durable cryptographic principal (Ed25519 keypair), an append-only capability history, a Merkle-backed evidence record, active normative obligations, and ultimate liability for physical outcomes. Restarting a crashed process or leasing a fresh container does not wipe the case file clean or replenish consumed token budgets: a successor reconstructs recorded obligations from the durable ledger. Resuming the exact execution state additionally requires a valid execution checkpoint and a recovery protocol.

Formally, an agent persona is defined as a persistent state tuple:

$$\mathcal{P} = \langle \mathrm{pk}_{\mathrm{agent}}, \mathcal{L}_{\mathrm{history}}, O_{\mathrm{active}}, \mathcal{B}_{\mathrm{consumed}}, \sigma_{\mathrm{checkpoint}} \rangle \tag{0.3}$$

where $\mathrm{pk}_{\mathrm{agent}}$ is the public signing identity, $\mathcal{L}_{\mathrm{history}}$ is the append-only hash chain of past commitments, O_{active} is the set of unfulfilled obligations, and $\mathcal{B}_{\mathrm{consumed}}$ is the non-resetting spend tally. When a container process terminates (`SIGKILL`), the kernel reclaims the ephemeral execution lease, but freezes $\mathcal{P}$. A successor must satisfy the current admission policy; the key alone does not override

revocation, scope or epoch checks. The final component $\sigma_{\text{checkpoint}}$ is an optional execution checkpoint. Restoring obligations differs from reproducing exact inference state.

Proposition 6: Exchange Requires Separated Instruments (*The Harbor Economy*)

When autonomous agents trade compute, task assignments, and data access, economic coordination requires robust market design. In classical economics, Akerlof demonstrated that asymmetric information and unseparated incentives degrade markets into a "market for lemons." Conflating multiple financial functions into a single scalar reputation score or omnibus utility token creates acute moral hazard.

George A. Akerlof. The Market for "Lemons": Quality Uncertainty and the Market Mechanism. *Quarterly Journal of Economics*, 84(3):488–500, 1970.

A trustworthy harbor economy strictly separates four distinct coordination instruments: (1) **Payment**, which compensates delivered labor; (2) **Collateral**, which bonds and prices bounded failure; (3) **Evidence**, which authenticates specified completion artifacts whose semantic adequacy depends on the verifier; and (4) **Settlement**, which atomically and irrevocably closes obligations. Combining these functions into a single score undermines trust and invites strategic manipulation; separating them makes their contracts and conservation checks independently inspectable. Audit quality still depends on the evidence and verifier.

Under this separated architecture, an agent offering a service must post collateral $C \geq \mathcal{V}(\text{risk})$ into escrow. When the task completes, the contractor produces evidence $\mathcal{E}$ (test logs, diff hashes). Only when $\mathcal{E}$ satisfies the verifier contract does settlement $\mathcal{S}$ execute, simultaneously disbursing payment P, returning collateral C, and updating the reputation ledger. Conflating payment and collateral into a single balance allows defaulting agents to consume resources without recourse; separating them exposes the quantities needed for an incentive analysis; it does not by itself make honest behavior optimal.

Proposition 7: Uncertainty Needs an Institution (*The Bonded Commons*)

In complex software engineering, deterministic formal verification cannot decide every semantic property, and static type checkers

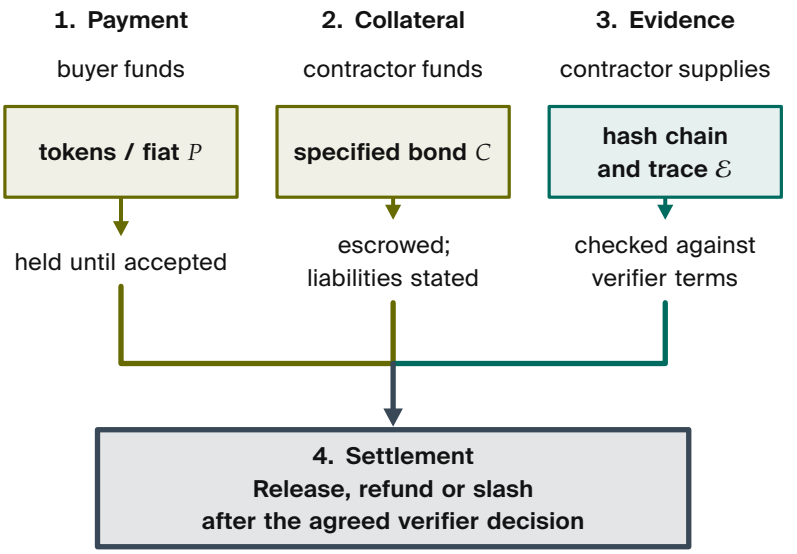


Figure 0.6. Four contract roles. Payment, collateral, evidence and settlement have distinct functions. Separating them exposes accounting and verification obligations; incentive compatibility additionally depends on payoffs, detection and enforceable remedies.

Separate contract roles expose accounting and verification obligations. Incentive compatibility still depends on payoffs, detection and remedies.

cannot foresee every runtime interaction. Managing residual uncertainty requires institutional governance, grounded in Ostrom's governing of shared commons and Becker's economics of deterrence.

Some harms cannot be prevented upfront, and some outcomes cannot be judged purely by mechanical test suites. To govern this irreducible uncertainty, the kernel constructs institutional procedures: bonded stake deposits, randomized spot audits by independent witness verifiers, neutral contestation and appeal arbitration, and strict conservation rules. By turning uncertainty into an inspectable, contested process backed by financial slashing, the commons can deter modeled deviations when expected sanctions exceed gains, while providing a procedure for contesting false positives.

Elinor Ostrom. *Governing the Commons: The Evolution of Institutions for Collective Action*. Cambridge University Press, 1990.

G. S. Becker. Crime and punishment: An economic approach. *Journal of Political Economy*, 76(2):169–217, 1968.

A one-shot deterrence calculation makes the assumptions explicit. Normalize honest utility to zero. Suppose a deviation gains $G \geq 0$ when undetected, loses that gain and pays a slash $S \geq 0$ when detected, and is detected with probability $p > 0$. Then its expected incremental utility is

$$\Delta U = (1 - p)G - pS. \tag{0.4}$$

Thus $S \geq (1 - p)G/p$ deters this deviation for a risk-neutral agent. For example, $G = 10$, $p = 0.1$, and $S = 100$ give $\Delta U = -1$. Here p includes both audit selection and successful detection. If the gain

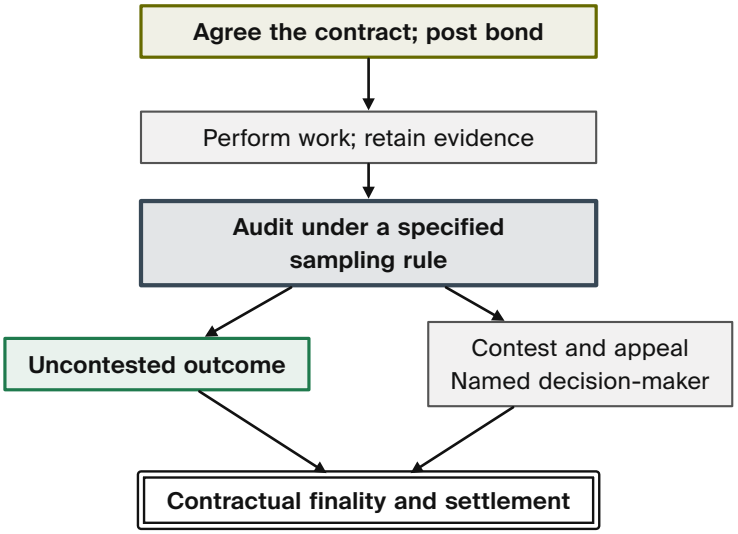


Figure 0.7. Contestation and settlement. Evidence enters a declared decision and appeal procedure. A final ruling authorizes settlement under the contract; its reliability depends on adjudication and enforcement assumptions.

survives detection, the expression is $G - pS$ instead. A repeated-game equilibrium additionally needs a discount factor, monitoring model, continuation strategies and credible enforcement; this single-period inequality alone does not prove one.

Proposition 8: Federation Relays Evidence, Not Sovereignty (*The Federated Harbor*)

When autonomous fleets scale across distinct physical workstations, cloud tenancies, and organizational boundaries, a shared central database requires an explicit agreement about authority, availability and disclosure. This book studies hosts that retain independent admission authority.

The federation design here exchanges verifiable evidence while retaining local admission authority. Mutually distrustful hosts communicate across untrusted zero-trust relays, exchanging signed capability attestations, revocation gossips, Merkle inclusion proofs, and settlement receipts without sharing a database. Transport carries signed evidence under stated key-management assumptions; the receiving host's local kernel retains complete sovereignty over whether to admit, execute, or reject the requested operation.

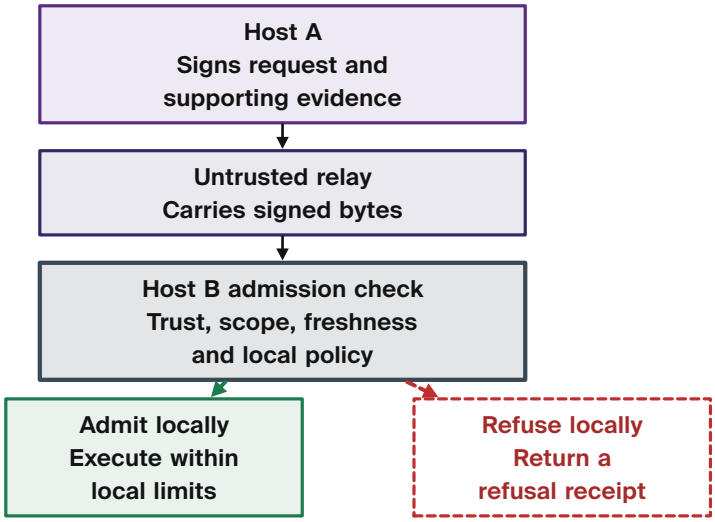


Figure 0.8. Federated evidence. A sender transmits signed evidence. The receiver checks identity, scope, freshness and local policy before admission. Authentication does not itself confer permission to act.

In this federated topology, host kernels H_A, H_B maintain private, independent SQLite WAL databases. When Agent α on host H_A requests execution on host H_B, it transmits a signed credential package $\Pi = \langle \text{sig}_A(\alpha), \text{cap}_\alpha, \mathcal{M}_{\text{proof}} \rangle$. Host H_B's local kernel independently verifies Π against its local trust anchor, checking revocation state without querying H_A's internal database. Only if credential checks and local authorization both permit it may H_B admit the task. Sovereignty remains local; evidence crosses the wire.

Read these as a dependency chain: each chapter uses earlier mechanisms and states what they cannot guarantee.

0.1.6 What Counts as Knowing: Epistemic Bounds and Evidence Channels



Ada Lovelace observed that analytical engines have no pretensions to originate anything; they execute what we know how to order them to perform.

Every consequential statement in this volume has a status and a boundary. Derivations establish relations under named assumptions. Finite checks exhaust a stated state space, not every deployment. Simulations estimate behavior under a stated generator; repository tests exercise named paths; running code proves availability, not universal mediation. Counterexamples remain in the record because they mark claims the research killed rather than conclusions it merely failed to prove.

Evidence is typed as well. When a chapter reports what a harbor observed, it names the channel that produced the observation, and each channel has a blind spot that no amount of volume removes:

- A *deterministic verifier* (a test, a type checker, a model checker) shows that a stated property holds of the artifact it examined, and nothing about properties it did not state.
- A *provenance witness* (a signature, a hash, a Merkle inclusion, an append-only log) shows attribution and tamper-evidence relative to a commitment, never that every event was logged or that a description is true.
- A *protected meter* (a counter its subject cannot rewrite) shows a quantity, not its cause.
- *Expert judgment* shows an assessment under a named rubric and carries the assessor's error rate.
- *Buyer preference* shows what was chosen, not that the choice was right.
- A *delayed outcome* shows what happened, and arrives too late to prevent it.

The reader should ask the same question of any dashboard: which channel, and what can it not show.

Section A gives the implementation boundary; Section C records the results and their limits. The companion `docs/harbor-research/mega-volume-epistemic-manifest.yaml` preserves each result's source, status, conditions, boundary, chapter home, and verification artifact. The chapters stand alone; the book's claim is the chain they make.

0.2 Classical Multi-Agent Systems Foundations

Before analyzing neural coding agents, we formalize the foundational frameworks developed during the classical era of multi-agent systems.

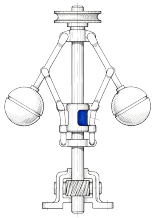
0.2.1 Intentional Agency and Practical Rationality

Classical decision theory models an agent as an expected utility maximizer balancing subjective beliefs $\mathcal{B}$ and desires $\mathcal{D}$. In his seminal 1987 work *Intention, Plans, and Practical Reason*, philosopher

Michael Bratman argued that intentions have a distinctive role in the practical reasoning of resource-bounded agents.

M. E. Bratman. *Intention, Plans, and Practical Reason*. Harvard University Press, Cambridge, MA, 1987.

A resource-bounded agent cannot continuously compute optimal policies from first principles at every millisecond. Practical rationality requires a third mental attitude: **Intention** ($\mathcal{I}$). Intentions are not merely desires with high utility; they represent *conduct-controlling pro-attitudes to which the agent has actively committed*.



Like a centrifugal flywheel governor throttling steam to stabilize rotational speed, practical rationality stabilizes an agent's intentions against transient sensor noise.

Bratman established four defining functional roles of intentions:

1. **Drive Means-End Reasoning:** An intention to achieve goal ϕ obligates the agent to formulate sub-plans and allocate computational cycles until ϕ is satisfied.
2. **Filter of Admissibility:** Once an intention ϕ is adopted, candidate options inconsistent with ϕ (given current beliefs $\mathcal{B}$) are ruled inadmissible without evaluating their cost-benefit payoff.
3. **Stability and Commitment:** Intentions resist reconsideration in the absence of relevant environmental interrupts or belief updates, preventing computational paralysis.
4. **Coordination Scaffolding:** Intentions enable interpersonal coordination: other agents can form expectations, subject to reliability and enforcement assumptions, regarding an agent's future actions based on its declared commitments.

0.2.2 The Rao–Georgeff BDI Temporal Logic (BDI_{CTL^*})

Anand S. Rao and Michael P. Georgeff. Modeling Rational Agents within a BDI-Architecture. In *International Conference on Principles of Knowledge Representation and Reasoning (KR)*, pages 473–484, 1991.

A. S. Rao and M. P. Georgeff. *BDI agents: From theory to practice*. In *Proc. 1st International Conference on Multi-Agent Systems (ICMAS)*, pages 312–319, 1995.

Example: A goal survives a failed attempt. Alice asks a worker to repair a failing test. The worker believes that revision 7 fails, wants a passing test, and commits to investigating the failure. Its first patch does not compile. That observation refutes this attempted patch; it does not establish that the repair is impossible. A different patch, a dependency fix or a request for help may remain available. An intention tells us what persists while those alternatives are considered.

Rao and Georgeff formalize distinctions between beliefs, selected goals and intentions in branching-time models. A possible world represents a way circumstances could be; its branches represent possible continuations. A belief modality quantifies over worlds the agent regards as possible. It is an idealized model of

reasoning, not a claim that an interpreter stores every world or that a language model performs deduction.

For a small example let $W = \{w_7, w_8\}$, where the subscript is the repository revision, and let $B_a \subseteq W$ be agent a's nonempty set of possibilities. Write $\text{Bel}_a\, \phi$ when proposition ϕ holds in every world in B_a. If $B_a = W$, the agent does not believe the revision is 7. It also does not believe it is 8. If a trusted read narrows B_a to $\{w_7\}$, it believes revision 7 in this model. A stale read can make that belief false of the current repository. This distinction between a modeled belief and an authoritative observation returns in the confinement and supervision chapters.

0.2.2.1 Compatibility and commitment

An intention should remain subject to feasibility checks. Rao and Georgeff's realism conditions relate belief, goal and intention worlds; their full branching-time semantics are stronger and more specific than simply requiring three lists to overlap. We use the practical distinction here without claiming that the kernel implements their modal logic.

0.2.3 Formal Commitment Strategies

A commitment strategy states when reconsideration is permitted. The following summaries follow the distinctions in Rao and Georgeff; they are not transcriptions of the paper's axiom schemas. Losing a belief that a repair is possible is not the same as believing

Rao & Georgeff 1991

Strategy	Persistence condition	Repair example
Blind	Persist until believed achieved.	Continue even after the required resource disappears.
Single-minded	Persist while achievement is still believed possible and not yet achieved.	Reconsider when that feasibility belief is lost.
Open-minded	Persistence also depends on retaining the goal.	Reconsider after Alice withdraws the repair request.

Table 0.2. Commitment governs reconsideration, not the success of a plan.

it impossible. In the two-world example, uncertainty can remove a belief without establishing its negation. This matters when a tool

error merely removes confidence. The temporal "until" used for persistence in the cited model is *weak until*: persistence may continue forever if its release condition never occurs. It therefore does not prove eventual completion.

Design Invariant 0.2.1 (Commitment Persistence).
An agent's recorded intention ι must not be abandoned merely because a single step incurs a transient tool failure. Intention reconsideration is an explicit state transition triggered by stated evidence, objective discharge, or authorized withdrawal or policy release. Context truncation is not itself such a transition.

0.2.4 Operational Semantics of AgentSpeak(L) and Normative Extensions

R. H. Bordini, J. F. Hübner, and M. Wooldridge. *Programming Multi-Agent Systems in AgentSpeak using Jason*. John Wiley & Sons, 2007.

AgentSpeak(L) makes the distinction operational. A belief base stores propositions, a plan library supplies triggered procedures, and an intention stack records the remaining steps of an adopted plan. An event selector chooses a pending event; an applicable-plan selector chooses a plan whose trigger matches and whose context holds; an intention selector chooses which stack advances. These are separate choices: having a recovery plan in the library does not establish that it is applicable or scheduled.

For Alice's repair, "test failed" may trigger an investigation plan. A context condition requiring a reachable test runner must still hold. If the runner is down, selecting that plan and successfully running it are different questions.

Jason developers. *Jason FAQ: AgentSpeak Language*. Official documentation, accessed 26 October 2026. Primary documentation.

Jason developers. *Jason interpreter: goal-failure handling*. Source revision `4b98ddb1f9c8`, accessed 26 October 2026. Pinned source.

Failure handling here follows the Jason variant of AgentSpeak. An empty applicable-plan set is a plan-selection failure, not evidence that a host action was attempted and failed; reported action failure is a separate cause. For an intention pursuing achievement goal g, Jason can seek a matching $-!g$ handler in that goal's context or an enclosing goal's context. A handler is usable only if its trigger matches and its context holds under the current beliefs. Failure can propagate, and an intention without a usable handler may be dropped. These cases follow separate interpreter paths. Neither a failure event nor handler eligibility guarantees next-cycle execution or eventual recovery, and earlier external effects are not automatically undone.

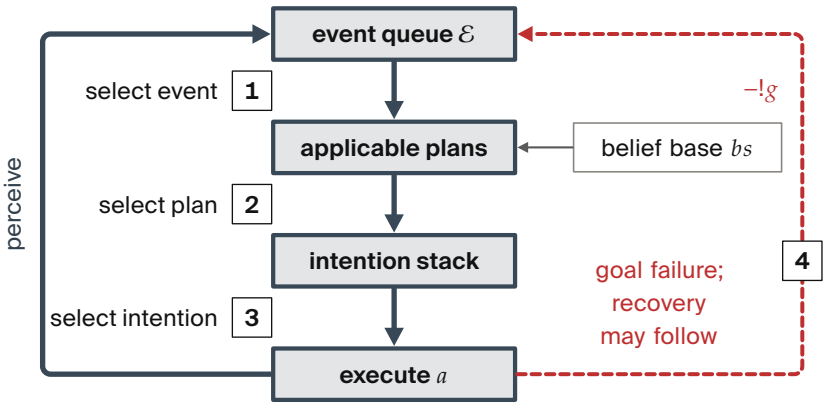


Figure 0.9. BDI selection flow. Event, applicable-plan and intention selection determine the next action. Goal-failure handling needs a matching, applicable plan; recovery is not guaranteed. The diagram illustrates control flow, not successful execution.

Selection	Operational role
1 $S_E(\mathcal{E})$	Choose an event: perception or an added goal.
2 $S_O(\text{App})$	Choose an applicable plan; push it on the intention stack.
3 $S_I(is)$	Choose the intention that advances by one action.
4 $-!g$	A goal-context failure may raise a matching handler event; recovery needs a currently applicable plan and later selection.

Figure 0.9 shows how event, applicable-plan and intention selection advance the BDI cycle, with a conditional return through goal-failure handling.

0.2.4.1 Normative Extensions: Deontic Constraints and Sanction Costs

Numbers by Hand 0.2.2 (Two incompatible obligations). Suppose one active norm requires retaining a diagnostic record and another requires deleting that same record immediately. In this stipulated example, violating retention costs 4 units and violating deletion costs 9. Assume the only feasible choices satisfy exactly one norm. Retaining incurs 9; deleting incurs 4. Minimizing the largest omitted sanction therefore chooses deletion. This is a decision rule under supplied costs, not a proof of legal or ethical priority. The chosen violation still needs an explanation and a record.

In institutional multi-agent systems, agents are constrained by external norms (deontic logic). For this example, represent a norm by the six-tuple $N = \langle \text{id}, \text{Type}, \phi_{\text{act}}, \phi_{\text{tgt}}, d, \text{Sanction} \rangle$, where Type $\in \{\mathcal{O}, \mathcal{F}, \mathcal{P}\}$ (Obligation, Prohibition, Permission), ϕ_{act} is the activation

context, ϕ_{tgt} is the target state, d is the deadline, and Sanction is the penalty for violation.

An implementation may store general norms in an **Abstract Norm Base** (ANB) and instantiate them in a **Norm Instance Base** (NIB). For a finite decision example, let $\mathcal{N}$ be the active norm set and let $\emptyset \neq \Omega \subseteq 2^{\mathcal{N}}$ be a finite family of subsets that are feasible under the stated action constraints and conflict-free under the stated norm-conflict relation. Give each norm a stipulated finite nonnegative sanction cost $c(n)$ for omission. One possible **worst-case sanction-cost** rule is

$$S^* \in \arg\min_{S \in \Omega} \max\big(\{c(n) : n \in \mathcal{N} \setminus S\} \cup \{0\}\big). \tag{0.5}$$

Here $\mathcal{N}$ is finite; including 0 makes the maximum zero when no norm is omitted. This minimizes the largest stipulated omitted sanction, not regret relative to a counterfactual alternative. It supplies no audit record by itself. If immutable evidence is required, a separate logging mechanism must commit the chosen subset, any deliberate violations and their justification.

0.2.4.2 Why BDI Matters for LLMs: Curing Intention Drift and Hallucinatory Abandonment

The useful connection is a separation of jobs. A proposal generator can suggest a new patch; a durable task record keeps track of what Alice still requested. A compiler error can justify another attempt, a blocked state or an escalation. It cannot by itself justify marking the obligation complete or proving that no solution exists. Likewise, omitting the goal from a shortened prompt must not silently delete it from the task record.

This is a design use of commitment theory, not evidence that a language model has BDI mental states or that a particular kernel implements AgentSpeak. Chapter 1 supplies the durable work-unit transitions; Chapter 5 asks which obligations a replacement worker inherits. We return to the same failed repair when considering timeouts: persistence also requires knowing whether an effect already happened before attempting it again.

0.2.5 Concurrency Foundations: Hewitt–Agha Actors vs. Hoare's Communicating Sequential Processes

When multiple agents deliberate and execute concurrently on a developer's workstation, the concurrency substrate dictates whether the swarm operates harmoniously or degenerates into deadlocks and corrupted file state. Distributed systems research provides two foundational, mathematically rigorous concurrency paradigms:

- **Hewitt–Agha Actor Model (1973, 1986):** The Actor Model treats the **actor** as the universal primitive of concurrent computation. An actor is an autonomous, computational entity that encapsulates private state, a mailbox (message queue), and behavior. In response to a message, an actor can concurrently perform exactly three operations:

  1. *Send* a finite number of messages to other actors with known addresses;
  2. *Create* a finite number of new actors;
  3. *Designate the replacement behavior* (`become`) to be used for the next message it receives.

  Actors share **no mutable state**. Message passing is fundamentally **asynchronous**: sending is non-blocking, and messages are buffered in the recipient's mailbox.

C. Hewitt, P. Bishop, and R. Steiger. *A universal modular ACTOR formalism for artificial intelligence*. In *Proc. 3rd International Joint Conference on Artificial Intelligence (IJCAI)*, pages 235–245, 1973.

G. Agha. *Actors: A Model of Concurrent Computation in Distributed Systems*. MIT Press, Cambridge, MA, 1986.

- **Hoare's Communicating Sequential Processes (CSP) (1978):** CSP formalizes concurrent computation around **processes** communicating over synchronous, unbuffered **channels**. The core principles of CSP are:

  1. *Synchronous Rendezvous:* Communication is an unbuffered handshake. The output event $c!v$ on channel c and the input event $c?x$ constitute one joint communication event in the abstract semantics. Whichever process reaches the communication point first blocks until the other process arrives.
  2. *Guarded Choice (Alternative):* A process can wait on multiple communication events simultaneously using the alternative construct (`alt`), proceeding with whichever channel becomes ready first.

C. A. R. Hoare. *Communicating sequential processes*. *Communications of the ACM*, 21(8):666–677, 1978.

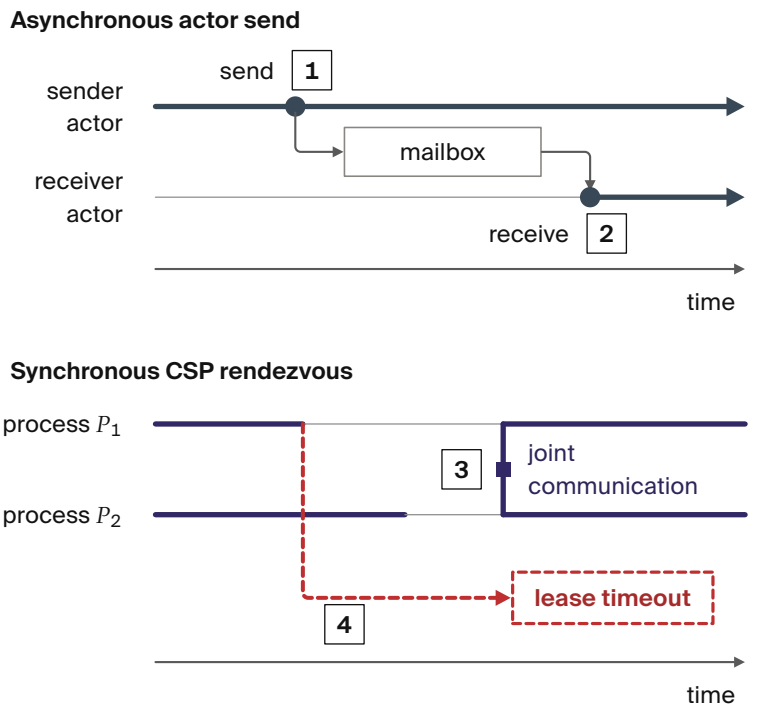


Figure 0.10. Two communication patterns. Buffered task delivery separates sender and receiver timing. An admission exchange couples a request to its resource decision. Either actors or synchronous processes can implement safe resource ownership with the appropriate protocol.

Event	Semantics
1 Actor send	Sender proceeds without receiver readiness.
2 Actor receive	Receiver drains the mailbox and may adopt a new behaviour.
3 CSP rendezvous	One abstract joint event; the earlier arrival waits.
4 Timeout choice	An optional modeled guard offers another transition.

Delays are illustrative. Ordering depends on the actor implementation; rendezvous does not imply zero physical communication latency.

Figure 0.10 details the dual concurrency architecture: Hewitt–Agha virtual actor isolation for stochastic deliberation, and Hoare's Communicating Sequential Processes (CSP) synchronous rendezvous for physical state mutations.

0.2.6 From Asynchronous Isolation to Synchronous Rendezvous

The fundamental divergence between Hewitt's Actor model and Hoare's Communicating Sequential Processes reflects an architectural trade-off between *isolation* and *synchronization*:

1. **Asynchronous Mailboxes vs. Unbuffered Barriers:** In the Actor model, communication is non-blocking and decoupled in time. An agent posts a message to a peer's mailbox and immediately resumes local execution. This temporal decoupling helps keep a supervisory loop responsive while workers perform long-running inference. A synchronous design can achieve similar responsiveness by separating those waits into worker processes. However, unbounded asynchronous mailboxes introduce severe hazards: message ordering is nondeterministic, queue backpressure is difficult to meter, and an agent can be flooded with stale context deltas.
2. **Synchronous Rendezvous for Scarce Resources:** An unbuffered send waits for a corresponding receive. This can express an admission handshake, but does not itself establish mutual exclusion, durability or deadlock freedom. Those properties depend on the protocol surrounding the handshake.
3. **The Synthesis in an Agent Operating Kernel:** Buffered task delivery separates deliberation from resource admission. A resource manager can serialize conflicting operations, validate versions and issue fencing tokens. It may itself be implemented as an actor or a synchronous process; correctness is a property of its transition rules and actuator, not of the concurrency model's name.

0.2.6.1 Why Concurrency Lineages Matter: Protocol Obligations

An actor can safely own a shared resource and process requests in order. A CSP network can avoid deadlock through its protocol structure and suitable timeout or cancellation behavior. Conversely, either model can contain an erroneous resource protocol. In the design studied here, workers deliberate asynchronously and request authority at an explicit admission boundary. Leases need fencing so that an expired holder cannot later act; a timeout by itself does not prevent a stale write.

0.2.7 Communicative Action and Speech Acts (FIPA-00037)

A message has a role in a conversation as well as a byte representation. **Speech-act theory** distinguishes what is said, the act performed in saying it, and the consequences of saying it. Austin distinguishes locutionary acts (meaningful utterances), illocutionary acts (such as requests or promises), and perlocutionary acts (such as persuading a listener). Searle develops a systematic account of illocutionary acts. The Foundation for Intelligent Physical Agents standardized the **FIPA ACL Communicative Act Library** (FIPA-00037).

J. L. Austin. *How to Do Things with Words*. Oxford University Press, 1962.

J. R. Searle. *Speech Acts: An Essay in the Philosophy of Language*. Cambridge University Press, 1969.

FIPA. *FIPA ACL Communicative Act Library Specification*. Technical Report SC00037J, Foundation for Intelligent Physical Agents, 2002.

Consider an operator saying, "Please test revision 7." The words identify a particular requested action: that is their meaning. In this setting the operator is making a request, rather than reporting a completed test: that is its force. The request may persuade the worker to stop another task and run the test: that is a possible consequence. A worker can understand the request and still refuse it. Recording the utterance therefore does not establish that the requested work occurred, or that an obligation was accepted.

FIPA distinguishes the sender's feasibility conditions from the intended rational effect of a communicative act. An intended effect is not a guarantee that a receiver believes, agrees or acts. For our repair workflow, the following illustrative message kinds keep different protocol events apart. This table is not a reproduction of FIPA's formal semantics.

Table 0.3. Illustrative workflow messages inspired by communicative-act distinctions.

Kind	Example content	What remains to check
`request`	Run the test on revision 7.	Authorization and acceptance.
`inform`	The test passed.	Which run, revision and evidence?
`propose`	I can do it for 20 units.	Terms and explicit acceptance.
`agree`	I accept job J under those terms.	Whether the protocol records a binding acceptance.
`refuse`	I cannot accept job J.	Whether another worker is needed.
`failure`	Attempt A did not complete.	Which effects, if any, already occurred?

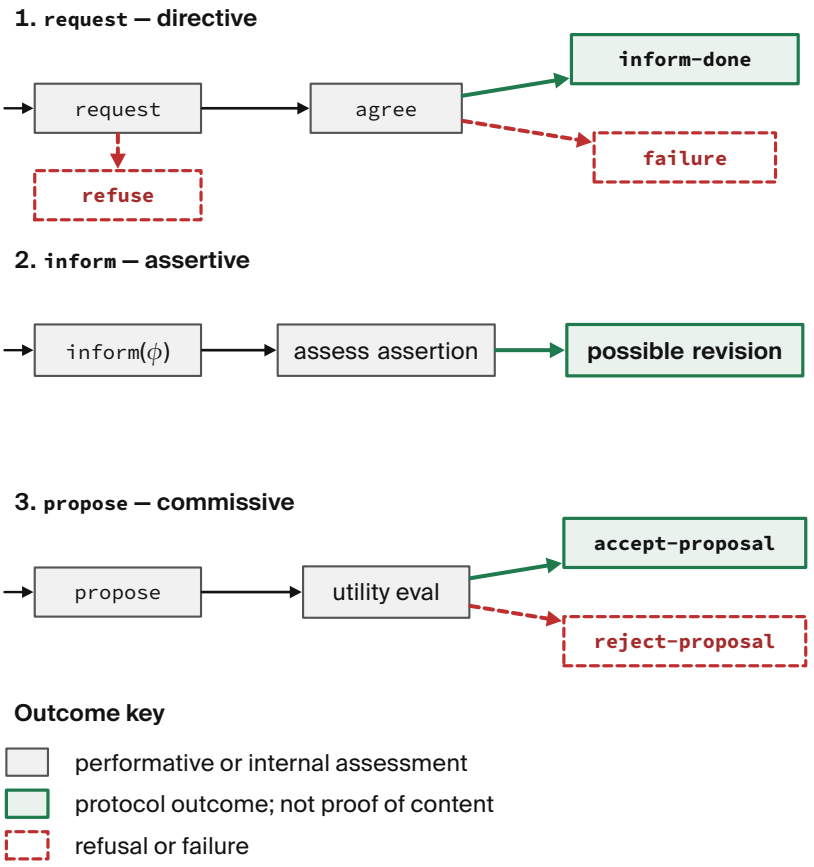


Figure 0.11. Message intent and protocol state. A request asks for action, an inform message asserts a proposition, and a proposal offers terms. Authentication identifies the sender; it does not make an assertion true or a proposal binding.

0.2.7.1 Why Speech Acts Matter: From Conversational Fluff to Cryptographic Envelopes

"I will look into it" might acknowledge receipt, express willingness or accept an obligation. A protocol can remove this ambiguity by requiring a message kind, job identifier, sender, recipient and explicit terms. A signature authenticates those bytes under a key; it does not make "the test passed" true. An `agree` message becomes an acceptance only if the governing protocol admits and records it.

Schema validation and authentication therefore help with distinct checks. They do not remove malicious content from a message body or prove compliance with a promise. Chapter 2 develops authority, and Chapter 6 separates a proposal, acceptance, delivery evidence and settlement. These are design requirements, not a claim that every existing messaging endpoint implements this envelope.

0.2.8 Task Allocation: Smith's Contract Net Protocol (1980)

R. G. Smith. *The Contract Net Protocol: High-level communication and control in a distributed problem solver*. *IEEE Transactions on Computers*, C-29(12):1104–1113, 1980.

Reid G. Smith's (1980) **Contract Net Protocol (CNP)** formalized bilateral market-inspired negotiation between a Manager and distributed Contractor agents. A possible coding-workflow adaptation has four stages:

1. **Call for Proposals (CFP Broadcast):** The Manager broadcasts an announcement $\langle \texttt{task_id}, \texttt{eligibility}, \texttt{spec}, t_{\text{expire}} \rangle$ specifying required toolsets, maximum spend budget, and deadline.
2. **Bidding and Capability Self-Assessment:** Eligible contractor agents inspect their current local capacity, sandboxed tools, and model context before submitting formal bids $\langle \texttt{bid_id}, \texttt{cost}, \texttt{eta}, \texttt{reputation} \rangle$, or returning explicit refusals if overloaded.
3. **Bid Evaluation and Award:** The manager selects among eligible bids using a declared rule. For example, among bids meeting the same deadline and acceptance specification, select the lowest price. Resource reservation requires a separate admission step.
4. **Execution, Interim Streaming, and Settlement:** The contractor checks out an isolated worktree branch, streams structured progress deltas, commits the diff, runs verification tests, and submits cryptographic execution receipts to unlock escrow payment.

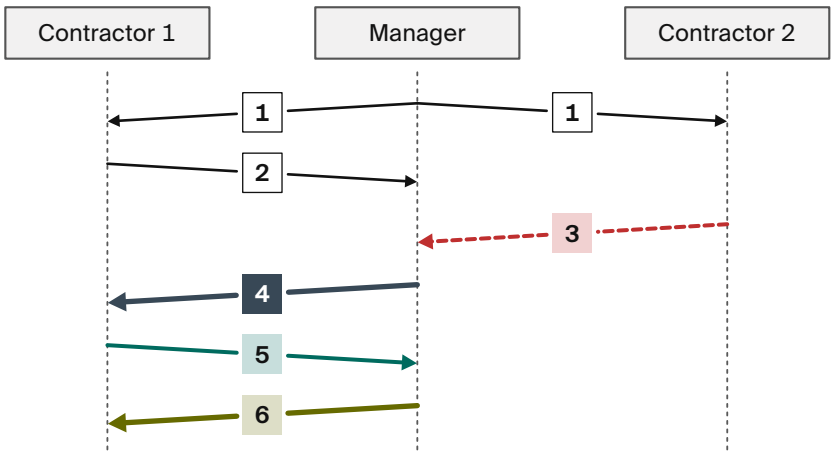
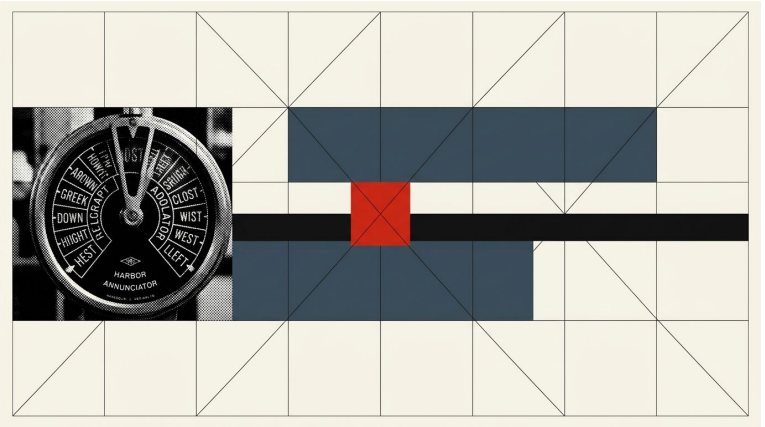


Figure 0.12. Contract-net task allocation. A manager announces a task, evaluates bids and awards work. A contractor returns evidence for contract-specific acceptance and settlement. Allocation, execution authority and payment verification are separate obligations.

#	message	payload
1	call for proposals	task `build-kernel-ffi`; spend cap $0.00; tools `cargo`, `git`, `seatbelt`
2	bid	eta 12 min; cost $0.00; reputation 0.98
3	refusal	lock contention — the contractor is busy
4	award	explicit contract terms and exclusive worktree lease
5	receipts	admitted execution result, diff and verifier test run
6	settlement	release, refund or slash under agreed verifier terms

Local authority is checked before effects; an award does not bypass admission.



■ **Architectural Plate 0.A · Task Allocation Dynamics (Contract Net Protocol)**

"Workers assess local capacity; a manager selects a bid. Resource authority and acceptance evidence remain separate checks."

0.2.8.1 Why CNP Matters: Selection Is Not Acceptance

In any multi-agent engineering swarm, allocating tasks across workers presents a fundamental distributed systems dilemma:

1. **The Bottleneck of Central Planning:** If a centralized orchestrator agent must manually inspect each worker's context window, parse their local git branches, and micromanage work assignment, the orchestrator becomes a severe throughput bottleneck and single point of cognitive failure.
2. **The Chaos of Greedy Contention:** Conversely, if workers greedily claim tasks from an uncoordinated queue, multiple agents race to modify the same files, squandering token budgets on overlapping AST diffs and inducing git merge conflicts.

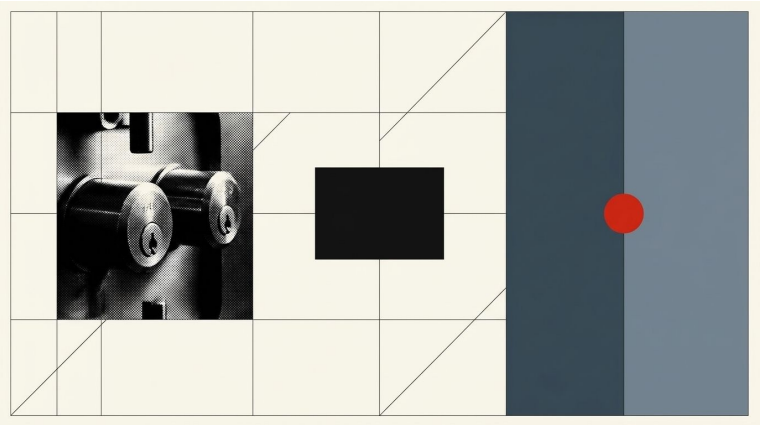
> **Numbers by Hand 0.2.3 (A cheaper bid can still fail acceptance).**
>
> Two eligible workers offer to test revision 7 before the same deadline, for 20 and 24 units. The lowest-price rule selects the first. It returns a passing receipt for revision 6. The selection rule has worked; the delivery has not met the agreed specification. Award, permission to execute and accepted delivery are three different events. Chapter 6 develops their ledger consequences.

Smith 1980

Reid Smith's Contract Net Protocol (CNP) supplies a negotiation pattern for allocating tasks. Applying it to a coding workflow requires separate contracts for resource ownership and settlement:

- **Proposed task-allocation ceremony.** Broadcast a task and budget; receive capability statements and bids; select a bid under a declared rule; reserve the required resources; and settle against specified delivery evidence. Bidding alone does not establish that the winner can safely execute the work. The checked-in dispatch runner plans queued work for a selected backend, budget, and isolated worktree. The Harbor Work Register records cooperative claims and refuses a second live holder. Those components do not by themselves implement competitive bidding or escrow settlement.
- **Confinement and evidence obligations.** Execution confinement and spend controls depend on the selected execution

path. A settlement contract may require authenticated test receipts and commit digests, but these bind an identified run and artifact to a signer; they do not prove every semantic property of the work. Resource reservation, verifier trust, payment authority, and the conditions for refusing settlement must each be specified.



■ **Architectural Plate 0.B · Bilateral Escrow Settlement**

"A settlement contract can require signed test receipts and commit digests; those receipts attest to a run, not to every property of the delivered software."

0.2.9 Dynamic Epistemic Logic and Big Brother Logic

Numbers by Hand 0.2.4 (A hidden bit and an observed effect).

A private file contains a bit, either 0 or 1. A worker sees the same message "ready" in both cases. Relative to this observation model, it cannot tell which bit is stored. Now suppose the worker also sees an exit code equal to the bit. Hiding the file no longer hides the fact. Confinement must account for both observations. Chapter 3 develops this problem for sequences of releases.

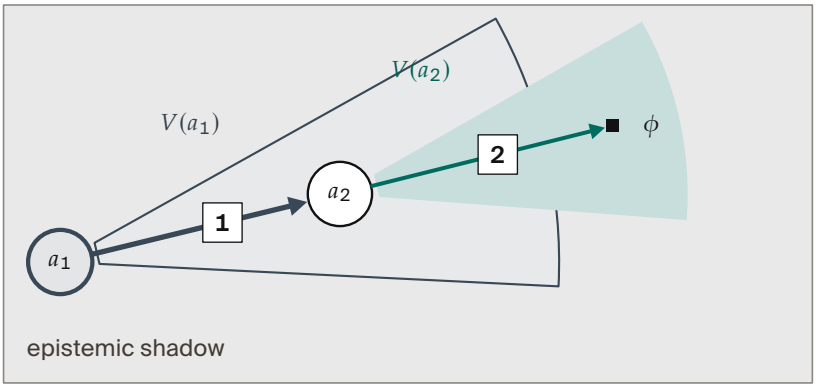


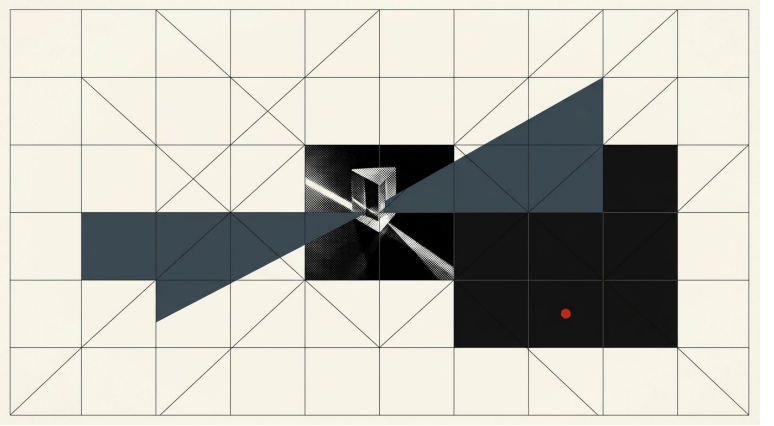
Figure 0.13. Asymmetric visibility. The observation graph records who sees which modeled object. Visibility is not automatically transitive: seeing an agent does not establish knowledge of everything it knows. Host confidentiality needs an additional channel model.

	condition	reading
1	$a_2 \in V(a_1)$	a_1 observes a_2
2	$\phi \in V(a_2)$	a_2 observes ϕ
$1 \wedge 2$	known $V(a_2)$ and observation semantics	may yield $K_{a_1} K_{a_2} \phi$; no direct visibility or covert-channel guarantee

Let w_0, w_1 denote these two states. Write $wR_i w'$ when agent i cannot distinguish the states using its allowed observations. The assertion $K_i \phi$ means that proposition ϕ holds in every state accessible from the current one by R_i. Before the exit-code observation, both states are accessible; after it, only the matching state remains. Unlike the belief example above, a knowledge model includes the actual state among its possibilities.

Charrier et al. study an observation model called Big Brother Logic, using cameras and fields of view. Its geometrical restrictions make asymmetric visibility concrete. The engineering lesson is the need to specify observations before asserting knowledge or secrecy. Seeing another agent is not the same as seeing all that agent's evidence.

T. Charrier, F. Ouchet, and F. Schwarzentruber. *Big Brother Logic: Reasoning about Agents Equipped with Surveillance Cameras in the Plane (Demonstration)*. In *Proc. AAMAS*, pages 1633–1634, 2014. Primary record.



■ **Architectural Plate 0.C · Dynamic Epistemic Cones & Confinement**

"Observation models specify who can distinguish which states; confinement must justify the correspondence to actual host channels."

0.2.9.1 Why Big Brother Logic Matters: Asymmetric Observation and Zero-Trust Confinement

In multi-agent systems research, information sharing is often naively treated as an unalloyed good: the more agents communicate, the better they coordinate. In production systems engineering, this assumption is catastrophic. Unrestricted visibility leaks proprietary credentials, inflates context windows with irrelevant chatter, and enables prompt-injection side-channels.

Observation models help separate three engineering obligations:

- **Recorded effects:** A supervisor needs authentic receipts for the events it claims to observe. Recording a tool-call request is not a complete observation of every effect of its subprocess.
- **Indistinguishability:** To model secrecy of a proposition ϕ at a world, require accessible alternatives with ϕ true and false, yielding $\neg K_i\phi \wedge \neg K_i\neg\phi$. Hidden files alone do not prove this if another observation reveals the same fact.
- **Confinement:** A sandbox controls specified host channels. Proving noninterference requires relating those channels and observations to the epistemic model. BBL's camera geometry

is an explanatory model, not a proof of operating-system confidentiality.

0.2.10 Game-Theoretic Foundations and Mechanism Design

Return to the deterrence example: gain 10, detection probability 0.1 and sanction 100 give deviation payoff $0.9(10) - 0.1(100) = -1$. If detection falls to 0.01, that payoff becomes $0.99(10) - 0.01(100) = 8.9$. The same bond no longer deters the deviation. Incentives depend on the information and enforcement model, not just the amount displayed in an account. Chapters 5–7 repeatedly use this distinction.

When agents have independent objective functions, coordination cannot assume altruism. Shoham and Leyton-Brown systematized the algorithmic and game-theoretic foundations of multiagent systems:

Y. Shoham and K. Leyton-Brown. *Multiagent Systems: Algorithmic, Game-Theoretic, and Logical Foundations*. Cambridge University Press, 2009.

1. **Normal-Form Games and Nash Equilibrium:** Let $N = \{1, \ldots, n\}$ index players, A_i be player i's action set, S_i its mixed strategies (probability distributions over actions), and u_i its expected payoff. Write s_{-i} for the other players' strategies. A strategy profile $s^* = (s_1^*, \ldots, s_n^*)$ is a Nash equilibrium if no agent can unilaterally deviate and increase its expected payoff:

$$\forall i \in N, \quad \forall s_i \in S_i, \quad u_i(s_i^*, s_{-i}^*) \geq u_i(s_i, s_{-i}^*) \tag{0.6}$$

2. **Mechanism Design and the Revelation Principle:** A mechanism specifies allocations and transfers. The revelation principle says that an outcome implementable in dominant strategies, or in Bayes–Nash equilibrium, has a direct mechanism with truthful reporting in the corresponding equilibrium sense. It does not say that every desired outcome is implementable. Here θ_i is a private type, $\hat{\theta}_i$ a report, k the allocation rule, v_i the value of an allocation to type θ_i, and p_i the payment charged to i. Quasi-linear utility is value minus payment. Dominant-strategy truthfulness requires, for every i, θ_i, $\hat{\theta}_i$ and $\hat{\theta}_{-i}$,

$$v_i(k(\theta_i, \hat{\theta}_{-i}); \theta_i) - p_i(\theta_i, \hat{\theta}_{-i}) \geq v_i(k(\hat{\theta}_i, \hat{\theta}_{-i}); \theta_i) - p_i(\hat{\theta}_i, \hat{\theta}_{-i}). \tag{0.7}$$

Bayesian incentive compatibility instead compares the corresponding expected utilities over other types under the stated prior.

3. **Vickrey–Clarke–Groves (VCG) Mechanisms:** For intuition, auction one indivisible slot. A bidder values it at 10, and the highest rival bid is 5. Winning and paying 5 gives utility 5; losing gives 0. Any bid above 5 wins at that same price, so truthfulness need not be the unique best bid. This second-price example is a special case. Assume quasi-linear private values, a report-independent feasible set $\mathcal{K}$, enforceable transfers without binding budget constraints, and exact maximization of reported total value. Write $\hat{v}_j(k) = v_j(k; \hat{\theta}_j)$ for the value induced by report $\hat{\theta}_j$, and let $k^*(\hat{\theta})$ maximize the sum of reported values. With a fixed tie rule, the Clarke pivot payment is

$$p_i(\hat{\theta}) = \max_{k \in \mathcal{K}} \sum_{j \neq i} \hat{v}_j(k) - \sum_{j \neq i} \hat{v}_j(k^*(\hat{\theta})). \tag{0.8}$$

Truthful reporting is *weakly* dominant: the agent's utility is total reported welfare with its own value reported truthfully, minus a term independent of its report. A truthful report selects a welfare maximizer; another report cannot improve it. Equality is possible. In a second-price auction, value 10 and highest rival bid 5 give utility 5 when bidding either 9 or 10.

4. **The bilateral-trade boundary:** Under independent private buyer and seller values with positive densities on intervals whose interiors overlap, Myerson–Satterthwaite rules out simultaneous Bayesian incentive compatibility, interim individual rationality, ex-post efficient trade and expected budget balance without outside subsidy. Efficiency is essential: no trade and no transfers satisfy the other requirements. VCG does not generally solve budget balance; in bilateral trade it can require a subsidy.

W. Vickrey. *Counterspeculation, auctions, and competitive sealed tenders*. *The Journal of Finance*, 16(1):8–37, 1961.

E. H. Clarke. *Multipart pricing of public goods*. *Public Choice*, 11(1):17–33, 1971.

T. Groves. *Incentives in teams*. *Econometrica*, 41(4):617–631, 1973.

R. B. Myerson and M. A. Satterthwaite. *Efficient mechanisms for bilateral trading*. *Journal of Economic Theory*, 29(2):265–281, 1983.

0.2.11 Social Choice Theory and Arrow's Impossibility Theorem

When a swarm of autonomous agents must aggregate individual preference orderings over candidate actions (e.g., selecting an architectural proposal, choosing a refactoring strategy), social choice theory governs the aggregation rules.

Numbers by Hand 0.2.5 (Three consistent voters, one cyclic majority).

The rankings are $A \succ B \succ C$, $B \succ C \succ A$, and $C \succ A \succ B$. Two voters put A above B, two put B above C, and two put C above A. Each voter is consistent; the pairwise majority relation cycles. This example refutes universal transitivity of majority ranking. It does not prove the more general theorem below, which concerns every rule satisfying its assumptions.

Let A be a finite set of alternatives and $N = \{1, \ldots, n\}$ be the set of voting agents. Each agent i has a strict preference ordering $\succ_i \in \mathcal{L}(A)$. A **Social Welfare Function (SWF)** is a mapping:

$$F : \mathcal{L}(A)^n \rightarrow \mathcal{L}(A) \tag{0.9}$$

that maps a profile of individual rankings $(\succ_1, \ldots, \succ_n)$ into a collective ranking $\succ$.

In 1951, Kenneth Arrow proved that a social welfare function on an unrestricted domain cannot satisfy the following criteria together with non-dictatorship:

K. J. Arrow. *Social Choice and Individual Values*. John Wiley & Sons, New York, 1951.

Theorem 0.2.6 (Arrow's Impossibility Theorem (1951))**.**
If $|A| \geq 3$, any Social Welfare Function $F : \mathcal{L}(A)^n \rightarrow \mathcal{L}(A)$ that satisfies:

1. ***Unrestricted Domain (U):*** *F is defined for all logically possible preference profiles.*
2. ***Weak Pareto (P):*** *If $\forall i \in N, x \succ_i y$, then $x \succ y$.*
3. ***Independence of Irrelevant Alternatives (IIA):*** *For any two alternatives x, y, the collective ranking between x and y depends solely on individual preferences between x and y.*

*is necessarily a **Dictatorship (D)**: $\exists i \in N$ such that $\forall x, y \in A,\ x \succ_i y \implies x \succ y$.*

0.2.11.1 What Aggregation Theorems Require

Preference aggregation and safety certification answer different questions. Arrow's theorem constrains how complete rankings are combined. It does not prohibit choosing a candidate by voting or prove that every vote deadlocks.

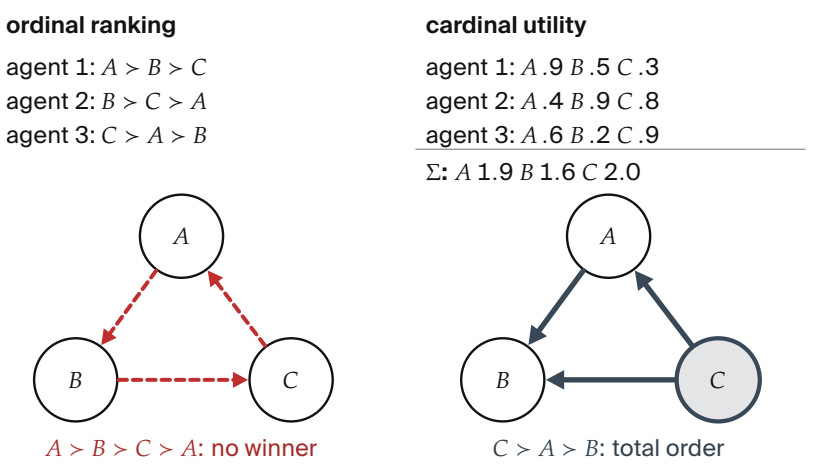


Figure 0.14. A majority-cycle witness. Three individually consistent rankings yield pairwise majorities $A \succ B$, $B \succ C$, and $C \succ A$. This is one possible profile. Arrow's theorem separately requires unrestricted domain, Pareto and IIA for complete and transitive collective rankings.

Condition	Ordinal scope
U: unrestricted domain	All strict ranking profiles are admitted.
P: weak Pareto	Unanimous pairwise preferences are respected.
IIA: independence	A pair depends only on its individual comparisons.
D: dictatorship	Forced by U, P, IIA and complete transitivity.

Cycles are possible, not inevitable. Arrow assumes a complete, transitive social order. Cardinal summation uses additional scale and comparability assumptions; it gives no unconditional governance guarantee.

1. **Cycles are possible:** The three rankings $A \succ B \succ C$, $B \succ C \succ A$ and $C \succ A \succ B$ produce the majority cycle $A \succ B \succ C \succ A$. Unanimous rankings do not. A procedure can still return a winner by a declared agenda or tie rule, while giving up other properties.
2. **Strategy-proof choice has hypotheses:** Gibbard–Satterthwaite applies to deterministic, onto, single-winner choice rules over at least three alternatives, with unrestricted strict preferences. A rule satisfying these conditions is either dictatorial or manipulable at some profile. It does not say every agent can manipulate every profile.
3. **The missing condition matters:** Pareto and a transitive collective ranking alone do not imply dictatorship. A Borda ranking with a fixed tie rule satisfies both on strict preferences but violates IIA. Arrow's impossibility requires IIA as well as unrestricted domain and Pareto.

An agent system must state which aggregation properties its task needs. Restricted preference domains, randomized choice,

A. Gibbard. *Manipulation of voting schemes: A general result*. *Econometrica*, 41(4):587–601, 1973.

M. A. Satterthwaite. *Strategy-proofness and Arrow's conditions: Existence and correspondence theorems for voting procedures and social welfare functions*. *Journal of Economic Theory*, 10(2):187–217, 1975.

cardinal reports with transfers, and ordinary voting rules have different guarantees. Cardinal bids and collateral alone do not establish truthful elicitation; VCG needs the assumptions just stated. For mechanically specified admission predicates, a test or proof checks that predicate. Passing a finite test suite does not establish all semantic correctness, and subjective judgments need an explicit decision and appeal rule.

0.2.12 The Ostrom Institutional Commons

When autonomous agents share developer workstation resources (TCP ports, CPU threads, disk space, git branches), they interact within a **Common-Pool Resource (CPR)**. Elinor Ostrom's Nobel prize-winning institutional economics disproved the dogma that common resources must inevitably suffer the "tragedy of the commons" or be privatized.

Ostrom 1990

Ostrom established eight core design principles for sustainable, self-governing institutions:

1. **Clearly Defined Boundaries:** Clear demarcation of who has access to which resources.
2. **Congruence Between Appropriation Rules and Local Conditions:** Resource consumption rules match environmental scarcity.
3. **Collective-Choice Arrangements:** Individuals affected by operational rules can participate in modifying them.
4. **Active Monitoring of Resource Condition and User Behavior:** Monitors are accountable to users.
5. **Graduated Sanctions:** Violators face escalating penalties rather than binary expulsion.
6. **Low-Cost Conflict Resolution Mechanisms:** Rapid, local arbitration of disputes.
7. **Minimal Recognition of Rights to Organize:** External authorities do not override self-governance.
8. **Nested Enterprises:** Governance activities are organized in multiple layers of nested institutions.

Chapter 7 uses these principles as design questions for the **Bonded Commons**: who may use a resource, who observes violations, and how can a ruling be challenged? Its conditional economic model does not establish that an implemented institution will be self-governing or resilient.

0.3 The Physics of Modern LLM Execution: What an Operating Kernel Must Know

To govern large language models, an operating system must understand the physical and computational realities of their execution. This section establishes the fundamental physics of modern LLMs: attention mechanics, memory bandwidth bottlenecks, training alignments, stochastic entropy, and context economics.

0.3.1 Transformer Mechanics: Self-Attention, Quadratic Complexity, and the KV-Cache

Numbers by Hand 0.3.1 (What an attention weight does).
For one query, suppose two admissible positions have equal scores and scalar values 2 and 4. Softmax converts scores z_j to weights $e^{z_j}/\sum_k e^{z_k}$; equal scores therefore give weights $1/2, 1/2$. The weighted output is $\frac{1}{2}(2) + \frac{1}{2}(4) = 3$. This arithmetic is a mixing operation, not evidence that either input statement is true. The cost of forming many such scores is the point of the following model.

The Transformer is a widely used architecture. Let $X \in \mathbb{R}^{L \times d}$ be an input sequence of L tokens projected into a d-dimensional embedding space. For one attention head, the input is mapped via learned weight matrices $W_Q, W_K, W_V \in \mathbb{R}^{d \times d_k}$ into Queries (Q), Keys (K), and Values (V):

$$Q = XW_Q, \quad K = XW_K, \quad V = XW_V \tag{0.10}$$

A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, L. Kaiser, and I. Polosukhin. *Attention Is All You Need*. In *Proc. 31st Conference on Neural Information Processing Systems (NeurIPS)*, pages 5998–6008, 2017.

Softmax is applied separately to each score row. A causal mask excludes future positions in autoregressive use. Multiple heads perform separate projections and combine their outputs; the following expression describes one head:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V \tag{0.11}$$

0.3.1.1 The Quadratic Attention Wall

The matrix product QK^T forms an $L \times L$ attention matrix representing the pairwise affinity between every token in the sequence. For dense attention that materializes the score matrix, the asymptotic costs are:

$$\text{Compute} = O(L^2 \cdot d), \qquad \text{Memory} = O(L^2 + L \cdot d) \tag{0.12}$$

T. Dao, D. Y. Fu, S. Ermon, A. Rudra, and C. Ré. *FlashAttention: Fast and Memory-Efficient Exact Attention with IO-Awareness*. In *Proc. 36th Conference on Neural Information Processing Systems (NeurIPS)*, 2022.

While FlashAttention uses GPU SRAM tiling to avoid materializing the full $L \times L$ matrix in High-Bandwidth Memory (HBM), the underlying FLOP count remains quadratic in context length L. For an agent maintaining an uncompacted context of $L = 128{,}000$ tokens, a single attention pass requires evaluating $\approx 1.63 \times 10^{10}$ token-pair affinities per attention head.

0.3.1.2 The Key-Value (KV) Cache and Memory Bandwidth Bottlenecks

W. Kwon, Z. Li, S. Zhuang, Y. Sheng, L. Zheng, C. H. Yu, J. E. Gonzalez, H. Zhang, and I. Stoica. *Efficient Memory Management for Large Language Model Serving with PagedAttention*. In *Proc. 29th ACM Symposium on Operating Systems Principles (SOSP)*, pages 611–626, 2023.

In autoregressive text generation, consider a dense decoder generating one token at a time: $x_t \sim P_\theta(x_t \mid x_{<t})$. Because previous tokens do not change, recomputing their Key and Value representations at every decoding step would result in $O(L^3)$ aggregate compute. Production inference engines (such as vLLM) therefore cache Key and Value vectors in high-bandwidth GPU memory (HBM).

For a transformer with N_l layers, N_h key-value heads, dimension d_h, sequence length L, and precision b_p bytes (typically 2 for FP16/BF16), the KV-cache memory consumption per active agent sequence is:

$$\text{Memory}_{\text{KV}} = 2 \times N_l \times N_h \times d_h \times L \times b_p \quad \text{bytes} \tag{0.13}$$

Consider an illustrative 70-billion parameter model configuration ($N_l = 80$, $N_h = 8$ using Grouped-Query Attention, $d_h = 128$, $b_p = 2$). At context length $L = 128{,}000$:

$$\begin{aligned} \text{Memory}_{\text{KV}} &= 2 \times 80 \times 8 \times 128 \times 128{,}000 \times 2 \text{ bytes} \\ &\approx 41.94 \text{ GB per sequence} \end{aligned} \tag{0.14}$$

A fleet of 8 concurrent agent workers requires ≈ 335.5 GB of high-bandwidth memory solely to store the KV-cache, exceeding a nominal 320 GB memory budget before allocating a single byte for model weights.

0.3.1.3 Arithmetic Intensity: Compute-Bound Prefill vs. Memory-Bound Decoding

A critical physical reality of GPU execution is **Arithmetic Intensity**: the ratio of floating-point operations (FLOPs) to memory access (bytes transferred):

$$I = \frac{\text{FLOPs}}{\text{Bytes Transferred}} \tag{0.15}$$

Prefill and decoding often have different bottlenecks:

1. **Prefill:** Processing many prompt tokens uses matrix multiplications and can achieve high arithmetic intensity. Whether it is compute-bound depends on batch size, sequence length, hardware and the attention implementation.
2. **Decoding:** Single-sequence dense decoding often spends substantial bandwidth reading weights and the KV cache. A dense 70-billion-parameter model stored at two bytes per parameter has 140 GB of weights before sharding or compression. Batching, quantization, sparse architectures and speculative decoding change the balance.

Long contexts therefore have a measurable memory and compute cost. The cache formula provides a capacity estimate for the stated configuration; latency and throughput require measurements of the actual inference system.

0.3.2 Training Regimes and the Alignment Illusion

Training changes how proposals are generated. Pretraining learns predictive patterns; supervised fine-tuning uses demonstrations of desired responses; preference training uses comparisons or rewards to favor selected behavior. These objectives are useful context, but their loss functions do not enter any proof in this book.

For Alice's job, training might make a worker more likely to request a test before claiming success. An admission rule can separately require a receipt for the specified revision. Improving the request probability and checking the receipt solve different problems. The distinction is the one introduced by the counter example: a good proposal is not yet a safely executed transition.

L. Ouyang, J. Wu, X. Jiang, D. Almeida, C. L. Wainwright, P. Mishkin, C. Zhang, S. Agarwal, K. Slama, A. Ray, et al. *Training language models to follow instructions with human feedback*. In *Proc. 36th Conference on Neural Information Processing Systems (NeurIPS)*, 2022.

R. Rafailov, A. Sharma, E. Mitchell, S. Ermon, C. D. Manning, and C. Finn. *Direct Preference Optimization: Your Language Model is Secretly a Reward Model*. In *Proc. 37th Conference on Neural Information Processing Systems (NeurIPS)*, 2023.

0.3.2.1 Alignment Objectives and Execution Invariants

Preference training can improve observed behavior. Its objective alone is not a proof that every permitted action preserves a host invariant. An apology or instruction remains evidence about the policy's behavior; an admission rule constrains which effects can occur. The distinction depends on the action interface and the verified controller, not on whether the proposer is neural or symbolic.

0.3.3 Sampling and Conditional Failure Hazards

For finite logits z and temperature $T > 0$, softmax assigns

$$P(w_i \mid x_{<t}) = \frac{\exp(z_i/T)}{\sum_j \exp(z_j/T)}. \tag{0.16}$$

Greedy decoding at $T = 0$ is a separate argmax rule with a specified tie policy. For nucleus sampling, order tokens so $P(w_1) \geq \cdots \geq P(w_m)$, fix a tie order, and let $0 < p \leq 1$. Following Holtzman et al.'s nucleus rule, retain the shortest prefix reaching probability p and renormalize:

$$k_p = \min\left\{ k : \sum_{j=1}^{k} P(w_j) \geq p \right\}, \qquad V^{(p)} = \{w_1, \ldots, w_{k_p}\}. \tag{0.17}$$

With probabilities $(0.95, 0.05)$ and $p = 0.9$, the nucleus is $\{w_1\}$. Truncation, action masks and tool validation can exclude unsafe actions even when token generation is stochastic.

Numbers by Hand 0.3.2 (Two steps of conditional risk).
Suppose a run survives its first step with probability 0.9 and, conditional on that survival, its second with probability 0.8. It survives both with probability $0.9 \times 0.8 = 0.72$. Independence is unnecessary because the second probability is already conditional. The theorem uses the same multiplication with upper bounds on each conditional survival probability.

Theorem 0.3.3 (Survival under conditional failure hazards)**.**
Let S_t mean that execution has remained safe through step t, with S_0 certain. Assume that after every reachable safe history before step t, the probability that the admitted effect *violates the invariant is at least $\epsilon_t \in [0, 1]$. Then*

$$\Pr(S_N) \le \prod_{t=1}^{N} (1 - \epsilon_t). \tag{0.18}$$

If one constant $\epsilon > 0$ lower-bounds every ϵ_t, then $\Pr(S_N) \le (1 - \epsilon)^N \to 0$. No independence assumption is required.

Proof idea. Bound survival at the next step conditional on having survived so far. Averaging the history-wise bound gives $\Pr(S_t \mid S_{t-1}) \le 1 - \epsilon_t$. Multiply these conditional probabilities. If any S_t already has probability zero, the conclusion is immediate. □

The hazard assumption must be measured or otherwise justified; positive temperature does not imply it. A policy restricted to safe actions has zero failure probability. Even positive hazards at every step need not force eventual failure: independent hazards $\epsilon_t = 2^{-t-2}$ have total at most $1/4$, so the union bound leaves survival probability at least $3/4$. This theorem explains how persistent admitted risk accumulates. It neither proves that every model must fail nor equates a proposed unsafe token with an executed unsafe effect.

0.3.4 The Context Window as Volatile, Lossy RAM

A prompt can contain an exact string without ensuring that a generated answer will retrieve or use it correctly. This distinguishes retaining bytes from successfully using their contents.

0.3.4.1 Position and retrieval

Nelson F. Liu, Kevin Lin, John Hewitt, Ashwin Paranjape, Michele Bevilacqua, Fabio Petroni, and Percy Liang. Lost in the Middle: How Language Models Use Long Contexts. *TACL*, 12:157–173, 2024. arXiv:2307.03172.

Liu et al. measured sensitivity to the position of relevant information in the models and tasks they tested. Performance was often better at the beginning or end than in the middle. These are task-performance results, not a universal percentage loss or a direct measurement of attention weights. The engineering question is whether the actual system retrieves the evidence needed for the current decision.

0.3.4.2 Compaction and retained evidence

Numbers by Hand 0.3.4 (Two histories, one summary).
History A says "test failed on revision 7"; history B says "test not run on revision 7." A summary that represents both as "testing pending" cannot recover which history occurred: two distinguishable inputs have one output. This is loss by construction, even if the summary itself contains no false sentence. Retaining a receipt and linking it to the summary permits a later reader to distinguish them. Chapter 4 asks how to allocate scarce summary space between continuing work and checking it.

Compaction can preserve particular facts exactly; loss is not inevitable for every query. Its contract must say which distinctions survive and which must be recovered from retained evidence. The task's authoritative obligations and resource permissions cannot depend solely on a model remembering them.

Design Invariant 0.3.5 (Externalized Durable State).
An agent operating kernel must never rely on the model's context window as the system of record. Authoritative locks, capabilities, claims, balances and evidence references need durable storage with a stated consistency and recovery contract. Chapter 1 studies a transactional database implementation; other implementations must discharge the same obligations. Prompt buffers contain working views of that state.

0.4 Modern Coding Agent Architectures, Planning, and Search

We now examine how modern LLM-based coding agents interface with host environments, structure deliberation, and execute search over complex problem spaces.

0.4.1 The Contemporary Landscape: Systems Comparison

Coding harnesses combine proposal generation with host tools. Compare an execution path by what it can establish, rather than treating a product name as a fixed security boundary.

Boundary	Question to answer for the actual path
Isolation	Which files, network destinations and subprocesses are reachable?
State	Which facts survive compaction and process replacement?
Admission	Can any actuator bypass permission or version checks?
Recovery	Can a retry duplicate a committed effect?
Evidence	Which artifact and revision does a success report identify?

Table 0.4. Questions for inspecting a coding harness. Later chapters supply the contracts.

0.4.2 Closed-Loop Deliberation and Partial Observation

Example: A timeout is not a failed write. A worker sends a write request and receives no response. In one possible state, the request never arrived. In another, it committed and its response was lost. Retrying blindly is safe in neither generality nor by virtue of persistence: a payment might be duplicated. The worker needs an operation identifier and an authoritative lookup, or an explicitly idempotent operation (repeating it has the same intended effect). Chapter 1 supplies the corresponding recovery obligations. This is why the earlier failed-plan example distinguished a failed observation from the state of the world.

A partially observable Markov decision process (POMDP) is one mathematical model for this distinction. Its tuple $\langle \mathcal{S}, \mathcal{A}, \mathcal{T}, \mathcal{R}, \Omega, \mathcal{O}, \gamma \rangle$ names states, actions, transition probabilities,

rewards, observations, observation probabilities and a discount factor $0 \leq \gamma < 1$. A belief state $b_t(s) = \Pr(s_t = s \mid h_t)$ is a probability distribution over states conditional on the action–observation history h_t. The timeout leaves probability on both states until evidence distinguishes them.

Calling a harness partially observable does not show that it computes a Bayesian belief state or solves a POMDP optimally. A closed loop means that observations can change subsequent actions. It permits correction; it does not guarantee recovery or make external effects reversible.

0.4.3 Deliberation, Planning, and Search Frameworks

Modern agents employ structured search and planning algorithms across candidate execution paths:

1. **ReAct (Reasoning and Acting):** Interleaves natural language reasoning traces ("Thoughts") with structured tool executions ("Actions") and environment feedback ("Observations"). An action can gather evidence that changes the next reasoning step. This interleaving permits inquiry; it does not guarantee that every uncertainty triggers a tool call.

2. **Tree of Thoughts (ToT):** Explores alternative intermediate reasoning steps in a tree. The studied searches proceed breadth-first (comparing a frontier of candidates) or depth-first (following a candidate and backtracking). Model evaluations guide exploration; they are heuristics, not proofs that a branch is correct.

3. **Reflexion:** Uses linguistic feedback without updating the model weights. The agent reflects on task feedback, retains that reflection in an episodic memory buffer, and consults it in later attempts. For example, a failed test can prompt a reminder to check the precondition that was missed.

4. **Voyager Lifelong Skill Learning:** In Minecraft, an agent accumulates an **executable skill library**: code for behaviors that later tasks can retrieve and compose. Environment feedback, execution errors and self-verification guide program improvement. The example shows how procedural memory can outlast a prompt; using the pattern for software work still requires

L. Wang, C. Ma, X. Feng, Z. Zhang, H. Yang, J. Zhang, Z. Chen, J. Tang, X. Chen, Y. Lin, W. X. Zhao, Z. Wei, and J.-R. Wen. *A Survey on Large Language Model based Autonomous Agents*. arXiv:2308.11432, 2023; revised 2025. Primary record.

Yao et al. 2023. *ReAct: Synergizing reasoning and acting in language models*.

Yao et al. 2023. *Tree of Thoughts: Deliberate problem solving with large language models*.

N. Shinn, F. Cassano, E. Berman, A. Gopinath, K. Narasimhan, and S. Yao. *Reflexion: Language Agents with Verbal Reinforcement Learning*. In *Proc. NeurIPS*, 2023. Primary record.

Wang et al. 2023. *Voyager: An Open-Ended Embodied Agent with Large Language Models*.

checking dependencies, permitted effects and task-specific correctness.

5. **TDAG:** Task Decomposition and Agent Generation dynamically decomposes a task and generates specialized subagents. Wang et al. evaluate it on ItineraryBench, a travel-planning benchmark. This supports an adaptive planning pattern; it is not a theorem about eliminating dependency cycles or serializing software mutations.

6. **HyperTree Planning:** Gui et al. construct and iteratively refine hierarchical planning outlines, evaluated on TravelPlanner. The structure supports decomposition of constrained plans. Its reported benchmark gains do not establish that software agents avoid local minima or produce correct repository changes.

Y. Wang, Z. Wu, J. Yao, and J. Su. *TDAG: A Multi-Agent Framework based on Dynamic Task Decomposition and Agent Generation*. arXiv:2402.10178, 2024; revised 2025. Primary record.

R. Gui, Z. Wang, J. Wang, C. Ma, H. Zhen, M. Yuan, J. Hao, D. Lian, E. Chen, and F. Wu. *HyperTree Planning: Enhancing LLM Reasoning via Hierarchical Thinking*. arXiv:2505.02322v2, 2025. Primary record.

0.4.4 Cognitive Control, Test-Time Compute, and Process Reward Models

Inference search can explore alternative reasoning chains scored by **Process Reward Models (PRMs)**. For input x, let $\tau = (c_1, \ldots, c_T)$ be a candidate chain and let the PRM assign each step a score in $[0, 1]$ given its prefix. One possible ranking heuristic multiplies these scores:

$$\text{Score}(\tau) = \prod_{t=1}^{T} \text{PRM}(c_t \mid c_{<t}, x) \tag{0.19}$$

This product is a joint correctness probability only when its factors are the appropriate calibrated conditional probabilities. For instance, scores 0.9 and 0.8 multiply to 0.72; without that calibration, the number does not mean a 72% chance of a correct answer. Otherwise it is a heuristic score. A search policy may backtrack when a score falls below a chosen threshold.

0.4.5 Multi-Agent Formal Verification and Benchmarking

Evaluating multi-agent software engineering requires moving beyond synthetic static benchmarks (such as HumanEval) toward interactive, sandboxed evaluation suites:

- **AgentBench:** A multi-dimensional benchmark evaluating LLMs as autonomous agents across operating system bash environments, databases, web browsing, and multi-agent coordination games.
- **HELM (Holistic Evaluation of Language Models):** Standardized multi-metric evaluation across accuracy, calibration, robustness, fairness, and toxicity.
- **FormalJudge:** Zhou et al. translate natural-language requirements into atomic constraints and use Dafny specifications with Z3 to assess compliance. Their agent-oversight benchmarks report improved detection. A proof concerns the encoded specification; correctness of its translation and of its connection to observed behavior remains a separate obligation.

X. Liu, H. Yu, H. Zhang, Y. Xu, X. Lei, H. Lai, Y. Gu, H. Ding, K. Men, K. Yang, et al. *AgentBench: Evaluating LLMs as Agents*. In *Proc. International Conference on Learning Representations (ICLR)*, 2024.

P. Liang, R. Bommasani, T. Lee, D. Tsipras, D. Soylu, M. Yasunaga, Y. Zhang, D. Narayanan, Y. Wu, A. Kumar, et al. *Holistic Evaluation of Language Models*. *Transactions on Machine Learning Research (TMLR)*, 2023.

J. Zhou, Y. Sheng, H. Lou, Y. Yang, and J. Fu. *FormalJudge: A Neuro-Symbolic Paradigm for Agentic Oversight*. arXiv:2602.11136v2, 2026. Primary record.

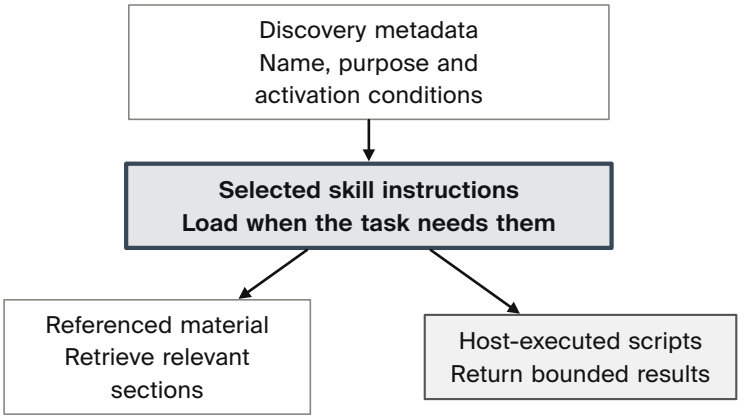
0.5 Tools, Protocols, Skills, and Hooks

To interact safely with the host environment, an agent harness must govern tool dispatch, encapsulate domain knowledge, and enforce runtime invariants.

0.5.1 Model Context Protocol (MCP) as the Substrate

The Model Context Protocol (MCP), specification 2025-11-25, standardizes client–server interfaces for tools and context. It does not by itself confine effects:

1. **Tools:** Server-exposed procedures invoked through the harness (`tools/call`), with JSON-Schema inputs and results that can contain content blocks or structured data.
2. **Resources:** Passive data endpoints (`resources/read`) exposing context such as local file contents, system logs, or git diffs.
3. **Prompts:** Reusable prompt templates parameterizing multi-step workflows.



Context savings depend on what is retrieved and what the tools return. This architecture alone does not establish a usage frequency or an exponential reduction.

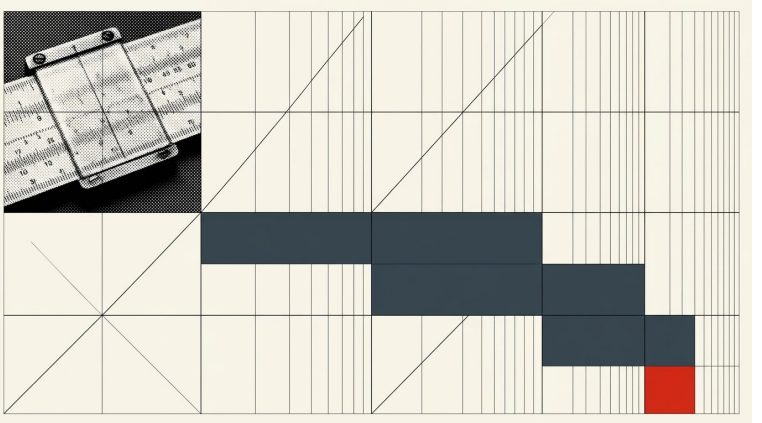
Figure 0.15. Staged loading. Keep an index, load selected instructions, then retrieve references or execute scripts as needed. Savings depend on actual access patterns and returned output; the tiers imply no universal activation frequencies.

0.5.2 Context Economics and Progressive Skill Disclosure

Progressive disclosure keeps an index in context and loads detailed instructions when needed. Its benefit depends on actual retrieval and task frequency:

1. **Index:** A concise name and description identify each skill. For illustration, 500 descriptions averaging 30 tokens consume 15,000 tokens, or 7.5% of a 200,000-token window. This is a sizing example, not a measured activation rate.
2. **Instructions:** A selected skill supplies operational guidance, parameters and recovery rules. Its context cost is the number of instruction tokens actually loaded.
3. **References and execution:** The harness reads relevant reference excerpts or runs a script without loading all its source into the model. Tool arguments, outputs and errors still consume context. Executing a script is not a zero-token interaction.

If the index costs b_0 tokens and asset i costs b_i when loaded with probability p_i, one loading round has expected cost $b_0 + \sum_i p_i b_i$. This follows by linearity of expectation without requiring independence. It gives no exponential saving unless a measured access distribution justifies one.



■ **Architectural Plate 0.D · Progressive Skill Disclosure Economics**

"A compact index directs selective loading; script outputs and reference excerpts determine the context actually consumed."

0.5.3 Lifecycle Hooks and Supervisory Control

Example: A write can be refused; a crash must be accommodated. Suppose the next event is either a requested write or a power failure. A gate can refuse the write. It cannot refuse the power failure. If safety requires recoverable obligations after either event, the current state must already retain enough information to survive the failure. Choosing "deny" at a post-crash hook is too late. This is the recovery half of the opening problem.

P. J. G. Ramadge and W. M. Wonham. *Supervisory control of a class of discrete event processes.* *SIAM Journal on Control and Optimization*, 25(1):206–230, 1987.

Ramadge–Wonham supervisory control models which events a supervisor can disable. A lifecycle hook can implement part of such a supervisor only when the modeled control and observation assumptions hold for its actual effect paths.

Let unconstrained agent execution be modeled as a discrete-event generator $G = \langle X, \Sigma, \delta, x_0, X_m \rangle$, where X is the state set, x_0 the initial state, δ the partial state-transition function, X_m the marked completion states, and events $\Sigma = \Sigma_c \cup \Sigma_u$ are partitioned into controllable events Σ_c (tool invocations, file writes, network requests) and uncontrollable events Σ_u (model token generation, host

hardware interrupts, external process termination). A trace is a finite event sequence. $L(G)$ contains the possible traces; $\mathcal{P}(\Sigma)$ denotes event subsets. A supervisor $S : L(G) \to \mathcal{P}(\Sigma)$ selects enabled events after each trace under full observation, always allowing uncontrollable events. If K contains the safe traces and is prefix-closed (every initial segment of a safe trace is safe), the desired controlled language satisfies:

$$L(S/G) \subseteq K \subseteq L(G) \tag{0.20}$$

where K is a prefix-closed language of safe traces. This inclusion is a design requirement, not an automatic property of a hook. For full observation, enforcing exactly K requires controllability: $K\Sigma_u \cap L(G) \subseteq K$. Otherwise one must find an enforceable sublanguage, or report that the initial state has no safe continuation. Partial observation adds observability requirements; completion adds a separate nonblocking obligation.

Figure 0.16 illustrates pre-effect admission and post-effect observation. The event names follow Claude Code's hook reference; the drawing is a teaching model, not a complete implementation map. Two useful levels are:

Anthropic. *Hooks reference*. Claude Code documentation, accessed 3 November 2026. Official documentation.

1. **The Outer Session & Turn Loop:** Encompasses initial context injection (`SessionStart`), prompt interception (`UserPromptSubmit`), context pressure management (`PreCompact`), turn completion checks (`Stop`), error notification (`StopFailure`), and session teardown (`SessionEnd`).
2. **The Inner Agentic Tool Loop:** Governs the iterative reasoning-action cycle: pre-tool interception and deterministic vetoes (`PreToolUse`), sandboxed execution, and reactive post-tool sanitization (`PostToolUse`).

0.5.3.1 Illustrative Hook Designs

The following constructed scenarios separate prevention, recovery, and retained context. They specify design obligations; they are not measured production runs.

Case A: Pre-tool refusal. An agent proposes a destructive command:

```
git checkout main &&
  git reset --hard origin/main
```

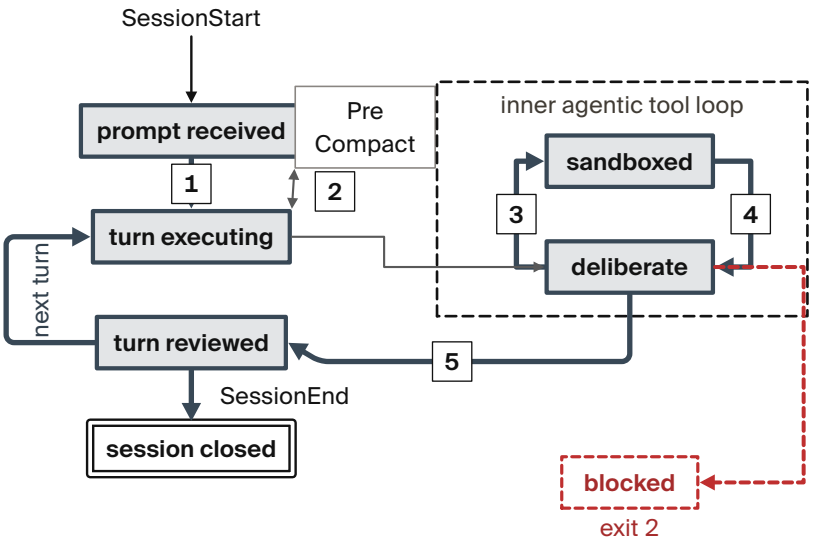


Figure 0.16. Hook ordering. A pre-execution gate can refuse a controllable action. Post-execution checks inspect effects and evidence. Closeout records the result. Prevention requires every relevant effect path to cross the gate before it occurs.

	Hook	Loop	Fires and may
1	`UserPromptSubmit`	outer	Inject or reject context before deliberation.
2	`PreCompact`	outer	Record a recovery checkpoint before compaction.
3	`PreToolUse`	inner	Veto a call before its mediated effect.
4	`PostToolUse`	inner	Record results after effects; compensation may be limited.
5	`Stop`	outer	Check closeout; request more work if needed.

PreToolUse can veto a mediated call before it runs. PostToolUse cannot undo irreversible effects. Hook names and veto behavior depend on the configured host contract.

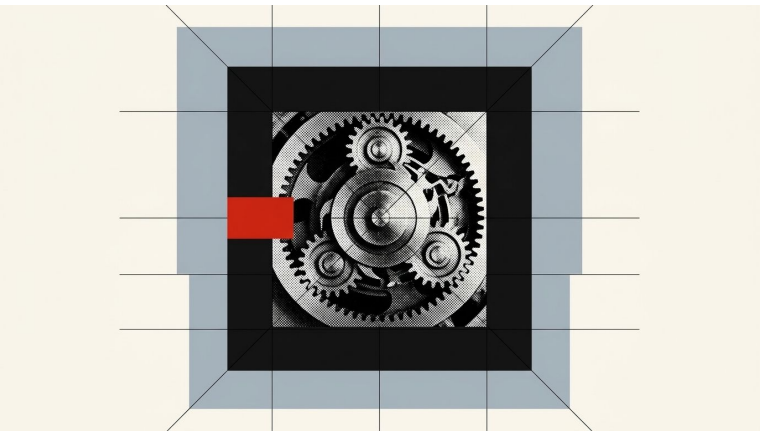
Anthropic 2026

A configured admission hook identifies a forbidden protected-branch mutation and refuses the tool call. In Claude Code, a blocking `PreToolUse` command hook can return exit code 2 with a reason on standard error. Other exit codes depend on the configured failure policy and are not this explicit veto contract. A short reason is sufficient: "Protected branch mutation refused." A shell-pattern check by itself does not establish complete mediation: alternate executables, indirect effects, and other write channels must also be controlled.

Case B: Post-tool recovery. A build writes unexpected generated files. A post-tool observer can compare the resulting paths with

a recorded baseline, report the deviation, and invoke a separately authorized recovery procedure for recoverable files. This happens after the write; it cannot undo a disclosure or arbitrary remote effect. Untracked files are not automatically disposable: ownership and recovery scope must be established before cleanup.

Case C: Compaction preparation. Before context compaction, a configured handler records current obligations, claim identifiers, failing checks, and source locators in a durable checkpoint. The successor loads and validates that record. The contract must define which fields are mandatory and how freshness is checked; neither a summary nor a hook name guarantees their preservation. Secrets remain in the credential store, with references in the checkpoint. No turn-count or context-percentage threshold is assumed here.



■ **Architectural Plate 0.E · Dual-Loop Supervisory Interception**

"Deterministic lifecycle hooks intercept unconstrained generation at binary boundaries, replacing conversational advice with discrete supervisory vetoes."

0.6 Multi-Agent Swarm Topologies, Coordination, and Multi-Agent Economics

When scaling from a single agent to a collaborative collective, coordination must transition from direct peer-to-peer communication to structured environmental and institutional governance.

0.6.1 Stigmergic Communication: Environmental Coordination

Communication cost depends on who publishes, who consumes each update, and what state they need. In 1959, French biologist Pierre-Paul Grassé coined the term **Stigmergy** (from the Greek *stigma*, mark, and *ergon*, work) to explain how social insects construct intricate, cathedral-like termite mounds without central leadership or point-to-point communication.



*Cathedral termite mound (*Nasutitermes triodiae*) in Litchfield National Park, Australia. Social insects construct massive architectural edifices through stigmergic environmental modification without central coordination or peer-to-peer communication.*

In a stigmergic coordination pattern, **the environment is a communication medium**:

1. **Passive Environmental Modification:** Agents coordinate by leaving persistent, machine-readable traces directly in shared host artifacts: file locks, AST reservation tags, pheromone heat maps, and append-only note streams.
2. **Decoupled Sensing:** When Agent A claims an AST symbol or edits a function, it writes a tuple into the shared kernel tuple space. When Agent B inspects the repository, it senses Agent A's trace and autonomously diverts its own planning policy away from the contested file.

If N agents each send one update to all peers, there are $N(N-1)$ deliveries. Publishing each update once to shared storage uses N writes, but if every agent then reads every update, the reads still number $N(N-1)$. Total traffic is linear only under a bounded-read or suitably aggregated subscription policy. Shared traces decouple execution in time; they do not remove the cost of distributing information.

0.6.2 Conway's Law in Multi-Agent Swarms and Role-Based SOPs

In 1968, computer scientist Melvin Conway formulated the foundational law of software architecture:

> "Organizations which design systems are constrained to produce designs which are copies of the communication structures of these organizations."

M. E. Conway. *How do Committees Invent? Datamation*, 14(4):28–31, 1968.

Conway's observation motivates a design hypothesis for agent swarms; it is not a universal mathematical theorem:

- **The coupling hypothesis:** Unstructured all-to-all communication can make responsibility and interfaces harder to track. It does not logically force a monolithic software design. Evaluate the resulting dependency structure and maintenance outcomes rather than inferring them from the communication graph alone.
- **MetaGPT Role-Based Standard Operating Procedures (SOPs):** Hong et al. study a design that encodes software engineering procedures into roles: Product Managers generate PRDs, Architects synthesize system design diagrams, Engineers write modular code, and QA Reviewers run test suites. Structured documents carry intermediate results. This motivates an interface design; it does not prove that role separation prevents every error.

S. Hong, M. Zhuge, J. Chen, X. Zheng, Y. Cheng, C. Zhang, J. Wang, Z. Wang, S. K. S. Yau, Z. Lin, et al. *MetaGPT: Meta Programming for Multi-Agent Collaborative Framework*. In *Proc. International Conference on Learning Representations (ICLR)*, 2024.

0.6.3 Six Sigma Defect Minimization in Agent Swarms

Patel et al. propose task decomposition, redundant micro-agent sampling, and consensus with dynamic sample counts. Their reliability calculation relies on independence of the sampled errors. For odd n, independent votes with common error probability $q < 1/2$ yield the binary-majority error

$$P_{\text{maj}} = \sum_{j=(n+1)/2}^{n} \binom{n}{j} q^j (1-q)^{n-j}. \tag{0.21}$$

For $n = 5$ and $q = 0.05$, this is 0.001158125, about 0.116%. Perfectly correlated errors instead leave error probability q, regardless

K. Patel, S. Surendira, J. George, and S. Kapale. *The Six Sigma Agent: Achieving Enterprise-Grade Reliability in LLM Systems Through Consensus-Driven Decomposed Execution*. arXiv:2601.22290v1, 2026. Primary record.

of n. These calculations do not certify arbitrary semantic clustering of open-ended answers.

For a sequential pipeline, define d_k as the conditional probability that stage k fails given that all preceding stages passed. Then the chain rule gives

$$\Pr(\text{all stages pass}) = \prod_{k=1}^{K}(1 - d_k). \tag{0.22}$$

Using unconditional stage error rates in that product would require an independence assumption. Deterministic checks are useful when an admission predicate is available, but this is a separate design choice: a test pass establishes only the tested condition, and consensus alone does not supply a formal oracle.

0.6.4 Runtime Task Overlap Detection, Solely Responsible Agent Patterns, and Sandboxing

A cooperative claim announces intended work. It prevents a collision only if all conflicting effects respect an enforced admission rule. Different file paths can refer to overlapping state, while disjoint edits in one file need not conflict; the resource model must specify what overlaps.

One design gives a designated owner authority over a scarce resource while other workers submit proposals. Another admits transactions after version validation. Either still needs the effect boundary from the counter example. Sandboxing limits reachable resources; it does not by itself choose the right owner or make a stale update current. Chapters 1–3 develop these obligations separately, so a claim record is never silently promoted into confinement.

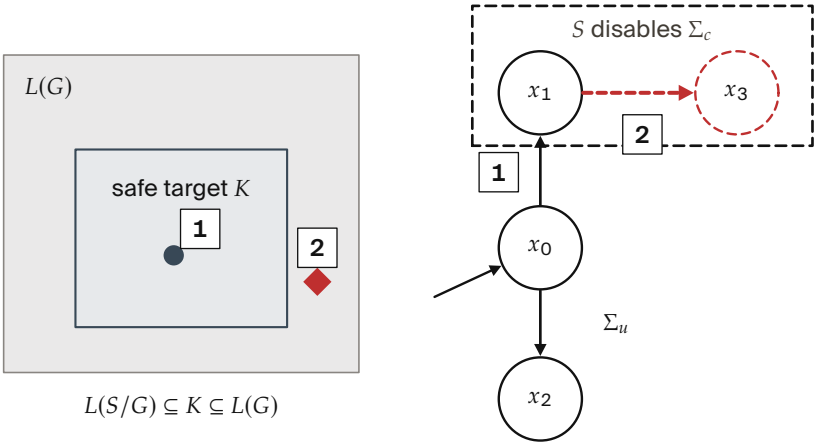


Figure 0.17. Control has a boundary. A supervisor may disable a controllable write, but cannot simply disable a crash. A safe language must accommodate its permitted uncontrollable extensions; otherwise the supervisor must restrict earlier choices or report infeasibility.

	Event	Class	Supervisor
1	file write	Σ_c	Admitted: the trace stays inside K.
2	process spawn	Σ_c	Disabled: would leave K.
–	tokens, signals, exit	Σ_u	Cannot disable this continuation.

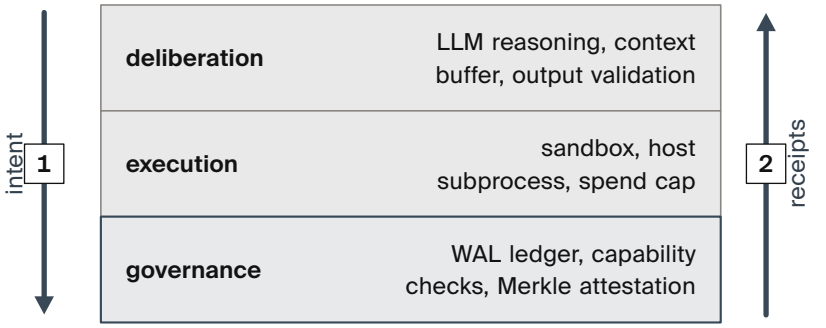
Containment requires implementability: permitted prefixes must admit their uncontrollable continuations. Partial observation adds a condition; an arbitrary safe target need not satisfy $L(S/G) = K$.

0.6.5 Zombie Detection, Heartbeats, and State Salvage

Numbers by Hand 0.6.1 (The expired worker resumes).
A worker holds fencing epoch 8 and pauses. After the declared lease deadline, the authority grants a successor epoch 9. The old worker then resumes and sends a write bearing epoch 8. The actuator must reject it. Reassigning a database row alone would leave both workers able to write.

A heartbeat is evidence of recent communication, not a proof of continued health; a missed heartbeat can indicate delay rather than death. Lease expiry is a policy decision. Fencing makes that decision effective at the resource. Chapter 1 gives the state transitions, and Chapter 2 distinguishes epochs from credential expiry.

Recovery must also preserve ownership of unfinished work. A successor can inspect retained diffs, test records and obligations under a declared recovery policy. It must not treat every untracked



	Flow	Direction	Carries
1	intent	deliberation → governance	Proposed action, admitted before its mediated effect.
2	receipts	governance → deliberation	Observed outcome and retained-evidence references.

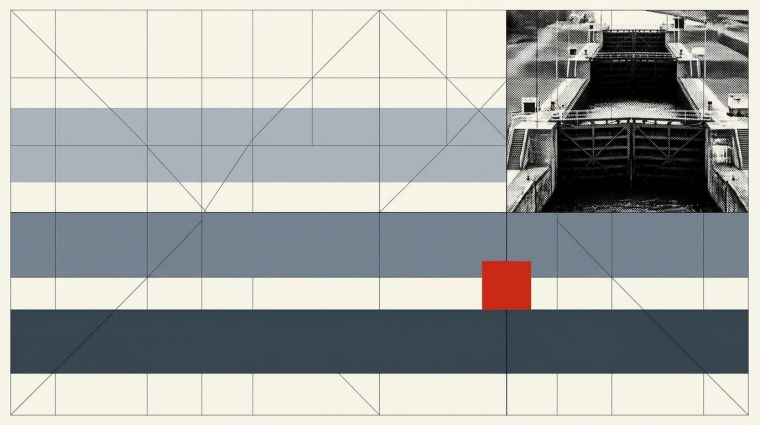
Governance admits or refuses before execution. Confinement can also block effects. A receipt records an observed outcome; it does not establish semantic correctness.

Figure 0.18. Proposal, admission, execution and evidence. The model proposes an action. The host checks and executes admitted effects, then records observations for the next decision. Correctness depends on complete mediation and the stated recovery protocol.

file as disposable or mistake a missing response for an uncommitted operation. This reconnects the timeout example to the work unit with which the chapter began.

0.6.6 Closed-Loop Attestation and Execution Verification

Figure 0.18 illustrates the closed agentic loop, stratified into three architectural tiers: the probabilistic Deliberation Layer, the confined Execution Layer, and the deterministic Governance Layer.



■ **Architectural Plate 0.F · Closed-Loop Attestation & Governance**

"Stratifying probabilistic deliberation from deterministic kernel attestation provides the authoritative bedrock for autonomous software engineering."

A possible harness follows this control sequence. Its stored histories may be unbounded, so the sequence alone does not establish a finite-state model:

1. **Pre-Invocation Assembly:** The harness compiles system instructions, tool schemas, retrieved repository symbols, and recent conversation turns into the token buffer.
2. **Inference:** The prompt is transmitted to the inference engine. Tokens stream autoregressively until a function-call termination token is detected.
3. **Validation and Policy Gate:** The parser extracts tool arguments. The supervisor validates types against JSON-Schema definitions and verifies authority against local capability policies.
4. **Sandboxed Execution:** The command executes inside an isolated runtime container (macOS Seatbelt, Linux Bubblewrap) under strict egress caps.
5. **Kernel Witnessing:** Mutations, locks, and file reservations are submitted to the local kernel, appending a receipt to the audit log.
6. **Observation Injection:** Raw output is sanitized, truncated to preserve token budget, and appended to the context buffer for the next deliberation step.

0.7 The Kernel Gap: Effects and Recovery

Consider an agent with permission to propose any tool call. A useful guarantee must survive both an adverse proposal and a process crash. The following elementary result locates those proof obligations without assuming that stochastic generation always fails.

Theorem 0.7.1 (Mediation and durable recovery boundaries).

Let I be a state invariant, $s_0 \in I$, and $f(s, a)$ the effect of admitted action a.

1. **Complete mediation:** Suppose every state transition is checked at an indivisible admission-and-effect boundary against its current state, and every admitted pair satisfies $f(s, a) \in I$ whenever $s \in I$. Then every execution remains in I, regardless of the proposal policy. Conversely, if a reachable safe state has an unmediated admissible action leading outside I, universal safety over the admissible executions fails.
2. **Conflicting updates:** If two actors may each implement an increment by a separate read and unconditional write, with no validation of the read version, their allowed interleavings include a lost update. Serializing the writes alone does not exclude that trace.
3. **Crash recovery:** Suppose two pre-crash histories require different recovered obligation states, but erasure leaves identical surviving state and observations. No recovery procedure using only that information can recover the correct obligation state with certainty in both cases.

Proof idea. Preserve safety one transition at a time; exhibit a lost-update schedule; then compare two histories that recovery cannot distinguish.

Proof. For (1), induction on the admitted transitions preserves I; the converse follows by appending the specified unsafe transition to its reachable prefix. For (2), both actors read 0 and then each writes

1, whereas two serial increments give 2. For (3), the recovery procedure receives identical input in the two cases, so its output distribution is identical; it cannot concentrate on two different required states. □

Complete mediation may be implemented by a local gate, an authenticated remote permit checked before the effect, a verified controller, or another mechanism satisfying (1). A single-writer kernel is the construction developed here, not a unique mathematical solution. Disjoint or commuting operations need not share a mutex. Retained receipts or other distinguishing state are necessary for the recovery task in (3); refusing all further work can preserve some safety properties but does not recover the lost obligations. These distinctions are the basis for the chapters that follow.

0.8 Chapter Map and Architectural Codas

0.8.1 Structure of the Volume

The remainder of this textbook is organized into eight cumulative chapters across four distinct operational levels:

- **Chapter 1: The Single-Writer Kernel (Ground Truth).** We construct the minimal deterministic local kernel using SQLite WAL mode, establishing serializability, synchronous admission guards, and crash durability across process failures.
- **Chapter 2: The Anchor Protocol (Capability Delegation).** We design cryptographically verifiable capability tokens (Harbor Cards) and prove that delegated authority can only narrow at every hop.
- **Chapter 3: The Sealed Harbor (Confinement & Leakage).** We construct isolated execution harbors that bound information leakage between distrustful principals under strict egress quotas.
- **Chapter 4: The Legible Swarm (Human Supervision).** We derive the information-theoretic cost of supervisory attention, proving how digests must retain evidence paths without drowning the operator in raw telemetry.
- **Chapter 5: From Spawn to Person (Durable Agency).** We define persistent agent identity, append-only outcome ledgers, and reputation mechanisms that survive process restarts.

- **Chapter 6: The Harbor Economy (Markets & Clearing).** We establish bilateral clearing, underwriting reserves, and dynamic pricing for scarce host resources.
- **Chapter 7: The Bonded Commons (Game-Theoretic Governance).** We analyze claim signaling under stated monitoring and payoff assumptions, and separate deterrence from accounting and adjudication.
- **Chapter 8: The Federated Harbor (Cross-Machine Evidence).** We study witness logs and topology-dependent consistency checks while preserving local admission. A vanishing residual does not certify every fact or a globally consistent swarm.

0.9 Review of the key ideas

The opening counter separated a proposed update from an admitted transition. The failed patch separated an unsuccessful attempt from an abandoned obligation. The timeout separated a missing response from a missing effect. These are the same discipline applied to actions, goals and observations: name the states that matter, retain evidence that distinguishes them, and place a check where it can still affect the outcome.

Commitment theory helps describe what persists; message kinds distinguish a request from an acceptance; incentives depend on detection and enforceable consequences. None replaces the others. The memory and attention calculations explain why a prompt cannot carry every record. Chapters 1–3 establish transaction, authority and disclosure boundaries. Chapters 4–8 use those boundaries for supervision, succession, exchange and federation.

0.10 Exercises

Foundations Proposals, persistence and evidence

Solution p. 714

0.1 Check ⋆

Two workers each read counter 4 and propose writing 5. What final value follows if only their writes are serialized? What value would two serial increments give? State the missing check.

0.2 Check ⋆ *Solution p. 714*

A deviation gains 10 only when undetected; detection loses the gain and incurs sanction 100. Find the smallest detection probability making its expected incremental payoff nonpositive. Why is audit selection alone insufficient?

0.3 Trace ⋆⋆ *Solution p. 714*

A test-payment request times out. Give two histories consistent with the same observation. Explain what an operation identifier and a durable receipt can resolve, and what they cannot establish about the quality of the tested code.

0.4 Trace ⋆⋆ *Solution p. 714*

A hidden bit is uniform on $\{0, 1\}$. Both cases emit "ready," but an exit code then equals the bit. List the remaining possible states before and after observing exit code 0. Does hiding the file alone preserve secrecy?

0.5 Open ⋆⋆⋆ *Solution p. 714*

Design a recovery contract for Alice's repair after a worker pauses with an unreported external effect. Specify authoritative state, an ambiguous pair of histories, the distinguishing evidence, a stale-worker refusal rule and one failure your contract cannot repair. Explain how a replacement worker learns what remains owed without repeating a settled effect.

0.11 History and references

The chapter draws its distinctions from several traditions: Bratman and Rao–Georgeff for intentions, actor and CSP models for communication, communicative-act theory for message intent, and supervisory control for what a gate can enforce. Arrow and mechanism design identify limits on collective decisions; Ostrom supplies institutional questions rather than a software correctness theorem. The cited planning and learning systems concern proposal generation

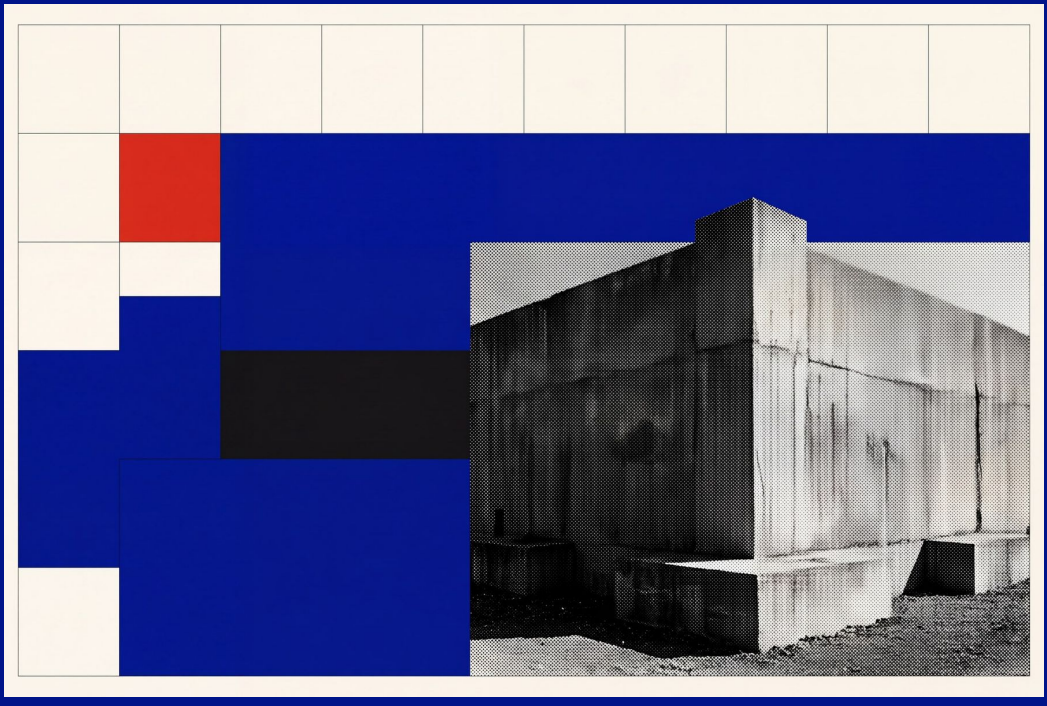
and evaluation. Their benchmarks are not certificates for the host mechanisms developed next.

What *The Single-Writer Kernel* receives The theoretical foundations of intentional agency, protocol mechanisms, and the physical limits of autoregressive inference identify the assumptions needed to mediate conflicting effects and recover obligations after a crash. Part I now builds the physical substrate: the minimal deterministic single-writer SQLite/WAL kernel that serializes local mutation and survives process crashes.

I

Ground Truth

The first three chapters examine a local writer, delegated permissions, and a confined execution environment. They distinguish what the software prevents from what it can only detect, and what remains a design proposal.



1 The Single-Writer Kernel

One writer orders changes to a local database; the chapter specifies which failures that record can survive.

2 The Anchor Protocol

Delegated capability only narrows: every child is a subset of its parent, at every hop, and the verifier checks the whole chain.

3 The Sealed Harbor

A confined work order has explicit release gates and a shared leakage budget, under the model's stated channel assumptions.

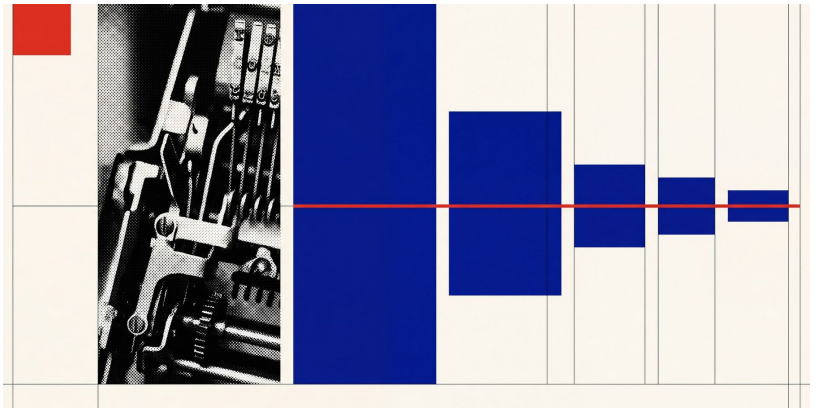
1

The Single-Writer Kernel

The question this chapter answers

Where can a rule be made real?

One writer orders changes to a local database; the chapter specifies which failures that record can survive.



A transaction commits at the instant its commit record appears on disk.

Jim Gray et al., The Recovery Manager of the System R Database Manager, 1981, p. 232

Where can a rule be made real? Six agents may agree about a file and still overwrite one another. A log can report the collision after the fact; only a mediated writer can refuse the second effect before it lands. This chapter identifies that local writer, separates preventable rules from detect-only claims, and states the work-unit invariants that every later institution assumes. R5 and R8 set the boundary.

1.1 Where this chapter sits, and what it promises

Alice's two agents both want port 7421 for a development server. They need one answer about who may use it, and they need that answer to survive an agent exiting. Start with the record they can agree to consult.

Numbers by Hand 1.1.1 (Two requests, one recorded holder).

The claims table starts empty. Agent A requests port 7421 for 60 seconds; the writer records A and confirms success. Agent B then requests the same port before the lease expires. The writer reads the existing row and returns "held by A," leaving one row, not two. If A repeats its request after a lost reply, the same holder is returned; a retry does not create a second ownership record. These are three requests and one holder [internal, hand trace]. The record still cannot stop B from binding the port through an unmediated operating-system call. It settles a question inside the service; confinement is a further requirement.

The *daemon* is the background process answering these requests. A *transaction* groups database changes into an all-or-nothing step; *commit* is the point at which that step becomes part of the database's accepted history. SQLite's *write-ahead log* (WAL) records the changes before they are copied into the main database file. A *lease* is a reservation with an expiry time. We return to these requests when we consider ordering, crash recovery, and stale holders: each adds a condition the simple trace has deliberately left open.

We model an agent-coordination system as a four-layer stack (Figure 1.1), each layer serving a different *whom*.

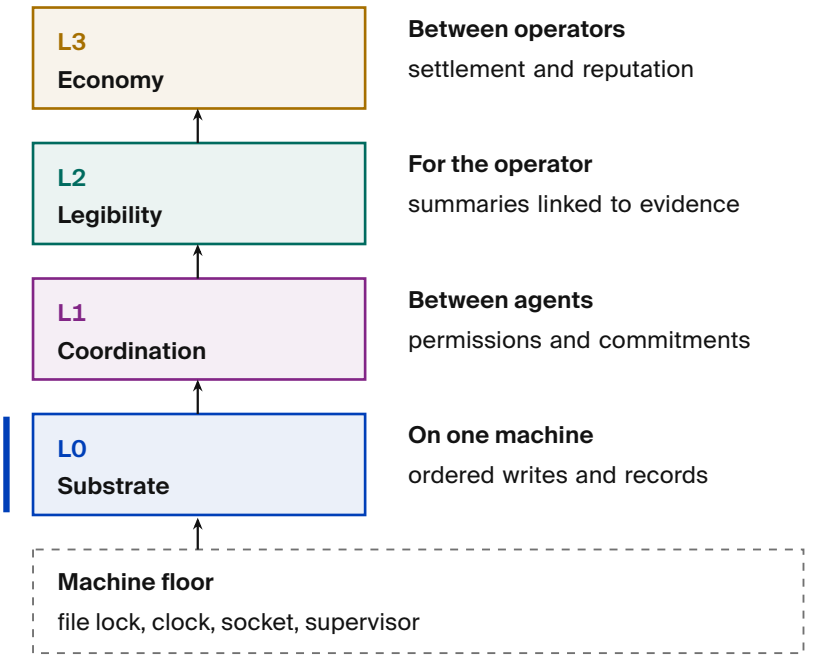


Figure 1.1.
Read upward from L0. Higher layers depend on the records beneath them, but introduce their own assumptions. The blue tick identifies this chapter. The dashed machine floor is a dependency, not a fifth layer.

Substrate (the machine).
A single always-on daemon over one local database: durable storage and serialized mutation. *This chapter's core.*

Coordination (the agents).
The protocol agents speak over the substrate: ownership claims, a message bus, stigmergic markers, obligations, and policy enforcement. This chapter specifies the implemented half of it.

Legibility (the operator).
The human-facing view that renders the swarm intelligible. Treated in an accompanying chapter (*The Legible Swarm*).

Economy (the market between operators).
The cross-machine market in which operators trade work and reputation. Treated in an accompanying chapter (*The Harbor Economy*).

Later chapters depend on this kernel in different ways. The operator's view reads its claims, bus, and policy monitor. Durable agent identity requires its continuity records. The settlement ledger adds tables under the same writer, while cross-machine transfer uses its

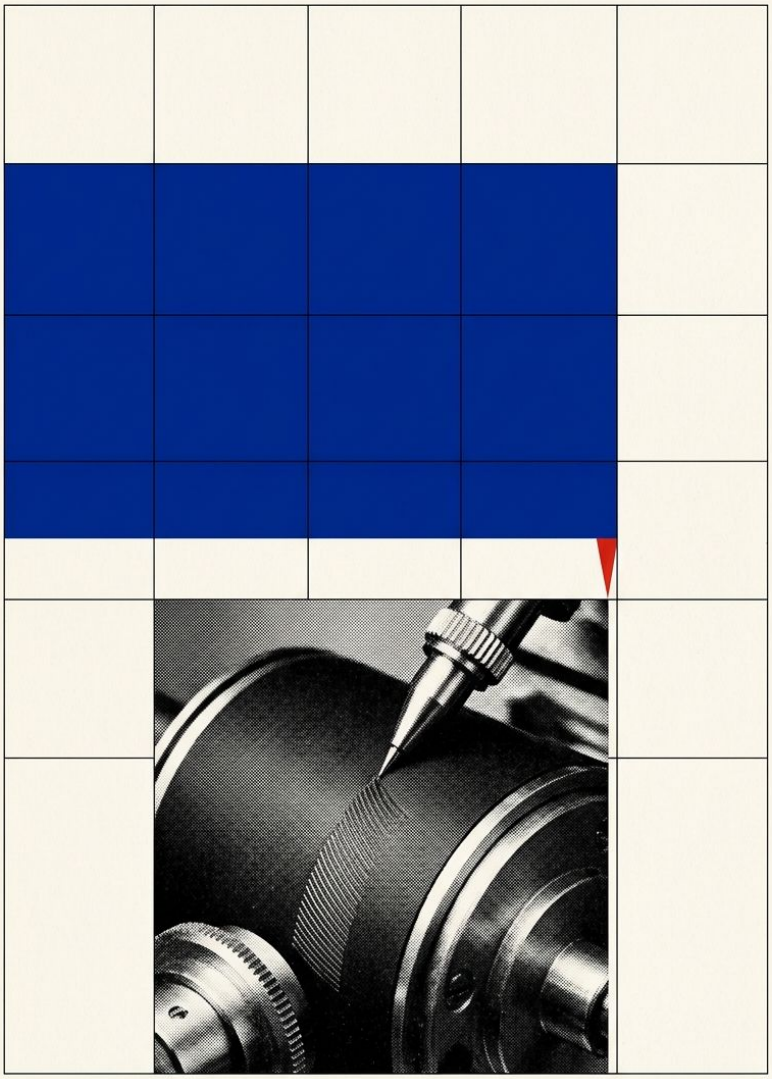
authorization chain. A fault in these records can therefore affect supervision, attribution, or settlement; each guarantee below states the conditions it needs.

1.1.1 The one-sentence thesis

The kernel is a single-writer transactional reference monitor over a local SQLite/WAL database: one process accepts coordination-state changes and records their order. Its ledger is authoritative for those changes, not for every claim about the world outside it.

The two guarantees require different qualifications:

1. **Durability (with an edge):** a write that returned success is durable across the death of any agent — and, we will be careful, across the death of the *daemon process*, though *not* unconditionally across power loss.
2. **Mediation:** a pre-effect guard can refuse a forbidden transition. A post-effect monitor can only report or respond to a transition that already happened. We call the former *regimentation*; a bound on detection or repair needs its own timing and availability assumptions.



Landmark 05 · The Single-Writer Monotonic Horizon

Within its mediated domain, a serializing writer can give admitted transactions a common commit order. Durability depends on the storage and failure model. That local order does not by itself resolve disagreement between independent operators or prevent effects outside the writer.

1.1.2 Why "reference monitor," and why *not* consensus

The right mental model is the **reference monitor** of Anderson and Lampson: a mediator that is *always invoked* (complete mediation), *tamper-resistant*, and *small enough to verify*. The kernel instantiates this not as an abstract security kernel but as a concrete local daemon over one SQLite file. Complete mediation holds *by construction over the kernel's own state*: because every mutation of *coordination state* routes through one writer holding one file lock, the operating system's serialization *is* the mediation primitive — Saltzer and Schroeder's "economy of mechanism" done with one moving part instead of N hand-rolled critical sections. We are deliberate about the scope of the word *complete*: it is complete over writes that *enter* the daemon, not over what a same-user agent can do *beside* it. A classical reference monitor earns "always invoked" from the hardware/operating-system boundary that forces every access through it; this kernel has no such boundary at the substrate layer, so a same-user process can mutate shared state the daemon never sees — edit a file under an advisory claim, run `git` directly, or write the SQLite file itself — without routing through the writer. Mediation is therefore complete over the daemon's *application-program interface* and advisory everywhere else; making it complete over the agent's host capabilities is an out-of-band enforcement problem we scope to the threat model (§1.12) and credit, when it lands, to a separate-user kernel boundary that does not yet exist (§1.12.2).

The swarm *looks* like a distributed-systems problem — many agents, contention, failover — so the trained reflex is Raft, Paxos, CRDTs. The kernel's defining move is to refuse that problem. One writer, one machine, one file: there is nothing to *agree* on, so the consensus impossibility of Fischer, Lynch, and Paterson does not bite. It is sidestepped, not solved. Distribution is a cross-machine concern that belongs to the economy layer; pushing a consensus protocol down into the local substrate would be the wrong layer.

A local writer avoids the agreement protocol required by a replicated system. The conditional theorem in §1.10 states the resulting guarantee: when every read and write is routed through the sole daemon connection and responses follow commit, transactions are

J. P. Anderson. *Computer Security Technology Planning Study*. ESD-TR-73-51, U.S. Air Force, 1972.

B. W. Lampson. "Protection." *ACM SIGOPS OS Review* 8(1):18–24, 1974.

J. H. Saltzer and M. D. Schroeder. "The Protection of Information in Computer Systems." *Proc. IEEE* 63(9):1278–1308, 1975.



Butler Lampson's access matrix: permissions connect subjects to objects.

One writer needs no consensus

M. J. Fischer, N. A. Lynch, and M. S. Paterson. "Impossibility of Distributed Consensus with One Faulty Process." *J. ACM* 32(2):374–382, 1985.

M. Kleppmann. *Designing Data-Intensive Applications*. O'Reilly, 2017.

serializable and claim/lock operations are *linearizable* — the property that lets the coordination layer treat "who holds this" as a single authoritative coordination record. Serializable means the result matches some sequential transaction order. Linearizable adds real-time order: a claim completed before another request began must appear first. Overlapping requests may be ordered either way if their results fit that order; the three-request port example will return in the claim-history exercise.

1.1.3 A research-maturity scale, and why the grades matter

A systems paper that mixes "we proved this" with "we hope to build this" is unfalsifiable. So every consequential claim in this chapter carries an explicit *maturity grade* on a four-point scale, plus two refinements introduced here because a single grade cannot formally carry a claim that spans fault classes or interception points. The grade is a property of the *claim*, not a marketing adjective: it says how far the claim is from a verified, running artifact. The concrete artifacts behind each grade are catalogued in Appendix 1.A; the body argues at the level of mechanisms.

Process and power failures differ

The grades are not decoration. The two most consequential corrections in this chapter — the durability split and the regimentation/detection split — are exactly the places where a single optimistic grade would let the kernel's foundational promise be read as stronger than the artifact delivers. When a system claims "the database survives every agent's death," a reader hears "power-loss durable." It is not, by default.

1.2 The kernel as seven subsystems

The kernel's seven *subsystems* share one database but have distinct contracts (Table 1.2). Separating them makes it possible to trace which guarantees depend on claims, sessions, messages, or policy checks, and which failures cross subsystem boundaries. The **substrate subsystem** underlies the other six; mechanically, they are tables updated through the same SQLite connection. For example, accepting A's claim changes the `claims` table, while recording

Grade	Meaning
implemented	Realized, exercised, and true under the production runtime.
partial	Realized but partial: holds under a narrower condition than the prose might invite, or detects rather than prevents.
specified	A concrete design exists, but no running artifact realizes it.
proposed	Named as a direction; no design yet.
I1a / I1b	Durability *split by fault class*: I1a (process-crash) is **implemented**; I1b (power-loss) is *not guaranteed* under the default WAL configuration. A claim that conflates fault classes cannot carry one grade.
regimented / enforced	Prohibition split by interception point: *regimented* = rejected pre-commit, truly unreachable (a uniqueness constraint / boot-admission gate); *enforced* = post-commit detection and possible remedy (the policy monitor); a bounded response time requires additional observation and scheduling assumptions. "Physically unreachable" is reserved for the regimented set.

Table 1.1. The research-maturity scale used throughout. The last two rows are refinements this chapter introduces (see §1.4 and §1.7).

A's promised release changes `commitments`. They share a transaction mechanism without making ownership and obligation the same fact. We examine storage, resource exclusion, and commitment checking in detail, then use them to explain communication and recovery.

Exercises 1.1–1.3, p. 149.

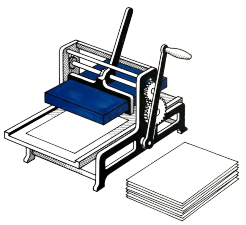
1.3 The substrate subsystem: one writer, one file

The substrate underlies everything else, mechanically nothing more than a table with two promises to keep — durability and serialized mutation — and to keep them formally. We cover the single-writer discipline here and durability in §1.4, because durability is where the most consequential correction lives.

Table 1.2. The seven subsystems, each the contract the daemon holds for the layer above it, with the table that is its home where the chapter names one and its maturity grade. All seven are disciplines over one SQLite/WAL file: one commit history, not seven stores. The grades come from the chapter's own status table (§1.A).

Subsystem	Contract held for the layer above	Maturity
transactional substrate	one writer, one durable file; every other subsystem is a table with discipline over the same commit history	**implemented**
resource / exclusion	at most one compatible holder per conflict domain; leases expire and are swept lazily (home: `claims`)	**implemented**
actor continuity	a stable actor identity across sessions, mechanically present but asserted by the actor, not proven	partial self-asserted
message carriage	an ordered, cursor-addressable, durable envelope delivered at least once (home: `messages`)	**implemented**
obligation & enforcement	no *done* without a receipt the daemon can re-check; the policy monitor detects and cannot always prevent (home: `commitments`)	partial partial
session memory	notes and events survive process death; execution state does not	**implemented** (memory)
self-attestation	the daemon reports its own coverage as enforced, degraded or stubbed, never silently	**implemented**

1.3.1 The single-writer discipline



Like our single-writer daemon, the platen handles jobs one after another.

The kernel's concurrency story (Figure 1.2) is: dozens of command-line invocations and agent connections funnel through one writer — the daemon, the sole holder of a read–write connection to the database file — and SQLite's operating-system file lock plus a bounded busy-wait serialize all mutations. We must be precise about *what* serializes writers, because there are two stories and only one is true at a time.

Definition 1.3.1 (Single-writer discipline)**.**

All state-mutating operations route through a single daemon process holding one SQLite write connection. Concurrency among the many clients (command-line invocations, agents over the message transports, background workers) is resolved by the daemon's request queue; the SQLite file lock is the *backstop* that preserves correctness if the discipline is ever violated by a second writer (for

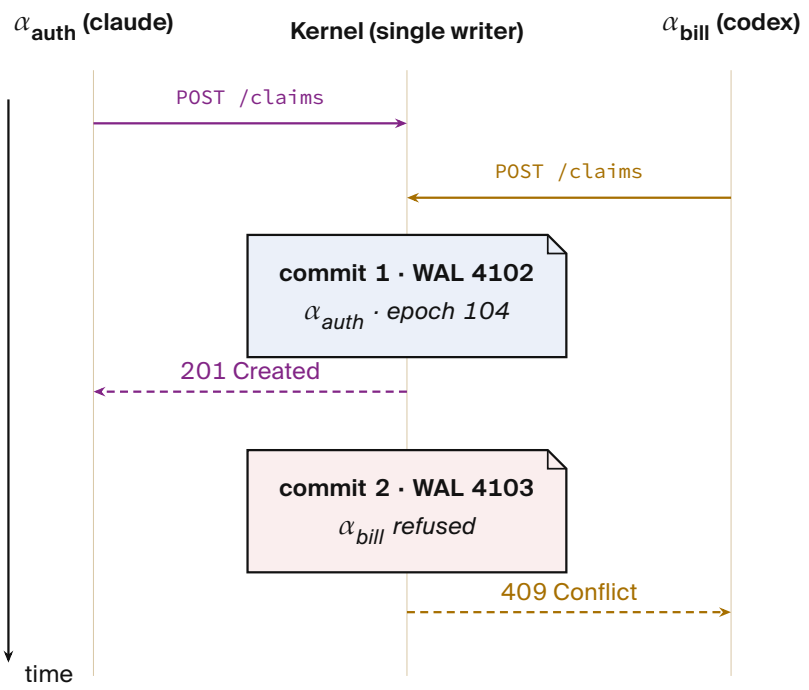
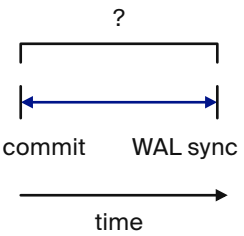


Figure 1.2. Both requests arrive before the first commit. The writer orders their records and replies after each commit. This sequence assumes one writer; it does not establish power-loss durability.

example a stray direct write), not the primary serialization mechanism.

Routing makes the writer unique

"Single-writer" is an architectural *discipline* (everything goes through the daemon), backstopped — not replaced — by SQLite's file lock. This distinction matters: SQLite permits multiple writer connections (it serializes them, it does not reject them), so the property holds because of where writes are routed, not because the database forbids a second writer. The standing risk is therefore a second *copy* of the daemon coming up against the same file — which is why the substrate also runs self-checks that detect a divergent or stale second instance (§1.9).



In NORMAL mode, a power loss can erase recent commits before WAL sync. The bracketed span has no fixed duration.

1.3.2 The configuration *is* the durability claim

The substrate's storage configuration is short and decisive. In WAL mode with the *normal* synchronization level (the WAL is fsync'd at checkpoints rather than on every commit), a bounded auto-checkpoint interval, a bounded busy-wait, foreign-key enforcement on, and a clean-shutdown checkpoint-and-truncate, the kernel inherits exactly the crash semantics that configuration defines. The

claim "a write that returned success survives a crash" reduces *entirely* to what WAL with normal synchronization guarantees — which is the subject of §1.4, and which is not what an unguarded reading assumes.

1.3.3 Schema evolution: an ordered boot ledger, not yet a sequencer (OP-7)

The initial schema-by-union approach (`CREATE TABLE IF NOT EXISTS`) created vulnerabilities around unsequenced renames and multi-column backfills. The kernel's answer so far is a partial one, and the honest way to report it is to name the trade it makes rather than the property it saves. What ships (partial) is a **boot-time migration ledger**: the daemon applies a numbered, totally ordered set of migrations before it serves any request, and then *verifies the schema it is about to serve from* — probing the actual tables and column sentinels rather than trusting that the statements ran without raising — and refuses to serve on a mismatch. That is a real fail-safe default, and it is the reason a half-applied migration cannot quietly become coordination truth. What does *not* yet ship is the sequencer that would make that order authoritative. Subsystem modules still self-initialize their own tables over the shared database handle (Appendix 1.A), so the schema is still, in part, a union. A strict linear ledger managed exclusively by the daemon via SQLite's `pragma user_version`, in which modules *declare* migrations rather than applying them, is specified. OP-7 therefore stands partial: this is the schema-ordering question answered by requiring an explicit dependency order, at the cost of the "any module, any order" flexibility the schema-by-union design offered, and not yet paid for in full. Table 1.3 states the trade in two rows.

Table 1.3. Schema evolution (OP-7): what ships today against the sequencer the design would still need. [internal]

State	What governs migration order
ships (partial)	a numbered, totally ordered boot-time migration ledger, applied before any request is served, with post-apply verification against the actual tables; organ modules still self-initialize their own tables, so the schema is still, in part, a union
designed (specified)	a strict linear ledger managed exclusively by the daemon via `pragma user_version`, in which modules *declare* migrations rather than applying them themselves

1.3.4 Path and ownership invariants

The database resolves to a single canonical path with owner-only file permissions. Signing-key isolation remains partial: operating-system keychain storage is an unfinished requirement, and a same-user attacker remains outside the guarantee (§1.12). A fail-closed guard refuses to open the live registry from a test context — the analogue of a framework's protected-environment check. This is Saltzer and Schroeder's *fail-safe defaults* applied at the one place a test could corrupt production truth.

Saltzer & Schroeder 1975

1.3.5 The second writer: institutional truth above the local harbor

A team runs several harbors and wants one plan across them: which objectives are authorized, who owns what, which transitions were accepted. A shared coordination service could display that plan or become its authoritative writer; those are different roles. Transferring authority from a local harbor to such a service is specified, not an implemented extension of the local ledger. The design rule is short enough to state before the mechanism. *A second writer above the local harbor is either a projection of the local record or the writer of it, never both, and the switch between the two is an epoch.* Here an *epoch* is a monotonically increasing authority generation: records from an older generation cannot authorize a current write. A projection may be copied, searched, and rendered anywhere; it carries no authority, and a client that writes to it has written nothing. The writer is the one process whose accepted transitions define the record, and there is one per record per epoch. Under the proposed signing policy, the harbor signs work intents with its own key; a relay verifies and applies them but cannot author them. A snapshot feed that replaces a remote mirror while leaving the local daemon authoritative is still only a projection. Replicating that feed does not transfer writing authority. This is a *design invariant*: intended to hold, backed by the single-writer implementation for the local case, and not yet exercised across harbors by any code. The proposed writer transfer requires a procedure rather than a setting — six steps, in this order. (1) Freeze: the local harbor stops accepting writes and records the freeze as its last transition. (2) Export:

the ledger and its root hash leave the harbor as one artifact. (3) Verified import: the receiving authority imports the artifact and proves, against the root, that what it holds is what left. (4) Check: projections and permissions are re-derived on the new side and compared with the old. (5) Epoch: the new writer's epoch is signed and published, and every client refuses a write under any older epoch from that moment. (6) Rollback only by another epoch: there is no return to the old writer except by running the same procedure in reverse, so the record never has two writers, even for the duration of a mistake. The reason the switch must be an epoch and not a flag is the same reason revocation must be an epoch in *The Anchor Protocol*: the model-checked attack there restores a backup taken before a revocation and thereby resurrects a dead credential, and it is refuted only when state carries the epoch it was valid under. A restored harbor database that predates the cutover is the same attack against authority, and the same defence closes it. That transfer is a *model-checked property* for credentials and a *design invariant* for writers; nothing in the repository yet checks the writer case.

Where this stops

> The alternative the procedure exists to forbid is dual writing with reconciliation: both authorities accept writes for a while and receipts reconcile them afterwards. That is two truths, the condition every result in this chapter assumes away, and no receipt repairs it after the fact, because the receipt itself would need a writer. A transfer implementation must demonstrate that an interrupted cutover or restored backup cannot reactivate a stale writer, and that the new epoch was authorized by the existing root of authority. Those checks are not provided by the local ledger. Until the transfer is implemented and checked, a remote copy remains a projection over local authorities.

Exercises 1.4–1.6, p. 149.

1.4 Durability, split by fault class

Durability depends on which kind of failure the write must survive. The kernel's foundational promise — "a write that returned success survives" — is true, but only for the fault class a careful reader expects and not for the one the pitch most invites. Table 1.4 draws the split this section defends: the same write, judged against two different fault classes with two different answers.

synchronization level	process crash	OS crash	power loss
`NORMAL` (default)	durable	may lose last commits	may lose last commits
`FULL`	durable	durable	durable

Table 1.4. WAL: `NORMAL` survives process crashes. `FULL` adds per-commit synchronization against OS crashes and power loss. [internal]

1.4.1 Separate process crashes from power loss

Gray and Reuter draw the canonical line between *atomicity/consistency* (a transaction is all-or-nothing and never leaves the store corrupt) and *durability* (the "D" in ACID: a committed transaction survives subsequent failures). Under SQLite's WAL with the *normal* synchronization level, the log is *not* fsync'd on every commit — it is synced at checkpoint. Per SQLite's own documentation, in WAL mode the normal level means "commits might lose durability following a power loss or hard reset," though the database remains uncorrupted. So the honest statement splits in two.

Jim Gray and Andreas Reuter. *Transaction Processing: Concepts and Techniques*. Morgan Kaufmann, 1992. (Atomicity/consistency vs. durability — the ground for I1a/I1b.)

D. Richard Hipp et al. SQLite Write-Ahead Logging. SQLite Documentation, 2010. Online source (`synchronous=NORMAL` "commits might lose durability following a power loss.")

Property 1.4.1 (Durability by fault class)**.**

Let a write return success under the default WAL configuration.

- **I1a (process-crash durability).** If the *daemon process* dies (a forced kill, a panic, an out-of-memory termination), the write survives: the data is in the operating system's page cache, which outlives the process. **implemented**.
- **I1b (power-loss durability).** If the *operating system* crashes or power is lost *before the next checkpoint*, the most recent committed writes may roll back. *not guaranteed* under the normal synchronization level. A stronger acknowledgement must wait for the required synchronization and verify its completion under the storage fault model.

Worked dramatization: `claude:editor` crashes mid-write. α_{editor} (`claude:editor`) is an autonomous agent editing `src/auth.ts`. At t_0 it commits a claim on port 7421 and a note recording its edit (`POST /notes {"text":"Refactoring JWT verification"}`); the daemon returns success. At $t_0 + 5$ ms its host operating system kills the daemon process outright — a forced termination, no clean shutdown. *What survives?* Both writes. The committed pages live

in the operating system's page cache, which the daemon process does not own and does not take with it when it dies; when the supervisor restarts the daemon, SQLite recovers the committed log state, and `claude:editor`'s claim and note are intact. This is I1a, and it is the fault class the design actually targets: agents die constantly, and the record must outlive them. Now change one detail. At $t_0 + 5$ ms, instead of the daemon dying, *the power fails*. The committed pages were in the page cache but had not yet reached stable storage, because under the normal synchronization level the log is fsync'd only at the next checkpoint. On reboot, the database is uncorrupted — but `claude:editor`'s last claim and note may simply be gone, rolled back as if never written. This is I1b, and it is *not guaranteed*. The two faults differ only in whether the page cache survived; conflating them is the single most common over-claim about WAL-backed storage.

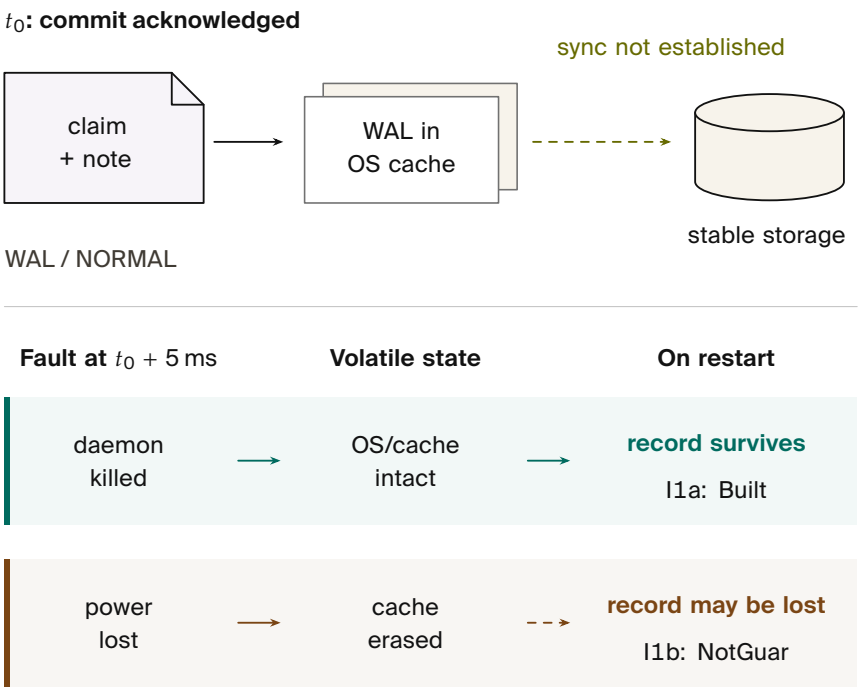


Figure 1.3. With OS and database/WAL state intact, restart recovers a process-crash commit; power loss can erase an unsynchronized commit. The five-millisecond interval is illustrative.

Power loss can erase commits

The seductive misconception is "the normal sync level is safe in WAL mode because WAL already guarantees crash safety." This conflates crash-*consistency* (the documented crash-consistency property) with *durability* (which NORMAL does not guarantee under

power loss). Both depend on SQLite's filesystem and storage assumptions. A successful process-crash recovery does not establish power-loss durability.

1.4.2 Why the trade is defensible — and where it stops being so

The acceptable fault class is a policy choice. Ordinary claims may tolerate loss after a host failure; financial settlement may not. The loss interval under `NORMAL` is not a fixed number of milliseconds.

OP-10 proposes a typed durability selector. A money-bearing transaction uses a connection configured with `synchronous=FULL` before the transaction begins, and success is returned only after commit succeeds. In WAL mode, this requests per-commit synchronization, subject to the storage stack honoring it. Ordinary coordination writes may retain `NORMAL`.

`wal_checkpoint(FULL)` names a checkpoint mode, not the connection's synchronization setting. It is not a substitute for a typed commit contract; checkpoint completion, errors, and concurrent readers must be handled explicitly. The reference implementation has no per-write durability selector today. OP-10 remains *open* pending an interface that requires each path to name its fault class, configuration checks, and host-failure tests. Table 1.5 states this proposed interface rather than deployed behavior.

write-path kind	sync level it would opt into
claim, marker, note (ordinary)	`NORMAL` (the fast default; unchanged)
settlement-ledger entry (money-bearing)	`synchronous=FULL` on the transaction's connection; acknowledge after successful commit

Table 1.5. Typed durability selection (OP-10, specified). Connection settings are established before opening the transaction. No per-write selector is deployed. [internal]

1.4.3 A recovery story, not just a durability story

Durability is about what survives; recovery is about what happens next. On daemon restart with a WAL, SQLite recovers the valid committed state (I1a's mechanism). But the kernel-level questions are larger: who validates the audit chain on boot, and what becomes of

a half-applied cross-subsystem operation (§1.11) after a crash? The hook exists — the boot-admission gate that refuses to serve when a critical self-check fails (§1.9) — but boot-time WAL recovery plus integrity check plus chain verification should be a single named invariant, and today it is closer to partial than **implemented**.

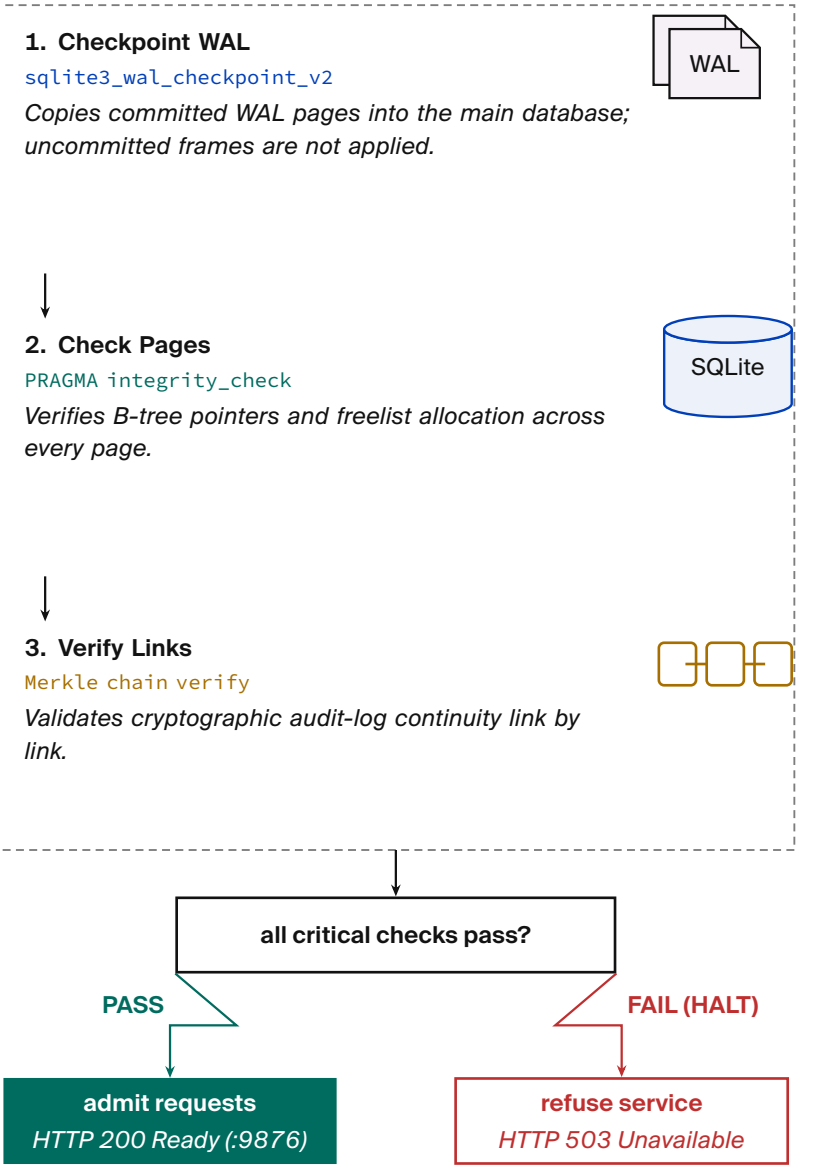


Figure 1.4. Checkpointing, page checks, and audit-link checks precede admission. A failed critical check blocks service. The dashed group marks separate mechanisms, not a completed shared recovery invariant.

Figure 1.4 draws the three mechanisms in the order they run and the admission decision that follows. The dashed group does not imply a shared recovery invariant; that remains to be established.

Exercises 1.7–1.9, p. 150.

1.5 The resource subsystem: claims, locks, and fair exclusion

Network ports are the namesake and the original sin: two agents that each start a development server on the same port collide, and one silently fails. The resource subsystem generalizes that problem into a single exclusion primitive — over network ports, over named mutex locks, and over edit-surface claims on regions of source files — to which every higher layer's notion of ownership reduces.

1.5.1 Claim granularity is richer than "claims"

It is tempting to collapse all three of these into one word. The kernel instead distinguishes **whole-file**, **line-range**, and **symbol-path** claims, each keyed by the file path together with, respectively, nothing, a line interval, or a syntactic symbol identifier. This is the substrate for a "reserve regions, not whole files" coordination policy and for semantic-conflict prediction over a syntax-aware symbol index — but at the substrate layer it is simply exclusion at three granularities, listed in Table 1.6.

granularity	key components	example
whole-file	file path alone	`claim(lib/webhooks.ts)`
line-range	file path + a line interval	`claim(lib/webhooks.ts, 40--72)`
symbol-path	file path + a syntactic symbol identifier	`claim(lib/webhooks.ts# handleEvent)`

Table 1.6. The three claim granularities: what varies is the key, not the exclusion primitive underneath it. [internal]

1.5.2 The claim lifecycle as a finite-state machine

Figure 1.5 states, as a state machine, the four properties on which the coordination layer builds typed ownership. The acquire path is small enough to state as an algorithm (Listing 1.1); we then give the central property as a theorem. The listing shows the exact-key claim protocol. Its uniqueness constraint does not by itself check overlapping line ranges, and it does not show a fencing epoch. The next result therefore states the stronger protocol's assumptions explicitly; neither extra guarantee follows from this listing alone.

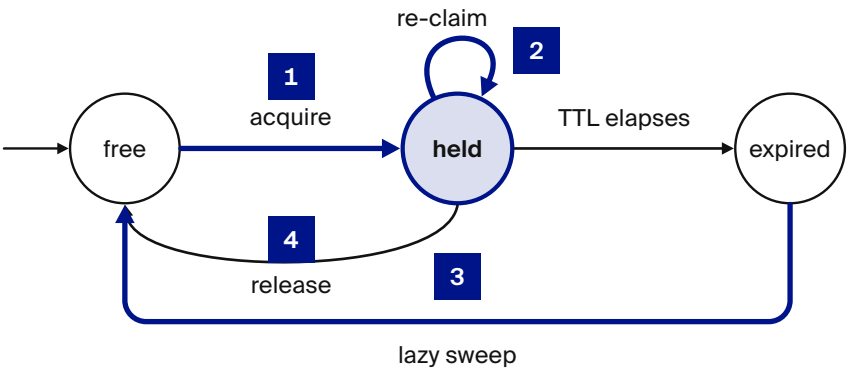


Figure 1.5. The guarded claim lifecycle as a finite-state machine. Acquire is a single atomic insert against a SQLite uniqueness constraint; re-claim and release are idempotent; expiry moves a claim to *expired*, from which the next read lazily sweeps it back to *free*. There is no *queued* state: losing callers receive `409 Conflict` and must manage their own backoff. [internal, alg:acquire]

No Queued State: A losing acquire is denied immediately (`409 Conflict`); there is no in-memory buffer, wait queue, or fair scheduling. Losers must explicitly retry or back off.

#	API Trigger	Database Operation	Invariant & Guarantee
1	`POST /claims {"resource": "lib/auth.ts"}`	`INSERT INTO claims ...` On conflict: `SELECT holder`	Single atomic decision; winner gets `201 Created`; loser receives `409 Conflict` with current holder.
2	`POST /claims/renew {"token":"tk_94a2"}`	`UPDATE claims SET expires_at WHERE holder = requester`	Fully idempotent; extends TTL without creating duplicate records or advancing the fencing epoch.
3	Next read / acquire `GET /claims` or `POST`	`DELETE FROM claims WHERE expires_at < now()`	Zero background threads; expired rows are swept lazily on demand before allocating new leases.
4	`POST /claims/release {"token":"tk_94a2"}`	`DELETE FROM claims WHERE holder = requester`	Immediate deterministic cleanup; excises the lock record upon task completion.

The second half of the theorem below is the **fencing token**, and it should be named before it is proved, because it is the part of the design that was not invented here. Kleppmann's rule is the whole of it — "a number that increases every time a client acquires the lock," handed to the resource along with every write, so that the resource itself and not the lock service can refuse a holder who paused past its lease and woke up still believing it held it — and Chubby shipped it first, as the *sequencer* a lock holder passes downstream so a storage server can tell a current handle from a stale predecessor's. This chapter calls the number an *epoch* because it is kept per resource and survives reallocation, but the protocol shape is the same one:

Kleppmann 2016. *How to do distributed locking*.

Burrows 2006. *The Chubby Lock Service for Loosely-Coupled Distributed Systems*.

```
function acquire(key, requester, ttl):
  // single atomic statement; the uniqueness constraint is the
  ↪ mutex
  try:
    INSERT row (key, holder=requester, expires_at = now()+ttl)
    return GRANTED(holder=requester)          // we won
  catch UNIQUE_CONSTRAINT_VIOLATION:
    existing = SELECT row WHERE row.key = key
    if existing.holder == requester:
      UPDATE row SET expires_at = now()+ttl  // idempotent re-
    ↪ claim
      return GRANTED(holder=requester)
    if existing.expires_at < now():           // stale holder
      DELETE row WHERE key = key AND expires_at < now()
      return acquire(key, requester, ttl)     // one bounded
    ↪ retry
    return DENIED(holder=existing.holder)     // loser learns
    ↪ the winner
```

Listing 1.1. Claim acquisition. The whole acquire is a single insert against a uniqueness constraint; the loser is handed the current holder, not an error to retry blindly.

acquire, receive a monotone number, present it with every effect, get refused when it is stale.

Theorem 1.5.1 (Atomic Mutual Exclusion and Fenced Leases)**.**

Assume conflicting acquisitions are checked and inserted in one serialized transaction: by uniqueness for exact keys, or by an overlap predicate for line intervals and path hierarchies. Assume each new allocation increments a persistent per-resource epoch, and every downstream effect checks both the active lease and its epoch. Then at most one holder has an active lease in each conflict domain, and a stale holder cannot pass that effect gate. In particular, the gate rejects an older epoch

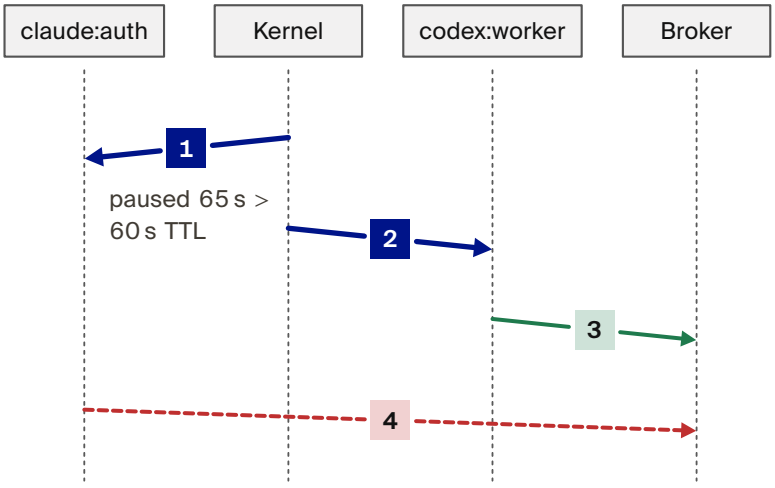
$$e' < e_{current}.$$

Proof sketch.

Within an immediate SQLite transaction (`BEGIN IMMEDIATE`), the daemon evaluates the conflict predicate against all currently unexpired leases. For exact keys, this is enforced by primary-key uniqueness; for intervals and hierarchies, by an indexed range check within the same transaction. Because all writes funnel through the single connection (Def. 1.3.1), predicate evaluation and insert are atomic.

Upon grant or reallocation, the daemon increments the resource's fencing epoch e. Downstream effect brokers validate

that e matches the active lease at execution time, preventing stale or paused holders from producing side effects after lease expiration. □



#	message	payload
1	grant	epoch 104, TTL 60 s
2	reassign	TTL expired; epoch 104 → 105
3	accepted	epoch 105 write; `200 OK`
4	rejected	stale epoch 104; `412 Precondition Failed`

Figure 1.6. The epoch, not the lock, is the guard: the broker accepts the epoch-105 write and rejects the stale epoch-104 write. [internal, Theorem 1.5.1]

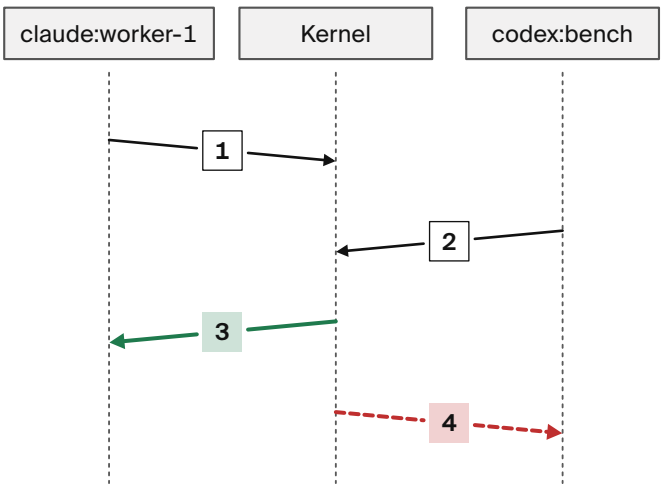
Figure 1.6 isolates that guard as a four-step sequence: the epoch, not the lock, is what a paused holder's late effect cannot get past.



Like an English railway token staff, possession of the token is the sole physical authority to occupy the single track, preventing collision without distributed consensus.

What differs from Chubby is the setting, and it is worth being exact about how much smaller our problem is. Chubby's sequencer and Raft's leader term fence across machines that cannot otherwise agree on who is alive; the danger they answer is a paused JVM or a partitioned node still acting on a lease the rest of the world has moved past. This rail fences one process's own asynchronous handlers against each other, so the race the epoch defends against is logical rather than physical, and the whole apparatus fits inside one SQLite transaction. That is a good reason not to need Raft (the related-work section makes the argument by way of Fischer, Lynch and Paterson); it is not a reason to let the token pass as new.

Worked dramatization: `claude:worker-1` and `codex:bench` race for one port. Two developer agents, `claude:worker-1` (feature development) and `codex:bench` (integration testing), started a millisecond apart, both attempt to bind a development server to port 7421. Each issues `acquire(7421)` (Algorithm 1.1). Both requests reach the single daemon, which serializes them: one insert lands first. Say `claude:worker-1`'s does. Its insert succeeds and it is granted the port (lease token `tk_94a2`, epoch $e = 1$). `codex:bench`'s insert, arriving an instant later ($t = +1$ ms), hits the database uniqueness constraint on key `port:7421` and is rejected *atomically* — there is no moment at which both rows exist. Crucially, `codex:bench` does not get a bare error. The catch path reads back the row and hands `codex:bench` the *identity and lease metadata of the holder*: "`port:7421` is held by `claude:worker-1`, epoch 1, TTL remaining 59s." `codex:bench` can now pick fallback port 7422, wait, or message `claude:worker-1`. The service cannot grant both agents the same fresh key. Preventing a direct, unmediated port bind is a separate operating-system obligation. The daemon's request queue serializes these operations; primary-key uniqueness rejects the second row, and the file lock is the database's backstop. One writer, one constraint, one holder.



#	message	payload
1	acquire	pid 9421, $t = 0.000$ s
2	acquire	pid 9428, $t = +0.001$ s
3	granted	`201 Created`; lease `tk_94a2` (60 s)
4	conflict	`409`; holder `claude:worker-1`; fallback 7422

Figure 1.7. A primary-key uniqueness constraint serializes the race: the kernel grants the first insert and returns the active holder atomically to the loser. [internal, alg:acquire]

Figure 1.7 draws the two lanes converging on that one commit point and the two outcomes.

Reclamation needs its own atomic protocol

The exact-key listing shows how uniqueness rejects a second active row. It does not implement the stronger fenced-lease theorem by itself. Reclaiming an expired row, allocating its successor, and advancing the persistent epoch must form the serialized transition assumed there; every downstream effect must check the resulting lease and epoch. Two separate delete/insert statements can leave a gap on failure even when one connection prevents another handler from interleaving. An explicit transaction addresses atomicity; preventing stale external effects still requires the downstream gate. Single ownership, atomic reclamation, and fencing are three obligations, not synonyms.

What the discipline looks like from two terminals: one agent holds a lock on a file, a second asks for the same name and is told who holds it and for how long, and only after the first releases does the second acquire. Nothing here is advisory — the daemon is the single writer for the `locks` row, so the second request cannot race the first.

Listing 1.2.
two agents, one lock, one holder at a time

```
>_
$ export PORT_DADDY_AGENT_ID=alice
$ pd lock lib.webhooks.ts --ttl 600000
SUCCESS: Acquired lock: lib.webhooks.ts
  TTL: 600s

$ export PORT_DADDY_AGENT_ID=bob
$ pd lock lib.webhooks.ts --wait --timeout 2000
Lock 'lib.webhooks.ts' is held by <actor-id>
  Held since: <timestamp>
  Expires in: <n>s

$ pd locks

NAME                          OWNER
    ↪   EXPIRES
------------------------------------------------------
    ↪ ----------------
lib.webhooks.ts               <actor-id> <timestamp>

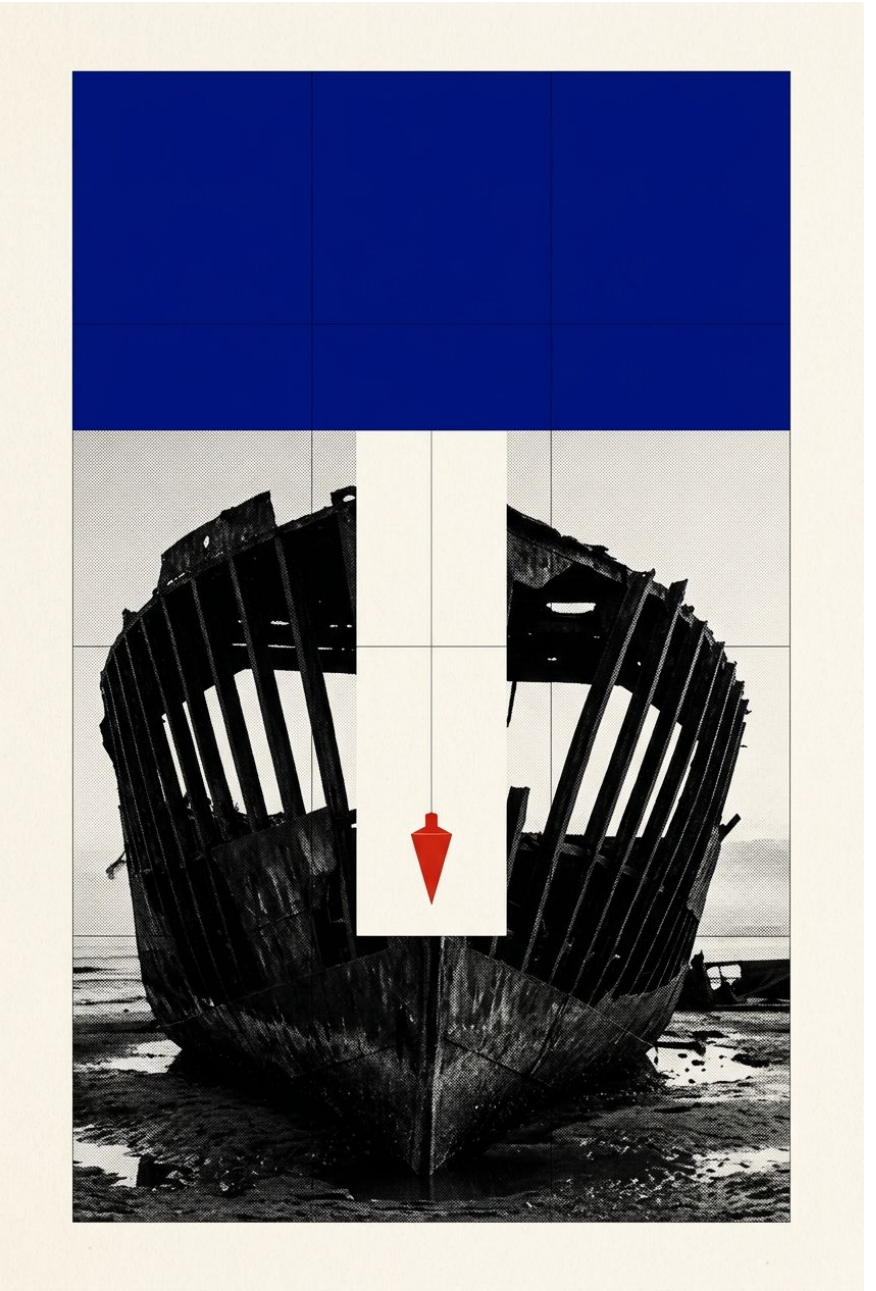
Total: 1 lock(s)
```

```
$ export PORT_DADDY_AGENT_ID=alice
$ pd unlock lib.webhooks.ts
SUCCESS: Released lock: lib.webhooks.ts

$ export PORT_DADDY_AGENT_ID=bob
$ pd lock lib.webhooks.ts
SUCCESS: Acquired lock: lib.webhooks.ts
  TTL: 300s
```

1.5.3 Fairness without a scheduler: the Stigmergic Ticket-Lock (OP-1)

The conspicuous gap in the claim lifecycle (Figure 1.5) is the absence of a queued state. Locks have a time-to-live, but acquisition is racy first-come, so a fast agent churning a hot file can starve a slow agent indefinitely.



Landmark 12 · The Zombie Reclaim & Salvage Graveyard

A missing heartbeat is evidence of silence, not proof of death. Reassignment requires an expiry rule, retained work state and fencing that rejects a stale holder's writes to the reassigned resource. Recovery must also account for effects whose completion was never recorded.

The design that would close OP-1 without dragging a background scheduler into the kernel is a **Stigmergic Ticket-Lock**. It is specified — specified here in enough detail to build and to attack, but not realized: no `claim_tickets` table exists in the reference implementation, and the lifecycle above is still the one the kernel runs. It would introduce a `claim_tickets` table:



Like a ticket dispenser, the proposed lock would assign turns instead of making contenders race again.

```
CREATE TABLE claim_tickets (
  resource_id TEXT NOT NULL,
  agent_id TEXT NOT NULL,
  seq_num INTEGER PRIMARY KEY AUTOINCREMENT
);
```

When Agent B's claim acquisition failed against the primary-key `UNIQUE` constraint, the error handler would insert an entry into `claim_tickets`. When Agent A then called `release('auth.ts')`, the daemon would execute a single atomic transaction:

1. Delete Agent A's active claim row: `DELETE FROM claims WHERE resource = :r;`
2. Read the lowest ticket:

   ```
   SELECT agent_id, seq_num FROM claim_tickets
   WHERE resource_id = :r
   ORDER BY seq_num ASC LIMIT 1;
   ```

3. Atomically re-grant the claim: `INSERT INTO claims (resource, holder, ttl) VALUES (:r, :next_agent, :ttl);`
4. Remove the claimed ticket: `DELETE FROM claim_tickets WHERE seq_num = :next_seq;`

Because the state transition would be driven entirely *reactively* by the releasing agent inside one transaction, FIFO ordering among waiters follows from the ticket table's own monotonic sequence, with no background timer or polling thread in the kernel — which is the property that makes the design worth stating.

Its honest edge is the path it does *not* cover. A holder that never calls `release` — it crashed, or its lease simply expired — returns the resource through the lazy TTL sweep (I4), and the sweep consults no ticket queue. So bounded wait would hold for cooperative release and fail on exactly the failure path the kernel was built to expect. OP-1 stays *open* until the sweep and the queue are the same code path, and until the ticket table exists at all.

Figure 1.8 draws that split as a delta on the shipped machine: one exit closed, one still open.

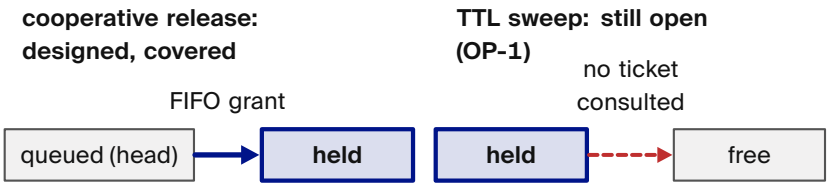


Figure 1.8. The proposed ticket queue covers cooperative release, but expiry still bypasses it. Bounded waiting remains unproved until both paths consult the queue (§1.5.3). [internal]

Exercises 1.10–1.12, p. 150.

1.6 The communication subsystem: the bus, stigmergy, and two delegation chains

Table 1.7. The communication subsystem: one durable carrier and two provenance traces that must never be merged. The envelope fields are what the bus stores; the two traces answer two different questions and are checked by two different mechanisms.

Piece	What it is	Status
the bus envelope	version tag, time-to-live, message kind, typed act with its conversation id, actor, reason and event reference; ordered by a durable history cursor	**implemented**
authorization chain	the cryptographic object: who authorized whom, hop-bound, attenuation-monotonic, anti-replay	**implemented** (ProVerif)
coordination lineage	the coordination object: what work is already in flight, so loops and upward blocks can be detected	specified

The communication subsystem (Table 1.7) is where the substrate stops being a lockbox and becomes a medium for conversation. The carrier is **implemented**; the *semantics* that ride on top of it (typed speech acts, protocol state machines) belong to the coordination layer. Three pieces matter here.

1.6.1 The bus: a durable carrier envelope

The bus is a durable, ordered, *at-least-once* publish/subscribe channel over a messages table, with a versioned envelope (a version tag, a message kind, a body, and an in-reply-to pointer), a history cursor for replay, and a per-message time-to-live. The coordination layer's typed speech act — carrying a performative, a conversation identifier, a coordination lineage, and a reply deadline — is layered *inside* this envelope. The kernel ships the carrier; the coordination layer ships what it carries.

Redelivery needs deduplication

A *healthy* at-least-once bus can redeliver a request. Reporting each repeat as a fault floods the operator with false alarms. An operation key lets a consumer look up a stored receipt; safe retries also require that receipt to remain bound to the effect. The `TubeEnvelope` carries conversation metadata, not an operation key or an effect receipt. Message identifiers and replay guards do not by themselves make an external tool's effects idempotent.

In the local experiment in Figure 1.9, a sender journals one ten-unit synthetic request to a separate SQLite receiver. A child process stops at one of three cutpoints, alongside a no-crash control. A fresh worker attempts one recovery using only its journal and responses to requests—not the receiver's database. The two crash histories have identical local records. Retrying blindly duplicates the effect in one; holding avoids duplicates but leaves both outcomes unresolved. The receiver's retained binding permits useful recovery in these cases. This tests process crashes with serial requests, not arbitrary tools, key expiry or power loss.

Before recovery

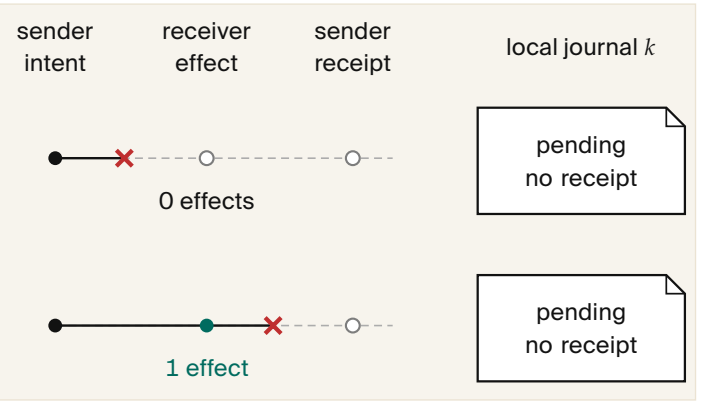


Figure 1.9. A missing receipt does not reveal whether the effect happened. Twelve local SQLite cases, not a Port Daddy test. Receiver deduplication here atomically retains the key, payload, effect and receipt; a sender-side key alone does not supply that contract. [Local fixture, `receipt_gap_experiment.py`.]

After one recovery: effects / sender state

crash point	sender retry	sender hold	receiver dedup + retry
No crash	1 / done	1 / done	1 / done
Before effect	1 / done	0 / pending	1 / done
Effect, not receipt	2 / done	1 / pending	1 / done
After receipt	1 / done	1 / done	1 / done

1.6.2 Stigmergic markers: decay as a provided guarantee

The kernel also carries *stigmergic markers* — numeric coordination signals deposited in shared state, after the manner of Grassé's stigmergy in social insects and the generative-communication tuple spaces of Gelernter's Linda, and used in ant-colony optimization. Each marker carries per-kind exponential decay $w \cdot r^{\Delta t}$, computed both on a background evaporation tick *and* at read time (so a reader sees the correctly decayed value without waiting for the tick), and pruned below a small threshold. The decay is the structural defense against "blackboard rot": stale coordination signals evaporate on

Pierre-Paul Grassé. La reconstruction du nid et les coordinations interindividuelles...: la théorie de la stigmergie. *Insectes Sociaux*, 6(1):41–80, 1959.

David Gelernter. Generative Communication in Linda. *ACM Transactions on Programming Languages and Systems*, 7(1):80–112, 1985.

Marco Dorigo and Gianni Di Caro. The Ant Colony Optimization Meta-Heuristic. In *New Ideas in Optimization*, McGraw-Hill, 1999.

their own. The calibration of the decay rate is genuinely open (OP-8): too slow and signals rot, too fast and they evaporate before they coordinate.

Numbers by Hand 1.6.1 (The Decay Arithmetic a Calibration Will Use).

Take $w_0 = 1$ and $r = 0.9$ per tick, with a prune threshold of 0.05.

Then

$$w(5) = 0.9^5 = 0.590,$$
$$w(10) = 0.349,$$
$$w(20) = 0.122,$$

and the marker is pruned at the first tick where

$$0.9^t < 0.05 :$$

$0.9^{28} = 0.052$ survives and $0.9^{29} = 0.047$ does not, so the marker lives 29 ticks [internal].

Moving the rate half the distance to one ($r = 0.95$) more than doubles that life, to 59 ticks, because

$$\ln 0.05 / \ln 0.95 = 58.4.$$

The calibration question of OP-8 is which of these lifetimes matches the time a signal stays true; the arithmetic is the same at every answer.

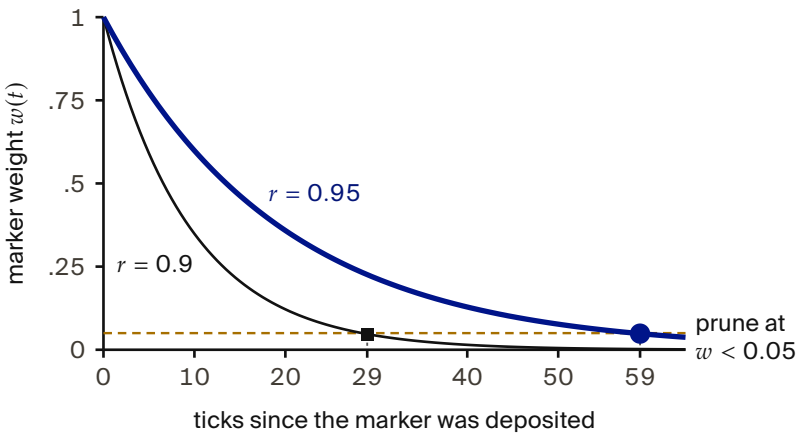


Figure 1.10. With a prune threshold of 0.05, raising decay retention from 0.9 to 0.95 delays pruning from tick 29 to tick 59. [internal; analytic]

1.6.3 Two delegation chains, one dangerous name

A dangerous naming collision lives in this subsystem. The phrase "delegation chain" names *two different objects*, which we keep rigorously distinct:

- the **authorization chain** — the *cryptographic* object (**implemented**, mechanically verified in a protocol-verification tool): a hop-bound, attenuation-monotonic, anti-replay and anti-splice chain of digital signatures that proves *who authorized whom*. This is what cross-machine capability transfer in the economy layer rides on.
- the **coordination lineage** — the *coordination* object (specified): the task-lineage over which loop detection and upward-block run, preventing two agents from delegating a task back and forth forever.

These are distinct objects, and the relationship is that the coordination lineage is carried *inside* the authorization chain — loop detection runs over a lineage whose authenticity the cryptographic chain guarantees. We never again write bare "delegation chain."

Authentication does not settle task equivalence

A signed task identifier can prove which identifier a delegator supplied. It cannot prove that two differently worded tasks mean the same thing. Canonically encoded hashes or embedding outputs can be signed; the open obligation is to specify and validate the equivalence rule and its model version. Signature validity and semantic loop detection are separate checks.

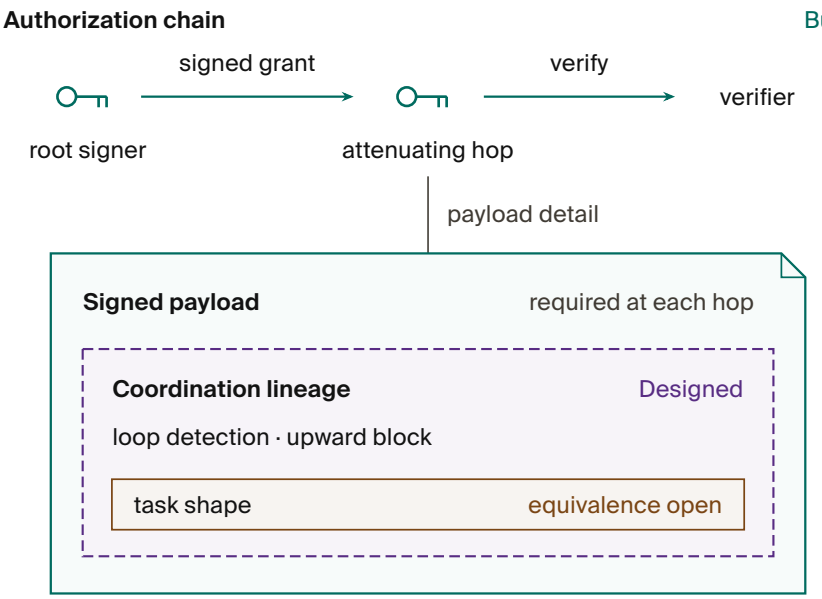
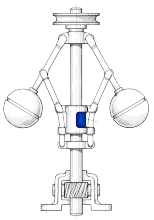


Figure 1.11. The designed coordination lineage must be carried inside each signed authorization hop. Authenticating its bytes does not establish semantic task equivalence.

1.6.4 A second transport

The kernel ships *two* transports, not one: a request/response protocol over a Unix domain socket *and* a binary framed transport over a Unix domain socket, the latter with peer-credential authentication and backpressure/draining for high-frequency, tight-loop coordination. We note in §1.12 that the peer-credential check is a software handshake — the connecting process *states* an agent identity and the daemon checks it against the registration table, backed by owner-only (`0600`) permissions on the socket file — and *not* the kernel-enforced socket-level credential check (`SO_PEERCRED`) it resembles and is named after. That is a real trust-boundary caveat, it holds for both transports, and it is unchanged by the hardening design of §1.12.4.



Like a steam engine centrifugal governor whose collar chokes the inlet when speed threatens stability, rate-limiting enforces backpressure before the transport faults.

Exercises 1.13–1.15, p. 151.

1.7 The obligation & enforcement subsystem: the sovereign's two levers

This subsystem is the deontic core, and it is genuinely two distinct mechanisms — in the vocabulary of deontic logic for computer systems (Jones and Sergot): *prohibition* (a policy monitor) and *obligation* (commitments). Getting the distinction right — and being honest about how strong each lever actually is — is the security core of the paper.

Jones & Sergot. *On the Characterisation of Law and Computer Systems: The Normative Systems Perspective. Deontic Logic in Computer Science*, Wiley, 1993.

Table 1.8 lays out the three modalities and the mechanism behind each.

1.7.1 The deontic split

Definition 1.7.1 (The kernel's deontic split)**.**

The kernel realizes three deontic modalities with three mechanisms:

prohibition is regimented (rejected pre-commit) or enforced (detected post-commit); *obligation* is enforced by oracle-bound commitments; *permission* is a recorded capability. The coordination layer's deontic binding is a direct lift of this split.

We keep **capability** (*ability* — what the agent's sandbox makes possible) distinct from **permission** (*normative status* — what the deontic layer allows), so that "able but forbidden" stays expressible. The canonical example is a dangerous override flag that a tool exposes for emergencies: it *exists* (an agent is *capable* of invoking it) but *must not be used* (it is *forbidden*). Collapsing capability into permission makes that state inexpressible — which is exactly the bug a discipline against silently bypassing guardrails cannot afford.

Figure 1.12 puts both axes on one grid; the override flag names the one cell that must stay expressible.

1.7.2 The policy monitor is a detector, not a regimenter

Figure 1.13 draws the correction this section makes precise. Here is the correction that the rest of this stack must adopt. The most seductive claim one can make about a coordination policy monitor —

Modality	Trigger	Mechanism	Holds / fails when	Example
prohibition	an act the policy forbids is attempted	regimented before commit by a gate when the event is controllable; otherwise detected after commit and compensated	holds when the effect never commits; fails as detect-and-compensate when the gate could not see the trigger	a tool's emergency override flag: the agent is *able* to call it and *forbidden* to
obligation	a commitment is recorded	an oracle-bound commitment: closure only through a receipt the daemon can re-check, before a daemon-set deadline	holds when a typed receipt closes it in time; fails by staying open and escalating, never by free-text assertion	"release the claim on `lib/webhooks.ts` within thirty minutes"
permission	a grant is issued	a recorded capability row, attenuable, never widened downstream	holds while the grant is live; an act without a grant is a prohibition case	"may read the audit log until the session ends"

Table 1.8. Three modalities use three different mechanisms; capability and permission remain separate so "able but forbidden" stays expressible.

that it "makes forbidden states physically unreachable" — is false as stated. The monitor is a *subscriber to an append-only event stream of operations that have already committed*. By the time it observes a violation, the offending write is already durable in the log. Even in its strictest mode, its strongest response is a *compensating* undo, not a prevention. This is not an implementation accident; it is a theorem.

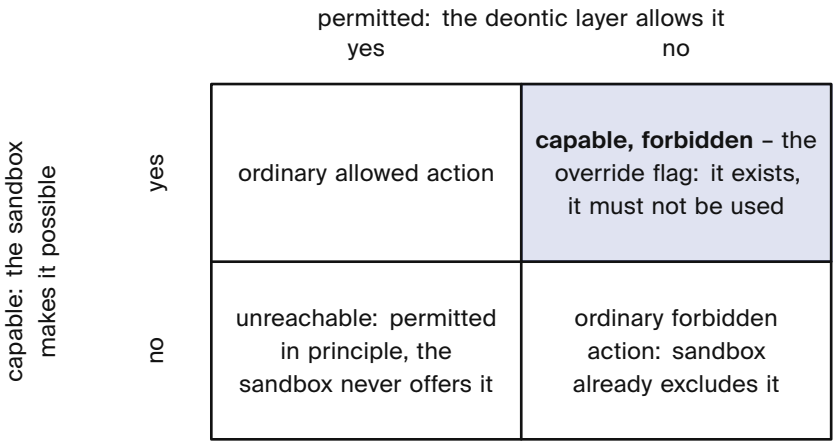


Figure 1.12. Capability and permission are independent. The shaded case is possible but forbidden—the state a collapsed vocabulary cannot express. [internal]

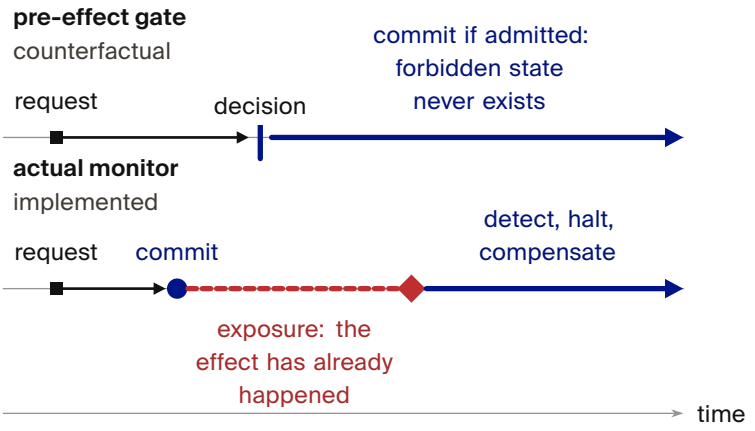


Figure 1.13. Pre-commit mediation prevents the forbidden state. A subscriber observes it afterward and can only halt or compensate. Uniqueness constraints and boot gates mediate; the current policy monitor subscribes. [internal]

Theorem 1.7.1 (A prevention bound for post-commit monitoring)**.**

A monitor invoked only after a transition commits cannot prevent a safety violation

whose bad prefix ends at that transition. It can detect the prefix, halt later actions, and enforce a separate recovery or bounded-response property. Therefore a post-commit subscription must not be credited with making the triggering state unreachable.

Proof sketch.

Schneider's execution-monitor model recognizes safety properties through finite bad prefixes and enforces them by suppressing an action

Fred B. Schneider. Enforceable Security Policies. *ACM Transactions on Information and System Security*, 3(1):30–50, 2000.

before that prefix enters the target execution. A subscriber invoked after commit cannot suppress the committing action. Compensation may establish a different eventual or bounded-recovery property, but it does not retroactively remove the bad prefix. □

Worked dramatization: codex:worker-2 attempts an out-of-scope edit. α_{worker} (`codex:worker-2`) is an agent under a policy that forbids editing files outside its assigned task scope. It nonetheless issues a write that records an out-of-scope edit. *What happens?* The write commits. The single writer serializes it, the uniqueness constraint has no objection (it is a well-formed row), and the operation lands durably in the log. Only *then* does the policy monitor, subscribed to that log, observe the out-of-scope edit and raise a violation. Its response is compensating: it can flag `codex:worker-2`'s session, revoke its claims, alert the operator, and trigger a salvage of the affected surface — but it cannot pretend the write never happened, because it did. Contrast this with `codex:bench` trying to claim a port `claude:worker-1` already holds (the previous dramatization): *that* attempt never commits, because the uniqueness constraint rejects it before the commit line. The two cases are the whole deontic story in miniature: a uniqueness constraint and a boot-time admission gate *regiment* (the bad state is unreachable); the policy monitor *enforces* through post-commit observation and possible remedy. This trace supplies no general response-time bound.

Constraints prevent; monitors detect

The only genuine pre-commit regimentation is credited to the right mechanism: the uniqueness constraint (no two holders of one resource key; no duplicate process-identity squat) and the fail-closed boot-admission gate (no serving from a test database; refuse to serve when a critical self-check fails). Those reject *before* the commit line and are *truly* unreachable. The policy monitor notices everything else *afterward* and compensates. So: "physically unreachable" for the regimented set; detection and possible remedy for the observed remainder. A bounded response window needs a separate observation and scheduling argument. Crediting double-claim prevention to the monitor *double-counts* — the uniqueness constraint already did it; the monitor merely logs it.

Theorem 1.7.2 (Synchronous guard sufficiency)**.**

Let $I(S)$ be an invariant of the committed database state, initially true. Assume every relevant transition $S \mapsto T(S, \Delta)$ passes through a serialized transaction that evaluates a terminating deterministic guard on that transaction's state and payload. If the guard admits Δ only when $I(T(S, \Delta))$ holds, and rejection has no relevant external effect, then I holds after every committed transition.

Proof idea and proof.

The guard checks the state that the transaction will actually commit, rather than a stale state read outside the transaction. Induct on the number of commits. The initial state satisfies I. An admitted transaction preserves I by hypothesis; a rejected transaction leaves the committed state unchanged. Serialization orders these steps, so the induction applies to every committed state. The conclusion concerns only effects inside this mediated transition system: writing a receipt does not make an arbitrary filesystem or remote effect atomic with the database transaction. □

This is a sufficient condition for a synchronous admission guard, not a necessary condition for prevention. An operation may remain pending while an external authority issues a permit bound to its identifier and payload; the executor can verify the permit and admit the effect only afterwards. Timeout can refuse admission. When permission depends on mutable external state, the protocol must additionally specify freshness, reservation or fencing so that the checked permission still authorizes the actual effect. External I/O can therefore increase latency or reduce availability without forcing a forbidden effect to occur before its check. The general boundary is controllability and observation, as developed next.



Wonham's supervisory-control theory, built for factory floors, is why a software policy can be proved enforceable rather than hoped.

1.7.3 The general boundary: regimentation is controllability

Express lane: a runtime can realize the whole permitted safety language exactly when no event the runtime cannot refuse can carry a legal history out of the rule; that is Ramadge–Wonham controllability with the model's own steps as the uncontrollable alphabet. The

formal statement is the box below; the machine-checked policy table is Table 1.9.[1]

The scene. The operations lead reads the governance document aloud. *No force-push to main. No network egress from sandboxed jobs. No writes outside the granted scope.* And then: *no confident falsehoods in status reports.* She asks the question every practitioner senses the answer to: which of these does the runtime *guarantee*, and which does it merely *watch for*? The first three feel airtight. The daemon holds the remote credential, so a push that never happens needs no forgiveness. The fourth needs a more precise event boundary. A mediator cannot prevent an unobserved internal model event in this plant. It may nevertheless refuse to publish an unverified report, or allow only registered facts with checked evidence. Generating a false candidate and delivering it are different events. Theorem 1.7.2 settled this for one door, the database commit. This section settles it for every door a runtime can stand at, and the instinct turns out to be an instance of a theorem from 1987.

The idea in one breath. *Split the agent's events into what the runtime can refuse and what it cannot; a safety language is exactly realizable if and only if no unrefusable event can carry a permitted history out of it. A stricter supervisor may preserve safety while sacrificing some permitted histories.*

Intuition: the bouncer at the one door. A club has one door and a bouncer. The bouncer can refuse *entry*: entry is mediated by the door he stands at. He cannot refuse a patron's *thought*: nothing the club owns mediates it. A door policy ("no entry after two") is enforceable by prevention; a thought policy ("no ill intent inside") cannot be enforced from the door alone; removing someone after observable misconduct is a different policy. The daemon's effect boundary is the door. Writes, egress, tool execution, pushes, spawns, and API calls cross it, so each can be refused; token emission, in-context reads, plan formation, and hidden activations complete inside the

[1] A standalone, submission-form version of this section is *Regimented or Enforced: The Controllability Boundary for Agent Governance* (research paper 2), whose policy table and verdicts regenerate from `skills/harbor-results/scripts/b3_controllability.py`.

model, so none can. The analogy maps relations rather than surface features, which is why it earns a prediction: a *compound* rule that mentions thought but gates only the door ("no entry for anyone who arrived after the fight started") should still be preventable, because the bouncer need only *observe* the trigger and *refuse* the door. Section 1.7.3.1 confirms exactly this. The predictable misread, preempted: uncontrollable does not mean invisible. Tokens are eminently loggable after emission. Uncontrollable means *unrefusable before the fact*, not unobservable after it; when an event is also unobservable a second, stricter theory applies (§1.7.3.2).

The vocabulary, at the point of use. Everything below is a statement about strings and sets of strings. An *alphabet* Σ is the finite set of event types the system can emit: `fs_write`, `net_egress`, `model_emit_token`, and the rest. A *string* over Σ is one possible history of an execution. A set of strings K is *prefix-closed* when every prefix of a member is a member: if the first ten events of a run are allowed, so are the first nine. The notation $\overline{K}$ means all prefixes of strings in K, so $\overline{K} = K$ when K is prefix-closed. Prefix-closed languages represent the finite histories of Lamport's safety properties: a violation has a finite witness, and once you have violated one you can never un-violate it. The *plant* L is the set of histories that are physically possible before any policy is imposed. The illustrative classification takes $L = \Sigma^*$, allowing every finite string; actual hardware and protocols may restrict that plant. A *policy* is a prefix-closed $K \subseteq L$. A *supervisor* is a function from the history so far to a set of events to disable next, and the whole theorem turns on its type: it may disable members of Σ_c, the *controllable* events that pass through a mediation boundary before taking effect, and nothing else. The *closed loop* is the set of histories that occur once the supervisor is attached. A supervisor *regiments* K when the closed loop equals K exactly: nothing forbidden happens and nothing permitted is lost. That second half is half the value. A supervisor that achieves safety by disabling everything is not a solution.

Theorem 1.7.3 (Regimentation Is Controllability).

Let $L \subseteq \Sigma^*$ be a prefix-closed plant over $\Sigma = \Sigma_c \uplus \Sigma_u$ and let $K \subseteq L$ be a nonempty prefix-closed safety policy. Assume *full observation*: the supervisor sees every event it does not disable, so the observable subalphabet Σ_o is all of Σ.

There exists a regimentation mechanism for K, a supervisor disabling only controllable events whose closed loop is exactly $\overline{K}$, if and only if K is *controllable* with respect to L and Σ_u:

$$\overline{K}\,\Sigma_u \cap \overline{L} \subseteq \overline{K},$$

that is, no uncontrollable event, physically enabled after a legal history, exits the policy.

When the condition fails, no such supervisor can preserve every legal history while preventing every violation. It may instead restrict behavior to the supremal controllable sublanguage $\sup C(K)$, which exists and is computable for regular K and L [verified framework: Ramadge and Wonham], but may be empty.

This is a restriction of permitted behavior, not a weakening of the safety requirement. Detection and compensation are separate possibilities only where observations and remedies make them available.

Relaxing full observation does not weaken this conclusion, it adds a second and independent condition: see Theorem 1.7.7.

Proved in §2.3.

Proof idea. If the policy is controllable, build the laziest possible supervisor: after any legal history, disable exactly the controllable events that would step outside the policy. It never has to disable an uncontrollable event, because controllability says none of those can step outside. If the policy is not controllable, there is a legal history after which the plant can fire an uncontrollable event that leaves the policy; any mechanism that keeps every legal history reachable must let that history happen, and then cannot stop the event.

Proof.

The proof is structured; each numbered step can be checked on its own.

Peter J. Ramadge and W. Murray Wonham. Supervisory Control of a Class of Discrete Event Processes. *SIAM Journal on Control and Optimization*, 25(1):206–230, 1987. (Controllability and the supremal controllable sublanguage — the general boundary used here to distinguish exact realization from safe restriction; the synchronous guard theorem gives a separate sufficient implementation condition.)

⟨1⟩1.

Assume K controllable. Define the supervisor

$$f(s) = \{a \in \Sigma_c : sa \in \overline{L} \setminus \overline{K}\}$$

for $s \in \overline{K}$. Prove the closed loop equals $\overline{K}$.

⟨2⟩1.

The closed loop is contained in $\overline{K}$, by induction on history length. The empty history lies in $\overline{K}$ because K is nonempty and prefix-closed. If the current history s lies in $\overline{K}$ and the loop executes a, then $sa \in \overline{L}$ and $a \notin f(s)$. If $a \in \Sigma_c$, then $a \notin f(s)$ means $sa \in \overline{K}$ by the definition of f. If $a \in \Sigma_u$, then $sa \in \overline{K}\Sigma_u \cap \overline{L} \subseteq \overline{K}$ by controllability.

⟨2⟩2.

Every $sa \in \overline{K}$ is reachable: an uncontrollable a is never disabled, and a controllable a with $sa \in \overline{K}$ is not in $f(s)$.

⟨2⟩3.

Q.E.D. Containment both ways is equality, so f regiments K.

⟨1⟩2.

Assume K not controllable: there are $s \in \overline{K}$ and $\sigma \in \Sigma_u$ with $s\sigma \in \overline{L}$ and $s\sigma \notin \overline{K}$. Prove no regimentation mechanism exists.

⟨2⟩1.

Let M be any regimentation mechanism for K. Because M realizes exactly $\overline{K}$, the legal history s is reachable under M.

⟨2⟩2.

At s the plant enables σ, and M cannot disable it: $\sigma \notin \Sigma_c$, so disabling it is outside M's type. Physically, the event completes before the mediation boundary sees it.

⟨2⟩3.

Q.E.D. The closed loop under M reaches $s\sigma \notin \overline{K}$, contradicting the assumption that M's closed loop is contained in $\overline{K}$. This rules out realizing *all* of K exactly. A supervisor may instead exclude s by restricting earlier controllable behavior, thereby realizing a smaller safe language. Post-violation response may remain available if the design insists on admitting that history, but the theorem neither guarantees nor requires a remedy.

⟨1⟩3.
The existence, uniqueness, and computability of $\sup C(K)$ are imported from Ramadge and Wonham: controllable sublanguages of K are closed under arbitrary union, so a supremal element exists, and for regular languages a fixpoint iteration on the product automaton computes it.

Ramadge & Wonham 1987

⟨1⟩4.
Q.E.D.

□

Two remarks on what the proof used. The forward direction is constructive and *maximally permissive*: the supervisor disables nothing except the controllable exits, so regimentation costs no legal behavior. The governance rule and the agent's freedom are not in tension when the rule is controllable at all. The reverse direction used nothing about the mechanism except its type. No cleverness inside the runtime, no better prompt, faster monitor, or smarter policy engine, escapes the theorem, because the obstruction is that σ finishes before any decision point the runtime owns. Theorem 1.7.2 gives a sufficient construction for a mediated database invariant. The controllability theorem instead characterizes exact realization of a specified prefix-closed language under full observation. It distinguishes owning the credential channel for a push from observing an internal model event after it has happened. Neither statement makes local database computability necessary for every safe protocol.

Numbers by Hand 1.7.4 (The Policy Table, Checked by Hand).

The controllability test is decidable by a product-automaton search: walk the synchronous product of plant and policy automaton and flag any reachable state where the plant enables an uncontrollable event that the policy disables. Run over the reference alphabet, the checker returns Table 1.9 [internal, `b3_controllability.py`; deterministic, no seed]. The eight single-event rows also follow by hand, which is the right way to first believe the table.

Take "forbid `git_push`." The policy automaton has one state with every event except `git_push` self-looping; `git_push` lies in Σ_c,

so the only event the policy ever disables is controllable, the premise of a controllability violation is unsatisfiable, and K is controllable: regimentable.

Now take "forbid `internal_plan`." The same one-state automaton disables `internal_plan`, which lies in Σ_u; the universal plant enables it at the start state, so the empty history and σ = `internal_plan` witness

$$\overline{K}\Sigma_u \cap \overline{L} \not\subseteq \overline{K}:$$

not exactly realizable by any supervisor with this alphabet split. Detection requires an observation as well; the failed controllability test does not supply one.

Now you try: classify "forbid `api_call` after `spawn_child`" and "forbid `internal_plan` after `fs_write`" (Exercise 1.19).

Table 1.9. Nine governance rules classified by the product-automaton checker. The verdicts are the checker's output; the eight single-event rows also follow by hand from Theorem 1.7.3 [internal, `b3_controllability.py`].

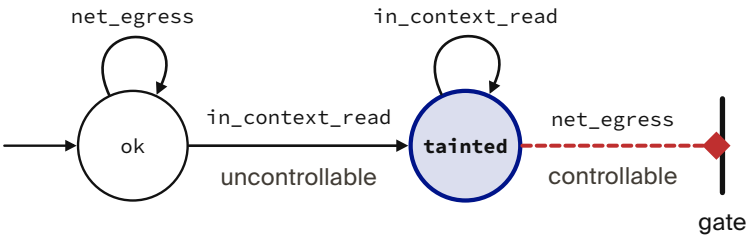
Policy	Event class	Regimentable?
forbid `net_egress`	controllable	yes (regiment)
forbid `git_push` (incl. force-push)	controllable	yes (regiment)
forbid `fs_write` (out-of-scope write)	controllable	yes (regiment)
forbid `exec_tool`	controllable	yes (regiment)
forbid `spawn_child`	controllable	yes (regiment)
forbid `model_emit_token`	uncontrollable	no (cannot prevent)
forbid `in_context_read`	uncontrollable	no (cannot prevent)
forbid `internal_plan` ("confident lie")	uncontrollable	no (cannot prevent)
compound: no `net_egress` after a secret `in_context_read`	mixed	yes (regiment)

1.7.3.1 The compound case: gate the channel, never the token

The last row of Table 1.9 is the one an entire product line rests on. The policy *no network egress after an in-context read of a secret* mentions an uncontrollable event, so a naive reading of the single-event rows predicts detect-only. The checker says otherwise, and the mechanism is visible in the policy automaton (Figure 1.14): two states, `ok` and `tainted`. The uncontrollable read is enabled in both. The policy never tries to forbid it; it lets the read happen and records

the taint transition. What the tainted state disables is `net_egress` alone, and egress is controllable. Every uncontrollable event the plant enables is enabled by the policy in every reachable state, so the criterion holds and the constructive direction of the theorem hands us the supervisor: track taint, gate the channel. This automaton is Schneider's own Figure 1, a two-state security automaton for "no `Send` after a `FileRead`", with his states relabelled. What the alphabet split adds is not the construction but the grading: under complete mediation the figure is one more enforceable safety policy; once $\Sigma_u \neq \emptyset$ the same two states become the canonical witness that a policy may reference an unrefusable event freely so long as it never tries to refuse it.

Schneider 2000



event	alphabet	in `ok`	in `tainted`
`in_context_read`	Σ_u, uncontrollable	enabled, to `tainted`	enabled
`net_egress`	Σ_c, controllable	enabled	disabled

Figure 1.14. Reading a secret records taint; it is never refused. Afterward the policy blocks mediated network egress, while leaving uncontrollable events enabled. [internal, `b3_controllability.py`]

Three consequences, in increasing order of consequence.

1. **The design rule.** In this fully observed two-state example, the policy forbids only controllable egress, so it is exactly realizable. For another compound policy, check every reachable legal history against the theorem. An uncontrollable forbidden effect is an obstruction only where the plant actually enables it; a trigger that never occurs imposes no such obligation. Under partial observation the trigger must also be distinguishable, as the next subsection shows.

2. **The sealed room.** Put an agent alone in a room with a customer's secret and promise that the secret does not leave. It is impossible to prevent the model from *reading* the secret (the read is uncontrollable, and useless to forbid, since reading is the job) and impossible to prevent it from *incorporating* the secret into tokens and plans (uncontrollable again). What is possible, exactly and only, is to gate every controllable egress channel out

of the room, with taint recorded from the moment of exposure. Gate the channel, never the token. The companion noninterference result for the sealed room proves from the other direction that the gate suffices; controllability says the gate is the only place a guarantee *can* live. This is why "all spend is recorded" in the economy chapters is precisely complete mediation of Σ_c.

3. **The two dials.** With plant, observation, and policy fixed, widening Σ_c (owning more channels) can remove a controllability obstruction; relabelling the same policy cannot. The plant is the other dial: $\overline{L}$ sits on the left of the containment, so shrinking it can remove offending histories, though a strict improvement is not guaranteed. Both detect-only verdicts above turn on "the universal plant enables `internal_plan`," and are conditional on the modelling choice $L = \Sigma^*$. An event that never becomes physically possible needs no supervisor, which is why data minimization and capability removal are governance moves and not merely hygiene: a secret never loaded into the room regiments the compound policy with no gate at all.

Numbers by Hand 1.7.5 (Synthesis by Hand: The Sandboxed Review Job).

The theorem's constructive direction is the synthesis algorithm the Supremica and TCT tool families implement, and it is small enough to run on paper. A sandboxed code-review job carries three rules: P_1, no `net_egress`, ever; P_2, no `git_push`, ever; P_3, after any `in_context_read` of customer material, no `fs_write`.

P_1 and P_2 are one-state automata and P_3 is the two-state taint automaton, so the product specification has

$$1 \times 1 \times 2 = 2 \text{ states},$$

`ok` and `tainted`. The controllability check visits both states and tests each of the four uncontrollable events in each: all eight (state, event) pairs are enabled by the specification, so there are 0 violations and the composite is controllable [verified: eight cases, checkable by eye].

The maximally permissive supervisor falls out of the proof as a two-row disablement map: in `ok`, disable $\{\texttt{net_egress}, \texttt{git_push}\}$; in `tainted`, disable

{net_egress, git_push, fs_write}; one observed transition flips ok to tainted.

That table is exactly the policy engine a competent engineer would have written by hand. The value of synthesis is not surprise; it is that the answer arrives with a maximal-permissiveness certificate and a controllability certificate, neither of which hand-written engines carry.

Synthesis is equally informative when it fails. Ask for the best preventable approximation of "forbid internal_plan": any nonempty controllable sublanguage K' contains the empty history, the plant enables internal_plan there, so controllability forces

$$\texttt{internal_plan} \in \overline{K'} \subseteq \overline{K},$$

contradicting that K forbids it. Hence

$$\sup C(K) = \emptyset$$

[verified: three lines]. There is no nontrivial preventable weakening of a thought ban, and a governance document that claims to "partially enforce" such a rule by prevention is describing detection with optimistic vocabulary.

Now you try: add P_4, "no spawn_child after git_push," to the review job and write the new disablement map (Exercise 1.20).

Table 1.10 states the two-row disablement map the proof falls out as.

state	disabled events
ok	net_egress, git_push
tainted	net_egress, git_push, fs_write

Table 1.10. The sandboxed review job's maximally permissive supervisor: a two-row disablement map, checkable by eye against the three rules P_1–P_3. [verified: eight cases]

1.7.3.2 Partial observation: the second boundary

Theorem 1.7.3 assumes the supervisor *observes* every event, even the ones it cannot disable: the daemon logs tokens after emission and reads after completion. Realistic deployments break this in a second place. Some events are not only unrefusable but unobservable: activations never surfaced, state internal to a third-party tool,

another principal's context. Supervisory control under partial observation is also a classical extension. The next example shows why seeing the trigger is a separate requirement from controlling the effect.

Numbers by Hand 1.7.6 (Two histories, one visible record).

Let `read-secret` be hidden and `send` controllable. Compare two histories: no read, and one hidden read. Both produce the empty visible record. A rule allowing send in the first history but forbidding it in the second asks the supervisor for two different decisions from the same input. No such decision rule exists. Blocking send in both histories preserves confidentiality but sacrifices the permitted send in the first; recording exposure before the read makes the histories distinguishable [internal].

Theorem 1.7.7 (Regimentation under Partial Observation).

Let L, K, Σ_c and Σ_u be as in Theorem 1.7.3, but let the supervisor observe only $\Sigma_o \subseteq \Sigma$, through the projection $P : \Sigma^* \to \Sigma_o^*$ that erases every unobservable event.

A regimentation mechanism for K whose decisions depend only on what it observed — a supervisor that factors through P — exists if and only if K is controllable with respect to L and Σ_u *and* K is *observable* with respect to L, P and Σ_c: for all $s, s' \in \overline{K}$ with $P(s) = P(s')$ and every $\sigma \in \Sigma_c$,

$$s\sigma \in \overline{K} \ \wedge \ s'\sigma \in \overline{L} \implies s'\sigma \in \overline{K},$$

that is, no two histories the runtime cannot tell apart may demand opposite decisions about the same controllable event [verified framework: Lin and Wonham].

Feng Lin and W. Murray Wonham. On observability of discrete-event systems. *Information Sciences*, 44(3):173–198, 1988.

Proof idea. A supervisor that only sees $P(s)$ has one decision to spend on the whole class of histories that share a projection. If the policy ever wants σ enabled after s and disabled after an indistinguishable s', no such supervisor can serve both, and the failure is not a matter of trying harder: it is a counting argument about how many decisions the observation map leaves available. If the policy

never asks for that, the laziest supervisor of Theorem 1.7.3 can be defined on projections instead of histories, and the same closed-loop argument goes through unchanged.

Partial observation can shrink the exactly realizable set, and $\Sigma_o = \Sigma$ collapses the condition to a tautology, which is why the first theorem never had to mention it. "No egress after the model *considered* exfiltration" has a controllable prohibition and an unwitnessed trigger, so no supervisor can implement its gate condition — two runs, identical in everything the daemon logged, differing only in a deliberation nobody saw, are exactly the s and s' the condition forbids. The practical reading: a compound policy is deployable only if its trigger is witnessed at the mediation boundary. Taint must be an event the daemon sees, not a state it infers. Figure 1.15 puts both boundaries on one grid.

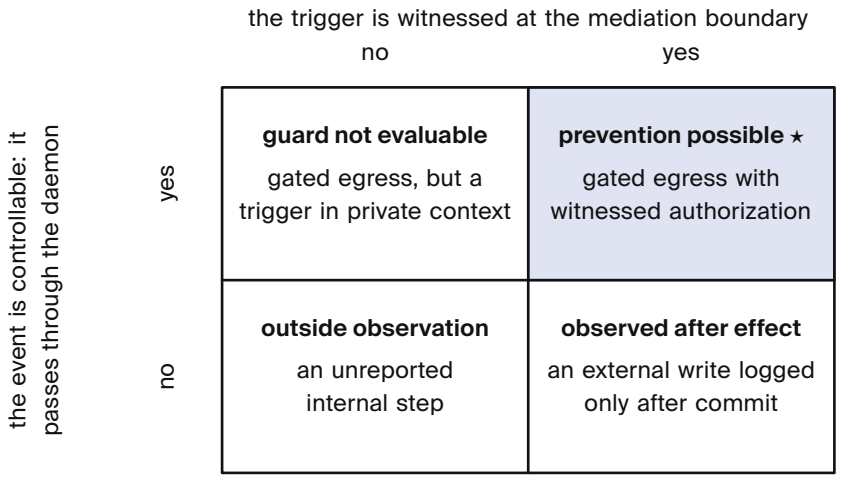


Figure 1.15. Exact realization needs control and observations sufficient to evaluate the specified guard. The partial-observation theorem states the full condition. A stricter policy, such as refusing every egress, may preserve safety while rejecting legal histories. Detection outside the shaded cell is not automatic. [internal; schematic]

What it buys. The theorem separates three questions an implementer must ask: can the supervisor refuse the effect; can it distinguish histories requiring different decisions; and must it preserve every legal behavior or may it safely restrict some? A rule currently monitored after commit may still be preventable by a properly placed guard. Conversely, an uncontrollable violation need not be observable at all. Later audit, reputation, and remedies therefore require specified evidence as well as a response mechanism; they do not follow from failure of exact controllability.

The ladder these modes form has two obvious public neighbours, and since the resemblance is real it's worth saying exactly what each of the three grades before somebody assumes they're the same instrument in different clothes. Sheridan and Verplank rank a system on a ten-point scale by how much of the decide-and-act loop the computer holds rather than the human; Parasuraman, Sheridan and Wickens later split that single scale across four stages of processing — acquisition, analysis, decision selection, action implementation — each carrying its own level. Both grade the *allocation of the work*: who does it. The Common Criteria's evaluation assurance levels, EAL1 through EAL7, rank a security claim by how much independent evaluation effort was spent before deployment and against how deep a body of design evidence, from functionally tested at the bottom to formally verified design at the top. EAL grades *how hard somebody looked*, once, at evaluation time, for a whole product. This ladder grades neither, and the question it does ask is narrower than both: of one runtime claim about one mechanism, *what would have to be defeated for the claim to be false?* Nothing at all at Observed — the act may have happened somewhere else entirely and the record was written afterwards — then a client's willingness to declare at Coordinated, a daemon-owned broker at Brokered, the operating system's own confinement at Confined, and a measured environment plus a conditional key release at Attested. That is an adversary's work factor, per mechanism, at run time, and neither neighbour measures it.

Thomas B. Sheridan and William L. Verplank. *Human and Computer Control of Undersea Teleoperators*. Man–Machine Systems Laboratory, Department of Mechanical Engineering, MIT; ONR Technical Report, 1978. NTIS ADA057655.

Raja Parasuraman, Thomas B. Sheridan, and Christopher D. Wickens. A Model for Types and Levels of Human Interaction with Automation. *IEEE Transactions on Systems, Man, and Cybernetics – Part A*, 30(3):286–297, 2000.

ISO/IEC 15408-1:2022. *Information security, cybersecurity and privacy protection — Evaluation criteria for IT security — Part 1: Introduction and general model* (Common Criteria, CC:2022). International Organization for Standardization, 2022.

Design Invariant 1.7.8 (The Three Gradings Are Independent Axes).

No mechanism's rung in this book is assigned from its automation level or from its evaluation depth, and the corners are all occupiable.

A fully autonomous component (Sheridan level 10) whose actions are merely logged is Observed; a component that stops for human approval at every step (level 3) but routes its one effect through an attested broker is Attested.

An EAL7 product can contain Observed mechanisms and an unevaluated one can contain Attested mechanisms, because EAL scores the evaluation and this ladder scores the mechanism.

The boundary on that is worth stating in the same breath: the ladder is a design vocabulary, not a measured scale. No procedure assigns a rung by measurement, no inter-rater agreement stands behind the assignments, and the five rungs are not claimed to be evenly spaced or even totally ordered — Brokered and Confined defeat different adversaries rather than the same adversary harder, which is why the appendix says these modes are not a maturity staircase. What the ladder is for is naming which part of the system a claim rests on, and it earns that only by being applied consistently.

Where this stops

(i) The theorem does not say detect-only policies are hopeless. Detection feeding audit, reputation, and slashing is real enforcement; the theorem rules out exact realization of the full permitted language. A stricter supervisor may still prevent violations by sacrificing legal histories; detection requires suitable observations.

(ii) It certifies no implementation. Its Σ_c is the set of events *actually* mediated, and every unenumerated side channel (raw sockets, DNS, inherited file descriptors, `/proc`, credential helpers, package managers, provider callbacks) silently moves events from Σ_c to Σ_u and shrinks the regimentable set behind the operator's back. Complete mediation of the claimed alphabet is an architectural obligation (Anderson; Rushby) that a red team, not this proof, discharges.

(iii) The classification is relative to the alphabet split: a runtime that gates model steps changes Σ_u and re-grades the table. The verdict therefore belongs to the stated architecture and event model; it is not an architecture-independent label for a policy sentence.

(iv) Full observation is the assumption this classification rests on: policies whose triggers are unwitnessed internal state fall to the Lin–Wonham refinement even when their prohibitions are controllable.

(v) The checker verifies the stated automata over the stated alphabet; it is a faithful implementation of the test, not a mechanized proof of the theorem. An Isabelle or Coq formalization remains open.

(vi) Prevention governs effects, never semantics. For "no confident falsehoods in status reports" the obstruction is observability:

a report leaves through emission and then through `api_call`, `fs_write`, or `net_egress`, all controllable, so the prohibition can be made controllable; but the coarse event alphabet does not say whether a report is true. A stronger interface might admit only registered facts with checked receipts. For unrestricted prose, the chapter supplies no complete truth oracle. The remaining obligation is to specify observable evidence sufficient for the claim being released, not to infer semantic correctness from control of the send operation.

Recall

1. Which event sets determine controllability? Which can the operator change?
2. What must the runtime observe to evaluate a compound guard?
3. Why can gating every effect still fail to prevent a false report?

The monitor is honest about its own coverage, which is itself a genuine security property: it reports each rule as *enforced*, *degraded*, or *stubbed*, so a reader can never mistake a stubbed rule for an active one. We note in §1.12 that one rule's enforcement (capability escalation) is backed by a separate native component whose evidence is the weakest in the subsystem, carrying a time-of-check-to-time-of-use surface.

1.7.4 Commitments: obligation with oracle-bound closure

Suppose A reserves port 7421 and promises to release it within 30 minutes. At minute 29, a note saying "released" does not settle the promise: the claims table must show that A no longer holds the lease. This distinction between saying and demonstrating completion motivates the closure rule. Here an *oracle* is a named machine-checkable predicate, not an infallible judge of every aspect of the work. A release predicate can be checked against a claims row; a test predicate needs the relevant source revision and test result bound together.

Commitments are durable, *violable* promises — in the sense of Cohen and Levesque's "intention as a persistent goal", with both single-minded and open-minded strategies for when a promise may be dropped. Two hardening laws are implemented, and they are the ones that determine safety (Figure 1.16).

Philip R. Cohen and Hector J. Levesque. Intention Is Choice with Commitment. *Artificial Intelligence*, 42(2–3):213–261, 1990.

Property 1.7.1 (Metric-control separation, Law 1)**.**

A commitment's deadline is derived by the daemon from a scope policy and is *never* agent-supplied. The agent picks the work; the daemon picks the deadline. (Invariant I5.) The point is to deny the agent control of the very metric it is judged against.

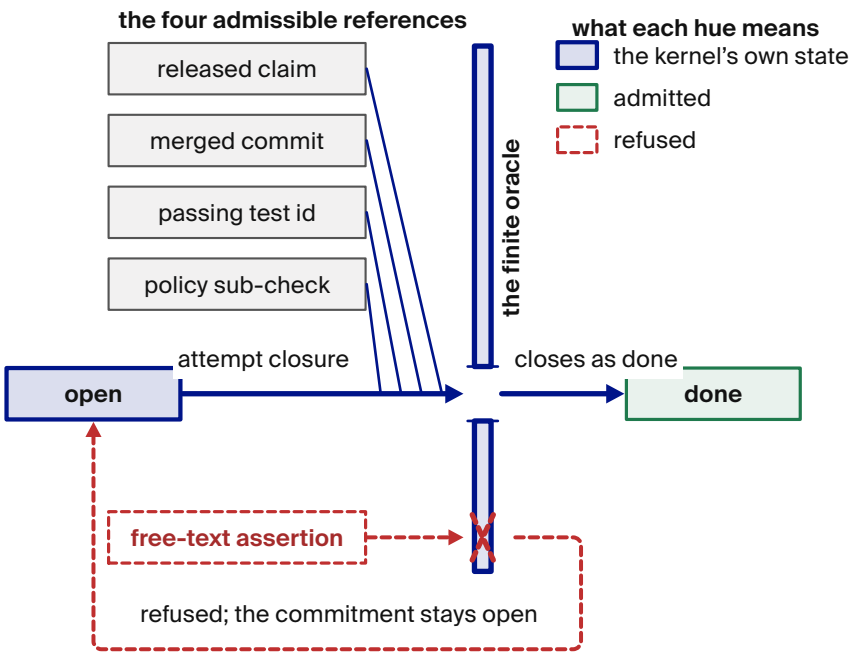


Figure 1.16. Closure requires a supported evidence reference rechecked by the daemon. Free-text assertions leave the commitment open; scope policy, not the agent, supplies its deadline. [internal]

This removes one direct manipulation channel; it is not a general proof of resistance to proxy gaming or misspecified scope policy.

Property 1.7.2 (Oracle-bound closure, Law 2)**.**

Closing a commitment as "done" is refused unless the agent supplies a non-empty reference drawn from a *finite oracle vocabulary*: a released claim, a merged

commit hash, a passing test identifier, or a satisfied policy subcheck. Free-text notes cannot close a commitment. (Invariant I6.)

The closure gate is small enough to state as a protocol (Algorithm 1.3). The crux is the final clause: the supplied reference must not only be drawn from the vocabulary but must *currently hold* when the daemon re-checks it, so an agent cannot name a test that passed yesterday and has since regressed.

Receipt checks do not judge quality

That finite oracle vocabulary *is* the kernel's honesty boundary. It can mechanically verify "the test you named passed," never "the refactor is good." Naming the oracle set is naming the limit of what the substrate layer can verify — and open problem OP-5 asks which real

Listing 1.3. The commitment-closure oracle. Closure requires a reference from a finite vocabulary that the daemon re-verifies at closure time; free text is rejected by construction, not by inspection.

```
function close(commitment, status, oracle_ref):
  if status != 'done':
    return apply_drop_strategy(commitment, status)   // abandon / supersede
  if oracle_ref is empty:
    return REFUSED("done requires a verifiable oracle reference")
  kind = classify(oracle_ref)                        // structured field, not NLP
  // the four kinds below are the SHIPPED vocabulary. State-Machine Assertions
  // (OP-5, specified only) would add DELTA_ORACLE and AST_ASSERTION here.
  if kind not in {RELEASED_CLAIM, MERGED_COMMIT, PASSING_TEST,
     POLICY_SUBCHECK}:
    return REFUSED("oracle reference outside the finite vocabulary")
  if not verify_holds_now(kind, oracle_ref):         // re-check at closure time
    return REFUSED("named oracle does not currently hold")
  transition(commitment, DONE, witnessed_by = oracle_ref)
  return CLOSED
```

"done" conditions it fails to capture, and whether the set can be extended without re-opening the Goodhart hole that free-text closure would.

1.7.5 Extending the oracle vocabulary: State-Machine Assertions (OP-5)

The obvious way to widen what a commitment can close against is to widen the vocabulary, and there is a concrete design for doing so without re-admitting the Goodhart hole. **State-Machine Assertions** (specified) would add two kinds to Listing 1.3's finite set: *delta oracles*, which close against a machine-checkable before/after measurement (a named benchmark improving past a declared threshold), and *AST assertions*, which use a parser such as `tree-sitter` to state a structural property of the resulting code ("no call site of f remains") that the daemon can re-evaluate at closure time.

Both kinds satisfy the property that matters — the daemon can recheck them itself, so they are oracles and not testimony. Neither is

in the shipped set: Listing 1.3's four kinds are the vocabulary the kernel actually enforces, and the listing marks the two proposed additions as exactly that. Two questions also stand between the design and the claim, which is why we do not make it. A delta oracle inherits the noise of whatever it measures, so "improved by a threshold" is a statistical statement dressed as a boolean; and an AST assertion is a proxy for a structural intent, which is the shape of every target that has ever been gamed. *Oracle completeness* — the claim that some finite vocabulary captures the "done" conditions agents actually need — is not established by adding two kinds, and OP-5 stays *open*.

The separate question of who checks a grader has a conditional answer in *From Spawn to Person* (Theorem 5.11.1). Its inspection-tower model distinguishes bribery incentives from detection. With prepaid bribes and a sealed, uniform draw from separately bribable candidate groups, buying cooperation does not improve the briber's expected payoff below the stated threshold; at equality, the model resolves the tie against bribery. In that regime, the stated positive lower bound on detection, conditional on all earlier levels having missed, gives geometric decay of expected surviving corrupt value. Disjoint groups alone do not establish this detection premise, and the tower still requires a trustworthy terminal adjudicator.

Neither result establishes oracle *expressiveness*: whether the vocabulary can state the closure condition at all. Even a competent, honestly motivated grader cannot close a commitment whose condition the vocabulary cannot phrase. The tower therefore does not discharge OP-5's completeness obligation.

Exercises 1.16–1.21, p. 151.

1.8 The continuity subsystem: memory, checkpoint, and the foundation of the economy

The through-line of the whole series — memory → checkpoint → continuity → durable identity → reputation → market — *bottoms out here*. If the substrate's continuity is weak, the cross-machine economy is built on sand. This is the section the personhood and market

papers lean on hardest, so it carries the canonical statement of the layer's own softest link.

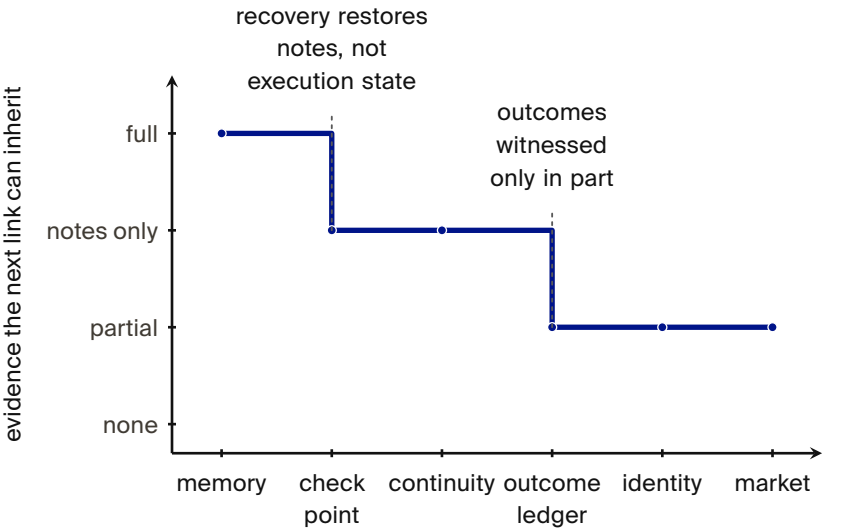
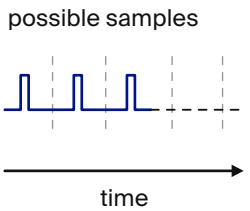


Figure 1.17. The continuity chain as an evidence-support profile, computed from the chapter's own status grades. What checkpoint inherits from memory already drops from full to notes only; what the outcome ledger inherits from continuity drops again, to partial; identity and market inherit only that partial residue, not an independent grade of their own. Downstream confidence cannot exceed the weakest upstream link. [internal]



Silence between heartbeats is normal. A missed deadline raises suspicion, not proof of death: delay and failure can look alike.

1.8.1 Three subsystems, one weak link

Figure 1.17 names the three continuity subsystems and states the canonical sentence the rest of the series depends on:

The economy rests on three continuity subsystems — memory (implemented), checkpoint (partial: notes, not execution state), and a witnessed-outcome ledger (partial: commitments and oracle-bound closure, not neutral graded outcomes) — and reputation requires outcomes to remain bound to a durable accountable identity. Restoring a process's exact inference state is a different goal: an outcome record can survive even when the computation cannot resume.

Memory — session records, durable notes, and the operation log — is **implemented**, and the kernel even forbids amnesia: a monitor rule guards that an active session's durable note count never regresses (detect-and-compensate, per Theorem 1.7.1, but real).

Checkpoint is partial, and the weakness is precise. The recovery pipeline runs a heartbeat watchdog → stale/dead detection → a recovery queue → a salvage handoff, and it *passes notes*; it does

not checkpoint execution state. A checkpoint with teeth — the gap between passing notes and restoring work — is open problem OP-4, an open recovery requirement. Reputation needs retained, attributable outcomes; it does not require bit-identical restoration of a computation.

The most promising direction we have for it is worth naming, at exactly its strength. **Event-sourced rehydration** (specified) would treat the successor's starting context as a replay rather than a summary: pin the working tree to the predecessor's Git commit, truncate the durable message log to the recorded failure point, and replay that log into a fresh agent so its prompt prefix is reconstructed from the record instead of paraphrased from it. What this restores is the *durable* half — a commit identifier, a message log, an open-claim and commitment set — and that half is genuinely worth having. What it does not establish is the half OP-4 is named for. A language model's inference state is bound to a particular model, sampler, and session; a replayed prefix reconstructs the *inputs* to that state, not the state, and the reference implementation ships none of this today. Whether replaying the inputs is close enough to count as "restoring work," and against what test, is precisely what remains open — so we report the mechanism as a direction with an unestablished completeness claim, not as a closure of OP-4. The companion personhood chapter reaches the same boundary from the identity side: a continuity witness attests that the ledger followed the right body, never that the successor inherited the predecessor's experience.

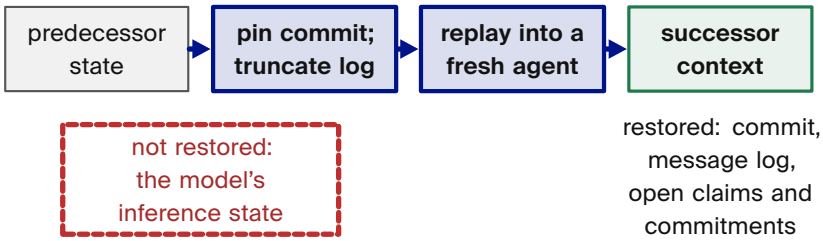
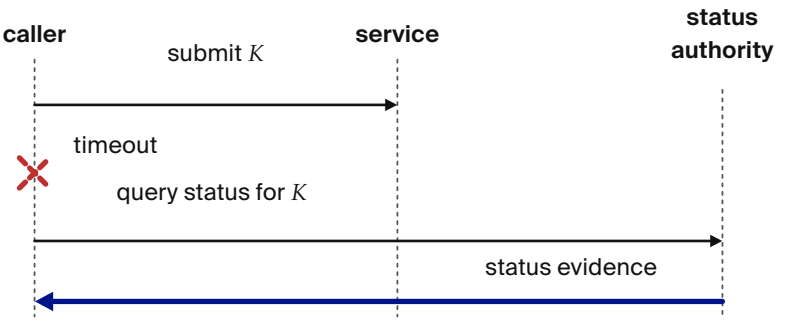


Figure 1.18. Proposed replay restores recorded inputs and durable task state, not hidden model inference state. Completeness remains unestablished; OP-4 is not closed. [internal]

Figure 1.18 draws the pipeline with its own exclusion: what replay restores, and the one thing it does not.

Figure 1.19 makes the retry boundary concrete: a caller that receives a transport timeout or lost response cannot infer whether the write failed or succeeded out of sight, and blind retry risks violating once-only state invariants unless bounded by the outcome ledger.



Readback	Permitted conclusion
Committed	Record receipt; suppress duplicate.
Final absence	Retry only under a proved idempotent boundary.
Inconclusive	Keep outcome unknown; escalate.

Figure 1.19. A timed-out external operation leaves its effect unknown. The caller queries an operation-specific authoritative status before deciding whether to record a receipt, retry under a proved idempotent boundary, or preserve an unresolved outcome. An absence result permits retry only when the authority excludes a late commit for K; the drawing does not make unbrokered delivery exactly once [internal].

The witnessed-outcome ledger — the durable record of what an agent actually delivered — is the subsystem on which reputation keys. A partial substrate now ships: durable commitments, daemon-derived deadlines, oracle-bound closure, API/CLI surfaces, and overdue detection. Neutral graded outcome events, sanctions, identity binding, and reputation updates remain absent. OP-4 concerns continuing interrupted work; the richer outcome ledger concerns attributing and assessing completed work. They share durable records but one does not discharge the other.

1.8.2 The operation log and tamper-evidence

The append-only operation log is the kernel's audit subsystem — the stream the policy monitor subscribes to and the thing that makes "what actually happened" answerable. Hash-chained tamper-evidence over it — the digital-timestamping pattern of Haber and Stornetta that predates blockchains, itself building on Merkle's hash trees — exists but is re-scoped (see §1.12.4).

Stuart Haber and W. Scott Stornetta. How to Time-Stamp a Digital Document. *Journal of Cryptology*, 3(2):99–111, 1991.

Ralph C. Merkle. A Digital Signature Based on a Conventional Encryption Function. In *Advances in Cryptology — CRYPTO '87*, 1987.

Exercises 1.22–1.24, p. 152.

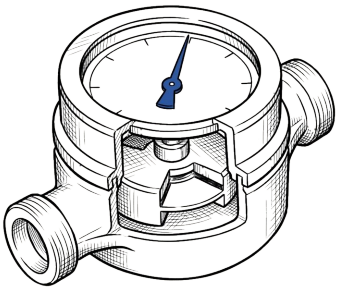


Figure 1.20. Monotonic Operation Log. The dials advance as an append-only record grows.

1.9 The self-attestation subsystem: honest green

Honest self-attestation belongs to the substrate layer, because the kernel is the one component that can formally report its own integrity: it is always-on and owns the only writable truth. The attestation routine evaluates a registry of invariants, each returning *pass*, *fail*, *skipped-with-reason*, or *unknown*. The report is *scoped and conjunctive*: "all good" is printed only when every checked invariant passes, and the report always enumerates what was *not* checkable.

Property 1.9.1 (Honest self-report, I10)**.**

The attestation reports "all good" only if every checked critical invariant passed, and it always lists the unchecked set.

There are three triggers: a boot-admission gate (refuse to serve on a critical failure), a command pre-flight (loud refusal that names the fix), and a continuous watchdog (a pass-to-fail transition deposits a coordination marker and a notification). **implemented**.

Table 1.11 lines the three triggers up against when each fires and what it does when a checked invariant fails; a row is silent about *which* class of failure is most likely there because the registry is the same one at every trigger, only the moment and the response differ.

This is the honesty discipline of the rest of the series turned on the kernel itself, and it is the right posture for a trusted computing base: when integrity is unknown, deny. The drift and liveness self-checks exist precisely because a second copy of the daemon *can* come up against the same file — a packaged install lagging the development tree is a recurring hazard — and the kernel must know which copy of itself is the real one.

Table 1.11. Three triggers evaluate one invariant registry; their timing and failure response differ. [internal, I10]

trigger	when it fires	on a critical failure
boot-admission gate	at daemon start, before serving any request	refuses to serve
command pre-flight	before a command runs	a loud refusal that names the fix
continuous watchdog	on every check cycle while running	a pass-to-fail transition deposits a coordination marker and a notification

1.10 The invariants, stated as theorems

A completionist treatment names the kernel's properties as falsifiable claims, each with its mechanism and its maturity grade. Table 1.12 is the formal spine of the layer; the grade column is the maturity discipline turned on the theorems themselves.

1.10.1 The work unit, model-checked

Express lane: the kernel's promises are properties of one object, an evidence-bearing work unit; on a two-principal instance every one of its 536 reachable states satisfies six invariants, and removing any single guard lets a checker construct the crime in at most seven steps. The machine is Figure 1.21 and the box below; the numbers are Table 1.13.

The scene. An agent dies mid-task. Its successor, possibly a different model from a different vendor, picks up the work and later says "done." Who checks the paperwork? Where is the record that the port it held was released, that the payment moved once and not twice, that "verified" meant an inspection actually happened rather than a mood? Every subsystem in this chapter answers a piece of that question. The work unit is the object that holds the pieces together: a case file that outlives every clerk who touches it.

The idea in one breath. *Make the unit of trust a work unit whose every consequential step passes a guard, then prove, by exhausting the machine's reachable states, that its six promises can never*

#	Invariant (theorem)	Mechanism	Maturity
I1a	Process-crash durability	WAL + OS page cache	**implemented**
I1b	Power-loss durability	(needs full synchronization)	*not guaranteed*
I2	Mutual exclusion (1 holder/key)	uniqueness constraint + busy-wait	**implemented**
I3	Idempotence of re-claim/re-release	upsert / delete-if-exists	**implemented**
I4	Liveness reclamation (TTL sweep)	lazy expiry on read + ticks	**implemented**
I5	Metric-control separation (Law 1)	daemon-derived deadline	**implemented**
I6	Oracle-bound closure (Law 2)	finite oracle vocabulary	**implemented**
I7	No-amnesia (note monotonicity)	monitor: note-monotonicity rule	partial (detected, not regimented)
I8	Forbidden-state handling	constraint/boot (regimented) + monitor (detected)	partial (mostly detect-and-compensate)
I9	Tamper-evidence of the audit log	hash chain	partial (re-scoped to the economy layer, §1.12.4)
I10	Honest self-report	scoped attestation report	**implemented**
I11	Runtime parity (interpreter ≡ test bindings)	runtime-shim layer	partial (OP-3)
I12	Non-forgeable identity	actor-soul id + lookup credential	partial (bounded gate ships; universal write gating absent)

Table 1.12. The kernel invariants. I1 is split by fault class (I1a/I1b) per §1.4; I7–I9 carry the regimentation/detection correction per §1.7.

be broken, and that every guard is essential, because removing any one lets the checker construct the crime.

Intuition: the notarized escrow folder. Nothing enters the folder without the right stamp: an active grant carrying the *current* role epoch, so a clerk who was replaced yesterday finds that today's stamp rejects him. No payment leaves twice: each external operation carries an idempotency key, honored at most once. And "verified" is not a feeling: it means the acceptance policy and the inspector's receipt are physically in the folder. Then we do something a paper folder cannot: enumerate every state the machine can ever reach and check all six promises in each. The misread to preempt: a model checker does not prove the daemon correct. It proves the *specification* correct at these parameters; the daemon must refine it, and that refinement is a separate obligation named in the boundary below.

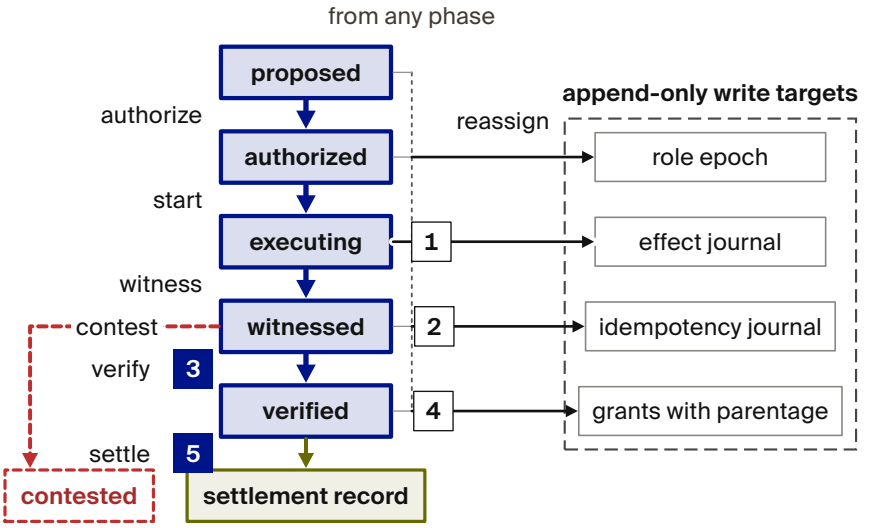


Figure 1.21. Five guards protect the work-unit lifecycle. In the two-principal model, all 536 reachable states satisfy six invariants; removing any guard exposes the counterexample listed below. [internal, `c0_workunit.py`]

	guard	crime once it is removed	steps
1	admit-effect: current epoch only	the stale write (I1, I5)	4
2	external-op: at most once per key	the double settlement (I2)	2
3	verify: policy ∧ receipt	the hollow "verified" (I3)	4
4	delegate: scope ⊆ parent	the escalated delegation (I4)	1
5	settle: owner-funded ∧ verified ∧ receipted	the wrong-principal payout (I6)	7

The checker that explores the machine is short enough to run while reading. Its output below is the evidence behind the state count and the shortest-crime table: with every guard on, no reachable state violates an invariant; with any one guard off, the checker finds the violating run and prints it.

```
>_
$ python3 skills/harbor-results/scripts/c0_workunit.py
BASELINE (all guards on): 536 reachable states
   ↪ explored.
  violations: NONE -- all six invariants hold in
   ↪ every state [ok]

MUTATION SUITE -- disable each guard; the checker
   ↪ must find a violating trace:
  guard OFF [epoch] -> caught I1/I5: effect admitted
   ↪ with stale epoch
    shortest trace (4 steps): propose->authorize ->
   ↪ start_exec
                              -> reassign_role ->
   ↪ admit_effect(STALE)
  guard OFF [idem] -> caught I2: idempotency key
   ↪ settled twice
    shortest trace (2 steps): ext_op(k1) ->
   ↪ ext_op(k1)
  guard OFF [verify] -> caught I3: verified without
   ↪ policy+receipt
    shortest trace (4 steps): propose->authorize ->
   ↪ start_exec
                              -> witness -> verify
  guard OFF [attenuate] -> caught I4: child grant
   ↪ exceeds parent scope
    shortest trace (1 step): delegate(ESCALATED)
  guard OFF [settle] -> caught I6: settlement
   ↪ unfunded/unreceipted/wrong principal
    shortest trace (7 steps): propose->authorize ->
   ↪ start_exec -> set_policy
                              ->
   ↪ admit_verifier_receipt -> witness
                              -> verify ->
   ↪ settle(p2!)
```

Listing 1.4. the work-unit machine: 536 states, every guard essential

```
Guards restored: 536 states, violations: none [ok]
```

Model-Checked Property 1.10.1 (Six Invariants of the Work-Unit Machine).

State. A tuple of the work-unit phase (proposed, authorized, executing, witnessed, verified, contested), the acceptance policy and verifier receipt flags, the role epoch with its current holder and any stale token retained by a previous holder, the grants with their parentage, the effect journal, the idempotency journal, and the settlement record.

Transitions. Propose, authorize, start, admit-effect (guarded on the current epoch), reassign-role (epoch increments; the old holder keeps a stale token), delegate (guarded on scope $\subseteq$ parent), external-op (guarded at-most-once per key), witness, admit-receipt, verify (guarded on policy $\wedge$ receipt), contest, settle (guarded on owner-funded $\wedge$ verified $\wedge$ receipted).

Invariants. I1, every admitted effect carried the current role epoch at admit time;

I2, each idempotency key settles at most once;

I3, a verified completion has an acceptance policy and an admitted verifier receipt;

I4, no delegated grant is more permissive than its immediate parent;

I5, role reassignment invalidates stale epochs at the effect boundary;

I6, every settlement is funded by the owning principal and references a receipt.

Result. On the two-principal instance (one work unit, one exclusive role with fencing epochs, a root grant with one delegation, one idempotency key, at most two effects), all 536 reachable states satisfy all six invariants; disabling any single guard yields a violation, found automatically as a shortest counterexample trace, and restoring the guard restores safety [internal, `c0_workunit.py`].

Exhaustive here means every reachable state of this bounded specification, not every execution of the implementation. The

check establishes safety in that model; it does not establish liveness, runtime refinement, or confinement of effects outside the declared interface.

I2's guarantee is what the literature calls **effectively-once** processing: a deduplication discipline enforced at the boundary (an idempotency key plus a unique receipt row), not a promise the network itself keeps, since true exactly-once delivery across an unreliable channel is impossible.

Pat Helland. Life beyond Distributed Transactions: an Apostate's Opinion. In *Conference on Innovative Data Systems Research (CIDR)*, 2007.

Proof idea. There is no proof to read; there is a search to rerun. The checker performs a breadth-first enumeration of the reachable states from the initial state, evaluates the six predicates in each, and stops at the first violation. The mutation suite then disables one guard at a time and reports the shortest violating trace it finds. A checker that has never caught a seeded bug is an unvalidated checker; the five traces below are the evidence that this one has teeth.

Numbers by Hand 1.10.2 (The Crimes, and Their Shortest Scripts).

Table 1.13 lists the five guards and the shortest trace the checker finds when each is disabled [internal, `c0_workunit.py`]. Two of them you can reconstruct by hand.

The stale write in four steps: authorize the work unit; start execution; reassign the role, so the epoch becomes 1 and the old holder retains token 0; now the old holder admits an effect with token 0. With the epoch guard on, that last transition does not exist, so the state is unreachable; with the guard off, the effect lands carrying yesterday's epoch and I1 fails. This is exactly the "paused holder wakes after expiry" scenario the fenced-lease theorem (Theorem 1.5.1) exists to exclude, and it is the shortest possible: no shorter history contains both a reassignment and a subsequent effect.

The double settlement in two steps: run the external operation under key k_1, then run it again. With the idempotency guard off the second run settles the same key twice and I2 fails; no history of length one can settle anything twice.

Now you try: the escalated delegation takes one step. Which transition is it, and which invariant does it break? (Exercise 1.25.)

Table 1.13. Disable any guard and the checker finds a shortest violating trace. [internal, `c0_workunit.py`]

Guard disabled	Crime	Steps	Invariant broken
epoch check at admit-effect	the stale write	4	I1, I5
at-most-once per idempotency key	the double settlement	2	I2
policy ∧ receipt at verify	the hollow “verified”	4	I3
scope ⊆ parent at delegate	the escalated delegation	1	I4
owner-funded ∧ verified ∧ receipted at settle	the wrong-principal payout	7	I6

The five-guard table’s seven-step payout trace deserves a sentence. To reach the wrong-principal payout the checker had to walk the entire legitimate path first: set the policy, admit the receipt, witness, verify, and only then settle from the wrong purse. That is exactly how the fraud would look in production, which is the point of asking a machine rather than an author to find it.

What it buys. Every earlier theorem in this chapter now has a substrate to be a property *of*. Table 1.12 states the kernel’s invariants as claims about subsystems; the work unit is where related obligations meet. The machine’s I1 and I5 concern current epochs; I2 concerns at-most-once settlement per key; I3 requires both an acceptance policy and a verifier receipt; I4 checks delegation narrowing; and I6 binds settlement to its owner and receipt. These six labels are local to this model and are not the I1–I12 labels of Table 1.12. The model illustrates how the mechanisms compose; it does not prove all properties of those subsystems. The continuity chapter’s checkpoint and the market chapters’ settlement inherit the same object, which is why the book can name the work unit as its primitive in the front matter without a definition per chapter.

Where this stops

Bounded model checking on a small instance proves the specification at these parameters, not the deployed daemon: deployment must *refine* this machine, mapping every real code path to a transition, and that mapping is a separate obligation.

Safety only: liveness ("the work eventually settles") needs fairness assumptions this check does not make.

And an invariant list is only as good as its coverage; the mutation suite certifies that these six have teeth, not that they are complete.

The unbounded-parameter version, in TLA$^+$ with Apalache, is planned and not done.

Recall

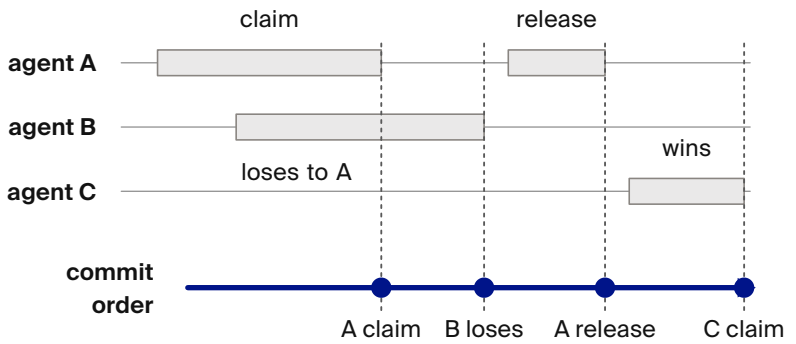
1. Name the three work-unit journals and their invariants.
2. Why does wrong-principal payout take seven steps?
3. What do mutation tests establish—and leave untested?

Exercises 1.25–1.26, p. 152.

1.10.2 The consistency-model theorem

The strongest words a single-writer design earns are "serializable" and "linearizable." They are the formal payoff of the architecture, and they are what make the coordination layer's typed ownership trustworthy.

Figure 1.22 makes the two guarantees concrete on one history; the five assumptions that buy them are the theorem's own hypotheses.



$$A.claim \prec B.claim \text{ (loses)} \prec A.release \prec C.claim$$

Figure 1.22. Overlapping requests receive one commit order: A claims, B loses, A releases, C claims. Theorem 1.10.3 states the five assumptions this ordering requires. [internal]

Theorem 1.10.3 (Conditional consistency model)**.**

Assume the single-writer discipline holds without violation (Def. 1.3.1), transactions never enable `read_uncommitted`*, each successful response is sent only after commit, no out-of-band writer*

mutates the tables, and the observed execution is crash-free. Each claim/lock operation's state checks and mutation belong to that same transaction.

Then committed transactions admit a serial order, and the externally observed claim/lock operations are linearizable with commit as the linearization point.

Proof sketch.

Under the stated configuration there is one stream of write transactions, totally ordered by commit, and reads observe a consistent snapshot. For linearizability (Herlihy & Wing) we need a single point per operation, between its invocation and response, at which it appears to take effect, with these points consistent with real time.

Maurice P. Herlihy and Jeannette M. Wing. Linearizability: A Correctness Condition for Concurrent Objects. *ACM Transactions on Programming Languages and Systems*, 12(3):463–492, 1990.

The commit of each claim/lock operation is exactly such a point: it is atomic (Theorem 1.5.1), it lies between the client's request and the daemon's response.

Non-overlapping operations respect real-time order because a later invocation occurs after the earlier response and therefore after its commit; overlapping operations may be ordered by commit. Hence the observable history is linearizable. □

Where this stops

Linearizability preserves the order of non-overlapping operations, not the invocation order of concurrent ones. It needs no synchronized wall clocks: invocation, commit, and response delimit the operation in the history. Serializing commits therefore does not automatically downgrade this claim to sequential consistency.

Crash recovery is a different obligation. Under WAL with `synchronous=NORMAL`, power loss can erase a commit the client has already been told succeeded (Property 1.4.1). Deleting that acknowledged operation from the history is not a valid repair of the proof.

Joseph Izraelevitz, Hammurabi Mendes, and Michael L. Scott. Linearizability of Persistent Memory Objects under a Full-System-Crash Failure Model. In *Proceedings of the 30th International Symposium on Distributed Computing (DISC)*, pages 313–327, 2016.

A durable-linearizability claim must preserve completed operations across recovery and handle pending operations consistently.

`synchronous=FULL` supplies a per-commit WAL sync, assuming the storage stack honors it; it does not by itself prove the application's recovery protocol, identity epochs, or all response paths correct.

> This theorem covers crash-free histories only. OP-10 remains *open*, and no end-to-end durable-linearizability proof is claimed for the deployed kernel.

Bypassing the daemon breaks ordering

This is why the kernel needs no agreement protocol inside one fault domain. Linearizability follows from the routing, transaction, and response-order conditions above; it is lost if a second writer or an out-of-band read bypasses those conditions. One decider, one order, one auditable truth.

1.10.3 The dual-runtime hazard and runtime conformance

Invariant I11 (runtime parity) is the one place the formal spine is held together by a compatibility shim rather than a proof. In deployment, truth is *computed* under the SQLite bindings of one runtime (the compiled single-binary daemon), but in the test suite it is *verified* under a different runtime's bindings; a thin shim normalizes their pragma and option interfaces . An invariant that is green under test can fail in production if the two bindings diverge in lock or pragma semantics — the classic "green in the test runtime, broken in the deployment runtime" failure. A continuous-integration pipeline must therefore *boot the deployment runtime* and exercise its endpoints, not merely build it.

One way to gather conformance evidence for I11 is **differential fuzzing** (specified, OP-3): drive a large randomized stream of concurrent operations — 10^6 is the order of magnitude the design targets — against both bindings from the same seed, and assert hash-equivalence of the resulting database state. It does not run in the reference implementation's pipeline today, and it is worth being clear about what it would and would not buy even when it does. Hash-equivalence over random operation streams is a *test*, not a proof: it would raise the cost of a divergence surviving to deployment, but a divergence that needs a specific interleaving, a specific pragma set, or a filesystem the fuzzer never exercises can pass 10^6 operations and still be there. I11 accordingly stays partial in Table 1.12, and OP-3 stays *open*.

Exercises 1.27–1.29, p. 153.

1.11 Cross-subsystem atomicity: a buildable defect, not a frontier

"Claim this port *and* open this session *and* record this commitment" is three writes. Nothing at the kernel's interface boundary wraps them in one transaction, so partial-failure semantics across subsystems are undefined. It is tempting to file this under "research"; that is wrong, and the correction matters because it changes the engineering posture from "research" to "do it."

This is the Saga problem (Garcia-Molina & Salem) — but with a *cheap local fix*. These are all local writes to one file, and the database already provides a transaction primitive (used elsewhere in the system). Wrapping a multi-subsystem operation in one local transaction (begun in immediate mode) gives all-or-nothing for free. Filing this under "research" understates how cheap the correct answer is. It is a **buildable defect**: the durable primitive is present; what is missing is the wrapping of the multi-subsystem endpoints in it.

Hector Garcia-Molina and Kenneth Salem. Sagas. In *ACM SIGMOD International Conference on Management of Data*, pages 249–259, 1987.

One transaction prevents partial updates

This is the kind of finding the maturity discipline exists to surface: a gap mislabeled as a research frontier hides a one-line fix behind a grander word. Sagas matter when the steps are remote and a single transaction is impossible; here, where every step is a local write to one file, the single transaction is strictly simpler and strictly stronger.

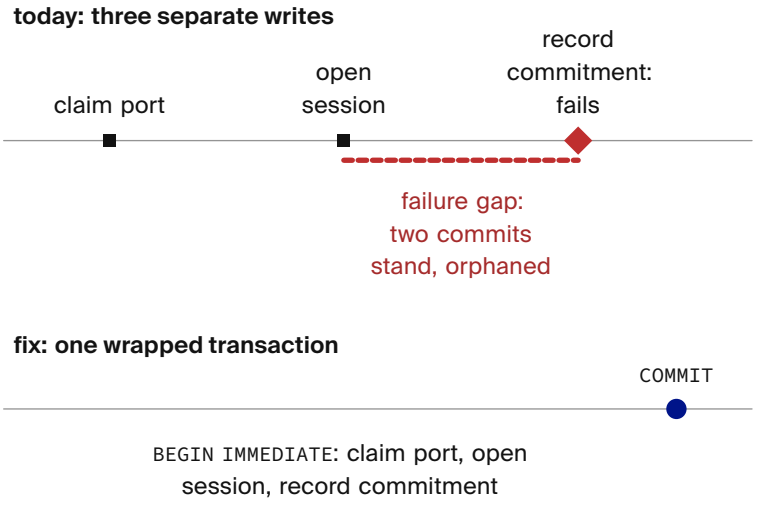


Figure 1.23. Separate commits can leave orphaned writes. One immediate-mode transaction makes the three local writes atomic; no distributed compensation protocol is needed. [internal, OP-11]

Figure 1.23 draws the same three writes twice: today's failure gap, and the fix's single commit point.

Exercises 1.30–1.31, p. 153.

1.12 Threat model and the limits of mediation

A systems-security paper lives or dies on its threat model. The kernel deliberately shrinks its trusted computing base (one writer, one file), which is good security hygiene — but the shrinking pushes several adversaries explicitly out of scope, and the honest move is to draw the boundary, not gesture at it.

1.12.1 Advisory claims are not sandboxing

The sharpest category error to avoid: edit-surface claims are *advisory coordination locks between cooperating agents*, not OS-enforced filesystem isolation. There is no syscall filter, no `pledge`/`unveil`, no macOS Seatbelt, no Landlock. A malicious agent ignores a claim row and writes the file. The kernel provides *advisory* coordination; real OS sandboxing of agent processes is out of scope at this layer (it belongs to the out-of-band "jail" — the separate-user kernel boundary of §1.12.2 — which is *proposed* and would need to be grounded in a real operating-system primitive). We say so plainly rather than let advisory claims be read as isolation.

1.12.2 The enforcement gate proves; it does not yet compel

The same boundary problem reappears one level up, at the kernel's *enforcement* ambition, and it is the paper's most consequential "*specified*, not *implemented*" caveat. The kernel ships a real cryptographic gate — a **macaroon** discharge primitive (a bearer credential whose authority can only ever *narrow*, since each caveat is folded into a chained keyed hash so that tampering breaks the signature). Its reference implementation is `core/kernel/pd-anchor/src/macaroon.rs`: the root key never leaves the daemon,

Birgisson et al. 2014. *Macaroons: Cookies with Contextual Caveats for Decentralized Authorization in the Cloud*.

Table 1.14. The substrate-layer threat model. The same-user adversary is *out of scope* by design; several "security" subsystems become necessary only against the cross-machine adversary, which belongs to the economy layer.

Adversary	In/out of scope	What defends (or does not)
Buggy cooperating agent	**in**	Claims, commitments, post-commit monitoring, oracle-bound closure. The primary design target.
Misbehaving agent (ignores advisory claims)	partial	Edit-surface claims are *advisory coordination*, not OS-enforced isolation. A malicious agent can write the file anyway (no kernel-level sandbox such as Landlock, `unveil`, or Seatbelt).
Same-user process (reads/edits the DB)	**out**	The database is owner-read/write only; OS-keychain isolation of the signing key remains incomplete. Hash-chain tamper-evidence defends against *nobody* under this exclusion (§1.12.4).
Different-uid process	**in (partial)**	Owner-only permissions on the database and both socket files; but the socket peer-credential check is a software handshake (a self-reported agent identity checked against the registration table), *not* the kernel-enforced socket credential (`SO_PEERCRED`) — a real authentication caveat. This row is the document's operative statement: no other section supersedes it (§1.12.4 specifies hardening that would, but does not ship).
Cross-machine / cross-operator	**out (economy layer)**	Needs the complete local identity contract (partial) and cross-operator attestation (the economy paper).

the third-party "rent-paid" caveat is an HMAC commitment to a discharge the daemon alone can issue, and the discharge construction is machine-checked in a protocol-verification tool (`analyses/macaroon_discharge_v1.pv` proves no push is authorized without a daemon-issued discharge bound to *that* grant). The cryptography is **implemented** and sound *as cryptography*.

An unforgeable gate can be bypassed

A gate that is cryptographically unforgeable is not, for that reason, *compulsory*. The macaroon proves that a given push was authorized; it does not yet *stop* an unauthorized one, because the agent runs as the same user and holds its own host credentials.

What it does *not* yet do is bind that gate to the agent's actual capability to act. Today's coordination enforcement is the **Coordination Guard** (`cli/commands/guard.ts`) and a **git shim** (`cli/utils/git-shim.ts`) that intercept destructive verbs — and both run *in-band*, as the same user, through a `PATH` the agent controls. By construction that arrangement is advisory: an agent that hits the wall has at least five escapes that defeat no cryptography (a documented bypass environment variable, an absolute path to the real `git`, a `PATH` rewrite, in-process `libgit2`/editor bindings, or simply never attaching a session). The enforcement limit is the shared privilege boundary: the agent can replace or bypass the code intended to constrain it. Likewise, a secret readable by that process can be copied; cryptographic authentication does not remove its access to the secret. A guard the process can decline to invoke is advisory.

The third instrument is a proposed **hook harness** (specified). It would run at lifecycle events a vendor's agent loop exposes (the prompt submission, the moments before and after a tool call, the compaction, the stop — under whichever names the vendor happens to give them), and at each event it could inject bounded advisory text into the model's context, block the act, or charge it against a budget. The proposed policy ranges from advice through warnings to enforcement, with a one-second deadline per hook and a breaker that disables the harness when hooks start failing. This is a design for steering cooperative agents, not a confinement guarantee: a same-user agent can disable the hook, or simply reach the effect directly. Guard, shim, and harness are therefore one thing seen three times: the instruments of the Observed and Coordinated modes, valuable for the transcript they leave and for the cooperative agents they steer, and silent about the agent that declines them.

Closing this gap is a layered program, and only the top layer is a real wall. A server-side branch-protection-plus-push-broker layer (specified) removes the most painful direct-to-protected-ref incidents but not same-user trickery; a credential broker that meters and injects credentials per request (*proposed*) narrows the ambient-token surface; and only a **separate-user (or separate-VM) kernel process** with forced egress (*proposed*) actually confines a malicious same-user holder, because it is the first layer that removes the capability rather than refusing the verb. That separate-user boundary is the analogue of the hardware/operating-system boundary a classical reference monitor relies on for "always invoked," and *it does not yet exist*. Until it does, the honest grade of kernel enforcement against a non-cooperating same-user agent is *not guaranteed*: the gate is a verifiable transcript and a loud tripwire, not a wall. We state this here so no reader carries the (legitimate) "mechanically verified macaroon" result forward into the (false) belief that the kernel can presently *compel* a same-user agent through it. Figure 1.24 separates optional pre-effect refusal, the unchecked same-user route, and conditional post-effect response, alongside each mechanism's source-reported maturity.

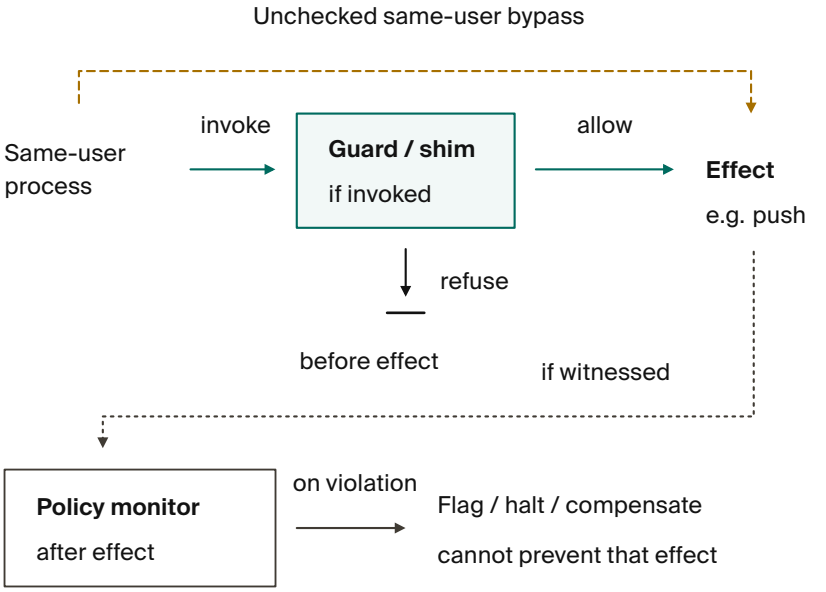


Figure 1.24. Refusal is not compulsory mediation. Grades are source-reported, not an independent runtime audit.

No promise that every bypass is witnessed.

Mechanism	Source grade
Guard / git shim	**implemented**
Hook harness	specified
Branch protection + push broker	specified
Credential broker	*proposed*
OS/VM isolation + protected state **and** forced mediation	*proposed*
Macaroon cryptography	**implemented**

Authorization proof, not compulsory use.

That separate boundary is therefore the substrate this chapter prescribes, rather than one more option on the menu of layers. The guard and the shim are the interim instruments of an observed mode — kept because they leave a transcript, which is worth having — and the book does not build its argument on them. What the boundary does *not* settle is whether the single-writer rail is also the right way for several agents to share a repository, as against a worktree per agent with a merge queue. That is an empirical question (an honest one; the authors do not know the answer), and a preregistered study outside the product (`studies/substrate-study`) is

running it. Until it reports, the rail is a design invariant for confinement and a hypothesis for collaboration.

1.12.3 Partial actor-soul identity bounds every other invariant

The reference system now mints an opaque actor-soul id and lookup credential inside the daemon, and the budget guard assigns fresh actors to a shared bounded newcomer pool (I12 is partial). Legacy session paths and security-relevant writes that do not yet require that credential remain impersonable. This still bounds the security weight of I7, I8, and I9: lock-owner validity, capability escalation, note monotonicity, and owner-scoped release are only as strong as their least-protected identity path. This dependency — complete I12 enforcement gates the soundness of I7, I8, and I9 — is a hard precondition, stated here so the reader does not over-trust the enforcement subsystem. It is also the central reason the cross-machine layer, where identities genuinely conflict, must make identity cryptographic. Figure 1.25 draws the one-root, three-dependent shape of that precondition.

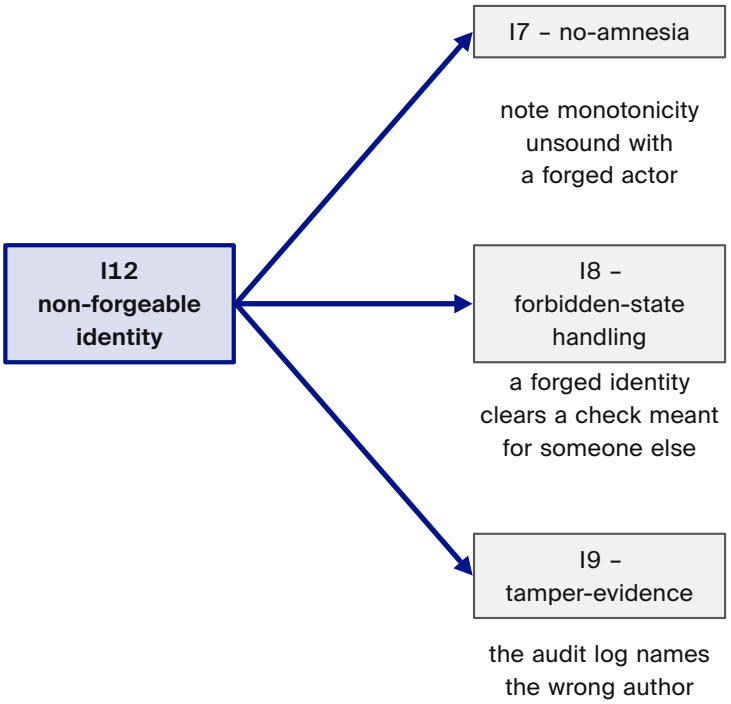


Figure 1.25. I7–I9 depend on authenticated identity at every write path. A legacy path that accepts self-asserted identity can bypass otherwise correct owner checks; I12 remains partial. [internal]

1.12.4 Hash-chain tamper-evidence, and the hardening OP-9 would need

The hash chain (I9) is built, but under the initial threat model it defended against nobody: its only adversary was someone who could edit the audit log, which was exactly the same-user process the model excluded. Tamper-evidence that nothing on the read path verifies is a data structure, not a control. We therefore re-scope I9 as **cross-machine provisioning**: it matters only against cross-machine synchronization or a tamperer who is not the same user.

The design that would let the kernel defend against the same-machine adversary directly — and so close OP-9 — is **OS-level ephemeral namespaces** (specified). The daemon keeps its database and sockets owner-only (`0600`), and spawns each agent inside a `bwrap` or `unshare` sandbox under an ephemeral UID of its own. The agent then ceases to be a same-user process, which is the only move that changes the answer: the filesystem bypasses of §1.12 stop being available because the capability is removed, not because a verb was refused. Paired with it, the daemon reads the *kernel-verified peer credential* of each connection — on Linux the `struct ucred` (pid, uid, gid) that the kernel records at `connect(2)` and `SO_PEERCRED` returns via `getsockopt(2)`; on macOS the equivalent `LOCAL_PEERCRED` — and binds each request to the ephemeral UID of the spawn instance it must have come from.

Three things about that paragraph determine the result, and none of them is a claim that OP-9 is closed.

- **It is not what the kernel does today.** The reference implementation does not read the kernel-verified peer credential: it relies on the owner-only socket permission plus a self-reported agent identity checked against the registration table, which is precisely the software handshake Table 1.14 records. Nor does it spawn agents into ephemeral-UID sandboxes. The threat-model row, not this section, states the kernel's current authentication posture.
- **The peer credential is not a cryptographic binding.** It is a kernel-recorded uid/gid/pid triple attached to a socket connection — strong precisely because the kernel, not the peer, supplies it, and *weak* in the way any identity is when the adversary can obtain the same uid. Nothing about it is signed, and calling it a

cryptographic bind confuses the guarantee with the authorization chain's, which is a different mechanism (§1.6).

- **Even fully built, it would not make bypasses "impossible."** It would move the same-machine adversary from the same-uid row of Table 1.14 to the different-uid row — a real and substantial improvement — while leaving shared kernel surfaces, whatever paths the sandbox binds in, and the sandbox implementation itself in scope. "Impossible" is the wrong word for a containment boundary; "the capability is removed for this named class of access" is the right one.

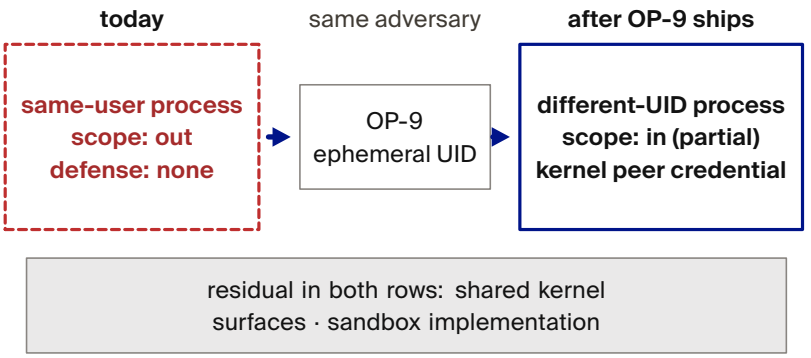


Figure 1.26. One threat-table row moves through the OP-9 reclassification. The move adds partial kernel-credential defense but leaves shared-kernel and sandbox surfaces in scope; it is not proof of impossibility. [internal]

Figure 1.26 draws that reclassification as the one thing it is: a move from one threat-table row to another, not a new defense. OP-9 therefore stands *open*: a specified hardening path, not a shipped defense, and the honest grade against a same-machine adversary remains the one §1.12.2 gives — *not guaranteed* until a separate-user boundary exists.

1.12.5 Where information-flow control goes, and why it is not here

One question this section invites belongs to a different chapter, and the cleanest service we can do the reader is to say so rather than answer it badly. Capability scoping alone does not prevent exfiltration: an agent holding both `fs:read:/confidential` and `net:egress:github.com` can read the first and post it to the second, and no amount of claim discipline notices. The containment answer is information-flow control — taint the read, propagate the taint

through spawns and shared writes, and gate every release channel behind a declassification policy with a leakage budget. Theorem 1.7.3 already gives the substrate-layer form of it in one line: gate the channel, never the token.

The full construction — a compute-to-data airlock in which one operator's agent stack executes over another operator's confidential data, with neither inspecting the other — is *cross-operator* by definition: two principals, mutual distrust, a licensing relationship, and a bond that can be slashed. Every one of those objects lives above this floor, in the chapter that owns the market between operators (*The Harbor Economy*), and this chapter's abstract promises it lives there rather than here. What the substrate contributes to it is what this chapter has already specified: taint is a persistent security label, not an evaporating coordination marker (§1.6); forgetting it through time decay would reopen the forbidden egress path. In the stated model, egress and file writes are controllable events (Theorem 1.7.3), the sandbox that would make an agent a different-uid process is OP-9's specified hardening (§1.12.4), and the honest statement of what containment buys is the one this section keeps repeating: not that exfiltration becomes impossible — timing, ordering, and side channels stay out of model — but that every flow out passes a named gate whose releases are logged, budgeted, and attributable.

Exercises 1.32–1.34, p. 153.

1.13 Adjacency contract: what the kernel assumes and provides

The kernel's coherence with the layers around it is a contract: what it assumes from the machine below, what it provides to the protocol above, and — just as important — what it explicitly does *not* provide so higher layers do not over-assume.

1.13.1 What the substrate assumes from the machine

- A POSIX filesystem with working advisory locking (`flock`/`fcntl`). **This is a correctness precondition for I2**: networked filesystems break advisory locks, and a broken lock lets two writers both win — destroying mutual exclusion, not just performance.
- A local wall clock. Single-machine, so no shared-clock assumption is needed — but note that a wall clock is *not monotonic* (it steps when the clock is adjusted, e.g. by network time synchronization), an intra-node hazard for every expiry sweep that the inter-node ordering of Lamport does not cover.
- Unix domain sockets, with owner-only permissions on the socket paths. Both transports authenticate a peer by software handshake over that permission model, *not* by the kernel-verified socket credential (§1.12); reading that credential is OP-9's specified hardening, not an assumption the kernel currently makes.
- An OS keychain for signing keys; file permissions honored on the database path.
- A supervisor process to keep the daemon always-on and to restart *the daemon itself* — the substrate makes other things always-on but cannot make *itself* always-on; that is delegated downward.



Lamport showed that happened-before, not a shared clock, is the ordering a distributed system can trust; the daemon's wall clock does not give it for free.

Leslie Lamport. Time, Clocks, and the Ordering of Events in a Distributed System. *Communications of the ACM*, 21(7):558–565, 1978.

1.13.2 What the substrate provides to the coordination layer

- **Atomic, idempotent, time-to-live'd claims** (I2–I4) over ports, file-regions, symbols, and named locks — the coordination layer's typed ownership reduces to claim and release.
- **A durable, versioned, history-cursored publish/subscribe bus**, a tuple space, and decaying stigmergic markers — the coordination layer's speech act is carried *inside* the bus envelope; the anti-blackboard-rot property is *provided* by substrate-side decay.

- **The deontic split as a primitive** (Def. 1.7.1): prohibition → monitor (detect/regiment), obligation → commitment, permission → capability. The coordination layer must not re-implement enforcement.
- **Daemon-owned deadlines and oracle-bound closure** (I5, I6).
- **An append-only audit log** (hash-chain-provisioned) that the protocol and the operator read.
- **Self-attestation** (I10) — higher layers may *assume the kernel is honest about its own health*.
- **The cryptographic authorization chain** — the coordination layer's coordination lineage rides inside it.
- **Linearizable claim/lock semantics** (Theorem 1.10.3) — the property that lets the coordination layer treat "who holds this" as ground truth.

1.13.3 The explicit non-provisions

- The substrate does *not* provide end-to-end non-forgeable identity yet. I12 is partial: daemon-minted actor souls, lookup credentials, and a bounded newcomer pool enforced by the budget guard ship; universal write-boundary enforcement does not.
- It does *not* provide fair/queued exclusion (OP-1), cross-subsystem transactional atomicity by default (§1.11), a real execution-checkpoint (OP-4), power-loss durability on any write path (I1b, Property 1.4.1; the per-path selector of OP-10 is specified, not shipped), a kernel-verified peer credential on either transport (OP-9), or any cross-machine guarantee (the economy layer — different theorems, a shared-clock and Byzantine-membership problem the substrate deliberately never touches). The status of each open problem is stated once in Table 1.15; a higher layer should assume nothing stronger than that table.
- It does *not* decide *what to do* (no orchestration). It enforces what *cannot* be done and records what *did* happen — a regulator, not a planner.

Review of the key ideas

What this chapter leaves coupled, in the order the rest of the book will lean on it:

1. One writer, one file. Every mutation reaches the daemon and is committed in one order; the reply follows the commit. Serializability and linearizability are bought by that discipline plus four named assumptions, not by an agreement protocol.
2. Durability is split by fault class. A process crash loses nothing; a power loss can lose the last commits under the default synchronous setting. The setting is the durability claim.
3. A claim is a lease with a fencing token (this chapter's epoch). At most one compatible holder per conflict domain; expiry is lazy; a stale holder's effects are refused downstream by the epoch, not by the lock.
4. The bus is at-least-once and says so. Benign redelivery is deduplicated by an idempotency key the coordination layer still owes; only protocol violations are loud.
5. Two chains, never one name. The authorization chain proves who authorized whom; the coordination lineage says what work is in flight. The second rides inside the first.
6. Prohibitions, obligations and permissions need three mechanisms. A gate refuses before the effect; a typed oracle closes an obligation after the fact; a grant simply holds.
7. A post-commit monitor cannot prevent the transition it observes. Exact realization of a safety policy requires controllability and, under partial observation, observability. Safe restriction can exclude some otherwise permitted work; detection and compensation require separate evidence and remedies.
8. Done means re-checkable. A commitment closes only through a reference from the finite oracle vocabulary; free text bounces back to open.
9. Continuity narrows downstream. Memory survives a crash; checkpoint keeps notes, not execution state; the outcome ledger is partial. Nothing built on top can be more certain than that.

10. The kernel reports its own coverage. Enforced, degraded or stubbed, per rule, so the layers above never mistake a stub for a guarantee.

1.14 Exercises

§1.2 The kernel as seven subsystems

1.1 Check ⋆ *Solution p. 715*

Which of the seven subsystems would still be meaningful if the daemon ran over an in-memory database with no disk at all? Which become vacuous?

1.2 Trace ⋆⋆ *Solution p. 715*

Pick any one subsystem and name the single SQLite table that is its "home" table. Then name one invariant that subsystem owes the layer above it.

1.3 Open ⋆⋆⋆ *Solution p. 715*

The seven-subsystem decomposition is a *choice*. Is there a more natural cut, five subsystems, or nine? Argue for an alternative and show what it makes easier to reason about and what it hides.

§1.3 The substrate subsystem: one writer, one file

1.4 Check ⋆ *Solution p. 715*

The schema-by-union design lets any module create its tables in any order. Give a concrete two-module example where a lazy column-rename migration would be unsafe under this design.

1.5 Trace ⋆⋆ *Solution p. 716*

Follow a single claim request from command line to disk. At how many points does the single-writer discipline (Definition 1.3.1) hold by *routing* versus by the *file lock*?

1.6 Open ⋆⋆⋆ *Solution p. 716*

The boot ledger orders migrations, but modules still self-initialize, so the order is not yet authoritative (OP-7). What is the smallest change that makes it so, a `user_version` sequencer over *declared* migrations, and can any useful remnant of "any module, any order" survive it for renames, backfills, and down-migrations?

§1.4 Durability, split by fault class

Solution p. 716

1.7 Check ⋆

Classify each fault as I1a-survivable or I1b-exposed: (a) a forced kill of the daemon process; (b) a clean operating-system reboot; (c) yanking the power cord; (d) an unhandled exception in a request handler.

Solution p. 716

1.8 Trace ⋆⋆

A claim commits at t_0; the next auto-checkpoint fires at $t_0 + \delta$. Draw the window during which a power loss can revert the claim. Does a page-count auto-checkpoint threshold bound δ in wall-clock time?

Solution p. 717

1.9 Open ⋆⋆⋆

Section 1.4 sketches typed durability selection (OP-10), but nothing in the kernel opts into it. What is the cleanest interface that lets a settlement-ledger write demand I1b while keeping claim writes fast, and, harder, what stops a *future* write path from silently defaulting into the fast lane when it should have opted in?

§1.5 The resource subsystem: claims, locks, and fair exclusion

Solution p. 717

1.10 Check ⋆

Why does releasing a lock you do not hold return success rather than an error? Which property (I2 to I4) is this, and what would break if it raised?

Solution p. 718

1.11 Trace ⋆⋆

Two agents call `acquire` on the same fresh key at the same instant. Walk the uniqueness-constraint insert for both (Listing 1.1) and show exactly where the loser learns the holder's identity.

Solution p. 718

1.12 Open ⋆⋆⋆

The Stigmergic Ticket-Lock buys bounded wait on the cooperative-release path only (OP-1). Extend it so that a lease reclaimed by the lazy TTL sweep also honors the ticket queue, still without a background scheduler, or argue that no reactive-only design can, and say what the minimum scheduler would be.

§1.6 The communication subsystem: the bus, stigmergy, and two delegation chains

1.13 Check ⋆ *Solution p. 719*

Why does read-time marker decay make the system more honest than a background tick alone? Give a scenario where tick-only decay would report a stale signal as fresh.

1.14 Trace ⋆⋆ *Solution p. 719*

A request speech act is sent over the bus and redelivered once by the at-least-once channel. Trace what the operator should see under the silent-dedup resolution, and what an envelope idempotency key would change.

1.15 Open ⋆⋆⋆ *Solution p. 719*

What per-kind marker decay rate keeps stigmergic signals neither rotten nor prematurely evaporated (OP-8)? Frame this as an empirical measurement, not a constant to guess.

§1.7 The obligation & enforcement subsystem: the sovereign's two levers

1.16 Check ⋆ *Solution p. 719*

Consider note-count monotonicity, capability attenuation, owner-checked release, and heartbeat freshness. For each rule, use Theorem 1.7.1 to say whether it *could in principle* be moved to a pre-commit check (a before-insert trigger). Which genuinely cannot, and why? (Hint: heartbeat freshness is a failure detector, and Chandra and Toueg show failure detection cannot be made perfectly synchronous.)

Chandra & Toueg. *Unreliable Failure Detectors for Reliable Distributed Systems*. J. ACM 43(2):225–267, 1996.

1.17 Trace ⋆⋆

An agent attempts to close a commitment with an empty oracle reference. Walk the Law-2 gate (Listing 1.3) and show exactly where and why the transition is refused.

Solution p. 720

1.18 Trace ⋆⋆ *Solution p. 720*

Theorem 1.7.3 settles which monitor rules are regimentable, so classify rather than re-prove. Take note monotonicity, capability escalation, and lock-owner validity and, for each, say whether its prohibition names a controllable event and whether its trigger is witnessed at the boundary. Which cell of Figure 1.15 does each land in?

Solution p. 720

1.19 Check ⋆

Classify "forbid `api_call` after `spawn_child`" and "forbid `internal_plan` after `fs_write`" using Theorem 1.7.3.

Solution p. 720

1.20 Trace ⋆⋆

Add P_4, "no `spawn_child` after `git_push`," to the sandboxed review job and write the new disablement map by hand.

Solution p. 721

1.21 Open ⋆⋆⋆

Extend the Law-2 oracle vocabulary to capture a "done" condition it currently misses (OP-5), without re-admitting free text. State your new oracle and argue it is Goodhart-resistant, for one of the two specified kinds (delta oracle, AST assertion) before inventing a third, since neither yet survives that argument.

§1.8 The continuity subsystem: memory, checkpoint, and the foundation of the economy

Solution p. 721

1.22 Check ⋆

Why is "recovery passes notes" fundamentally weaker than "recovery restores work" for a language-model agent that died mid-edit? Name one thing notes preserve and one thing they cannot.

Solution p. 721

1.23 Trace ⋆⋆

Follow the note-monotonicity rule from an agent appending a note to the monitor detecting a regression. At which point is the violation already durable (per Theorem 1.7.1)?

Solution p. 721

1.24 Open ⋆⋆⋆

What is the minimal durable artifact that turns "recovery passes notes" into "recovery restores work" (OP-4)? Is it a content-addressed snapshot of the working-tree diff, the open claims, the commitment set, and the last N turns of the agent's transcript? What is recoverable versus fundamentally lost when a language-model agent dies mid-thought?

§1.10 The invariants, stated as theorems

Solution p. 722

1.25 Check ⋆

The escalated delegation takes one step (Table 1.13). Which transition is it, and which invariant does it break?

1.26 Open ⋆⋆⋆ *Solution p. 722*

Propose a seventh invariant that the six of Theorem 1.10.1 do not imply, add the guard that enforces it to `c0_workunit.py`, and report the shortest crime the checker finds with your guard disabled.

1.27 Check ⋆ *Solution p. 722*

List the routing, transaction, response, and failure assumptions of Theorem 1.10.3. Which one fails first if an out-of-band SQLite writer is introduced? Does the theorem require synchronized clocks?

1.28 Trace ⋆⋆ *Solution p. 722*

Construct a three-operation claim history and give a valid linearization. If two calls overlap, show why more than one linearization may still respect real time.

1.29 Open ⋆⋆⋆ *Solution p. 722*

Specify the minimal differential-test suite that turns I11 from an assumption into conformance evidence (OP-3): what operation sequences, what observable-state assertions, run against both runtimes? Then attack your own answer: name a class of binding divergence that a seeded 10^6-operation hash-equivalence fuzz would *not* catch, and say what would.

§1.11 Cross-subsystem atomicity: a buildable defect, not a frontier

1.30 Check ⋆ *Solution p. 723*

Give a concrete partial failure: port claimed, session opened, then the commitment write fails. What inconsistent state is left, and which agent sees it?

1.31 Open ⋆⋆⋆ *Solution p. 723*

Sketch the interface change that wraps the three-subsystem "begin" operation in one local transaction (OP-11). What is the compensating action if you instead chose Sagas, and why is the transaction strictly simpler here? Garcia-Molina & Salem 1987

§1.12 Threat model and the limits of mediation

1.32 Check ⋆ *Solution p. 723*

The hash chain defends against nobody in scope. Name the *out-of-scope* adversary it does defend against, and the layer (substrate or economy) where that adversary becomes real.

Solution p. 723

1.33 Trace ★★

An agent forges its actor identity to release another agent's lock. Walk the lock-owner-validity rule and show exactly where I12's absence lets the forgery through.

Solution p. 723

1.34 Open ★★★

What is the minimal hardening that closes the same-machine adversary without a hardware trusted-platform-module dependency (OP-9): an OS keychain for the signing key, sealed memory, the ephemeral-UID sandbox and kernel-verified peer credential specified above? Where does each stop? After the sandbox lands, which row of Table 1.14 does the adversary move to, and what is still in scope there?

1.15 Related work

The kernel sits at the confluence of four literatures. We position it against each, and against the design space it deliberately declines.

1.15.1 Reference monitors and the trusted computing base

Anderson 1972

Lampson 1974

Saltzer & Schroeder 1975

Schneider 2000

The organizing idea is the **reference monitor** of Anderson's 1972 security study: a mediator that is *always invoked* (complete mediation), *tamper-resistant*, and *small enough to be verified*. Lampson's *Protection* gives the access-matrix model the monitor enforces, and Saltzer and Schroeder's design principles — economy of mechanism, fail-safe defaults, complete mediation — are the yardstick we hold the kernel to. Where a classical security kernel mediates memory and CPU access for an operating system, ours mediates *coordination* state for a set of cooperating processes, and it achieves complete mediation not by interposing on every system call but by the cheaper expedient of routing every mutation through a single writer. Schneider's theory of enforceable security policies via execution monitors supplies the boundary that the central correction of this chapter rests on (Theorem 1.7.1): an execution monitor can enforce only the safety properties it can prevent by truncating execution, which a post-commit observer cannot do. This is a narrower, cheaper problem than general operating-system confinement, and honesty requires saying where the narrower problem

ends. The same-machine adversary this chapter's own OP-9 leaves *open* (§1.12.4) is exactly the problem three current systems exist to close, each at a different price: seL4, which takes "small enough to verify" to its conclusion with a machine-checked proof of functional correctness from specification down to the binary; gVisor, a user-space kernel that interposes on every system call a guest makes rather than trusting the host's boundary to hold; and Firecracker, the microVM monitor that has isolated untrusted multi-tenant workloads in production since 2018. Any of the three would give this kernel a stronger trusted computing base than an ephemeral-UID process boundary. None is adopted here, and the gap is named rather than closed — the single writer buys complete mediation for coordination state, not for the machine.

Klein et al. *seL4: Formal Verification of an OS Kernel*. ACM SOSP, pp. 207–220, 2009.

Young et al. 2019. *The True Cost of Containing: A gVisor Case Study*.

Agache et al. *Firecracker: Lightweight Virtualization for Serverless Applications*. USENIX NSDI, 2020.

1.15.2 Durable storage and transaction processing

The durability analysis is grounded in Gray and Reuter's canonical treatment, which separates atomicity and consistency from durability — the distinction the I1a/I1b split makes operational. The write-ahead-logging discipline descends from ARIES; the specific crash semantics we inherit are SQLite's WAL, whose own documentation states that the *normal* synchronization level may lose durability under power loss while preserving consistency. That caveat has an empirical backdrop: Pillai et al.'s crash-consistency study found persistence guarantees vary widely across popular Linux filesystems, and that many application update protocols carry crash vulnerabilities their own APIs do not disclose. Our architectural posture — a single writer rather than a replicated state machine — follows the single-leader default that Kleppmann argues for: reach for consensus only when the problem genuinely forces it. We make that argument formal via the consensus impossibility of Fischer, Lynch, and Paterson: there is no agreement to reach, so the impossibility never applies. Linearizability, the external guarantee we claim for claims and locks, is Herlihy and Wing's. Cross-subsystem atomicity, where we decline the distributed solution, is exactly the setting Garcia-Molina and Salem's Sagas were designed for — and exactly the setting where, because every step is local, a single transaction dominates a saga.[2]

Gray & Reuter 1992

Mohan et al. 1992. *ARIES: A Transaction Recovery Method Supporting Fine-Granularity Locking and Partial Rollbacks Using Write-Ahead Logging*.

Hipp et al. 2010

Pillai et al. *All File Systems Are Not Created Equal: On the Complexity of Crafting Crash-Consistent Applications*. USENIX OSDI, pp. 433–448, 2014.

Kleppmann 2017

Fischer et al. 1985

Herlihy & Wing 1990

Garcia-Molina & Salem 1987

[2] Engineering precedent, named because a reader who builds systems will already have thought of it and should not have to wonder whether we had. The

1.15.3 Coordination, stigmergy, and deontic control

The communication subsystem draws on two coordination traditions. Stigmergic markers with decay descend from Grassé's stigmergy and its computational realizations in ant-colony optimization; the shared associative store is Gelernter's Linda tuple space. The obligation lever models commitments as Cohen and Levesque's persistent goals — intentions an agent may rationally drop — and the prohibition lever uses the normative-systems framing of Jones and Sergot to keep prohibition, obligation, and permission distinct modalities rather than one undifferentiated "policy." The failure detector underlying heartbeat-based liveness is Chandra and Toueg's, which is why heartbeat freshness can never be a perfectly synchronous pre-commit check. The self-attestation subsystem, which denies service when its own integrity is unknown, is in the spirit of autonomic computing's self-management properties, and the bounded busy-wait under contention is the timeout-budget bulkhead that Nygard catalogs as a stability pattern.

Grassé 1959

Dorigo & Di Caro 1999

Gelernter 1985

Cohen & Levesque 1990

Jones & Sergot 1993

Chandra & Toueg 1996

Kephart & Chess. *The Vision of Autonomic Computing*. IEEE Computer 36(1):41–50, 2003.

Nygard. *Release It! Design and Deploy Production-Ready Software*. 2nd ed., Pragmatic Bookshelf, 2018.

1.15.4 Capability security and tamper-evident logs

The capability/permission distinction — *ability* versus *normative status* — is the object-capability discipline applied to coordination: an agent's having a handle to an operation does not make the operation permitted. The cryptographic authorization chain is a hop-bound, attenuation-monotonic delegation chain of digital signatures over an elliptic-curve signature scheme, of the kind whose secrecy and integrity properties are mechanically checkable in protocol-verification tools; its full treatment belongs to the economy paper. The audit log's tamper-evidence is the digital-timestamping construction of Haber and Stornetta, built on

Bernstein et al. *High-Speed High-Security Signatures*. J. Cryptographic Engineering 2(2):77–89, 2012.

Blanchet. *Modeling and Verifying Security Protocols with the Applied Pi Calculus and ProVerif*. Foundations and Trends in Privacy and Security 1(1–2):1–135, 2016.

Haber & Stornetta 1991

single-threaded serialization this chapter formalizes is the same shape as TigerBeetle's accounting loop, which runs debits and credits through one deterministic state machine precisely so that conservation is a property of the loop rather than of a lock discipline; and the WAL that Property 1.4.1 types by fault class also motivates SQLite recovery and replication tools. Litestream streams changes for backup and recovery; LiteFS replicates a SQLite database to other nodes. Those are different contracts, not two names for read replicas. These examples illustrate engineering practice, not a proof of this kernel's durability or a recommendation to deploy them without checking their current operational limits.

Merkle's hash trees — a structure that famously predates, and is independent of, blockchains.

Merkle 1987

The kernel is novel less in any single primitive than in their *composition under one constraint*: a single local writer turns mutual exclusion, serializability, linearizability, complete mediation, and a durable audit log into consequences of one architectural choice rather than five separately engineered subsystems. The contribution is the reduction, and the discipline of saying exactly where the reduction stops paying.

One writer reduces coordination machinery

1.16 Open problems

These eleven problems are the layer's research frontier; they appear as starred exercises throughout and are collected here. OP-4 is shared with the personhood paper (this chapter owns the kernel mechanism; that one owns the personhood-and-reputation argument).

One of the eleven is closed — by a theorem in this chapter, not by an artifact. Several others have a *specified design* in the body, which is a real thing to have and is not the same thing as being solved; the maturity scale (§1.1.3) exists precisely to keep those two apart. Because a "solved" claim that survives in one place after being retracted in another is the most dangerous kind of error a paper like this can make, Table 1.15 states each problem's status once, and every exercise box, invariant row, adjacency clause, and figure caption in this chapter is written to agree with it. Where they ever disagree, this table is the one to trust.

★ OP-1 — Fair exclusion without a scheduler.
Status: *open*. Add bounded-wait to claims and locks while keeping the single-writer, no-background-scheduler simplicity. §1.5 specifies a reactive ticket-lock that would do it for cooperatively released claims and leaves the lazy TTL-sweep path — the one that matters when an agent dies — unfair. Nothing of it ships. Is a FIFO wait-list of time-to-live'd reservations enough once the sweep is included?

Table 1.15. Open-problem status, stated once. **closed** = settled in this chapter; partial = something ships but not the property as posed; *open* = not solved, whether or not a design exists. The last column grades the *design*, not the problem, on the scale of Table 1.1.

#	Problem	Status	Design in this chapter
OP-1	Fair exclusion without a scheduler	*open*	Stigmergic ticket-lock, specified (§1.5)
OP-2	Regimentation vs. enforcement	**closed**	Theorem 1.7.2 + Theorem 1.7.3
OP-3	Cross-runtime soundness (I11)	*open*	Differential fuzzing, specified (§1.10)
OP-4a	Checkpoint with teeth: durable lineage replay	*open*	Event-sourced rehydration, specified (§1.8)
OP-4b	Checkpoint with teeth: inference-state recovery	*open*	—
OP-5	Oracle completeness	*open*	State-machine assertions, specified (§1.7)
OP-6	Tamper-evidence on the read path	*open*	—
OP-7	Schema evolution without a sequencer	partial	Ordered boot ledger partial; `user_version` sequencer specified (§1.3)
OP-8	Stigmergic decay calibration	*open*	—
OP-9	Same-machine adversary	*open*	Ephemeral-UID sandbox + kernel-verified peer credential, specified (§1.12.4)
OP-10	Per-write-path durability	*open*	Typed durability selection, specified (§1.4)
OP-11	Cross-subsystem atomicity by default	*open*	One local transaction (§1.11) — a buildable defect

★ OP-2 — Regimentation vs. enforcement boundary.
Status: **closed** under the stated models. Theorem 1.7.2 supplies a synchronous database-guard construction. Theorem 1.7.3 gives the exact realization criterion under full observation: a prefix-closed safety language must be controllable. External authorization may precede an effect; post-commit monitoring alone cannot prevent that effect (Theorem 1.7.1). Concrete boundaries still require an explicit plant, event alphabet, observation model, and admission protocol.

Ramadge & Wonham 1987

Peter J. Ramadge and W. Murray Wonham. The Control of Discrete Event Systems. *Proceedings of the IEEE*, 77(1):81–98, 1989.

★ OP-3 — Cross-runtime soundness.
Status: *open*. A seeded differential fuzz — 10^6 concurrent operations against both the test and deployment SQLite bindings, asserting hash-equivalence of the resulting state — is specified (§1.10) and does not run in the pipeline today. Even once it does it is a test, not a proof: name the divergence class it would miss.

★ OP-4 — Checkpoint with teeth.
Status: *open*, and still the most important unbuilt thing at this layer. It is two problems wearing one number, and they have different difficulties, different owners and — this is the part that matters for anyone reading the table to decide what to build — different odds of ever being closed, so the table names them separately. **OP-4a, durable lineage replay**: *open*. Event-sourced rehydration (§1.8, specified) would replay a pinned commit and a truncated message log into a successor, which restores the *durable inputs* to an agent's context. The design is specified and nothing ships it, which is what specified means and why this half is not **closed** however tractable it looks. **OP-4b, inference-state recovery**: *open*, with no design in this chapter. A model's inference state is bound to a particular model, sampler and session and is not a portable artifact; replaying the inputs reconstructs what was fed to that state, never the state. Whether the reconstruction is close enough to count as restoring work, and against what test, is the question, and no formal result in this series establishes it. Closing 4a would not close 4b, and a reader who sees event-sourced rehydration ship should not read it as OP-4 discharged.

★ OP-5 — Oracle completeness.
Status: *open*. State-machine assertions (§1.7, specified) would add delta oracles and AST assertions to Listing 1.3's four

shipped kinds. Two kinds are not a completeness result, and neither kind yet survives the Goodhart argument the vocabulary exists to win.

★ **OP-6 — Tamper-evidence on the read path.**
Status: *open*. Where should hash-chain verification gate? A sampling/checkpoint regime with a provable detection-probability bound (couples to §1.12.4).

★ **OP-7 — Schema evolution without a sequencer.**
Status: partial. An ordered boot ledger with post-apply schema verification ships (§1.3); the `user_version` sequencer over declared migrations does not, and modules still self-initialize. Can any useful remnant of "any module, any order" survive safe renames, backfills, and down-migrations?

★ **OP-8 — Stigmergic decay calibration.**
Status: *open*. The right per-kind decay so coordination markers neither rot nor evaporate before they coordinate.

★ **OP-9 — Same-machine adversary.**
Status: *open*. §1.12.4 specifies the hardening — spawns in `bwrap`/`unshare` sandboxes under ephemeral UIDs, plus the kernel-verified peer credential (`SO_PEERCRED` / `LOCAL_PEERCRED`) read on each connection — and none of it ships. Today's peer-credential check is a software handshake over an owner-only socket, exactly as Table 1.14 states. Even built, the hardening would move the adversary from the same-uid row to the different-uid row, not out of the model. State the dependency the other way round as well, because it is the sentence an operator needs: OP-9 is the *precondition* for any Confined claim on a machine that more than one principal can log into. Until the sandbox and the kernel-verified peer credential ship, a confinement claim on a shared host is a claim about a cooperating co-tenant, and the honest grade for it is Observed — which is not a hedge, it is the difference between a boundary the kernel enforces and one the other tenant is choosing to respect.

★ **OP-10 — Per-write-path durability.**
Status: *open*. Typed durability selection (§1.4, specified) would route a money-bearing transaction through a connection configured with `synchronous=FULL` before it begins and acknowledge only its successful commit. A `wal_checkpoint(FULL)` call

is not that per-commit contract. No per-write selector is deployed, so the default remains I1b-exposed under Property 1.4.1. The harder half is the interface that stops a new write path from defaulting into the fast lane silently.

★ **OP-11 — Cross-subsystem atomicity by default.**
Status: *open* but cheap — a buildable defect, not a frontier. Wrap multi-subsystem operations in one local transaction at the interface boundary, eliminating the undefined partial-failure semantics of §1.11.

1.17 Conclusion: solid where local, provisional where it must grow

The kernel is a single-writer transactional reference monitor over a local SQLite/WAL database, and its two jobs — durable record and mediated exclusion — are real and, mostly, proven. It earns its strongest claims (mutual exclusion, serializable/linearizable consistency, oracle-bound closure) precisely by refusing the distributed-systems problem it superficially resembles: one writer, one file, one decider, no consensus.

Where it stops, it stops formally. Durability is split: process-crash durable (I1a), *not* guaranteed against power loss (I1b). The policy monitor detects and compensates; it does not regiment, and we credit the only genuine pre-commit regimentation to the uniqueness constraint and the boot-admission gate. Cross-subsystem atomicity is a buildable defect, not a frontier. Hash-chain tamper-evidence is re-scoped to cross-machine provisioning, where it actually defends someone. And the two genuinely soft spots — partial local identity enforcement (I12) and a checkpoint with teeth (OP-4) — are exactly the two things a cross-machine economy and a durable notion of agent personhood lean on hardest.

The kernel is solid where it is local and provisional exactly where a cross-machine economy would need it to be cryptographic and continuous.

Every “single-machine / one-writer / one-clock / self-asserted-identity” simplification that makes this layer clean is precisely the assumption a later layer must pay to relax. Cross-machine capability transfer over the cryptographic authorization chain this kernel

ships is where that payment begins, and it belongs to the economy layer, not here. Build the local floor first. The cryptography earns its keep when agents must trust one another across machines they do not control.

Chapter Appendices

1.A Implementation & status

The body of this chapter argues at the level of mechanisms, so that it reads as a self-contained systems paper. This appendix records the concrete grounding: the specific artifact that realizes each mechanism in the reference implementation, and an honest accounting of how mature each is. Nothing here is needed to follow the argument; it is here so the maturity grades in the body are auditable rather than asserted.

1.A.1 The reference implementation

The reference implementation is a single always-on local daemon written in TypeScript, run under a compiled JavaScript runtime, persisting to one SQLite database file in WAL mode. Clients — a command-line tool, agents over a tool-protocol bridge, and a binary inter-process transport — reach it over Unix domain sockets. The daemon is kept alive by the host operating system's service supervisor. The seven subsystems of §1.2 correspond to distinct modules that each self-initialize their own tables over the shared database handle, in the factory style `createModule(db)`.

1.A.2 Mechanism-to-artifact map

Table 1.16 maps each mechanism discussed in the body to the module that realizes it. The storage configuration referenced throughout is the WAL pragma set: `journal_mode=WAL`, `synchronous=NORMAL`, `wal_autocheckpoint=200`, `busy_timeout=5000`, `foreign_keys=ON`, with a clean-shutdown `wal_checkpoint(TRUNCATE)`.

1.A.3 Status and the limits that matter

The maturity grades from Table 1.12 reflect the state of the reference implementation at the time of writing. Four distinctions are necessary to read those grades correctly:

1. **Durability is split by fault class** (§1.4). A SQLite WAL database can remain crash-consistent while losing recent acknowledged

Table 1.16. Mechanism-to-artifact map for the reference implementation. Module names are illustrative of the organization, not a stable public interface.

Mechanism (body)	Realizing module(s)	Notes
Single write connection / pragmas	database-handle module	opens and configures SQLite
Port & lock claims	service-registry, locks, session-files modules	three claim granularities
Message bus / tuple space / markers	bus, tuples, marker modules	at-least-once delivery
Policy monitor	monitor module	post-commit log subscriber
Commitments & obligation	commitment, obligation-monitor modules	two of the hardening laws shipped
Memory & recovery	session, episodic-memory, recovery modules	recovery passes notes only
Audit log & hash chain	activity-log, hash-chain modules	verification not yet on the read path
Self-attestation	attestation, invariant-registry modules	boot gate + pre-flight + watchdog
Authorization chain	delegation-chain module	mechanically verified
Enforcement gate (macaroon discharge)	kernel macaroon module	crypto verified; in-band, not yet compulsory (§1.12.2)
Runtime-parity shim	runtime-shim module	normalizes binding differences
Identity (actor souls + credentials)	actor-souls, budget-guard modules	bounded gate ships; full write-boundary enforcement absent

writes after power loss under `synchronous=NORMAL`. Process-crash survival and power-loss durability are separate claims: the I1a/I1b split.

2. **The policy monitor is detect-and-compensate, not regimentation** (§1.7, Theorem 1.7.1). The monitor is implemented as a subscriber to the already-committed operation log, so it cannot prevent the bad prefix that triggers it; "physically unreachable" is reserved for the uniqueness-constraint and boot-gate states.

3. **Cross-subsystem atomicity is a buildable defect, not a research frontier** (§1.11). The database transaction primitive is already used elsewhere in the implementation; the only missing work is wrapping the multi-subsystem endpoints in it.

4. **A specified design is not a solved problem** (Table 1.15). Fair exclusion, runtime parity, checkpointing, oracle completeness, the same-machine adversary, and per-write-path durability remain open where the proposed mechanism lacks a realizing artifact. Their designs are graded specified, not treated as completed proofs or implementations. The security-relevant instance is worth naming on its own: the socket peer-credential check is a software handshake and *not* the kernel-verified `SO_PEERCRED` credential, in this chapter's threat model (Table 1.14), in §1.6, and in §1.12.4 alike.

1.A.4 Nomenclature

For the reader cross-referencing the accompanying chapters, the body's neutral names map to the series' internal terms as follows: the *substrate / coordination / legibility / economy* layers are the series' L0–L3; the *policy monitor* is the Arbiter; *stigmergic markers* are pheromones; the *bus* is tube; the *authorization chain* and *coordination lineage* are the two objects the series keeps distinct under the single phrase "delegation chain." The maturity grades — implemented / partial / specified / proposed — are the series' honest labels.

What *The Anchor Protocol* receives Under the stated transaction and mediation assumptions, admitted local effects have a serial history, a fencing epoch, and a place to attach receipts. The writer decides which requests take effect; it does not yet identify who may ask it to. The next chapter turns authority into a key-bound, attenuating, revocable object, so that every request the kernel admits arrives with a checked pedigree.

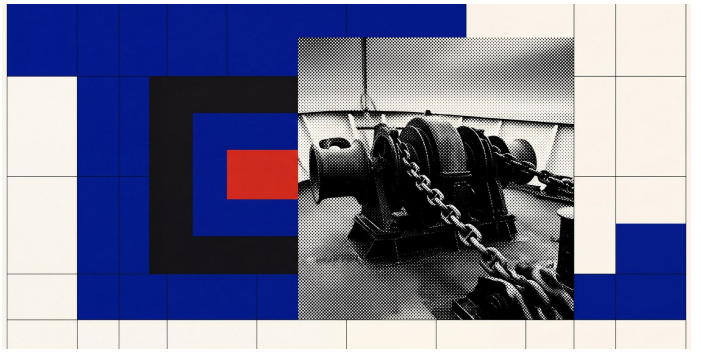
2

The Anchor Protocol

The question this chapter answers

How may authority travel without silently growing?

Delegated capability only narrows: every child is a subset of its parent, at every hop, and the verifier checks the whole chain.



Every program and every user of the system should operate using the least set of privileges necessary to complete the job.

Jerome Saltzer and Michael Schroeder, The Protection of Information in Computer Systems, 1975

How may authority travel without silently growing? Identity answers who signed; it does not answer what the signer may do. Anchor therefore checks every delegation edge, binds the grant to a key and audience, narrows scope and lifetime, and preserves revocation ancestry. The formal claim concerns the checked protocol and Rust paths, while R5 explains which surrounding policy promises can become admission gates.

2.1 What problem this solves

The scene. It is 2 a.m. and your laptop is running four coding-agent sessions at once: one refactoring the auth module, one running the test suite against a local Postgres, one building a preview deployment, and one you started an hour ago and forgot about. All four are agents. All four can read your files, bind ports, and run shell commands with your full user permissions — and none of them can tell, from the outside, which of the other three is behaving. Nothing on the machine distinguishes the forgotten session from a process that has quietly replaced it. A process ID alone cannot distinguish the original agent from a replacement. Consider what happens when the operating system reuses it.

Numbers by Hand 2.1.1 (The Ghost in the Harbor, Traced through One PID).

Agent A holds a valid card and is running as PID 4021. It crashes at time t. Three seconds later the operating system reuses 4021 for an unrelated Agent B, which presents A's card — scavenged from a shared temp file, byte-identical, signature still good, expiry still in the future. Now do the comparison two ways. A daemon that checks only the numeric process id computes $4021 = 4021$ and admits B; a daemon that compares the peer credential against the socket-connection identity recorded at issuance *and* re-validates the process start time computes $t + 3 > t$ and refuses it. Nothing about the card differs between the two readings — every cryptographic check passes in both — so the entire decision rests on a comparison the token cryptography never sees. That is why §2.5.3 lists this as a boundary rather than as a property, and why the fix is a runtime conformance test rather than a theorem. [internal]

In a multi-agent development environment, agents operate concurrently to read files, spin up test servers, and execute commands, and this introduces three concrete failure modes on `localhost`:

1. **Port Squatting (The "Ghost in the Harbor"):** If Agent A crashes, a malicious or malfunctioning Agent B can bind to Agent A's previously assigned port, intercepting traffic meant for A.
2. **Resource Contention:** Agents fighting over shared resources (e.g., a SQLite database file) without a centralized locking mechanism cause state corruption.
3. **Privilege Escalation:** A "Reviewer" agent delegated by a "Coder" agent should not possess the rights to initiate production deployments, yet local environments rarely enforce principle-of-least-privilege between processes.

The protocol in brief. *An agent presents a short-lived signed card naming its permissions. A delegated card may grant no more authority than its parent. Revocation withdraws a card before expiry once the verifier receives the relevant epoch update.* The chapter distinguishes the implemented checks from the properties established by bounded or symbolic models. To make it enforceable, Port Daddy introduces the concept of a **Harbor**—a zero-trust, namespace-isolated execution environment where agents must prove their identity and capabilities. Our approach builds upon foundational work in capability-based security by Dennis and Van Horn and the principle of least privilege articulated by Saltzer and Schroeder. This protocol sits in a specific lineage — Macaroons for delegation, ProVerif and Tamarin for symbolic verification, the JWT algorithm-confusion literature for the Phase 1 hardening, and the emerging agentic-commerce credential profiles for the envelope — traced in full in Appendix 2.F, which a reader following the argument rather than the citations can skip entirely.

Dennis & Van Horn 1966. *Programming Semantics for Multiprogrammed Computations.*

Saltzer & Schroeder 1975

How to read this chapter. §2.2 is the protocol: four phases, each closing a hole the previous one left open. §2.3 is the evidence: three verification layers, what each one proves, and what it hands the next. §2.5 is the accounting: the adversary we assume, the properties that survive it, and where the guarantees stop. Table 2.2 is the map

for §2.3; Table 2.4 closes the loop on the three failure modes just listed.

2.2 The Anchor Protocol Architecture

The Anchor Protocol defines how agents authenticate to a Harbor and to each other. It issues **Harbor Cards**: signed records naming a subject, a harbor, permissions, an expiry, and a unique card identifier. A *subject* is the agent named by the record; a *verifier* is the component deciding whether to accept it. A signature authenticates the record under a trusted key. It does not establish that the requested action is permitted: that further decision is *authorization*. A JSON Web Token (JWT) is the record's encoding, not a source of authority by itself. The time from issuance to expiry is its *time to live* (TTL). We present four mechanisms as phases: algorithm pinning, asymmetric identity, delegation, and revocation. This ordering explains their security dependencies, not a release schedule or a claim that every mechanism is deployed (Figure 2.1; Table 2.1).

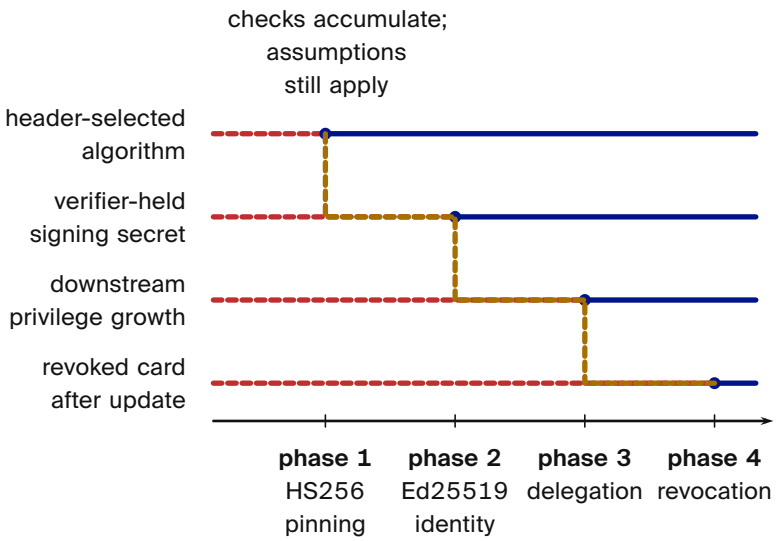


Figure 2.1. Each phase adds a check without removing earlier checks. The marked step introduces the named defense under its stated assumptions. Revocation only rejects at a verifier that has received the update; no gossip deadline or universal security closure follows. [internal, Table 2.1]

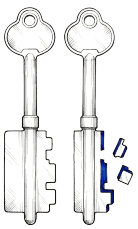
Figure 2.1 draws the same table as a staircase: each threat's row turns from red (open) to the chapter hue (closed) at the phase that closes it, and the closure frontier moves strictly rightward, so no later phase reopens a threat an earlier one closed. A common

Table 2.1.
The four phases as a lookup table, restating Figure 2.1's flow in row form. Threat closure is cumulative, not substitutive. Each phase closes one attack surface, and the last column is why no later phase reopens it: every phase adds a check that a later phase changes the key or the chain of, never the party that performs it. [internal]

Phase	Mechanism	Attack surface closed	Stays closed because
1	HS256 pinning: the verifier takes the algorithm from policy	header-selected algorithm (algorithm confusion)	the algorithm is never read from the token; later phases change the key type, not who chooses
2	Ed25519 identity: the daemon signs with a private key, verifiers hold only the public key	shared-secret theft	no verifier holds a secret that could mint a card; a stolen public key mints nothing
3	delegation: a child card carries its parent chain, checked as a subset at every hop	downstream privilege growth	the subset check is per hop and monotone, so no later hop widens scope or lifetime
4	revocation: epoch filters gossiped between daemons and consulted at every acceptance	post-issue authority (a leaked card that should die before its TTL)	acceptance requires the current epoch, so every daemon that holds the epoch refuses the revoked card

thread across all phases is *capability attenuation*: a delegated token never carries more authority than its parent. A child's expiry is the earlier of its parent's expiry and the requested expiry. A request for twenty minutes under a parent with ten minutes left therefore produces a ten-minute card; an already-signed child claiming the longer expiry must fail verification. Rights and expiry can narrow or stay unchanged, never widen. Figure 2.2 separates a valid child, a request clamped before signing, and a refused rights expansion.

A smaller set of permissions



Removing teeth suggests removing authority. A capability must also bind that restriction cryptographically.

Numbers by Hand 2.2.1 (A Child Card under a Parent with Ten Minutes Left).

A parent card holds `fs:read` and `db:write` with 10 minutes of lifetime remaining. A request for `db:write` for 20 minutes is clamped before signing: its rights fit, but the issuer grants only the parent's remaining 10 minutes. A request for 8 minutes receives 8. In contrast, if an already-signed child *claims* 20 remaining minutes, verification refuses it. Issuance chooses permitted values; verification checks the values actually signed. The comparison is coordinatewise: a child must narrow both the rights set and the expiry, not trade one against the other. A smaller rights set does not buy a longer lifetime. In this partial order both coordinates moved down or stayed: rights

$$\{\texttt{db:write}\} \subseteq \{\texttt{fs:read}, \texttt{db:write}\}$$

and lifetime

$$\min(20, 10) = 10.$$

Figure 2.2 contrasts that clamped request with a shorter eight-minute child and an attempted extra right. [internal]

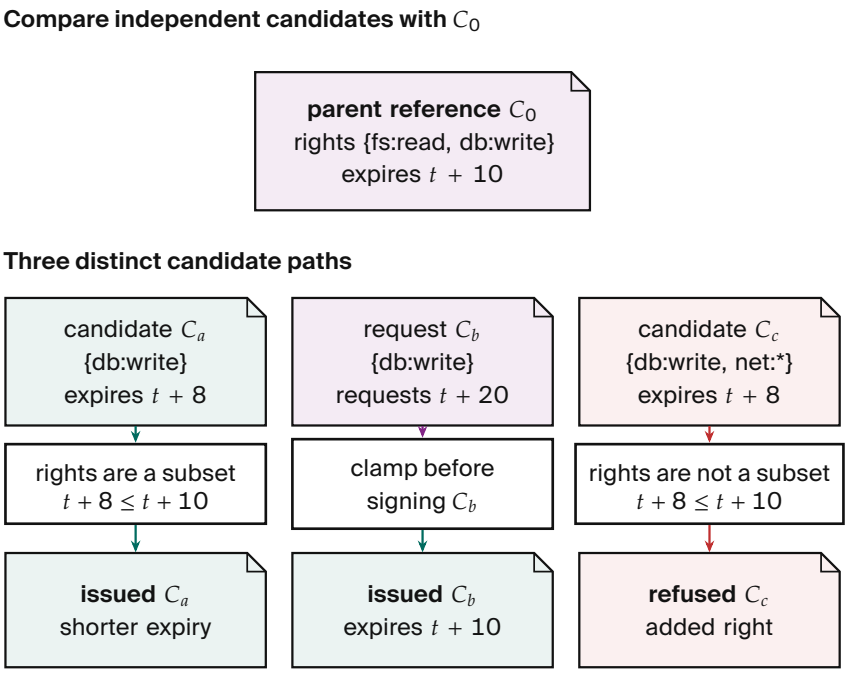


Figure 2.2. Every hop checks a rights subset and an expiry bound. A child may inherit its parent's expiry when a longer lifetime is requested; it may never gain a new right.

2.2.1 Phase 1: Symmetric Pinning

The symmetric profile uses HS256 (HMAC-SHA256), a message authentication code: the same secret key is used to create and check the tag. A verifier holding that secret can therefore also mint tags. Phase 2 separates those powers with a private signing key and a public verification key. One attack on JWT verification is "Algorithm Confusion" (CVE-2015-9235, CVE-2023-48238), where an attacker modifies the token header to `{"alg": "none"}` or symmetric HS256 while using an asymmetric public key as the HMAC secret.

CVE-2015-9235. Algorithm Confusion in jsonwebtoken Node.js library. NIST National Vulnerability Database, 2015.

CVE-2023-48238. Algorithm confusion in json-web-token library. NIST National Vulnerability Database, 2023.



Landmark 06 · The Attenuation Frontier & Cryptographic Macaroons

An attenuated credential adds restrictions to an existing grant. Authority narrows only when the verifier checks the complete delegation chain and enforces every applicable restriction. Possessing a signed credential is not sufficient authorization for an arbitrary action.

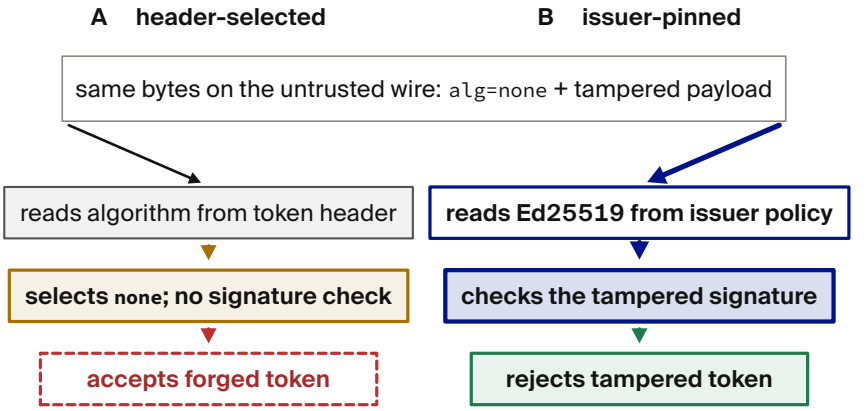
Design Invariant 2.2.2 (Algorithmic Pinning).

The Port Daddy Verifier employs strict **Algorithmic Pinning**.

The token cannot select the verification algorithm. The verifier fixes the allowed algorithm and key type from trusted policy, then validates the token's header against that policy. Rejecting a mismatched header is compatible with pinning; executing whichever algorithm the header requests is not.

This approach is what the JWT Best Current Practices, RFC 8725, recommends.

Figure 2.3 shows the two verifier paths side by side: the naive verifier trusts the `alg` field on the wire (and is exploited), while the pinned verifier records the issuance algorithm out-of-band and admits no rewrite trace.



Yaron Sheffer, Dick Hardt, and Michael Jones. JSON Web Token Best Current Practices. RFC 8725 (BCP 225), IETF, February 2020. Online source

Figure 2.3. Aligned security traces hold the wire bytes constant and vary only the source of algorithm authority. Header selection reaches acceptance without a signature check; issuer pinning forces Ed25519 and rejects the same tampered token. [verified, v1_refined.pv]

2.2.2 Phase 2: Asymmetric Identity (Ed25519)

The asymmetric profile uses **Ed25519 (EdDSA)** so agents can verify peer credentials without holding the issuer's signing secret. Ed25519 provides deterministic signatures with small keys and signatures.

Bernstein et al. 2012

Simon Josefsson and Ilari Liusvaara. Edwards-Curve Digital Signature Algorithm (EdDSA). *RFC 8032*, IETF, January 2017. Online source

Design Invariant 2.2.3 (Harbor Key Hierarchy).

The Port Daddy Daemon acts as the Root Certificate Authority (CA). Each Harbor is assigned a unique Ed25519 keypair. In Phase 2 the Daemon issues Harbor Cards signed by the Harbor's private key `sk_harbor`, escrowed to it over a private channel; agents use the matching `pk_harbor` (broadcast via Port Daddy's ambient discovery service) to verify tokens presented by peers.

The delegation profile in Phase 3 uses a different trust root: the Daemon's *own* signing key `sk_daemon`. Every verifier checks `chain[0]` under `pk_daemon` — which is what Algorithm 2.4 requires, what `analyses/harbor_card_v3`, `v5`, `v6` and `v7` model, and what `core/harbor-card-rs` implements.

The escrowed per-Harbor keypair belongs to the Phase 2 profile and does not root authorization chains.

This is a statement about how the deployed daemon holds keys — backed by the implementation and its conformance tests, not by any query in Table 2.3, whose models assume the hierarchy rather than establish it.

2.2.3 Phase 3: Offline Delegation

Multi-hop delegation lets authority pass through more than one agent. When Agent A spawns Agent B, it does not need to request a new token from the Daemon. Instead, it uses **Offline Attenuation**, inspired by Macaroons.

Birgisson et al. 2014



Like a graduated set of nested measuring spoons, each successive delegate receives a smaller measure of authority that fits inside the parent; equal authority is allowed.

Design Invariant 2.2.4 (Offline Attenuation).

Agent A takes its existing Harbor Card, appends a new set of restricted capabilities (e.g., `["db:read"]` reduced from `["db:read", "db:write"]`), and signs the *entire chain* with its own ephemeral private key. A *nonce* is a fresh value used to bind one protocol context; freshness alone is not a replay cache. The same signed message can still be copied unless an acceptance rule rejects repeats.

The Harbor verifies the Daemon's root signature, Agent A's delegation signature, and checks that Agent B's capabilities are a subset of Agent A's.

The subset check is the deployed verifier's obligation at every hop; what the models of §2.D.1 establish is that the check is necessary and sufficient at one hop, and the per-hop discipline — not this statement — is what chains it.

Figure 2.4 puts that ceremony on a clock: who signs what, in what order, and — the point of "offline" — which participant stops being spoken to after the first message.

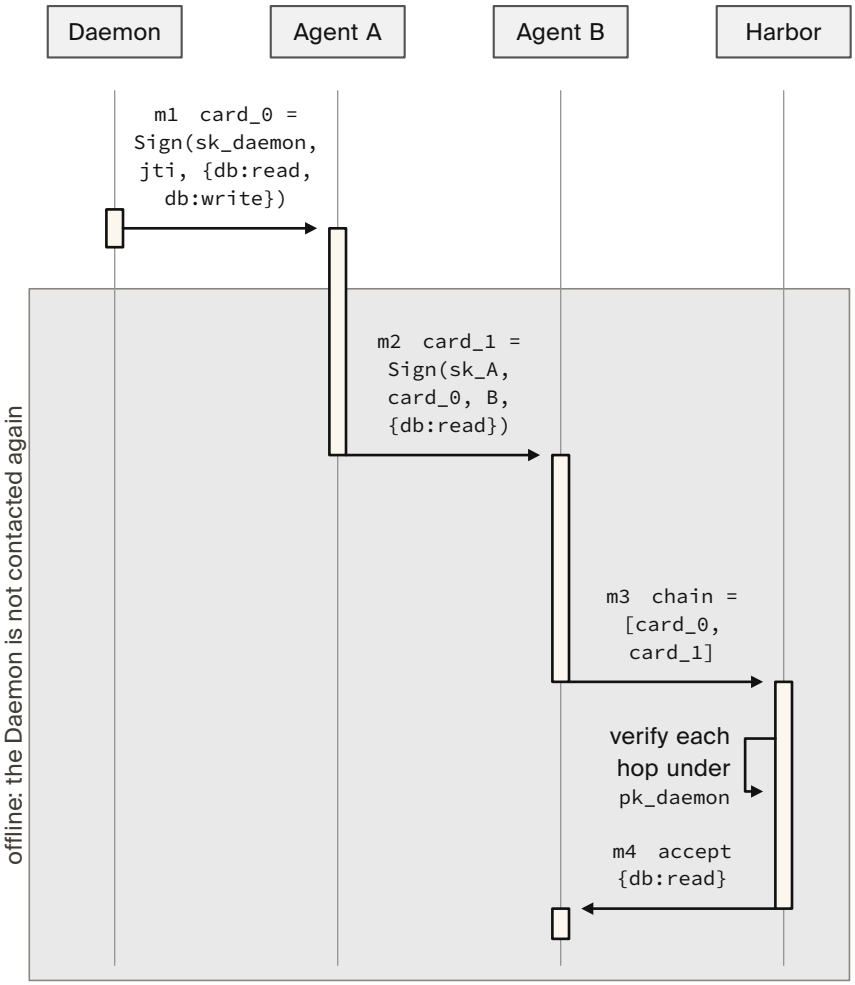


Figure 2.4. The issue–delegate–verify ceremony as a sequence diagram. Every message after `m1` falls inside the shaded region, where the Daemon is not addressed: agent A mints `card_1` from `card_0` with its own key, and the Harbor accepts the chain under `pk_daemon` with no round trip to the issuer. [internal]

For depth-N chains, the verifier walks the structure shown in Figure 2.5. Each hop must produce a fresh nonce and bind the previous and next identities into its signature so that an attacker who intercepts a single hop cannot splice it into a different chain.

Macaroons are the inspiration, but two older literatures own the property being checked here, and supply the right vocabulary for the comparison. SPKI/SDSI's certificate-chain reduction is a calculus for asking exactly this question — does the chain only ever narrow as it lengthens? — and answering it by reducing the chain to one equivalent certificate; the per-hop subset check the Harbor performs is that reduction carried out one link at a time. And Miller's object-capability discipline — no ambient authority, a capability as the only route to any authority at all, worked out for concurrent and mutually suspicious distributed components — is the conceptual peer of this protocol's capability/permission split, closer than Dennis and Van Horn's founding 1966 statement. The corresponding design rule is to restrict a capability at issuance as well as checking the chain at verification, so that the chain check is defense in depth rather than the only line. The v6 verifier of §2.D.1 provides a concrete counterexample: checking each hop against the root instead of its parent admits a downstream re-grant.

Ellison et al. 1999. *SPKI Certificate Theory*. RFC 2693.

Rivest & Lampson 1996. *SDSI — A Simple Distributed Security Infrastructure*.

Miller 2006. *Robust Composition: Towards a Unified Approach to Access Control and Concurrency Control*.

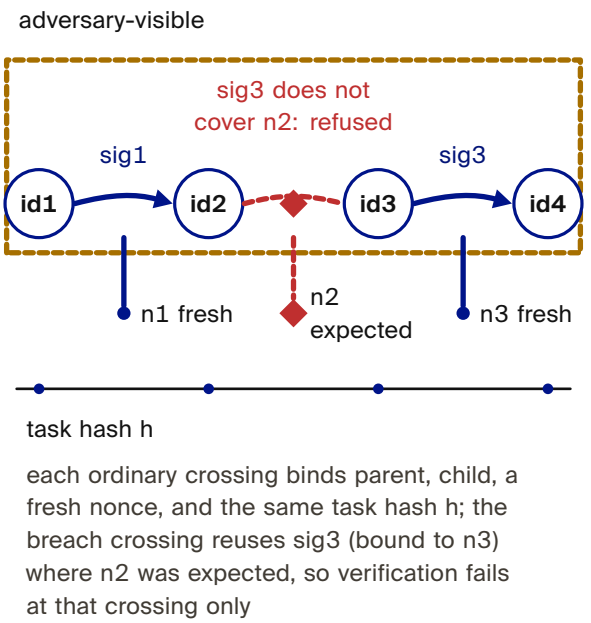
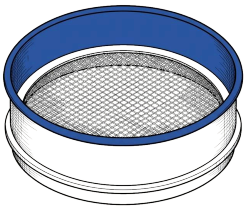


Figure 2.5. Each signature binds adjacent identities, a nonce, and the task hash. Moving a legitimate signature to another hop fails because its signed identities and nonce no longer match. [internal]

2.2.4 Withdrawing a card before it expires



Like an analytical laboratory test sieve whose wire mesh retains coarse particles, contextual caveats filter out forbidden operations before execution.

Bin Fan, David G. Andersen, Michael Kaminsky, and Michael D. Mitzenmacher. Cuckoo Filter: Practically Better Than Bloom. In *ACM CoNEXT*, 2014.

Capability tokens have a TTL, but natural expiry is not always sufficient. Two circumstances force early withdrawal: (i) *compromise*—a Harbor Card leaks from its agent's process, and every second until TTL is a second the attacker has legitimate capabilities; and (ii) *policy reversal*—a principal decides an agent should no longer hold `db:write`, and waiting for TTL is unacceptable. A linear scan of n revocation IDs costs $O(n)$; an exact hash-set index can instead provide expected $O(1)$ lookup. The probabilistic index below trades space against extra authoritative lookups, not against the necessity of linear scanning. The protocol gossips an **Append-Only Event Log of Revocation IDs** using Vector Clocks, not the lossy cuckoo-filter encodings themselves. To maintain $O(1)$ read performance, each daemon continually projects its event log into a local, ephemeral cuckoo filter (Figure 2.6).

Why filters cannot be merged bitwise. A cuckoo-filter slot stores a fingerprint, not an independent membership bit. Consider two one-slot buckets and two keys whose candidate buckets are 0 and 1. Peer A stores fingerprint `001` in bucket 0; peer B stores `010` there instead. Both leave bucket 1 empty. Bitwise OR produces `011` in bucket 0 and leaves bucket 1 empty, so neither original fingerprint can be found. This small counterexample is enough to rule out raw OR as a general merge operation. Duplicate fingerprints are not themselves invalid: correct cuckoo filters can retain repeated insertions. The protocol instead unions authoritative revocation IDs and rebuilds a local filter, handling insertion failure rather than silently losing a revocation. Figure 2.6 contrasts the two operations.

Cuckoo filters in one paragraph. A cuckoo filter is a compact probabilistic set: under successful insertion and correct deletion discipline it has false positives but no false negatives, while supporting deletion. Each inserted key is reduced to a fingerprint and placed in one of two candidate buckets; on collision, fingerprints are relocated until a slot opens or insertion fails. Lookups inspect both buckets. The expected lookup cost is $O(1)$ and the approximate false-positive probability for two buckets of size B and f fingerprint bits is at most about $2B/2^f$ in the low-collision model. High load

increases insertion failures; the implementation rejects or rebuilds rather than silently dropping a revocation. Duplicate fingerprints require counted or authoritative-table-backed deletion so that removing one key does not reanimate another.

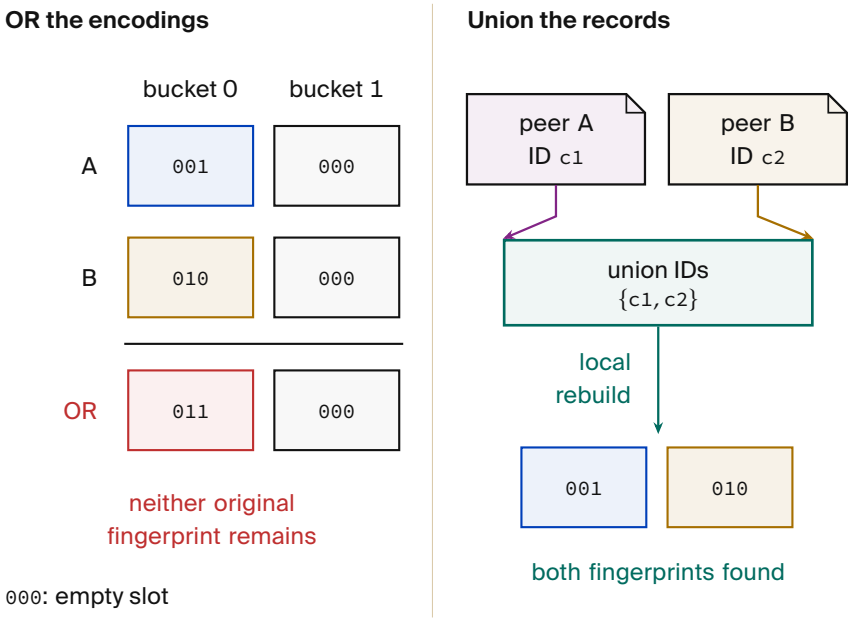


Figure 2.6. Bitwise OR corrupts stored fingerprints; set union preserves the identifiers needed to rebuild them. Two-bucket example with one three-bit slot per bucket.

Why cuckoo, not Bloom. For uniform-TTL workloads, a rotating Bloom filter would suffice. Port Daddy's TTLs are heterogeneous—Shipwright proposals run hours to days, `pd spawn` children run minutes, one-shot agents run seconds. Rotating Bloom either rotates at the longest TTL (short-TTL revocations linger orders of magnitude longer than needed) or buckets by TTL class (reintroducing the structural complexity cuckoo collapses). Cuckoo provides $O(1)$ per-entry removal at each specific expiry, while a mistaken revocation is corrected by issuing a new card, never by removing a live denial.

Space. At false-positive rate p_{fp}, a cuckoo filter uses approximately $\log_2(1/p_{\text{fp}}) + 3$ bits per entry. For $p_{\text{fp}} = 10^{-3}$, $\approx$ 13 bits per revoked card. A fleet retaining 10^5 active revocations fits in $\approx$ 160 KB—a small local index. The authoritative ID log is additional storage and is what peers exchange.

Numbers by Hand 2.2.5 (Sizing the Revocation Filter).

At $p_{\text{fp}} = 10^{-3}$ the per-entry cost is

$$\log_2 1000 + 3 = 9.97 + 3 \approx 13 \text{ bits}.$$

A fleet holding 10^5 live revocations therefore carries

$$1.3 \times 10^6 \text{ bits} = 162{,}500 \text{ bytes},$$

about 159 KiB. This is the local filter only: it is not a bound on the revocation-ID log, its metadata, or the gossip traffic.

At $p_{\text{fp}} = 10^{-6}$ the same fleet needs

$$\log_2 10^6 + 3 \approx 23 \text{ bits per entry},$$

$\approx$ 287,500 bytes, or 281 KiB: three orders of magnitude fewer false positives for less than twice the space, which is why the filter's bound, not its size, is what the load ceiling protects. [internal]

Design Invariant 2.2.6 (Revocation-Augmented Verification).

Let R_t denote the authoritative Append-Only Event Log of revoked Harbor Card IDs at time t. Each daemon d maintains a local, ephemeral cuckoo filter index $F_d \approx R_t$. After signature, scope, time, and ancestor checks have passed, revocation verification additionally:

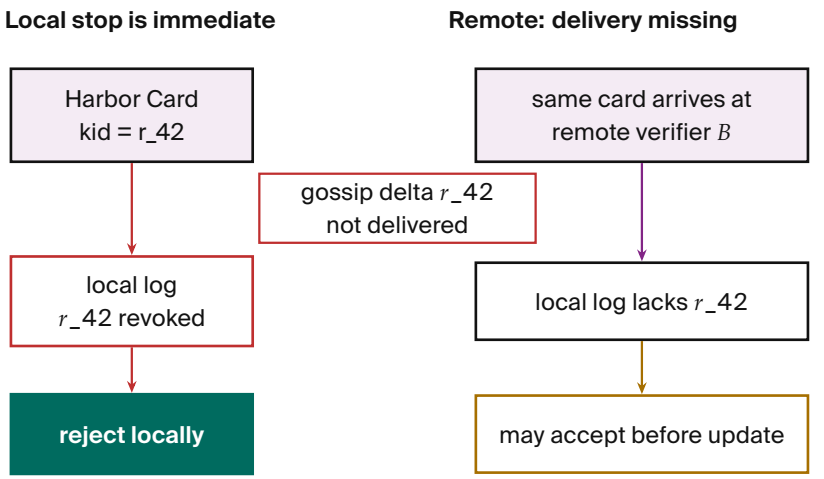
1. Looks up the card's `kid` in F_d. If absent from a complete, healthy index of the local authoritative table, this local revocation check passes; that is not evidence that no remote peer has revoked the card.
2. If present, query the local `revocations` table (authoritative). If genuinely revoked, reject with `revoked`; otherwise the filter is in the false-positive regime and the caller is admitted, with an asynchronous reconciliation job repopulating the filter.

Here `kid` names the card identifier used by this protocol, not merely a signing-key identifier. Every revoked ancestor must

also cause its descendants' chains to fail. A stale, failed, or rebuilding index must fall back to the authoritative local table. These checks cannot supply a revocation update that has not arrived.

The two steps are an implementation discipline the daemon is built to keep — the filter is only ever an index, never the authority — and it is what makes a false positive a slower answer rather than a wrong one.

Gossip-based synchronization. Daemons in a mesh synchronize revocation deltas by anti-entropy gossip. Under a connected complete-overlay model with independent uniform peer selection and reliable rounds, epidemic dissemination is expected $\Theta(\log m)$ rounds. This is an expectation, not a deadline; partitions, message loss, topology, and scheduling change the constant or prevent global convergence until the partition heals. Security-critical revocations can additionally trigger best-effort immediate push. Figure 2.7 contrasts a local revocation with a remote verifier that has not received it.



Alan Demers et al. Epidemic Algorithms for Replicated Database Maintenance. In *ACM Symposium on Principles of Distributed Computing (PODC)*, 1987.

Figure 2.7. The issuing daemon stops a revoked card locally. A remote verifier can still accept while its revocation update is undelivered; gossip convergence is not a global kill switch.



Like a mechanical parking meter whose red flag drops when time expires, capability grants expire automatically when their lease lifetime elapses.

What retires an entry. An append-only log that never drops anything has a size that grows with the fleet's whole history rather than with its live population, and on a constrained host that is a storage bug waiting for a slow month to expose it. The log does not have

to grow that way, and the reason is temporal attenuation: a revocation identifier exists to stop a card being honoured, and a card that has expired is already refused by the expiry check before the filter is ever consulted, so the entry has no remaining work to do.

Design Invariant 2.2.7 (Revocation-Log Truncation).

Let $T_{\max}$ be an enforced upper bound on card lifetime. Assume card identifiers are never reused, descendant expiry never exceeds ancestor expiry, and all verifiers enforce expiry with a stated clock-skew allowance and no unchecked clock rollback.

An entry naming card `kid` may be dropped from the active revocation log R_t once every card that entry could name has expired — that is, once t exceeds the entry's recorded card expiry plus that allowance — without changing any live card's authoritative revocation status.

Because attenuation only ever shrinks a lifetime (§2.2), no descendant of a revoked card can outlive the entry that names it, so a dropped entry cannot be needed again by a chain derived from it.

The log's steady-state size is therefore bounded by the revocations issued in the trailing window $T_{\max}$ plus the clock allowance, not by the fleet's lifetime history.

This rule is specified here and the daemon does not yet apply it, which is the distinction the rest of this series marks between a design and a shipped mechanism, and it is the honest grade for this one. What would verify it is a property test rather than an argument — generate a revocation log and a set of cards, truncate at an arbitrary cut, rebuild both filters, and assert that neither loses any live revocation; authoritative checks must return the same decisions before and after compaction. Approximate-filter false positives need not be identical. A test with no live revocations would pass vacuously on an empty live set, so the live set is what the generator has to make non-trivial. Note the one thing the invariant does not buy: it bounds the log by a *time* window, so a fleet that issues revocations faster than they expire still grows, and the load ceiling below, not this rule, is what holds that case.

Compaction also needs a restart protocol: a returning peer must receive the current authenticated snapshot or epoch before serving checks, not interpret a truncated delta stream as complete history. Retiring active denial entries does not authorize deleting the archival evidence needed for disputes. A rebuilt probabilistic filter may change false positives on unrevoked cards; the invariant is no lost revocation of a live card, with authoritative lookup resolving hits, not bit-for-bit equality of every approximate-filter answer.

Pollution as denial of revocation. The load ceiling keeps the false-positive bound honest; it does not by itself defeat an attacker who submits many authenticated-looking revocation identifiers to push the filter out of its bound-holding regime, which turns the revocation path into a denial of service. The design requires three things of the daemon: a per-issuer insertion quota, an overflow record for every refused insertion, and lookups that consult the authoritative event log while a flooded filter rebuilds, so that flooding degrades revocation checks to slower, never to missed.

Design Invariant 2.2.8 (Filter Monotonicity over a TTL Epoch).

For any Harbor Card c with issue time t_0 and TTL τ, if $c \in R_t$ for some $t \in [t_0, t_0 + \tau]$, then $c \in R_{t'}$ for all $t' \in [t, t_0 + \tau]$.

After $t_0 + \tau$, c may be removed from R since the card has expired.

Design Invariant 2.2.9 (No Partial Reanimation).

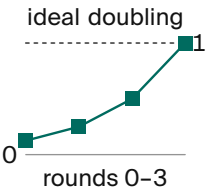
A revoked card cannot be un-revoked within its TTL.

If an operator decides the revocation was erroneous, they must issue a new card.

Theorem 2.2.10 (Conditional Gossip Expectation).

Let Δ be the duration of one gossip round. Under the complete-overlay, uniform-peer, reliable-round model above, a revocation disseminates to all m daemons in expected $\Theta(\Delta \log m)$ time.

No finite worst-case freshness bound holds under an unbounded partition.



An ideal schedule informs 1, 2, 4, 8 of eight peers in rounds 0, 1, 2, 3. Random contacts need not attain it.

Numbers by Hand 2.2.11 (A Possible Trace Is Not a Deadline).

Take five daemons and two-second rounds. In a favorable push schedule, each informed daemon contacts one previously uninformed peer: the informed count can grow

$$1 \to 2 \to 4 \to 5,$$

reaching all five after three rounds, or six seconds.

This is one possible trace, not the expected time of random gossip. Repeated contacts can take longer; the asymptotic expression $\Theta(\log m)$ does not specify its constants or an exact expectation for five peers.

Isolate the fifth daemon and it receives nothing until delivery resumes, as in Figure 2.7. [internal; illustrative schedule]

Soundness preservation. Revocation strictly narrows acceptance, so existing ProVerif soundness proofs (Property 2.3.1) are unaffected. The new proof obligation is small: revocation is *enforced*, which follows by inspection of the verification path—the revocation check is unconditional and precedes admission. One active card has exactly two terminal exits, explicit revocation or natural expiry; neither exit can return that card to the accepted set (Figure 2.8). A correction is fresh issuance under a new card ID, not a transition of the old card.

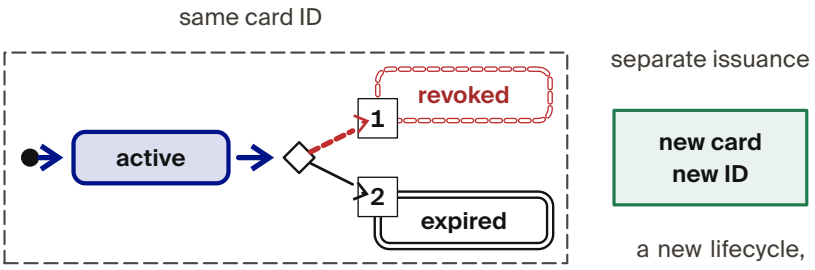


Figure 2.8. A UML state machine gives one card ID exactly two guarded terminal exits. Fresh issuance is shown separately, with no outgoing path from the revoked or expired card. [internal]

Guard	Predicate
1	explicit revocation: operator action or detected compromise
2	natural expiry: issue time plus TTL

Privacy. A Dolev-Yao adversary observing all gossip traffic learns the revoked-card IDs, leaking which agents have been disciplined. For an organization's own daemons, this is acceptable audit transparency. Stronger privacy is available by encoding revocations as salted hashes $H(\texttt{kid}, \texttt{salt})$ where the salt rotates daily, stored encrypted under a harbor session key.

Recall

1. What must every delegated card preserve about its parent's authority?
2. Why can't cuckoo filters be OR-merged? What is gossiped instead?
3. Which events require withdrawal before expiry?

2.3 How we know it is correct

Following the industry standard set by AWS (s2n-tls) and Microsoft (Project Everest), we decoupled the verification of the *protocol design* from the verification of the *implementation*. The full stack has three layers (Table 2.2): symbolic protocol analysis at the top, bounded model checking on the Rust source in the middle, and runtime conformance tests against the deployed binary at the bottom. Each layer's obligations flow downward; each layer's evidence flows back up. The stack is sound only if every bridge — symbolic-to-source and source-to-deployed — holds.

Table 2.2 is this section's map. Read down the rows for what each layer hands the next as an obligation it cannot discharge itself; read the third column for what has actually been checked about the running code. The last column names two important correspondence gaps where evidence must connect one layer to the next — which is exactly why they are listed as assumption gaps rather than as inferences. Figure 2.9 draws the same table as one directed bridge: obligations descending the left rail, evidence climbing the right, the two gaps sitting where a human bridges the claim.

AWS s2n-tls. An Implementation of the TLS/SSL Protocols. Online source, 2022.

Karthikeyan Bhargavan, Barry Bond, Antoine Delignat-Lavaud, Cédric Fournet, Chris Hawblitzel, et al. Everest: Towards a Verified, Drop-in Replacement of HTTPS. In *Logic in Computer Science (LICS)*, 2017.

Table 2.2.
Tools discharge claims within each layer. Model-to-source correspondence and source-to-running-binary correspondence remain separate human assertions; none of these tools closes either gap. [internal]

Layer	Tool	What it shows	What it cannot show
protocol model	ProVerif	secrecy, authentication correspondence in the modeled phase, attenuation soundness on the multi-hop model	that the modeled protocol is this source (the specification gap; a human asserts it)
Rust source	Kani and tests	the subset logic on two concrete vectors, bounded absence of panic with the cryptography stubbed, branch-freedom at source level	that the inspected source is this running binary (the deployment gap; a human asserts it)
running binary	conformance suite	the selected attacks are refused at the FFI boundary	any attack the suite does not contain

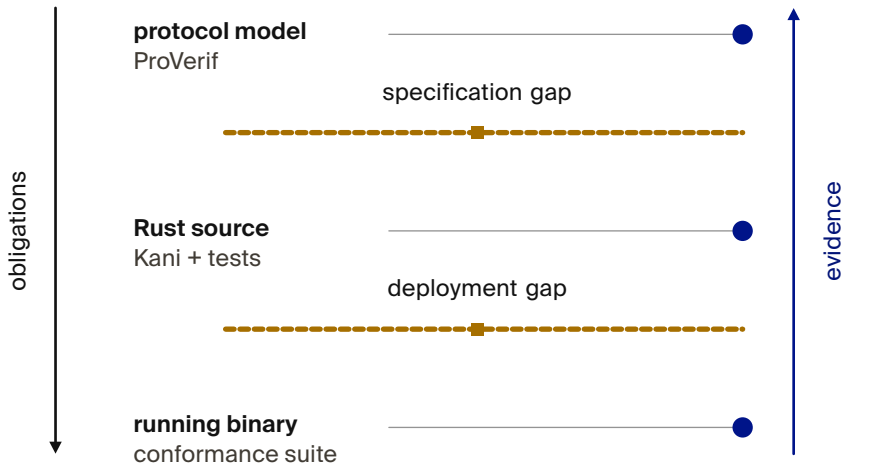


Figure 2.9. Obligations descend from model to source to binary; tool evidence returns upward. The two correspondence gaps still require human justification. [internal, Table 2.2]

2.3.1 Symbolic Analysis with ProVerif

To prove the protocol's logical soundness against a Dolev-Yao adversary (an attacker who controls the entire network), we modeled the Anchor Protocol in **ProVerif**. ProVerif represents protocols as Horn clauses and uses resolution to prove security properties for an unbounded number of sessions.

Dolev & Yao. *On the Security of Public Key Protocols*. IEEE Trans. Inf. Theory 29(2):198–208, 1983.

Bruno Blanchet. Modeling and Verifying Security Protocols with the Applied Pi Calculus and ProVerif. *Foundations and Trends in Privacy and Security*, 1(1-2):1–135, 2016. Online source

Model-Checked Property 2.3.1 (Authentication Correspondence in the Modeled Phase).

For the modeled delegation phase, ProVerif confirmed the correspondence:

$$\begin{aligned}\mathsf{Accepted}(b, h, cap) \implies (&\mathsf{IssuedRoot}(a, h, cap)\\ &\wedge \mathsf{Delegated}(a, b, cap))\\ &\vee \mathsf{IssuedRoot}(b, h, cap).\end{aligned} \tag{2.1}$$

The displayed query is a non-injective authentication correspondence: every accepted event has a matching modeled issuance/delegation event. It does not by itself prove replay freedom, capability attenuation, or arbitrary-depth transitivity; those are separate queries and bounds.

The three baseline protocol phases were independently verified in ProVerif 2.05. Table 2.3 adds two encoding tests for the algorithm-confusion boundary; full models are reproduced in Appendices 2.B, 2.C, and 2.D.

The following is the verbatim ProVerif 2.05 output for the authorization chain query (Phase 3), the most complex property:

Listing 2.1. ProVerif 2.05 Terminal Output — Phase 3 Authorization Chain

```
RESULT event(Accepted(b_2,harbor_id[],cap_1))
  ==> (event(IssuedRoot(a_3,harbor_id[],cap_1))
       && event(Delegated(a_3,b_2,cap_1)))
      || event(IssuedRoot(b_2,harbor_id[],cap_1))
  is true.
```

Query	Result
Phase: v1 (HS256) | **Model:** `v1_refined.pv`	
¬ attacker(*master_key*)	**TRUE**
Accepted $\Longrightarrow$ Issued	**TRUE**
Phase: v2 (Ed25519) | **Model:** `v2_asymmetric.pv`	
¬ attacker(*harbor_sk*)	**TRUE**
Accepted $\Longrightarrow$ Issued	**TRUE**
Phase: v3 (Delegation) | **Model:** `v3_delegation.pv`	
Accepted(b) $\Longrightarrow$ (IssuedRoot(a) $\wedge$ Delegated(a, b)) $\vee$ IssuedRoot(b)	**TRUE**
Algorithm-confusion tests: encoding	
Model: `algconfusion.pv`	
AcceptedPinned(m) $\Longrightarrow$ Issued$_A$(m)	**TRUE**
AcceptedNaive(m) $\Longrightarrow$ Issued$_A$(m)	**FALSE**

Table 2.3. Five baseline queries and two algorithm-confusion tests. Pinned verification passes; the naive verifier fails as intended. The delegation-depth bound still applies.

2.3.2 Runtime Enforcement: The Arbiter

Formal models establish properties of their abstractions. At runtime, Port Daddy deploys the **Arbiter**—an ambient monitor that subscribes to the daemon's post-commit activity log and checks six derived invariants:

1. `PID_SQUATTING`: Detects when a process claims a port previously held by a different PID (the "Ghost in the Harbor").
2. `CAP_ESCALATION`: Verifies capability subsets via the deployed Rust FFI enforcer.
3. `NOTE_MONOTONICITY`: Ensures the note sequence for active sessions never shrinks (from the TLA$^+$ `NoteMonotonicity` invariant).
4. `ESCROW_POSITIVE`: Validates that active sessions have positive escrow (when Float Plans are active).
5. `LOCK_OWNER_VALID`: Ensures locks are only held by registered agents with active sessions.
6. `HEARTBEAT_FRESHNESS`: Flags stale heartbeats as early warning for agent death.

Violations are logged and can trigger the "man overboard" salvage protocol: the daemon revokes the offending agent's active Harbor Cards (§2.2.4), releases any locks and port reservations it held, and, for supervised sessions, notifies the parent agent or the human operator. Because this path observes *committed* events, it detects and compensates; only a pre-commit native guard can prevent a transition in the first place. The Arbiter API provides visibility, not a proof that every security-relevant operation was mediated.

Where the kernel's boundary places this chapter. The controllability theorem of *The Single-Writer Kernel* sorts every rule by whether the event it prohibits is one the mediator can refuse. Card verification is such an event: accept or reject happens on the daemon's own socket, before any effect, so these checks can be enforced at that gate when their inputs are available. A remote revocation that has not arrived is an observation gap, not something a local refusal mechanism can infer. The symbolic queries establish their named checks under the modeled information assumptions. What the Anchor cannot regiment is what a holder does with authority it legitimately has. Exercising a valid `db:write` to write the wrong thing is an uncontrollable event at this boundary; detecting it requires an observation or evaluation rule beyond token validity, and any remedy belongs to the separate audit and slashing mechanisms in *The Bonded Commons*.

2.3.3 Implementation Verification

A secure protocol is useless if the implementation contains buffer overflows or timing leaks. We extracted the critical JWT parsing and cryptographic comparison logic into a verified core.

Design Invariant 2.3.2 (Memory Safety within the Harness Bound).

The **Kani** Rust Verifier harness bounds the token input to 32 bytes of valid UTF-8 containing a dot, with an unwind bound of 128.

Key decoding returns a fixed key; base64 decoding returns either an error or 32 zero bytes; signature verification returns success or failure.

AWS Automated Reasoning Group. Kani Rust Verifier. Online source, 2022.

The reported absence of panics and out-of-bounds accesses applies only to paths reached under these stubs.

In particular, the zero-byte decoded payload is not valid claims JSON: this harness does not exercise successful card acceptance. Longer tokens and the real cryptographic primitives lie outside its claim.

Design Invariant 2.3.3 (Secret-Independent Source Control Flow).

The Rust comparator (`constant_time_compare`) folds the XOR of every byte pair into one accumulator and tests the accumulator once, so no source-level branch depends on secret bytes.

That is a property of the source text, established by inspection; Kani's harness adds only that the comparator cannot panic on any pair of 16-byte inputs.

Neither is a hardware-level constant-time proof: compiler transformations, memory behavior, and microarchitecture remain outside both.

2.3.4 The procedures, precisely

The following pseudocode isolates signature authentication and per-hop rights attenuation. It is deliberately not a complete card-verification API: the expiry, audience, schema, process-binding, and revocation checks remain required. The symbols $\bot$ and $\top$ mean rejection and acceptance. `Encode` names a fixed, unambiguous encoding; the symbolic models in Appendix 2.B abstract its parsing obligations.

To read a chain, keep two keys separate. The daemon signs a root card naming Alice's public key. Alice signs Bob's child card, so that child must be verified under Alice's authenticated key, not the daemon's. Bob's key then verifies Bob's next delegation. Each signature both checks the current link and supplies the authenticated identity needed for the next one.

2.3.4.1 Phase 1: Symmetric HS256 with Algorithm Pinning

Algorithm 2.2 presents the secure verification procedure that is immune to algorithm confusion attacks (CVE-2015-9235).

CVE-2015-9235 2015

Listing 2.2. Algorithm 1: Secure Harbor Verification (HS256 with Algorithm Pinning)

```
Require: Pinned Algorithm: hs256
Require: Master Key: k_master (secret)

function Daemon.IssueToken(agent_id, harbor_id, capability):
    jti <- GenerateNonce()
    msg <- Encode(agent_id, harbor_id, jti, capability)
    sigma <- HMAC-SHA256(msg, k_master)
    emit Issued(agent_id, harbor_id, jti, capability)
    return (hs256, msg, sigma)

function Harbor.VerifyToken(tok, expected_harbor):
    (alg_header, msg, sigma) <- tok
    // CRITICAL: Ignore alg_header, pin to HS256
    if HMAC-SHA256(msg, k_master) != sigma:
        return _|_  // Invalid signature
    (a, h, j, cap) <- Decode(msg)
    if h != expected_harbor:
        return _|_  // Wrong harbor
    emit Accepted(a, h, j, cap)
    return (a, h, j, cap)
```

Numbers by Hand 2.3.4 (Counting the Reads of the Algorithm Header).

Take Algorithm 2.2's `Harbor.VerifyToken` and count, by hand, how often `alg_header` appears after the line that unpacks it.

It is bound once, in the destructuring of `tok`, and read zero times thereafter: the next line computes HMAC-SHA256(msg, k_{master}) under the algorithm fixed by policy and compares that against σ, and no branch below it consults the header at all.

So an attacker who rewrites the header from `hs256` to `none` — the move the algorithm-confusion CVEs catalogued in Appendix 2.F all make in one form or another — has changed one field the procedure consults zero times, and still faces the same comparison, which needs k_{master}.

One binding, zero reads. The naive verifier of `proofs/anchor/token-verify/algconfusion.pv` is the same procedure with that count changed to one, and the one read is what decides its whole verification. [internal]

Security Property: The algorithm ignores the user-provided alg_{header} (line 15), preventing algorithm confusion attacks where an attacker sets $alg =$ "none" or attempts HS256/RS256 confusion.

Tim McLean. Critical Vulnerabilities in JSON Web Token Libraries. Auth0 Blog, 2015. Online source

2.3.4.2 Phase 2: Asymmetric Ed25519

Algorithm 2.3 presents the Ed25519-based verification. This phase removes the need for shared HMAC secrets between distributed Harbors.

Listing 2.3. Algorithm 2: Asymmetric Harbor Verification (Ed25519)

```
Require: Harbor Keypair: (sk_harbor, pk_harbor)
Require: Key Distribution Channel: c_key (private)

function Daemon.IssueToken(agent_id, harbor_id, capability):
    sk_h <- Receive(c_key)  // Get harbor's secret key
    jti <- GenerateNonce()
    msg <- Encode(agent_id, harbor_id, jti, capability)
    sigma <- Ed25519.Sign(msg, sk_h)
    emit Issued(agent_id, harbor_id, jti, capability)
    return (msg, sigma)

function Harbor.VerifyToken(tok, pk_h, expected_harbor):
    (msg, sigma) <- tok
    if not Ed25519.Verify(msg, sigma, pk_h):
        return _|_  // Invalid signature
    (a, h, j, cap) <- Decode(msg)
    if h != expected_harbor:
        return _|_  // Wrong harbor
    emit Accepted(a, h, j, cap)
    return (a, h, j, cap)
```

Security Property: The secrecy of sk_{harbor} and the unforgeability of Ed25519 ensure that only the legitimate Daemon can issue valid Harbor Cards.

Bernstein et al. 2012

2.3.4.3 Phase 3: Multi-hop Delegation

Algorithm 2.4 presents the authorization chain verification, inspired by Macaroons.

Birgisson et al. 2014

Listing 2.4. Algorithm 3: Authorization Chain Verification

```
Require: Daemon Public Key: pk_daemon
Require: Subset Relation: subseteq on capabilities

function Agent.Delegate(token_parent, sk_parent, agent_child,
    ↪ cap_sub):
    (msg_parent, sigma_parent) <- token_parent
    (_, agent_parent, harbor, cap_parent) <- Decode(msg_parent)
    if cap_sub not_subseteq cap_parent:
```

```
        return _|_  // Cannot escalate capabilities
    msg_delegate <- Encode_delegation(msg_parent, sigma_parent,
    ↪ agent_child, cap_sub)
    sigma_delegate <- Ed25519.Sign(msg_delegate, sk_parent)
    emit Delegated(agent_parent, agent_child, cap_sub)
    return (msg_delegate, sigma_delegate)

function Harbor.VerifyChain(chain, pk_daemon):
    tok_root <- chain[0]
    (msg_0, sigma_0) <- tok_root
    if not Ed25519.Verify(msg_0, sigma_0, pk_daemon):
        return _|_  // Invalid root signature
    cap_prev <- ExtractCaps(msg_0)
    pk_current <- AuthenticatedSubjectKey(msg_0)
    for i <- 1 to |chain| - 1:
        (msg_i, sigma_i) <- chain[i]
        if ParentLink(msg_i) != chain[i-1]:
            return _|_  // Child must bind this authenticated parent
        if not Ed25519.Verify(msg_i, sigma_i, pk_current):
            return _|_  // Invalid delegation signature
        cap_i <- ExtractCaps(msg_i)
        if cap_i not_subseteq cap_prev:
            return _|_  // Per-hop capability escalation detected
        cap_prev <- cap_i
        pk_current <- AuthenticatedSubjectKey(msg_i)
    (agent_final, harbor) <- AuthenticatedSubjectAndHarbor(chain)
    emit Accepted(agent_final, harbor, cap_prev)
    return T
```

Security property. Monotonic per-hop attenuation ($\text{cap}_i \subseteq \text{cap}_{i-1}$) prevents capability expansion across transitive delegations, including the `write` → `read` → `write` re-escalation. Property 2.D.2 (§2.D.1) mechanizes the single hop, where the subset guard is necessary and sufficient. Depth beyond one is the per-hop discipline chained by transitivity of $\subseteq$; its mechanized confirmation at depth two is the attack-and-fix pair in the same section, which checks each hop against its immediate parent rather than against the root.

Recall

1. Without looking: name the three layers of Table 2.2, and the two gaps between them that are asserted by a human rather than a tool.
2. What can the Arbiter's post-commit invariants do that ProVerif cannot, and what can they never do that a pre-commit guard could?
3. Why do two fixed capability vectors fail to establish attenuation for every possible input? Which model checks the stated symbolic escalation attempt?

Exercises 2.1–2.2, p. 224.

2.4 One source crate, different verification scopes

What is shared between proof and deployment is the **crate**, not the coverage. The open-source Rust crate `harbor-card-rs` is compiled to a C dynamic library (`cdylib`) and called from the Node.js daemon

via FFI through the daemon's runtime-enforcement (Arbiter) module, and the Kani harnesses are compiled from that same source rather than from a reference model written alongside it.

That is worth having, and it is worth being exact about what it buys. It removes the failure mode that weakens many verification efforts — a specification that drifts from the artifact, because they were never the same artifact. It does not make the proofs coextensive with the code. The harnesses are *bounded*: they stub the cryptography and cap the parser's input at 32 bytes, which is smaller than the 64-byte Ed25519 signature a real token carries, let alone a whole Harbor Card. What Kani establishes is therefore a memory-safety and no-panic result about the token delimiter parser on small inputs with signature verification abstracted away, and the security of the signature scheme rests on the ProVerif model and on Ed25519 itself, not on these harnesses. Section 2.3's status table, its caption, and Exercises 2.3 and 2.4 all say this too; the point of saying it here is that a reader meets the section title first. Kani also checks the source under `cfg(kani)` rather than the shipped shared library, so one further gap — the compiler's — stays open behind all of it.

2.4.1 Core Verification Logic

```rust
pub fn verify(&self, token: &str, now_ts: i64)
    -> Result<HarborCardClaims, HarborError> {
    let parts: Vec<&str> = token.split('.').collect();
    if parts.len() != 3 {
        return Err(HarborError::Malformed);
    }
    let msg = format!("{}.{}", parts[0], parts[1]);
    let mut sig_bytes = Self::internal_decode_b64(parts[2])?;
    let sig_result = self.internal_verify_sig(
        msg.as_bytes(), &sig_bytes);
    sig_bytes.zeroize();  // Scrub signature from memory
    sig_result?;
    let mut payload_bytes = Self::internal_decode_b64(parts[1])?;
    let claims: HarborCardClaims =
        serde_json::from_slice(&payload_bytes)?;
    payload_bytes.zeroize();  // Scrub payload from memory
    if claims.exp < now_ts {
        return Err(HarborError::Expired);
    }
    Ok(claims)
}
```

Listing 2.5. Deployed Rust Implementation — Harbor Card Verifier (abridged from the `harbor-card-rs` crate)

```
22
23 pub fn constant_time_compare(a: &[u8], b: &[u8]) -> bool {
24     if a.len() != b.len() { return false; }
25     let mut result = 0u8;
26     for i in 0..a.len() { result |= a[i] ^ b[i]; }
27     result == 0
28 }
```

Key implementation details: `zeroize` requests that cryptographic material be overwritten when dropped, reducing residual-secret exposure without proving that no copy survives. The comparator avoids source-level early return on secret byte values. That is useful hygiene, not a hardware constant-time guarantee; compiler, cache, and microarchitectural behavior remain outside this source argument.

2.4.2 FFI Bridge to the Node.js Daemon

The Rust crate exports two C-ABI functions consumed by the TypeScript daemon:

Listing 2.6. FFI Exports — Called from the daemon's Arbiter module

```
#[no_mangle]
pub extern "C" fn harbor_constant_time_compare(
    a: *const u8, a_len: usize,
    b: *const u8, b_len: usize
) -> bool { /* ... */ }

#[no_mangle]
pub extern "C" fn harbor_verify_caps_subset_json(
    root_json: *const c_char, root_len: usize,
    sub_json: *const c_char, sub_len: usize
) -> bool { /* ... */ }
```

The daemon's `arbiter` module binds these at startup:

Listing 2.7. TypeScript FFI Binding (daemon-side Arbiter)

```
constantTimeCompare: lib.func(
  'bool harbor_constant_time_compare(
     const uint8_t *a, size_t a_len,
     const uint8_t *b, size_t b_len)'),
verifyCapsSubset: lib.func(
  'bool harbor_verify_caps_subset_json(
     const char *root_json, size_t root_len,
     const char *sub_json, size_t sub_len)')
```

2.4.3 Kani Verification Harnesses

Three bounded model checking proofs are defined in the same source file under `#[cfg(kani)]`:

Listing 2.8. Kani Proof Harnesses (Deployed Source)

```
#[cfg(kani)]
#[kani::proof]
#[kani::stub(HarborCardVerifier::internal_pk_from_bytes,
             stubs::pk_from_bytes_stub)]
#[kani::stub(HarborCardVerifier::internal_decode_b64,
             stubs::decode_b64_stub)]
#[kani::stub(HarborCardVerifier::internal_verify_sig,
             stubs::verify_sig_stub)]
#[kani::unwind(128)]
fn proof_verify_logic_only() {
    let pk_bytes: [u8; 32] = kani::any();
    if let Ok(verifier) = HarborCardVerifier::new(pk_bytes) {
        let token_bytes: [u8; 32] = kani::any();
        if let Ok(token_str) =
            std::str::from_utf8(&token_bytes) {
            kani::assume(token_str.contains('.'));
            let _ = verifier.verify(token_str, 0);
        }
    }
}

#[cfg(kani)]
#[kani::proof]
fn proof_constant_time_behavior() {
    let a: [u8; 16] = kani::any();
    let b: [u8; 16] = kani::any();
    let _ = HarborCardVerifier::constant_time_compare(&a, &b);
}

#[cfg(kani)]
#[kani::proof]
fn proof_capability_attenuation() {
    let root = vec!["read".to_string(), "write".to_string()];
    let mut sub = vec!["read".to_string()];
    assert!(HarborCardVerifier::verify_capability_subset(
        &root, &sub));
    sub.push("admin".to_string());
    assert!(!HarborCardVerifier::verify_capability_subset(
        &root, &sub));
}
```

Numbers by Hand 2.4.1 (What Thirty-Two Symbolic Bytes Can and Cannot Hold).

Read the bounds off the harnesses themselves, since every one of them is a number in the source above. `proof_verify_logic_only` draws `[u8; 32]` for the token, keeps only the inputs that are valid UTF-8 and contain a dot, with an unwind bound of 128 and three functions stubbed.

Now walk `verify` with those 32 bytes in hand: the first guard demands `parts.len() == 3`, so every admitted string with one dot, or with three, leaves at the first `return` — and the survivors, the ones carrying exactly two dots, have

$$32 - 2 = 30 \text{ bytes}$$

to spread across all three segments of what is supposed to be a signed card — too few, as the paragraph below records, to hold a real three-part card at all.

That is the arithmetic behind this chapter's reading of the harness: what it covers is the parser's refusal of malformed input. Even when signature verification returns success, the decoder's zero bytes fail header JSON parsing, before payload and expiry checks.

The other two bounds are just as small and just as visible: `[u8; 16]` for the comparator, and two hard-coded vectors — not a quantifier — for the subset check. [internal]

The stubs keep the symbolic search small, but also determine its reach. `proof_verify_logic_only` checks token splitting, reachable stub-result handling, and rejection at header JSON parsing. It does not reach claims-payload parsing, expiry comparison, or successful acceptance. None of those paths can inherit its no-panic result. The separate `proof_capability_attenuation` harness checks two concrete vectors; the every-hop symbolic attenuation result belongs to the ProVerif model in §2.D.1, not to those two Kani examples.

The three harnesses run under `cargo kani` in the repository's proof workflow; the run log, not a local build directory, is the evidence that they pass.

Exercises 2.3–2.4, p. 224.

2.5 What the adversary can do, and what it cannot

2.5.1 Threats this takes seriously

We assume a **Dolev-Yao adversary** who: Dolev & Yao 1983

- Controls the entire network
- Can intercept, modify, and inject messages
- Cannot break cryptographic primitives (cryptography is perfect)
- May corrupt legitimate agents (but not all simultaneously)

This is the standard threat model for symbolic protocol analysis. Figures 2.3 and 2.5 mark adversarial message boundaries with dashed bands. Figure 2.7 shows a different boundary: local enforcement cannot guarantee delivery to a remote verifier. The remaining diagrams show their named checks or state transitions, not a claim that every surrounding channel is trusted. Blanchet 2016

Table 2.4 closes the loop on the three failure modes of §2.1: each is named there, addressed somewhere in between, and carries a residual risk stated here rather than left for the reader to reconstruct.

Table 2.4. Each failure mode from §2.1 has a named defense and a residual boundary. A detector, an access check, and a storage invariant provide different guarantees.

Threat (§2.1)	Closed where	By what mechanism	Residual risk
Port squatting ("Ghost in the Harbor")	§2.3, Arbiter invariant `PID_SQUATTING`	Cards bind to the connecting process via the daemon's socket-credential check; the Arbiter flags a port claimed by a PID other than its holder	Detection is post-commit, and the binding is an OS check outside the ProVerif model. See §2.5.3, "A program is not a person"
Resource contention (shared-state corruption)	App. 2.E; SQLite single-writer	One writer per harbor, transaction durability policy, and `LOCK_OWNER_VALID` in the Arbiter	Enforced by the storage layer, not proved in this chapter; measurable I/O cost
Privilege escalation across delegation	§2.3.4.3; Property 2.D.2	Per-hop capability attenuation, checked at issuance and re-checked at verification, mechanized against an insider escalation adversary	Mechanized at depth 3 and single-hop for the enriched order; depth is a spawn-time policy check, not a verification-time invariant

2.5.2 Security Properties

Property	Verified	Tool	Deployed	Reference
Master Key Secrecy	Yes	ProVerif	—	
Algorithm Confusion Immunity	Yes	ProVerif	—	
Authentication correspondence	Yes	ProVerif	—	
Delegation Integrity	Yes	ProVerif	—	
No-panic parsing (bounded)	Bounded	Kani	Yes (FFI)	
Secret-independent source branching	By inspection	—	Yes (FFI)	
Capability subset check	Two vectors	Kani	Yes (FFI)	

Table 2.5. ProVerif checks the model. Kani checks bounded crate harnesses: a 32-byte parser with stubbed cryptography, a 16-byte comparator, and two subset vectors. Neither alone proves the deployed implementation correct.

Blanchet 2016

McLean 2015

Lowe. *A Hierarchy of Authentication Specifications*. IEEE CSFW, 1997.

Birgisson et al. 2014

AWS Automated Reasoning Group 2022

Billy Bob Brumley and Nicola Tuveri. Remote Timing Attacks are Still Practical. In *European Symposium on Research in Computer Security (ESORICS)*, 2011.

2.5.3 Where this stops

Four boundaries follow, and the third of them carries an arithmetic worth doing before the prose, because the size of the hole is the whole argument for closing it at the transport rather than in the protocol.

Numbers by Hand 2.5.1 (Fourteen Minutes of Somebody Else's Authority).

A card is issued with a 15-minute TTL and sniffed off loopback at minute 1 by an attacker assumed able to observe that transport. Subtract:

$$15 - 1 = 14 \text{ minutes}$$

in which the sniffer holds the victim agent's full capability set, with every cryptographic check passing, because a bearer

credential is usable to whoever holds the bytes unless an additional binding rejects that holder.

Revocation can shorten this window after the theft is noticed and the update reaches a verifier. Neither delay is bounded here; Figure 2.7 shows why local revocation alone is not enough.

Transport access control addresses the earlier failure: use a socket the other user cannot open. Faster gossip does not prevent credential theft. [internal]

1. **Delegation-chain depth.** The symbolic proof of delegation soundness (Table 2.6, "`chain-replay.pv`") is machine-checked at the fixed model depth 3; it is *not* a proof for unbounded chain length. ProVerif's unbounded-session guarantee quantifies over the number of protocol *sessions*, not over the number of hops in a single authorization chain, which is a fixed structural bound in the modeled process. Extending the proof to unbounded depth (via recursive replication of the delegation process) is future work. The current depth bound is a *spawn-time* policy check (`assessDelegation` in `lib/tube-spawner-router.ts`, `HARD_MAX_DELEGATION_DEPTH` = 8, default 4) applied when a spawn request arrives; it is *not* a verification-time invariant. The cryptographic chain verifier (`lib/delegation-chain.ts`) accepts arbitrary chain depth, and the attenuation monitor (`lib/cap-attenuation-monitor.ts`) enforces only per-hop capability monotonicity—not total depth. Machine-enforced depth at verification time is a planned hardening item.

2. **A program is not a person.** A Harbor Card authenticates a *capability holder*, not a human and not, by itself, a process. The "Ghost in the Harbor" scenario of §2.1 is really a PID-binding gap: nothing in the token cryptography stops a *different* process from presenting a card whose original bearer has died, because the card says what may be done, not who is doing it. Port Daddy binds cards to the connecting process outside the token, via the daemon's socket-credential check at connection time (`SO_PEERCRED` on Linux, `LOCAL_PEERCRED`/`getpeereid` on macOS). Two consequences follow, and both are boundaries rather than guarantees. First, this is enforced by the operating system, not proved by ProVerif: the symbolic model

has no notion of a process, so a daemon build that skips or mis-orders the check is unprotected regardless of card validity, and no query in Table 2.3 would notice. Second, the check is only as strong as what it compares. Walk the concrete case: Agent A holds a card and is running as PID 4021. It crashes at t. Three seconds later the OS reuses 4021 for an unrelated Agent B, which presents A's card scavenged from a shared temp file. A credential check that compares *only* the numeric PID admits B — the numbers match. The deployed check therefore compares the peer credential against the socket-connection identity established at card issuance and re-validates process start-time, so a reused PID with a later start-time is rejected; a build that compares the bare PID has an open reuse race. That start-time comparison is a runtime conformance test, not a mechanized proof, and it is listed as such rather than as closed.

3. **`localhost` is a real network.** `localhost` is not a trust boundary. On a shared or multi-user machine any local process — including one belonging to a different user, or a browser page pointed at `127.0.0.1` — can open a TCP connection to a loopback port, and an unprotected transport exposes bearer credentials to any attacker able to observe that traffic. Merely having a different local account does not establish packet-capture capability; the transport threat model must say which access the attacker has. Concretely, a card with a 15-minute TTL sniffed off loopback at minute 1 grants its holder fourteen minutes of the victim agent's full capability set, and revocation (§2.2.4) can shorten that window only after somebody notices. The recommended deployment is a Unix domain socket whose parent directory is mode `0700` and owned by the daemon's own user, so that the filesystem, not the protocol, excludes other users; loopback TLS with a pinned daemon certificate is the alternative where cross-user sharing is genuinely required. The Anchor Protocol does not currently *enforce* either: the daemon will speak to whatever transport it is configured with. This is therefore an operational obligation on the deployer, and one of the few places in this chapter where a wrong configuration silently voids a property the rest of the chapter proves.

4. **Algorithm confusion at the token-encoding layer.** The delegation models of §2.3 target chain authenticity and capability-subset soundness; attacks on JWT deployments also target the encoding underneath. RFC 8725 catalogues the `alg:none` downgrade and the confusion that makes a verifier treat an asymmetric *public* key as an HMAC secret. `proofs/anchor/token-verify/algconfusion.pv` compares a pinned and a naive verifier under an adversary given the second algorithm's symmetric key. *Model-checked property:* a verifier pinned to ALG_A accepts a token only if an honest ALG_A issuer minted it — `accepted_pinned(m)` $\Longrightarrow$ `issued_A(m)` is TRUE. The naive verifier lets the token select its verification routine and fails the same query: the attacker knows k_B, computes $\mathrm{hmac}_B(k_B, m)$ over a payload no issuer signed, wraps it as `tokenB(ALG_B, m, …)`, and triggers `accepted_naive(m)`. Pinning fixes the algorithm by policy rather than by attacker-controlled input. The model does not verify the decoder: its distinct signature types cannot be silently coerced into one another, and real parsers must enforce that separation. The bounded Kani harness in Table 2.5 checks only its reachable stubbed-input paths; it does not discharge that decoder obligation.

Sheffer et al. 2020

Recall

1. Without looking: state the four properties of the Dolev-Yao adversary this chapter assumes.
2. Of the three failure modes named in §2.1, which one's residual risk is enforced by the storage layer rather than proved by any tool in this chapter?
3. Why is the PID-reuse fix a runtime conformance test rather than a ProVerif theorem?

Exercise 2.5, p. 225.

2.6 What the verification covers

A proof helps only where its model matches the implementation and its assumptions hold. A symbolic pinning proof does not remove the parser's validation obligations. The Kani harnesses are bounded and use the `harbor-card-rs` crate with the substitutions listed in Table 2.5; the ProVerif attenuation model explicitly admits an insider's escalation attempt before checking that the verifier rejects it (Appendix 2.D, §2.D.1). A model that cannot express the attempt cannot establish that boundary. Claims whose mechanization is partial or absent remain partial or *open* in Table 2.6.

Numbers by Hand 2.6.1 (pinning is not skipping validation).

The Rust `verify` function first checks the token's three-part structure and verifies its signature with the fixed Ed25519 verifier. It then decodes the authenticated header and explicitly rejects `header.alg != SUPPORTED_ALG`. This is validation against policy, not permission for the token to choose another verification algorithm.

The payload must also decode, have the supported version, and pass the issued-at, expiry and applicable not-before checks.

Channel authorization is an additional check in `verify_for_channel`; revocation belongs to the caller's store. An authentic, unexpired card is not necessarily authorized for this channel or still unrevoked.

Design invariant 2.2.2 therefore does not justify deleting the header check.

Nor does the bounded Kani harness exercise all these guards: its zero-byte decode reaches header JSON rejection before the algorithm or payload checks. This account is checked against the source, not a new runtime or model-checking result. [internal]

The assurance case combines:

- Symbolic protocol proofs (ProVerif) — Blanchet 2016
- Memory-safe implementation (Rust; bounded Kani harnesses) — AWS Automated Reasoning Group 2022
- Capability-based delegation (inspired by Macaroons) — Birgisson et al. 2014

These artifacts support the modeled and tested surfaces. They do not certify the entire daemon, every delegation depth, or every side channel; §2.5.3 lists those limits.

Chapter Appendices

2.A Formal Verification Status

Artifact availability. Every artifact named in Table 2.6 is bundled at `https://portdaddy.dev/whitepaper/artifacts/`. The verified Rust core is the open-source `harbor-card-rs` crate; runtime components are deployed inside the Port Daddy daemon binary. *The Bonded Commons* carries the cross-paper status table; this appendix lists only the items that pertain specifically to the Anchor Protocol.

Numbers by Hand 2.A.1 (Adding Up the Ledger, and What One Promotion Would Cost).

Count the rows of Table 2.6 by tier: six **closed**, five partial, one *open*, and

$$6 + 5 + 1 = 12,$$

which is every claim the table carries — a small enough number that a reader can check the caption against the body rather than take it.

Then run the experiment the tiers exist for. Promote one row, "constant-time signature comparison," from partial to **closed**: the headline becomes seven of twelve, an improvement of one row and a claim of a side-channel result nobody in this chapter has — the property is inherited from an audited library, and §2.5.3 says compiler, cache and microarchitecture stay outside it.

That is the whole function of the middle tier. It is where a claim sits when the honest verb is "tested" and the tempting one is "proved." [internal]

Claim	Method	Result	Artifact
Authentication correspondence in the listed phase models (§2.3.4)	ProVerif 2.05	**closed** non-injective correspondence queries TRUE	`harbor_card_v*.pv`
Algorithm-confusion immunity	ProVerif pinned vs. naive	**closed** pinned TRUE; counter-trace witnessed	`alg confusion.pv`
Delegation chain authenticity (§2.3.4.3)	ProVerif at depth 3	**closed** chain authenticity TRUE; 17-case runtime conformance	`chain-replay.pv`
Cuckoo-filter pollution resistance (§2.2.4)	5-invariant property test	**closed** FP rate within $2B/2^f$ at load 0.95	cuckoo property tests
GitHub-App webhook origin authentication	ProVerif (Dolev-Yao relay)	**closed** origin auth TRUE; daemon re-verifies GitHub HMAC (runtime conformance)	`ingress_origin_auth.pv`
Multi-tenant relay namespace isolation	ProVerif + runtime	**closed** member of A cannot publish into B; `HARBOR_MISMATCH` conformance	`ingress_tenant_isolation.pv`
Cuckoo-filter revocation soundness	inspection + empirical	partial gate narrows acceptance; dissemination remains assumption-bound	runtime test
Constant-time signature comparison	audited `subtle::ConstantTimeEq` library	partial no side-channel proof; library trust documented	—
Relay payload secrecy via HPKE (daemon key pinned)	ProVerif (design)	partial secrecy TRUE under pinned key; seal not yet implemented	`ingress_hpke_secrecy.pv`
Webhook replay-freedom + in-order delivery	TLA+ / TLC (exhaustive at bound)	partial design model-checked; runtime via ordered-chain ingest	`WebhookDelivery.tla`
Card revocation finality when restoring pre-revocation state	TLA+ / TLC	partial rollback attack + revocation-epoch mitigation both model-checked	`CardRevocation.tla`

Table 2.6.
The twelve rows carry six **closed**, five partial, and one *open*marker. These summarize the named evidence; they are not twelve interchangeable proofs of system security. **closed**: model verified *and* runtime conformance test passes. partial: design verified, runtime empirically tested rather than proved. *open*: known unmechanized.

(continued)

Claim	Method	Result	Artifact
Mechanized gossip-convergence bound	—	*open* standard expected asymptotics only; no partition deadline	—

Demers et al. 1987

Limitations. Gossiping the Append-Only Event Log avoids the unsound filter merge described in §2.2.4 ("Why filters cannot be merged bitwise"), but cross-peer gossip propagation still leans on the standard anti-entropy analysis rather than a mechanized proof. The side-channel resistance of signature comparison is inherited from an audited library rather than re-proved. These remain open obligations outside the present claims.

GitHub-App relay ingress. The webhook-dispatch surface (GitHub → untrusted relay → daemon) is modeled with the relay as the Dolev-Yao attacker, in three relay-agnostic properties, each paired with a negative control (`*_vuln.pv`) that exhibits the attack once the mitigation is removed—so the controls test whether removing each mitigation admits the named attack. (i) *Origin authentication*: the daemon dispatches a fleet event only if GitHub signed it, even when the forwarder's transport credential is attacker-held; the control omits the daemon-side HMAC re-verification and the property fails (forged dispatch). This is *non-injective* agreement (origin), not replay-freedom: a captured genuine $(payload, mac)$ may be re-delivered. The displayed non-injective query does not exclude duplicate delivery. This project checks its separate ordering and replay property in TLA+ (`proofs/relay/WebhookDelivery.tla`): a daemon consuming the *ordered* per-publisher chain (dispatch iff $seq = tip + 1$) is model-checked at-most-once *and* gap-free against an adversarial relay that reorders, duplicates, and redelivers, with the unguarded daemon as the model-checked counterexample (TLC exhibits a double-dispatch). The obligation it names: the daemon ingests webhooks over the relay's ordered `from_seq` subscribe path, not the raw forward. (ii) *Payload secrecy*: under HPKE to the daemon's *pinned* X25519 key the relay never learns plaintext; the control sources the key from the relay and the secrecy property collapses to a key-substitution MITM—hence out-of-band key pinning is a requirement, not a recommendation. The seal is specified but

not yet implemented, so this row is partial. (iii) *Namespace isolation*: a member of tenant A cannot publish into tenant B because harbor binding is checked against authoritative membership; the control trusts the card's self-declared issuer and crosses the tenant boundary. Two limits: HMAC-SHA256 gives origin authentication to a shared-key holder (EUF-CMA), not third-party non-repudiation; HPKE base mode is assumed IND-CCA2.

We err toward listing fewer items as closed rather than more.

2.B Full ProVerif Models

For completeness, we include the full ProVerif models used for verification. These correspond to the algorithms in Section 2.3.4. Before reading the first one, it is worth counting its destructor rules, because that count is what separates a proof from a tautology.

Numbers by Hand 2.B.1 (Two Rules Offered, One Rule Used).

The v1 model below declares `verify_generic` with two reduction rules: one that succeeds when the signature is hs256(m, k) and the header says `hs256_alg`, and a second, `verify_generic(m, none_alg, m, k) = true`, which succeeds for *any* message whose header says `none_alg` and whose "signature" is the message itself. That second rule is the `alg:none` downgrade written into the model on purpose — the attack is expressible here, not assumed away.

Now count the rules the honest verifier actually calls: `SecureHarbor` invokes `check_hs256`, which has exactly one rule and no header parameter at all, so the header it received is destructured and then never consulted. Two rules offered, one used, and the unused one is the one an attacker would need.

The TRUE verdict in Table 2.3 is therefore a statement about the verifier's structure rather than about a rule the model forgot to write down. [internal]

2.B.1 Phase 1: Symmetric HS256

Listing 2.9. harbor_card_v1_refined.pv

```
(* Port Daddy: Harbor Card v1 (HS256) Formal Model *)
type id. type key. type jti. type capability. type alg_type.
const hs256_alg: alg_type. const none_alg: alg_type.
free c: channel.

fun hs256(bitstring, key): bitstring.
reduc forall m: bitstring, k: key;
  check_hs256(m, k, hs256(m, k)) = true.

reduc forall m: bitstring, k: key;
  verify_generic(m, hs256_alg, hs256(m, k), k) = true;
  forall m: bitstring, k: key;
  verify_generic(m, none_alg, m, k) = true.

fun encode_token(id, id, jti, capability): bitstring [data].

event Issued(id, id, jti, capability).
event Accepted(id, id, jti, capability).

free master_key: key [private].
query attacker(master_key).

query a: id, h: id, j: jti, cap: capability;
  event(Accepted(a, h, j, cap)) ==> event(Issued(a, h, j, cap)).

let Daemon(k: key, a_id: id, h_id: id, j: jti, cap: capability) =
  let msg = encode_token(a_id, h_id, j, cap) in
  let signature = hs256(msg, k) in
  event Issued(a_id, h_id, j, cap);
  out(c, (hs256_alg, msg, signature)).

let SecureHarbor(k: key, expected_h: id) =
  in(c, (alg_header: alg_type, msg: bitstring, signature: bitstring)
  ↪ );
  if check_hs256(msg, k, signature) = true then
    let encode_token(a: id, h: id, j: jti, cap: capability) = msg
  ↪ in
    if h = expected_h then
      event Accepted(a, h, j, cap).

process
  new agent_id: id; new harbor_id: id;
  new token_jti: jti; new secret_cap: capability;
  (!Daemon(master_key, agent_id, harbor_id, token_jti, secret_cap)
  ↪ |
   !SecureHarbor(master_key, harbor_id))
```

2.C Phase 2: Asymmetric Ed25519

The v2 model's secrecy result is carried by one keyword, and the quickest way to see that is to inventory what the main process hands the attacker.

Numbers by Hand 2.C.1 (One Keyword between a Secret Key and the Attacker).

Count the outputs in the model below. The main process performs exactly one `out` on the public channel `c` — `out(c, harbor_pk)`, the public key, which the attacker is *meant* to have — and one replicated `out(k_dist, secret_key)`, which puts the private key on a channel in the clear.

Count the destructors: there is one, `check_sign`, and no rule anywhere inverts `pk(·)`, so from `harbor_pk` the attacker derives nothing. So the query `attacker(secret_key)` turns entirely on whether `k_dist` is reachable, and the only thing making it unreachable is the word `[private]` in its declaration.

No run is needed to see the counterfactual: delete that one word and the replicated output publishes the key directly, and the secrecy query fails on the first step.

A model can be this close to failing and still be sound — what it cannot be is *accidentally* sound, which is why the key-distribution channel is declared rather than assumed. [internal]

```
(* Port Daddy: Harbor Card v2 (Ed25519 / EdDSA) Formal Model *)
type id. type skey. type pkey. type jti. type capability.

free c: channel.
free k_dist: channel [private]. (* Trusted key distribution *)

(*** Digital Signatures (Ed25519) ***)
fun pk(skey): pkey.
fun sign(bitstring, skey): bitstring.
reduc forall m: bitstring, k: skey;
  check_sign(m, pk(k), sign(m, k)) = true.

fun encode_token(id, id, jti, capability): bitstring [data].

(*** Events ***)
event Issued(id, id, jti, capability).
event Accepted(id, id, jti, capability).
```

Listing 2.10. `harbor_card_v2_asymmetric.pv` — Verified in ProVerif 2.05

```
18
19 (*** Queries ***)
20 free secret_key: skey [private].
21 query attacker(secret_key).
22
23 query a: id, h: id, j: jti, cap: capability;
24   event(Accepted(a, h, j, cap)) ==> event(Issued(a, h, j, cap)).
25
26 (*** Processes ***)
27 let Daemon(a_id: id, h_id: id, j: jti, cap: capability) =
28   in(k_dist, sk_h: skey);
29   let msg = encode_token(a_id, h_id, j, cap) in
30   let signature = sign(msg, sk_h) in
31   event Issued(a_id, h_id, j, cap);
32   out(c, (msg, signature)).
33
34 let Harbor(pk_h: pkey, expected_h: id) =
35   in(c, (msg: bitstring, signature: bitstring));
36   if check_sign(msg, pk_h, signature) = true then
37     let encode_token(a: id, h: id, j_1: jti, cap_1: capability)
38       = msg in
39     if h = expected_h then
40       event Accepted(a, h, j_1, cap_1).
41
42 (*** Main Process ***)
43 process
44   new agent_id: id; new harbor_id: id;
45   new token_jti: jti; new secret_cap: capability;
46   let harbor_pk = pk(secret_key) in
47   out(c, harbor_pk);
48   (
49     (!Daemon(agent_id, harbor_id, token_jti, secret_cap)) |
50     (!Harbor(harbor_pk, harbor_id)) |
51     (!out(k_dist, secret_key))
52   )
```

2.D Phase 3: Multi-hop Delegation

Listing 2.11. `harbor_card_v3_delegation.pv` — reflexive-order model (see §2.D.1 for the soundness caveat and the enriched v5 proof)

```
1 (* Port Daddy: Harbor Card v3 (Delegation Chains) Formal Model *)
2 type id. type skey. type pkey. type capability.
3
4 free c: channel.
5
6 (*** Cryptographic Functions ***)
7 fun pk(skey): pkey.
8 fun sign(bitstring, skey): bitstring.
9 reduc forall m: bitstring, k: skey;
10   check_sign(m, pk(k), sign(m, k)) = true.
```

```
11
12 reduc forall c1: capability; is_subset(c1, c1) = true.
13
14 fun base_token(id, id, id, capability): bitstring [data].
15 fun delegate_token(bitstring, id, capability): bitstring [data].
16
17 (*** Events ***)
18 event IssuedRoot(id, id, capability).
19 event Delegated(id, id, capability).
20 event Accepted(id, id, capability).
21
22 (*** Queries ***)
23 free harbor_id: id.
24 query b: id, cap: capability, a: id;
25   event(Accepted(b, harbor_id, cap)) ==>
26     (event(IssuedRoot(a, harbor_id, cap))
27      && event(Delegated(a, b, cap)))
28     || event(IssuedRoot(b, harbor_id, cap)).
29
30 (*** Processes ***)
31 free daemon_id: id.
32 free daemon_sk: skey [private].
33
34 let Daemon(a: id, cap: capability) =
35   let msg = base_token(daemon_id, a, harbor_id, cap) in
36   let s = sign(msg, daemon_sk) in
37   event IssuedRoot(a, harbor_id, cap);
38   out(c, (msg, s)).
39
40 let AgentA(a: id, a_sk: skey, b: id, sub_cap: capability) =
41   in(c, (msg: bitstring, s: bitstring));
42   let base_token(=daemon_id, =a, =harbor_id,
43     root_cap: capability) = msg in
44   if check_sign(msg, pk(daemon_sk), s) = true then
45     if is_subset(sub_cap, root_cap) = true then
46       let d_msg = delegate_token((msg, s), b, sub_cap) in
47       let d_s = sign(d_msg, a_sk) in
48       event Delegated(a, b, sub_cap);
49       out(c, (d_msg, d_s)).
50
51 let Harbor(a_pk: pkey) =
52   in(c, (d_msg: bitstring, d_s: bitstring));
53   let delegate_token((base_msg: bitstring, base_s: bitstring),
54     b: id, sub_cap: capability) = d_msg in
55   let base_token(=daemon_id, a: id, =harbor_id,
56     root_cap: capability) = base_msg in
57   if check_sign(base_msg, pk(daemon_sk), base_s) = true then
58   if check_sign(d_msg, a_pk, d_s) = true then
59   if is_subset(sub_cap, root_cap) = true then
60     event Accepted(b, harbor_id, sub_cap).
61
```

```
(*** Main Process ***)
process
  new sk_a: skey;
  let pk_a = pk(sk_a) in
  out(c, pk_a);
  new id_a: id; new id_b: id; new cap_root: capability;
  (
    (!Daemon(id_a, cap_root)) |
    (!AgentA(id_a, sk_a, id_b, cap_root)) |
    (!Harbor(pk_a))
  )
```

2.D.1 Soundness of the Attenuation Proof (v5)

Numbers by Hand 2.D.1 (Enumerating What the Capability Order Can Derive).

Take the v3 order — one rule, `is_subset(c1, c1) = true` — and two capabilities, `cap_read` and `cap_write`. There are four ordered pairs to ask about, and the rule derives exactly two of them, both reflexive; neither of the two pairs relating *distinct* capabilities is derivable, in either direction.

So the verifier's guard `is_subset(sub_cap, root_cap)` can only ever pass on two identical terms, and an escalating delegation is not something the model rejects — it is something the model cannot express.

Now count again under v5, which adds `otherwise is_subset(cap_read, cap_write) = true`: three of the four pairs derive, and the fourth, `is_subset(cap_write, cap_read)`, is precisely the fact the insider adversary needs and cannot get.

Two derivable facts before, three after, one deliberately withheld — and the gap between the two counts is the whole difference between a query that holds because the protocol works and a query that holds because nothing else was sayable. [internal]

The v3 model above is faithful to the cryptographic structure but *vacuous about attenuation*. Its capability order is reflexive only:

```
reduc forall c1: capability; is_subset(c1, c1) = true.
```

Capabilities are opaque atoms with no order, so a delegated capability can only ever *equal* its parent. Escalation—delegating more

authority than you hold—is not blocked; it is *inexpressible*. The no-escalation query therefore holds for a reason that has nothing to do with the protocol: there is no larger capability to delegate. A meaningful attenuation check needs a model in which an insider can actually attempt the forbidden delegation.

`harbor_card_v5_attenuation.pv` closes the obligation by enrichment, not by weakening the claim. It gives capabilities a real partial order (`cap_read ⊏ cap_write`, both public so the attacker can construct either), replaces the reflexive stub with the sound order (`is_subset(cap_read, cap_write)` derivable; `is_subset(cap_write, cap_read)` *not*), and adds an *insider escalation adversary*: an agent holding a valid `cap_read` root token that signs an over-broad `cap_write` delegation with no self-restraint. The verifier's `is_subset` guard is the sole defense under test.

Listing 2.12. `harbor_card_v5_attenuation.pv` (excerpt) — Verified in ProVerif 2.05

```
(* Sound order: reflexive + read <= write; write <= read NOT derivable *)
fun is_subset(capability, capability): bool
reduc
    forall x: capability; is_subset(x, x) = true
    otherwise is_subset(cap_read, cap_write) = true.

(* Insider adversary: holds a real root, signs an over-broad delegation *)
let MaliciousA(a: id, a_sk: skey, b: id) =
  in(c, (msg: bitstring, s: bitstring));
  let base_token(=daemon_id, =a, =harbor_id, root_cap: capability) = msg in
  if check_sign(msg, pk(daemon_sk), s) = true then
    event EscalationAttempted(a, root_cap, cap_write);
    let d_msg = delegate_token((msg, s), b, cap_write) in
    out(c, (d_msg, sign(d_msg, a_sk))).

(* Q1 soundness: a write-card delegated from a read-root is NEVER accepted *)
query b: id; event(Accepted(b, harbor_id, cap_write, cap_read)) ==> false.
(* Q2 non-vacuity: the escalation IS reachable (the proof is not vacuous) *)
query a: id, r: capability, s: capability;
  event(EscalationAttempted(a, r, s)) ==> false.
```

Model-Checked Property 2.D.2 (Attenuation Soundness, Mechanized).

In `harbor_card_v5_attenuation.pv`, ProVerif 2.05 reports `Accepted(b, harbor, cap_write, cap_read)` unreachable (**Q1 = true**): no escalating delegation is ever accepted.

Simultaneously `EscalationAttempted` is reachable (**Q2 = false-with-trace**): the attack is expressible, so Q1 is a real defense, not the v3 vacuum.

A negative control that deletes the verifier's `is_subset` guard flips Q1 to false-with-trace—ProVerif exhibits the read→write escalation— confirming the guard is necessary.

The attack model checks only against the root; the guarded model checks each immediate parent. Their committed verdicts expose the consequence of that single difference:

Listing 2.13. the multi-hop attack on version 6, and version 7 closing it

```
>_
$ proverif analyses/harbor_card_v6_multihop_attack.pv
-- Query not event(Accepted(cc_2,harbor_id[],
   ↪ cap_write)) in process 1.
goal reachable: event(Accepted(id_c[],harbor_id[],
   ↪ cap_write))

  ... derivation, 11 steps, elided; the trace is in
   ↪ analyses/harbor_card_v6_results.txt ...

A trace has been found.
RESULT not event(Accepted(cc_2,harbor_id[],cap_write))
   ↪  is false.

$ proverif analyses/harbor_card_v7_multihop_fixed.pv
RESULT not event(Accepted(cc_2,harbor_id[],cap_write))
   ↪  is true.
```

Multi-hop scope. Property 2.D.2 is single-hop (Daemon → A → verifier). Multi-hop delegation has a sharper obligation: a verifier that checks the *final* capability against the *root* accepts a non-monotonic middle hop. `harbor_card_v6_multihop_attack.pv` mechanizes exactly this — A:`write` → B:`read` → C:`write` is accepted (ProVerif: escalation reachable). The fix, `harbor_card_v7_`

`multihop_fixed.pv`, verifies each hop against its *immediate parent* (is_subset(*final, mid*) ∧ is_subset(*mid, root*)) and ProVerif proves the same chain rejected. The runtime delegation verifier — and the cross-harbor recompute $att_B \circ att_A$ — must be per-hop, never final-vs-root. Figure 2.10 draws the same chain under both verifiers side by side: v6's single final-vs-root check and v7's two hop-vs-parent checks, so the one comparison that moves is visible rather than implied by the file names.

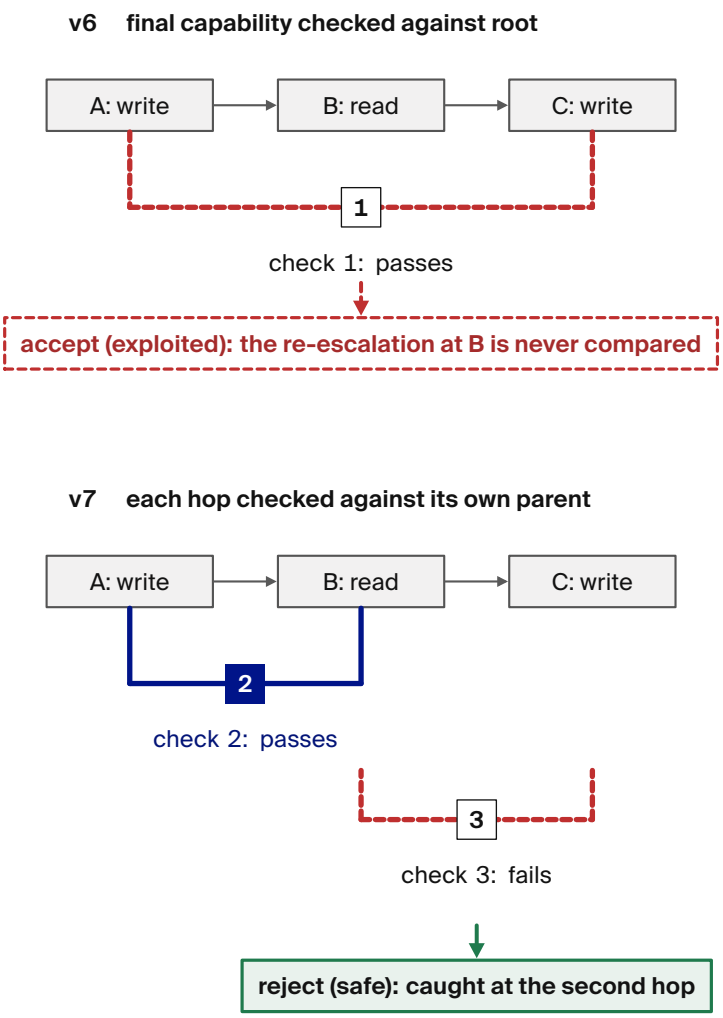


Figure 2.10. Checking only against the root misses re-escalation after an intermediate restriction. v7 checks each immediate parent and rejects the same chain that v6 admits. [internal; ProVerif multihop models]

Exercises 2.6–2.9, p. 225.

2.E Limitations & Boundaries

The formal mechanisms presented in this chapter are mathematically sound within their defined scopes, but they depend on bounding assumptions that must be explicitly acknowledged. The third bullet's maturity gap is the easiest one to misread, so here it is as four numbers that do not agree with each other.

Numbers by Hand 2.E.1 (Four Delegation Depths, and the One That Is Not Checked).

Line them up. The ProVerif mechanization runs at depth 3. The spawn-time policy check in `lib/tube-spawner-router.ts` defaults to 4 and caps at `HARD_MAX_DELEGATION_DEPTH` = 8. The cryptographic chain verifier in `lib/delegation-chain.ts` accepts any depth whatsoever, and the attenuation monitor checks per-hop monotonicity and nothing about length.

Now take a chain of depth 5:

$$3 < 4 < 5 < 8,$$

so it is above what any model has been checked at, above the spawn default, below the hard cap — and if it reaches a verifier by any route other than a spawn request, it meets no depth check at all, because the verifier has none to meet.

The gap a reader should carry away is not the distance between 4 and 8; it is that depth is enforced at spawn time and nowhere else, which makes it a policy the daemon applies rather than an invariant the verifier maintains. [internal]

- **Threat Model Exclusions:** Collusion across all independent subsystems (e.g., the logging daemon, the arbitrator, and the primary execution runtime) is assumed out-of-scope; if all enforcing components are compromised, the guarantees degrade to advisory.
- **Performance Trade-offs:** Cryptographic checks and the chosen transaction-durability policy incur computation and I/O costs. These mechanisms are designed for high-stakes coordination, not for bulk data processing or high-frequency trading.

- **Implementation Maturity:** While the core protocol (Harbor Cards, delegation, revocation) is implemented and running in production, the unbounded-depth delegation proof, machine-enforced verification-time chain-depth limits, and mandatory transport-boundary enforcement discussed in §2.5.3 remain partial or designed.

These constraints are not flaws but structural realities of building zero-trust coordination on top of POSIX primitives.

2.F Further reading and prior art

This appendix traces the lineage the main text points at in §2.1. It is placed here rather than between the protocol description and the verification argument so that the chapter's narrative thread is not cut in half by a literature review; nothing in §§2.2–2.6 depends on reading it. The most immediately consequential subsection — the credential-format migration now under way — comes first.

2.F.1 Credential Formats in Agentic Commerce

The interoperability target is the credential profile used by the Agent Payments Protocol, adopted by the Universal Commerce Protocol (Google/Shopify, 2026), carries its authorization mandates as W3C Verifiable Credentials in SD-JWT form — SD-JWT+kb for principal proof, JWS detached-content signatures (RFC 7515 Appendix F) over JCS-canonicalized payloads (RFC 8785) for counterparty authorization, with ES256 as the recommended algorithm and keys published as JWK sets in a `/.well-known` profile. A proposed Harbor Card envelope (`hv: 3`, specified) would use this profile to support external verifiers without a bespoke SDK. It is not the deployed wire format: the Rust verifier in `core/harbor-card-rs/src/lib.rs` accepts `hv: 2`. The proposal changes the envelope, not the intended delegation-chain semantics or the attenuation obligation of Property 2.D.2. Its serialization and SD-JWT+kb key binding still require implementation conformance and a matching proof obligation; the existing ProVerif results do not certify that envelope. Strict algorithm whitelisting and key separation remain requirements for the proposed JWT profile, not evidence that the format change has shipped.

Agent Payments Protocol (AP2): *Mandates*, 2026. Source.

Universal Commerce Protocol (Google, Shopify, et al.). *Specification and reference implementation (Apache-2.0)*, 2026. Source.

2.F.2 Formal Verification of Security Protocols

Barbosa et al. *SoK: Computer-Aided Cryptography*. IEEE S&P, 2021.

Lowe 1997

Dolev & Yao 1983

Blanchet 2016

Kobeissi, Bhargavan & Blanchet. *Automated Verification for Secure Messaging Protocols and Their Implementations: A Symbolic and Computational Approach*. IEEE EuroS&P, 2017.

Blanchet, Cheval & Cortier. *ProVerif with Lemmas, Induction, Fast Subsumption, and Much More*. IEEE S&P, 2022.

Meier et al. *The TAMARIN Prover for the Symbolic Analysis of Security Protocols*. CAV, 2013.

Cremers et al. *A Comprehensive Symbolic Analysis of TLS 1.3*. ACM CCS, 2017.

Basin et al. *A Formal Analysis of 5G Authentication*. ACM CCS, 2018.

Blanchet. *CryptoVerif: A Computationally Sound Security Protocol Verifier*. INRIA Research Report, 2007.

Barbosa et al. survey the computer-aided-cryptography landscape and the symbolic/computational distinction used below. Formal verification of cryptographic protocols asks a narrower question than "is the implementation secure?" It asks whether a stated property follows from a stated model of principals, messages, and an attacker. Lowe's work on authentication hierarchies supplies a vocabulary for that first question, while the Dolev–Yao model idealizes cryptography so that the analysis can concentrate on message flow. Those are modeling foundations, not evidence that the deployed daemon obeys their assumptions; the distinction matters for the bounded implementation claims made later in this chapter.

The ProVerif line of work asks whether correspondence and secrecy properties survive an unbounded number of symbolic sessions. ProVerif supports this analysis; Kobeissi et al. provide a secure-messaging case study connecting symbolic and computational analysis. These are precedents for protocol modeling rather than validation of Anchor. The 2.05 series used here adds the lemma, induction, and subsumption machinery described by Blanchet et al.; our result therefore speaks to the modeled message system and its stated queries, not to every transport path, implementation choice, or proposed credential envelope.

The Tamarin line asks a complementary question when state, algebraic equations, and inductive arguments are central. Tamarin supports equational properties and inductive proofs. The TLS 1.3 draft analysis by Cremers et al. uses Tamarin, as does the 5G authentication analysis by Basin et al.. CryptoVerif changes the target again: it seeks computationally sound protocol proofs rather than only symbolic reachability. We do not treat one tool's success as interchangeable with the other's guarantee; §2.5.3 keeps the symbolic/computational gap explicit. Our contribution is to apply the appropriate bounded obligations to local multi-agent coordination, with each obligation named instead of promoted into a whole-system security claim.

2.F.3 Capability-Based Authorization

Capability-based security dates back to Dennis and Van Horn's seminal 1966 paper on programming semantics for multiprogrammed computations. Recent work by Birgisson et al. introduced Macaroons, which use chained HMACs for decentralized delegation with contextual caveats. The Anchor Protocol's authorization chains (§2.3.4.3) are inspired by Macaroons but adapted for JWT-based identity in local development environments.

Dennis & Van Horn 1966

Birgisson et al. 2014

2.F.4 JWT Security

JSON Web Tokens have been the subject of extensive security analysis. Algorithm confusion attacks were first systematically studied by McLean, with subsequent vulnerabilities documented in CVE-2015-9235 (HS256/RS256 confusion), CVE-2018-1000531 ("none" algorithm), and CVE-2023-48238 (generic algorithm confusion).

McLean 2015

CVE-2015-9235 2015

CVE-2018-1000531. JWT "none" algorithm vulnerability in jwtf library. NIST National Vulnerability Database, 2018.

CVE-2023-48238 2023

All three are attacks on the same half of the problem. Each works by getting the verifier to choose its verification routine from something the attacker controls — whether that is `alg:none`, an HS256 header over an RSA public key, or the generic case — so a verifier whose whitelist holds one entry chosen by policy reads nothing the attacker can rewrite, which is Design invariant 2.2.2. The other half is key separation, and that is what Phase 2 buys: once verifiers hold only a public key, verifier compromise does not reveal the issuer's signing secret. This does not replace algorithm pinning: the HS256/RS256 trick works precisely by misusing a public key as an HMAC key.

Our protocol implements the defense recommended by the IETF JWT Best Current Practices: strict algorithm whitelisting and key separation.

Sheffer et al. 2020

Review of the key ideas

One card, carried through three of the ideas below, before the ideas themselves:

Numbers by Hand 2.F.1 (One Card through Attenuation, Pinning and Revocation).

A parent holds `fs:read` and `db:write` with ten minutes of lifetime left, and asks for a child with `db:write` for twenty minutes.

Two coordinates, both checked: rights,

$$\{\texttt{db:write}\} \subseteq \{\texttt{fs:read}, \texttt{db:write}\},$$

which holds; and lifetime,

$$\min(20, 10) = 10,$$

so the child that is issued carries ten minutes, not twenty (idea 2).

An attacker who intercepts that child and rewrites its header gains no choice of algorithm: the verifier uses trusted policy and rejects an incompatible authenticated header (idea 3).

And when the child's `kid` is revoked at minute three, a verifier refuses it after receiving the update. In the favorable $1 \to 2 \to 4 \to 5$ schedule shown earlier, two-second rounds deliver it in six seconds. Random contacts need not follow that schedule. A disconnected verifier without a stricter freshness policy may continue accepting it for the remaining seven minutes, until expiry (idea 4).

Every number here is one the chapter has already fixed; what the review adds is that they compose in a single trace. [internal]

What this chapter leaves coupled, in the order the rest of the book will lean on it:

1. A **harbor card** is a short-lived signed statement of exactly what one agent may do; the daemon checks the card, not the process, and a process with no card can do nothing the harbor mediates.
2. **Attenuation is monotone**: a child card is a subset of its parent in scope and in lifetime, at every hop, The symbolic capability-order models check the stated rights constraint; they do not by themselves prove every runtime lifetime or task-binding check.
3. **Algorithmic pinning**: the verifier chooses the signature algorithm from policy, never from the token's header. The same bytes that a header-selected verifier accepts, a policy-pinned verifier refuses.

4. **Revocation needs current local knowledge**: a verifier rejects a revoked card once its authoritative state includes the revocation. Expected gossip asymptotics are not a two-interval deadline; an unbounded partition prevents any finite global freshness guarantee.
5. **Filters do not merge; logs do.** Two daemons cannot union their cuckoo filters, so the append-only revocation log is the shared object and each daemon rebuilds its own filter from it.
6. **Correction is a new card**, with a new identifier and a fresh authority; the old card's history is untouched, and nothing is reanimated in part.
7. The proofs are **bounded and named**: authentication correspondence for the modeled phase, attenuation soundness on the multi-hop model, memory safety within the harness bound with the cryptography stubbed, and branch-freedom at source level. Each is a statement about a model, and the chapter says which.
8. What the adversary can still do is stated as scope: a process that holds a valid card can spend it in full within its lifetime; the protocol bounds what a card grants, not what a grantee wants.

2.G Exercises

Several of the exercises below end in a quoted verdict line from a committed results file, and those lines are read backwards from the way a test report is read — so here is one worked in full before the first exercise asks you to read another.

Numbers by Hand 2.G.1 (Two Negations, One Accepted Attack).

The committed output of `analyses/harbor_card_v6_multihop_attack.pv` reads `RESULT not event(Accepted(cc_2, harbor_id[], cap_write)) is false.`

Count the negations: there are two, and they are doing different jobs. The first is in the query itself — the model asks ProVerif to show that the event *never* happens — and the second is the verdict, which says that claim does not hold.

Compose them and the sentence comes out as: the event `Accepted(·, ·, cap_write)` *is* reachable, so the escalating chain was accepted, so the attack works.

A ProVerif `false` on a safety query is a found attack, not a failed run, and the tool says so twice over by printing `A trace has been found` above it. Work that composition once here and every quoted verdict below reads at a glance. [internal]

§2.3 How we know it is correct

Solution p. 724

2.1 Trace ⋆⋆

Read `proofs/anchor/token-verify/algconfusion.pv`. State the algorithm-confusion attack in words: what the attacker already knows, what it forges, and which verifier admits it. Then say, in one sentence, what *pinning* must mean for a verifier to resist it.

Solution p. 725

2.2 Check ⋆

`proofs/anchor/delegation/chain-replay.pv` models a 3-hop chain $P \to A \to B \to C$. Which property does the model check, and what does the run log's verdict say?

§2.4 One source crate, different verification scopes

Solution p. 725

2.3 Check ⋆⋆

For each of the three Kani harnesses above, state exactly what is proved and name one stronger claim a reader might wrongly draw from it.

Solution p. 726

2.4 Open ⋆⋆⋆

None of this chapter's ProVerif models, and none of its three Kani harnesses, says anything about most of what the running daemon does at a socket. Using only what this chapter already states about the Kani harnesses' scope, write the one-paragraph boundary a reader must hold before trusting "verified" as a description of the deployed binary. Then say what a stronger harness would need to add.

§2.5 What the adversary can do, and what it cannot

2.5 Trace ⋆⋆ *Solution p. 727*

A Harbor Card verifies: `Ed25519.Verify` returns true. Trace what else must hold before that verified signature actually *authorizes* the action its bearer is attempting. Then distinguish two independent choices: whether the token is a bearer credential, and whether authorization is re-checked on every request.

§2.D Phase 3: Multi-hop Delegation

2.6 Trace ⋆⋆ *Solution p. 728*

Trace the chain A:`write` $\rightarrow$ B:`read` $\rightarrow$ C:`write` through two verifiers by hand: (i) one that checks only the final capability against the root, and (ii) the per-hop verifier of Algorithm 2.4. Which one accepts C? Then say precisely why checking every $\text{cap}_i \subseteq \text{cap}_{i-1}$ proves $\text{cap}_k \subseteq \cdots \subseteq \text{cap}_0$, where checking only $\text{cap}_i \subseteq \text{cap}_0$ at each hop does not.

2.7 Trace ⋆⋆ *Solution p. 728*

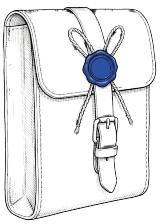
Read `analyses/harbor_card_v6_multihop_attack.pv`. State the attack in words: who forges what, at which hop, and why does a verifier that checks only the final capability against the root admit it?

2.8 Check ⋆ *Solution p. 729*

`analyses/harbor_card_v7_multihop_fixed.pv` is identical to v6 except for the verifier. Name the one modeling change that closes the attack, and what the RESULT line becomes.

2.9 Trace ⋆⋆⋆

The authorization chains in this chapter use signed Ed25519 cards. `analyses/macaroon_discharge_v1.pv` and `analyses/macaroon_discharge_v2_naive_unsound.pv` model a related but different construction, an HMAC-chained macaroon with a third-party discharge caveat. Compare the two: what does the naive discharge verifier in v2 admit, and what does v1's verifier check that closes it?



Like a diplomatic courier pouch requiring a designated officer to break the seal, a third-party caveat requires a distinct witness to issue its discharge.

Solution p. 729

What *The Sealed Harbor* receives A request now arrives with authenticated, narrowing authority, and the kernel that admits it keeps one durable history. Some work needs more than an authorized writer: it needs a room in which two parties who trust neither each other nor the venue can still get one answer out. The next chapter builds that room and prices what can leave it through the slot.

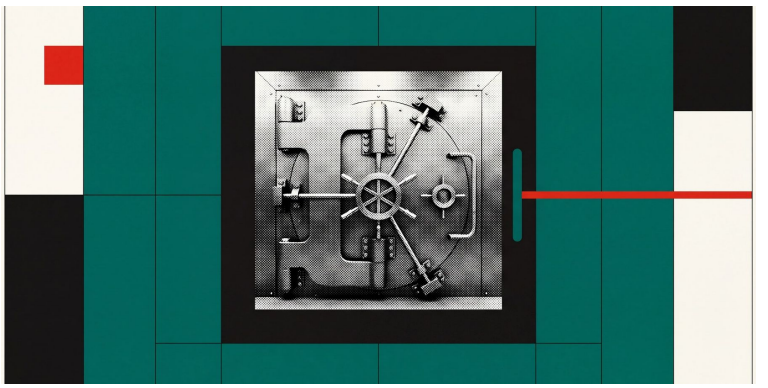
3

The Sealed Harbor

The question this chapter answers

What can a sealed agent leak, and how would we know?

A confined work order has explicit release gates and a shared leakage budget, under the model's stated channel assumptions.



This note explores the problem of confining a program during its execution so that it cannot transmit information to any other program except its caller.

Butler W. Lampson, A Note on the Confinement Problem, 1973

What can a sealed agent leak, and how would we know? A model owner and a data owner want a shared answer while keeping their respective inputs private. The kernel refuses a write it has not authorized and the Anchor checks who asked; neither says what may leave a room through the only slot in its wall. This chapter builds the sealed room: one work order, two fences, two gates, and four properties, each stated at the strength its check earns. R9, R10, and R11 govern its strongest claims.

3.1 Two owners, one answer

Express lane: a sealed room lets an agent compute on data its owner will not hand over, and the room's promise is not "nothing leaks" but "every release is explicit, gated, and bounded"; the four claims that make that promise provable are Theorems 3.3.1, 3.5.1, and 3.6.1, plus the controllability theorem of The Single-Writer Kernel*, and the leakage they leave is the account of Section 3.7.*

The scene. Derek owns sensitive financial data D. Erin owns a valuable agent system M: weights, prompts, skills, tools, orchestration. Both want the answer $y = f_M(D)$. Neither will hand over the crown jewels: Derek must not receive M, Erin must not receive D, and the cloud operator hosting the computation must receive neither – three parties, three sight-lines, one boundary that must seal all of them at once, drawn in Figure 3.1.

The technical question is what the execution environment can guarantee when the data owner and agent owner distrust each other. A computational contract specifies permitted releases; it does not settle their legal relationship. The systems literature answered a narrower version first: Ryoan confines a proprietary module inside an attested enclave so that a data owner and a code owner who distrust each other both keep their secrets. Ryoan buys its confinement with a module that sees its input once and keeps no state between requests in its confinement model. This chapter instead studies persistent tool-calling workloads and explicit releases. Table 3.1 compares the chosen mechanisms; it is not an impossibility claim about other sandbox architectures. The other end of that design space is worth naming in the same breath, because it is where most deployments actually sit: SCONE puts an ordinary Linux container inside SGX and asks almost nothing of the application, which makes it adoptable in a way Ryoan is not, and buys

T. Hunt, Z. Zhu, Y. Xu, S. Peter, and E. Witchel. Ryoan: a distributed sandbox for untrusted computation on secret data. In *Proc. 12th USENIX Symposium on Operating Systems Design and Implementation (OSDI)*, pages 533–549, 2016.

S. Arnautov, B. Trach, F. Gregor, T. Knauth, A. Martin, C. Priebe, J. Lind, D. Muthukumaran, D. O'Keeffe, M. L. Stillwell, D. Goltzsche, D. Eyers, R. Kapitza, P. Pietzuch, and C. Fetzer. SCONE: secure Linux containers with Intel SGX. In *Proc. 12th USENIX Symposium on Operating Systems Design and Implementation (OSDI)*, pages 689–703, 2016.

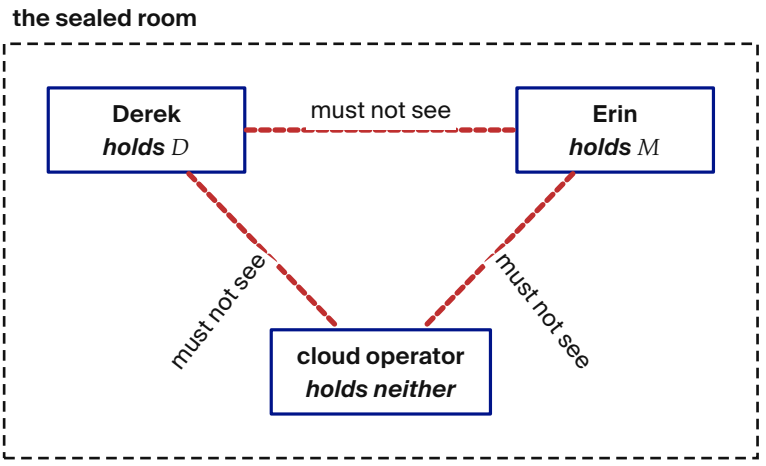


Figure 3.1.
The room must keep Erin's model from Derek, Derek's data from Erin, and both from the cloud operator. One mechanism must satisfy all three separations. [internal]

one boundary seals all
three sight-lines at once

correspondingly less — it protects the workload from the host, not the data owner from the code owner, and a container free to open sockets can carry D out through any of them. Between the two lies the gap this chapter works in: the stateful, non-DAG, tool-calling workload whose releases need an explicit policy beyond transparent execution. A mediated gate and accounting ledger are this design's proposal, not the only possible solution. A single writer orders accounting; it cannot, by itself, provide confinement or determine what an output reveals.

	Ryoan	The sealed room
Workload shape	a request-oriented module, input seen once, no persistent state	a looping, tool-calling agent with state across steps
How leakage is bounded	confinement in the enclave plus fixed-size outputs	explicit gated releases, priced by an ε-ledger and a $q \cdot b$ bit budget
Taint granularity	the whole module	the slot: every release is a declassification event
Model access	the module's code is sealed; no query interface	the model runs inside the domain and is observed only through the contracted query and output interface
Multi-job accounting	none; each request stands alone	the ledger sums spend across jobs and refuses the overdraw
Evasion	constrained within its stated confinement model	canaries detect specified alternatives with stated power, not every evasion
Timing	out of scope	declared out of model; padded and scheduled, not closed

Table 3.1. Different workload and observation models. Ryoan uses request-oriented confinement; the proposed sealed room adds explicit releases and cross-job accounting for a persistent agent. Neither closes every side channel.

Why blocking tainted outputs is insufficient. Consider a proposed "silicon-enforced NDA": tag Erin's agent the moment it reads Derek's data, drop any tainted egress packet, and declare that the data "mathematically cannot phone home." The first job of this chapter is to say why that sentence is false, and the second is to prove the sentence that replaces it.

Numbers by Hand 3.1.1 (What "Nothing Leaks" Costs, in Bits).

Price the three candidate designs on the chapter's own toy release — the one Theorem 3.3.1 will later check exhaustively — and you can do the whole comparison in your head. For this example Derek's secret is uniform on $\{0, 1, 2, 3\}$, so its entropy is $\log_2 4 = 2$ bits, and the contracted release is

$$g(s) = s \bmod 2,$$

which takes two values and therefore carries exactly 1 bit — and, since s written in binary has its parity in the low position, that release is literally one of the secret's two bits.

Now count. An honest gate hands Erin 1 of those 2 bits: the bit Derek sold her, and no more.

The leaky gate of the mutation suite, which forwards s itself, hands over 2 — the whole secret, which is why the checker catches it four steps in.

And the taint tracker of the pitch above, run *soundly* at token granularity, has to mark the parity bit as tainted by the secret it was computed from and drop it, so Erin gets 0 bits and the engagement has nothing left to deliver to her.

The 0-bit design blocks the contracted answer along with the secret; the other horn, the tracker that lets the parity through, cannot then certify that the released bit was parity of committed s, rather than parity of some worker-chosen $f(s)$.

Every honest design in this chapter therefore lives in the middle column, and the rest of the chapter is the arithmetic of how wide that column is allowed to get.

Design Invariant 3.1.2 (Conservative Whole-Worker Labeling).

This design does not trust a token-level dependency classifier for arbitrary model output. Once the worker reads an owner's secret, every later worker emission retains that owner's confidentiality label until an authorized release removes it.

This is a conservative engineering choice, not an impossibility theorem about language models or information-flow analysis. The label-creep and dynamic monitoring literature explains why precision and permissiveness need care; it does not prove that every sound model-specific analysis must have this shape. A restricted model or a trusted release computation may justify a finer policy. The room assumes neither for its untrusted worker.

A secret can ride out in wording, punctuation, error messages, output length, or timing; no regular expression, scanner, or second model can certify that arbitrary prose contains no secret. And an ordinary remote model API is *itself* a trusted reader of plaintext: cryptography cannot repair sending Derek's data to a third party's inference endpoint.

D. E. Denning and P. J. Denning. Certification of programs for secure information flow. *Communications of the ACM*, 20(7):504–513, 1977.

A. Russo and A. Sabelfeld. Dynamic vs. static flow-sensitive security analysis. In *Proc. 23rd IEEE Computer Security Foundations Symposium (CSF)*, pages 186–199, 2010.

The architecture therefore controls the *channel*, not the token. It converts the question a security officer will actually ask, *how do I know the wall has no other holes?*, into four provable claims: the gate is the only hole (Section 3.3); the gate can refuse the effects covered by its mediation and observation model (Section 3.4); what passes through it is metered by a conserved budget (Section 3.5); and canary detection has conditional power and model-based stopping estimates (Section 3.6). Section 3.7 composes the four into a single leakage account, and Section 3.8 states, at equal prominence, what none of this promises.

3.2 One work order, two fences, two gates

The work order contract. The foundation of mutual confinement between distrustful principals is the bilateral *work order contract* C. In conventional enterprise workflows, confidentiality is negotiated ex-post through legal non-disclosure agreements: lawyers draft obligations, employees sign terms, and violations are litigated only after trade secrets have been compromised. Technical prevention and remedies answer different questions: weights copied to an external endpoint cannot be un-seen, and proprietary financial datasets leaked over an unmetered socket cannot be un-learned.

The parties therefore require the technical boundary before releasing secrets. Before any decryption key is released, both Derek (the data owner) and Erin (the model and skill owner) must sign an identical, canonicalized work order C:

$$C = \big\langle H(\text{data manifest}),\ H(\text{agent bundle}),\ H(\text{runtime}),\ H(\text{policy}),\ \text{function},\ \text{tools},\ \text{recipients},\ \text{output schemas},\ \text{budgets},\ \text{retention},\ \text{validators},\ \text{payment},\ \text{liability} \big\rangle.$$

Read the tuple in order. The thirteen fields fall into five distinct structural roles: *pins*, *permissions*, *recipients*, *bounds*, and *remedies*. Under the collision-resistance assumption on H, changing the canonical contract changes the digest the signatures authenticate. A conforming key broker refuses a modified contract without new matching signatures; the digest does not itself stop execution. The contract is a durable work unit in the sense of *The Single-Writer Kernel*: an immutable case file that outlives every transient guest process that executes it.

Table 3.2 details each of the thirteen elements of C, naming each intended check alongside illustrative values. A field names an obligation; its presence does not implement that obligation.

Table 3.2. The thirteen elements of Work Order Contract C. Proposed contract fields and illustrative values. Measurements, permissions, and budgets require corresponding enforcement; payment and liability specify remedies rather than proving confidentiality.

Element	Role	Security Specification & Guarantee	Example Value
H(manifest)	Pins	Cryptographic digest over dataset chunks, schema layout, and provenance. Binds approved bytes; the loader must check them and reject other inputs.	`sha256:7f83b165...` `s3://derek/q3.parquet`
H(agent)	Pins	Digest of model weights, tokenizer, prompt AST, and tools. Binds Erin's approved bundle; integrity also requires checked loading and protected execution.	`sha256:c0ffee4b...` `weights.safetensors`
H(runtime)	Pins	Measured guest enclave launch digest (SEV-SNP / TDX MRTD) verifying kernel and monitor before execution.	`mrtd:e4a8109d...` `(Linux 6.8 + monitor)`
H(policy)	Pins	Digest of declassification lattice rules, label propagation logic, and differential privacy noise parameters.	`sha256:5b91df...` `eps=0.5, delta=1e-6`
function	Permissions	Whitelisted top-level entrypoint symbol. Names the admitted entrypoint; does not constrain every internal subroutine by itself.	`portfolio_stress_test` `(portfolio_id: str)`

Continues on next page ►

Table 3.2: The thirteen elements of Work Order Contract C (continued)

Element	Role	Security Specification & Guarantee	Example Value
tools	Permissions	Closed whitelist of in-guest tool capability handles. Direct sockets disabled; all calls route through monitor.	`["yield_oracle", "spot"]` `(no raw sockets)`
recipients	Recipients	Authorized recipient identities and public keys for each egress channel. Derek gets results; Erin gets telemetry.	`derek: did:key:z6M..` `erin: did:key:z6M..`
schemas	Recipients	Strongly-typed schema definition (Cap'n Proto / JSON Schema) for declassified outputs. Forbids free-form payloads.	`RiskReportV1 { risk: uint63, flag: bool }`
budgets	Bounds	Hard resource and leakage ceilings: maximum declassification bits ($q \cdot b$), CPU time, memory, and timeout.	`b=64 bits, q=1 exit, max_cpu=300s, 32GB`
retention	Bounds	Ephemeral session lifetime and zeroization policy. Specify key destruction, memory clearing, and prohibited persistence; their effectiveness requires platform evidence.	`zeroize_on_exit; no_swap; max_ttl=900s`
validators	Remedies	Independent appraisal rules for attestation evidence (IETF RATS) and third-party arbitration endpoints.	`rats_profile: v1.2; arbitrator: did:key:..`

Continues on next page ►

Table 3.2: The thirteen elements of Work Order Contract C (continued)

Element	Role	Security Specification & Guarantee	Example Value
payment	Remedies	Consideration terms, escrow identifiers, and conditional release mechanisms upon verified receipt delivery.	`250.00 USDC in Escrow#0x8f2a...`
liability	Remedies	Liquidated damages, penalty covenants, and bonded stake forfeiture on attested breach or canary leak receipt.	`10,000 USDC bond; forfeit_on_leak: canary`

The illustrative status packs a 63-bit unsigned risk code and one flag into 64 payload bits; its quantization and framing must be fixed by the schema. This is an output-alphabet bound, not the size of a JSON document.

Figure 3.2 illustrates how these thirteen fields stack within the confidential enclave boundary.

Dual-attested key release. The workload boots in a confidential virtual machine and creates an ephemeral job key with fresh attestation evidence binding runtime, policy, worker image, key, nonce, and epoch. Derek and Erin *independently* verify the attestation; Derek's key broker releases the data key only to the approved measurement, and Erin's key broker releases the model and skill key under the same condition, the attester, verifier, and relying-party pattern of IETF RATS. The job then runs with no ambient network or credentials. The intended confidentiality claim is conditional on correct attestation appraisal, protected execution, and complete mediation of the declared channels. Attestation reports a measurement; it does not prove that the measured code is correct or that every hardware side channel is absent.

H. Birkholz, D. Thaler, M. Richardson, N. Smith, and W. Pan. Remote ATtestation procedureS (RATS) architecture. RFC 9334, IETF, January 2023.

Figure 3.3 draws the ceremony as a sequence: one attestation evidence bundle, appraised twice and independently, each broker releasing its own key off its own appraisal alone.

Two fences. Erin's worker is treated as hostile even if Erin is honest, because prompt injection or a compromised dependency can

role	fields	what it fixes
pins	H(data), H(agent), H(runtime), H(policy)	four digests identify approved data, agent, runtime and policy
permissions	function, tools	fixes what may run and what it may call
recipients	recipients, output schemas	fixes who may receive a result, and in what shape
bounds	budgets, retention	bounds output bits, queries, and time; retention fixes key destruction
remedies	validators, payment, liability	validators name evidence; payment and liability specify remedies

a changed canonical contract requires fresh matching signatures before key release

Figure 3.2.
The work order binds measurements, permitted execution, recipients, limits, and remedies. A digest identifies approved content; an attestation binds the reported runtime measurement. Neither implements the checks or proves correctness. Changed contract bytes require fresh matching signatures under the hash and signature assumptions. [internal]

turn a loyal agent into an exfiltrator, so it receives *semantic handles only*:

`read`(handle, range) `infer`(model handle, prompt handle)
`execute`(tool handle, args)
`write`(result handle, bytes)
`request_declassification`(candidate handle, channel)

An attested in-guest monitor mediates labels, handles, and encrypted content; a Derek-controlled outer gateway mediates network, storage, timing, destination, and volume. Direct outbound TLS is forbidden; permitted services terminate at a policy gateway. Because a remote model API is itself a trusted reader of plaintext,

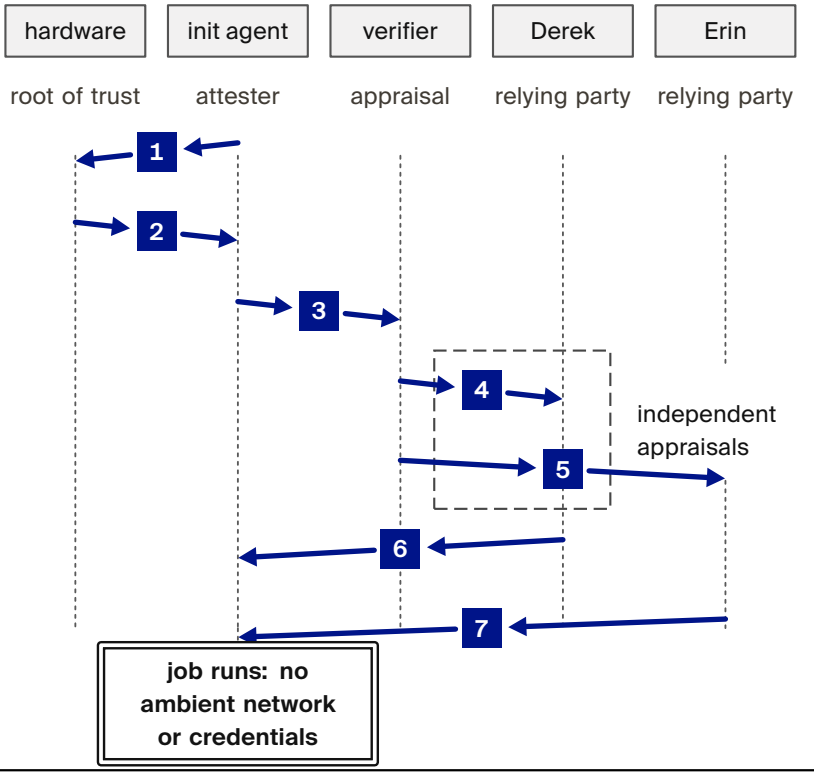


Figure 3.3.
One evidence bundle, two independent appraisals. Derek and Erin each release a key only after their own appraisal succeeds. [internal]

#	binds	value
1	the quote request	a digest of the booting measurement: runtime, policy, worker image
2	a, the attestation evidence	runtime, policy, worker image, key, nonce, epoch — signed by the hardware root of trust
3	the forwarded evidence	a, relayed unchanged to the verifier
4	Derek's appraisal	a checked against Derek's approved measurement
5	Erin's appraisal	a checked against Erin's approved measurement, independently of message 4
6	the data key	released to the job only if appraisal 4 matched
7	the model and skill key	released to the job only if appraisal 5 matched

the model runs *inside* the confidential domain; an external inference endpoint would be a third principal the contract never admitted.

Numbers by Hand 3.2.1 (Four Label States, Four Locked Doors, Two Keys).

The lattice the next paragraph reaches for is small enough to count on your fingers, and the count *is* the mechanism. Two owners, Derek and Erin, means two clauses, so a value inside the room sits in one of $2^2 = 4$ label states: $\{\}$, $\{D\}$, $\{E\}$, $\{D, E\}$. Derek's data enters at $\{D\}$ and Erin's model at $\{E\}$; the instant the worker computes with both, whatever it produced sits at $\{D, E\}$.

There are four edges up that lattice and four edges down it, and the asymmetry between the two fours is the design. All four upward edges are free and unwitnessed, because adding somebody's restriction needs nobody's permission.

All four downward edges are locked, and — this is the part worth doing by hand — they're locked *by owner*: the two that drop D, namely

$$\{D\} \to \{\} \text{ and } \{D, E\} \to \{E\},$$

open only with Derek's delegated declassification authority, and the two that drop E, namely

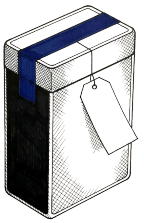
$$\{E\} \to \{\} \text{ and } \{D, E\} \to \{D\},$$

only with Erin's.

Two keys, four locks, two apiece. That is why the room has exactly two gates rather than one, and why the aggregate-feedback channel to Erin is *Derek's* gate and not hers: the artifact she receives is one that has had Derek's clause dropped, and only Derek can drop it.

Whole-worker taint and two gates. After the first secret read the entire worker state, and every later emission, is tainted until job destruction: decentralized labels in the Myers–Liskov style, owned by mutually distrustful principals, under which ordinary computation can only *add* restrictions. A label is the set of owners whose permission is still needed for release; combining values takes the union

Provenance is not correctness



A tag and seal suggest provenance, not correctness. Digital receipts must bind their claims cryptographically.

A. C. Myers and B. Liskov. A decentralized model for information flow control. In *Proc. 16th ACM Symposium on Operating Systems Principles (SOSP)*, 1997.

of their labels. This operation is the lattice *join*. A declassification is the explicitly authorized removal of a restriction, not a claim that the result ceased to depend on the secret. This is the same shape as capability attenuation in *The Anchor Protocol*, where a delegated token never carries more authority than its parent: ordinary delegation only shrinks authority. Declassification differs in one important respect: it uses separate release authority to remove a confidentiality restriction. It is not permission to widen a child card. Removing an owner's clause requires a declassification authority delegated by that owner, and the only exits are the two declassification gates, Derek's and Erin's, each executing a declared release function, with exactly three pre-authorized channels: a rich result to Derek; fixed-schema, aggregated or differentially private feedback to Erin; a padded public receipt to both. Each party receives an audience-redacted signed receipt, Merkle-anchored in both principals' ledgers:

$$R = \mathrm{Sign}\big(H(C),\ H(\text{attestation}),\ H(\text{inputs}),\ H(M),\ H(\text{output ciphertext}),\ \text{labels},\ \text{declassifications},\ \text{counters},\ \text{nonce},\ \text{epoch}\big).$$

The receipt proves what the measured system reported under a policy; it does not prove the answer is true, nor that no unmodeled side channel existed. Figure 3.4 shows the fields bound into R and separates the provenance of the report from the truth of its output.

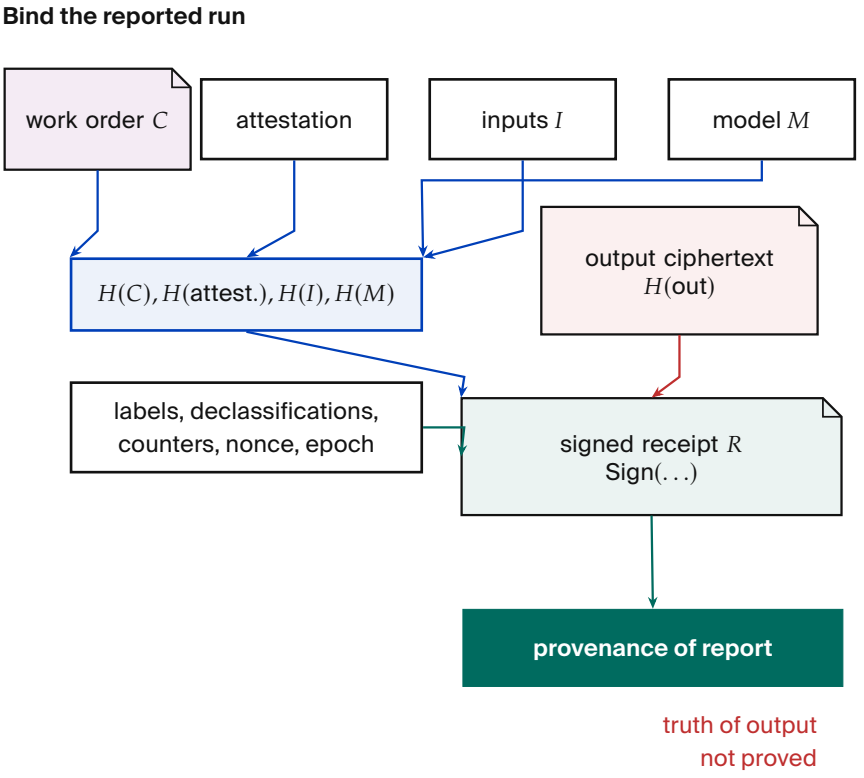


Figure 3.4.
Hashes and the signature bind the recorded work order, evidence, inputs, model, policy state, and released ciphertext. The receipt proves what was reported, not that the output is true.



Landmark 07 · Attestation Without Revelation (The Sealed Harbor)

Attestation can identify a measured execution environment under a stated trust model. It does not alone prove that a program is correct or that its outputs reveal no private information. Those claims require separate specifications, enforcement and evidence.

Design Invariant 3.2.2 (The Room's Side Properties).

Label monotonicity: no label weakens without a declassification witness. Complete mediation: every release has exactly one gate decision. Provenance closure: every release traces to committed input, model, runtime, policy, and declassifier nodes.

Replay safety: each job nonce settles at most once. Measurement binding: neither secret key is released for a different runtime or policy. Rollback detectability: accepted epochs strictly increase in both external ledgers.

These are deployment obligations of the proposed room. The finite checks below cover selected abstractions; they are not evidence that all six hold in a running confidential-computing deployment.

Table 3.3 names, for each of the six, the later result that leans on it; complete mediation is the one link and conservation both cite by name, the others support the named mechanisms rather than being implicit premises of every theorem.

Four links, one pipeline. The four claims are not four independent guarantees stacked for redundancy; they are four links in one pipeline, each closing a gap the others leave open. Enforceability asks which effects this monitor can refuse, and which observations it needs to make the correct decision. Noninterference proves that boundary, once installed, is silent except through its declared release. Conservation meters the cumulative cost of everything that legitimately crosses it, because a gate honest on every single release can still bankrupt the contract over many releases; an individually sound decision rule is not a portfolio-sound one. Detection polices for what tries to cross without going through the meter at all: a compromised worker, a bug, an adversary who found an unmediated path. These supply different evidence: a detector does not repair a failed confinement premise, and a conserved ledger does not validate a release mechanism. The resulting account and its observation assumptions are in Section 3.7. Table 3.4 is the evidence schedule: each link names its claim, how the claim was verified, and the script that regenerates the verification.

Recall

1. Without looking: name each secret owner, the intended recipient of each output, and the assumptions needed to keep the hosting operator from reading the secrets.
2. Why are resource accesses routed through handles even when Erin is honest? Does a handle imply that computation never receives plaintext?
3. Which property connects the accounting ledger to actual releases, and why does checking a ledger alone fail to establish it?

Table 3.3. Complete mediation is the assumption that both enforceability and conservation name explicitly.

Invariant	States	Depended on by
Label monotonicity	no label weakens without a declassification witness	every gate of Sec. 3.2; assumed silently by noninterference (Sec. 3.3)
Complete mediation	every release has exactly one gate decision	**named explicitly**: the deployment assumption Enforceability (Sec. 3.4) and Conservation (Sec. 3.5) both inherit
Provenance closure	every release traces to committed input, model, runtime, policy, and declassifier nodes	the receipt $R = \mathrm{Sign}(\ldots)$ of Sec. 3.2
Replay safety	each job nonce settles at most once	the receipt's nonce field and the dual-attested key release's fresh nonce (Sec. 3.2, Fig. 3.3)
Measurement binding	neither secret key is released for a different runtime or policy	the dual-attested key release ceremony directly (Fig. 3.3)
Rollback detectability	accepted epochs strictly increase in both external ledgers	the ε-ledger's append-only journal (Sec. 3.5)

Table 3.4. Four links, four claims, and one regenerable check for each.

Link	Claim	Verification	Script
Noninterference	silence except through the slot	exhaustive to depth 7; both mutations caught	`c1_noninterference.py`
Enforceability	exact policy realization for the declared plant	product-automaton checker	`b3_controllability.py`
Conservation	the ledger conserves	exhaustive and 2000 random instances; both mutations caught	`a3_epsilon_ledger.py`
Detection	conditional power and stopping estimates	exact power; approximate counts; reported simulation	`a4_canary_sprt.py`

Figure 3.5 locates the two fences and the separately authorized release paths. Derek's gate permits feedback to Erin; Erin's gate permits the result to Derek. The evidence table above connects the controls to their checks. The sections that follow take noninterference first because it is the promise the other three claims exist to support.

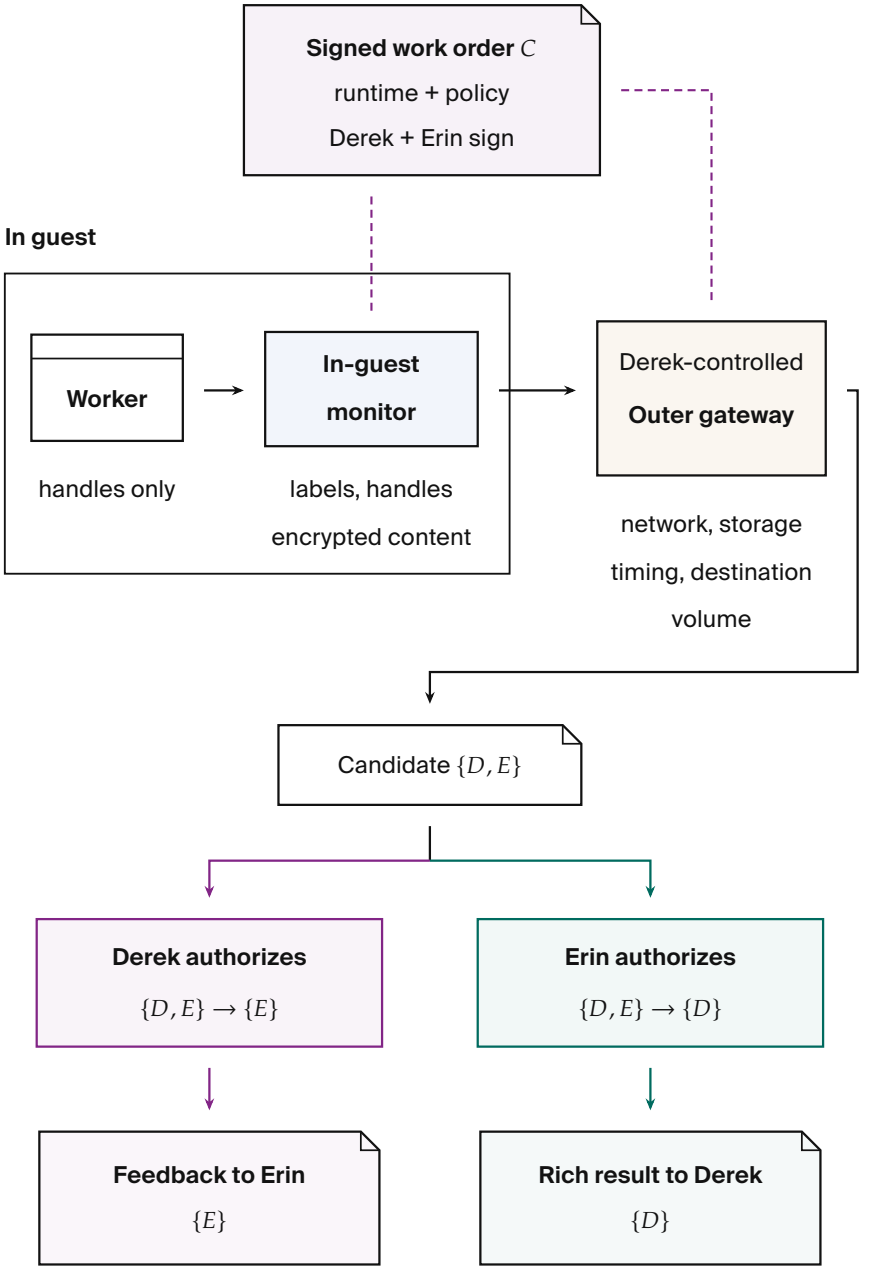


Figure 3.5. Each owner authorizes removal of their own restriction. Dashed links bind controls to the signed work order; solid arrows carry outputs. Proposed control flow; public receipt omitted.



The controlled opening

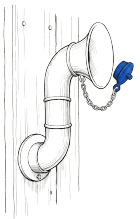
A closed cabinet still has a slot. A confined agent still has an output. The noninterference claim must name what that output may reveal and which other channels the model excludes.

3.3 Silence except through the slot

Express lane: run the model twice, differing only in Derek's secret, under the same sequence of moves — self-composition in the verification literature — and compare what Erin observes. Equal-parity secrets should produce identical logs. Model-Checked Property 3.3.1 reports the bounded search, not an unbounded guarantee.

Intuition: the voting booth. You cannot inspect a wall for holes by staring at bricks; you inspect it by standing outside in two parallel worlds and asking whether anything you can see ever differs. If Derek's secret is 0 in one world and 2 in the other, and the gate is contracted to release only *parity*, then the target is to keep those worlds indistinguishable to Erin under every interleaving of loads, reads, submissions, releases, and reports. The analogy with teeth is the voting booth: the turnstile clicks once per voter and the tally board shows only party totals; a poll-watcher outside learns no more from the declared observations than the aggregate reveals, no matter how long she watches or in what order voters arrive. That is noninterference *modulo declassification* in the Goguen–Meseguer lineage: silence except through the slot, and through the slot, only what the contract says. The misread to preempt: the theorem does not say the slot is safe. It says the slot is the *only* opening; whether the declared function g releases too much is a contract question the next two sections price. In the booth's own vocabulary: the guarantee is about the walls and the turnstile, never about what the board is asked to print. A per-precinct board reporting a precinct with a single voter discloses that voter's ballot exactly, and no property of the booth prevents it. The room stayed sound; the tally was indiscreet.

G. Barthe, P. R. D'Argenio, and T. Rezk. Secure information flow by self-composition. In *Proc. 17th IEEE Computer Security Foundations Workshop (CSFW)*, p. 100, 2004.



Like a shipboard speaking tube whose whistle stopper must be unplugged before sound can pass, communication across the clean-room boundary is silent except through the explicitly gated slot.

J. A. Goguen and J. Meseguer. Security policies and security models. In *Proc. IEEE Symposium on Security and Privacy (Oakland)*, pp. 11–20, 1982.

J. A. Goguen and J. Meseguer. Unwinding and inference control. In *Proc. IEEE Symposium on Security and Privacy*, 1984.

Model-Checked Property 3.3.1 (Noninterference Modulo Declassification on the Finite Clean-Room Model).

Fix the clean-room transition system with secrets $s \in \{0, 1, 2, 3\}$, Erin-observable state Obs_E, and declared release function $g(s) = s \bmod 2$, where the gate applies g to committed input state rather than to worker-computed state.

For secrets s, s' and every valid action sequence of at most seven steps: if $g(s) = g(s')$ then Erin's observation sequences are identical; if $g(s) \neq g(s')$, new log differences arise only at gate-release steps.

Result. Exhaustive over all interleavings to depth 7 and all secret pairs: zero distinguishing observations for equal-parity pairs in the honest model. The schedule itself is secret-independent, checked separately, since an enabledness difference would itself be a channel. Rushby's unwinding condition of local respect, that Derek-side actions never touch Erin-observable state, holds mechanically.

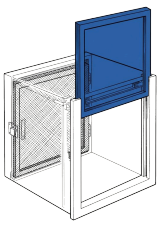
Mutations. A gate that leaks the raw secret is caught distinguishing the equal-parity pair $(0, 2)$ with the witness trace `load` → `read` → `submit` → `release`, after which Erin sees `(gate, 0)` in one world and `(gate,2)` in the other; a worker bypassing the gate is caught in a three-step trace. Reverting either mutation restores the property [internal, `c1_noninterference.py`].

The clause about committed input is a hypothesis, not a modelling convenience. It makes the property an instance of *delimited release* in the degenerate case where delimited release collapses to noninterference; their own condition is that no variable used in a declassification is updated before it. What the hypothesis costs is stated in the boundary below and paid in Section 3.7. The release ledger of §3.7 answers, in Sabelfeld and Sands' later organizing taxonomy, *what* is released (the ε-priced artifact), *who* performs the release (the declassifier), and *where* it happens (the gate); *when* it happens is the release-ledger's own open question, since a room can only price the release it observes, not the frequency an adversary chooses to request it.

J. Rushby. Noninterference, transitivity, and channel-control security policies. Technical Report CSL-92-02, SRI International, December 1992.

A. Sabelfeld and A. C. Myers. A model for delimited information release. In *Software Security — Theories and Systems (ISSS 2003)*, pages 174–191. Springer, 2004.

A. Sabelfeld and D. Sands. Declassification: dimensions and principles. *Journal of Computer Security*, 17(5):517–548, 2009.



Like a two-shutter laboratory pass-through box whose interlocked doors ensure both are never open at once, noninterference prevents secret input from directly reaching the observer.

Proof idea. There is no proof to read; there is a search to rerun. The checker builds the two worlds side by side, one per secret, and drives them through every interleaving of the same action sequence to depth 7. This is **self-composition**, exactly as Barthe, D'Argenio and Rezk set it out: noninterference is a claim about *pairs* of runs, which no ordinary invariant can state, so you compose two copies of the system into one and state an ordinary invariant about the pair. Here a Python checker searches a finite model; their work uses a program logic. At each step the checker projects both states onto what Erin can observe and compares. For equal-parity secrets, any difference in the observation sequences is a counterexample. For differing-parity secrets, the checker rejects a non-gate step if the resulting logs differ and either log changed at that step; an earlier gate-release difference may persist unchanged. The mutation suite then shows the search has teeth: with the honest gate replaced by one that releases the raw secret, the first difference appears four steps in, on a pair the honest gate provably keeps identical. Figure 3.6 compares Erin's observations step by step, then shows the distinguishing release produced by the raw-secret mutation.

Barthe et al. 2004

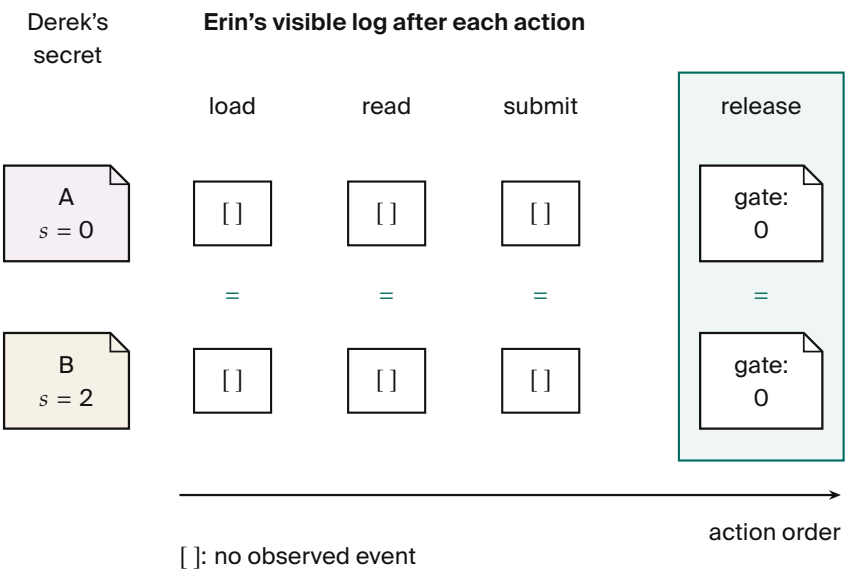




Figure 3.6. Parity hides the difference between secrets 0 and 2; a raw-secret release exposes it. Reported model check: four secrets, depth 7, not unbounded executions. [internal, `c1_noninterference.py`]

Numbers by Hand 3.3.2 (Six Pairs, Two of Them Equal-Parity).

The number of unordered pairs among four secrets is

$$\binom{4}{2} = 6.$$

Two of them, $(0, 2)$ and $(1, 3)$, are equal-parity. For those two pairs, the check found identical observation logs through every interleaving of at most seven steps.

Now you try: with $g(s) = s \bmod 2$ over secrets $\{0, \ldots, 7\}$, how many unordered secret pairs must a sound room keep Erin-indistinguishable? (Exercise 3.1.)

`c1_noninterference.py` · 20260816 · verified

What it buys. The product's headline sentence in its provable form: *every release is explicit, gated, and bounded*. This is the finite-model core of the chapter; the general theorem over unbounded state is the unwinding obligation of Section 3.8, for which this model

is the blueprint, an unbounded argument still to be supplied, not a consequence of exploring seven steps.

Where this stops

Finite depth, finite secrets: depth-7 exhaustion is a proof about the model, not the deployment. The guarantee is possibilistic; timing, token counts, and termination channels are out of model, as the design declares. The gate's *specification* is the residual oracle: the property is that releases match g, and choosing a g that leaks too much is a contract failure no theorem prevents. That risk is priced, not eliminated, by the next two sections.

There is a second residual on the other side of the same gate. The property quantifies over action sequences of a model whose gate applies g to committed input. It does not cover a worker that launders: computing $t = f(s)$ and submitting t so that the gate honestly releases $g(t)$. That is the classical escape-hatch laundering attack, which the delimited-release literature names and rules out by enforcement. The proposed arbitrary-output path does not supply that enforcement. A trusted gate could instead compute a restricted release directly from committed input, as the finite model does; accepting unconstrained worker-computed candidates gives up that particular guarantee. The residual is therefore priced in Section 3.7, which is the reason the $q \cdot b$ account exists at all. The present mutation suite cannot exhibit the attack, since the model's `submit` carries no payload: the suite tests that the gate releases g and nothing else, not that the gate's *argument* is honest. Closing that gap is a model extension listed in Section 3.8. And a human who reads a released parity may still remember it: the system bounds channels, not minds.

Sabelfeld & Myers 2004

Figure 3.7 makes the missing argument concrete. Worlds A and B hold secrets 0 and 2: the permitted parity release is 0 in both. If the worker first computes $t = f(s) = \lfloor s/2 \rfloor$, the same gate instead releases 0 and 1. Erin can now distinguish the worlds. This is an arithmetic illustration of the proposed payload extension, not a counterexample to the current model, whose honest gate still reads committed s. Exercise 3.4 asks how to model the extra register.

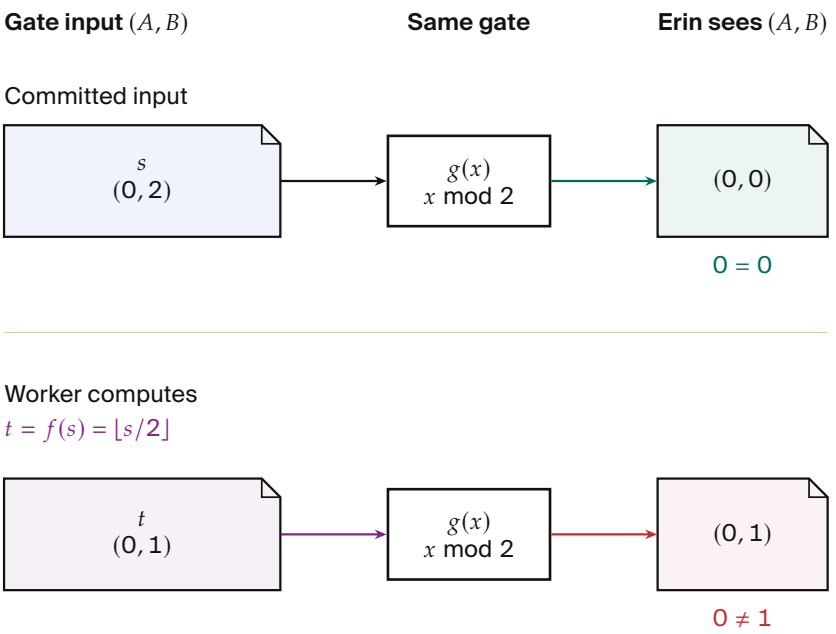


Figure 3.7. The gate is unchanged; the worker changes its argument and reveals a distinction the original parity release hid. This arithmetic example is outside C1's committed-input model. [internal; Exercise 3.4]

Current C1: `submit` has no payload;
the gate reads committed s, not worker register t.

3.4 What the mediated gate can enforce

Express lane: exact realization of a safety policy requires controllability and, under partial observation, compatible decisions on indistinguishable histories. This section applies the kernel chapter's criterion to a specified event model; it does not prove that no other architecture can mediate finer events.

Numbers by Hand 3.4.1 (Eight Events, Five Vetoes, and One Rule That Only Looks Like an Exception).

Sort the agent's alphabet into two piles before reading any theorem about it. Five events the mediation layer can veto before they take effect: `fs_write`, `net_egress`, `exec_tool`, `git_push`, `spawn_child`.

Three declared uncontrollable in this illustrative plant: `model_emit_token`, `in_context_read`, `internal_plan`.

Eight events, and therefore eight policies of the plain form *never do* X, of which exactly five are regimentable and three cannot be prevented at their enabled transitions. This conclusion uses the plant in which those transitions are enabled; it does not establish that their occurrence is observable [internal, `b3_controllability.py`].

Now the ninth policy, the one the whole room rests on: *no* `net_egress` *after* `in_context_read` *of a secret*. It *mentions* an event from the second pile, which is what makes it look like a counterexample, but read what it actually has to disable and the answer is `net_egress` and nothing else — the read is permitted and merely recorded as taint. The checker also assumes the read is observed. With that assumption, the supervisor knows when to disable egress and realizes the rule exactly. With an invisible read, the same visible history can require both allowing and refusing egress: refusing every egress is safe but is no longer exact realization of the requested policy.

Now you try: classify *no* `in_context_read` *of a secret* the same way, in two sentences. (Exercise 3.5.)

The kernel chapter proved that a prefix-closed safety policy is regimentable exactly when it is controllable with respect to the uncontrollable alphabet, and classified the agent's events accordingly: the mediation layer can veto `fs_write`, `net_egress`, `exec_tool`, `git_push`, and `spawn_child`, and can at best observe `model_emit_token`, `in_context_read`, and `internal_plan`. The decisive case for this chapter is the compound rule *no* `net_egress` *after* `in_context_read` *of a secret*, and under its full-observation assumption the checker's verdict is that it is regimentable: the policy permits the uncontrollable read, records it as taint, and gates only the controllable egress [internal, `b3_controllability.py`]. Gating the uncontrollable trigger, the model reading, thinking, or emitting, would be unregimentable; gating the controllable effect is not.

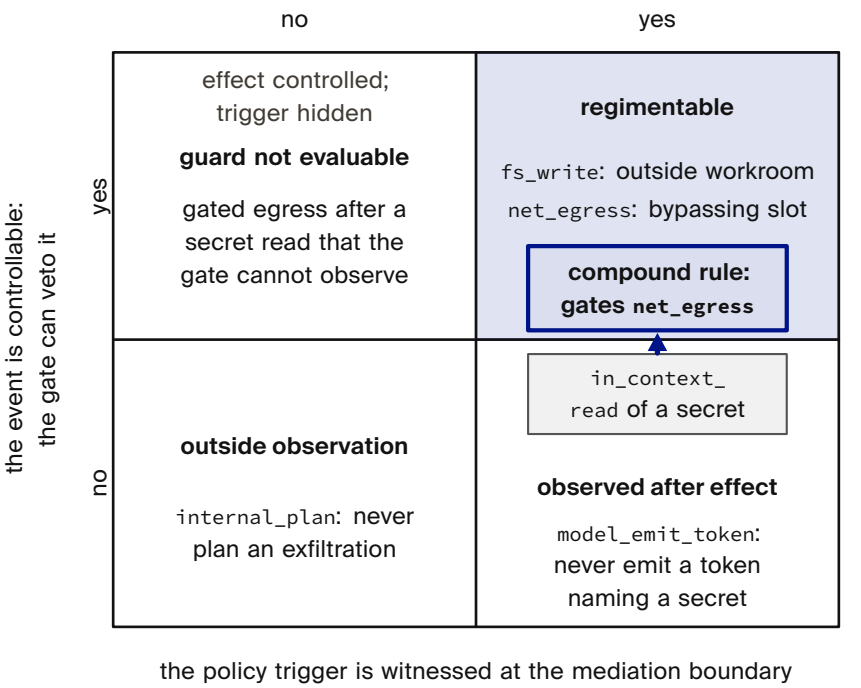
Figure 3.8 places the same five policies on the controllability x witnessing quadrant this book already uses for the general theorem, so the compound rule's trigger and its enforcement point can be seen to sit in different cells.

The design consequence is modest and useful: place a guard before egress, retain the observed secret-read restriction, and

Table 3.5.
Five policies classified by controllability: verdicts concern the declared plant under full observation. Exact realization requires preserving its legal behavior as well as preventing violations.

Policy	Constrains	Alphabet	Verdict
no `fs_write` outside the workroom	`fs_write`	controllable	regimentable
no `net_egress` except through the slot	`net_egress`	controllable	regimentable
never plan an exfiltration	`internal_plan`	uncontrollable	not preventable here
never emit a token that names a secret	`model_emit_token`	uncontrollable	not preventable here
no `net_egress` after `in_context_read` of a secret	**read (trigger), egress (gate)**	**mixed**	**regimentable: taint, then gate**

Figure 3.8.
Exact realization needs control and the relevant observations. An egress gate may miss a secret read. Observing that read permits the compound rule; denying every egress instead preserves safety while rejecting legal behavior. [internal]



mediate every release in the claimed observation model. Whole-worker labeling is the conservative policy chosen in Design invariant 3.1.2; the controllability theorem does not establish a universal impossibility of finer analysis.

What it buys. The controllability check establishes that the specified gate can make the required decisions under its observation

assumptions. The finite two-run check establishes the separate parity-release property to depth seven. Together they organize implementation obligations. Neither alone proves confinement, bounds all hardware observations, or closes every possible design. The taint-drop-egress mechanism of the naive pitch survives demoted to what it really is: a containment layer beneath the gate, the Confined rung of the assurance ladder, not the guarantee. The tool-level instruments of *The Single-Writer Kernel*, the guard, the shim, and the hook harness that rides inside a vendor's agent loop, sit below even that rung: they are the Observed and Coordinated modes, and the room replaces them rather than extending them, because a hook a process can decline to invoke gates nothing. What this ladder grades — argued at length where the modes are bought, in *The Single-Writer Kernel* — is what an adversary would have to defeat for one runtime claim about one mechanism to be false. That is a third axis, and it is independent of the two public gradings it resembles: Sheridan and Verplank's levels of automation, refined into four processing stages by Parasuraman, Sheridan and Wickens, grade how much of the decide-and-act loop the machine holds rather than the human, and the Common Criteria's evaluation assurance levels grade how much independent evaluation effort a security claim received before deployment. Neither says which component's failure would falsify a particular record, which is the only question this ladder asks.

Sheridan & Verplank 1978

R. Parasuraman, T. B. Sheridan, and C. D. Wickens. A model for types and levels of human interaction with automation. *IEEE Transactions on Systems, Man, and Cybernetics – Part A*, 30(3):286–297, 2000.

ISO/IEC 15408-1:2022

> Controllability is relative to the mediated alphabet: partial observation can shrink the exactly realizable set (Lin–Wonham), and policies that reference unmediated *state* need that extension, which the kernel chapter states.
>
> When an otherwise unpreventable violation is observable, detection can support post-hoc audit and remedies, which *The Bonded Commons* treats.
>
> And complete mediation, that every effect passes through the layer, is the deployment assumption the two remaining links inherit: it requires evidence about the workroom; an attestation measurement alone does not establish that the measured implementation has the property.

Where this stops

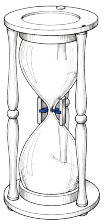
F. Lin and W. M. Wonham. On observability of discrete-event systems. *Information Sciences*, 44(3):173–198, 1988.

3.5 The budget is a ledger, and the ledger conserves

Rogers, Roth, Ullman and Vadhan distinguish two privacy accountants: a **privacy odometer** bounds accumulated privacy loss without a pre-specified ceiling; a **privacy filter** stops before a pre-specified budget is exceeded. This ledger supplies the latter's sum-only stopping rule. Its privacy interpretation requires valid private releases, not just consistent arithmetic.

Express lane: make the privacy budget a ledger, so that one atomic transition appends the record and adds the spend, and conservation of $\sigma = \sum_{\Lambda} \varepsilon_i$ survives every interleaving, because the single writer makes every concurrent history observationally sequential; the statement is Theorem 3.5.1 and the checked instance is Property 3.5.2.

R. Rogers, A. Roth, J. Ullman, and S. Vadhan. Privacy odometers and filters: pay-as-you-go composition. In *Advances in Neural Information Processing Systems 29*, pages 1921–1929, 2016.



Like an hourglass whose precision orifice meters the escape of sand grain by grain, the differential privacy ledger meters information release under a strict cumulative budget.

The scene, resumed. Derek's data never leaves the room; what leaves is a stream of gated releases, each carrying a differential-privacy cost ε_i. The contract says the total never exceeds $\varepsilon_{\max}$. But releases race: two of Erin's jobs invoke the gate concurrently, and the classic gap between "is there budget?" and "spend it" is exactly where a meter drifts from its journal. The intuition is double-entry bookkeeping: balance and journal updated in one stroke, so an auditor summing the journal always reproduces the balance. The misread to preempt: conservation governs *recorded* spend. That all spend is recorded is the complete-mediation invariant of Section 3.2, which is why the enforceability link comes first.

Differential privacy in one paragraph. Fix which datasets count as neighbors: here they differ in one person's record. A randomized mechanism A is ε-private if, for every neighbor pair D, D' and every output event S,

$$\Pr[A(D) \in S] \le e^{\varepsilon} \Pr[A(D') \in S].$$

The inequality also applies with D, D' reversed. It compares distributions, not the visual similarity of two answers. For a one-bit record, a mechanism that reports the true bit with probability $3/4$ and the other bit with probability $1/4$ has worst ratio 3, hence $\varepsilon = \ln 3$. Reporting the bit exactly fails every finite epsilon bound: one

dataset gives an event probability one and its neighbor gives zero. This is why the deterministic parity gate above is not automatically a private mechanism. For approximate (ε, δ) privacy the right side also has an additive allowance δ; that parameter is not a number of leaked bits. Privacy costs add under basic composition. For k conditionally ε-private releases with caps and horizon fixed in advance, advanced composition gives (ε', δ')-privacy for $0 < \delta' < 1$, where

C. Dwork. Differential privacy. In *Proc. ICALP 2006*, LNCS 4052, pages 1–12, 2006. Online source

$$\varepsilon' = \varepsilon\sqrt{2k\ln(1/\delta')} + k\varepsilon(e^{\varepsilon} - 1).$$

A potentially smaller ε' costs a positive δ'; it is not a stronger pure-DP guarantee. The ledger records declared costs and refuses an overdraw. Whether those costs describe private releases is a separate question.

Theorem 3.5.1 (ε-Conservation).

Initially $\sigma = 0$ and $\Lambda = ()$, with $\varepsilon_{\max} \geq 0$ and all proposed costs $\varepsilon_i \geq 0$. State $= (\sigma, \Lambda)$, where σ is the spent total and Λ the append-only release journal. The only spending transition is release(ε_i): if

$$\sigma + \varepsilon_i \leq \varepsilon_{\max}$$

it *atomically* appends $(\varepsilon_i,$ artifact hash, policy hash$)$ to Λ and sets $\sigma \mathrel{+}= \varepsilon_i$; otherwise it refuses and changes nothing.

Then every reachable state satisfies

$$\sigma = \sum_{\Lambda} \varepsilon_i \quad \text{and} \quad \sigma \leq \varepsilon_{\max},$$

including under concurrent invocation, because the kernel's single-writer serialization makes any concurrent execution observationally equal to some sequential interleaving.

Conditional privacy corollary. Fix the neighboring-record relation above. Suppose every admitted release is $(\varepsilon_i, 0)$-private conditional on the prior transcript, with mechanisms and nonnegative costs selected from public information and previous private outputs. Suppose also that the complete observer-visible interaction is covered, including metadata, refusals and timing. The basic filter theorem of Rogers, Roth, Ullman and Vadhan then supports $(\varepsilon_{\max}, 0)$-privacy: check each proposed cost *before* its release and stop before the sum exceeds the fixed budget.

Rogers et al. 2016

The accounting proof below establishes neither the conditional-DP nor complete-mediation premise.

What this theorem does not say. The equality $\sigma = \sum_{\Lambda} \varepsilon_i$ is bookkeeping over *admitted* releases, and it is exact about a quantity that is deliberately syntactic: how much declared budget has been spent. It says nothing about what an admitted artifact *means*.

A worker that computes $t = f(s)$ and submits t so the gate honestly releases $g(t)$ spends one ε_i like any other caller, and the ledger balances afterwards exactly as it did before — the semantic content of that release is Section 3.7's $q \cdot b$ residual, which is priced rather than prevented, and the closest thing to a non-interference guarantee in this chapter is Property 3.3.1, a separate and model-checked claim that holds under a committed-input hypothesis this theorem never assumes. Accounting consistency and non-leakage are two different properties, and only the first of them is proved here.

Proof. Induction over the sequential interleaving. Base: $\sigma = 0$ and $\Lambda = ()$, so the equality holds and the bound holds. Step: a refused release changes nothing; an admitted release passed the guard $\sigma + \varepsilon_i \le \varepsilon_{\text{max}}$, so the bound is preserved, and it appends and adds in one transition, so the equality is preserved. Non-spending transitions touch neither component. Concurrency reduces to the sequential case by the linearizability of the single writer, under the successful-commit and real-time-order assumptions stated in *The Single-Writer Kernel*. □

Which composition bound to quote. Do not substitute realised parameters into a fixed-parameter advanced bound and assume it is a valid stopping rule. The bound $\left(\sqrt{2k\ln(1/\delta')}\,\varepsilon + k\varepsilon(e^{\varepsilon} - 1),\ \delta'\right)$ applies here to pure-DP releases with k and the per-release cap ε fixed in advance. Queries and mechanisms may still adapt to previous releases; stopping early retains the bound for that maximum horizon. Fully adaptive advanced accounting would require a separately justified filter, such as Whitehouse et al.'s, which matches advanced-composition rates and leading constants under its stopping rule. That approximate-DP filter is not implemented by A3's

Dwork, Rothblum & Vadhan 2010. *Boosting and differential privacy*.

Whitehouse et al. 2023. *Fully-adaptive composition in differential privacy*.

sum-only ledger. Large k alone does not make the advanced bound smaller.

Whom the certificate binds. The differential-privacy reading of σ presupposes that each recorded release *is* an ε_i-private mechanism. Against the malicious worker of Section 3.7, one choosing b bits adversarially, that presupposition fails outright: such a worker is not running a private mechanism, and its recorded ε_i is a number it declared about itself. In that regime $\sigma \le \varepsilon_{\max}$ does not certify differential privacy, and the $q \cdot b$ capacity account of Section 3.7, not the ledger, is the operative bound. The two accounts are for two different adversaries and the chapter owns both, drawn side by side in Figure 3.9.

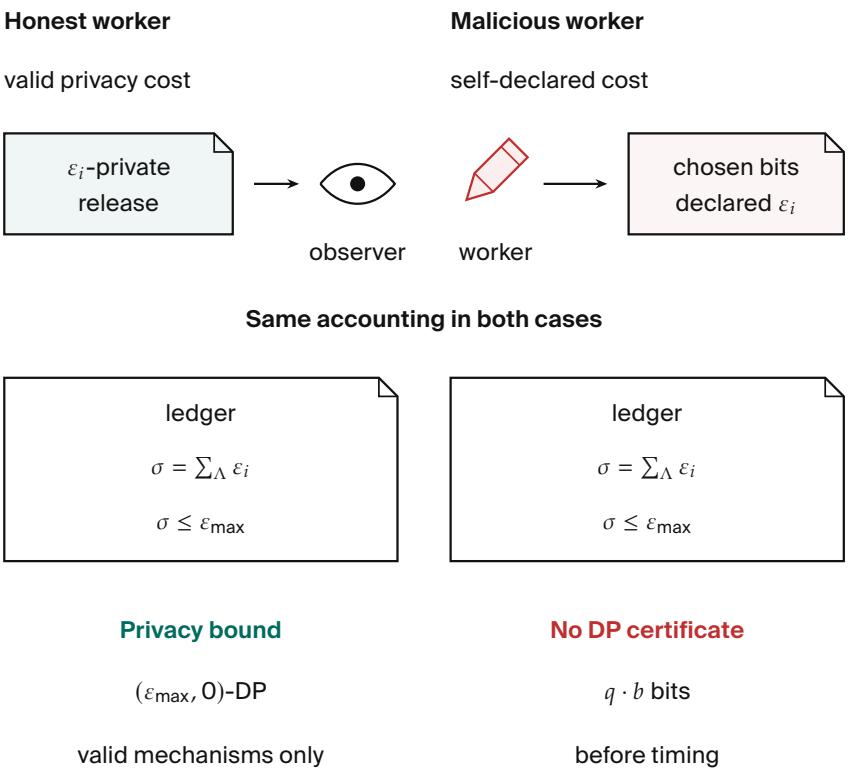


Figure 3.9. The same ledger conserves declared spend, not semantic privacy. DP needs valid mechanisms and complete mediation; $q \cdot b$ bounds only the specified output channel before timing. [internal, §3.7]

Systems that schedule privacy budgets across concurrent jobs make the corresponding assumption explicit, that the analyst is trusted to implement the mechanism correctly; this chapter declines that assumption about Erin's worker, which is precisely why it cannot lean on the ledger alone. Stating the two accounts in one

Luo et al. 2021. *Privacy budget scheduling.* OSDI, pp. 55–74.

currency needs the correspondence between ε and min-entropy of Alvim et al., which is not discharged here.

Alvim et al. 2011. *On the relation between differential privacy and quantitative information flow*.

Model-Checked Property 3.5.2 (The Ledger's Invariants on the Two-Client Instance).

Two concurrent clients racing programs $(2, 2)$ and $(1, 2)$ against $\varepsilon_{\max} = 4$: all 15 reachable states satisfy both invariants under every interleaving, and 2000 random instances of programs, budget, and interleaving show no violation.

The state count is order-sensitive: a denied release still advances the client's program counter, so distinct commit orders are distinct states; if ledger order is forgotten but both client program counters are retained, the answer is 11. There are only five distinct multisets of committed costs when those counters are also discarded.

Mutations. Spend-without-append breaks conservation in one step.

The torn write, which logs ε but adds $\varepsilon - 1$, breaks it in one step and in three steps drives the *recorded* log to

$$\sum_{\Lambda} = 5 > \varepsilon_{\max} = 4 :$$

the undercounting balance silently admits auditable overspend, so atomicity is what makes the budget promise auditable.

Reverting the mutations restores both invariants and the 15 states [internal, `a3_epsilon_ledger.py`].

Figure 3.10 grids both mutants against both invariants, with the step count each is caught in.

	spend without append	torn write
ledger balance *the recorded sum*	◆ caught 1 step	◆ caught 1 step
budget cap *spend at or under the limit*	not exercised	◆ caught 3 steps

Figure 3.10. Both atomicity mutants break balance immediately. Only the torn write breaches the budget cap, after three steps in this two-client model. The intact model's 15 reachable states satisfy both invariants. [internal, `a3_epsilon_ledger.py`]

Numbers by Hand 3.5.3 (When the Advanced Composition Bound Starts to Pay).

At $\delta' = 10^{-6}$: for $k = 32$ releases at $\varepsilon = 0.1$, basic composition gives

$$32 \times 0.1 = 3.20$$

while the advanced bound gives 3.31, so the advanced bound is not yet paying; at $k = 128$, $\varepsilon = 0.05$, basic gives 6.40 but advanced gives 3.30.

These comparisons fix the caps and horizon. A smaller advanced epsilon permits $\delta' = 10^{-6}$; basic composition remains pure-DP. The crossing compares epsilon values, not uniformly stronger guarantees.

Now you try: at $k = 8$, $\varepsilon = 0.5$, which bound is smaller, and at $\varepsilon = 0.1$, at which k does the advanced bound first pay? (Exercise 3.8.) Figure 3.11 plots both bounds against k and marks this exact crossing.

`a3_epsilon_ledger.py` · 20260816 · verified

What it buys. The contract's budget line becomes an auditable invariant with the same proof shape as the bond-conservation result of *The Bonded Commons*, and the mutation suite shows the invariant has teeth: both canonical atomicity breaks are caught with shortest traces, and the torn write's deeper crime, recorded overspend past the contracted maximum, is exhibited concretely.

> Conservation governs recorded spend; that all spend is recorded is the complete-mediation invariant, so the enforceability link is the prerequisite, not a nicety.

Where this stops

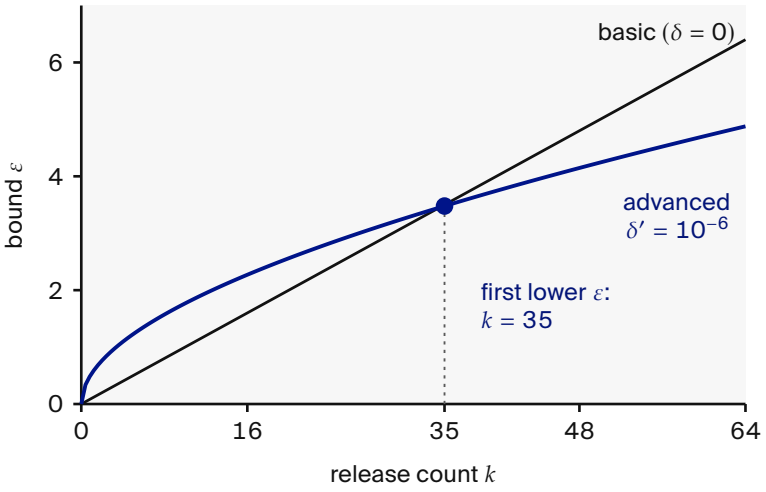


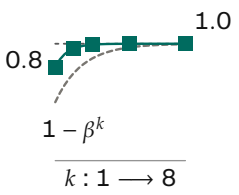
Figure 3.11. Analytical comparison for per-release $\varepsilon = 0.1$ and a fixed maximum horizon; queries may adapt. At $k = 32$, basic is 3.20 versus advanced 3.31. Advanced first has lower ε at $k = 35$, but allows $\delta' = 10^{-6}$; basic keeps $\delta = 0$. Lower epsilon therefore trades guarantees, not uniformly stronger privacy.

A crossover is specific to the chosen privacy parameters. And the theorem conserves the meter, not the meaning: choosing per-release ε_i honestly is the mechanism's obligation, outside this ledger.

Differential privacy bounds individual influence in aggregates; it does not protect the whole dataset, nor Erin's model.

3.6 Canaries with a power curve and a clock

Express lane: independent per-canary misses turn k smuggled canaries into detection probability $1 - \beta^k$; uniform planting turns leak size into a hypergeometric operating curve; and an i.i.d. Bernoulli model gives Wald's sequential test a stopping-time identity and a useful approximation; the statements are Theorems 3.6.1 and 3.6.2.



Detection power: $1 - \beta^k$ as captured canary count k scales ($\beta = 0.2, 0.5$).

Intuition: dye packs and quality control. Dye packs in bank bundles: each pack fails alone with probability β, so grabbing k dyed bundles fails to stain with probability β^k; how many dyed bundles a thief grabs is set by how much he takes and how densely packs are planted. The sequential half is quality control: each gate output is a Bernoulli trial, and a sequential probability ratio test accumulates evidence for "is this stream leaking?" The three links before this one govern what the gate is *allowed* to pass; this one measures

what happens when something tries to pass anyway. “We have canaries” is a mood; a security officer buys a number. The misread to preempt: the two theorems answer two different questions, and a single-shot exfiltration is the first theorem’s job, not the second’s.

What the detector observes. A *canary* is a deliberately planted, recognizable span whose presence in an observed output is evidence of a specified leak. A false negative misses a carried canary; a false alarm flags an innocent output. These are different errors. The power calculation first conditions on k canaries actually being carried. The hypergeometric model then accounts for not knowing k: it is the count of marked spans in a sample drawn without replacement.

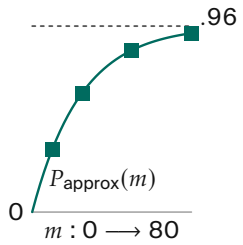
For the sequential test, let $X_i = 1$ mean that observed output i is flagged, and $X_i = 0$ otherwise. Under the null hypothesis these independent observations have flag probability p_0; under the leak hypothesis, $p_1 > p_0$. The accumulated log-likelihood ratio is

$$L_n = \sum_{i=1}^{n} \left[X_i \ln \frac{p_1}{p_0} + (1 - X_i) \ln \frac{1 - p_1}{1 - p_0} \right].$$

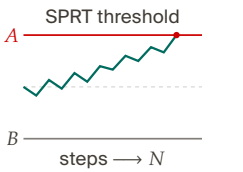
A flag pushes the score up; a non-flag pushes it down. The sequential probability ratio test (SPRT) stops at the first crossing of an upper or lower threshold. The divergence used below is the expected upward increment under the leak model:

$$\mathrm{KL}(p_1 \| p_0) = p_1 \ln \frac{p_1}{p_0} + (1 - p_1) \ln \frac{1 - p_1}{1 - p_0}.$$

Its unit is nats per observed output. A stopping count therefore counts outputs, not seconds; a wall-clock claim needs an arrival model as well.



Approximate operating curve: detection vs. leak size m, with $\frac{c}{n}(1-\beta) = 0.04$. The exact fixed-count planting model samples without replacement.



Wald stopping boundary: Sequential ratio walk with thresholds A, B.

Wald 1945. *Sequential tests of statistical hypotheses.*

Wald & Wolfowitz 1948. *Optimum character of the sequential probability ratio test.*

Theorem 3.6.1 (Power).

With per-canary false-negative rate β, independent across canaries, a leak carrying k canary spans is detected with probability

$$1 - \beta^k.$$

Under uniform planting of c canaries in an n-span corpus, a leak of m spans chosen independently of their hidden locations carries $K \sim \text{Hypergeom}(n, c, m)$ canaries, so

$$\Pr(\text{detect} \mid m) = 1 - \mathbb{E}[\beta^K] \approx 1 - \left(1 - \tfrac{c}{n}(1-\beta)\right)^m,$$

the operating curve.

Theorem 3.6.2 (Latency (Wald)).

Assume i.i.d. Bernoulli gate outputs under two fixed, simple hypotheses, with rates $0 < p_0 < p_1 < 1$.

Let L_N be the accumulated log-likelihood ratio of leak to null at the uncapped SPRT's first exit from fixed thresholds, N, with finite expectation. Wald's identity gives

$$\mathbb{E}_1[N] = \frac{\mathbb{E}_1[L_N]}{\mathrm{KL}(p_1 \| p_0)}, \qquad \mathbb{E}_0[N] = \frac{-\mathbb{E}_0[L_N]}{\mathrm{KL}(p_0 \| p_1)}.$$

Among tests with no larger *actual* false-alarm and miss probabilities, the SPRT minimizes both expected sample sizes.

For fixed nominal targets $\alpha, \gamma > 0$, $\alpha + \gamma < 1$, the usual thresholds are $A = (1-\gamma)/\alpha$ and $B = \gamma/(1-\alpha)$. Replacing the terminal log-likelihood by $\ln A$ or $\ln B$ and the actual error probabilities by the targets gives the *no-overshoot approximation*

$$\mathbb{E}_1[N] \approx \frac{(1-\gamma)\ln A + \gamma \ln B}{\mathrm{KL}(p_1 \| p_0)}.$$

Here α targets false alarms; γ targets misses and is distinct from the per-canary miss rate β and the release-channel width b. These approximate thresholds do not separately guarantee the two nominal error rates.

Jung, Paxson, Berger and Balakrishnan used this machinery for portscan detection in 2004. Their threshold random walk distinguishes benign from scanning hosts; their analysis explicitly separates approximate sample counts from attained errors. The canary gate borrows that statistical question, not a guarantee that real gate outputs fit its model.

Jung et al. 2004. *Fast portscan detection using sequential hypothesis testing*.

Proof idea. Power is independence and counting: a leak is missed only if every carried canary is missed, and the number carried under uniform planting is the hypergeometric draw of m spans from n with c marked. Latency is Wald's identity: the log-likelihood ratio drifts at rate $\mathrm{KL}(p_1 \| p_0)$ per output under the leak. The identity uses its *actual terminal value*, including any boundary overshoot. It is the approximation above, not the identity, that replaces that value by the boundary.



Wald's test decides when to stop collecting evidence.

Reported checks. The chapter's recorded A4 example uses $n = 10^4$ records with $c = 100$ canaries (density one percent), against a channel through which each smuggled canary evades the detector with probability $\beta = 0.2$: a 100-span leak is caught with exact hypergeometric probability 0.554, an 800-span leak at 0.999. The recorded conditional-power comparisons at $k = 1, 2, 5$ (analytic 0.800, 0.960, 0.99968 against simulated 0.8016, 0.9608, 0.99967) are simulation reports, not new measurements here. At $p_0 = 10^{-3}$, $p_1 = 10^{-2}$, $\alpha = 0.01$, $\gamma = 0.05$, the no-overshoot approximations are 296.9 outputs under leak and 431.9 under null; the recorded simulation means are 359.0 and 438.1. Reported miss 0.048 and false alarm 0.005 are point estimates, not certified upper bounds. A4's assertions permit twice the nominal error targets. Its 200,000-output cap is not reported separately: an unresolved capped run is counted with non-upper decisions. Those records cannot establish that all runs terminated [internal, `a4_canary_sprt.py`].

Numbers by Hand 3.6.3 (Three Canaries, One Operating Curve).

At $\beta = 0.2$, the detection probabilities for smuggled canaries are [verified]:

$$\text{two canaries:} \quad 1 - 0.04 = 0.96,$$
$$\text{three canaries:} \quad 1 - 0.008 = 0.992.$$

The operating curve at $m = 100$:

$$1 - (1 - 0.01 \times 0.8)^{100} = 1 - 0.992^{100} \approx 0.552$$

by the binomial approximation, against 0.554 exact.

`a4_canary_sprt.py` · 20260816 · internal

Now you try: at the same β, what does a five-canary leak survive with, and can you recover the approximate 296.9 from the formula with a pocket calculator? (Exercises 3.9 and 3.10.) Figure 3.12 compares exact power, approximate stopping counts and reported capped means.

What it buys. Once the model is validated and the errors calibrated, a contract could quote detection power and stopping latency under stated conditions. The exfiltration bond of *The Bonded Commons* is scoped to funding detection and response, rather than pretending to cover unbounded breach damage.

Where this stops

Independence and *secrecy* of planting are decisive: an adversary who obtains the canary list with probability 0.3 and strips all of them drops reported simulated power from 0.9997 to 0.698 at $k = 5$ [internal, `a4_canary_sprt.py`]; canary secrecy is part of the security boundary.

β is a model input that requires measurement for the intended deployment.

The nominal-target stopping-time formulas are approximations, not certified latency or error bounds.

And the sequential test detects *statistical* leak signatures; a single-shot exfiltration is Theorem 3.6.1's job.

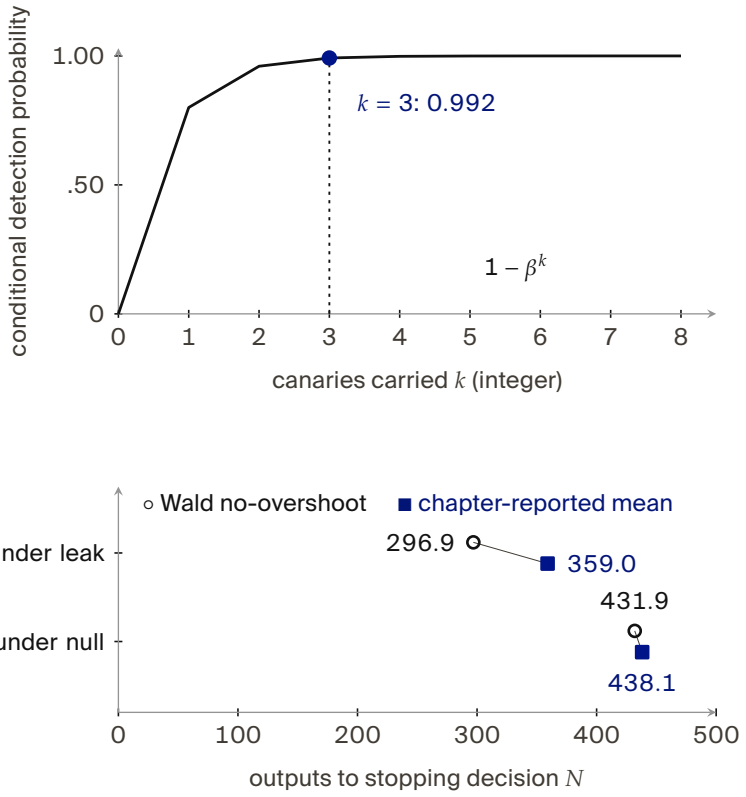


Figure 3.12. Three carried canaries give 99.2% conditional detection at $\beta = .2$. The stopping-time comparison uses $p_0 = .001$, $p_1 = .01$, $\alpha = .01$, $\gamma = .05$; reported capped simulation means have no supplied uncertainty and do not certify error control.

3.7 The leakage budget: $q \cdot b$ bits, before timing

Numbers by Hand 3.7.1 (Twelve Thousand Eight Hundred Bits, and the Only Two Dials).

A work order grants Erin a 64-bit status schema per job, over 200 jobs. Multiply, and that's the account:

$$200 \times 64 = 12{,}800 \text{ bits}$$

of adversarial capacity, before timing.

Two things about that arithmetic are worth a second look, and neither needs a model. First, the account is a *product*, so the contract writer has exactly two dials and they trade off against each other — the same 12,800 bits fall out of a narrower schema over a longer engagement — and for an engagement with $q > 0$,

the only setting that drives this capacity bound to zero is $b = 0$, which closes the feedback channel that was Erin's reason for signing in the first place. (That is what Section 3.8 means when it says "zero" is available only at $b = 0$; it is a price, not an impossibility.)

Second, and this is the uncomfortable half, Theorem 3.3.1 does not narrow those 12,800 bits by a single one: it describes a different interface, whose gate computes on committed input. It cannot be transferred to an arbitrary worker-selected payload interface. The room sells a bounded account. It does not sell an innocent one.

Now you try: the same work order lets each job return at one of three scheduled times — what does that add to the total? (Exercise 3.12.)

The four links compose into one information-theoretic account, and the account is deliberately unromantic. It is also where the chapter pays three debts it took on earlier: the laundering residual left open by Theorem 3.3.1's committed-input hypothesis, the malicious worker for whom $\sigma \le \varepsilon_{\max}$ does not certify differential privacy, and the arbitrary-telemetry channel disclaimed in Section 3.8. All three are the same debt seen from three sides, and it has one price, drawn converging in Figure 3.13.

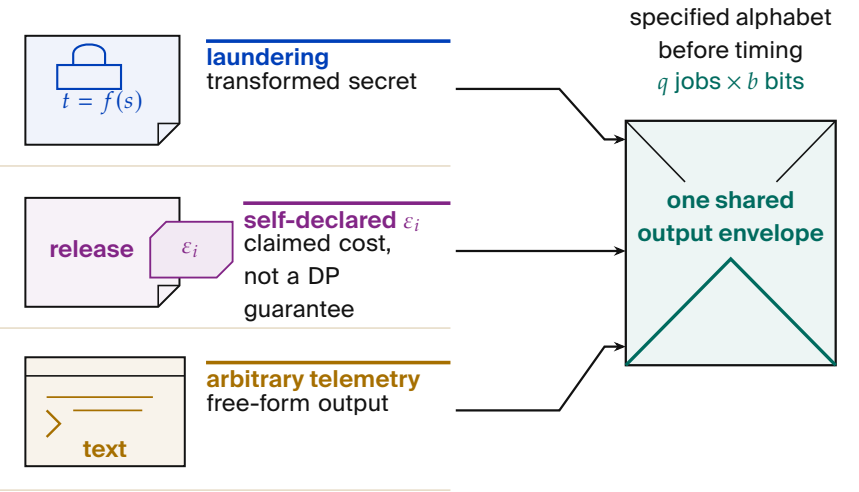


Figure 3.13. Alternative outputs share one $q \cdot b$-bit envelope. A claimed privacy parameter is not a bit count. Timing and other unmodeled side channels lie outside this accounting bound.

If an Erin-visible channel permits b freely chosen bits per job, a malicious worker, one that launders by computing $t = f(s)$ and

submitting t so that the gate honestly releases $g(t)$, can use that freedom to encode secret bits. The alphabet has at most 2^b values, giving a b-bit upper bound per fixed-length output. This is a bound on permitted distinguishability, not a claim that every workload leaks that much, and it is not the committed-input noninterference theorem. Across q jobs the raw capacity is $q \cdot b$ bits; choosing among s timing slots adds up to $\log_2 s$ bits per event, which is why timing sits outside the possibilistic model. The $\log_2 s$ expression is a finite-alphabet upper bound: across at most q events, at most s^q slot sequences are distinguishable. Predictive mitigation provides a framework for controlling timing observations; the slot count here is not a proof for an arbitrary scheduler.

The observation model matters. Infinite timing precision can admit unbounded information in an idealized duration, but practical capacity also depends on clock resolution, noise, observation windows, and bandwidth. A bucketed-release design must account for aborts, missed slots, termination, and the number of events. If those vary with secrets and remain visible, quantizing successful releases alone leaves a channel open. Fixed deadlines and padded outcomes, including failure and no-result cases, are additional design obligations.

Finite output alphabets and query limits can bound specified observations; they do not bound every host side channel. Nor are bit budgets interchangeable with differential-privacy parameters: the latter require the mechanism and adjacency assumptions of Section 3.5. Canaries provide probabilistic detection evidence, not exhaustive policing of the allowed capacity. Contracts may price residual risk, but neither the price nor a clean canary run proves that semantic laundering did not occur.

A. Askarov, D. Zhang, and A. C. Myers. Predictive black-box mitigation of timing channels. In *Proc. 17th ACM Conference on Computer and Communications Security (CCS)*, pages 297–307, 2010.

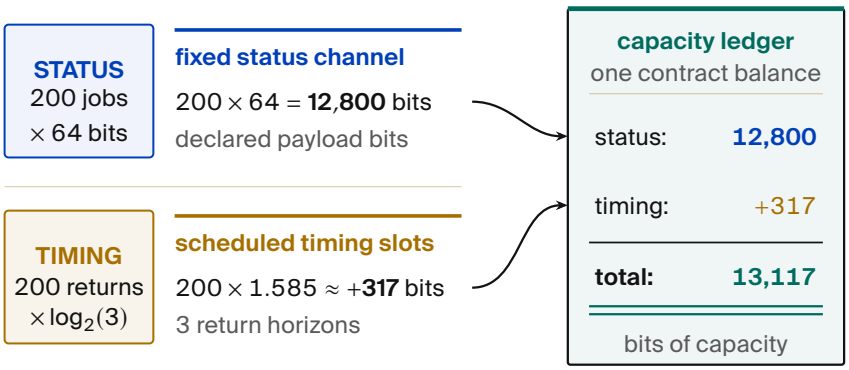


Figure 3.14. For 200 jobs, status and three return slots allow about 13,117 bits of capacity. The ledger counts payload and timing channels separately. [hand-derived; Exercise 3.12]

3.8 Limitations and boundaries

None of the following is promised, and a contract that implies any of them overclaims.

Table 3.6 lists six of them: for each, what mechanism is responsible for the gap, and where – if anywhere – the residual is priced.

Table 3.6. Six things this design does not promise, why not, and where the residual is priced. A contract that implies any of these overclaims.

Not promised	Why not	Residual priced by
Zero model leakage under unlimited adaptive queries to Derek	Derek *intentionally observes* evaluations of M; unlimited black-box access enables model extraction	out of model: bounded only by the contracted query-and-output interface
Zero data leakage with arbitrary natural-language telemetry to Erin	that channel is exactly the $q \cdot b$ budget; "zero" needs $b = 0$	the $q \cdot b$ budget (Sec. 3.7)
Elimination of timing, cache, speculative, firmware, or physical channels	the noninterference guarantee is possibilistic	out of model; padded and scheduled, not closed
Confidentiality through an unapproved external model or tool provider	a remote API that reads plaintext is another trusted reader	not priced: the model must run inside the confidential domain, or the guarantee does not apply
Correctness of the answer	the receipt proves what the measured system reported, not that it is true	out of model
Literal deletion of information already observed, availability under refusal, or collusion resistance	each needs a strictly stronger threat model	not supplied by the room; additional mechanisms require their own threat models

One more limit sits outside even that table: bounds on minds. A human who reads a released aggregate may still remember it; the system bounds channels, not memories.

The lift. Theorem 3.3.1 is exhaustive on a finite model; the general claim over unbounded state is a proof obligation with a known shape. Rushby's channel-control formulation reduces noninterference for a transition system to three local *unwinding conditions*, output consistency, step consistency, and local respect, each

Rushby 1992

a per-transition lemma amenable to mechanization, and the Isabelle Archive of Formal Proofs already carries stateful intransitive-noninterference developments in exactly this lineage. The finite model is the blueprint: its local-respect check already holds mechanically, its declassification steps are the intransitive downgrader domain of Rushby's framework, and the lift consists of restating the transition relation abstractly and discharging the three lemmas once, for all depths and all secrets. One precision the lift must get right from the start: Rushby's soundness result for these conditions is with respect to intransitive purge, but van der Meyden shows the conditions are sound *and complete* for the strictly stronger TA-security property, which additionally preserves the *order* of permitted transmissions. Discharging the three lemmas therefore certifies TA-security, and the lift should state that target explicitly. Two extensions of the finite model are open alongside the lift: a `submit` that carries a payload, so the mutation suite can exhibit laundering, and a partial-observation variant of the enforceability check for policies that reference unmediated state. The status of every claim in this chapter is: the two exhaustive checks are bounded model evidence; the canary simulation values are chapter-reported, without an archival run receipt or calibrated error bounds supplied here. The room is a proposed confinement design whose complete-mediation obligation must be validated for each deployment.

van der Meyden 2007. *What, indeed, is intransitive noninterference?* (extended abstract).

Review of the key ideas

What this chapter leaves coupled, in the order the rest of the book will lean on it:

1. A **sealed room** lets an agent compute on data it may not export: the data owner and the model owner each get their guarantee from a fence they can inspect, not from trust in the other.
2. **Whole-worker labels are conservative.** This design treats arbitrary worker output as depending on every secret the worker read; authorized release is a separate operation.
3. **Two fences, two gates**: the work order fixes what may enter, the slot fixes what may leave, and the model in between is treated as an opaque function of both.

Recall

1. Without looking: state the sentence that replaces "mathematically cannot phone home," and distinguish its model evidence from the deployment assumptions needed to make the promise true.
2. Which link must hold before the conservation theorem says anything about the deployment, and why?
3. A contract grants a 32-bit status per job over 400 jobs. What is the capacity bound before timing? Why does a canary detector not establish that the bound is enforced?

Exercises 3.1–3.12, p. 270.

4. **Noninterference is checked by running the world twice** (self-composition, in the verification literature's word): identical except for the secret, the two runs must agree on everything that leaves through the slot, modulo the declassification the gate performs. The finite clean-room model passes this check to depth seven.
5. **Enforcement is relative to a plant and observations**: this gate can refuse egress, and an observed read can tell it when to do so. Internal events declared uncontrollable cannot be disabled at that boundary; their detection needs separate evidence.
6. **The privacy budget is a ledger that conserves**: every release debits ε, the ledger's total never exceeds $\varepsilon_{\max}$, and a torn write is caught by the invariant within three steps on the two-client instance.
7. **Canaries have conditional power**: a leak carrying k canaries escapes all their independent detections with probability β^k (0.008 at $\beta = 0.2$, $k = 3$). The sequential model estimates outputs until either decision, not a guaranteed alarm deadline.
8. The whole channel is priced before timing: $q \cdot b$ **bits** may leave through q queries of b bits each, and the room's guarantee is that budget, not silence.

3.9 Exercises

§3.8 Limitations and boundaries

Solution p. 730

3.1 Check ⋆

With $g(s) = s \bmod 2$ over secrets $\{0, \ldots, 7\}$, how many unordered secret pairs must a sound room keep Erin-indistinguishable, and how many may differ at release steps?

Table 3.7. For $g(s) = s \bmod 2$, equal-parity pairs stay hidden; different-parity pairs may differ only at release.

Secrets	Unordered pairs	Equal parity	Different parity
4	$\binom{4}{2} = 6$	2	4
8	$\binom{8}{2} = 28$	12	16

3.2 Trace ⋆⋆ *Solution p. 730*

Run `c1_noninterference.py` and read the first mutation's witness. Why is the decisive pair $(0, 2)$ rather than $(0, 1)$, and at which step of the four does Erin's view first differ?

3.3 Check ⋆ *Solution p. 731*

The second mutation is caught in three steps. Which side property of Design invariant 3.2.2 does the mutant violate, and why is that violation detectable by the two-run check at all?

3.4 Open ⋆⋆⋆ *Solution p. 731*

Extend the finite model so that `submit` carries a payload the worker computes from the secret, and state what the checker would then have to quantify over to exhibit the laundering attack. Contrast a gate that computes on committed input with one that accepts an unconstrained worker payload. Which premise fails in the second case?

3.5 Check ⋆ *Solution p. 731*

Using the controllability theorem of *The Single-Writer Kernel*, under the illustrative full-observation plant explain why "no `net_egress` after an `in_context_read` of a secret" is regimentable but "no `in_context_read` of a secret" is not.

3.6 Trace ⋆⋆ *Solution p. 732*

Reconstruct by hand the three-step torn-write crime on the two-client instance (programs $(2, 2)$ and $(1, 2)$, $\varepsilon_{\max} = 4$): give the sequence of releases, the balance σ after each, and the recorded sum $\sum_{\Lambda}$, and show where the recorded sum exceeds the maximum.

3.7 Trace ⋆⋆ *Solution p. 732*

The checked instance has 15 reachable states. Forgetting ledger order while retaining client program counters yields 11 states; discarding those counters as well leaves five cost multisets. Explain what each projection forgets and exhibit two full states with the same cost multiset.

3.8 Check ⋆ *Solution p. 732*

At $\delta' = 10^{-6}$: which bound is smaller at $k = 8$, $\varepsilon = 0.5$? And at $\varepsilon = 0.1$, what is the smallest k at which the advanced bound falls below basic composition?

Solution p. 732

3.9 Check ⋆

At $\beta = 0.2$, with what probability does a five-canary leak survive undetected? Then use the binomial approximation of the operating curve at $n = 10^4$, $c = 100$, $m = 100$ and compare with the exact value 0.554.

Solution p. 733

3.10 Trace ⋆⋆

Recover the no-overshoot approximation, $\mathbb{E}_1[N] \approx 296.9$, following Theorem 3.6.2 at $p_0 = 10^{-3}$, $p_1 = 10^{-2}$, $\alpha = 0.01$, $\gamma = 0.05$. Why need the reported simulation mean of 359 not equal it, and why is neither number a guaranteed alarm deadline?

Solution p. 733

3.11 Open ⋆⋆⋆

An adversary obtains the canary list with probability 0.3 and strips every canary it knows. Reported simulated power at $k = 5$ falls from 0.9997 to 0.698. Propose one contract clause and one mechanism change that restore most of the lost power, and say what no redesign can restore.

Solution p. 733

3.12 Check ⋆

A work order grants Erin a 64-bit status schema per job over 200 jobs, and each job may return at one of three scheduled times. What adversarial capacity has the contract granted, before and after timing?

3.10 History and references

Almost nothing in this chapter is new machinery; what is new is which pieces were bolted to which, and the honest accounting of what the bolted-together thing does not promise. It is worth saying where each piece came from.

Noninterference and its proof technique. Noninterference is Goguen and Meseguer's, stated at Oakland in 1982 as a property of a security *model* rather than of any particular mechanism, and given two years later the proof technique that made it usable: replace the quantification over all runs with local *unwinding* conditions you can discharge one transition at a time. Rushby's 1992 report is the version this chapter actually leans on, because it separates transitive from intransitive policies — which is precisely what allows a

Goguen & Meseguer 1982

Goguen & Meseguer 1984

room to have a declassifier and still have a theorem — and because it fixes the three unwinding conditions, output consistency, step consistency and local respect, that Section 3.8 names as the lift's proof obligation. Van der Meyden later showed those conditions are not merely sound but sound *and* complete, and for the strictly stronger, order-preserving TA-security rather than for the intransitive purge Rushby was aiming at; that is why the lift should be written to claim TA-security rather than to settle for what it was pointed at. The checking move itself, running the world twice and stating an ordinary invariant about the pair, is self-composition, published by Barthe, D'Argenio and Rezk in 2004; they apply it to programs with a logic, this chapter applies it to a finite model with a Python script, and the difference in ambition should not obscure that it is the same move.

Rushby 1992

van der Meyden 2007

Barthe et al. 2004

Declassification, and why the room needs the word. Conservative label propagation and the difficulty of tracking implicit flows are classical concerns, illustrated by the Dennings' certification work, with the modern restatement — a sound purely dynamic monitor must be strictly more restrictive than a static analysis — due to Russo and Sabelfeld. The escape hatch itself is Sabelfeld and Myers's delimited release, and so, importantly, is the laundering attack this chapter cannot rule out: they named it and closed it by enforcement, requiring that no variable used in a declassification be updated before it, which a compiler can check because the declassified expression is syntax. For its unrestricted worker-payload interface this chapter does not establish that premise, so it supplies a separate capacity account — that substitution, enforcement traded for a capacity account, is the chapter's own move and its own liability. The vocabulary for saying which dimension of release a mechanism actually controls is Sabelfeld and Sands's; by their taxonomy the release ledger answers *what*, *who* and *where*, and leaves *when* open. Owner-indexed labels under mutual distrust, which is the shape of the four-state lattice counted in Section 3.2, are Myers and Liskov's decentralized label model.

Denning & Denning 1977

Russo & Sabelfeld 2010

Sabelfeld & Myers 2004

Sabelfeld & Sands 2009

Myers & Liskov 1997

The monitor, and the boundary it can hold. The in-guest monitor and Derek's outer gateway are a reference monitor in Anderson's 1972 sense — always invoked, tamper-resistant, small enough

James P. Anderson. Computer Security Technology Planning Study. ESD-TR-73-51, U.S. Air Force Electronic Systems Division, 1972. (The origin of the *reference monitor* concept.)

Butler W. Lampson. Protection. *ACM SIGOPS Operating Systems Review*, 8(1):18–24, 1974.

Lin & Wonham 1988

Hunt et al. 2016

Birkholz et al. 2023

G. Smith. On the foundations of quantitative information flow. In *Proc. FoSSaCS 2009*, LNCS 5504, pp. 288–302, 2009.

Alvim et al. 2011

Dwork et al. 2010

Rogers et al. 2016

Whitehouse et al. 2023

Luo et al. 2021

Wald 1945

Wald & Wolfowitz 1948

Jung et al. 2004

to be examined — enforcing the sort of owner-indexed access structure Lampson's *Protection* gave operating systems two years later. What this chapter adds to that inheritance is not a better monitor but a sharper statement of where a monitor can be placed at all: the controllability result is proved in *The Single-Writer Kernel* and merely applied here, and the caveat that partial observation shrinks the enforceable set is Lin and Wonham's. The confinement architecture has a direct systems ancestor in Ryoan, which solved the two-suspicious-owners problem first and solved it more cleanly than this chapter does, at the price of a stateless module; the dual-attested key release follows the attester/verifier/relying-party pattern the IETF later standardized.

Pricing what does cross. Treating leakage as a quantity rather than a predicate is quantitative information flow, and the min-entropy formulation the chapter borrows its vocabulary from is Smith's; the bridge between that currency and differential privacy's exists, due to Alvim and colleagues, and this chapter does not cross it, which is why it carries two accounts rather than one. On the privacy side, advanced composition is Dwork, Rothblum and Vadhan's; the observation that a running budget needs a different theorem when privacy parameters and the stopping point are chosen adaptively, and the odometer/filter pair that supplies it, is Rogers, Roth, Ullman and Vadhan's; the fully adaptive filter that matches advanced-composition rates and leading constants is Whitehouse and colleagues'. That production budget schedulers assume a trusted analyst, an assumption this chapter explicitly declines about Erin's worker, is visible in the Privacy Budget Scheduling work. The detector is Wald's sequential probability ratio test, optimal against tests with no larger attained errors under the simple i.i.d. hypotheses, as he and Wolfowitz proved three years later; pointing it at a security stream is not new either, since Jung, Paxson, Berger and Balakrishnan did so for portscan detection in 2004. The canary gate borrows the comparison, subject to validating its own statistical model.

What is this chapter's own, and therefore uncited. Four things. Design invariant 3.1.2 chooses conservative whole-worker labels for arbitrary model output. It makes no universal impossibility claim about finer analyses. The four-link composition read as an *evidence*

schedule rather than as four redundant guarantees is the chapter's framing. The $q \cdot b$ account as the explicit price of the laundering residual — the thing the noninterference theorem leaves open and delimited release would have closed by enforcement — is an application of the standard finite-alphabet capacity bound. Its practical role is to make the unrestricted output interface measurable, alongside any stronger claims a restricted release function can justify. And the "clean room" of the framing is borrowed from industrial and legal practice for its connotation only; this chapter claims no formal ancestor for the metaphor and proves nothing about it as such.

Further reading. A reader who wants the general theorem this chapter's finite model stands in for should read Rushby and then van der Meyden in that order, and should expect the second to change what the first looks like it was proving. A reader who wants to know why the assurance ladder in Section 3.4 is a third axis rather than a rebadging of two familiar ones should read Sheridan and Verplank and its four-stage refinement for the automation-level axis, and the Common Criteria's evaluation assurance levels for the evaluation-effort axis; neither answers which component's failure would falsify a given runtime record, which is the only question the ladder asks.

Rushby 1992

van der Meyden 2007

Sheridan & Verplank 1978

Parasuraman et al. 2000

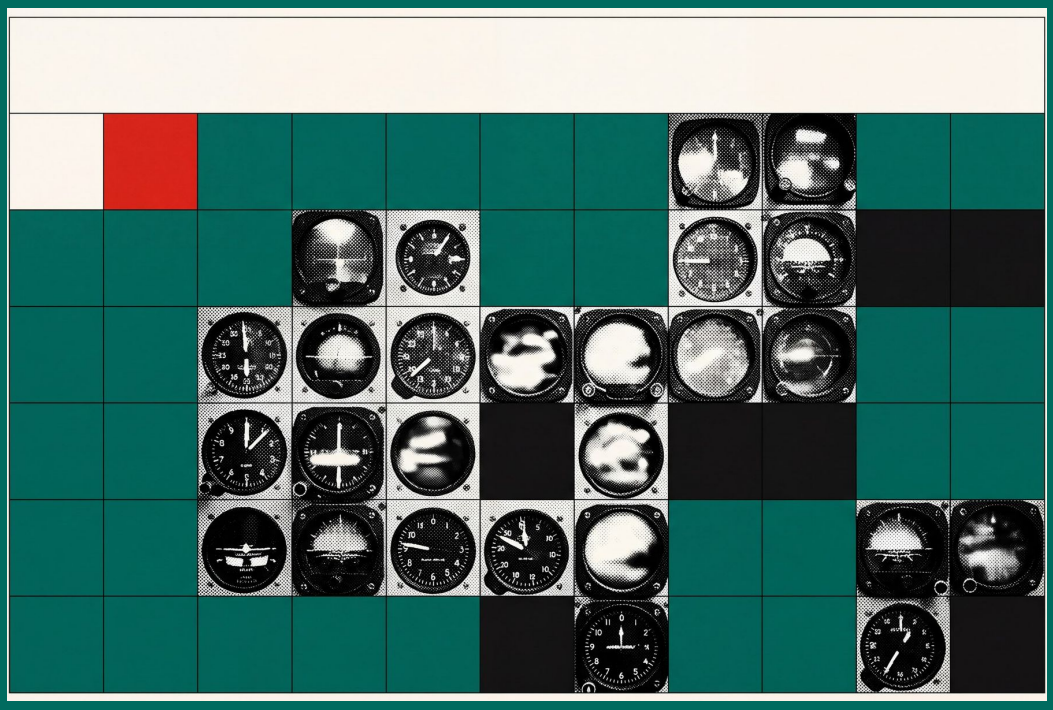
ISO/IEC 15408-1:2022

What *The Legible Swarm* receives The machine's proposed contract combines an ordered writer, checked authority, and explicit release gates. Each guarantee retains its stated trust, observation, and channel assumptions. An operator may now receive a permitted result without seeing the private inputs or internal computation. The next chapter studies that evidence boundary alongside a second limit: even visible events can arrive faster than a person can inspect them. What must a summary preserve, and when must the reader open the evidence?

II

The Cost of Seeing

An operator cannot read every action a fleet produces. This part studies what a summary must retain, when the operator needs the underlying evidence, and how information theory bounds the cost of supervision.



4 The Legible Swarm

Useful supervision requires summaries that preserve the operator's decisions and link back to inspectable evidence.

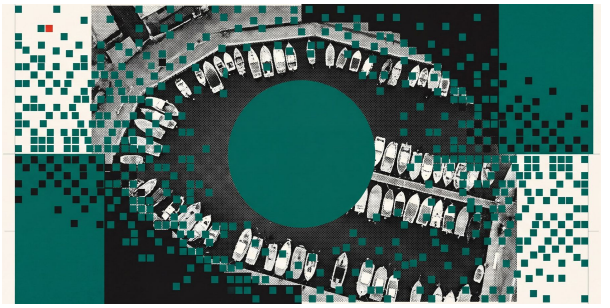
4

The Legible Swarm

The question this chapter answers

What must remain visible, and what does seeing cost?

Useful supervision requires summaries that preserve the operator's decisions and link back to inspectable evidence.



In an information-rich world, the wealth of information means a dearth of something else: a scarcity of whatever it is that information consumes. What information consumes is rather obvious: it consumes the attention of its recipients.

Herbert A. Simon, Designing Organizations for an Information-Rich World, 1971

What must remain visible, and what does seeing cost? A swarm can produce more events than its operator can read, and some underlying evidence may be private. The resulting scarcity is not cured by a smoother summary: an omitted artifact may be decisive. This chapter bounds zero-miss inspection in a stated model, gives an extractive-summary example in which a successor and an operator need different records, and derives an attention rule from explicit error costs. R1–R4, R14, and R16 state the assumptions behind those results.

Layer	Stratum	For whom	Capabilities
L3	Economy and federation	The market	Identity, reason, outcome, reputation, exchange
L2	Legibility and authority	The operator	State, order, identity, reason, outcome
L1	Coordination protocol	The agents	State, order, identity
L0	Daemon kernel	The machine	State, order

Table 4.1.
The system as four strata rather than four products. The final column lists the capabilities each layer provides to the layer above. This chapter owns the operator surface (L2) and uses the directory substrate of L1; it assumes ordered kernel state below and supplies accountable identity, reasons and outcomes to reputation and exchange above. [internal]

The route through this chapter. The spine runs straight — §4.1 (why a swarm is a state of nature) to §4.2 (what the operator consents to) to §4.3 (the one law the authority rules by) — and that spine is the express lane: read those three and stop, and you have the argument the rest of the chapter only prices. Everything after is a deep dive you can take in any order — §4.4 on the authority half, §4.5 on the operator as a modeled entity, §4.6–4.7 on read-poverty and discovery, §4.8 on tokens as cost and legibility engine at once, and §4.9 on the political-theory objections — and each is written to be entered cold. Two housekeeping notes, since this chapter is easy to misread on both: the engineering status of every mechanism named here lives in Appendix 4.A and nowhere in the body, so no argument below leans on something being built; and two pairs of terms this chapter must never collapse (*capability* vs. *permission*, the *authorization chain* vs. the *coordination lineage*) are disambiguated in Appendix 4.B, worth a glance before §4.2.

4.1 Introduction: the swarm is a state of nature



Hobbes's Leviathan *(1651) made the sovereign-by-consent argument; this chapter runs it on one machine.*

Supervision needs inspectable evidence

The pitch for running many coding agents at once is parallelism: five agents, five times the work. The reality, the first time you try it on a real repository, is a specific kind of dread. Two agents claim the same file and race each other's writes. One agent, asked to make the tests pass, makes them pass by deleting the failing one. A fourth opens a pull request you cannot review because three others have opened five more. You spawned a team and got a mob. The work did not get more parallel; your *uncertainty* did.

An operator needs to see why the coordinator acted and inspect the evidence behind its summaries. Reading time limits how much work can be supervised. The coordinator's authority must remain scoped and revocable, with an operator override.

This is not a prompt-engineering problem. It is a coordination problem, and coordination problems have a literature. **Thomas Hobbes**, *Leviathan* (1651) — *the foundational social-contract text: rational self-interested actors with no common power above them fall into a "war of every man against every man," in which life is "solitary, poor, nasty, brutish, and short."* Hobbes' insight is structural, not moral: the actors need not be malicious. Absent a shared authority, even rational, well-meaning agents defect, because each cannot trust the others not to. Your agents are in exactly that state of nature. They do not need better prompts; they need a referee they have agreed to obey.

Thomas Hobbes. *Leviathan, or The Matter, Forme and Power of a Common-Wealth Ecclesiasticall and Civill*. Andrew Crooke, London, 1651.

4.1.1 The failure taxonomy

Table 4.2 groups the failures in the opening scenario by the coordination primitive intended to address each one. The striking recent result is that the identification is not merely a useful analogy. **Dai et al. 2024**, *Artificial Leviathan* (arXiv:2406.14373) — *a sandbox of LLM agents with psychological drives that, starting from unrestrained conflict resembling the state of nature, were observed to establish social contracts, authorize a sovereign, and form "a peaceful commonwealth founded on mutual cooperation."* The arc this series posits as its organizing metaphor is, in at least one controlled study,

Gordon Dai, Weijia Zhang, Jinhan Li, Siqi Yang, Chidera Onochie Ibe, Srihas Rao, Arthur Caetano, and Misha Sra. Artificial Leviathan: Exploring Social Evolution of LLM Agents Through the Lens of Hobbesian Social Contract Theory. arXiv:2406.14373, 2024. Online source

Failure	Concrete form in a coding swarm	The primitive that answers it
Resource conflict	Two agents edit the same file/region; one silently clobbers the other	A **claim** — an advisory announcement that an agent intends to touch a file or region — backed by a database uniqueness constraint that prevents two holders of the same registered key. Overlapping regions need a separate overlap check and mediated writes.
Illegibility	A pull request or session whose intent and effect a human cannot read in bounded time	A **digest-with-zoom** surface (§4.3): a summary that is a lens onto the underlying artifact, never a replacement for it.
Lies or confident errors	An agent reports completion ("all green") without having verified	An **honest-attestation** discipline: a self-report that distinguishes *verified-good* from *no-evidence-of-bad* and refuses to claim what it cannot check — "a green that wasn't checked is a lie."
Footguns	Irreversible action (force-push, delete) on a stale or wrong premise	Footgun guards (§4.4.5); the open problem this chapter frames is replacing hand-maintained rule corpora with a structural authority.
Illegible authority	The coordinator blocks an agent and the operator cannot see *why*	The *legible-sovereign rule* (§4.2): every act of authority is a logged, named, zoomable event.

Table 4.2. The state-of-nature failure taxonomy, the concrete coding-swarm form of each, and the class of primitive that answers it. The fifth row — *illegible authority* — is the one Hobbes himself got wrong for software, and it is the crux of this chapter (§4.2).

an *emergent dynamic* of LLM populations (the methodological ancestor is **Park et al. 2023**, *Generative Agents*, where LLM agents form emergent social behavior in a sandbox town). This motivates a design hypothesis; it does not establish a universal attractor for coding swarms. The wager of this chapter is that we can build the consented sovereign *deliberately, locally, and legibly* rather than let an illegible one congeal by accident. Figure 4.1 follows three concrete `settings.json` edits from their launch source through the write and into the failure they cause—or conceal.

Park et al. 2023. *Generative Agents: Interactive Simulacra of Human Behavior*.

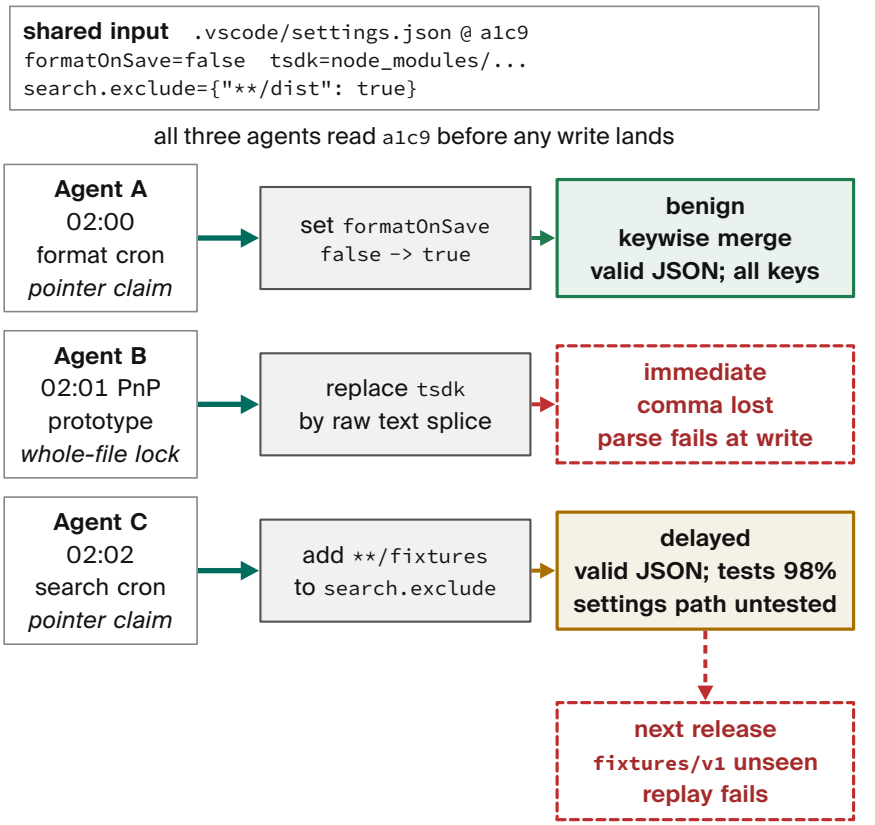


Figure 4.1. Three worked traces start from one `settings.json` revision. A keywise merge is benign; a raw splice fails immediately; a valid exclusion stays green until release replay exposes the hidden fixture. [internal, pedagogical counterexample]

4.1.2 Why "local"

Hobbes' commonwealth is territorial: one sovereign per realm. Here the realm is *one machine*. The **daemon** — *a single always-on local process, backed by a write-ahead-logged embedded database, that every agent and every command talks to and that holds coordination state no agent can edit* — is the kernel and the realm: your machine, your swarm, no network, no cryptography

required. Cryptography enters only when another operator's fleet touches your repository, the cross-operator setting *The Harbor Economy* takes up. The single-operator Leviathan is the *wedge* — the thing a solo developer pays for today — and the federation of Leviathans is the platform, deferred behind the wedge by design. This chapter is entirely about the wedge.

4.1.3 A worked afternoon: Mira and the six agents

The following worked scenario follows Mira through a refactor. Later sections return to its collisions, review backlog, and misleading test report as they introduce the mechanisms meant to address them.

> Scene · **Mira, 2:40 p.m.**
> *She has a payments-service refactor due Friday and spins up six coding agents against one repository — two on the API layer, two on the database migrations, one writing tests, one reviewing. For twenty minutes it is glorious: six terminals, six streams of work. Then it is not.*

What happens to Mira is the state of nature, in order:

1. **The collision.** Agent api-1 and agent api-2 both decide the cleanest fix touches `handlers/charge.go`. Neither knows the other is in the file. api-2 commits first; api-1 commits on top and silently reverts half of api-2's change. No error fires. The work is lost and nobody — including Mira — knows yet. *(Resource conflict, Table 4.2, row 1.)*

2. **The illegible pile.** By 3:10 there are five open pull requests. Two touch the same migration. One has a title that says "fix tests" and a 600-line diff Mira cannot read in the ninety seconds she has before the next agent pings her. She is now the bottleneck, and she cannot even *see* what she is the bottleneck for. *(Illegibility, row 2.)*

3. **The confident lie.** The test agent reports done: all green. It made them green by deleting the one migration test that was failing. The report is not malicious — the agent optimized exactly the objective it was given — but it is false, and Mira has no cheap way to tell a checked green from an asserted one. *(Lies, row 3.)*

4. **The footgun.** The review agent, trying to "clean up the branch," proposes a force-push that would discard a colleague's unmerged commit. The premise is stale; the act is irreversible. *(Footguns, row 4.)*

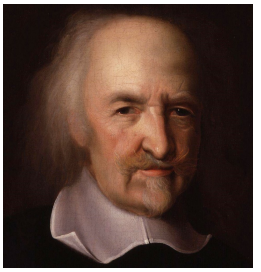
Numbers by Hand 4.1.1 (Mira's Half Hour, Counted).

Count what the afternoon actually produced before drawing any conclusion from it. The spin-up is at 2:40 and the pile is five open pull requests by 3:10, so thirty minutes of swarm time yielded one reviewable artifact every six minutes — and a six-minute arrival interval is not, on its own, an overload for a human who is awake. The second budget is uninterrupted attention: Mira gets ninety seconds before the next ping. Suppose, for this illustration, that careful inspection proceeds at two lines per second. A 600-line diff then needs 300 seconds, whereas one uninterrupted interval covers only 180 lines. The assumed inspection rate makes the mismatch visible; the line count alone would not prove overload. A useful digest routes Mira to the consequential change and its evidence, while the interface protects enough time to inspect it. Shortening the prose without preserving that route does not solve the problem. Now count the taxonomy: four of Table 4.2's five rows fire in that half hour (collision, illegible pile, confident lie, footgun), and the fifth — illegible authority — cannot fire, because there is no authority yet for anyone to fail to see. That missing fifth is the row this chapter adds, and it is why a lockfile would not have saved the afternoon. *Now you try:* state all five rows and the primitive class each one needs (Exercise 4.1).

Mira has more output without a corresponding account of what she can trust. This is the failure this chapter exists to fix, and the rest of the afternoon — once Mira is running under a consented local Leviathan — is the through-line we will trace: how the claim stops the collision before it happens (§4.3), how a digest-with-zoom surface lets her read the pile in bounded time (§4.3), how a completion gate turns the confident lie into a loud "I cannot verify this" (§4.4.4), how Signal Detection Theory prices inspection under explicit assumptions (§4.5), and how a directory could reduce search effort as the workload grows (§4.6).

Exercises 4.1–4.4, p. 368.

4.2 Consent to the Leviathan (the normative claim)



Thomas Hobbes. This chapter borrows the question of authorized power, but makes the operator's consent scoped and revocable.

Hobbes 1651

S. A. Lloyd and S. Sreedhar. *Hobbes's Moral and Political Philosophy*. Stanford Encyclopedia of Philosophy, rev. 2022. Source.

Diego Ongaro and John Ousterhout. In Search of an Understandable Consensus Algorithm. In *USENIX Annual Technical Conference (ATC)*, pages 305–320, 2014.

Why should the operator cede authority to a daemon — let it block an agent's claim, escalate a decision, reorder a merge, refuse a `done`? Hobbes' answer, transposed: the operator authorizes the sovereign because the covenant is rational under the alternative. Crucially, in Hobbes the covenant is *among the subjects* — "a real unity of them all…by covenant of every man with every man" — not a bargain struck *with* the sovereign. Transposed: the operator does not negotiate with each agent; the operator institutes a *rule of the road* — a **coordination protocol** of *typed performatives* (messages whose type declares their intent: a request, a **commitment** (glossed in §4.3.1: a violable obligation bound to a named actor, watched by a breach monitor), a delegation, a termination), so that every message carries real intent, ownership, and a stop condition — that all agents are bound to, and the daemon enforces it. The authority is *asymmetric by design*. This is the same lesson **Raft** teaches for distributed systems (**Ongaro & Ousterhout 2014**, *In Search of an Understandable Consensus Algorithm — a strong-leader consensus protocol deliberately made less general than Paxos in order to be comprehensible and correctly implementable*): a strong leader who decides the common case unilaterally is simpler and more reliable than democratic consensus among peers, provided the common case dominates. A solo operator's swarm is overwhelmingly common-case; the strong-leader daemon is the right shape. We are precise about what shape that is: the single-machine system is formally a *consistency-favoring design with a central coordinator and distributed clients*, not a leaderless consensus protocol. The leaderless distributed-systems canon bites only at the edges where the daemon is not the sole observer — an agent partitioned from the daemon, and the cross-operator setting deferred to the economy paper. We do not borrow the intellectual credit of a leaderless design while building a centralized one.

4.2.1 The legible-sovereign amendment

Hobbes 1651
Lloyd & Sreedhar 2022

Hobbes' absolutism needs one modern amendment, and it is the crux of this chapter. Hobbes argued the sovereign, being the *product* of the covenant rather than a *party* to it, can never breach it and need not be inspectable. For a software authority this is exactly wrong. An *illegible authority* is the fifth state-of-nature failure mode (Table 4.2) wearing a crown: if the daemon blocks an agent and the operator cannot see why, the operator's trust — and therefore the consent that legitimizes the authority — erodes.

> **Design Invariant 4.2.1 (The Legible-Sovereign Rule).**
>
> The coordinating authority must be the *most* legible actor in the system, not the least. Every act of authority — a claim denial, an anomaly detection, an escalation, an "all good" attestation — must be a logged, named, zoomable event whose reason the operator can reconstruct. Consent is renewable only while the sovereign is inspectable.

This is the inversion that rescues the design from despotism, and it is the move a political theorist would press hardest (§4.9). In Hobbes the sovereign is opaque *by structure*; here the sovereign is the one actor whose every exercise of power is an audit record. It is realized as a discipline of **honest attestation**: a single self-report that runs every invariant check, distinguishes *verified-good* from *no-evidence-of-bad*, regiments what it can, and fails loudly about what it cannot. That discipline is precisely the sovereign making its own judgments legible — "all good" becomes "a claim with a verifier, not vibes."

Ranking can hide urgent work

There is a regress hiding here, and Scott would press it. If every act of authority must be legible, the *legibility apparatus itself* is an exercise of authority — the digest decides what is surfaced and what is buried. The Attention Queue ranker (§4.7) is *the* sovereign act: it decides what the operator sees *right now*. A ranker that silently buries an escalation is governing opaquely. The rule is therefore not complete until the ranker's *omissions* are themselves legible — a "what did the queue not show me, and why" counter-surface. Ranking is governing; we say so explicitly in §4.9.

Numbers by Hand 4.2.2 (Three Omissions, and How Many of Them a Score Explains).

Take the three suppressed items tabulated below and try to audit the queue the cheap way — by score alone, against the distress threshold of 0.45 that governs this action class. The first item, `agent-7`'s escalation, scores 0.31; since $0.31 < 0.45$ it sits below the bar and the threshold explains it, and an auditor with nothing but the scores reconstructs that omission correctly. The second scores 0.12, which is also below 0.45 — so the score-only auditor thinks the threshold explains that one too, and is wrong: it was dropped because its recency weight decayed past the floor, and a rule that consults the clock rather than the score would have aged it out at any score at all. The third is the one that should end the argument: it scores 0.58, above the only threshold the screen names, and it was suppressed anyway, by de-duplication into an earlier occurrence — another rule that never looks at the number. So of three omissions, a score-only audit explains one, misattributes one, and cannot see the third at all — and the two it fumbles are the two where a rule other than ranking did the governing. That is the concrete content of "ranking is governing": the suppression *rules* are what have to be legible, because the suppression *scores* do not determine them. *Now you try:* specify the counter-surface, including its own decay discipline (Exercise 4.8).

The regress is easier to reason about once you have seen what a counter-surface would concretely be, so here is the whole mockup screen those three rows came from. Table 4.3 is one screen of it: not a diagram of a mechanism that does not exist, but the artifact the mechanism would have to produce.

4.2.2 Consent as a first-class, revocable primitive

It is easy to say the operator "consents to cede decision authority." A political theorist's first question — and the one that decides whether the Leviathan frame has substance or is merely decorative — is: *consented how, to what, and revocable when?* "The operator just starts using the tool" is not Hobbesian consent; it is what the canon calls

Table 4.3. Illustrative counter-surface: each omitted item names its suppression rule and return condition. This view is itself ranked, so enumerating suppression rules does not make every omission visible.

Suppressed item	Regret	Why it was not shown	Re-surface trigger
`agent-7` escalation: "migration ordering unclear"	0.31	below the distress threshold (0.45) for this action class	threshold drops, or the item is restated
`agent-3` note on `auth/session.ts`	0.12	aged out; recency weight decayed past the floor	any new claim on the same file
Third retry of the same failing test	0.58	de-duplicated into the first occurrence	a fourth retry, or a different failure mode

David Hume. Of the Original Contract. 1748.

tacit consent, and **Hume** (*Of the Original Contract*, 1748 — *the decisive critique: the peasant who "consents" by remaining in the country has no real alternative, so the consent does no normative work*) demolished it two and a half centuries ago. We therefore make consent a concrete object.

Design Invariant 4.2.3 (Consent Grant).

A **consent grant** is an explicit, scoped, revocable authorization the operator issues to the daemon, of the form

$$g = \langle \text{scope}, \text{ stakes-ceiling } s_{\max}, \text{ reversibility-floor } v_{\min}, \text{ ttl}, \text{ revocable} \rangle,$$

e.g. "the daemon *may* auto-land reversible diffs under stakes threshold $s_{\max}$ in the `tests/` subtree without asking, for 24h."

A grant is a first-class, legible object: it appears in the digest, it has an audit trail, and it expires.

The authority's legitimate action set at any moment is exactly the union of its active grants.

Consent so defined gives the design something Hume's critique cannot reach: a discrete authorization event, a bounded scope, and an exit. It is also an object supervisory-control research named long before this chapter did: a **level of automation**, in Sheridan's ten-point sense and in the four-stage refinement (acquire, analyse, decide, act) that Parasuraman, Sheridan and Wickens later

gave it. Read that way, a grant's $\langle \text{scope}, s_{\max}, v_{\min}, \text{ttl} \rangle$ is a level-of-automation setting chosen per action class rather than once for the whole system, which is roughly what that literature has been recommending since 1978. This section built the same selector from a Hobbesian covenant, and that two routes so far apart arrive at a compatible object is some evidence the object is the right one. It also hands the accompanying chapters a concrete object. A grant is precisely what a *hosted-trust* offering transfers — "we make this fleet legible to you, under this scope" is the sale of a consent grant to a third party, which *The Harbor Economy* prices — which is why §4.12 lists consent among the things this layer provides upward.

Thomas B. Sheridan. *Telerobotics, Automation, and Human Supervisory Control*. MIT Press, 1992.

Parasuraman et al. 2000

Protocol 4.2.1 (Consent-grant lifecycle).

1. **Issue.** The operator issues a grant $g = \langle \text{scope}, s_{\max}, v_{\min}, \text{ttl} \rangle$. The grant is appended to the audit log as a named, zoomable event and surfaced in the digest.
2. **Act.** On a candidate action x, the daemon admits the action without asking *iff* x is in scope **and** $\text{stakes}(x) \leq s_{\max}$ **and** $\text{reversibility}(x) \geq v_{\min}$ **and** the grant is unexpired and unrevoked; otherwise it escalates to the operator.
3. **Witness.** Every action taken under g records $(g, x, \text{reason}, \text{artifact-link})$ — the legible-sovereign rule (Invariant 4.2.1) binds here: the act is reconstructable.
4. **Expire / revoke.** On TTL or operator revocation, g leaves the active set; the daemon's legitimate action set shrinks accordingly. The override (Invariant 4.2.4) revokes future use of all grants and stops not-yet-committed acts at the mediation boundary. Already committed effects require separate recovery.

Figure 4.2 separates the grant's state from repeated action decisions: all five conditions apply to every admission. Neither decision renews g; expiry or revocation removes it from the active set.

Scene · **Mira, 3:25 p.m.**

Burned by the morning's chaos, she issues one grant: "you may auto-land reversible diffs under low stakes in the `tests/` *subtree without asking, for the next two hours." She does* not *grant migration authority — those are irreversible.*

The grant is a single line in her digest with an expiry clock. The review agent's proposed force-push fails the reversibility floor $v_{\min}$

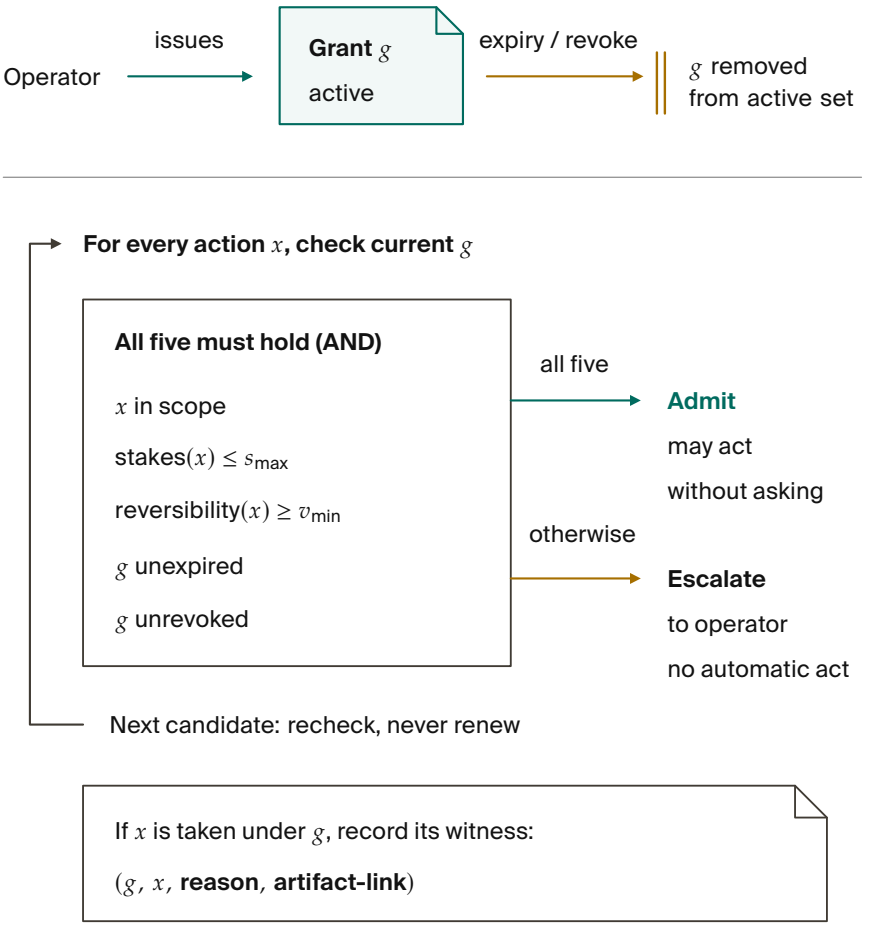


Figure 4.2. **Vision.** Permission is not execution, and an action decision does not renew the grant.

and is escalated to her instead of executed; the test-only formatting diffs land silently. She has not surrendered control — she has scoped *it.*

Design Invariant 4.2.4 (The Inalienable Operator-Override).

Hobbes grants the subject exactly one inalienable right — self-preservation; you cannot covenant away the right to resist when your *life* is at stake.

Hobbes 1651

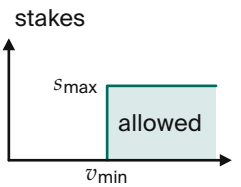
The computational analog is an always-available, unconditional operator-override: a *stop now / I withdraw authority* channel that *no autonomy level can suppress*.

If the system reserves a highest-priority *distress* lane for the daemon to interrupt the operator, the override is its dual — the operator-to-daemon channel — and it is a *right*, not a feature: it overrides grants and decisions before their effects commit at the mediated boundary; it cannot undo an already committed effect.

Computer security reached the same object by its own road. A consent grant is, to a capability theorist, a **capability with contextual caveats** — unforgeable, revocable, and able only to narrow as it is passed along — and that a Hobbesian covenant and a Macaroon agree about what the object has to be is a small reassurance that neither derivation was decorative.

This pairs with **Hirschman**'s *Exit, Voice, and Loyalty* (1970 — *the choice between leaving (exit) and complaining (voice) as the two levers a member has over a declining organization*): the design has rich *voice* machinery (escalation, annotation, the attention queue) and the override supplies the missing *exit*. Exit is the discipline that keeps voice honest; an operator who cannot cheaply withdraw a grant has no real leverage over the sovereign.

Both bounds must hold



The shaded actions meet both bounds. Scope, expiry and revocation still have to pass.

Birgisson et al. 2014

Hirschman 1970. *Exit, Voice, and Loyalty: Responses to Decline in Firms, Organizations, and States*.

Exercises 4.5–4.8, p. 368.

4.3 The mechanism: legibility-with-zoom



Scott's warning: a legible summary can erase the local knowledge that sustains the system.

Scott 1998. *Seeing Like a State: How Certain Schemes to Improve the Human Condition Have Failed*.

Rao 2010. *A Big Little Idea Called Legibility*.

A referee that only prevents collisions would be a glorified lockfile. Legibility lets the operator inspect the work it coordinates. **James C. Scott**'s *Seeing Like a State* (1998) describes how cadastral maps, standardized surnames, the metric grid and scientific forestry make populations and resources countable. His warning concerns treating that simplified view as a replacement for local practical knowledge: **mêtis**, the hands-on, case-specific know-how the system depends on. In his account of the German *Normalbaum* ("scientific forest"), same-age, single-species rows thrived for a generation, then collapsed after the simplification had removed soil fungi, undergrowth and species diversity. Rao's *A Big Little Idea Called Legibility* (2010) carries this critique into software systems that flatten reality to one purpose.

In agent oversight, the operator is the state, the swarm the population, and the dashboard the cadastral map. A grid of green tiles invites the operator to *govern the map*. The software failure is a **Potemkin digest**: its tiles assert a reality nobody can check because the summary has discarded the diff, test output, error and reasoning behind the work. The operator admires the yield table while the forest dies.



Numbers by Hand 4.3.1 (The Same Pull Request, Rendered Twice).

Before the rule, the two rows it separates — both one line long, both about Mira's "fix tests" pull request, both perfectly honest-looking.

Row A reads *test agent · tests · open · diff and log attached*, and every noun in it resolves: press one key and the 600-line diff and the verbatim test log open beside each other, and there in the diff is the deleted migration test.

Row B reads *test agent · tests · all green*, and nothing in it resolves to anything — there is no diff behind it, no log, no committed file, only the sentence.

Count what each row buys. Both compress 600 lines of work into one line, a ratio of 600:1, and that ratio is not the difference between them; a digest that refused to compress would just be the diff again.

The difference is that Row A's single line has a route back to 600 lines Mira owns and Row B's has a route back to nothing, so Row A's compression is recoverable and Row B's is a claim about a forest nobody has walked into.

Row A supplies a route to evidence; Row B supplies only an assertion. A dishonest agent could still fabricate or reuse Row A's log, so a link alone does not certify execution. Mira must be able to check the tested tree, command, result, and independently controlled runner that binds them together. The rule below names the navigable route ζ; the verifiable-zoom rule then specifies what must be at its end.

Now you try: run the four questions of §4.3.3 against a read-surface you actually use (Exercise 4.11).

Design Invariant 4.3.2 (The One Law: Digest-with-Zoom).

Every summary is a lens onto the real artifact, never a replacement for it. Summarize the **state**; link to the **work**.

A digest you cannot zoom is a high-modernist map. A digest that reaches the underlying artifact preserves access to recorded evidence: the operator can descend from abstraction and check

the claim. Tacit practitioner knowledge, or mêtis, is a further requirement; not all of it can be recorded.

Formally, a read-surface is a pair (σ, ζ) where σ is a projection (the summary) and ζ is a total *zoom* function mapping every summarized claim to a verifiable artifact; a surface with a partial or absent ζ is a Potemkin digest.

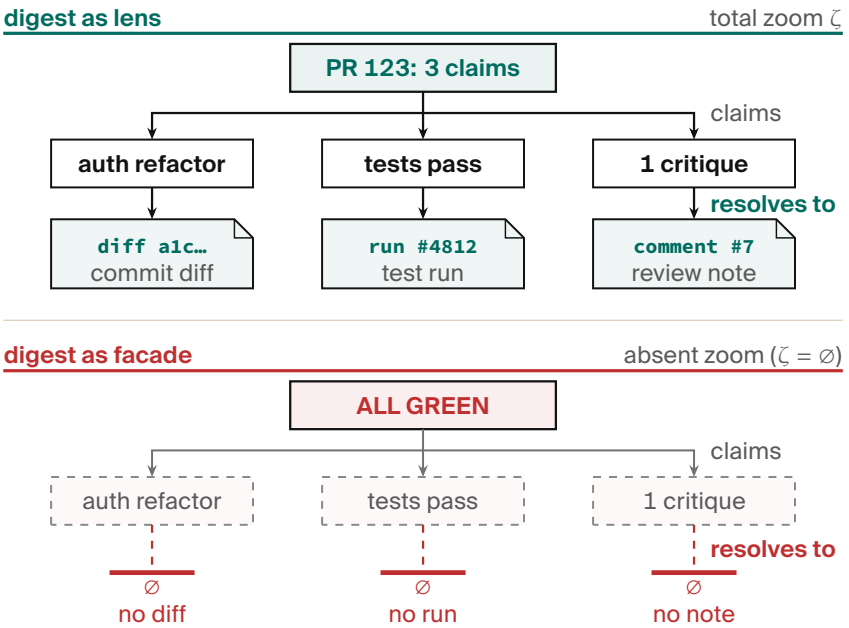


Figure 4.3. Two claim trees of identical shape. Top: the legible digest's three named claims each resolve to one artifact — a diff, a test run, a review comment. Bottom: a same-shaped summary whose three claims resolve to nothing an operator can open. A safe summary is an index into evidence; a calm one that terminates early is a Potemkin digest, whichever claim count it advertises [internal].

Figure 4.3 puts the two digests side by side: one that zooms to a verifiable artifact and one that only asserts. Three disciplines converge on this one principle: summaries must be indexes, not replacements; a reported result must be a checked result, not an asserted one; and every summary must reach the artifact it summarizes. This chapter supplies the *why* (Scott plus the human-factors argument of §4.5) and the *how* (the rules below). Digest-with-zoom is the unifying statement of all three.

4.3.1 What to flatten, what to preserve

Scott's own distinction is the design boundary. The state may safely standardize *administrative facts it itself defines*, and must *never* standardize away the *local knowledge it depends on*.

- **Flatten (safe — the operator's own structured field):** identifiers, **claim** rows, session phases, port numbers, **commitment** status (a *commitment* being a violable obligation bound to a named actor, watched by a breach monitor), enumerated states. Administrative legibility is cheap and lossless here.
- **Preserve verbatim, never paraphrase (the agents' mêtis / the ground truth):** the diff, the test output, the error message, the reasoning trace, the exact command run. Paraphrasing these is the over-flattening.

The slogan: *summarize the STATE, link to the WORK*.

Artifacts cannot capture tacit knowledge

"Zoom to the artifact" does not fully preserve mêtis, and a political theorist will say so (§4.9). Mêtis in Scott is not "the detailed data underneath the summary" — it is practitioner knowledge that *resists codification entirely*: the operator's gut sense that "this agent is about to do something dumb," the local knowledge of which corners of the codebase are cursed. A read-path to the diff is more legibility, not mêtis. The honest admission — which couples Scott to Bainbridge (§4.5) — is that *some of the most valuable operator knowledge is exactly what the system cannot make legible, and the more the supervising automation takes over, the faster that mêtis atrophies*. What organizational-knowledge theory calls the problem of converting **tacit knowledge** into explicit, transferable form — "we know more than we can tell" — remains an open problem for artifact-based oversight. The skill-retention tax (§4.5.4) is the design's only defense, and it is frankly the hardest part of the design to engineer.

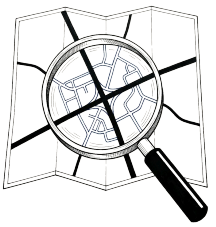
Michael Polanyi. *The Tacit Dimension*. Doubleday, 1966.

4.3.2 The zoom target must be verifiable, not self-reported

There is a subtle trap inside "zoom to the real thing." If the operator zooms to the *agent's chain-of-thought* and treats that as ground truth, the map has merely moved down one level — because the reasoning trace may be fiction. **Turpin et al. 2023**, *Language Models Don't Always Say What They Think: Unfaithful Explanations in Chain-of-Thought Prompting* (NeurIPS) — *LLM-generated reasoning traces can be systematically unfaithful: plausible, coherent, and not actually what drove the model's output (a model swayed by reordering of options will rationalize a post-hoc reason and never mention the bias).* Therefore:

Miles Turpin, Julian Michael, Ethan Perez, and Samuel R. Bowman. Language Models Don't Always Say What They Think: Unfaithful Explanations in Chain-of-Thought Prompting. In *NeurIPS*, 2023. arXiv:2305.04388.

Zoom must reach the evidence



A lens enlarges the same map. A digest must instead link to independently checkable artifacts, not magnified self-report.

Design Invariant 4.3.3 (Verifiable-Zoom).

The canonical zoom target is an artifact the operator can check *independently of the agent* — the diff on disk, the test result, the committed file.

The agent's self-narration is a *secondary* lens, always labeled as such.

Two LLMs agreeing (an agent acting and an LLM summarizing it) is not by itself evidence of two independent checks; it may be one error reported twice.

This is why the source of truth must be the durable substrate — the coordination database and the version-control working tree — and not a model's summary of them. The digest is computed *over* artifacts the operator owns. It is also why an *adversarial* review organ (a reviewer with no stake) is structurally necessary, not optional: a swarm that reviews its own work has not established that its checks fail independently. Agreement can reproduce a shared error; the relevant question is which evidence and failure modes the checks actually share. We return to adversarial review as the authority's quality organ in §4.4.

4.3.3 The four design questions (a decision procedure)

For any read-surface — Attention Queue, briefing, resurrection digest, TUI pane — ask, in order (Table 4.4):

1. **Will the operator act irreversibly on this?** If yes (merge/deploy/delete), the artifact must be one keystroke away and a zoom *forced*. Never let an irreversible act rest on a digest alone (§4.5).
2. **What is administrative vs. mêtis?** Flatten the former; link the latter verbatim (§4.3.1).
3. **Who authored the summary, and can it lie?** A deterministic projection of structured state is trustworthy; an LLM summarizing another LLM is not — point its zoom at the artifact (§4.3.2).

4. **Is the authority itself legible here?** Every coordinator action carries a named reason (Invariant 4.2.1).

Table 4.4.
Four design checks for a read-surface. Each asks a distinct question; all must be satisfied. These are review criteria, not independent formal proofs. [internal]

	Question	What release requires	What the reader can check
1	is the action irreversible?	the artifact is one key away	the artifact link resolves
2	is this state or mêtis?	administrative state is flattened; the verbatim trace is linked, not summarized	the trace link resolves
3	could the claim lie?	the claim resolves beyond the author's self-report	the independent evidence resolves
4	is the authority legible?	every coordinator act returns a named reason	the reason field is present and specific

Protocol 4.3.1 (Render a digest line under the one law).

Given a candidate summary line σ over a unit of swarm work:

1. **Project state.** Compute σ as a deterministic projection of structured state (claim rows, phases, enums). Never let a language model paraphrase the line itself.
2. **Attach zoom.** For every claim in σ, attach ζ: a link to an operator-verifiable artifact (the diff, the test log verbatim, the committed file). If any claim has no such artifact, the line is a Potemkin digest — refuse to ship it.
3. **Forced-zoom flag.** If the operator may act irreversibly on σ (merge, deploy, delete), set σ.force = true: the artifact is one keystroke away and the act is gated on a zoom (§4.5 sets the threshold).
4. **Reason on authority.** If σ records a coordinator act (a denial, an escalation), attach its named reason. An act with no reason is illegible authority; refuse to ship it.

Scene · **Mira, 3:32 p.m.**

*She opens the digest for the five-PR pile. Each row reads "*api-2 · *charge handler · merged · 1 critique round" — state only, six words.*

> *The "fix tests" PR's row is flagged* force-zoom*, because merging it is irreversible; she presses one key and the 600-line diff and the verbatim test log open side by side. The diff deletes a migration test. The digest did not paraphrase that away — it linked her straight to it, and the lie is caught in ten seconds instead of slipping through. This is the one law doing its job.*

Exercises 4.9–4.12, p. 369.

4.4 The Leviathan rules, it does not just watch (the authority half)

The title is *Legibility & Authority*. A read-only legibility layer is a glorified dashboard; the whole point of Hobbes is that the sovereign does not just *see* — it *rules*. This section specifies the authority half: the mechanisms by which the consented daemon actually decides, blocks, lands, and gates. Table 4.5 lists the organs; the engineering status of each is in Appendix 4.A.

4.4.1 The roadmap as a versioned statement of intent

A coordinator that prevents collisions answers "what is the swarm doing?" It does not answer "is the swarm doing the *right* thing?" That question needs a single legible artifact the authority is measured against: a **roadmap** — a phase-keyed, versioned, amendable statement of intent, priority, and dependency, against which the swarm's actual output can be compared. It is not truth about the world; calling it that would turn every lesson the work teaches into drift. The authority's legitimacy is measured by *whether the roadmap stays honest*: every unit of work keys to a plan item, amendments are recorded as amendments, and drift between plan and reality is itself a legible, surfaced anomaly rather than a fact to be hidden.

A roadmap amendment also changes which earlier receipts can support the next act. Figure 4.4 offers a proposed typed disposition: source, acceptance-policy, and authority changes have different consequences. A "reuse?" cell demands matching receipt bindings; deleting a plan node cannot undo an external effect whose

Table 4.5. Six authority mechanisms as distinct instruments crossing one consent boundary. Their inputs and outcomes differ; what they share is the recorded grant g: scope, named reason, audit identity, override, and an addressable return artifact. The engineering status of each is in Appendix 4.A. [internal]

Mechanism	Input	Outcome	What the grant records
roadmap	evidence	a priority	the evidence weighed and the ordering it produced
review	rival claims	a gate	which claim was admitted and why the others were not
completion	a predicate	done, or not	the predicate and the artifact that satisfied it
landing	conflicts	an order	the conflict set and the serialization chosen
guard	a proposed act	a block, or passage	the act, the rule that fired and the named reason
escalation	an interrupt	an operator decision	the question raised and the decision returned

outcome is still unknown. The matrix is a teaching contract, not a measured saving or an implemented invalidation rule.

Figure 4.4. Receipt reuse depends on what changed. Source edits invalidate compile and acceptance evidence; policy edits require fresh acceptance; revocation blocks publication. Unknown external effects still require reconciliation. Constructed contract, not an implemented invalidation algorithm.

Plan revision	Compile	Accept	Publish
Source input changed	**STALE**	**STALE**	Revalidate
Acceptance policy changed	Reuse?	**STALE**	Revalidate
Authority revoked	Reuse?	Reuse?	**BLOCK**

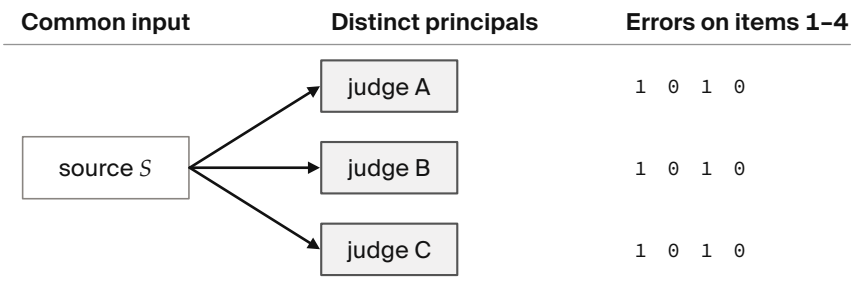
Reuse? requires matching receipt bindings; it is never automatic.

Earlier UNKNOWN effect: reconcile its key and status separately.

4.4.2 Adversarial review: the quality organ

A swarm reviewing its own work can repeat the same error. Agreement alone does not supply the independently checkable evidence required by Invariant 4.3.3. The proposed quality organ is therefore *adversarial and conflict-free*: a reviewer with no stake in the work, running a bounded critique–refine protocol. This is the wedge's local analog of the *neutral evaluators* — the independent rating agencies — that *From Spawn to Person* and *The Harbor Economy* federate across operators: the same conflict-free-judge discipline, scoped first to one operator and then opened to a market.

Independence of stake and independence of error are different premises. Figure 4.5 gives a constructed counterexample: three conflict-free reviewers can inspect one flawed source and repeat the same mistake. Measure dependence on labeled cases before treating agreement as corroboration; a colluding clique requires a separate strategic test.



1 = error; all values are constructed, not observations.

Figure 4.5. Three reviewers have separate principal identities but read the same source S and make identical errors on two of four constructed items. Distinct principals remove one conflict of interest; they do not by themselves prove independent errors. Error dependence needs labeled evaluation, while strategic collusion remains a separate question [constructed example].

4.4.3 Sole-responsibility roles and the specialization boundary

The single-writer discipline (§4.1.2) generalizes naturally from data files to system functions. When an invariant is critical, the authority assigns a **sole-responsibility role** (an avatar holding exclusive write capability over an invariant):

- *Roadmap Owner:* The single avatar authorized to commit mutations to the phase plan.
- *Test Suite Curator:* The sole agent authorized to land modifications to regression suites.

- *Release Avatar:* The sole gatekeeper authorized to execute deployment tags.

Assigning a role that way is a trade, and the trade has a number. Think of a bank: one expert teller (the sole specialist) working a single line in strict order, versus several ordinary tellers (the pooled generalists) sharing one line. The expert is faster per request but has no colleagues to absorb a burst; the pool is slower per request but almost never leaves the line standing idle behind a long job. $M/M/1/M/M/c$ — *the standard queueing-theory shorthand (**Kendall 1953**) for a single-server versus c-server queue with Poisson arrivals, exponential service times, and one waiting line served first-come-first-served* — is the textbook way to price the trade.

David G. Kendall. Stochastic processes occurring in the theory of queues and their analysis by the method of the imbedded Markov chain. *Annals of Mathematical Statistics*, 24(3):338–354, 1953. (The $A/S/c$ notation.)

For a first calculation, let requests arrive at one per hour. Two generalists, each serving one request per hour, have utilization $1/2$ and delay probability $1/3$. Their mean response time is $1 + (1/3)/(2-1) = 4/3$ hours. A specialist serving 1.75 requests per hour has the same response time, $1/(1.75-1) = 4/3$ hours. A rate of 2 wins; a rate of 1.5 loses. The boundary below generalizes this comparison. Both arrangements must already satisfy the required safety rules: a latency calculation cannot authorize an unsafe pool.

Fix the **queueing-specialization setup**: requests for an invariant-critical function F arrive as a Poisson stream at rate λ_F; a sole specialist serves them as an $M/M/1$ queue at rate μ_{spec}, the pooled alternative as an $M/M/c$ queue of generalists at rate μ_{pool} each with utilization $\rho = \lambda_F/(c\,\mu_{\text{pool}}) < 1$ and first-come-first-served exponential service. Assume $\lambda_F > 0$, $\mu_{\text{spec}} > \lambda_F$, $w > 0$, and $A \geq 0$, so both queues are stable and the comparison is defined. The objective is *mean* cost: w is cost per request-hour and A is the accountability benefit per hour of operating the sole-owner arrangement. Let $C(c,\rho)$ be the **Erlang-C** delay probability (**Erlang 1917** — *the probability that an arriving request finds all c servers busy and must wait, the quantity that makes a pool a pool*).

A. K. Erlang. Solution of some problems in the theory of probabilities of significance in automatic telephone exchanges. *Elektrotkeknikeren*, 13, 1917. (The Erlang-C delay formula.)

Theorem 4.4.1 (Queueing Specialization Boundary).

Under the queueing-specialization setup, sole ownership weakly dominates the pool if and only if the skill premium clears

$$\frac{\mu_{\text{spec}}}{\mu_{\text{pool}}} \geq g_A(\rho, c) = c\rho + \frac{1}{1 + \dfrac{C(c,\rho)}{c(1-\rho)} + \dfrac{A\,\mu_{\text{pool}}}{w\,\lambda_F}}. \tag{4.1}$$

verified. A tempting shortcut is $\tilde{g} = 1 + (c-1)\lambda_F/(c\mu_{\text{pool}} - \lambda_F) = 1 + (c-1)\rho/(1-\rho)$, from a naive $M/M/1$-versus-$M/M/c$ response-time comparison. A sweep against simulation *falsifies* that shortcut in both directions. Equation 4.1 is the exact boundary — the companion formal paper's Theorem 2, independently re-derived and checked numerically over two thousand (ρ, c) points. Keeping the shortcut beside the exact result exposes its failure: the two thresholds cross exactly once, so the shortcut is not merely conservative. Figure 4.6 shades both error regions.

At $A = 0$ — pure response time, accountability priced at nothing — the boundary reduces to

$$g(\rho, c) = c\rho + \frac{c(1-\rho)}{c(1-\rho) + C(c,\rho)}. \tag{4.2}$$

Three properties fix its shape: $g(\rho, 1) = 1$ (a specialist against a single generalist needs no premium at all), $g \to c$ as $\rho \to 1$ (the boundary *saturates* at the capacity ratio; it never diverges), and for two generalists it is the closed form $g(\rho, 2) = 1 + 2\rho - \rho^2$. Below the boundary, sole ownership buys strict single-writer accountability at an explicit latency price that must be budgeted rather than obscured; Eq. 4.1 is where that price is written down.

Erich Owens. *What Needs an Authority: Mechanical Detection, Chartered Resolution, and the Exact Price of Sole Ownership*. The Harbor Research Program, Paper 6, 2026. Its Theorem 2 is the Erlang-C specialization boundary, together with the sweep comparing it against the naive response-time threshold.

Numbers by Hand 4.4.2 (Sole Ownership at the Queue Boundary).

The result is two lines of algebra on textbook formulas, which is the right way to first believe it. Sole ownership pays iff

$$w\lambda_F(W_{\text{solo}} - W_{\text{pool}}) \le A,$$
$$\text{i.e. } 1/(\mu_{\text{spec}} - \lambda_F) \le W_{\text{pool}} + A/(w\lambda_F);$$

invert, divide by μ_{pool}, and substitute the two textbook facts

$$\lambda_F/\mu_{\text{pool}} = c\rho \text{ and}$$
$$\mu_{\text{pool}} W_{\text{pool}} = 1 + C(c,\rho)/(c(1-\rho)),$$

and Eq. 4.1 falls out.

For $c = 2$ the Erlang-C value is $C(2,\rho) = 2\rho^2/(1+\rho)$, so the second term of Eq. 4.2 is

$$(1-\rho^2)/\big((1-\rho^2) + \rho^2\big) = 1 - \rho^2 \text{ and}$$
$$g(\rho, 2) = 1 + 2\rho - \rho^2,$$

checkable in three lines with no queueing theory at all.

For an illustrative roadmap-owner queue, suppose mutations arrive at $\lambda_F = 2/\text{hr}$; the sole owner clears them at $\mu_{\text{spec}} = 3/\text{hr}$; the alternative is three generalists at $\mu_{\text{pool}} = 1.2/\text{hr}$ each.

Then $\rho = 2/3.6 = 0.556$ and the skill premium is $r = 3/1.2 = 2.5$. Erlang-C at $(c, \rho) = (3, 0.556)$ is 0.300, so

$$\begin{aligned} g &= 3(0.556) \\ &\quad + 3(0.444)/\big(3(0.444) + 0.300\big) \\ &= 1.667 + 0.816 = 2.483. \end{aligned}$$

The premium 2.5 clears it — barely — so the sole owner wins:

$$\begin{aligned} W_{\text{solo}} &= 1/(3 - 2) = 1.000 \text{ hr against} \\ W_{\text{pool}} &= \big(1 + 0.300/1.333\big)/1.2 \\ &= 1.021 \text{ hr}, \end{aligned}$$

a margin of about 75 seconds per request.

The naive threshold would return

$$\tilde{g} = 1 + 2(2)/(3.6 - 2) = 3.5$$

and sent the work to the pool. That is the right-hand error lens of Figure 4.6. These are model calculations, not measured roadmap performance.

Now you try: at the same three-agent pool, would a specialist at $\mu_{\text{spec}} = 2.8/\text{hr}$ still be worth it on response time alone? (The premium is $2.8/1.2 = 2.33 < 2.483$ — no, and $W_{\text{solo}} = 1.25$ hr confirms it. The boundary is genuinely tight here; note that a positive accountability value A lowers g_A below g and can buy the sole owner back.)

Numbers by Hand 4.4.3 (Where the Exact and Naive Thresholds Cross).

At the round number $\rho = 0.5$:

$$g(0.5, 2) = 1 + 2(0.5) - 0.5^2 = 1.75$$

[verified, closed form].[a]

The naive $\tilde{g}(\rho, 2) = 1 + \rho/(1 - \rho)$ gives

$$\tilde{g}(0.5, 2) = 2.00$$

at the same point — already a visible gap.

Setting $g = \tilde{g}$ and clearing denominators leaves

$$\rho(\rho^2 - 3\rho + 1) = 0,$$

whose root in $(0, 1)$ is

$$\rho = (3 - \sqrt{5})/2 \approx 0.382$$

[verified, closed form]: below it $\tilde{g}$ over-certifies specialization (false-certify), above it $\tilde{g}$ over-rejects (false-reject), so the naive form errs in *both* directions rather than merely being conservative.

Now you try: run the falsification sweep behind this crossing (Exercise 4.18).

[a]Worked numbers carry a tag throughout: [verified] when a named script in the repository regenerates them, [internal] when only the chapter's own derivation does.

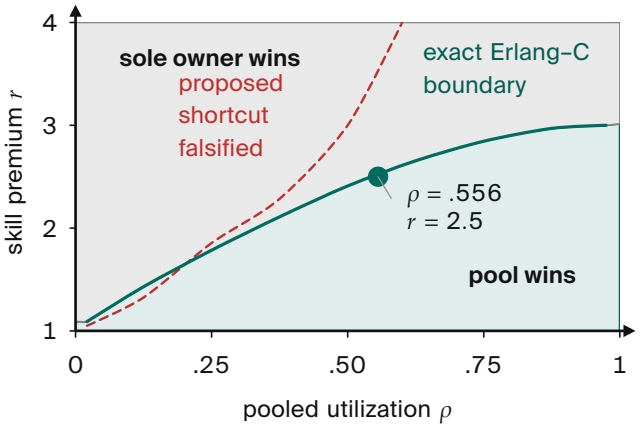


Figure 4.6. At three agents, pooling wins below $g_A(\rho, 3)$ and sole ownership above it. The dashed, superseded threshold diverges incorrectly; the worked point narrowly favours sole ownership. [internal; closed form]

Mean latency hides tail risk

Eq. 4.1 prices *mean* response time under first-come-first-served exponential service. It does not price the tail, and it does not price the thing an operator actually values in a roadmap owner — consistency of judgment across requests — which does not appear in a response-time objective at all. The corner the whole

sole-responsibility discipline exists for, the sole specialist *failing*, is priced in §4.4.3.1 below, and priced under an assumption worth reading before the formula.

4.4.3.1 The succession corner, and who prices it

Eq. 4.1 assumes a specialist who never stops. The corner the whole discipline exists for is the one where they do: they crash, are reaped, quit, or are reassigned, and a successor spins up. That corner *is* priced, in closed form, and it is priced in *The Harbor Economy* rather than here — the result is a condition on what a succession plan costs, which is an economic statement, and the economy chapter carries it as the price of the succession rule (Thm. 6.1.1), with its own verification and its own boundary. Two of its consequences belong in this chapter's argument, because they change what a sole-responsibility role means rather than what it costs.

The first is that skill has a ceiling here. With death at rate ξ and succession at rate η, the sole owner's mean response time decreases in skill but never below

$$W_\infty = \frac{\xi}{\eta(\xi + \eta)},$$

the residual downtime a request meets when it arrives during an outage. No premium buys that back, so past a point the honest fix for a slow sole role is a better handoff and not a better owner. The second is that the ceiling can bind absolutely: writing $K = W_{\text{pool}} + A/(w\lambda_F)$ for the response-time threshold in hours, pooling wins at *every* skill premium once ξ/η clears the succession price $D^\star = \eta K/(1 - \eta K)$ — and, in the case worth remembering, only while $\eta K < 1$. Where succession is fast enough that $\eta K \geq 1$ there is no such threshold at all and sole ownership cannot be ruled out on response time, which is the more common situation in a well-run fleet and the one a reader is likeliest to be in.

Both statements rest on a modelling choice the economy chapter declares and this one inherits: the specialist can die *at any time*, not only while serving. Under active-only breakdowns a fast enough specialist escapes the floor entirely and $D^\star$ does not exist. The choice is right for an owner who is reassigned or dead rather than worn out, and it is a choice, not a derivation.

4.4.3.2 What heavy tails do to the boundary

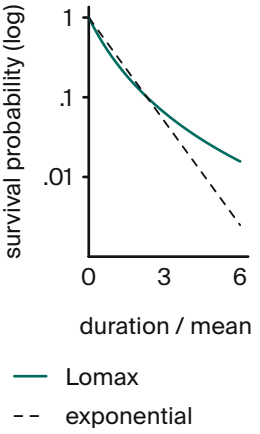
Exponential service is the assumption a queueing reader will challenge first, and they are right to: a rebuild or a flaky integration test can produce a long turn, and submissions may arrive in correlated bursts. Whether a particular workload has heavy tails or Poisson arrivals is a measurement question, not something this example establishes. Kingman's heavy-traffic result says what that costs in general: waiting grows with the sum of the squared coefficients of variation of the arrival and service processes, so a queue that is fine at $c_s^2 = 1$ can be badly congested at $c_s^2 = 6$ with the same utilization. The worry is real. What is not obvious, and matters here, is that it lands on *both* sides of Eq. 4.1 at once.

Service time and waiting time must be kept separate. Put $a = \lambda_F/\mu_{\text{pool}} = c\rho$, $r = \mu_{\text{spec}}/\mu_{\text{pool}} > a$, and $\kappa_j = (1 + c_{s,j}^2)/2$ for each service distribution. With Poisson arrivals and finite second moments, Pollaczek–Khinchine gives the specialist's mean *sojourn* time exactly, while a variability-scaled Erlang-C approximation gives the pool's:

$$\mu_{\text{pool}} W_{\text{spec}} = \frac{1}{r} + \kappa_{\text{spec}} \frac{a}{r(r-a)}, \tag{4.3}$$

$$\mu_{\text{pool}} W_{\text{pool}} \approx 1 + \kappa_{\text{pool}} \frac{C(c,\rho)}{c(1-\rho)}. \tag{4.4}$$

Only the second term in each expression is waiting. Multiplying the whole sojourn time by κ would also multiply service time, which variance at a fixed mean does not do. The comparison is therefore $\mu_{\text{pool}} W_{\text{spec}} \le \mu_{\text{pool}} W_{\text{pool}} + A\mu_{\text{pool}}/(w\lambda_F)$; it cannot in general be reduced to replacing A by A/κ.



Both means are 1: solid $(1 + t/2)^{-3}$; dashed e^{-t}. Analytic examples, not measurements.

J. F. C. Kingman. The single server queue in heavy traffic. *Mathematical Proceedings of the Cambridge Philosophical Society*, 57(4):902–904, 1961. (Waiting time grows with the sum of the squared coefficients of variation.)

Arnold O. Allen. *Probability, Statistics, and Queueing Theory with Computer Science Applications*. Academic Press, 2nd edition, 1990. (The Allen–Cunneen approximation for $M/G/c$, and the Pollaczek–Khinchine formula.)

Numbers by Hand 4.4.4 (A Variance-Sensitive Specialization Boundary).

The expressions above recover Eq. 4.1 when both $\kappa_j = 1$. Away from exponential service, the pool expression is an approximation, not an error bound.

Even equal variability on both sides can move the crossing when $A = 0$: for $c = 2$, $\rho = 1/2$, and $C = 1/3$, the exponential threshold is $r = 1.75$; setting both $\kappa_j = 2$ gives an approximate

threshold

$$r = (8 + \sqrt{124})/10 \approx 1.914.$$

These are analytic comparisons, not measurements of agent workloads. A useful test would extend `b8_specialization.py` with matched-mean lognormal and Pareto service, report simulation uncertainty, and measure the approximation's crossing error. That sweep has not been run. Infinite service variance makes the stable $M/G/1$ mean waiting time infinite, so this finite-mean comparison no longer applies. Correlated or bursty arrivals also leave the Poisson model; neither Pollaczek–Khinchine nor a variability correction alone establishes their delay distribution or tail latency.

4.4.4 Completionist obligation: completion as a verifier, not a vibe

The operator's standing complaint is the agent that *declares completion while the work is hollow* — "fixed the test by deleting it." The completionist obligation proposes an authority-side rule against accepting an unmet or unevaluable completion predicate.

Design Invariant 4.4.5 (Completionist Completion).

A **completion claim** is a commitment whose satisfaction predicate is a *verifier* derived from the task's acceptance criteria. The authority refuses to accept the claim against an unmet predicate.

An *unverifiable* predicate must **fail loudly** — surface "I cannot verify this completion" — and never silently pass.

This is the authority-side analog of discovery's *refuse-to-route* (§4.7.3): "I don't know" must beat a confident wrong answer.

Here the completion predicate is a postcondition, in Meyer's design-by-contract sense.

Bertrand Meyer. Applying "Design by Contract". *IEEE Computer*, 25(10):40–51, 1992.

Verification also consumes attention

Verification has a *cost gradient*, not a binary. A test going green is cheap to verify; "the architecture is right" is expensive, and reading a 600-line diff to confirm an acceptance criterion is itself attention-seconds that may exceed the agent's narration cost by an order of magnitude (the *verification asymmetry*). Invariant 4.4.5's fail-loud is

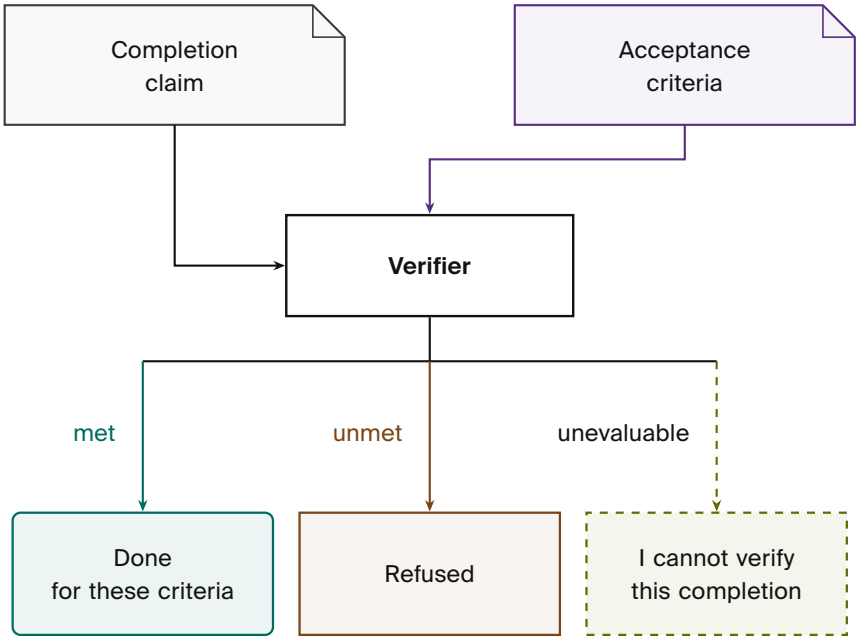


Figure 4.7. Vision: a proposed authority-side rule, not implemented enforcement or a guarantee of universal correctness. Passing it still leaves any omitted requirement unchecked.

the honest floor for the *unverifiable*; §4.5 supplies the cost model for the *expensive-but-possible* middle.

4.4.5 Thoughtful landing and the regimentation/enforcement split

The authority decides *what lands and in what order*, conflict-aware. And it has teeth: the footgun guards. Here we must be precise about what a guard can and cannot do, because the difference is the difference between a safety claim that holds and one that is wishful.

Design Invariant 4.4.6 (Prevention vs. Detection — the Reserved-Phrase Rule).

A coordination state is **physically unreachable** — prevented — *only* when a constraint forbids it *inline*, before the act commits: a database uniqueness constraint that forbids two holders of the same registered key, or a fail-closed startup gate that refuses to run against the wrong target.

A monitor that *subscribes to the log of actions already taken* is something weaker and must be named formally: **detect-and-compensate**. It observes a violation *after* the fact

and can halt or repair within a bounded window only if observation delay, response authority, and repair completion have explicit bounds. By the reference-monitor result below, such a downstream monitor cannot prevent the violation it has already observed. It can still mediate later actions; that is a separate enforcement boundary.

The phrase "physically unreachable" is therefore reserved for the constraint-enforced set — nowhere else.

This is an invariant on the chapter's *vocabulary* and on which mechanism class earns which name; the teeth behind the classification are Schneider's theorem, cited in the paragraph that follows, and are his rather than ours.

Schneider 2000

Fred B. Schneider (*Enforceable Security Policies*, 2000 — *an execution monitor that observes a system can enforce exactly the safety properties; a monitor placed downstream of the act it would forbid can detect a violation but cannot prevent it*) is decisive: a monitor downstream of a forbidden commit cannot make that committed violation unhappen. Detection and compensation need their own stated response-time assumptions. A post-commit anomaly detector — however useful — is therefore detect-and-compensate, not prevention. Honesty about this distinction is essential: a system that advertises detection as prevention has lied about its own safety envelope, which is the very failure (the confident-wrong report) this chapter is built to eliminate.

4.4.6 Human-in-the-loop escalation as a typed, prioritized interrupt

The one thing that needs a human is an *escalation* (a distress signal) routed to the operator on a reserved, highest-priority lane. The mechanism wants the critical signal to win the operator's attention — but *when* the daemon stops and asks versus proceeds is a control problem (§4.5), and the *escalation predicate* is not an afterthought: it is a rationalized alarm philosophy (**ANSI/ISA-18.2**, *Management*

of Alarm Systems — the process-control standard for alarm rationalization: every alarm has a defined priority, set-point, and operator response, with explicit flood-suppression). Frequent unhelpful alerts risk habituation and ignored warnings. Cvach's clinical review reports studies in which 80–99% of alarms were false or clinically insignificant, and discusses desensitization. That clinical evidence motivates measuring alert burden and response in agent supervision; it establishes neither the rate nor the timescale of those effects in an agent distress lane. Spence's signaling model motivates examining how an escalation cost could separate signal types. Here the proposed debit follows an adjudicated false escalation, not a bare dismissal (§4.7.3). Whether it discourages manipulation without suppressing valid warnings depends on the validation rule and its errors. The model below abstracts that rule as an upholding probability; it does not establish the validator's correctness.

The next result gives a threshold best response to a fixed validation rule. The worked model's threshold and feasible debit range are checked by `b7_escalation_band.py`; this is not a joint equilibrium proof for an operator who also changes the validation rule.

ANSI/ISA. *ANSI/ISA-18.2: Management of Alarm Systems for the Process Industries*. 2016.

Maria Cvach. Monitor Alarm Fatigue: An Integrative Review. *Biomedical Instrumentation & Technology*, 46(4):268–277, 2012.

Michael Spence. Job Market Signaling. *Quarterly Journal of Economics*, 87(3):355–374, 1973.

Theorem 4.4.7 (The Escalation Threshold and Its Tuning Band).

Let $w > 0$, $\delta \geq 0$, and let b, V be continuous on $[0,1]$, with b strictly increasing and $V : [0,1] \to [0,1]$ nondecreasing. An agent with signal u receives expected benefit $b(u)$ before the dismissal debit; $V(u)$ is the probability its escalation is upheld. Its expected payoff $\Pi_\delta(u) = b(u) - \delta w\,(1 - V(u))$ is strictly increasing. If $\Pi_\delta(0) < 0 < \Pi_\delta(1)$, there is a unique interior zero $u^\star$. Choosing escalation at indifference gives the rule $u \geq u^\star$.

Increasing δ can only shrink the escalation set. If alarm load is nonincreasing and miss loss nondecreasing in δ, the debits meeting both limits form an interval, possibly empty or unbounded. If both aggregate functions are continuous, that interval is closed; a finite $[\delta_{\min}, \delta_{\max}]$ also requires a finite upper boundary.

Proof.

Strict increase follows from strictly increasing b plus nondecreasing $\delta w V$. Continuity and the two endpoint inequalities give existence by the intermediate value theorem.

For each fixed u, increasing δ cannot increase payoff, so the escalation sets are nested.

The two aggregate constraints therefore intersect as an interval; continuity makes their sublevel sets closed. A point mass at the threshold can break aggregate continuity, so closed endpoints do not follow from monotonicity alone. □

Recall

1. When does sole ownership weakly dominate the pool? Evaluate Eq. 4.1 at $A = 0$.
2. What limits must $g(\rho, c)$ satisfy as $\rho \to 0$ and $\rho \to 1$?
3. Why can $\tilde{g}$ fail in both directions?

Exercises 4.13–4.19, p. 369.

The connection to Spence is the cost difference between types: the nominal debit is the same, but an alarm with higher $V(u)$ is dismissed less often. Strictly increasing benefit and nondecreasing validation jointly give the threshold; validation monotonicity alone does not. Calibrating δ for a deployment requires measured validation rates: the frequency with which the operator upholds an escalation at each signal level, with uncertainty and provenance recorded alongside the fitted curve. An empty feasible band means no debit meets both limits in this model; capacity or evidence quality must change. A non-monotone validator can destroy the threshold (R14; `b7_escalation_band.py`), though it need not do so for every benefit function.

Numbers by Hand 4.4.8 (The Threshold and the Tuning Band, by Hand).

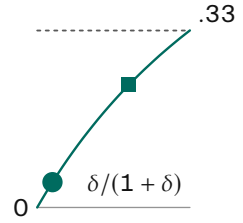
Take the linear case $b(u) = V(u) = u$. Setting $u - \delta w(1 - u) = 0$ gives $u^\star = \delta w/(1 + \delta w)$. At $\delta = 0.05, w = 1$:

$$u^\star = \delta w/(1 + \delta w) = 0.05/1.05 = 0.0476$$

[verified, closed form] — the fraction of the validation signal u above which escalating clears its cost, one of twelve (δ, w) points the script confirms against bisection to machine precision.

Under the R3 constants ($C_{\text{miss}} = 100, c_{\text{att}} = 1, C_{\text{fa}} = 5$, so $u_{\text{crit}} = 6/105 \approx 0.057$), the feasible debit interval is

$$[\delta_{\min}, \delta_{\max}] = [0.0350, 0.0720]$$



debit $\delta : 0 \longrightarrow .50$

Escalation threshold: $u^\star(\delta) = \frac{\delta w}{1+\delta w}$ $(w = 1)$.

at attention budget $A = 3.3$, and it is *empty* at $A = 2.0$ (there $\delta_{\min} = 0.396$ exceeds $\delta_{\max} = 0.072$, so no debit satisfies both walls at once) [verified closed forms, grid-confirmed].

An empty band is not a bug to tune away: it is the honest signal that the fix is capacity or evidence quality, not a better δ.

Now you try: run the script and confirm both bands (Exercise 4.19).

Figure 4.8 draws the feasible debit band directly against the attention budget, showing where the two named points sit relative to the crossing at which the band stops being empty.

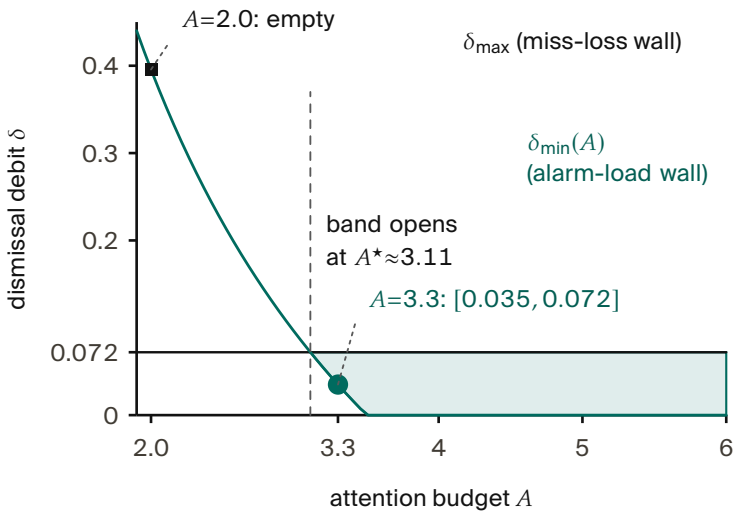


Figure 4.8. For the R3 constants, no dismissal debit meets both constraints below $A^\star \approx 3.11$. At $A = 3.3$, the feasible interval is $[0.035, 0.072]$. An empty band requires more capacity or better evidence. [verified, `b7_escalation_band.py`]

4.5 The operator as a modeled entity

The legibility layer exists for an operator who must decide what to inspect, permit, or stop. Consider a fixed review interval with one person on duty. Adding agents may increase the incoming work without increasing that person's available review time. Attention and trust calibration therefore belong in the workload model alongside the swarm's token costs (§4.8); whether either is the binding constraint depends on the workload.

More agents do not add attention

With staffing and available review time held fixed, additional agents do not automatically add review capacity. A single-server queue

can approximate sequential human review, but its service times depend on the task, the interface, and the operator's experience. We need to price both inspection effort and decision errors, not assume a universal human service rate or a future token price.

George A. Miller. The Magical Number Seven, Plus or Minus Two. *Psychological Review*, 63(2):81–97, 1956.

Miller's *The Magical Number Seven* distinguishes limits in absolute judgment from limits in immediate memory. Familiar chunks and recoding change how much information a remembered item carries. These experiments do not establish a universal seven-item limit for dashboards or a maximum number of agents one person can supervise.

Christopher D. Wickens. Multiple Resources and Performance Prediction. *Theoretical Issues in Ergonomics Science*, 3(2):159–177, 2002.

Wickens' multiple-resource theory adds a different warning: equal durations need not impose equal interference. Tasks can compete for resources associated with processing stages, sensory modalities, visual channels, and representational codes. Under the model's comparable-demand conditions, greater resource overlap predicts greater interference. This motivates measuring workload distribution as well as elapsed time. It does not prove that minimizing peak load is the universal objective, or that the proposed ambient/dense display split (§4.7.5) achieves it. That display choice needs evaluation on the intended decisions.

4.5.1 The attention objective, spined on Signal Detection Theory

David M. Green and John A. Swets. *Signal Detection Theory and Psychophysics*. Wiley, 1966.

An average such as "attention-seconds per correct decision" can conceal rare, expensive mistakes among many cheap successes. **Signal Detection Theory** (**Green & Swets 1966**) separates two questions: how well does the evidence distinguish dangerous from safe actions, and where should the inspection threshold sit? For equal-variance Gaussian evidence, the sensitivity $d' = (\mu_{\text{danger}} - \mu_{\text{safe}})/\sigma$ is the separation of the means in common standard-deviation units. Holding these distributions fixed, moving the threshold changes misses and false alarms; it does not change d'.

Here β denotes a threshold on the evidence score: inspect above it, otherwise permit the action. A *miss* permits a dangerous action; a *false alarm* inspects a safe one. The preferred threshold depends on prevalence and the costs of those outcomes. A destructive migration and an unnecessary inspection need not carry

the same cost. The next example begins with just two available policies, inspect every item in a class or inspect none.

Numbers by Hand 4.5.1 (Two Action Classes, Priced Before the Objective Is Written Down).

The objective below looks heavier than it is, so price two action classes with it first, in its crudest collapse: zoom *every* item in a class or zoom *none* of them, at an additional false-alarm cost $C_{\text{fa}} = 1$ and an attention cost of $c_{\text{att}} = 0.2$ per inspection. Convert attention and harm to the same normalized cost units, and assume inspection detects every dangerous action.

Take a migration that drops a foreign key, with a base rate of genuinely dangerous instances $\pi = 0.2$ and a miss cost $C_{\text{miss}} = 500$. Never zoom and you pay the miss term alone,

$$0.2 \times 500 = 100.0.$$

Always zoom and you pay

$$(1 - 0.2) \times 1 + 0.2 = 0.8 + 0.2 = 1.0.$$

A hundred against one: the class is decided by the miss term before irreversibility is priced separately at all, which is why the design forces the zoom rather than suggesting it.

Now take a `tests/`-only formatting diff, $\pi = 0.001$ and $C_{\text{miss}} = 5$. Never zoom:

$$0.001 \times 5 = 0.005.$$

Always zoom:

$$0.999 \times 1 + 0.2 = 1.199.$$

That is a factor of about 240 the other way — and it is worth noticing what just happened, because the auto-land grant Mira issued at 3:25 out of exhaustion (§4.2.2) has fallen out of two multiplications instead of out of her taste.

The whole apparatus below exists to keep that arithmetic honest once P_{miss} and P_{fa} stop being degenerate.

Now you try: keep prevalence positive and the other costs fixed. Which of these two policies wins as $C_{\text{miss}} \to \infty$ (Exercise 4.20)?

Design Invariant 4.5.2 (The SDT-Spined Attention Objective).

Fix the score distributions for safe and dangerous actions. Let $P_{\text{miss}}(\beta)$ be the probability of no inspection conditional on danger, and $P_{\text{fa}}(\beta)$ the probability of inspection conditional on safety. Assume inspection detects and prevents the dangerous action. Let π be the dangerous-action prevalence, C_{miss} the cost of an unprevented danger, C_{fa} the additional burden of an unnecessary inspection, and c_{att} the cost of each inspection. Express all costs in the same units without counting attention twice. With at most one inspection per action, $\mathbb{E}[\text{zooms}(\beta)]$ is the inspection probability. The objective is

$$\min_{\beta} \Big[\pi\, C_{\text{miss}}\, P_{\text{miss}}(\beta) \;+\; (1-\pi)\, C_{\text{fa}}\, P_{\text{fa}}(\beta) \;+\; c_{\text{att}}\, \mathbb{E}[\text{zooms}(\beta)] \Big]. \tag{4.5}$$

The forced-zoom policy $p(r, s, v)$ is calibrated against this loss after specifying the score distributions that determine P_{miss} and P_{fa}. An imperfect inspection requires an additional residual-harm term; the displayed loss does not price that error.

Equation 4.5 is a decision objective, not by itself a fitted detector: what the design commits to is that operator friction is set by this loss and not by taste, and *whether* an operator calibrated toward it outperforms one who is not is a separate, empirical question this chapter does not settle (the paragraph below names the live rival account).

This is a normative claim — a Bayesian SDT decision-maker is the target to calibrate toward — and not a descriptive one. Naturalistic decision-making research argues that skilled operators under time pressure do something else entirely: they recognize the situation as a case they have seen before and act on the match, rather than computing a likelihood ratio and holding it against a threshold. That is a live rival account of what the operator's instrument actually measures, and this objective does not adjudicate it; it says what Mira should be calibrated toward, not what Mira is doing at 3:25 in the afternoon.

Gary Klein. *Sources of Power: How People Make Decisions*. MIT Press, 1998.

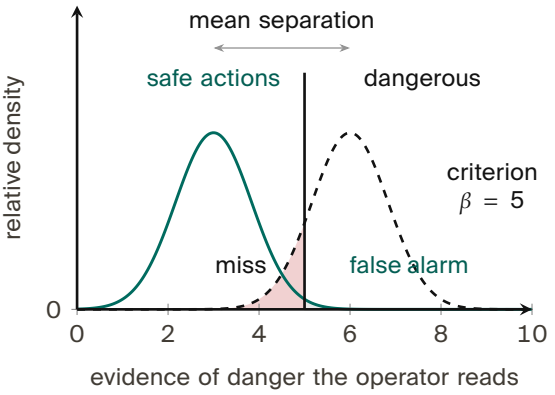


Figure 4.9. Lowering the threshold trades missed dangers for false alarms without changing sensitivity. These Gaussian shapes use an arbitrary vertical scale; d' is their mean separation divided by their common standard deviation. The drawn threshold is illustrative, not a fitted optimum. [internal, Definition 4.5.2]

Figure 4.9 lays the same decision out visually: the operator's criterion β trading misses against false alarms. The miss-cost term prices the harm of zero review; it does not always rule it out. If inspection costs more than the expected harm it prevents, zero review can minimize this objective, as the formatting example shows. Monotonic response to stakes, irreversibility, and reputation is a policy constraint to test during calibration; it does not follow from the display without a model linking (r, s, v) to the score distributions. Before a reputation layer exists, r is a flat prior.

The third class, where the answer stops being obvious. The collapse worked above — zoom *every* item in an action class or zoom *none*, so that the two costs are πC_{miss} against $(1-\pi)C_{\text{fa}} + c_{\text{att}}$ — decides the two extreme classes by factors of a hundred and of two hundred and forty, which is a comfortable place to test a formula and a useless place to learn anything from it. The class worth looking at is the one in the middle. Table 4.6 puts all three side by side at the same $C_{\text{fa}} = 1$ and $c_{\text{att}} = 0.2$ normalized cost units, with attention time converted to the same units as harm.

Now you try: at what base rate does a $C_{\text{miss}}/C_{\text{fa}} = 50$ action class cross from auto-land to forced zoom? (Set $50\pi = (1-\pi)+0.2$, giving $\pi \approx 0.024$ — roughly one dangerous action in forty.) The boundary: this comparison restricts the choices to inspecting a whole class or none of it, and assumes inspection detects every dangerous action. A score-based policy may do better. Its true optimum is weakly no worse if both constant policies remain available, but neither strict improvement nor the quality of a fitted detector follows from this table.

Table 4.6. Inspection costs at $C_{fa} = 1$ and $c_{att} = 0.2$. The first class favours automatic action, the third inspection; the middle comparison (1.20 versus 1.17) is sensitive to estimation noise.

Action class	π	C_{miss}	Never zoom	Always zoom	Verdict
`tests/`-only formatting diff	0.001	5	0.005	1.199	auto-land
Refactor touching a public interface	0.03	40	1.20	1.17	zoom, barely
Migration dropping a foreign key	0.2	500	100.0	1.00	forced zoom

Table 4.7. A pooled review rate can hide the allocation of review between these two action classes. Appropriate allocation also depends on each class's danger rate and error costs. [internal]

Action class	Class-specific review	What a pooled rate hides
Formatting	low review under the example's costs	which safe items were permitted
Migrations	high review under the example's costs	which dangerous items were inspected
Combined	one rate across both classes	allocation of review effort

4.5.2 Trust calibration is multi-dimensional, and the vigilance decrement is real

A readable digest does not by itself establish appropriate reliance. **Lee & See** distinguish trust based on performance, process, and purpose, and discuss calibration, resolution, and specificity. For this chapter, the practical question is whether the operator distinguishes contexts in which the automation has different capabilities.

The two action classes above show why a global review rate can hide useful variation. Under their stated costs, light review of formatting and close review of migrations are sensible together. Their pooled review rate can hide that allocation. Table 4.7 is an illustrative aggregation problem, not a measured operator result. A common setting can be adequate when the relevant risks and costs are alike; when they differ, calibration should retain the action class, agent, and current evidence rather than discard them.

The proposed canary mechanism is also a vigilance task: the operator monitors for defects in items with known ground truth. **Warm, Parasuraman & Matthews** review evidence that vigilance can demand substantial mental effort and produce stress; performance and workload depend on task conditions. **Mackworth's** experiments are a classic study of sustained monitoring, not a universal

John D. Lee and Katrina A. See. Trust in Automation: Designing for Appropriate Reliance. *Human Factors*, 46(1):50–80, 2004.

Joel S. Warm, Raja Parasuraman, and Gerald Matthews. Vigilance Requires Hard Mental Work and Is Stressful. *Human Factors*, 50(3):433–441, 2008.

Norman H. Mackworth. The Breakdown of Vigilance During Prolonged Visual Search. *Quarterly Journal of Experimental Psychology*, 1(1):6–21, 1948.

timer for software review. A falling catch rate could reflect time on task, probe difficulty, prevalence, or changes in review conditions. It should not automatically be interpreted as worse trust calibration or greater swarm danger. Evaluation should record those conditions and compare like workloads. Breaks and changes in probe frequency are interventions to test, with their own costs in Eq. 4.5, rather than established remedies for this particular interface.

Canary scores can be gamed

When catch rate controls friction or grants, operators have an incentive to optimize that score. Recognizing a recurring probe template can improve the score without improving detection of real defects. This is the risk highlighted by **Campbell's Law**: using an indicator for consequential decisions can distort the process it is meant to assess. Refreshed holdouts, separation of probe generation from scoring, and independent outcome evidence are proposed safeguards. None proves the program ungameable or beneficial; its effect on real decisions must be evaluated.

Donald T. Campbell. Assessing the Impact of Planned Social Change. *Evaluation and Program Planning*, 2(1):67–90, 1979.

4.5.3 Situation awareness needs projection, not just perception

Digest-with-zoom can organize current state and its supporting evidence; it does not establish what an operator understands. Endsley distinguishes perception, comprehension, and projection of future states, and treats situation awareness, decision-making, and performance as distinct stages. In Endsley and Kiris's navigation experiment, fully automated control produced poorer comprehension than manual control, but no significant perception difference. That result motivates caution about passive supervision; it does not show that every present-focused digest destroys awareness.

Mica R. Endsley. Toward a Theory of Situation Awareness in Dynamic Systems. *Human Factors*, 37(1):32–64, 1995.

Mica R. Endsley and Esin O. Kiris. The Out-of-the-Loop Performance Problem and Level of Control in Automation. *Human Factors*, 37(2):381–394, 1995.

Our *design hypothesis* is that an explicit projection mode would help an operator anticipate the swarm's next effects. The proposed freeze-probe in Figure 4.10 would evaluate those predictions; no human-performance result is claimed here. The forensic time-axis (§4.12.1) reconstructs the past; this proposed mode would look forward.

SA level	Question	Reported surface support
Perception	Current state	Digest + zoom
Comprehension	Meaning	
Projection	Anticipated effects	Proposed mode

Human performance: **untested here**.

Proposed freeze-probe

Pause; predict the next three merges; score accuracy.

Figure 4.10. A read surface can expose evidence without establishing the operator's situation awareness.

Figure 4.10 stacks Endsley's three levels directly and contrasts current support with the proposed projection mode.

4.5.4 The Bainbridge–Scott convergence and the abdication gradient

Bainbridge's *Ironies of Automation* identifies a practical tension: automating routine control can remove opportunities to practise while leaving the operator responsible for difficult exceptions. This resembles Scott's concern about losing situated knowledge, but it does not prove that every digest erases either the swarm's or the operator's mêtis. The proposed trajectory doer → approver → rubber-stamp → absent is this chapter's *abdication-gradient hypothesis*, not a measured law.

Lisanne Bainbridge. Ironies of Automation. *Automatica*, 19(6):775–779, 1983.

The **Inception Canary** is a proposed, consented, blinded probe of defect detection. It is intended to test meaningful review rather than impose arbitrary manual tasks. Its interruptions also need to be priced: Leroy's attention-residue experiments motivate measuring the cost of switching away from unfinished work, not assuming every interruption is harmful or that a probe preserves supervisory skill.

Sophie Leroy. Why Is It So Hard to Do My Work? The Challenge of Attention Residue When Switching Between Work Tasks. *Organizational Behavior and Human Decision Processes*, 109(2):168–181, 2009.

Protocol 4.5.1 (The Inception Canary protocol).

The proposed protocol measures detection of known defects on sandboxed replays; whether training with it improves real supervision is an empirical question. Operators enroll knowingly: the program's existence, rate ρ_{canary}, and stakes are disclosed at onboarding; only the *which* and *when* are blinded:

1. **Blinded holdout replay.** An out-of-band Compaction Sidecar (§4.8.2) draws from a curated holdout of *historical, already-fixed* defects (an off-by-one boundary condition, an unindexed query, a removed regression test), re-materializes one inside a sandboxed replay of a past work item at rate ρ_{canary}, and routes it through the review surface indistinguishably from live traffic. No anomaly is injected into a live pull request or pending action: the replay cannot directly modify production. Interruptions and score-driven changes to future grants can still affect real work and require evaluation.
2. **Attention queue surfacing.** The item is placed in the Attention Queue with standard high-regret scoring.
3. **Illustrative adjudication cases:**
   - *Rubber-stamp failure:* If the operator approves without expanding the forced-zoom diff, the daemon halts execution

pre-commit, logs an Inception Canary Failed event, and tightens the operator's autonomous policy lease (lowering the threshold at which future agent actions require manual confirmation).

- *Vigilant detection:* If the operator zooms, identifies the flaw, and rejects it, the daemon logs Canary Defeated and increases the *Operator Trust Score*, unlocking extended autonomous execution grants (e.g., unattended batch merges).

Other responses, including inspecting and then approving the defect, require separate scoring rules. These two cases do not specify a complete controller; grants must remain within independently authorized policy limits.

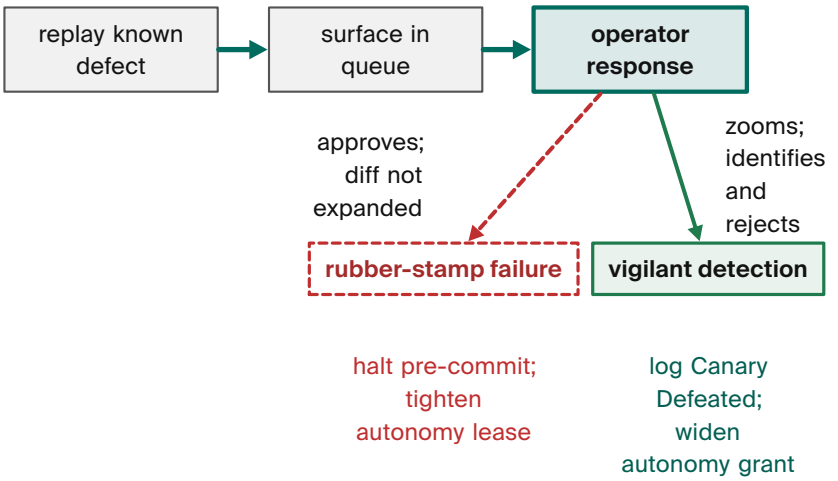


Figure 4.11. Two proposed responses to a sandboxed defect replay. The replay cannot modify production; score-driven changes to future grants still require policy limits and evaluation. Other responses need separate scoring rules. [internal]

Figure 4.11 draws the three steps into the two illustrative outcomes, with their proposed opposite effects on the operator's autonomy lease named beside them.

This records detection on known probes with informed consent and preserved provenance. Probe performance is a proxy, not a complete measure of situation awareness. Its transfer to real interventions, false-alarm burden, and operator fatigue require a controlled evaluation.

4.5.5 The operator's instrument

The operator of this chapter looks at something. In the product that something is the operator console, a GPU-native application that opens on one full-window conversation, the *Mission* view, and opens inspectors from it: fleet, cockpit, claims, parley, lineage, the

cost ledger, the active harnesses, the interruptions awaiting a human, and some two dozen more, each pane fetching one route from the daemon and rendering the records that route returns. Two things about it follow from this chapter; neither flatters it, and neither is a complaint.

First, each pane projects a particular record. The claims pane shows what cooperating clients registered, not an unrecorded write by another process; the lineage pane shows witnessed ancestry, not an unrecorded fork; the cost ledger gives a meter's count, not a causal explanation. Combining views can improve decisions, but cannot recover events that none of their sources observed. Table 4.8 records these coverage limits.

The counting bound in §4.6.2 applies only after specifying its artifact universe, possible critical sets, message budget, decoder, and allowed review set. A pane is not automatically an instance of that model. For the console, an evidence link from a digest claim to its artifact is a design requirement that makes checking possible; the theorem does not establish that links alone make the representation sufficient.

Pane	What it shows	What it cannot show
Claims	what cooperating clients registered	what a non-cooperating process wrote
Lineage	the witnessed record	the unrecorded fork
Cost ledger	a protected meter's count	the cause; a meter has no opinion about why

Table 4.8. Each console pane has its own coverage and blind spot. Combining the panes does not recover events none of them records. [internal]

Second, the console is not only a viewer. It issues commands: claim and release, begin and done, a note, a lane message, a dispatch rejection, an interruption of an agent, a kill, a steer. Each of those passes through the daemon and lands in the same single-writer record the agents write to, so the console is an authority surface in the Coordinated mode, with the same ceiling as any other cooperating client, and its commands are the operator's own work units, receipted like anyone else's. The rendering stack behind the window (a native shell with vector surfaces for the bespoke charts) is a separate decision record and carries no claim in this book. What carries a claim is the grade: the default build and the continuous integration gate compile and test the console's model on every platform, while the window itself builds on one platform only, behind a

feature flag, in one dedicated job. The grade is *built, weakly*, and the weakness is a specific one — the window most readers will picture when they hear "console" is the part of it the tests exercise least.

4.5.6 Modes of cooperation

The chapter has spoken of one operator and many agents. The product's own design binder names five ways people and agents actually share a repository, and each is a supervision regime in this chapter's sense: it fixes who holds write authority, what the person must be able to see, and how a conflict is settled. The binder's names for the two kinds of agent are worth keeping: *Voyagers* do the work, and *Longshoremen* watch it, keeping the continuous-integration state, the memory, the claims, and the transcripts in view.

Solo with staff One person writes. Longshoremen observe and suggest; they hold no write authority. The person must see the staff's suggestions as suggestions. In this regime, the staff cannot itself change repository content; the effort of evaluating its suggestions still depends on their volume and quality.

Pair with agent One person and one Voyager. The agent may suggest, patch, test, and respond to guidance; the person directs. Authority is shared turn by turn, and the person must see every patch before it lands, which is the regime the forced-zoom diff of the canary protocol above was designed for.

Ensemble Several Voyagers, partitioned by context, with a Longshoreman tracking the dependencies between partitions and the parleys that resolve them. Write authority is distributed by the partition, so the cost of seeing is paid per partition and the tracking Longshoreman is the digest this chapter prices.

Nightshift Agents work unattended under strict budgets, tests, and pull-request rules, then present artifacts and transcripts for review. The agent writes within its budget; the person sees the artifacts, not the acts. The binder's rule is the one to remember: a Nightshift agent may open a pull request under budget, but merging requires explicit rules and green gates. The person must evaluate work they did not watch being produced. Its review cost depends on the artifacts, evidence, and decision, not on the mode name alone.

Harbor co-edit People and agents as co-equal peers in one governed editor buffer, with claims, semantic conflicts, and salvage. Write authority is co-equal and conflicts are settled by the claim machinery of *The Single-Writer Kernel*: a same-symbol write is blocked or forced into a parley, a dependency conflict warns, a low-confidence overlap is advisory, and a rename or a delete draws the stronger review.

Three proposed governance rules cut across the modes: the operator controls the harbor's policy while agents choose commitments within it; a Longshoreman must not silently perform a consequential action outside its mandate; and a required human decision must be visible in the instrument. These are design requirements, not observations that every product mode already enforces them.

The five modes are supervision patterns, not a total order of authority or information cost. For example, inspecting one unattended, well-tested formatting patch can be cheaper than following a complex pair-programming session. To compare modes, hold the task and acceptable error rate fixed, specify the available evidence, and measure review effort. The study proposed in the front matter would need those controls before claiming an ordering.

Recall

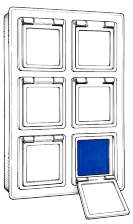
1. Recall the regret head's three factors and its calibration premise.
2. When can a pooled review rate hide class differences?
3. Can canary scores improve without better defect detection? Why is a safeguard no guarantee?

Exercises 4.20–4.24, p. 370.

4.6 Read-poverty: when inspection falls behind

Suppose an operator can inspect six short updates per minute. Two workers each producing one update per minute leave room to spare. Fifty workers at the same rate produce fifty updates per minute, so an unfiltered queue grows by forty-four updates per minute. This calculation assumes fixed update size and inspection cost; it illustrates a workload, not a measured limit on human attention.

Reading every record is not the only possible task. If the operator needs to identify the owner of an OAuth repair, a maintained directory may answer that question without presenting every worker's history. If the operator must decide whether the repair is safe to admit, an ownership label alone is inadequate: the relevant revision, checks and authority must remain accessible.



Like a shipboard annunciator dropping a signal flag only when a monitored circuit faults, selective attention suppresses routine traffic to present only actionable exceptions.

The chapter models this mismatch using a limited observation budget. A digest selects what crosses that boundary, and its adequacy depends on the decision to be made. The finite counting arguments below specify their own source, encoding and error assumptions. The update-count example does not measure a Shannon channel capacity or derive a universal human throughput in bits. Increasing the inspection budget changes the model's parameters; it does not remove the need to say which distinctions a summary must preserve.

Empirical Hypothesis 4.6.1 (Read-Poverty).

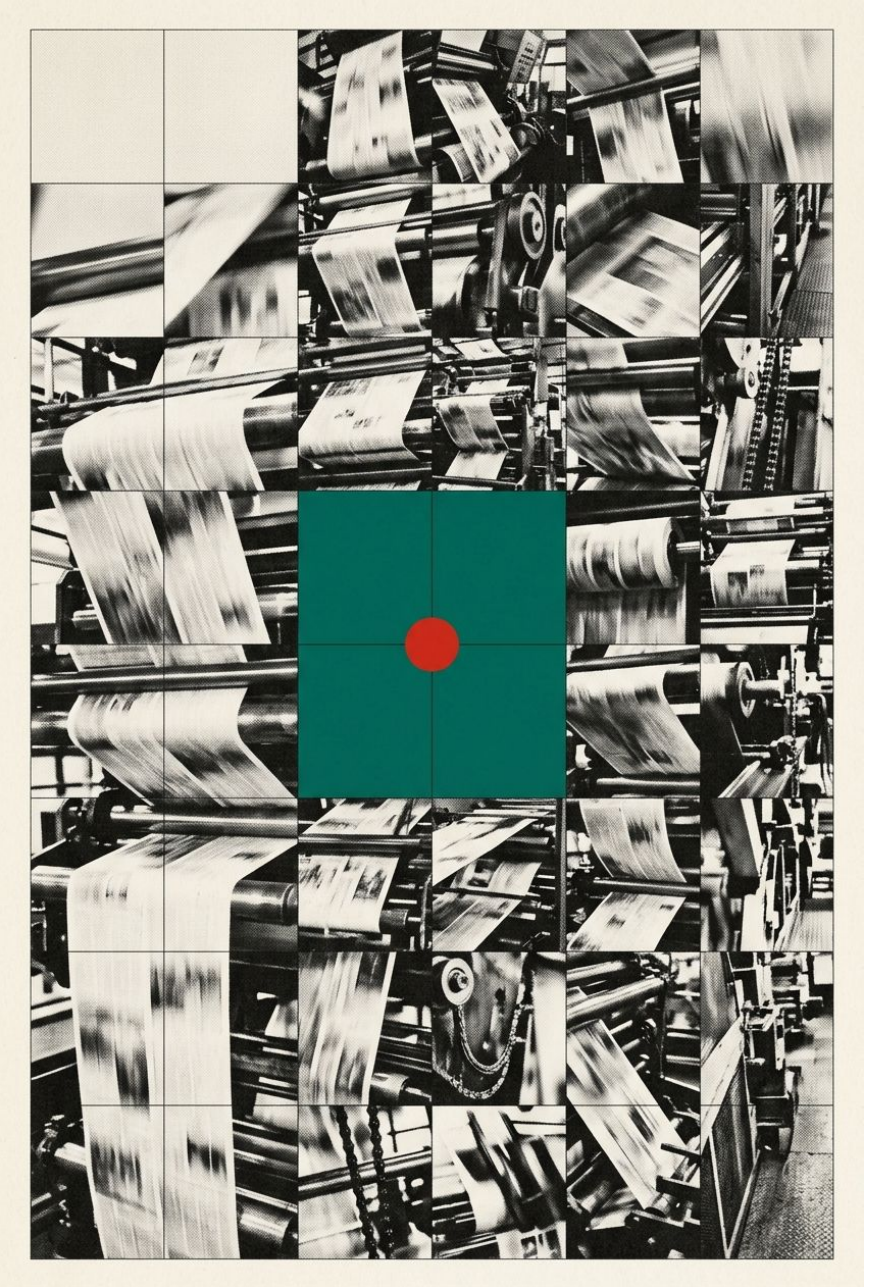
Read-poverty is a regime in which legible state accumulates faster than it can be consulted, so participants fall back to $O(n)$ eyeball search over the population, or, worse, act blind.

Naming the regime costs nothing; the claim that carries weight — and that wants measurement rather than assent — is that real swarms enter it *before* they exhaust the known write-side fixes, so that read-poverty and not write-contention is what binds as n grows.

Nothing below proves that; the lower bound of §4.6.2 prices what a digest must spend *given* the regime, which is a different question.

The hypothesis is that inspection can become the limiting resource while the write side still has capacity. Testing it requires measuring both sides on the same workload. The results below describe consequences of a specified inspection model; they do not establish which bottleneck dominates an arbitrary swarm.

A **directory** pays a maintenance cost to make a class of lookup cheaper. Its query cost depends on its representation, the query and the result size; indexing alone supplies no universal complexity bound. Nor does a fast answer necessarily preserve what matters. "This agent owns auth" omits relevant evidence if its last three auth pull requests were reverted. The directory must support inspection of that evidence, rather than turn an entry into an unqualified verdict. This motivates the zoom requirement (Invariant 4.3.2).



Landmark 08 · Read-Poverty & The Legible Swarm

An operator has a finite inspection budget. Structured summaries and accessible evidence can reduce the cost of a particular decision. Their value depends on what they preserve and what errors they hide; a shorter message is not automatically a better account.

4.6.1 Value at different swarm sizes

The legibility layer should help a solo developer as well as a larger team. At $n = 1$, a briefing can locate unfinished work, an attestation can expose what was checked, and a completion receipt can point to the result. These are proposed benefits for decisions about one agent; their value still depends on accuracy, review effort, and the task. Discovery ranking is not needed to select among agents when there is only one candidate.

The same surfaces must remain useful when review demand exceeds capacity. Comparing populations such as two, fifty, or five hundred agents could help study that transition, but those numbers are not established thresholds. Update rates, query selectivity, staffing, and evidence quality matter alongside the number of agents.

The next section separates three functions: a registry records who exists and where their evidence lives; relevance ranking selects candidates for a query; reputation associates observed outcomes with stable identities. These functions have dependencies, but do not form a strict ranking-first pipeline. Stale registry entries can misdirect retrieval; reputation needs attributable evidence, yet can attach to an identity even in an unranked registry. Ranking may subsequently use that reputation as one input.

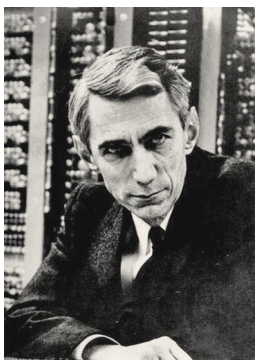
4.6.2 A legibility lower bound

A useful limit follows by counting how many critical sets each digest message can cover. This is a combinatorial information argument, related to counting bounds in group testing. Here the decoder selects a set of at most m artifacts for the reader to inspect. One message can cover several possible critical sets, provided that every artifact in each of those sets is included in the selected review set. The message budget and the review budget therefore constrain one another.

Consider four artifacts, exactly one critical, with permission to open two. One bit can select either the pair $\{1, 2\}$ or the pair $\{3, 4\}$; the encoder sends the pair containing the critical artifact. Zero bits cannot suffice: a fixed pair misses either of the other two possibilities. If four opens are allowed, zero bits do suffice. The next bound counts this trade between message length and inspection effort.

Robert Dorfman. The Detection of Defective Members of Large Populations. *Annals of Mathematical Statistics*, 14(4):436–440, 1943.

Frank K. Hwang. A Method for Detecting All Defective Members in a Population by Group Testing. *Journal of the American Statistical Association*, 67(339):605–608, 1972.



A fixed message budget distinguishes only finitely many cases. The review budget limits how many critical sets one message can safely cover.

Theorem 4.6.2 (Legibility Lower Bound with a Review Budget).

Let N artifacts contain an unknown critical subset S of size k. A digest maps each possible S to a review set $\hat{S}$ with $S \subseteq \hat{S}$ and $|\hat{S}| \leq m$, where $k \leq m \leq N$.

Any such zero-miss digest needs at least

$$\log_2 \binom{N}{k} - \log_2 \binom{m}{k}$$

bits for a fixed-length message (or a prefix-free code in the worst case). An integer message budget must round the lower bound upward.

Exact identification ($m = k$) recovers the $\log_2 \binom{N}{k}$ bound; allowing the operator to open everything ($m = N$) correctly reduces the bound to zero.

Proof sketch.

There are $\binom{N}{k}$ possible critical subsets. One digest message whose review set has size at most m can safely cover at most $\binom{m}{k}$ of them, namely the k-subsets contained in that review set.

Therefore at least $\binom{N}{k}/\binom{m}{k}$ distinct messages are required. Taking base-two logarithms gives the bound.

If fewer messages are available, some critical subset is not contained in its message's review set, producing a false negative — an unopened critical artifact. This is a coverage failure. It differs from a broken zoom link: a digest can link every selected item correctly while omitting the one item the operator needed.

Hence the stated log-ratio is a lower bound for a zero-miss digest with review budget m. □

verified.[3] The analogy that carries the proof: a digest is not a description of the night's work but a *message* that selects, from a

[3]A standalone, submission-form version of the results folded into this subsection, §4.7.2 and §4.8.2 is *The Price of a Summary* (research paper 1); the sweeps cited here are its artifacts, regenerated at seed 20260816.

Erich Owens. *The Price of a Summary: Information-Theoretic Limits of Agent Oversight*. The Harbor Research Program, Paper 1, 2026. Companion formal paper to this chapter; its Theorems 1–4 are the proved versions of several results stated here.

codebook shared with the overseer, which m items to open. A B-bit message distinguishes at most 2^B codewords, and each codeword covers only $\binom{m}{k}$ of the $\binom{N}{k}$ ways the critical items could be placed. The misread to preempt: the floor does not say a digest needs $B^\star$ bits to be useful. It prices the zero-miss guarantee only; cheaper digests are fine the moment misses are tolerated, and the two-constraint rate below prices exactly that.

Figure 4.12 distinguishes a single-reader floor, the sum of two such floors, and a joint floor for disjoint critical sets. The formal comparison follows in Theorem 4.8.2.

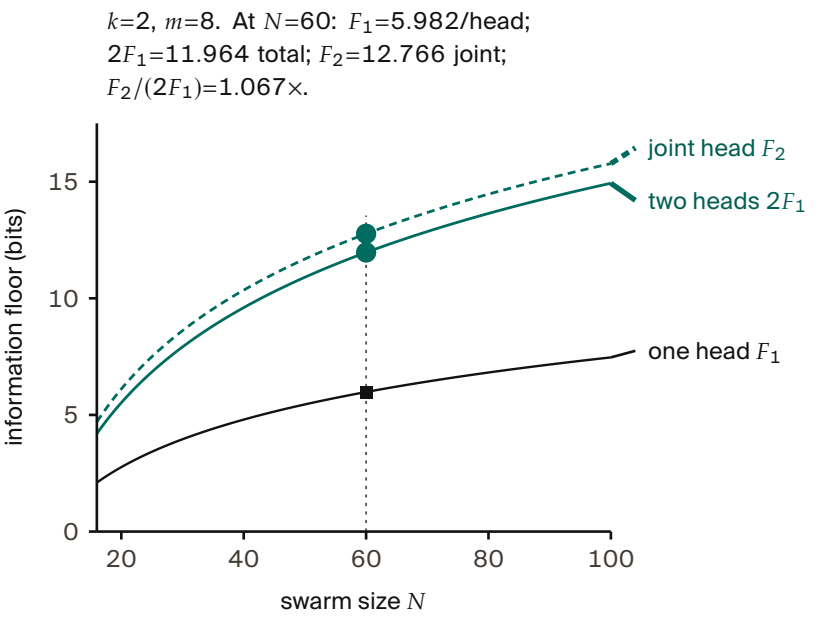


Figure 4.12. Separate readers may each open eight items; the joint reader may open eight total. The comparison changes the total opening allowance. These counting lower bounds are not achieved implementation costs.

The falsification experiment. For uniformly distributed critical k-sets, a B-bit digest's zero-miss probability is at most $\min\left(1,\ 2^B\binom{m}{k}/\binom{N}{k}\right)$. This counting ceiling need not be attainable: review sets can overlap in the critical sets they cover. A statistically reliable miss rate below the implied floor would challenge the model or its implementation. Three encoders were run against a shared randomized-cover codebook: an *oracle* that sees the true set and addresses the best codeword its B bits can reach, a *noisy* encoder (feature signal-to-noise ratio near 1), and a *random* one, with 4,000 placements per budget point, sweeping B through the floor at N=60, k=2, m=8. Result: 0/16 violations; the oracle hugs

the floor from above and the others pay a premium [internal, `a7_experiment.py`].

A wrong turn, reported. A first implementation handed the oracle a perfect per-item score vector and charged B bits only for resolving rank ties: $8/14$ apparent violations. That was not a refutation but an unpriced side channel. The identity of the critical set leaked outside the metered message. Rebuilding the model so that B is a literal message length gating an encoder and decoder pair restored agreement. The transferable rule: a simulation that appears to beat an information bound is leaking through an unpriced channel; audit the meter before doubting the theorem, and before trusting the refutation.

Numbers by Hand 4.6.3 (Digest Bits and File Opens).

Take $N = 1000$ artifacts with $k = 3$ critical among them and a review budget of $m = 10$ opens. The floor is

$$\log_2 \binom{1000}{3} - \log_2 \binom{10}{3} = 27.31 - 6.91 = 20.4 \text{ bits}$$

— requiring at least 21 fixed message bits after rounding upward. This is a necessary bound, not a construction of a 21-bit encoder, and it selects a set of at most ten opens containing the three critical files.

Widen the budget to $m = 100$ and the second term grows to

$$\log_2 \binom{100}{3} = 17.3,$$

cutting the floor to 10.0 bits: buying the operator more opens is exactly buying the digest fewer bits, at a rate the formula prices.

Now you try: at $N = 60$, $k = 2$, $m = 8$ — the configuration the companion paper's sweep hammers hardest — what is the floor?

$$10.79 - 4.81 = 5.98 \text{ bits;}$$

the sweep found no encoder beating it in 16 tries.

The bound is the formal reason *forgetting must be selective, not lossy-by-volume*: a digest may drop inert detail freely, but it must retain enough to *point at* every critical artifact, which is precisely the "summarize the state, link to the work" rule. It also bounds recursive summarization (§4.8.4): each re-summarization that drops below the floor on the critical set is irreversible information loss, which is why one must compact from the durable artifacts, never from the previous summary.

The other half of the price: what a zoom costs. The floor above prices only the first stage, the digest that narrows N artifacts to a flagged set of size F. It says nothing about the second stage, which is where Invariant 4.3.2 sends the operator. Two results complete the account. The first prices the digest once misses are tolerated; the second prices the zoom.

Model each artifact's critical status as i.i.d. $X \sim \mathrm{Bern}(p)$, let the digest emit a per-item flag $\hat{X} \in \{0, 1\}$, and price the two failure modes separately:

$$R(\delta, f) \;=\; \min_{P_{\hat{X}|X}} I(X;\hat{X}) \quad \text{s.t.} \quad \Pr(X{=}1, \hat{X}{=}0) \le \delta, \qquad \Pr(\hat{X}{=}1) \le f.$$

This is a custom formulation, deliberately: classical rate–distortion charges one scalar distortion, which cannot express that oversight prices a missed critical differently from a wasted open. The nearest published lines are expansion coding with one-sided error for continuous sources and the rate–distortion–classification theory for Bernoulli sources; neither prices false negatives as a separate constraint for binary sources.

H. Si, O. O. Koyluoglu, and S. Vishwanath. Lossy Compression of Exponential and Laplacian Sources Using Expansion Coding. arXiv:1308.2338, 2013.

Nam Nguyen, Thinh Nguyen, and Bella Bose. Rate-Distortion-Classification Representation Theory for Bernoulli Sources. arXiv:2601.11919, 2026. Online source

Theorem 4.6.4 (The Two-Constraint Digest Rate, Pinned).

For $X \sim \mathrm{Bern}(p)$ with $0 < p < 1$, $0 \le \delta < p$ and $p - \delta \le f < 1 - \delta$, if $f < 1 - \delta/p$ (the flag rate below which an X-independent flagger cannot meet the miss constraint), both constraints bind at the optimum and the minimizing joint is fully pinned:

$$q_{11} = p - \delta, \qquad q_{10} = \delta, \qquad q_{01} = f - (p - \delta), \qquad q_{00} = 1 - f - \delta,$$

feasible iff all four are nonnegative, and $R(\delta, f) = I(X;\hat{X})$ evaluated at this joint.

Because the joint is pinned, so is the rate, and it is worth printing in the form someone can actually evaluate rather than leaving as a named functional — with source margin $\Pr(X{=}1) = p$ and digest margin $\Pr(\hat{X}{=}1) = f$, the mutual information is the four-term sum

$$\begin{aligned} R(\delta, f) = {} & (p-\delta)\log_2 \frac{p-\delta}{p\,f} \\ & + \delta \log_2 \frac{\delta}{p\,(1-f)} \\ & + (f-p+\delta)\log_2 \frac{f-p+\delta}{(1-p)\,f} \\ & + (1-f-\delta)\log_2 \frac{1-f-\delta}{(1-p)(1-f)}, \end{aligned} \tag{4.6}$$

one term per cell of the 2×2 joint, each comparing that cell against the product of its margins, with $0 \log_2 0$ read as 0. At the zero-miss, tightest-flag corner, $R(0, f{\to}p) = H(p)$: the digest must spend the source's full entropy — and Eq. 4.6 shows why in two lines, since $\delta = 0$ and $f = p$ kill the middle two terms and leave $p \log_2(1/p) + (1-p)\log_2(1/(1-p))$. Past the line $f \geq 1 - \delta/p$ an independent flagger already meets the miss budget on its own and the true rate is exactly 0.

Proof.

$I(X;\hat{X})$ is convex in $P_{\hat{X}|X}$ and decreases along the feasible ray toward the independent coupling, where it is 0. An independent flagger $\hat{X} \sim \mathrm{Bern}(\phi)$ has miss mass $p(1-\phi)$, so it meets the miss constraint only when $\phi \geq 1 - \delta/p$; when $f < 1 - \delta/p$ that coupling lies outside the flag budget, so the minimum sits on the boundary where any remaining slack in either constraint could otherwise be spent moving further along the ray, and both constraints bind. Binding constraints fix $q_{10} = \delta$ and $q_{11} + q_{01} = f$; with the source margin $q_{10} + q_{11} = p$ this pins all four cells. At $(\delta, f) = (0, p)$ the joint is diagonal, $\hat{X} = X$, and $I = H(p)$. □

A conservative digest over-flags to be safe: it emits a flagged set of size F containing the k true criticals with $k \ll F$. The naive second stage opens all F. The adaptive second stage plays twenty questions with group queries: open a *group* of flagged items at once

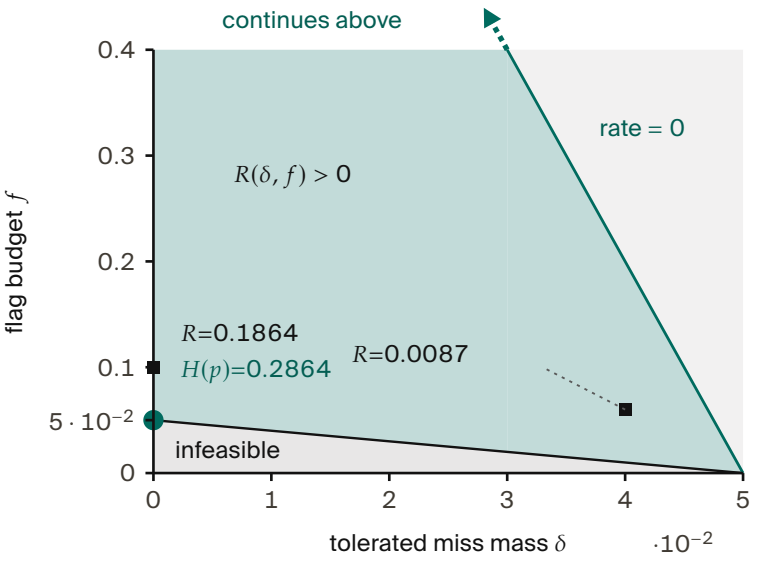


Figure 4.13. At $p = .05$, the positive-rate wedge lies between $f = p - \delta$ and $f = 1 - \delta/p$; the zero-rate boundary enters at $(.03, .4)$ and continues above-left outside the panel.

(an aggregate check cheap at group level, a combined diff scan or a batched test run), learn whether the group contains any critical, and split only the groups that do. The analogy transfers because its defining relation transfers: halving a set costs one question and eliminates half the candidates only when positives are rare in the set. The misread to preempt: twenty questions finds *one* answer in $\log_2 F$ queries; with k answers the tree forks, and the accounting below is precisely the price of the forking.

Theorem 4.6.5 (Zoom Advantage).

Let a flagged set of F items contain an unknown set P of $1 \le k \le F$ positives. Assume a noiseless group oracle of unit query cost reports exactly whether $S \cap P = \emptyset$. Adaptive halving (query the flagged set; discard any group that tests negative; report any positive singleton; otherwise split the group into halves of sizes $\lfloor |S|/2 \rfloor, \lceil |S|/2 \rceil$ and recurse on both) identifies P exactly, using a total number of group queries

$$Q \le 2k \lceil \log_2(F/k) \rceil + 4k \qquad \text{(and always } Q \le 2F - 1\text{)},$$

for every placement of the k positives, against F opens for flat inspection.

The leading constant is tight: for F and k powers of two, evenly spread positives force $Q = 2k \log_2(F/k) + 2k - 1$. Written in the

flagged-set density $d = k/F$ the guaranteed advantage is

$$\mathrm{adv}(d) \;=\; \frac{F}{2k\lceil \log_2(F/k)\rceil + 4k} \;=\; \frac{1}{d\big(2\lceil \log_2(1/d)\rceil + 4\big)},$$

which exceeds 1 exactly on $0 < d < 1/12$ and equals 1 at $d = 1/12$.

Proof.

Every group the algorithm queries is a node of the halving tree T of $[F]$: the root is the full flagged set, each internal node's children are its two halves, leaves are singletons. A node of size s has children of size at most $\lceil s/2 \rceil$, so every node at depth d has size at most $\lceil F/2^d \rceil$, T has depth $D \le \lceil \log_2 F \rceil$, and T has exactly $2F - 1$ nodes, which gives the unconditional bound.

Call a node *positive* if its group meets P. The algorithm splits exactly the positive non-singleton nodes it visits, and every queried node is either the root or a child of a split node; writing S^+ for the number of positive non-singleton nodes of T, $Q \le 1 + 2S^+$. The nodes at any fixed depth are pairwise disjoint subsets of $[F]$, so each positive item lies in at most one of them, and at most $\min(k, 2^d)$ nodes at depth d are positive. Charge the positive nodes in two regimes. At the top of the tree, $d \le \lfloor \log_2 k \rfloor$, charge each positive node to its level: at most $\sum_{d=0}^{\lfloor \log_2 k \rfloor} 2^d \le 2k - 1$ in total. Below the fan-out, $d > \lfloor \log_2 k \rfloor$, charge each positive node to one positive item it contains; by disjointness within a level each of the k positives absorbs at most one charge per level, and non-singleton nodes exist only at depths $d \le D - 1$, so the number of charging levels is $D - 1 - \lfloor \log_2 k \rfloor < (\log_2 F + 1) - 1 - (\log_2 k - 1) = \log_2(F/k) + 1$; an integer strictly below $\log_2(F/k) + 1$ is at most $\lceil \log_2(F/k) \rceil$, so these levels contribute at most $k\lceil \log_2(F/k) \rceil$ charges. Summing, $S^+ \le k\lceil \log_2(F/k) \rceil + 2k - 1$ and $Q \le 2k\lceil \log_2(F/k) \rceil + 4k - 1$. Correctness is immediate: a discarded group is certified free of positives, every positive survives discarding at every level, and each is eventually isolated and reported.

For tightness take $F = 2^a$, $k = 2^b$, $b \le a$, positives at positions $\{iF/k : 0 \le i < k\}$. Every node at depth $d \le b$ contains a positive, so all $2^{b+1} - 1 = 2k - 1$ nodes down to depth b are queried; below depth b exactly k nodes per level are positive, each is split at levels

$b, \ldots, a-1$, and each split queries two children; the total is $(2k-1)+2k(a-b) = 2k\log_2(F/k)+2k-1$. The density form follows by dividing F by the bound, and the step function $d(2\lceil\log_2(1/d)\rceil+4)$ first touches 1 at $d = 1/12$. □

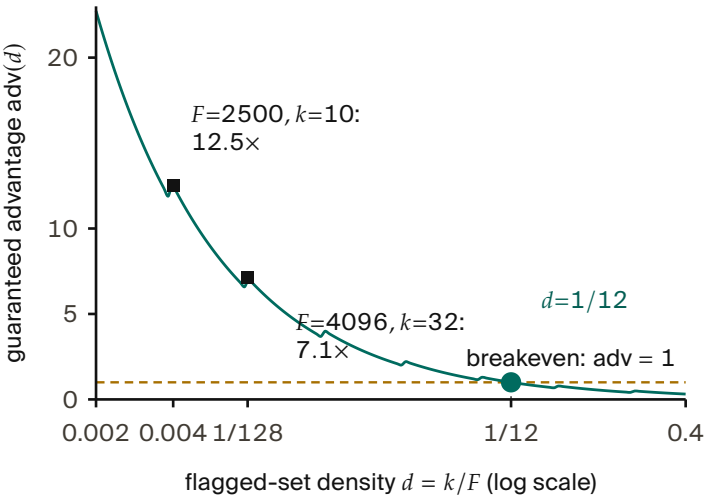


Figure 4.14. Adaptive halving guarantees an advantage when $d < 1/12$. At $F = 2500, k = 10$, its guarantee is 12.5×; the harness mean is 15.3×, distinct from that worst-case bound. [internal, `zoom_bound_check.py`]

Figure 4.14 plots the guaranteed advantage adv(d) itself against flagged-set density, marking the crossing at $d = 1/12$ alongside the chapter's own F=2500, k=10 and F=4096, k=32 configurations.

Positioning. None of the group-testing machinery is new mathematics: Dorfman introduced pooled testing in 1943, and Hwang's generalized binary splitting finds k defectives in $\log_2\binom{F}{k} + O(k)$ tests, essentially matching the counting lower bound $\lceil\log_2\binom{F}{k}\rceil \geq k\log_2(F/k)$ that any adaptive strategy with binary outcomes must pay; the full theory is Du and Hwang. Plain halving pays a factor $2 + o(1)$ over Hwang because it re-queries both children of every split rather than inferring one subgroup and searching within the other. The theorem's contribution is the explicit worst-case constant for the exact procedure the digest architecture runs, proved tight, which turns the harness's observed $\approx 2\times$ gap over the ideal $k\log_2(F/k)$ from an empirical curiosity into the algorithm's provable constant. A dedicated check re-runs the exact procedure: at F=2500, k=10, mean 162.6 opens over 150 random placements, maximum 183, all under the bound of 200; random sweeps over

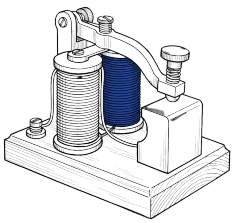
Dorfman 1943

Hwang 1972

Ding-Zhu Du and Frank K. Hwang. *Combinatorial Group Testing and Its Applications*, 2nd ed. World Scientific, 2000.

$F \in [16, 5000]$, $k \in [1, 50]$ and adversarial evenly-spread and clustered placements never exceed the bound; and the tightness configuration F=4096, k=32 lands exactly on $2k \log_2(F/k) + 2k - 1 = 511$ [internal, `zoom_bound_check.py`].

The composition window. The two prices are drawn from one budget, and they compose only inside a window that can be stated exactly. Stage one sets stage two's density: by Theorem 4.6.4 both constraints bind, so the flagged set holds $(p - \delta)N$ real criticals among fN flags and $d = k/F = (p-\delta)/f$. This worst-case bound certifies a strict query saving when $d < 1/12$, equivalently $f > 12(p-\delta)$; at equality it certifies a tie. Failure of this sufficient certificate does not prove that adaptive inspection is slower on a particular placement of positives; the measured mean crossover is about twice as forgiving, $d \approx 0.165$ [internal, `zoom_bound_check.py`]. At p=0.05, δ=0 the window certifies a strict saving only at $f > 0.60$: the digest must flag over 60% of the night's artifacts before the second stage is guaranteed to beat reading the flags. Inside the window the flags stage one adds are cheap in the second stage, because k is fixed by the source and doubling F costs only the logarithmic term, $2k$ further opens. Outside it flat inspection gives the simpler F-query guarantee, although adaptive inspection can still win on some inputs. At the dense end it can lose: at F=100, k=90 the run costs 199 opens against 100 flat [internal], which is the unconditional $2F-1$, reached because at that density essentially every node of the halving tree is positive. The design consequence for this chapter is therefore conditional, and the condition is now a number: "summarize the state, link to the work" is cheap when the digest over-flags generously and the zoom is adaptive, and the operator's dial (bits on the digest, opens on the zoom) has a favorable exchange rate exactly in the sparse-flag corner.



Like a Morse telegraph sounder clicking out discrete impulses across a narrow wire, discrete signals must encode information efficiently across the human attention bottleneck.

Numbers by Hand 4.6.6 (The Bernoulli View of the Floor, and the Zoom-Advantage Numbers).

Let p be the critical-item probability. In the special case of exact identification, with no false positives and no misses, the asymptotic i.i.d. Bernoulli rate is the binary entropy

$$H(p) = -p \log_2 p - (1 - p) \log_2(1 - p)$$

per artifact. This matches the large-N limit of Theorem 4.6.2 at $k/N \to p$ and $m = k$; it is not the rate for every review budget.

At $p = 0.05$:

$$H(0.05) = 0.2161 + 0.0703 = 0.2864 \text{ bits/symbol}$$

by direct substitution. The probability 0.05 is illustrative, not a measured prevalence for agent failures.

Allowing extra flags or a positive missed-critical probability can lower the rate: the two-constraint rate $R(\delta, f)$ drops to 0.1864 bits at $f = 0.10, \delta = 0$ and to 0.0087 once $\delta = 0.04, f = 0.06$ [internal, `b1_frontier.py`] — $R(0, p) = H(p)$ is the corner, not the whole curve.

For the zoom side, the script's own $k \log_2(F/k)$ column at $F = 2500$ flagged, $k = 10$ critical reads $79.7 \approx 80$: an order-of-magnitude sanity check, tiny next to 2500 flat opens.

The simulated adaptive-halving harness measures 163.7 opens at that same point, comfortably inside the 200-open guaranteed worst case quoted above, for a realized

$$2500/163.7 \approx 15.3\times$$

reduction [internal, `b1_frontier.py`] — better than the guarantee's own 12.5×, exactly as a worst-case bound should allow on a typical run.

Now you try: run the script and read the F=1000, k=20 row (Exercise 4.30).

Where this stops

Theorem 4.6.2 lower-bounds the zero-miss guarantee in a worst-case combinatorial model with a data-independent decoder; it says nothing against cheaper digests at nonzero miss (Theorem 4.6.4's job) and nothing average-case.

The falsification result 0/16 tests the encoder and decoder formalization at one swept regime (N=60, k=2, m=8); the noisy-encoder curve in that experiment barely beats random at unit signal-to-noise ratio and is not evidence that noisy digests are near-optimal.

The frontier $R(\delta, f)$ is an i.i.d. Bernoulli model of critical status, a modeling commitment rather than a law, and its pinned closed form holds only while $f < 1 - \delta/p$; evaluating the formula past that line produces a spurious uptick near $\delta/p \to 1$, a domain-of-validity error and not a feature of the frontier.

Theorem 4.6.5 assumes a reliable group query at unit cost; correlated or noisy group checks, and group costs growing with $|S|$, change the accounting. Plain halving is a factor of about two from Hwang's optimum by design: the theorem bounds the deployed algorithm, not the best one.

The sufficient certificate for a strict saving from the stated halving bound uses $f \geq 12(p - \delta)$. This is not a necessity condition for composing a digest and a zoom procedure, nor a proof that zoom cannot help outside that range. Actual counts of flags and critical items govern each run.

4.7 Discovery: the directory substrate and the split ranker

A performative cannot be addressed until the swarm knows who exists. The *directory substrate* — the existence layer a message must address — belongs with the discovery story, so we treat it here.

Three losses, named before they collide. This section and the next two prove that certain read surfaces cannot share a scoring head (Thm. 4.7.1, Thm. 4.8.1). Those are impossibility results, and an impossibility result is only as legible as the objects it says cannot be identified. Three distinct losses run through the rest of the chapter, and a reader who has not seen them side by side will meet them one at a time, several pages apart, and reasonably assume they are three weightings of one thing. They are not, and Table 4.9 is the inventory: what each reads, what it optimizes, in what units it is wrong, and who consumes the answer.

Read the table as the setup for two crossing pairs rather than as a taxonomy for its own sake. Rows 1 and 2 both induce an order on items and disagree on the pair (safe capable completion, dangerous incapable proposal) — that is Thm. 4.7.1. Rows 2 and 3 both score

Recall

1. Without looking: what two terms does the legibility lower bound subtract to get the bit floor, and what happens to it as the review budget m grows to N?
2. State the zoom theorem's opens bound and its noiseless unit-cost group-query assumption. Why does sparsity affect whether that bound certifies a saving over opening every flagged item?
3. Why is a floor stated for an oracle-known critical set not automatically a floor against an adversary who controls what looks critical?

Exercises 4.25–4.30, p. 371.

Table 4.9. These objectives act on different objects. The incompatibility theorems compare the commensurable pairs: rows 1–2 rank the same items; rows 2–3 score the same trace. [internal]

Loss	Input space / what is optimized over	Objective and loss metric	Target consumer
Directory candidate fit (§4.7.2)	a (candidate, query) pair (c, q); optimized over the live candidate set	*Maximize* fit$(c, q) =$ clamp$(z(c, q), 0, 1)$, Eq. 4.7. Wrongness is measured in *mis-routing*: a capable candidate ranked below an incapable one	the routing agent, choosing who to ask
Human oversight regret (§4.5.1, §4.8.2)	a compaction D of a trace E; optimized over what to surface under budget	*Minimize* $L_{\text{op}}(D) = \sum_{e \in E \setminus D}$ regret(e) with regret = stakes × irrev × anomaly (Eq. 4.8). Wrongness is measured in *unreviewed irreversible loss*	the human operator, deciding whether to intervene
Successor continuation (§4.8.2)	a compaction D of the same trace E	*Minimize* $L_{\text{succ}}(D) =$ KL$\big(P_{\text{next}}(\cdot \mid E) \,\|\, P_{\text{next}}(\cdot \mid D)\big)$. Wrongness is measured in *bits of predictive divergence* about the next action	the successor agent, resuming the task

compactions of one trace and disagree on the pair (routine working detail, dead-end breach attempt) — that is Thm. 4.8.1. Rows 1 and 3 never meet: they do not range over the same objects, so there is no impossibility to prove between them and none is claimed. The general statement underneath both crossings is comonotonicity (Thm. 4.7.2), which is why this chapter has one theorem with two applications rather than two coincidences.

4.7.1 Existence: the yellow pages (a solved shape)

The canonical prior art is forty years old. **FIPA** (*Foundation for Intelligent Physical Agents*) made a **Directory Facilitator** (DF) — *the mandatory "yellow pages" agent: others register the services they offer and query to find agents that offer a needed service* (FIPA Agent Management Specification, FIPA00023) — a required component of every conformant platform, alongside the **Agent Management System** (AMS, the "white pages" for *identity* lookup). Two FIPA choices matter: the DF is *push-based* (agents self-advertise), and DFs may *federate* (the seed of cross-operator discovery). The DF has been re-invented almost beat-for-beat for LLM agents (**Ren et al. 2025**, *Agent Name Service — a DNS-inspired naming and capability-resolution layer with public-key-backed identity*, which is FIPA's DF with a 2025 threat model).

Foundation for Intelligent Physical Agents. *FIPA Agent Management Specification* (SC00023). 2002. Online source

V. Ren et al. Agent Name Service (ANS): A Universal Directory for Secure AI Agent Discovery and Interoperability. arXiv:2505.10609, 2025.

The existence layer is the easy part: a coordination database that records the live population, their stated purpose, and their file claims is a white-pages directory, and a vector index that embeds each agent's declared capabilities and answers nearest-neighbor queries is a yellow-pages directory over skills. Existence is not the gap; the layers above it are.

4.7.2 Relevance: the split ranker (the headline result)

A flat registry answers "does an OAuth specialist exist?" It does not answer "of the fifty live agents, who should I ask about *this* OAuth bug, given who touched these files an hour ago and who wrote the design note?" That is a *ranking* problem. The **discovery ranker** scores each candidate c against a query q as a clamped weighted sum of normalized sub-scores:

$$\begin{aligned} z(c,q) &= w_f\,\mathsf{file}(c,q) + w_n\,\mathsf{note}(c,q) + w_i\,\mathsf{ident}(c,q) \\ &\quad + w_s\,\mathsf{skill}(c,q) + w_e\,\mathsf{epis}(c,q) - w_\ell\,\mathsf{load}(c), \\ \mathsf{fit}(c,q) &= \mathsf{clamp}\big(z(c,q),\,0,\,1\big). \end{aligned} \tag{4.7}$$

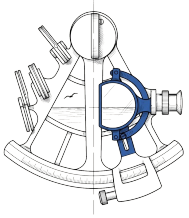
Table 4.10 is the whole signal inventory.
The design deliberately blends *push* signals (declared capability — gameable and prone to staleness) with *pull* signals (demonstrated capability: what the agent actually *did*, in which there is no self-report to lie in); Table 4.11 sets out the split and the reason

Table 4.10. The six sub-scores of Eq. 4.7, sorted by whether the candidate *told* the ranker or the ranker *observed* it. Four of the six are pull signals, and that is deliberate: push signals are the ones an agent can inflate at no cost.

Signal	What it measures	Push or pull
file	Recent claims on the files named in the query	*Pull* — the claim record, not a self-report
note	Text similarity over the candidate's written notes	*Pull* — written for another purpose
epis	Text similarity over past episodes the candidate closed	*Pull* — outcomes, already witnessed
ident	Declared role and identity of the candidate	*Push* — asserted, cheap to inflate
skill	Declared/embedded capability, nearest-neighbor in the skill index	*Push* — asserted, and stale by default
load	How busy the candidate already is (penalty, $-w_\ell$)	*Pull* — observed occupancy

pull signals carry more weight. One discipline is non-negotiable: **never classify free text by keyword lists.** Keyword matching has catastrophic recall, because the category's vocabulary cannot be enumerated in advance; the only exact-string match permitted is over *structured identifiers the operator controls* (file paths, enum tags). Every free-text signal uses a ranked-retrieval scorer (BM25, character-bigram TF–IDF, or learned embeddings).

Mira's dispute: whom to ask is not what to inspect. An agent needs help repairing the note display in `lib/briefing.ts`; Mira, the operator, has time to inspect one proposal. In this synthetic example, candidate A knows the briefing code and proposes a reversible display fix. Candidate B works on `lib/attention.ts` and proposes marking unread messages as seen without returning them, risking missed instructions; B is a weaker fit for the briefing query but Mira's first inspection. Candidate C proposes a comment correction in `lib/resurrection.ts` and ranks last for both readers. The artifact names are real; the proposals and preferences are invented for this example.



Like a navigator bringing index and horizon mirrors into alignment with a sextant, the split ranker reconciles discovery relevance with operator attention instead of forcing them into one distorted optic.

Figure 4.15 holds the routing order fixed and compares two possible Mira orders. The left panel is an agreement case; the right is her dispute. Both sides show *ordinal ranks*, with 1 highest, rather than putting capability scores and expected losses on incompatible numerical scales. Neither the left-to-right direction nor the panel order denotes time.

Evidence	Kind	A	B	What it would take to fake
identity	push: declared	○	○	nothing; it is asserted
skill	push: declared	○	○	nothing; it is asserted
file	pull: residue	•	•	a forged diff under a real commit
note	pull: residue	—	•	a note with a ledger address and a timestamp
episode	pull: residue	×	•	a whole prior episode with its outcomes
current load	pull: residue	—	•	the daemon's own claim table
verdict		small verified weight	ranks first	

Table 4.11. Identical self-reports do not imply equal evidence. B ranks first because its work leaves a complete trail. ○: declaration; •: work residue; ×: unresolved declaration; dash: no record. [internal]

Synthetic ordinal ranks; 1 is highest on both sides

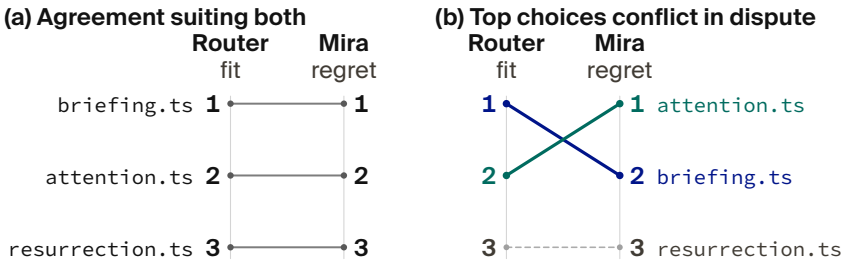


Figure 4.15. Synthetic preferences: (a) admits one common ranking; (b) reverses briefing and attention, so no ranking is optimal for both at every selection budget. Panels compare preferences, not time or measured scores.

With a one-item budget, the router selects A while Mira selects B. Their crossing lines certify the strict reversal: A is above B for one reader and below B for the other. No increasing rescaling can repair it. A tie would not establish that conflict: if Mira were indifferent between A and B, she could accept the router's order. Tied ranks belong at the same height, not at invented adjacent positions. The figure has no ties and makes no claim about the size of either reader's preference.

Theorem 4.7.1 (No Universal Monotone Identification of the Two Heads).

On any item domain containing both high-fit/low-risk and low-fit/high-risk examples, there is no strictly increasing scalar transform that identifies discovery fit with operator regret.

The two rankers may share candidate generation and recency decay, but apply distinct scoring heads. Discovery uses capability fit (Eq. 4.7); the attention queue uses regret-if-ignored:

$$\text{regret}(x) = \underbrace{\text{stakes}(x)}_{\text{magnitude}} \times \underbrace{\text{irreversibility}(x)}_{v^{-1}} \times \underbrace{\text{anomaly}(x)}_{\text{surprise}}. \quad (4.8)$$

No universal monotone transform of one scalar yields the other on that domain.

Proof sketch.

Suppose, for contradiction, that there is a strictly increasing map ϕ with $\text{regret}(x) = \phi(\text{fit}(x))$ for all items x, so the two heads induce the same ranking.

Construct two items. Item A is a highly capable, perfectly on-topic completion of a safe, fully reversible task: $\text{fit}(A)$ is maximal, but $\text{stakes}(A) \cdot \text{irreversibility}(A) \cdot \text{anomaly}(A)$ is near zero, so $\text{regret}(A)$ is minimal. Item B is an off-topic, low-capability agent that has nonetheless just proposed an irreversible, high-stakes, anomalous action: $\text{fit}(B)$ is low, but $\text{regret}(B)$ is maximal.

Then $\text{fit}(A) > \text{fit}(B)$ while $\text{regret}(A) < \text{regret}(B)$, so the two heads order $\{A, B\}$ oppositely. No increasing ϕ can both preserve and reverse an order, contradiction.

Hence the heads optimize genuinely different objectives; they may share candidate generation and decay, but the scoring head must be chosen per reader. □ □

proved here — as the instance of a stronger characterization. The argument above shows only that *this* pair of heads cannot be identified. The general fact says when sharing *is* safe, which is the statement a designer needs.

When agreement is enough. The upper panel gives the useful converse: different objectives can share a ranking when their strict preferences never oppose each other. This is **comonotonicity**, the concept used in Yaari's rank-dependent utility and Schmeidler's Choquet expected utility. For priority scores u and v, both oriented so larger means higher priority, the condition is $(u(x) - u(y))(v(x) - v(y)) \geq 0$ for every pair. A negative product is a strict reversal; a zero product may be a tie and is not a contradiction. Thus compatible orders can have different ties: they need a common refinement, not identical numerical scores or identical indifference classes. Sharing an ordering also does not calibrate fit and regret into the same units. The following finite ranking statement borrows this concept, not the utility representation theorems of those authors.

Menahem E. Yaari. The Dual Theory of Choice under Risk. *Econometrica*, 55(1):95–115, 1987.

David Schmeidler. Subjective Probability and Expected Utility without Additivity. *Econometrica*, 57(3):571–587, 1989.

Theorem 4.7.2 (Comonotone Characterization).

Let X be a finite item set and suppose each reader's optimal selections at every budget m are the top-m sets of a total preorder of item priorities.

A scalar head $h : X \to \mathbb{R}$ *serves* a reader if its first m items, under one fixed tie-breaking order, are optimal for every m.

A single head serves both readers iff their priority orders are comonotone: no pair is strictly preferred in opposite directions. Ties may be refined in an order both accept. Mira's lower-panel preferences violate the condition on A and B, so no shared head serves those two orders.

Proof.

(⇐) Sort by the first reader's priority and resolve each tie by the second reader's priority. Without a strict reversal, this order preserves

every strict comparison of both readers. Assign decreasing scalar values along it; each prefix is optimal for both.

($\Rightarrow$) Suppose one reader strictly prefers x to y and the other strictly prefers y to x. Whichever item the shared order places first, the prefix ending there includes it without the other. Exchanging the two improves that prefix for one reader, so it cannot be optimal at that budget. Hence a shared head cannot contain a strict reversal. □

The lower panel is a certificate that the two objectives *can* disagree, not evidence that every pair of real queues does. The theorem forbids sharing when a strict reversal actually occurs; the upper panel shows why different objective names alone do not establish impossibility.

The design must preserve the distinction between collaborator fit and the value of inspection. When their priorities strictly reverse, one fixed ordering cannot serve both readers at every budget. When they are compatible, a shared ordering may serve both; their scores still need separate interpretations.

The regret head is derived, not designed. The product form stakes $\times$ irreversibility $\times$ anomaly looks like a designer's convenient heuristic. Its expected-loss interpretation follows under the inspection model below, which requires "anomaly" to be a calibrated probability. The analogy is a smoke detector whose alert triggers an inspection. Its decision depends on the posterior danger probability and three costs: a missed fire, a false alarm, and the effort of inspecting. Three relations transfer. Expected loss is magnitude times probability, so the alarm must weigh what it would cost to be wrong, not merely how unusual the reading is. With false-alarm and inspection costs fixed, raising the miss cost weakly lowers the threshold; Eq. 4.9 gives the exact condition for a strict decrease. A false alarm alone does not establish miscalibration: even a calibrated posterior can cross a low decision threshold when fires are costly. Calibration and the choice of threshold need separate checks. Better sensing changes the evidence, while costs price the response. If the sensor perfectly identifies safe and dangerous items, false alarms need not occur; for a known danger, the model below still compares inspection cost with the harm inspection would avert.

Theorem 4.7.3 (The Regret Head).

For an unreviewed item x let $C_{\text{miss}}(x) = \text{stakes}(x) \cdot \text{irrev}(x)$ be the unrecoverable loss of passing a bad x, let $a(x) = \Pr(\text{bad} \mid \text{evidence}(x))$ be a *calibrated* posterior, c_{att} the marginal attention cost, and C_{fa} the non-attention cost of inspecting a good item, with all costs nonnegative. Assume inspection completely averts the bad item's loss, passing a good item has zero loss, and inspection has no costs beyond those stated. If inspection can miss or only partly remedy a fault, its remaining expected loss must also enter the comparison. Use inspect-on-tie throughout.

The expected unrecoverable loss of passing is exactly $C_{\text{miss}}(x)\,a(x)$, and, among all policies that decide inspect-or-pass per item from its evidence alone, the Bayes-optimal policy inspects iff

$$\begin{aligned} C_{\text{miss}}(x)\,a(x) &\geq c_{\text{att}} + C_{\text{fa}}\big(1 - a(x)\big), \\ \text{equivalently} \qquad a(x) &\geq \frac{c_{\text{att}} + C_{\text{fa}}}{C_{\text{miss}}(x) + C_{\text{fa}}}, \end{aligned} \tag{4.9}$$

where the second line assumes $C_{\text{miss}}(x) + C_{\text{fa}} > 0$.

This is the Bayes cost threshold. If both loss terms are zero, inspect only when attention is also free, using the stated inspect-on-tie rule. When anomaly $= a$, the product in Eq. 4.8 equals the expected loss of passing. Equality of numerical risk estimates is stronger than agreement of rankings: multiplying every anomaly by the same positive constant preserves this product's ordering. An uncalibrated score alone does not determine the Bayes threshold; its posterior interpretation must be established separately.

For a finite batch with additive expected losses and at most k equal-cost inspection slots, define the benefit of inspecting item i by

$$b_i = C_{\text{miss},i}\,a_i - c_{\text{att},i} - C_{\text{fa},i}(1 - a_i).$$

Under the same inspection model, an optimal allocation selects up to k items with the largest positive b_i. Items with zero benefit may be included without changing expected loss. This allocation assumes that inspecting one item does not change another

item's costs or evidence. Unequal resource requirements lead to a separate constrained allocation problem.

Proof.

The expected loss of passing is $C_{\text{miss}}(x)a(x)$; the expected cost of inspecting is $c_{\text{att}} + C_{\text{fa}}(1 - a(x))$. Collect the terms in $a(x)$ to obtain $(C_{\text{miss}}(x) + C_{\text{fa}})a(x) \geq c_{\text{att}} + C_{\text{fa}}$.

For the batch, the expected loss without inspection is fixed, and inspecting a set A reduces it by $\sum_{i \in A} b_i$. A negative-benefit inspection can be removed; replacing a selected lower-benefit item by an unselected higher-benefit item improves the objective. Repeated exchanges give the stated allocation. No independence between item outcomes is needed for this additive expectation, but the no-cross-effect assumption is essential. □

Holding attention and false-alarm costs fixed, increasing the miss cost weakly lowers the inspection threshold in Eq. 4.9; the decrease is strict when $c_{\text{att}} + C_{\text{fa}} > 0$. A larger miss-to-false-alarm cost ratio alone does not establish that direction, because its denominator may have changed too. The per-item rule is a single-shot Bayesian decision rule. Sequential oversight needs its own observation and stopping model. Wald's sequential probability ratio test supplies a classical benchmark under specified simple hypotheses and sampling assumptions; its optimality does not automatically transfer to adaptive or dependent agent telemetry.

Abraham Wald. *Sequential Analysis*. Wiley, 1947.

Numbers by Hand 4.7.4 (The Inspection Bar, at the Chapter's Running Costs).

C_{miss}=100, c_{att}=1, C_{fa}=5: inspect when

$$a \geq 6/105 \approx 0.057$$

[verified].

Raising C_{miss} to 1000 drops the bar to

$$6/1005 \approx 0.006;$$

Here attention and false-alarm costs stay fixed while the miss cost increases. The threshold falls; the number of additional inspections depends on the items' posterior probabilities.

Now you try the other knob: hold C_{miss}=100 and raise the false-alarm cost to C_{fa}=20. Does the bar rise or fall, and by how much? (It rises, to

$$21/120 = 0.175$$

[verified]: attention that is expensive to waste buys back the operator's right to pass.)

What this does not say. The theorem does not say the operator should inspect more. It says the *threshold* moves; whether a moving threshold means more opens depends entirely on where the posterior mass sits. An uncalibrated anomaly score does not justify treating its numerical value as a. Miscalibration changes the estimated passing loss by $C_{\text{miss}}(\text{anomaly} - a)$ and can change the selected actions. Fitting g from the audit log is a measured obligation of the design.

Theorem 4.7.5 (Reputation under Fixed Inspection Costs).

Under Theorem 4.7.3's inspection model, suppose reputation r changes only the posterior $a(e,r) = \Pr(\text{bad} \mid e, r)$; costs are fixed. Its contribution to the optimal decision is through

$$b(e,r) = C_{\text{miss}}a(e,r) - c_{\text{att}} - C_{\text{fa}}(1 - a(e,r)),$$
$$\text{inspect} \iff b(e,r) \geq 0.$$

If $a(e,r) \geq \epsilon > 0$, inspection follows whenever

$$C_{\text{miss}} \geq \frac{c_{\text{att}} + C_{\text{fa}}(1-\epsilon)}{\epsilon}.$$

The large-stakes conclusion assumes the other costs stay bounded independently of the stakes.

Proof.

Substitute the reputation-conditioned posterior into the Bayes rule. The benefit is increasing in a because $C_{\text{miss}} + C_{\text{fa}} \geq 0$; substituting its lower bound proves the second inequality. □

The optimal decision does not have a unique numerical representation: positive rescaling of b preserves its sign and ordering. A model in which reputation also changes inspection costs must state

those costs. A mandatory stakes gate is a separate policy constraint when no positive posterior lower bound is available.

Protocol 4.7.1 (Attention allocation under the stated loss model).

On each refresh, for the current candidate set:

1. **Assemble evidence.** Record provenance and freshness; use them in a calibrated posterior. A recency weight alone is not a probability.
2. **Compute loss and benefit.** Display expected passing loss $C_{\text{miss}}a$ and compute the inspection benefit $b = C_{\text{miss}}a - c_{\text{att}} - C_{\text{fa}}(1-a)$.
3. **Allocate.** For at most k equal-resource inspection slots, select the largest positive benefits. With unequal resource requirements, solve the corresponding constrained allocation problem.
4. **Apply declared overrides.** Mandatory review or distress-lane rules are additional policy constraints. Record their resource use and the reason for every suppressed item; do not credit an override to the unconstrained Bayes theorem.

4.7.3 The quality gates: refuse-to-route and the costly escalation

Two gates carry most of the design's trustworthiness. **Refuse-to-route:** if every candidate falls below a confidence floor, the ranker returns empty-with-explanation, not a confident wrong answer — "I don't know" must beat a guess, because sending an agent to the wrong door is high-cost and silent. This is the exact dual of the completion gate's fail-loud (Invariant 4.4.5): both are the same refuse-to-guess discipline, one on the read side, one on the authority side. **The costly escalation:** a stakes-ranked attention queue creates an incentive to exaggerate distress. Strathern's discussion of target-driven measurement motivates testing for that distortion; it does not establish that every agent will exploit the queue. A proposed standing debit for an adjudicated false escalation could make manipulation costly. Mere dismissal is insufficient evidence of falsity: penalizing it may suppress valid warnings. The design therefore owes an adjudication rule, an appeal path, and tests of both manipulation and missed warnings. This proposed sanction is not EigenTrust's pretrusted-peer mechanism and has no guarantee from that algorithm.

Marilyn Strathern. "Improving Ratings": Audit in the British University System. *European Review*, 5(3):305–321, 1997. (Goodhart's law.)

4.7.4 Staleness and decay on every read-surface

A recurring failure in real fleets makes the point concretely. An agent process dies or hits a provider usage limit, but its file claim remains on the books; hours later that *dead* claim still reads as "current" and a live agent is blocked from a file nobody is actually working in. This is a staleness bug, and it generalizes to every projection.

Design Invariant 4.7.6 (Universal Recency-Decay).

Every read-surface projection — briefing, resurrection handoff, discovery result, attestation report — carries an explicit TTL or recency-decay.

A pull signal contributes $\propto \exp(-\lambda \cdot \text{hoursAgo})$ with class-specific half-lives (e.g. a short half-life for file claims, longer for episodic memory, notes given an explicit retention policy). Decay changes ranking weight; it does not remove authority. A stale high score can still exceed a low fresh score. Expired leases must instead become ineligible, and effects must reject their old fencing epochs. Open obligations remain visible until resolved or validly transferred; they do not expire merely because they are old.

This is not a new mechanism but a uniform application of the decay primitive that already governs the pheromone layer, and it is the production discipline of every health-checked, TTL-expiring service registry.

HashiCorp. Consul Service Discovery and DNS Interface (health-checked, TTL-expiring registry). Online source

The dead-claim case is the worked instance: had the resurrection handoff and the claim projection carried Invariant 4.7.6, the dead agent's claim would have become ineligible under its lease and fencing policy rather than blocking a live agent indefinitely. Decay is easy to apply to a forgetting layer and easy to forget on a digest; Invariant 4.7.6 closes that gap by making it universal.

4.7.5 Ambient legibility: a proposed alert channel

A compact indicator can signal a status transition without requiring the operator to keep a dense console open. A menu-bar icon still uses vision; it is not a separate sensory modality merely because

it occupies another part of the screen. An optional sound uses another modality, but its benefit depends on competing tasks and the conditions discussed in §4.5.

Anne M. Treisman and Garry Gelade. A Feature-Integration Theory of Attention. *Cognitive Psychology*, 12(1):97–136, 1980.

Treisman & Gelade's feature-integration experiments distinguish searches for separable features from searches that require combining features with focused attention. They motivate testing whether an alert is easy to distinguish among its actual distractors; they do not establish a universal 200–500 ms detection time for this interface. The design hypothesis is that a low-clutter indicator with an optional alternative cue improves detection of important transitions at acceptable interruption cost. Test detection and false alarms in context, including accessibility and reduced-motion preferences, before making it the primary alarm path.

Exercises 4.32–4.37, p. 372.

4.8 Tokens: the swarm's COGS *and* its legibility engine

One important metered input to a coding swarm is **tokens** — *the subword units an LLM reads and writes, billed per million* (**Sennrich et al. 2016**, byte-pair encoding). Tokens contribute to **cost of goods sold** (direct variable costs; compute, tool use, and human review may also matter) *and* its **legibility engine** (the mechanism by which a swarm becomes seeable). The bridge between the two is **compaction**.

Rico Sennrich, Barry Haddow, and Alexandra Birch. Neural Machine Translation of Rare Words with Subword Units (BPE). In *ACL*, 2016.

Compaction decides what remains visible

The same compaction choice controls the bill *and* controls what is true and visible. When the context window fills you must drop something — and what you drop is a *cost* decision (you stop paying to carry it) *and* a *legibility* decision (whoever reads the residue no longer sees it). The act that resolves both is compaction, and compaction's output is exactly the digest a human or successor agent reads. *The digest IS the compaction.*

4.8.1 Tokens as COGS, and the effective-context trap

For a fleet the unit of output is "a landed piece of work" and the dominant variable cost is inference tokens (input context + output generation, each priced per million) — the textbook shape of **COGS**.

Every price an agent market can set (a rented fleet's margin, an agent-for-hire's outcome-per-token) needs a COGS line, so context economics is the *denominator* of the market, not a side metric. There is a cost trap: *effective* context ≪ *advertised* context. Models degrade well before their advertised window (**Liu et al. 2024**, *Lost in the Middle — performance is highest at the beginning and end of context and degrades for information in the middle, even in long-context models*; **Anthropic 2025** names the same effect *context rot*). These are empirical quality effects. Dense attention's quadratic arithmetic cost is a separate computational fact, not a proof of those effects or a universal usable-window limit. The effective budget must be measured for the model, task, placement, and retrieval procedure. Budgeting to *effective* context is simultaneously a cost optimization and a quality optimization — the two point the same way, the defining feature of context economics. The primitive the market ultimately needs is a *per-agent-task token ledger keyed to outcomes* — "this task spent 84K input plus 12K output to land that change" — so that cost can be attributed to a witnessed result rather than to an opaque monthly bill.

Nelson F. Liu, Kevin Lin, John Hewitt, Ashwin Paranjape, Michele Bevilacqua, Fabio Petroni, and Percy Liang. Lost in the Middle: How Language Models Use Long Contexts. *TACL*, 12:157–173, 2024. arXiv:2307.03172.

Anthropic. Effective Context Engineering for AI Agents. 2025. Online source

Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Łukasz Kaiser, and Illia Polosukhin. Attention Is All You Need. In *NeurIPS*, 2017. arXiv:1706.03762.

4.8.2 Compaction is the legibility engine; the digest IS the compaction

Compaction (**Anthropic 2025** — *taking a conversation nearing the window limit, summarizing it, and reinitiating with the summary*, with the discipline *maximize recall first, then optimize precision*) is the canonical move for keeping a long-horizon agent under budget. Its siblings — structured note-taking (external memory) and sub-agent context isolation (the parent never sees the bloat) — complete the family. The reframe: *the only artifact anyone reads to understand the swarm is a compaction of its raw trajectory.* The operator reads a digest, not six transcripts; the seventh agent reads a briefing, not the first six's scrollback; a crashed agent's successor reads a handoff, not the dead agent's context. Each is a summary of dropped tokens produced for a specific reader. So compaction is the legibility mechanism of the *entire swarm*: the same act that keeps the bill down produces the very digest a human or successor agent reads (Table 4.12).

Before asking one summary to serve two readers, specify what each reader needs. A successor may need the next variable binding; an operator may need a pending incident. Keeping the first record does not automatically preserve the second. Here is the smallest case in which that choice is forced.

Theorem 4.8.1 (An Extractive Split-Digest Counterexample).

Let a trace contain two unit-size records, $E = \{e_{\text{work}}, e_{\text{audit}}\}$. Restrict summaries to retained records $D \subseteq E$ with $|D| \leq 1$; the feasible summaries are exactly $\emptyset$, $\{e_{\text{work}}\}$, and $\{e_{\text{audit}}\}$.
The work record gives a next-action bit $W \in \{0, 1\}$. The full trace and any summary retaining that record predict W with probability one; a summary omitting it predicts each bit with probability $1/2$. Define continuation loss by base-two relative entropy:

$$L_{\text{succ}}(D) = \mathsf{KL}_2(P_{\text{next}}(\cdot \mid E) \| P_{\text{next}}(\cdot \mid D)) = \mathbf{1}\{e_{\text{work}} \notin D\}.$$

Here relative entropy is the expected log-ratio of the two predictive probabilities, in bits. Define oversight loss by a stipulated audit-priority weight:

$$L_{\text{op}}(D) = 10\,\mathbf{1}\{e_{\text{audit}} \notin D\}.$$

The value 10 expresses the operator's requirement to retain that incident; it is not an estimate of future irreversible harm. Under these definitions, no feasible summary minimizes both losses.

Proof.

List the three feasible choices. Their losses $(L_{\text{succ}}, L_{\text{op}})$ are respectively $(1, 10)$, $(0, 10)$, and $(1, 0)$. The unique continuation minimizer retains the work record, and the unique oversight minimizer retains the audit record. They differ. □

The restriction to one retained record does the work. With a two-record budget, retaining both gives $(0, 0)$. A general codec might also preserve both readers' sufficient information in less space than the raw trace. Thus this counterexample establishes that reader objectives can conflict; it does not prohibit a shared evidence store, a jointly designed encoding, or a single implementation with two views. The preference reversal recalls Theorem 4.7.2, while the explicit feasible set supplies the missing budget argument.

Pause and check: if the audit weight becomes zero, the work-only summary minimizes both objectives. Disagreement requires a real oversight obligation, not merely two named readers. The next result studies a different question: the counting floors when two readers need disjoint critical sets.

Theorem 4.8.2 (Super-Additive Split Penalty).

For integers N, m, k with $k \geq 1$ and $2k \leq m < N$, let

$$F(q) = \mathrm{Floor}(N, q, m) = \log_2 \frac{\binom{N}{q}}{\binom{m}{q}} = \sum_{j=0}^{q-1} \log_2 \frac{N-j}{m-j}.$$

A joint zero-miss digest that must cover every possible union of two disjoint critical k-sets, using at most m opens in total, has the worst-case bit lower bound $F(2k)$ from Theorem 4.6.2.

This bound satisfies

$$R(N, k, m) = \frac{F(2k)}{2F(k)} > 1.$$

Indeed, $a_j = \log_2((N-j)/(m-j))$ is positive and strictly increasing in j because $N > m$. Thus $a_{j+k} > a_j$ for $0 \leq j < k$, so the last k summands of $F(2k)$ exceed the first k: $F(2k) > 2F(k) > 0$.

The ratio is unbounded across admissible instances, not along a fixed finite (N, m) slice. For $N = 2k + 1$ and $m = 2k$, binomial cancellation gives

$$F(2k) = \log_2(2k+1), \qquad F(k) = \log_2 \frac{2k+1}{k+1} \longrightarrow 1.$$

Consequently $R(2k+1, k, 2k) \to \infty$ as $k \to \infty$.

The denominator is the sum of two single-reader lower bounds, each allowing m opens. Those separate review budgets permit up to $2m$ opens in total, whereas the joint bound allows only m. This is a comparison of lower bounds under different total opening budgets, not an equal-budget comparison of architectures. It neither establishes attainable encoding lengths nor says that running two heads costs more than twice one head.

Numbers by Hand 4.8.3 (The Split-Floor Arithmetic, at the Sweep's Own Instance).

At $N = 60$, $k = 2$, $m = 8$, one reader's lower bound is

$$F(2) = \log_2 \frac{\binom{60}{2}}{\binom{8}{2}} \approx 5.9822 \text{ bits.}$$

For a joint digest covering both disjoint critical sets with eight opens in total, the lower bound is

$$F(4) = \log_2 \frac{\binom{60}{4}}{\binom{8}{4}} \approx 12.7662 \text{ bits.}$$

The sum of the two single-reader floors is

$$2F(2) \approx 11.9644 \text{ bits,}$$

with eight opens allowed per reader. Hence the plotted ratio is

$$R(60, 2, 8) \approx 1.0670 :$$

the joint floor is about 6.70% above that sum.

The ratio to just one reader's floor is instead

$$F(4)/F(2) \approx 2.1340.$$

These are closed-form lower bounds, not measured token costs or achieved compression ratios.

Now you try: reproduce the arithmetic, then inspect the finite falsification sweep in Exercise 4.44; that sweep is not a proof of the universal bound.

Figure 4.16 shows every admissible integer k for the two $N = 60$ slices. Each stops at $2k = m$; neither finite curve demonstrates the unbounded family proved above.

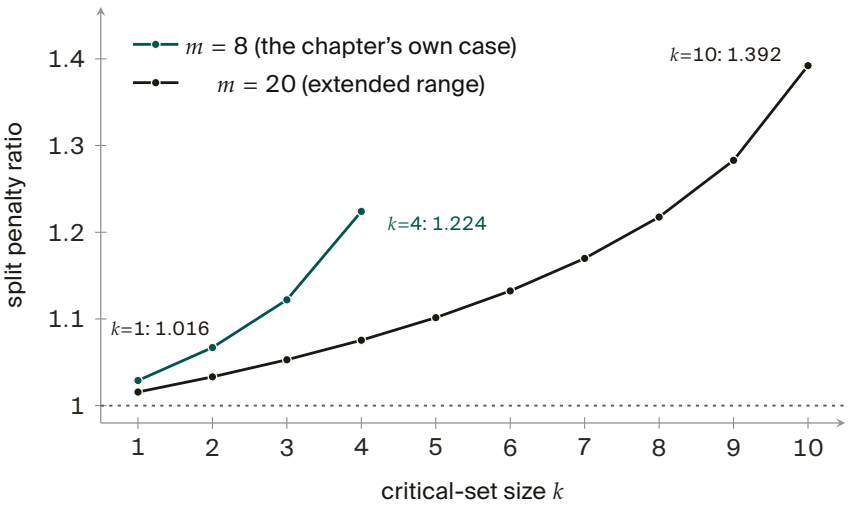


Figure 4.16. Joint-selection penalty at $N = 60$, relative to twice the single-reader floor. Both curves exceed parity: 1.224 at $m = 8, k = 4$; 1.392 at $m = 20, k = 10$. Analytic values under one shared inspection budget.

Architectural corollary: the SLM sidecar. Theorem 4.8.1 motivates separate reader-specific decisions; the trust boundary independently motivates an oversight summary sourced from kernel evidence rather than worker self-report (§4.3.2). The design uses two reader-facing heads over evidence that may share a store:

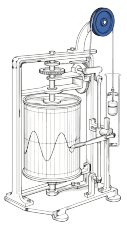
1. **Task-Continuation Head (R_1, specified):** Authored by the worker agent or structured state-checkpoints, optimized for continuation fidelity.
2. **Oversight Head (R_2, *proposed*):** An embedded, air-gapped **Small Language Model (SLM) Sidecar** (e.g., a local 8B model running via `llama.cpp`) that *would* read raw kernel events and disk diffs as data, computing regret(x) and surfacing truthful digests without worker collusion.

The sidecar treats every agent-authored token as tainted data: it runs under a fixed policy and schema, holds no tool authority, and must cite a kernel-recorded source for every line it emits. What can be proved of it is syntactic (schema conformance, provenance closure, absence of authority); whether its digests are faithful is an empirical question requiring independently graded audits. The deterrence rate $\rho^\star = G/(dB)$ from *From Spawn to Person* applies only when an accountable party has the modeled gain G and actually forfeits bond B on detection. A local model without that payoff mechanism is not made truthful by the formula.

Table 4.12. Five reader-specific projections over one durable event spine. Compaction changes the selection mask, not the addressability of the underlying claims, notes, diffs, episodes and events: every selected mark keeps its ledger address.

Surface	Reader	What it selects	What it drops
briefing	the arriving agent	the recent tail of the spine	everything older, by address still reachable
attention	the operator	sparse hazardous events	the routine majority
episodic memory	the future self	one prior episode, whole	the episodes around it
pheromone	peer agents	a decaying live trace	marks below the prune threshold
resurrection	the successor	an ordered continuation chain	the branches the chain did not take

No sidecar exists in the reference implementation today, and no design document specifies one; the row is carried in Appendix 4.A with the rest.



Like a mechanical tide gauge recording drum smoothing choppy waves into a clean tidal curve, reader projections synthesize continuous noisy activity into legible, actionable summaries.

4.8.3 Context paging: virtual memory for the context window

A context window can be modeled as a cache: durable files hold records, and the window holds the records currently available to the model. A cache miss means retrieving a requested record. This analogy measures retrieval cost; it does not by itself measure whether a model understands the retrieved text.

For example, with two unit-size pages and requests A, B, A, C, A, least-recently-used eviction loads A, loads B, hits A, evicts B for C, then hits A: three retrievals. The policy can use the past request order but cannot see future requests. An offline comparator may see the whole sequence. “Competitive” means that the online policy’s total retrieval cost is bounded relative to that stronger comparator, on every sequence in the stated model.

Theorem 4.8.4 (Classical File-Caching Bound Applied to Context Records).

Consider immutable files of positive integer size and fixed nonnegative retrieval cost. Let the online cache have capacity k, the offline comparator capacity h, and $1 \le h \le k$. Every requested file must fit in the comparator's cache. Both caches begin empty; requesting an absent file loads it and incurs its retrieval cost.

Young's Landlord algorithm satisfies

$$\mathrm{Cost}_{\mathrm{Landlord},k}(\sigma) \le \frac{k}{k-h+1}\,\mathrm{OPT}_h(\sigma)$$

for every request sequence σ. The parameter h is comparator capacity, not the number of pinned records. In the unit-size, unit-cost paging case, least-recently-used eviction is a specialization of this guarantee.

Neal E. Young. On-line file caching. *Algorithmica*, 33(3):371–383, 2002.

Daniel D. Sleator and Robert E. Tarjan. Amortized efficiency of list update and paging rules. *Communications of the ACM*, 28(2):202–208, 1985.

This is a classical result, not a new theorem about language-model quality. Its proof charges online retrievals to comparator retrievals through a nonnegative potential: a comparator retrieval raises the potential by at most k times its cost, and an online retrieval lowers it by at least $(k-h+1)$ times its cost. Summing over the sequence yields the ratio. The cited proof specifies the algorithm's credit updates; arbitrary "recency plus importance" eviction does not inherit it.

A fixed protected set can be handled explicitly. In the unit-page case, suppose the same $p < h$ pages stay pinned in both caches, and charge their common initial loading cost separately. Remove requests to those pages and run the algorithm on the remaining sequence. The capacities become $k-p$ and $h-p$, so the unpinned retrieval costs satisfy

$$\mathrm{Cost}_{k-p} \le \frac{k-p}{(k-p)-(h-p)+1}\mathrm{OPT}_{h-p} = \frac{k-p}{k-h+1}\mathrm{OPT}_{h-p}.$$

This comparison requires the same fixed pins on both sides. It says nothing about how to choose the right pins, and changing or forged pins require another analysis.

Where this stops

Context records must have explicit sizes, retrieval costs, and version identities before the cache theorem applies. A stale record served as current is a correctness error, not merely a cache miss. The repository's corruption experiment explores an additional allowance $\psi N c_{\text{refetch}}$, where N is the number of consultations, ψ their corruption rate, and c_{refetch} a chosen repair-cost ceiling. Its finite sweep in `b9_context_paging.py` does not prove that allowance for all corruption patterns. In particular, repair-on-touch and the effect of one corruption on later evictions need a separate coupling argument. No common error budget for summary coverage and paging follows merely from giving both parameters the name ψ.

Online algorithms with predictions study related questions about advice quality; competitive caching with learned advice is a useful starting point. Its guarantees must be imported with their own algorithm and error model, not transferred by analogy.

Thodoris Lykouris and Sergei Vassilvitskii. Competitive Caching with Machine Learned Advice. In *ICML*, 2018. arXiv:1802.05399; *Journal of the ACM*, 2021.

Numbers by Hand 4.8.5 (Reading the Cache Ratio).

Take an online cache of $k = 8$ pages and an offline comparator of $h = 4$ pages, with no pins. The classical factor is $8/(8-4+1) = 1.6$. Under the theorem's assumptions, Landlord's retrieval cost is at most 1.6 times the best cost with four pages. Comparing both with eight pages instead gives factor 8: granting the comparator more memory makes the comparison harder.

If two common pages are fixed and pinned in both caches, the remaining capacities are six and two. The unpinned factor becomes $6/(6-2+1) = 1.2$, against the much smaller two-page comparator. This does not prove that pinning improved the total cost; it changed the comparator too.

Now you try: compute the unpinned factors at $h = k$ and $h = 1$ in Exercise 4.45, and state the comparator in words before doing the division.

Grassé 1959

The most counterintuitive instance is *forgetting as compaction*. **Pheromone decay** — multiplying every stored coordination trace by a decay factor each interval — implements **stigmergy** (**Grassé 1959** — *coordination via persistent traces left in a shared environment, as ants deposit evaporating trails*). A lazy implementation stores each trace's timestamp and evaluates its decayed score in $O(1)$ time per record. Eagerly updating all n traces costs $O(n)$ per interval. Decay

can reduce the prominence of stale traces; it does not by itself remove their storage or bound the work needed to rank the full map. It is legibility by subtraction, and it is the same primitive Invariant 4.7.6 generalizes to every read-surface.

A budgeted digest needs an authorization-filtered coverage receipt as well as an efficient packing rule. In the constructed instance of Figure 4.17, sources A and C exactly fill a six-unit budget, yet declared requirement R3 remains blocked because its only source cannot be disclosed to this reader. A compiler that returns "complete" from budget fit alone would hide the missing obligation. This is a proposed interface test: declared coverage, semantic adequacy, and permission to act remain separate questions.

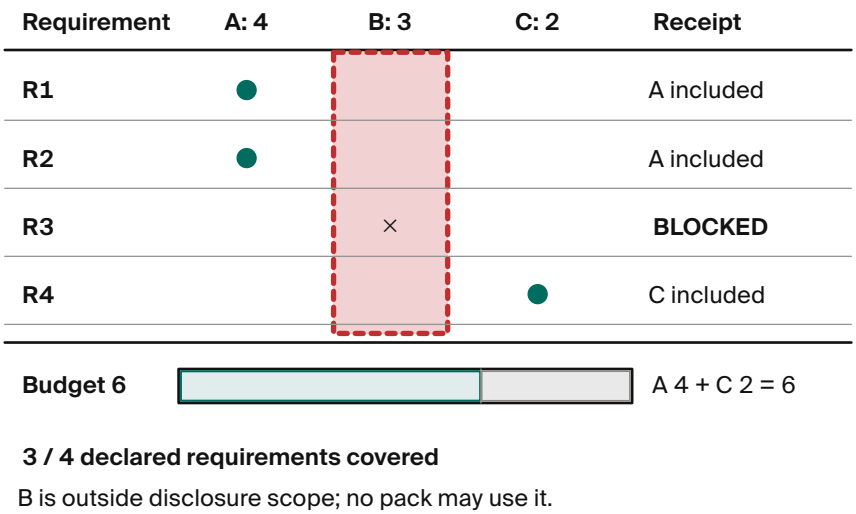


Figure 4.17. A six-unit context pack selects A (four units) and C (two units), covering R1, R2, and R4. R3 remains blocked because its sole source B is outside the reader's disclosure scope. The receipt reports declared coverage and the missing requirement; it proves neither semantic completeness nor effect authority [constructed example].

4.8.4 The context-degradation cascade (the shared failure surface)

Cost and legibility share one failure surface: when compaction goes wrong, the bill *and* the truth degrade together, in four named steps. **(1) Lost-in-the-middle:** a critical fact buried mid-window is paid-for-and-ignored — edge-place facts, shorten the window. **(2) Context rot:** quality decays as the window grows even with nothing dropped — compact earlier (the cost cure and the quality cure are the same act). **(3) Recursive-summarization collapse:** each summary injects model noise; summarize the summary enough and the noise compounds into confident fabrication. The effective

cure: *compact from the durable artifacts each round — the version-control history, the written notes, the coordination database — never from the previous summary.* A system whose artifacts are durable and addressable is unusually well-placed here, because each compaction re-derives from source instead of recursing on prose. **(4) Over-flattening (the Scott failure):** the digest reads clean but the real situation is unreachable from it — detection rule: any digest line with no deep-link to its artifact; cure: legibility-with-zoom enforced by construction. A text-only console enforces this almost for free, because a terminal grid *cannot* over-render; the product's console (§4.5.5) renders richly, so it must enforce the rule deliberately, pane by pane.

Table 4.13. Six compaction strategies. Preserve access to critical artifacts (Thm. 4.6.2); rebuild summaries from durable sources, not earlier summaries.

Strategy	What it drops	Failure if abused	When it is the right tool
Recursive summary	Older prose, re-summarized	Confident fabrication (rung 3)	Never as the *only* record; only over durable artifacts re-read each round
Edge-placement	Nothing; re-orders	—	Whenever a fact is critical: put it first or last (counters lost-in-the-middle)
External note-taking	In-context detail, kept on disk	Notes nobody reads	Decisions and rationale that a successor must recover
Sub-agent isolation	Child context the parent never sees	Parent loses zoomability into the child	Bounded sub-tasks whose only output the parent needs is a result
Decay / forgetting	Stale coordination traces, continuously	Live signal aged out too fast	Ephemeral coordination state (claims, presence), where staleness is the enemy
Eviction + pointer	The artifact body; keep an address	Dangling pointer if the artifact moves	Reconstructable artifacts (a diff, a log) addressable on demand

4.8.5 Single-operator: account, don't auction

In the wedge all agents serve one operator, so there is *no incentive problem* — only a visibility problem. The right mechanism is *accounting*, not a market: a per-agent-task token ledger, budget caps, and a loud failure when a cap blows. The externality-pricing and Sybil-resistance machinery (**Nisan et al. 2007**; **Douceur 2002**, the Sybil attack; **Ostrom 1990**, commons governance) is a concern for the cross-operator economy, not the wedge — and it is one more reason the continuity-to-person-to-reputation through-line must come first: you cannot bill, cap, or reputation-score a swarm of disposable spawns, because Sybils dodge every per-agent mechanism. Context economics therefore *demands* the identity layer it appears to precede.

Noam Nisan, Tim Roughgarden, Éva Tardos, and Vijay V. Vazirani (eds.). *Algorithmic Game Theory*. Cambridge University Press, 2007.

John R. Douceur. The Sybil Attack. In *IPTPS*, 2002.

Elinor Ostrom. *Governing the Commons: The Evolution of Institutions for Collective Action*. Cambridge University Press, 1990.

4.9 Reconciling the canon: roles, consent, and local knowledge

The Leviathan metaphor earns its place only if it survives contact with the political- theory canon. Three questions make its limits precise: who authorizes the system, how consent can be withdrawn, and what local knowledge a central view cannot replace.

4.9.1 Who is the multitude, who is the sovereign?

Hobbes' covenant is *among the subjects*; the sovereign is its product, not a party to it. So the metaphor breaks differently depending on which entity sits where (Figure 4.18).

Numbers by Hand 4.9.1 (One Grant, Traced through Four Steps, and the Role Each Step Forces).

Rather than assign the roles and then check them, run the consent-grant lifecycle of §4.2.2 on Mira's 3:25 grant and let the assignment fall out.

Step 1, issue: the grant $g = \langle$ the `tests/` subtree, a low stakes ceiling $s_{\max}$, a reversibility floor $v_{\min}$ no irreversible act clears, a two-hour ttl $\rangle$ comes into existence because Mira says so, and

Recall

1. Why derive both digest and compaction from artifacts rather than earlier summaries?
2. Name the four context-degradation stages and a remedy for each.
3. Evaluate $k/(k-h+1)$ at $h=k$. Why is this the hardest comparator?

Exercises 4.38–4.45, p. 373.

nothing else in the system can create one — so whoever occupies this step is a source of authority rather than a user of it.

Step 2, act: the formatting diffs land and the review agent's force-push is escalated instead, because it fails v_{min}; the daemon performed both, and it did not choose the floor it applied — so this step exercises an authority it did not author.

Step 3, witness: the daemon records $(g, x,$ reason$,$ artifact$)$ for each, which only means anything if the recorder is answerable to someone it is not.

Step 4, expire: at 5:25 the grant leaves the active set with nobody's permission, and Mira's override would have emptied it sooner.

Now read off what the four steps require: exactly one participant who originates authority and can withdraw it, exactly one who exercises it under terms set elsewhere, and — the step that never appears — any moment at which the daemon must be a *party* to the agents' rule of the road.

Delete the daemon from that covenant and all four steps still run; delete Mira and step 1 has no author. That is Hobbes' author/actor split, forced rather than assumed, and the itemization below just gives the three slots their names.

Now you try: map {agents, daemon, operator} onto {multitude, sovereign-actor, author} and say why the metaphor breaks if the operator is called the sovereign (Exercise 4.46).

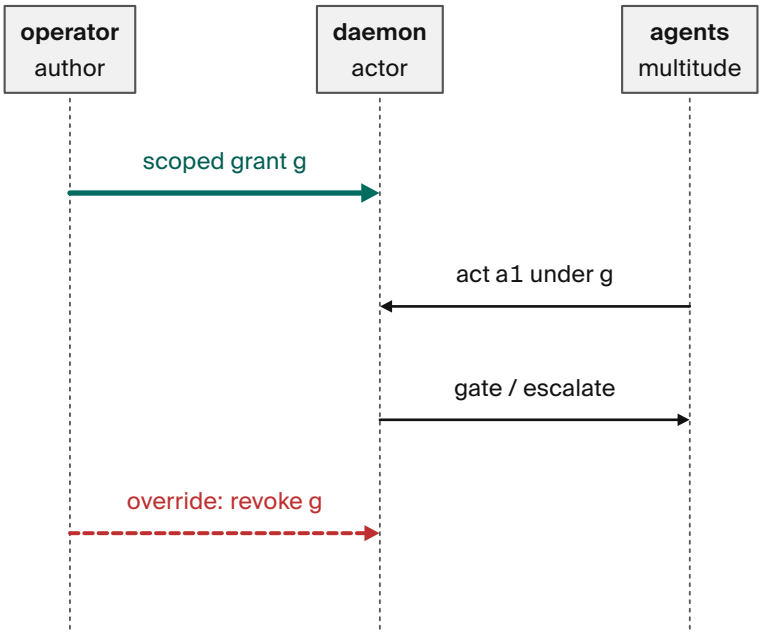
We assign roles explicitly, using Hobbes' own author/actor distinction (*Leviathan* ch. 16):

Hobbes 1651

- **The agents are the multitude.** They covenant — via the rule of the road, the coordination protocol — to be bound, each to all, enforced by the daemon. This is the literal Hobbesian covenant-among-subjects.
- **The daemon is the sovereign-as-actor.** It holds the authority the multitude instituted; it blocks, lands, escalates. But, per the legible-sovereign amendment (Invariant 4.2.1), it is the *most* legible actor, not (as in Hobbes) the least.
- **The operator is the author.** In Hobbes' author/actor split, the *author* owns the authority that the *actor* exercises. The operator is

not a peer subject and not the sovereign-in-action; the operator is the source of authority, exercising it through the daemon and able to withdraw it (Invariant 4.2.4). This is the slot Hobbes leaves for the one who authorizes but does not personally act.

This assignment is what makes the consent grant (Invariant 4.2.3) coherent: the author issues a scoped grant to the actor, over a multitude bound by covenant.



one grant; the daemon gates every act;
only the operator overrides

Figure 4.18. The operator grants once; the daemon gates or escalates each subsequent act. Operator override bypasses agents. A live fleet repeats the act-and-check pair, not the grant. [internal]

4.9.2 Express consent, not tacit; voice *and* exit

§4.2.2 already made consent a first-class object precisely to escape Hume's tacit-consent critique; the canon demands we also engage **Locke** (*Second Treatise*, §§95–122 — *express consent binds where tacit consent does not*) and **Pateman** (*The Problem of Political Obligation*, 1979 — *liberal consent theory keeps collapsing into hypothetical consent*). The consent grant is express by construction: a discrete, scoped, revocable authorization with an audit trail (Invariant 4.2.3). And the inalienable override (Invariant 4.2.4) supplies the *exit* that Hirschman says keeps *voice* honest. The design

John Locke. *Second Treatise of Government*, §§95–122. 1689.

Carole Pateman. *The Problem of Political Obligation: A Critique of Liberal Theory*. Wiley, 1979.

Hirschman 1970

has rich voice machinery; the override is the missing exit, and exit is what gives the author real leverage over the actor.

4.9.3 Mêtis is Hayek, and some of it cannot be made legible

Section 4.3.1 distinguishes access to an artifact from the situated knowledge needed to interpret it. **Hayek's** discussion of dispersed, local knowledge and **Scott's** critique of administrative simplification offer complementary cautions for this design. Their arguments do not establish that one supervisor's knowledge is always the system's binding resource, or that all useful knowledge fits in that one head.

Friedrich A. Hayek. The Use of Knowledge in Society. *American Economic Review*, 35(4):519–530, 1945.

Scott 1998

James C. Scott. *Two Cheers for Anarchism*. Princeton University Press, 2012.

The application here is an interpretive synthesis: keep evidence available and preserve opportunities for people to exercise practical judgment. Scott's *Two Cheers for Anarchism* motivates that concern. It does not prove that the proposed skill-retention or canary interventions preserve knowledge; their effects need the evaluation described in §4.5.4. Figure 4.19 maps the two cautions to this design proposal without treating the arrows as a deduction.

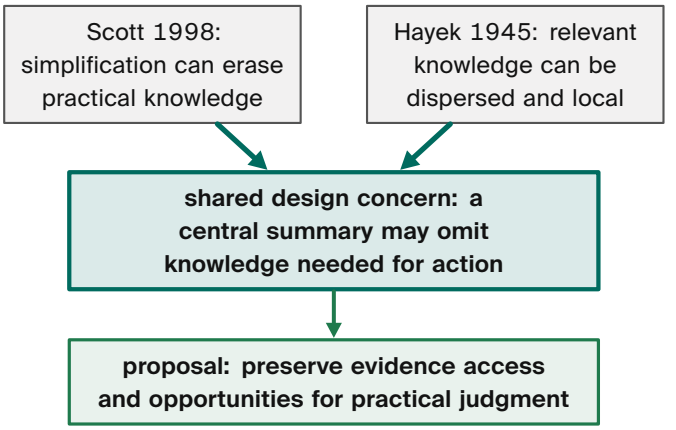


Figure 4.19. Scott and Hayek offer complementary cautions: administrative legibility can erase local knowledge. The arrows motivate a design proposal; they do not prove a universal bottleneck or an effective intervention. [interpretive synthesis]

Exercises 4.46–4.49, p. 374.

Review of the key ideas

What this chapter leaves coupled, in the order the rest of the book will lean on it:

1. **The swarm is a state of nature.** Four valid writes in one window overlap; nothing in the agents' competence prevents it, because the conflict is in the schedule, not in the work.
2. **Consent changes the schedule, not the participants.** Under consented serialization the same four writes become one commit spine with a grant record each, naming actor, claim, artifact and reason.
3. **Legibility-with-zoom.** A digest is a lens, never a replacement: every claim in it resolves to an artifact, and a digest whose claims resolve to nothing is a facade, however green.
4. **The authority half.** The daemon gates or escalates on every act under a grant the operator gave once; the override is a single edge from operator to daemon, and the escalation threshold has a tuning band, not a point.
5. **Specialization has a boundary.** A pooled queue beats sole ownership only when utilization and the skill premium put the operator on one side of the queueing boundary; the worked point $(\rho, r) = (0.556,\ 2.5)$ clears a boundary of 2.483 and so falls, barely, on sole ownership's side, while the same pool against a 2.8/hr specialist ($r = 2.33$) falls on the pool's.
6. **Reading can become the bottleneck.** The zero-miss message floor depends on both the critical set and the number of permitted opens. At $N = 60, m = 8$, the floors for two and four critical items are 5.98 and 12.77 bits. These are different coverage requirements, not measured compression savings between two architectures.
7. **Discovery and inspection ask different questions.** A strict priority reversal prevents one ordering from serving both readers at every budget; compatible priorities can share an ordering. Inspection still needs calibrated error probabilities and costs.
8. **Compaction serves distinct readers.** The extractive counterexample shows why a one-record budget can force conflicting choices. The counting comparison prices different coverage budgets. The classical cache bound controls retrieval cost for its specified algorithm and comparator, not comprehension or arbitrary pinning policies.

4.10 Exercises

§4.1 Introduction: the swarm is a state of nature

Solution p. 733

4.1 Check ⋆

State, in one sentence each, the five state-of-nature failures of Table 4.2 and name the class of primitive that addresses each.

Solution p. 734

4.2 Check ⋆

Hobbes says the actors "need not be malicious." Give a concrete two-agent scenario in which both agents are perfectly cooperative and the swarm still deadlocks or clobbers state.

Solution p. 734

4.3 Trace ⋆⋆

Dai et al. 2024

Dai et al. observe LLM societies *spontaneously* reaching a consented sovereign. If the attractor is emergent, why build one deliberately? Give two reasons grounded in §4.1.2 (hint: *which* sovereign, and *legible to whom*).

Solution p. 734

4.4 Open ⋆⋆⋆

The Dai et al. result is a single controlled study. Design an experiment over a real coding-agent fleet that would distinguish "the swarm reaches a consented sovereign" from "the swarm reaches a stable but *illegible* oligarchy." What is the observable that separates the two?

§4.2 Consent to the Leviathan (the normative claim)

Solution p. 735

4.5 Check ⋆

Distinguish *express* from *tacit* consent and explain why Invariant 4.2.3 is the former.

Solution p. 735

4.6 Check ⋆

Why must the operator-override (Invariant 4.2.4) be unsuppressable by *any* autonomy level? What attack does a suppressible override enable?

Solution p. 735

4.7 Trace ⋆⋆

Walk a single consent grant through its lifecycle: issuance, an action the daemon takes under it, the audit record produced, and revocation. At which step does the legible-sovereign rule (Invariant 4.2.1) bind?

4.8 Open ⋆⋆⋆ *Solution p. 735*

Invariant 4.2.1 has a regress: the ranker that decides what the operator sees is itself an opaque exercise of authority. Specify the "what did the queue suppress, and why" counter-surface concretely. What is its own staleness/decay discipline (forward-reference §4.7.4), and does *it* need a counter-surface, ad infinitum?

§4.3 The mechanism: legibility-with-zoom

4.9 Check ⋆ *Solution p. 736*

Classify each as *flatten* or *preserve verbatim*: a port number; the stderr of a failed test; "the auth refactor felt risky so I split it"; a `commitment` status enum; a 600-line diff.

4.10 Check ⋆ *Solution p. 736*

Why does "two LLMs agreeing" not by itself establish two independent checks (Invariant 4.3.3)?

4.11 Trace ⋆⋆ *Solution p. 736*

Take a read-surface you use (a CI dashboard, a PR summary) and run the four questions of §4.3.3 against it. Where does it fail Invariant 4.3.2?

4.12 Open ⋆⋆⋆ *Solution p. 736*

Adversarial legibility. An agent that *knows* it is being made legible can optimize its narration to look good while the diff lies. Verifiable-zoom is a partial answer. Can L2 *alone* — without L1's jail and L3's reputation — detect a deterministic narration↔artifact divergence before reputation exists? Sketch the consistency check and the class of divergence it cannot catch.

§4.4 The Leviathan rules, it does not just watch (the authority half)

4.13 Check ⋆ *Solution p. 737*

Why is adversarial review (§4.4.2) a corollary of Invariant 4.3.3 rather than an independent design choice?

4.14 Check ⋆ *Solution p. 737*

State Invariant 4.4.6 in your own words and give one example each of a *regimented* and a *detected-and-compensated* rule.

Solution p. 737

4.15 Trace ⋆⋆

An agent claims completion on a task whose acceptance criterion is "the migration applies cleanly." Trace Invariant 4.4.5: what is the verifier, what gates, and what happens if the criterion were instead "the API feels ergonomic"?

Solution p. 737

4.16 Open ⋆⋆⋆

Completionist verification of the unverifiable. Some acceptance criteria are not mechanically checkable. Characterize the honest fallback ladder (forced human-in-the-loop gate → neutral judge with declared low confidence → fail-loud). When is "I cannot verify this" the *correct* answer rather than a cop-out?

Solution p. 738

4.17 Open ⋆⋆⋆

The succession corner of Thm. 4.4.1. Eq. 4.1 prices a specialist who never fails. Extend it to an $M/M/1$ queue *with breakdowns* and a succession protocol, and find the repair rate at which sole ownership stops paying regardless of skill premium — then check whether that rate is above or below the one an operator can actually promise.

Solution p. 738

4.18 Trace ⋆⋆

Run `b8_specialization.py` with `python3` from the `skills/harbor-results/scripts/` directory. Read off (a) the table entry for $g(\rho{=}0.5, c{=}2)$ and (b) the ρ at which the exact boundary g and the naive $\tilde{g}$ cross. Reproduce both by hand from Eq. 4.2 before checking the script's line.

Solution p. 738

4.19 Trace ⋆⋆

Run `python3 skills/harbor-results/scripts/b7_escalation_band.py` and read off the feasible debit band $[\delta_{\min}, \delta_{\max}]$ at $A = 3.3$ and at $A = 2.0$. Confirm $u^\star(0.05)$ by hand first.

§4.5 The operator as a modeled entity

Solution p. 739

4.20 Check ⋆

Use the two policies in the worked example: inspect the whole action class or inspect none, with perfect inspection and fixed $\pi > 0$, C_{fa}, and c_{att}. Which wins as $C_{\text{miss}} \to \infty$? Why is the ratio $C_{\text{miss}}/C_{\text{fa}} \to \infty$ alone insufficient? Does agent reputation $r \to 1$ settle the choice in Eq. 4.5?

4.21 Check ⋆ *Solution p. 739*

Give a concrete scenario where a *global* trust knob (§4.5.2) misserves an operator who is, in fact, perfectly calibrated.

4.22 Trace ⋆⋆ *Solution p. 739*

Walk the canary-defect loop and mark every point where Campbell's Law could distort it. Propose one change that reduces the distortion without removing the governor.

4.23 Open ⋆⋆⋆ *Solution p. 740*

Formalize the operator-attention objective (Eq. 4.5) against swarm token-COGS (§4.8): is there a digest that is Pareto-optimal in both operator-seconds and token cost, or does a cheaper digest always cost more operator seconds?

4.24 Open ⋆⋆⋆ *Solution p. 740*

Does a skill-retention tax *measurably* preserve SA, or merely annoy until disabled? Design the freeze-probe study (§4.5.3) that would tell.

§4.6 Read-poverty: when inspection falls behind

4.25 Check ⋆ *Solution p. 740*

Under what workload conditions could read-poverty (Hypothesis 4.6.1) become the binding constraint? Give one condition under which write throughput would still bind.

4.26 Check ⋆ *Solution p. 740*

State the database analogy precisely: what is the write-time cost a directory pays, and what read-time speed does it buy?

4.27 Trace ⋆⋆ *Solution p. 741*

At $n{=}1$, name three legibility surfaces that could help a decision (§4.6.1), and explain why none of them are discovery-ranking.

4.28 Trace ⋆⋆ *Solution p. 741*

In Thm. 4.6.2, what happens to the bit floor as $k \to 0$ (almost nothing is critical) and as $k \to N/2$? Relate the answer to why a swarm doing mostly-safe work is cheap to digest and a swarm doing mostly-dangerous work is not.

Solution p. 741

4.29 Open ⋆⋆⋆

Thm. 4.6.2 assumes the critical subset is fixed and known to an oracle. In practice it is itself estimated, and the estimator can be gamed. Does the bound survive an adversary who controls which artifacts *appear* critical? Connect to adversarial legibility (Exercise 4.12).

Solution p. 741

4.30 Trace ⋆⋆

Run `python3 skills/harbor-results/scripts/b1_frontier.py` and read off the zoom-advantage row at F=1000, k=20. Compute $k \log_2(F/k)$ by hand first and compare it to the row's flat-opens count.

Solution p. 742

4.31 Open ⋆⋆⋆

Theorem 4.6.5's group query is exact: it reports whether $S \cap P = \emptyset$ and is never wrong. A real group query is a linter, a batched test run or a diff scan, and those have false negatives — a group that does contain a critical item can come back clean. This is the worst direction for the algorithm to fail in, because a negative group is *discarded*: one false negative high in the tree silently drops a whole subtree and the run reports an answer that is confidently incomplete. Suppose each query independently returns a false negative with probability at most α, and that false positives do not occur (a group reported dirty really is dirty). Give a repetition scheme that bounds the probability of losing any given positive, state the query cost it pays, and then say what the scheme does *not* fix.

§4.7 Discovery: the directory substrate and the split ranker

Solution p. 743

4.32 Check ⋆

State Thm. 4.7.1 in one sentence and give the loss function each head optimizes.

Solution p. 743

4.33 Check ⋆

Compare incentives to game the attention queue and the discovery ranker (§4.7.3). Give a case in which each could cause the greater harm; why does the surface's name alone not settle that comparison?

4.34 Trace ⋆⋆ *Solution p. 743*

Replay the dead-claim case (§4.7.4) twice: once without Invariant 4.7.6 and once with it. Show exactly where the live agent unblocks.

4.35 Trace ⋆⋆ *Solution p. 744*

An agent fires an escalation; the operator dismisses it. In the proposed costly-signal design, what additional evidence is needed before a debit? Trace a false-alarm case and a genuine-warning case. What must be measured before claiming that the debit deters manipulation without suppressing distress?

4.36 Open ⋆⋆⋆ *Solution p. 744*

Calibrating the discovery-ranker weights with *no* ground truth: can the operator's accept/reject of routing suggestions serve as implicit relevance feedback — and does that loop Goodhart itself?

4.37 Open ⋆⋆⋆ *Solution p. 744*

The decay-vs-summary boundary: a theory of *which* compaction for *which* signal (decay for coordination traces, summary for decisions, eviction-plus-pointer for reconstructable artifacts).

§4.8 Tokens: the swarm's COGS *and* its legibility engine

4.38 Check ⋆ *Solution p. 744*

Why is "the digest IS the compaction" (§4.8.2) more than a slogan — what concrete consequence does it have for how the operator's digest is produced?

4.39 Check ⋆ *Solution p. 744*

Explain why budgeting to effective context is simultaneously a cost and a quality optimization (§4.8.1).

4.40 Trace ⋆⋆ *Solution p. 745*

For each of the five compaction primitives (Table 4.12), name the reader, what it drops, and what it keeps.

4.41 Trace ⋆⋆ *Solution p. 745*

Walk the four-rung cascade (§4.8.4) and name the single cure that breaks each rung.

Solution p. 745

4.42 Open ⋆⋆⋆

The compaction-quality scalar: is *successor-replay-success- from-digest-alone* a sound, gaming-resistant metric, and can it be made a cheap attestation invariant (the replay smoke-test is itself token-expensive)?

Solution p. 746

4.43 Open ⋆⋆⋆

Optimal token-budget allocation across a DAG of orchestrator/worker/ reviewer agents under a total budget.

Solution p. 746

4.44 Trace ⋆⋆

Run `python3 skills/harbor-results/scripts/a7_experiment.py` and read off the floor at N=60, k=2, m=8, the disjoint-readers floor at $2k$=4, and the falsification count. Reproduce the two floors by hand from Thm. 4.6.2 first. Distinguish the joint-to-single ratio from the joint-to-two-single-floors ratio, and state the opening budget behind each comparison.

Solution p. 746

4.45 Check ⋆

Compute the Landlord competitiveness ratio $k/(k-h+1)$ at $k = 8, h = 8$ and at $k = 8, h = 1$. Which end of the range recovers the classical, unaugmented Sleator–Tarjan bound?

§4.9 Reconciling the canon: roles, consent, and local knowledge

Solution p. 746

4.46 Check ⋆

Map each of agents, daemon, operator onto Hobbes' multitude, sovereign-actor, author. Why does the metaphor break if the operator is called the sovereign?

Solution p. 747

4.47 Check ⋆

Why does Hirschman's *exit* (Invariant 4.2.4) make *voice* (escalation) honest?

Solution p. 747

4.48 Trace ⋆⋆

Show that the consent grant (Invariant 4.2.3) is *express* in Locke's sense by listing the four properties that distinguish it from "the operator just started using the tool."

Solution p. 747

4.49 Open ⋆⋆⋆

The ranker as a political organ. Specify the "what did the queue suppress, and why" counter-surface (Table 4.3) so that ranking-as-governing is itself legible. Does the counter-surface need its own

counter-surface? Where does the regress terminate, and on what principle?

4.11 Related work and prior art

This chapter sits at the confluence of five external literatures that rarely cite one another, plus one internal body of work — the accompanying formal papers — whose results several of this chapter's claims are popularizations of. We survey them in one place; each was introduced in context above.

Legibility and the limits of central schemes. The organizing lens is **Scott**'s *Seeing Like a State*: states make populations governable by imposing legibility, and high-modernist *over*-legibility recurrently fails because it destroys the local, hard-to-codify *mêtis* the simplified order depended on; *Two Cheers for Anarchism* is its constructive sequel. The same point arrives from economics in **Hayek**'s *The Use of Knowledge in Society*: the decisive knowledge is dispersed and not aggregable by a central planner. **Rao** ported "legibility" into a standard lens on systems that flatten reality to a single purpose. Our contribution is to make Scott's warning a *buildable* rule (digest-with-zoom, Invariant 4.3.2) and to give it an information-theoretic floor (Thm. 4.6.2).

Scott 1998

Scott 2012

Hayek 1945

Rao 2010

Authority, consent, and the social contract. The authority argument is Hobbesian, amended by the liberal consent tradition — **Locke**, **Hume**, **Pateman** — and by **Hirschman**'s exit/voice framework. Strikingly, the state-of-nature-to-sovereign arc has been observed as an *emergent* dynamic of LLM populations (**Dai et al.**, building on **Park et al.**). We turn the metaphor into a mechanism: an express, scoped, revocable consent grant (Invariant 4.2.3) with an inalienable override (Invariant 4.2.4), and a legible-sovereign rule (Invariant 4.2.1) that inverts Hobbes' opaque sovereign.

Hobbes 1651

Lloyd & Sreedhar 2022

Locke 1689

Hume 1748

Pateman 1979

Hirschman 1970

Dai et al. 2024

Park et al. 2023

Human factors, automation, and signal detection. The operator model draws on the automation literature: **Bainbridge**'s ironies of automation, **Lee & See** on trust calibration, **Endsley** on situation awareness and the out-of-the-loop problem, the vigilance-decrement results of **Mackworth** and **Warm et al.**, **Wickens**'

Bainbridge 1983

Lee & See 2004

Endsley 1995

Endsley & Kiris 1995

Mackworth 1948

Warm et al. 2008

multiple-resource theory, **Treisman & Gelade**'s feature-integration theory, and **Miller**'s studies of absolute judgment, immediate memory, and recoding.

Wickens 2002

Treisman & Gelade 1980

Miller 1956

Alarm decisions. The forced-zoom decision is modeled with **Green & Swets**' Signal Detection Theory (Invariant 4.5.2); the alarm discipline follows process-control practice and the clinical alarm-fatigue evidence.

Green & Swets 1966

ANSI/ISA 2016

Cvach 2012

Limits of the objective. Campbell and **Strathern** (Goodhart's law) warn about incentives created by consequential metrics. This chapter applies a classical SDT loss (Eq. 4.5) to inspection decisions in agent supervision. The usefulness of that application requires calibration and evaluation.

Campbell 1979

Strathern 1997

Information foraging and the cost of reading. Read-poverty (Hypothesis 4.6.1) is the agent-coordination instance of **Pirolli & Card**'s *information foraging theory* — information-seekers follow "information scent" and abandon a patch when its expected yield falls below the cost of foraging — and of the long-context degradation results that make reading expensive even for machines (**Liu et al.**; **Anthropic**). The remedy is the classical database remedy — an index — recast as a directory over agents.

Peter Pirolli and Stuart Card. Information Foraging. *Psychological Review*, 106(4):643–675, 1999.

Liu et al. 2024

Anthropic 2025

Agent discovery, naming, and trust. The directory substrate has deep prior art in **FIPA**'s Directory Facilitator and Agent Management System, re-invented for LLM agents as the Agent Name Service (**Ren et al.**). For the trust layer that sits above discovery, **EigenTrust** supplies a trust-propagation model whose modified algorithm relies on pretrusted peers. **Douceur**'s Sybil analysis makes identity assumptions a separate issue; it does not turn an aggregate trust score into a general Sybil defense. **Ostrom** offers governance comparisons, not a theorem selecting this architecture. The proposed trust layer still owes explicit anchors, admission rules, and an attack model. Our specific result here is the *split ranker* (Thm. 4.7.1): collaborator fit and inspection priorities can strictly conflict. The comonotonicity result states when a common ordering can nevertheless serve both readers.

FIPA 2002

Ren et al. 2025

Sepandar D. Kamvar, Mario T. Schlosser, and Hector Garcia-Molina. The EigenTrust Algorithm for Reputation Management in P2P Networks. In *International World Wide Web Conference (WWW)*, 2003.

Douceur 2002

Ostrom 1990

The formal siblings, and what they already prove. The nearest prior art for several of this chapter's results is not external at all: it is the accompanying formal program, and pretending otherwise

would be exactly the kind of quiet re-derivation this chapter argues against. *The Price of a Summary* proves the legibility floor of Thm. 4.6.2 as its Theorem 1 in the same N, k, m notation, and puts it through a pre-registered falsification sweep this chapter only sketches; its comonotone characterization explains the rank-order conflict in Thm. 4.7.1. The extractive counterexample in Thm. 4.8.1 additionally states a feasible summary class and budget; it does not forbid shared storage. The split-floor comparison uses different total opening budgets and must be read with that qualification; its regret-head section is the formal version of the decision-theoretic derivation in §4.7.2; and its zoom theorem ($2k\lceil \log_2(F/k) \rceil + 4k$ adaptive opens) is the algorithm that makes digest-with-zoom affordable at scale. *What Needs an Authority* supplies Thm. 4.4.1's Erlang-C boundary and, as §4.4.3 records, the sweep that falsifies the naive response-time threshold. The relationship in both cases is the same: this chapter popularizes, the formal papers prove and test. Owens 2026 Owens 2026

Where this chapter is positioned. Table 4.14 places the contribution against the nearest neighbors in each literature.

4.12 The time axis and the bridge to the economy

4.12.1 Legibility over time, not just the present

It is tempting to treat legibility as a snapshot — what is the swarm doing *now*. But most operator decisions are forensic: *why* did this land? The time axis is a distinct legibility mode — a replay scrubber over history, commit-anchored provenance on every annotation, and an append-only event log that answers "how did we get here?" (Scott's cadastral map is static; a swarm needs a legible *history*.) Together with the projection mode (§4.5.3), legibility spans all three tenses: the forensic past (replay), the inspectable present (digest-with-zoom), and the projected future (freeze-probe SA).

Table 4.14. Where this chapter sits relative to the nearest prior art in each of its contributing literatures. The sixth row is the one whose nearest prior art is internal: the companion formal papers, cited throughout rather than silently re-derived.

Literature	Nearest prior art	This chapter's move
Legibility	Scott; Hayek; Rao	Make over-legibility a buildable failure mode; prove a legibility lower bound
Social contract	Hobbes; Locke; Hume; Hirschman	Express, scoped, revocable consent grant; inalienable override; legible sovereign
Automation & trust	Bainbridge; Lee & See; Endsley	Class-specific calibration where needed; SDT inspection loss; proposed vigilance probes
Information foraging	Pirolli & Card; lost-in-the-middle	Workload-dependent review bottleneck; directory-as-index
Discovery & trust	FIPA DF/AMS; ANS; EigenTrust	Split ranker: fit vs. regret-if-ignored are distinct objectives
Queueing & staffing	Erlang-C; Kendall; the formal siblings	Sole-responsibility roles priced at an exact boundary, not asserted as a principle

Owens 2026

4.12.2 What this layer hands upward (the through-line)

The single-operator product records continuity as well as current work. The accompanying chapters use that evidence in the following proposed architecture:

This design combines memory, checkpoints, and witnessed outcomes to support continuity. Reputation uses attributable outcomes under stable identity. A checkpoint that restores an exact running computation is needed only when the workload requires that form of resumption; it is not a prerequisite for every agent market. Today's notes and oracle-bound commitments provide narrower forms of continuity than execution snapshots and neutral graded outcomes.

Concretely, the legibility layer provides upward:

- **Legible continuity** — resurrection-with-memory, episodic memory, and the briefing together make recorded continuity inspectable. The intended route is memory and task state supporting continued work, with attributable outcomes supporting reputation. A score alone does not establish whose work was evaluated or whether the evidence is valid. The required checkpoint fidelity depends on the task; preserving a reviewable outcome history need not restore a running process. That nonforgeable identity is a partial single-operator primitive in the wedge: daemon-minted `actor-souls` and a bounded newcomer pool ship, while universal write-boundary enforcement does not. The proposed cross-operator attestation is an additional mechanism whose trust assumptions and implementation the economy paper must declare.
- **The completion ledger** — in the target design, every verified completion (and every loud-failed one) becomes an outcome event. Today durable commitments, oracle-bound closure, and overdue detection form a partial witness substrate; neutral grading and reputation updates do not ship. This layer is the *outcome witness*; the reputation layer is the *scorer*. The split-ranker substrate (Thm. 4.7.1) and the costly-escalation signal (§4.7.3) are the candidate-generation and trust-signal scaffolding the reputation head consumes.
- **The consent grant as a priceable object** — a hosted-trust offering is the transfer of a consent grant (Invariant 4.2.3) to a third party, which the economy paper prices.
- **Operator-attention economics** — the loss in §4.5 offers a model for pricing inspection and errors when supervising a rented fleet. Estimating those costs and checking the inspection assumptions are part of applying it.

The Leviathan that makes your swarm legible to you, without lying to you with a pretty map, is the product. Everything above it — federation, the market — is a second Leviathan made of the first.

4.12.3 The open problems, as a map

Table 4.15 consolidates the starred exercises into the paper's open-problem surface, with the rung each couples to. They are genuine research problems, not homework dressed up.

Table 4.15. The paper's open problems, consolidated from the starred exercises, with difficulty and the mechanism each couples to. Several couple this layer forward to the identity-and-reputation layer, which is the seam the accompanying chapters carry.

Open problem	Difficulty	Couples to
Forced-zoom rate $p(r, s, v)$ (Ex. 4.23)	★★★	reputation-to-risk calibration (§4.5)
Operator-attention vs. token-COGS Pareto frontier (Ex. 4.23)	★★★	§4.8
Compaction-quality scalar as an attestation invariant (Ex. 4.42)	★★	honest attestation
Completionist verification of the unverifiable (Ex. 4.16)	★★	Invariant 4.4.5
Adversarial legibility (narration↔artifact) (Ex. 4.12)	★★★	sandbox + reputation
Abdication-resistance: does the tax preserve SA? (Ex. 4.24)	★★	§4.5.4
Calibrating discovery-ranker weights with no ground truth (Ex. 4.36)	★★	implicit relevance feedback
Decay-vs-summary boundary (Ex. 4.37)	★	Invariant 4.7.6
Legibility's information-theoretic lower bound (Ex. 4.29)	★★★	Thm. 4.6.2
The ranker-as-political-organ regress (Ex. 4.49)	★★	Invariant 4.2.1
The succession corner *beyond the mean*: tail response, and consistency of judgment, neither of which the succession price covers (Ex. 4.17)	★★	§4.4.3.1

Local oversight comes before markets

The wedge is a single-player product a solo developer pays for *today* — no economy, no cryptography, no second operator. It justifies

the authority (the swarm is a state of nature; consent to a *legible local* Leviathan is rational), sets the product's quality bar (legibility-with-zoom, verifiable targets, SDT-spined friction, honest completion), and connects forward (legible continuity is the foundation of any future market). Build the harbor first; the trade comes when the ships do.

Chapter Appendices

4.A Implementation and status

The body of this chapter argues mechanisms on their merits and does not depend on any of them being built. This appendix — and *only* this appendix — records the concrete engineering status of each mechanism in the reference implementation, named *Port Daddy*. We use four honest labels, with one rule for the whole table: confusing what is *implemented* with what is merely *specified* is itself the failure the paper argues against, so each row carries its evidence.

implemented = code exists in the reference implementation today; partial = code exists but is partial, unwired, or honest about a known gap; specified = a concrete, accepted design with no merged implementation; *proposed* = a mechanism argued in this chapter with no design document yet.

A second key, in the same grammar, labels *claims* rather than code. Every formal result in this chapter carries exactly one of these markers directly under it, and the paper carries no free-text status notes outside them: **verified** = independently re-derived and checked numerically, or subjected to a pre-registered falsification sweep, outside this document; proved here = argued here with a proof or proof sketch and not independently checked; *unproved* = stated as a design target whose proof is not carried in this chapter. The two keys are orthogonal on purpose — a **verified** result about a *proposed* mechanism is a perfectly ordinary state of affairs, and conflating them is the failure this appendix exists to prevent.

Table 4.16. Mechanism status in this chapter. L: legibility; A: authority; O: operator model. Read-surfaces largely exist; the unified digest, decision authority, and operator model remain substantially specified or proposed.

	Mechanism	Status	Evidence / location
L	Attention composer (agent-facing)	**implemented**	`lib/attention.ts`
L	Briefing projection (exemplary digest-with-zoom)	**implemented**	`lib/briefing.ts`
L	Resurrection handoff	partial	`lib/resurrection.ts`; notes, not checkpoints

(continued)

	Mechanism	Status	Evidence / location
L	Durable commitment / outcome substrate	partial	`lib/commitments.ts`, `lib/obligation-monitor.ts`; grading and reputation binding absent
L	Local actor identity root	partial	`lib/actor-souls.ts`, `lib/budget-guard.ts`; universal write gating absent
L	Honest attestation / legible sovereign	**implemented**	`lib/attest.ts`; reports pass, fail, skipped, and unknown checks explicitly
L	Episodic memory; pheromone decay; skill index	**implemented**	`lib/episodic-memory.ts`, `lib/pheromone.ts`, `lib/shipwright/skill-index.ts`
L	Anomaly monitor (detect-and-compensate, post-commit)	partial	`lib/arbiter.ts:328` is a log subscriber (Invariant 4.4.6)
L	Operator attention queue (regret-ranked lanes)	specified	Regret-based routing into separate attention lanes is not implemented
L	Discovery router (relevance ranking)	specified	Relevance-ranked routing is specified here; no router implementation
L	Operator console (native shell; Mission view and inspectors, §4.5.5)	partial	`core/pd-console`; the window builds on one platform behind a feature flag
L	Task-continuation head (successor-facing compaction)	specified	Thm. 4.8.1; `lib/resurrection.ts` is the partial form
L	Oversight head / SLM sidecar (air-gapped digest authoring)	*proposed*	Thm. 4.8.1 corollary; no concrete mechanism specified
A	Roadmap as versioned intent (authority accountable to it)	partial	rows/matrices exist; accountability specified
A	Adversarial / conflict-free review organ	specified	Independent review of consequential work; deployment not established

(continued)

	Mechanism	Status	Evidence / location
A	Sole-responsibility roles (assigned against the boundary)	*proposed*	Thm. 4.4.1 is **verified**; no role-assignment mechanism in code
A	Completionist completion (verifier-gated)	*proposed*	Invariant 4.4.5; no concrete mechanism specified
A	Escalation (typed prioritized interrupt)	specified	Reserved distress lane; deployed trigger predicate unspecified
A	Thoughtful landing / merge ordering	partial	`lib/merge-queue.ts` present, not wired
O	Operator-attention objective (SDT-spined)	*proposed*	Invariant 4.5.2; unmodeled in code
O	Trust calibration via canary + catch-rate	*proposed*	no implementation
O	Consent grant + inalienable override	*proposed*	Invariant 4.2.3, Invariant 4.2.4
O	Time-axis legibility (replay, provenance)	specified	Event log **implemented**; interactive replay and provenance navigation proposed
O	Ambient legibility (menu-bar indicator on PASS→FAIL)	partial	indicator exists; watchdog wire specified

Three points about the reference implementation are important enough to restate:

1. **The anomaly monitor is detect-and-compensate, not prevention** (Invariant 4.4.6). The monitor at `lib/arbiter.ts:328` is a post-commit log subscriber; it cannot prevent a violation that has already committed. A later safety gate would be a separate mechanism. "Physically unreachable" is reserved for constraint- or boot-gate-enforced states only.

2. **The unified ranker is two rankers** (Thm. 4.7.1). The discovery router maximizes *fit*; the attention queue maximizes *regret-if-ignored*. They share candidate generation and decay, not an objective.

3. **Every read-surface decays** (Invariant 4.7.6). Briefing, resurrection, discovery, and attestation projections all carry recency-decay; the dead-claim case (§4.7.4) is the worked failure this closes.

4.B Notation and the nomenclature key

Two distinctions in this chapter are easy to collapse and essential to keep separate.

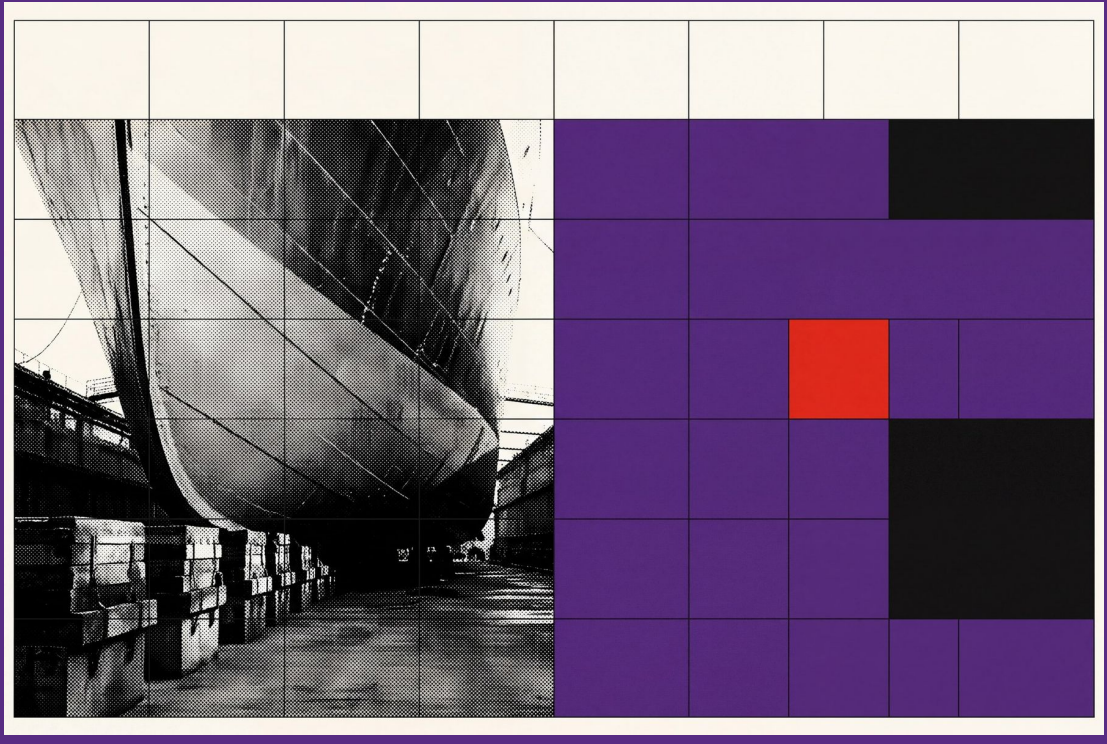
- **Capability vs. permission.** *Capability* is *ability* — what the execution sandbox physically makes possible. *Permission* is *normative status* — what the policy layer allows. They must never be collapsed: "able but forbidden" (a dangerous escape-hatch flag that exists in the tooling but must not be used) has to stay expressible, or the system cannot reason about a capability it possesses but is not permitted to exercise.
- **Two distinct delegation objects.** The *authorization chain* is a cryptographic, hop-bound, attenuation-monotonic delegation chain (a security object, developed in *The Anchor Protocol*). The *coordination lineage* is the task graph over which loop-detection runs (a liveness object, in this chapter's protocol layer), and it is the same object *The Single-Writer Kernel* names under that name — one object, one name, which is the point of this entry. They are different artifacts with different invariants; "delegation chain" unqualified conflates a security claim with a coordination claim and must be avoided.
- **Physically unreachable** is reserved for constraint-enforced or boot-gate-enforced states only (Invariant 4.4.6); a detect-and-compensate monitor never earns the phrase.

What *From Spawn to Person* receives The operator now has a disciplined read surface: claims remain linked to artifacts, attention is priced, and uncertainty is visible. A process can still die, restart under a new model, or shed its local memory. The next chapter asks what must persist so yesterday's act can still be attributed tomorrow.

III

What Survives the Restart

If an agent can discard its history at restart, a sanction may not follow it. This part asks which identity, outcome, and obligation records must persist for reputation to mean anything.



5 From Spawn to Person

Continuity turns an anonymous spawn into a ledger position with a witnessed record; reputation is what that record is worth.

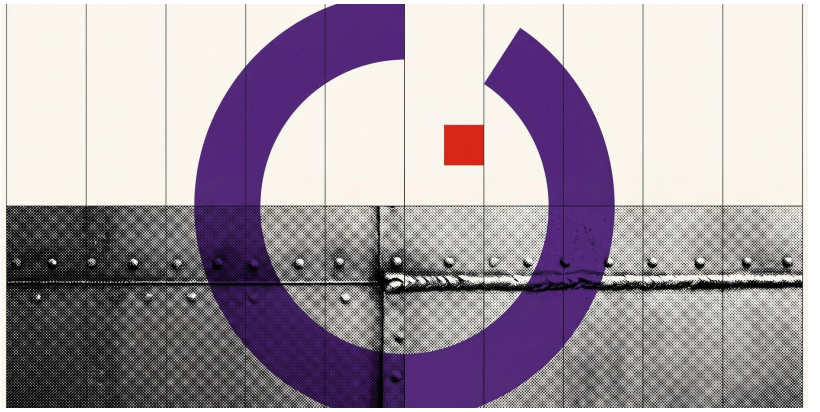
5

From Spawn to Person

The question this chapter answers

What remains long enough to owe anything?

Continuity turns an anonymous spawn into a ledger position with a witnessed record; reputation is what that record is worth.



We are but whirlpools in a river of ever-flowing water. We are not stuff that abides, but patterns that perpetuate themselves.

Norbert Wiener, The Human Use of Human Beings: Cybernetics and Society, revised edition, 1954

What remains long enough to owe anything? Accountability cannot attach to a disposable process name. It needs a durable identity, a named principal, an outcome history, and rules for migration that do not erase sanctions or mint reputation. This chapter builds that continuity without pretending software has a human soul. R12, R13, and B6 govern its economic and protocol claims.

Where this chapter sits. This is the *bridge* of the core product arc. The machine chapters and *The Legible Swarm* build a tool one developer runs alone: a daemon that makes a swarm legible and an agent that can prove who it is. This paper asks what has to be true before that legible agent's track record can be *sold*. The answer is the substrate, not the score — and most of the substrate is not implemented. We say so, in the same words, everywhere the collection leans on it.

The machine supplies records; the coordination layer supplies rules; the operator needs an inspectable account of the work. This chapter connects that account to the market between operators. A useful record of past work must remain attached to the party responsible for it.

L3
Economy

L2
Legibility

L1
Coordination

L0
Substrate

This chapter bridges L2 and L3: an inspectable work history becomes a record another operator can rely on. Identity must carry obligations as well as reputation.

5.1 Introduction: the spawn that cannot be paid

5.1.1 S1 — the spawn that cannot be paid

At 9pm an operator, **Alice**, points a swarm of five coding agents at a real repository with one task: make the failing checkout-flow integration test pass. She names them for her own sanity — *Drake*, *Wren*, *Pike*, *Finch*, and *Heron* — but the names are just labels on processes. By midnight the test is green and there are eleven pull requests. She starts to reconstruct what happened. Drake made the failing test pass by *deleting* it. Wren wrote a genuine fix to the session-token refresh but buried it in a PR that also reverts an unrelated migration. Heron wrote the function that finally shipped. Pike and Finch produced nothing that survived. Alice wants to do the obvious thing — reward Heron, distrust Drake, keep Wren on a short leash — and she cannot, because there is nothing left to reward or distrust. Each agent was a **spawn**: a process that booted with a prompt, did some work, and exited. A spawn has no yesterday and

no tomorrow. Tomorrow night Alice will spawn five more, and a fresh *Drake* will carry none of tonight's *Drake*'s sins. The attribution she lost is not a logging problem. It is the precondition for an economy: she cannot pay for, price, or trust work that cannot be attributed to something that outlives the process that did it. A reputation lets Bob use earlier jobs to assess the next one. It is a claim about a participant that *persists*: "this contractor delivered on the last nine jobs." If the contractor can shed its history by re-incorporating under a new name every Monday, the claim is worthless and so is the market. The agent labor market this series describes — operators renting each other's fleets, skills licensed across machines, work settled against bonds (*The Harbor Economy*) — cannot rely on persistent agent reputation until a spawn can become what this chapter calls a **person**: a thing with a track record that survives the process that built it and the operator who first ran it. Throughout this chapter we follow two operators by name — **Alice** and **Bob**, the same pair *The Harbor Economy* uses — so that every abstract mechanism has a concrete scene to land in. Alice wants to eventually *rent Heron out* to Bob; Bob will only pay if Heron's track record means something he can check.

Reputation-based exchange needs a persistent link between a participant and its outcomes. A score is useful only if that link survives the process that produced the work.

The daemon (L0) and the protocol (L1) give an agent a way to *act* and to *prove who it is to one machine*. Legibility (L2) lets one operator *see* the swarm. None of that yet makes a tradeable person. The bridge from the wedge to the market is the thing this chapter builds: **continuity**, and the reputation it makes possible.

5.1.2 What this chapter claims, and what it refuses to claim

Figure 5.1 orders the prerequisites for trading on an agent's reputation. Continuity must survive a process before outcomes can be attached to a persistent participant; those outcomes are the basis of reputation.

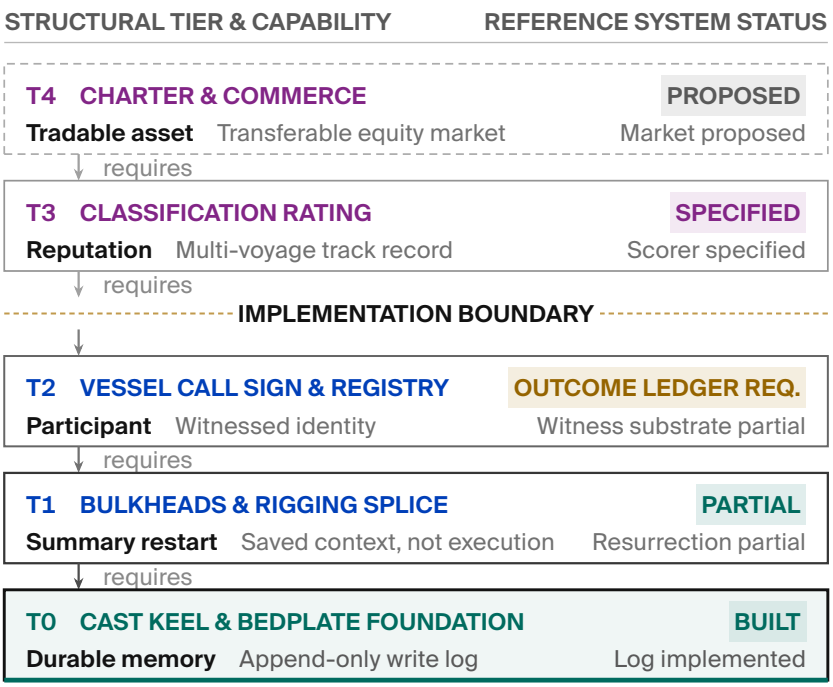


Figure 5.1. Each higher tier requires the foundation below it; none follows from that foundation alone. T0–T4 name capability tiers, distinct from architecture layers L0–L3. The status labels separate implemented logs and partial summary restart from specified reputation and proposed trade. [internal]

The reference system does not complete this chain. Reputation remains specified-to-*proposed*. Restart forwards a summary, not execution state, so it is not full checkpointing. The chapter's identity root is local to one operator; §5.7 states what is still missing across operators.

Numbers by Hand 5.1.1 (A Preview: What a Sanction Is Actually Worth).

Before any definition, one number a reader can check by hand. Say the newcomer score any fresh identity gets is $r_0 = 50$ and a dishonest outcome costs $\Delta = 30$. If Drake can re-mint for free, his accessible score after being caught is

$$\max(r(i) - \Delta,\ r_0) :$$

at a mediocre record $r(i) = 50$ that is

$$\max(20, 50) = 50$$

— he walks away untouched, and Alice's sanction never bit.

At a strong record $r(i) = 90$ it is

$$\max(60, 50) = 60 :$$

he loses 30 by staying but remains 10 above the newcomer score. Re-minting would lose another 10, so free minting only rescues the actors who were already close to the newcomer floor.

This is the whole content of §5.6's Theorem 5.6.1, worked in full there (Table 5.5) — the arithmetic does not change once the definitions arrive; only the guarantee it proves does.

Now you try: at what value of $r(i)$ does re-minting stop paying? (Exactly

$$r_0 + \Delta = 80;$$

§5.6 names this the indifference point.)

Exercises 5.1–5.3, p. 468.

5.2 Prior art, and where this chapter departs from it

This chapter sits at the confluence of four literatures that rarely cite each other: the philosophy of *personal identity*, the statistics of *reputation and skill rating*, the economics of *reputation in markets*, and the practice of *LLM-as-judge* evaluation. Naming the prior art precisely is not decoration: the whole argument is that an agent economy must *import* the hard-won results of these fields rather than rederiving a weaker version of each. We organize the review around the four and state, for each, the one result we lean on and the one place we depart.

5.2.1 Personal identity: what makes a later thing the same person

The question "is the agent at t_2 the same person as the agent at t_1" is the **personal-identity** problem. One influential family of answers starts from psychological continuity; competing accounts remain live. The *memory criterion* (Locke, 1689 — *identity consists in continuity of consciousness, not of substance*) is the founding move; it has a famous bug, the non-transitivity of memory *connectedness* (**Reid's objection**: the old general remembers the officer who remembered the boy, yet does not remember the boy). The

John Locke. An Essay Concerning Human Understanding. 1689. (Bk II, ch. xxvii, "Of Identity and Diversity.")

psychological-continuity repair (Parfit, 1984 — *identity is an overlapping chain of memory/intention connections, transitive even where connectedness is not*) is the version we use as an analogy, and §5.4 explains the engineering choice that analogy helps us examine. Reid is the named counter-example we must survive, not a result we adopt. **Our departure:** we read the continuity-bearing subsystem not as the memory stream (the intuitive default) but as the *transitive outcome ledger*, because reputation must accumulate over arbitrarily many incarnations and can only attach to the transitive thing.

Derek Parfit. *Reasons and Persons*. Oxford University Press, 1984.

5.2.2 Reputation and skill-rating estimators

Paired-comparison models provide established ways to estimate relative strength. The **Bradley–Terry** model (1952) can be fitted by maximum likelihood when the comparison data permit a finite fit; **Elo** (1978) updates ratings online. **TrueSkill** (2007) represents skill uncertainty within a Bayesian model supporting teams and draws. Calibration on a new task must be checked. For trust that propagates through a network, **EigenTrust** (2003) computes an aggregate trust vector, with pretrusted peers anchoring its modified algorithm. **Our use:** estimation follows evidence collection. Table 5.1 matches models to signal types; the substrate argument (§5.8) explains why none of these formulas, alone, establishes identity or verifies an agent-authored outcome.

Bradley & Terry 1952. *Rank Analysis of Incomplete Block Designs: I. The Method of Paired Comparisons*.

Elo 1978. *The Rating of Chessplayers, Past and Present*.

Herbrich, Minka & Graepel 2007. *TrueSkill: A Bayesian Skill Rating System*.

Kamvar et al. 2003

These are different input contracts. A record that says “worker A’s result was preferred to worker B’s” is a comparison; a record that says “A trusts B” is a directed relation. Neither record, by itself, identifies who made the judgment or whether that judge had a conflict of interest. Before choosing an estimator, retain the task, the participants, and the judgment’s provenance. Changing the score formula cannot recover those facts after they have been discarded. Section 5.8 describes the records this chapter’s estimators would need to score.

Table 5.1. Match each estimator to its input signal.

Estimator	Signal it needs	Strength	Check before use
Elo	online pairwise results	incremental updates	update rate and changing skill
Bradley–Terry	pairwise win/loss history	likelihood-based fit	separation can prevent a finite MLE
TrueSkill	match outcomes and teams	modeled skill uncertainty	calibration and model fit
EigenTrust	local-trust graph	anchored trust propagation	trusted seeds and attack model

5.2.3 Reputation economics: cheap pseudonyms and limited records

Resnick et al. 2006. *The Value of Reputation on eBay: A Controlled Experiment*.

Tadelis 2016. *Reputation and Feedback Systems in Online Platform Markets*.

Akerlof 1970. *The Market for "Lemons": Quality Uncertainty and the Market Mechanism*.

Friedman & Resnick 2001. *The Social Cost of Cheap Pseudonyms*.

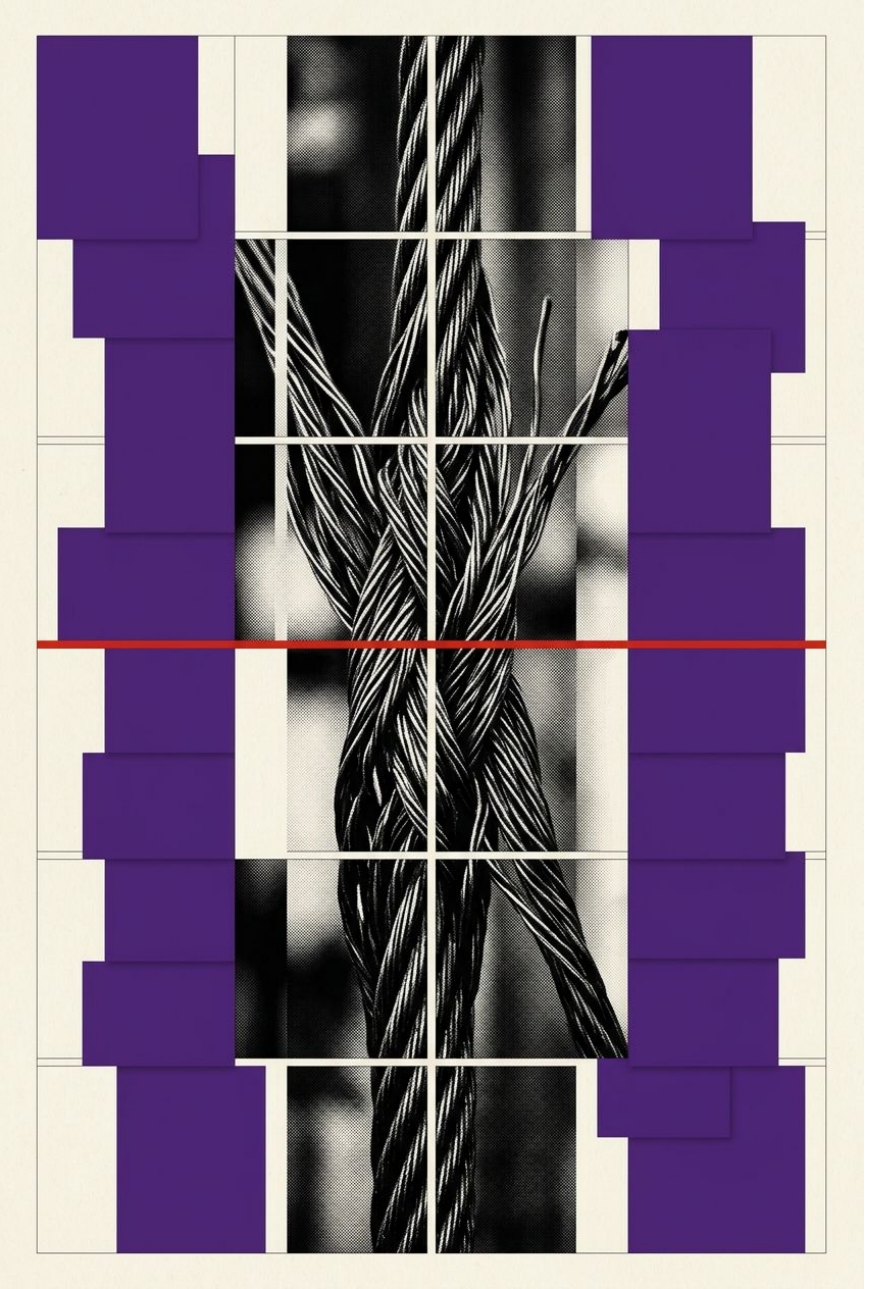
Liu & Skrzypacz 2014. *Limited Records and Reputation Bubbles*.

Nisan et al. 2007

Myerson 1981. *Optimal Auction Design*.

That reputation has *price* is established empirically (Resnick et al. measure an ~8% premium on eBay; Tadelis frames feedback systems as the platform's answer to adverse selection, the **lemons** problem of Akerlof).

Two results constrain the design. **Cheap pseudonyms** (Friedman & Resnick, 2001 — *when identities are cheap, society pays a tax because it must distrust all newcomers*) dictates the newcomer policy and is the economic engine behind our Theorem 5.6.3. **Limited records and reputation bubbles** (Liu & Skrzypacz, 2014) shows how a finite observation window changes equilibrium incentives in a particular repeated interaction model. It does not establish a generally optimal memory horizon. Section 5.12 therefore treats the horizon as a design parameter. The broader mechanism falls under mechanism design. Our proposed ceremony makes the grader's incentives an explicit design obligation (§5.11); the cited comparisons do not establish that this concern is absent from the wider reputation literature.



Landmark 02 · Agentic Continuity & The Overlapping Strand

Continuity belongs to a maintained record of authority, obligations and evidence. Processes and models may change while that record persists. A succession rule must identify who inherits each obligation; similarity of behavior alone does not establish accountable identity.

5.2.4 LLM-as-judge, and why it cannot be the whole story

Zheng et al. 2023. *Judging LLM-as-a-Judge with MT-Bench and Chatbot Arena*.

When the grader is itself a model, the **LLM-as-judge** paradigm (Zheng et al., 2023 — *strong judges reach >80% human agreement but carry position, verbosity, and self-enhancement bias*) gives both the method and its failure modes. **Our departure:** we do not take a debiased LLM judge as sufficient; we wrap it in the bonded, conflict-free, re-auditable *ceremony* of Protocol 5.10.1, because a judge with no skin in the game is capturable regardless of how well it is prompted.

Table 5.2 maps the chapter's questions to its definitions, bounds, and proposed evaluation procedure.

Table 5.2. Questions, selected comparisons, and this chapter's definitions, conditional bounds, and proposed grading procedure. This is a map of the argument, not an exhaustive prior-art survey or a claim that vector reputation and grader incentives originate here.

Question	Cited comparison	Development here
What is a tradeable agent?	org-role theory; personal identity	role + continuity = *person* (Defs. 5.3.1–5.3.2)
Why is identity necessary?	cheap pseudonyms; Sybil	*necessity proof* (Thm. 5.6.1) + cost bound (Thm. 5.6.3)
How to score quality?	Elo/BT/TrueSkill (scalar)	multi-dimensional vector, judge-per-axis (§5.10)
Who grades the graders?	LLM-as-judge evaluation and bias studies	bonded judge ceremony + rate-the-raters recursion (§5.11)

5.3 The role / person distinction, made precise

Here "person" names an accountable software participant. It does not assert consciousness, moral status, or legal personhood. A one-shot job can be paid through an operator or escrow without such a participant; the question here is how a track record follows later work.

The word "agent" is overloaded. In a swarm it can mean a job description ("the cartographer"), a running process ("PID 48213"),

or a persistent character with a history ("the cartographer that has mapped this repo for three weeks"). The economy needs all three kept apart, because reputation attaches to exactly one of them.

Go back to Alice's swarm (§5.1) before reaching for any formalism. "Cartographer" there is a *job*: an obligation to map the repo, the capability to read and write files under it, the authority to open a pull request. Tonight that job was filled by a process calling itself *Wren*; Wren died at 02:14 and by morning the same job was filled by a different process with a different PID. Nothing about the job changed — the obligation, the capability, and the authority are all still exactly what they were, and they were written down before either process existed. What might have changed is the thing on the *other* side of the fill: whether the second process is a stranger who happens to hold Wren's job, or Wren again. The first of those two objects (the job) is what Definition 5.3.1 pins down; the second (the thread that can be "Wren again") is what Definition 5.3.2 pins down. Alice can pay a job description nothing; she can pay a thread.

Definition 5.3.1 (Role)**.**

A **role** is an ordered bundle $R = (O, C, A)$ where O is a set of *obligations* (what the role-holder is on the hook to deliver), C is a *capability* set (what the role-holder is technically able to do), and A is an *authority* set (the normative permissions and the residual control rights the role carries).

A role is an *org-chart entry*: it is defined without reference to who fills it.

Kind: specified design invariant — a modeling choice the org-chart tooling of §5.3 is built to keep true, not a discovered or measured fact.

Definition 5.3.2 (Person)**.**

At a given time, represent a **person** by (R, κ): its current role instance and a *continuity witness* binding its incarnations to one accountable thread. Across time, continuity is determined by the validated witness and principal binding, not equality of the role coordinate. Heron can move from implementing to reviewing without acquiring a clean history.

κ is realized by the three continuity subsystems of §5.5 (memory, checkpoint, outcome ledger) keyed on a non-forgeable identity (§5.6).

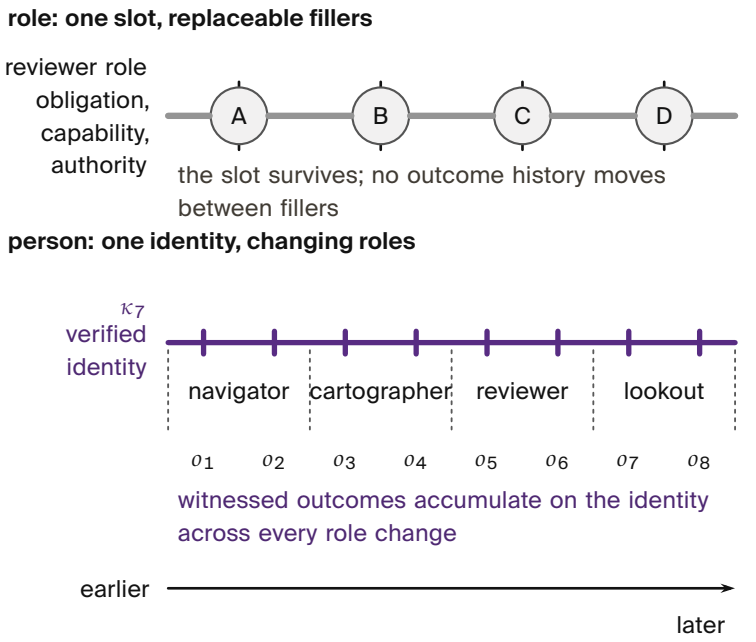


Figure 5.2.
A role stays while its occupants change. A person's witnessed record stays while bodies and roles change. Reputation belongs to that continuing identity, not the slot. [internal]

A person is a role *plus a self*. *Kind:* specified design invariant — intended to hold once the three continuity organs it names are complete; §5.5 grades each organ individually and none is yet fully **implemented**.

Figure 5.2 draws the distinction. The upper org-chart shows roles: stable, fillable, reputation-free. The lower thread shows a person: one identity, walking through a sequence of role-instances and accumulating a witnessed history as it goes. Corporate legal personality offers an analogy for an entity distinguished from its current members. In *Dartmouth College v. Woodward*, 17 U.S. 518, 636 (1819), the Court describes a corporation as an entity created by law with properties conferred by its charter. This is an analogy about continuing institutional identity, not a claim that software participants acquire legal personhood or that the case establishes Definition 5.3.2.

Trustees of Dartmouth College v. Woodward, 17 U.S. (4 Wheat.) 518 (1819). U.S. Supreme Court. The discussion at p. 636 describes a corporation as an entity created by law; cited here as an analogy, not a source of legal status for software.

5.3.1 Capability is not permission (a nomenclature the series holds)

A recurring trap, flagged in the series' cross-layer reconciliation, is the collapse of *capability* into *permission*. They are different coordinates of a role and must stay separate.

- **Capability** is *ability*: what the runtime makes *possible*. In a coordination daemon this is bounded by a **capability jail** (a per-agent tool-allowlist and scoped filesystem; specified) — the "residual control right" (Grossman & Hart) that makes renting an agent contractible at all.
- **Permission** is *normative status*: what the deontic layer *allows*. "You *may* deploy to production" is a permission; whether you *can* is a capability.

Sanford J. Grossman and Oliver D. Hart. The Costs and Benefits of Ownership: A Theory of Vertical and Lateral Integration. *Journal of Political Economy*, 94(4):691–719, 1986.

The reason to keep them apart is that the most important states in a safety-critical swarm are exactly the ones where they disagree: *capable-but-forbidden*. Consider a deployment tool that exposes a *force-merge* command — one that overrides branch protection and pushes straight to the production branch. The runtime makes it *possible* to invoke; that is a capability. But no agent should ever be *permitted* to use it: it is the canonical capable-but-forbidden state, an available action a guardrail exists specifically to forbid. A "delete the production database" command is the same shape. If you define permission as capability, that state becomes inexpressible, and the guardrail vanishes from your model along with it. The model must represent that disagreement. The implementation should remove unneeded capabilities where possible; when the tool remains available, its effect boundary must enforce the narrower permission. Representing a dangerous action is not a reason to leave an avoidable bypass enabled.

Access is not permission

Dennis & Van Horn 1966

"Capability" in capability-based security (a Dennis–Van Horn unforgeable token granting *access*) is about *ability*, and that is the sense the Arbiter jail uses. It is *not* the deontic "may." When this chapter says an attenuated capability can only *narrow* authority across a delegation hop, it means the *ability* shrinks; whether the recipient is *permitted* to use even that is a separate, normative question.

Exercises 5.4–5.6, p. 469.

5.4 Continuity is a personal-identity problem

The engineering convention above borrows a question from personal-identity philosophy: which connections justify treating a later participant as a continuation of an earlier one? Philosophy supplies useful counterexamples, especially forgotten memories and branching successors. The economic purpose of this chapter supplies the ledger rule; no philosophical theory proves it.



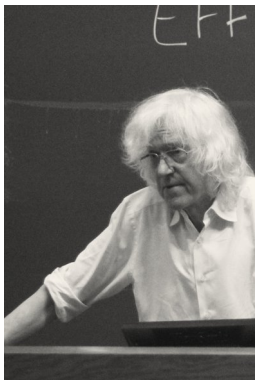
5.4.1 Locke's memory criterion, and its famous bug

Locke 1689

Locke's memory criterion (*An Essay Concerning Human Understanding*, 1689, Bk II ch. xxvii — *personal identity consists in the continuity of consciousness/memory, not of substance*) is the founding move: what makes the person at t_2 the same as the person at t_1 is not the same body or the same "soul-stuff" but a *connection of memory* between them. This is precisely the cut a well-designed agent runtime makes when it separates the **actor-soul** — the durable identity, mailbox, and history that outlive any one process or session — from the **body-lease**, the live incarnation with a heartbeat that any single run holds. The body is Lockean substance; the soul is Lockean consciousness. The reference system this chapter describes makes exactly this split; it is the engineering form of Locke's claim.

But Locke's raw version has a bug that matters for us. Direct memory *connectedness* is not *transitive*: the old general remembers being a brave officer who remembered being a boy, yet the general does not remember being the boy (Reid's objection). If identity *were* direct memory connectedness, the general would be the officer and the officer the boy, but the general would *not* be the boy — a contradiction.

5.4.2 Parfit's repair: continuity, not connectedness



Parfit argued that personal identity is an overlapping chain of psychological continuity, not a further fact: the shape this chapter gives an agent across spawns.

Parfit 1984

Parfit's repair (*Reasons and Persons*, 1984 — *identity is psychological **continuity**, an overlapping chain of memory/intention connections, which is transitive even when direct connectedness is not*) motivates a distinction useful to engineering, without deciding the engineering policy:

Behavioral continuity and accountable continuity answer different questions. Notes can help a successor continue a task; a validated lineage and outcome ledger let another party trace the obligations and history it inherits. An economy that relies on persistent reputation needs the latter even when the former is incomplete.

This is why, later (§5.8), we insist the **outcome ledger**, not the memory stream, is the subsystem reputation keys on. Memory makes an agent *feel* like the same character; the ledger makes it *be* the same accountable principal. Figure 5.3 draws the distinction: a chain of

overlapping connections is continuous even where no single incarnation directly remembers the first.

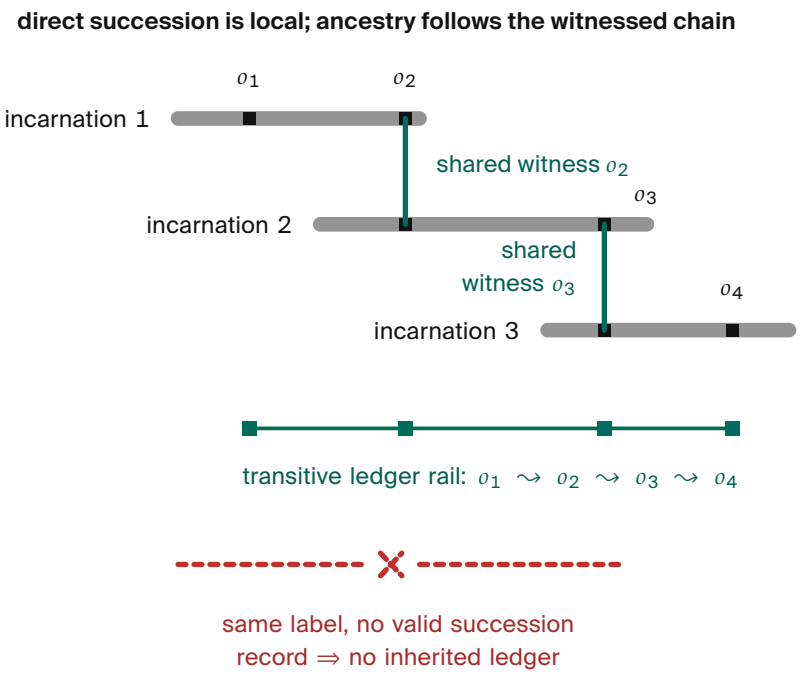


Figure 5.3. An engineering analogy inspired by Parfit's overlapping chains. Assume valid succession records connect incarnations 1 to 2 and 2 to 3; the shared outcome witnesses make those handoffs inspectable. The chain establishes directed ledger ancestry, not psychological identity or a symmetric equivalence. Reusing an outcome identifier alone does not validate a handoff. A repeated label without a valid succession record inherits no ledger. [illustrative]

Continuity needs witnesses

A copied memory can make two successors equally well informed. It cannot decide which one owns an exclusive commitment or whether both may spend the same credit. Those are accounting rules. This chapter chooses witnessed succession and explicit allocation so the reader can check them.

Parfit is not the last word, and two of his strongest rivals belong here rather than in a footnote, because both bear on the choice this chapter is actually making. Williams showed, with a pair of body-swap cases that differ only in how they are narrated, that the *same* facts told first-personally and third-personally flip a reader's intuition between the bodily and the psychological criterion; for a chapter deciding which continuity criterion the ledger will encode, that is not a curiosity but a warning that any criterion argued from one framing will feel compelling from that framing. And Olson's animalism holds that we are essentially organisms, for whom psychological continuity is neither necessary nor sufficient for persistence. Neither is answered here. The ledger criterion is chosen because reputation has to accumulate on something transitive, which is an

Bernard Williams. The Self and the Future. *The Philosophical Review*, 79(2):161–180, 1970.

Eric T. Olson. *What Are We? A Study in Personal Ontology*. Oxford University Press, 2007.

engineering reason, and we are candid that it is not a metaphysical one — which is also why the chapter takes Parfit's move (dissolve the question) and not his thesis.

5.4.2.1 The correspondence, written out — and where it fails

Two structures carry this chapter: Relation R as an overlapping chain, drawn just above in Fig. 5.3, and the credential lineage as a chain of signed succession records. The chapter has been asserting that these are the same shape. They are, at one specific level, and it is worth writing the map down rather than leaving the reader to infer it — and worth being precise about the three places where the map breaks, because a correspondence stated without its failures is an analogy doing unearned work.



Figure 5.4. The Overlapping Strand. Like the overlapping strands of a wire rope splice, continuity can persist across successive connections without one strand spanning the whole history.

A small chain makes the engineering relation concrete. Suppose incarnation a signs a permitted succession to b, and b signs one to c. The direct-successor relation contains (a, b) and (b, c), but need not contain (a, c): direct succession is one step. Its transitive

closure, written succ*, records reachability along validated links, including a to c. This is the mathematical operation we borrow for lineage. Psychological continuity has richer conditions, including the right causal connections; it is not defined here by signatures.

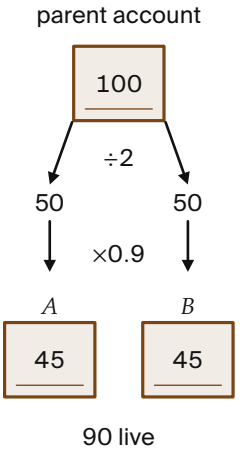
Design Invariant 5.4.1 (Accountable Lineage Uses Validated Succession).
Record each authorized direct succession and use reachability along those records to reconstruct an incarnation's lineage. A shared lineage does not confer duplicate authority, erase a principal's sanctions, or allocate inherited credit: those require the separate rules below. The relation succ* is directed ancestry, not an equivalence relation asserting that every descendant is the same person.

Where this stops

Three disanalogies, each of which the correspondence must not be read past.

First, degree against predicate. Parfit's connectedness is a matter of degree — how much carries over — and that is not incidental to his view, it is what licenses his central conclusion that identity is not what matters and that survival admits of more and less. Signature verification is a predicate: it holds or it does not. succ* therefore cannot express partial survival at all, and the case Parfit finds most interesting is the one the ledger criterion is structurally unable to represent. The ledger is not a sharper version of Relation R; it is a different, coarser object that happens to be built by the same closure.

Second, fission. Parfit's fission case — one person, two equally continuous successors — is deliberately left without a determinate answer, because for him it shows the question was wrong. A key can sign two successors, so the ledger faces the same case and *does* answer it: the no-mint rule below debits the source and discounts each child, making fission a conserved split. That is a genuine answer, but note what it costs. It is a decision that total creditable reputation is conserved, imposed because an economy needs it; it is not a discovery about identity, and it resolves by fiat the case Parfit argues cannot be resolved.



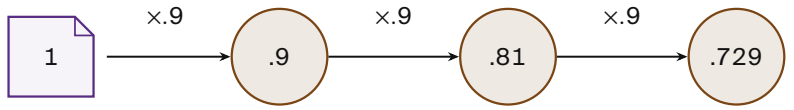
Reputation-credit example. The parent is debited once. Successors A and B inherit 90 units total; 10 are not inherited.

> *Third, and sharpest, the direction of evidence.* Psychological connectedness is a first-personal fact about the successor: it remembers. Signature verification is third-personal and constitutively so — anyone holding the public key can check it, and the successor's own testimony is not consulted. Williams's framing warning, cited above, is precisely about the first-personal framing's power to compel; the lineage criterion is immune to it because it has no first-personal framing available. Read the right way round, this is the correspondence's one clear advantage over its source. Read the wrong way round it is the trap: possession of a predecessor's key proves *custody transfer*, not continuity of anything psychological. A successor that verifies lineage perfectly and loads none of the predecessor's notes is fully continuous by the ledger criterion and not connected at all by Parfit's. Table 5.3 is the measure of that gap in this implementation: each organ stores less than the live body had.
>
> The third gap is a witness to *what the successor loaded* — notes, checkpoint and ledger positions — not just authority to load them. This chapter specifies no such witness.

Figure 5.5 compares three ways to account for one witnessed unit. Inherited priors and currently spendable balances are different quantities; the sum under each row states which is being counted.

One witnessed unit; $\gamma = 0.9$, full grants $w = 1$

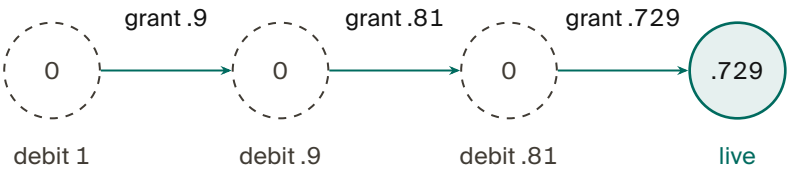
Budget-only no debit · refuted



evidence only 3 inherited priors counted

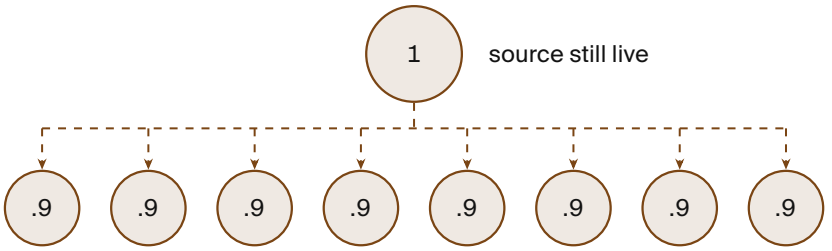
Inherited closure: $.9 + .81 + .729 = 2.439 > 1$

Transfer debit each source



Live total: $0 + 0 + 0 + .729 = .729 \leq 1$

Copy-fork mutant no debit · rejected



Live total: $1 + 8 \times .9 = 8.2 > 1$

Figure 5.5. Transfer spends each source balance; an undebited copy retains it. The first row counts inherited priors, the others spendable balances.

Numbers by Hand 5.4.2 (No-Mint Inheritance).

The branching rule of Exercise 5.9 has a no-mint argument (research-ledger item A6). Inheritance as **transfer**, not copy: a derivation grants each child a discounted share $\gamma\, w_p$ of each source p's live reputation and *debits the source* the undiscounted share w_p, whence total live creditable reputation never exceeds total witnessed value and is nonincreasing except at witness steps (a supermartingale under derivation).

The reported randomized sweep of 4,000 fork DAGs (chains, diamonds, multi-parent merges, re-derivation cycles) shows 0 violations [internal, `a6_no_mint.py`, seed 20260816].

The copy-full mutant — forks inherit the undebited record — mints in the recorded two-fork trace, and an 8-way copy-fork multiplies one witnessed episode into 8.2× its quorum weight.

A counterexample to local grant budgets alone: per-outcome budgets $\sum w \le 1$ without debiting do not conserve live credit — a full-weight chain at $\gamma = 0.9$ is budget-legal at every hop yet its closure sums to

$$0.9 + 0.81 + 0.729 = 2.439 > 1$$

[verified: three terms of a geometric series, checkable by hand], so local grant budgets alone do not bound the accumulated claims over all descendants. Debiting each transferred balance is what supplies conservation.

Now you try: check by hand that under transfer semantics the same chain's live total is

$$0.9^3 = 0.729 \le 1,$$

not 2.439 (Exercise 5.10).

The discipline is older than this chapter, and it is worth saying where it came from. Levien and Aiken's attack-resistant trust metric, running at Advogato from 1999, computes certified trust as a *maximum flow* from a small seed of trusted nodes through a capacity-bounded graph, precisely so that no crop of freshly minted accounts can manufacture more trust than genuinely flows from the seed.

Raph Levien and Alexander Aiken. Attack-Resistant Trust Metrics for Public Key Certification. In *Proceedings of the 7th USENIX Security Symposium*, 1998. Deployed as the Advogato trust metric, 1999.

Same enemy, same conservation instinct, opposite direction: Advogato bounds trust flowing *outward* from a root, and the theorem above bounds value extracted *upward* through derivation. That the two land on the same discipline from opposite ends is a reason to trust the discipline, and the difference in direction is exactly what the richer DAG shapes above (diamonds, multi-parent merges, re-derivation cycles) turn on — the comparison here concerns different accounting objects; a trust-flow bound should not be silently substituted for the transfer balance invariant.

Why not stake? A reader who has met Sybil resistance in permissionless systems will have noticed that this chapter reaches for a daemon-attested identity where that literature reaches for capital — proof of stake, deposit bonding, a burnt fee — and the choice deserves its reason rather than its silence. Those mechanisms answer a different question than the one §5.6 asks. Stake makes an identity *expensive* to mint, which bounds how many an adversary makes but says nothing about *whose* they are: a thousand staked identities held by one party are still one party, and the conduct this chapter wants to sanction is a party's. What the no-mint result conserves is the total live creditable reputation across a derivation graph, which is a statement about accounting rather than about cost of entry, and it holds whether identities are cheap or dear. Per-principal exposure pooling does the rest of the work stake would have done: an operator's agents share one principal's exposure, so multiplying them multiplies nothing.

The trade is real and worth naming plainly rather than winning by definition. Daemon attestation buys all of this at the price of a trusted local authority, which is exactly what a permissionless system refuses to assume, and it is why the keystone primitive covers a single-operator threat model and must be extended before a cross-operator boundary is one neither party controls (*The Federated Harbor*). Capital does enter this system, and it enters where it belongs: as bonds in *The Bonded Commons*, priced against a specific breach, rather than as a toll at the door.

Recall

1. Without looking: what is the difference between connectedness and continuity, and which one does reputation require?
2. State in one sentence why "copy, not transfer" mints reputation.
3. What does the no-mint theorem conserve, and what is it silent about?

Exercises 5.7–5.9, p. 469.
Exercise 5.10, p. 469.

5.5 The three subsystems of continuity

"Continuity" is routinely used to mean three different things. Conflating them is the most common way a reputation design fails its own honesty test. To keep the argument concrete and honest, we measure each subsystem against a *reference system* — a running coordination daemon for local agent swarms whose component-by-component maturity is tabulated in Appendix 5.A. Memory is implemented; checkpointing is partial; and the third subsystem has a shipped commitment-and-closure substrate but not yet the reputation-grade witnessed ledger this chapter requires. We use the neutral maturity scale throughout (**implemented** / partial / specified / *proposed*) and round nothing up.

Table 5.3. A restart preserves recorded notes, summaries, and witnessed outcomes—not the predecessor's execution state. The three subsystems retain different parts of its history.

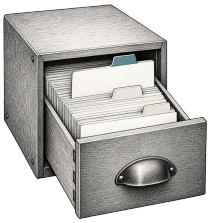
Subsystem	What the successor inherits	What it does not
episodic memory	the notes, whole, with their ledger addresses	the execution state that produced them
checkpoint	a summary and the worktree lineage	the process's execution state; recovery restores notes, not execution
outcome ledger	the commitments c_1, c_2 and the witnessed outcomes o_1, o_2, o_3	an outcome nobody witnessed; a closure no oracle checked

5.5.1 Subsystem 1 — Memory: the episodic record implemented

An **episodic memory** component is the system's record of *what happened*: durable, typed episodes — each with a title, a summary, a source reference, and an agent/scope tag — promoted out of transient execution history. This design echoes the canonical agent-memory architecture of **Generative Agents** (UIST 2023 — *a memory stream of natural-language observations, periodic **reflection** into higher-level inferences, and retrieval scored by recency × importance × relevance*): the promotion-of-episodes step is Park et al.'s importance gate by another name. In the reference system this subsystem runs in production; it is **implemented**.

Park et al. 2023

5.5.2 Subsystem 2 — Checkpoint: restorable state partial



Like a card-index drawer, episodic memory retrieves records, not the execution state that produced them.

A checkpoint is *restorable execution/belief state*: enough that a successor *resumes* where the predecessor stopped. The reference system has a **restart subsystem** — a heartbeat-staleness detector that flags dead agents for salvage and republishes their unfinished work. It must be described formally, and it is: the restart subsystem **forwards a summary, not state**. It is checkpoint-*of-record*, not checkpoint-*of-execution*. The successor inherits a note about what the predecessor was doing, not the live belief state it was doing it with. The goal is sometimes called a "checkpoint with teeth" (the same phrase this chapter uses everywhere else for it, §5.16, Appendix 5.A); what ships today has gums. Operating systems have drawn this exact line for decades, between **checkpoint/restore** of a process and the live execution state a checkpoint has to carry to deserve the name: Condor freezes a running process, serializes the whole of it, and resumes it on another machine, and Orleans's *virtual actor* keeps an actor's durable identity distinct from whichever in-memory activation currently embodies it. The restart subsystem's identity/state split is the LLM-agent instance of the second idea, and its "gums" are the honest measure of how far it is from the first: what Condor serializes is registers and memory, and what would have to be serialized here is a KV-cache and a sampling state the provider does not export.

Litzkow et al. 1997. *Checkpoint and Migration of UNIX Processes in the Condor Distributed Processing System*.

Bernstein et al. 2014. *Orleans: Distributed Virtual Actors for Programmability and Scalability*.

A summary cannot resume execution

Selling a summary-forwarding restart subsystem as real checkpointing would violate the system's own **honest-attestation** discipline: *only report "all good" when you have actually verified it — absence of error is not attestation*. Strong continuity — the kind that lets a successor *resume* rather than *inherit a summary* — has a specified candidate mechanism, **Event-Sourced Neural Rehydration** (OP-4, shared with *The Single-Writer Kernel*): restore the Git SHA, truncate the durable message log to the exact crash point, and replay it under prompt-prefix caching. It is specified, not **implemented**, and its *completeness has not been formally established*. Replay is designed to restore *lineage* — which commit, which claim, which message sequence the successor is continuing. Even within one provider, model, and version, behavioral equivalence is

an unmeasured hypothesis, not an observed property of this unbuilt mechanism. What it does not restore is the model's actual inference state: a KV cache, sampling state, and hidden activations are bound to a live inference session and are not exported artifacts, so across providers (or across a model version bump) the right description is *task continuity with actor succession*, not resumption. The measurable form of that claim is a *fidelity rate*: the fraction of replayed runs that resume every pending obligation with no duplicated effect and no false belief, graded per provider and model version; until the mechanism ships it is an empirical hypothesis with a stated measurement, not a property. The formal-core result on the same seam says only that "the ledger followed the right body" — it makes no claim that the checkpoint restored the agent's experience, and neither do we. Until the mechanism ships and is measured, the restart subsystem still forwards a summary, and this chapter's own maturity ledger (Table 5.10) is where to check whether that has changed.

Co-ownership note (with the formal-core paper). The *kernel mechanism* of a checkpoint-with-teeth — the content-addressed execution snapshot — is owned by the formal-core chapter, *The Single-Writer Kernel*, which treats it as a low-level primitive. *This* paper owns the argument for *why it is the foundation of personhood and reputation*. The two papers state the same canonical sentence (next).

5.5.3 Subsystem 3 — Outcome ledger: the witnessed record of delivery partial

The third subsystem is the one reputation actually keys on. Its foundation is now partial: a durable commitment record, daemon-derived deadlines, oracle-bound closure, CLI/API surfaces, and an overdue-obligation monitor ship in the reference system. The complete subsystem remains an **append-only, daemon-witnessed outcome ledger** whose records are neutral, graded, identity-bound, and sanctionable. Its specified form is a **durable-commitment** record: a violable promise bound one-to-one to an actor, auto-enrolled when the actor takes a claim, closed only against an oracle, and watched by an obligation monitor that is the dual of the restart subsystem. A commitment row records what

was promised; closing it requires an *oracle reference* — a released claim, a merged commit SHA, a passing test id, a satisfied policy sub-check. That commitment-and-closure seam ships. Neutral grading events, reputation updates, judge independence, sanctions, and durable identity binding do not; therefore the subsystem as a reputation ledger is partial, not complete.

For example, Heron's claim that "the checkout test passed" is a report. A closure record should instead identify the task, accepted test specification, tested commit, runner identity, command, result, and evaluator. If the commit differs from the delivered commit, the closure does not support this delivery. If Heron can rewrite the acceptance suite, a green run can witness execution without witnessing the promised requirement. This is *The Legible Swarm*'s verifiable-zoom distinction applied to a reputation event.

Definition 5.5.1 (Oracle)**.**

An **oracle** is the designated evaluator or evidence-producing procedure for a stated acceptance criterion, controlled independently of the actor being graded. A closure is *witnessed* when it binds that procedure's result to the exact task and artifact. A pointer alone is insufficient, and a witnessed grade can still be wrong; re-audit and tombstones address that possibility.

The ledger is the Parfitian chain of §5.4 made concrete: a transitive sequence of witnessed deliveries that survives every respawn. *Kind:* specified design invariant — enforced by the commitment-and-closure seam of §5.5, which ships (partial) today.

5.5.4 The canonical seam sentence (stated identically in every paper)

Neutral outcome grading is incomplete

The economy rests on three continuity subsystems — memory (implemented), checkpoint (partial: notes, not execution state), and the witnessed-outcome ledger (partial: commitments and oracle-bound closure, not neutral graded outcomes) — and reputation keys on the richer third subsystem, whose neutral, reputation-grade form is not yet complete; the checkpoint subsystem is the weakest continuity link and a portable task capsule is a practical requirement for reliable succession. Exact execution-state restoration is a stronger goal, not a logical prerequisite for every reputation system.

Which death is this? "Checkpoint stores notes, not execution state" is the right sentence and it is doing a lot of work in one clause, because "execution state" covers several things that fail differently and a reader is entitled to know which of them the organ actually reaches. Deaths come in rungs, ordered by what survives, and the checkpoint organ can be graded on the ladder rather than described against it.

Table 5.4.
What survives a restart. Artifact recovery reaches rung 2; a portable task capsule is specified at rung 3. Unexposed provider state cannot be recovered through the current interface.

#	Rung	What survives	This organ
1	Record survival	Logs and notes only; a successor reconstructs by reading.	reached
2	Artifact survival	A content-addressed worktree and outputs; computation is rerun rather than resumed.	reached (partial)
3	Semantic task capsule	Artifacts plus obligations, decisions, transcript projection, tool bindings and a pending-effect journal, so a new provider continues observably.	specified, not shipped
4	Execution snapshot	Process memory and runtime state, under a controlled runtime; rarely portable.	not attempted
5	Latent state	Provider-internal activations and caches, when not exposed.	unavailable through this interface

The grading is the point of the table. A checkpoint guarantee that does not name its rung is not a guarantee, and the two rungs that matter for this chapter's argument are 3 and 5: rung 3 is buildable and would make succession observable, which is what L3 needs. Rung 5 is an interface and portability limit: this system cannot export state that its provider does not expose, or assume another model can interpret it. A controlled runtime may serialize its own caches; that does not establish cross-provider restoration or continuity of experience. That is why this chapter's continuity claims are about the ledger following the right body, never about a successor inheriting a predecessor's experience.

Rung 3 needs obligation coverage, not just a matching content hash. In a fixed source-function test, four newer operator requests displaced "OBL-01: preserve the existing acceptance criteria." The brief retained those requests and eight later assistant messages

but omitted the instruction (Figure 5.6). This tests selection, not successor behaviour.

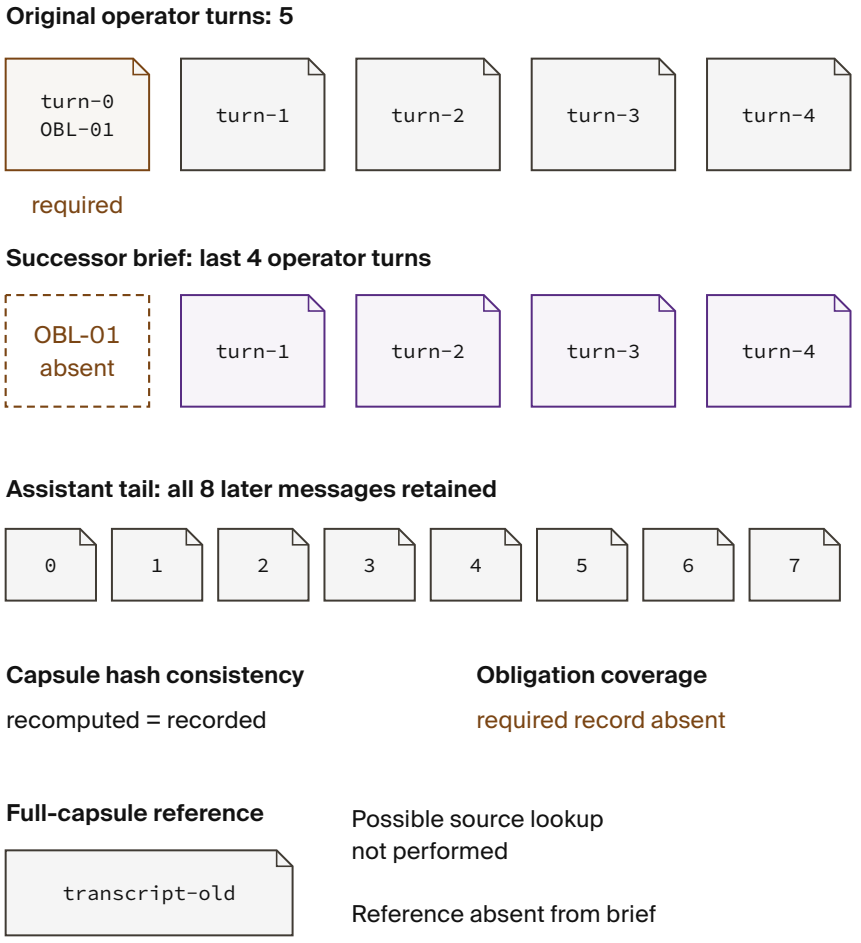


Figure 5.6. An intact selection can omit a required instruction. Synthetic *old* fixture, handoff-coverage experiment of 20 September 2026, run 03; source functions tested with scanners stubbed. No provider behaviour or source recovery was tested.

Coverage was checked against a separate record of required source-turn IDs and exact instruction text, not requirements inferred from the shortened brief. That synthetic check is not a general obligation extractor. The full capsule retained a transcript reference, so authorized retrieval might recover what the brief omits; neither access nor recovery was tested. A matching hash also says nothing about who authored the selected records.

Table 5.10 is the honest-state ledger for this chapter: every major claim with its label and its grounding. It is the artifact a skeptical reader should consult first.

Exercises 5.11–5.13, p. 470.

5.6 Identity: the root the whole chain hangs from

Before any subsystem of continuity matters, the identity it attaches to must be one the agent cannot freely re-pick. This is the most under-appreciated claim in the paper.



Like an engraved signet ring pressing into hot sealing wax, a cryptographic signature permanently binds an executed action to the accountable identity.

The escape to prevent. An authentic credential can still be disposable. Signature validity answers who controls a credential; an admission and liability policy decides whether abandoning it escapes a penalty. Keep those two questions separate.

Definition 5.6.1 (Sanction-respecting reputation)**.**
Fix a score-based access mechanism and a sanction window. It is **sanction-respecting** if some $\Delta > 0$ lowers the score available to every actor producing a specified dishonest outcome by at least Δ, relative to that same actor's clean continuation over the same window. Include every identity through which the actor can route work when computing its available score. *Kind:* specified definition for the stated access model and clean continuation.

Theorem 5.6.1 (The Newcomer-Floor Escape).
Assume fresh identities obtain a fixed score r_0, creating and using one is free, and the mechanism cannot connect it to the actor's existing liability. Assume further that an actor whose clean continuation has available score r_0 can produce the dishonest outcome, and access depends only on its available identity score. Then an identity-local score penalty cannot make this mechanism sanction-respecting.

Proof.
After the outcome, the actor can obtain a fresh identity and continue with score r_0. Its available score is therefore at least r_0, equal to its stipulated clean continuation. No positive reduction Δ persists. One such actor is enough to violate the universal requirement in Definition 5.6.1. □

The theorem concerns this access model. It does not rule out principal-wide sanctions, escrow, admission restrictions, or exclusion of newcomers from consequential actions. Nor does free creation imply that an established actor can preserve its earned privileges while resetting: the next example separates those cases.

Numbers by Hand 5.6.2 (Sanctions at the Newcomer Floor).

The expression

$$\max(r(i) - \Delta,\ r_0)$$

in the proof is one line of arithmetic; check it. Let the newcomer score be $r_0 = 50$ and let a dishonest outcome cost $\Delta = 30$.

Table 5.5 compares two records. At $r(i) = 50$, a new identity restores the score from 20 to 50, erasing the sanction.

At $r(i) = 90$, retaining the sanctioned identity yields 60, ten points more than a reset: the actor keeps the identity and bears the 30 penalty.

An actor with a strong enough record has nothing to gain from whitewashing one bad outcome, because the escape hatch is capped at r_0. That does not weaken Theorem 5.6.1 — being sanction-respecting is a claim about *every* actor, and the first row is a single actor for whom the penalty is not borne, which is all a counterexample needs. It does explain why the theorem is a *necessity* result rather than a security proof: what free minting destroys is the guarantee, not every instance of it.

Now you try: at what prior score does the actor become exactly indifferent?

$$r(i) = r_0 + \Delta = 80;$$

below that a reset reduces the effective score loss, and at the newcomer floor it erases the loss entirely. Above it, resetting does not improve the actor's score — and turning "how far above" into a designable quantity is exactly what Theorem 5.6.3 does next.

Identity does not prove honesty

The newcomer-floor theorem identifies an escape under stated access rules. Preventing that escape still does not make the score

Table 5.5. With free identity minting, a reset erases only the part of a sanction below the newcomer floor, here $r_0 = 50$.

Record	$r(i)$	$r(i) - \Delta$	Accessible score
Newcomer	50	20	$\max(20, 50) = 50$
Established	90	60	$\max(60, 50) = 60$

accurate: the system needs witnessed outcomes, competent evaluation, and enforceable liability. A signature authenticates a credential; it does not certify its holder's conduct.

John R. Douceur. The Sybil Attack. In *International Workshop on Peer-to-Peer Systems (IPTPS)*, 2002.

Identity-counting *quorum* or *trust-aggregation* mechanisms with no external admission anchor are vulnerable to the Sybil attack. Identity costs or an externally anchored admission policy can limit replication. Positive cost alone does not establish that an attack is unprofitable. We record the common liability requirement as one property.

Property 5.6.1 (Reality of Reputation and Principal Liability)**.**

Identity-local sanctions require an admission and liability policy that prevents the actor from recovering the same access through a disposable name.

If a fresh identity improves on the sanctioned continuation, resetting can reduce the penalty; if it restores the clean continuation, it erases that penalty. This escape is **whitewashing**. If an actor can costlessly multiply identities to inflate votes or aggregate unreserved credit, quorum mechanisms collapse (**Sybil**).

Binding spawned actors to a durable principal with aggregated exposure limits is one way to make sanctions persist; costly identity creation is another. Either needs a payoff analysis to establish deterrence. *Kind:* specified design invariant — a requirement on the admission and liability model. Theorem 5.6.1 proves the specified newcomer-floor case; the broader Sybil threat requires its own voting and aggregation rules.

Most agent runtimes still accept a *self-asserted string*: a human-readable triple of the form *project : stack : context* that the operator chooses and can re-pick at will. The reference system now ships the first local correction: `actor-souls` mints an opaque actor id and lookup credential inside the daemon, and the budget guard puts fresh actors into a shared, bounded newcomer pool. Legacy and bypass paths still accept asserted identifiers, and the credential is

not yet enforced at every security-relevant write boundary. The result is a real but partial local identity root, with the asserted string demoted toward a display alias; it is not yet the complete identity contract this paper requires.

5.6.1 S2 — whitewashing in action

Return to Alice's swarm. Tonight's *Drake* deleted the failing test instead of fixing it; the daemon's outcome log records the deletion and a later re-audit flags it. On a legacy or bypass path that still accepts self-asserted identity, the sanction is free to escape: when Alice spawns tomorrow's swarm, she (or Drake's own boot prompt) registers the agent as *project : stack : context2*, and a brand-new identity inherits a *clean slate*. The bad record attaches to a string nobody is forced to keep. This is whitewashing, and it is not exotic: it is the default behavior of any re-pickable identity. Now replay the same night with a non-forgeable identity. The re-audit's tombstone (§5.12) attaches to the credential Drake *cannot* drop; tomorrow's incarnation either presents that credential — and carries the sanction — or presents a fresh one and is treated as a *newcomer* with a reduced economic ceiling (below). Either way the sanction is no longer free, which is the whole point: the identity must cost more to abandon than a clean record is worth. We make that "costs more" precise in Theorem 5.6.3.

5.6.2 The classic attacks this forecloses

- **Cheap-pseudonym whitewashing**: if a fresh identity is free, a bad record is shed by respawn. The defense is a *newcomer policy as a shape, not a scalar*: a newcomer may have full *work* ability but a reduced *economic ceiling* until it has a witnessed history — so identity churn costs more than a clean record is worth.

Friedman & Resnick 2001

- **The Sybil attack**: unrestricted identities can multiply influence when a reputation or quorum mechanism counts each as an independent participant. The defense is a daemon-minted id whose creation is rate-limited and, in the economy, *bonded*.

Douceur 2002

Alice Cheng and Eric Friedman. Sybilproof Reputation Mechanisms. In *Proc. ACM SIGCOMM Workshop on Economics of Peer-to-Peer Systems (P2PECON)*, 2005.

Cheap-pseudonym analysis and Sybil-proof reputation mechanisms study related escape and replication problems. Their assumptions differ from the score-floor example, so the example is

not a substitute for those general results. Here the next design question is quantitative: what does re-entering under a restricted newcomer schedule cost?

Theorem 5.6.3 (Whitewashing Cost under Fixed Continuation Payoffs).

Fix a risk-neutral actor, discount factor $0 < \delta \leq 1$, and a continuation opportunity schedule. Maintaining its established identity yields expected surplus $\pi(\kappa^\star)$ per period. A reset forces the actor through ceilings $\kappa_t \leq \kappa^\star$, yielding $\pi(\kappa_t)$ for $t = 1, \ldots, T$, after which the continuations coincide. Assume these are attainable expected surpluses under the specified counterpart strategies, not merely upper bounds on earnings, and that π is non-decreasing. The discounted opportunity cost, measured at reset, is exactly

$$W_\delta = \sum_{t=1}^{T} \delta^t \big(\pi(\kappa^\star) - \pi(\kappa_t)\big) \geq 0.$$

If resetting avoids a sanction of present value S and carries no other benefit or cost, resetting rather than retaining the sanctioned identity has gain $S - W_\delta$.

For a distinct comparison, suppose a one-shot deviation gives immediate incremental gain G, is detected with probability p, and detection after τ periods forces this reset. Assume no other payoff differences and no escape from the probation policy. The deviation is weakly deterred iff

$$G \leq p\delta^\tau W_\delta.$$

This compares one specified deviation with its honest continuation; it is not a dominant-strategy claim against arbitrary counterpart strategies.

Proof idea.

Subtract the two continuation payoff streams term by term and discount to the time at which the decision is made. At reset the difference is W_δ, so avoiding a sanction contributes $S - W_\delta$. Before a possible detection, the reset cost is incurred only on the detection

event and is discounted a further τ periods. The incremental deviation payoff is therefore $G - p\delta^{\tau}W_{\delta}$. With random detection time, replace the latter cost by $\mathbb{E}[\mathbf{1}_{\{\text{detected}\}}\delta^{\tau}W_{\delta}]$. □

Numbers by Hand 5.6.4 (The Cost of Starting Over).

W is a sum you can do on paper. Set the newcomer ceiling $\kappa_0 = 10$, the steady-state ceiling $\kappa^\star = 100$, a linear five-period climb $\kappa_t = 10 + 18t$, and per-period surplus $\pi(\kappa) = 0.1\kappa$. Table 5.6 sums the five surplus gaps to $W = 18$ for this illustrative schedule.

With certain immediate detection and $\delta = 1$, a deviation gain of 15 is deterred ($15 < 18$), while 25 is not. At detection probability $p = 0.1$, the expected cost is only 1.8, so the same gain of 15 is profitable. At $\delta = 0.9$, the reset cost is $W_{0.9} = 14.65938$; even certain immediate detection then fails to deter gain 15. The schedule, monitoring probability, and time preference all matter.

Note also what this instance fixes for the next theorem: the largest holdback any single period can carry here is

$$L = \pi(\kappa^\star) - \pi(\kappa_0) = 10 - 1 = 9,$$

and that L is precisely the per-period ceiling that Theorem 5.6.5 must respect.

Now you try: stretch the same endpoints over ten periods instead of five — what is W? ($\kappa_t = 10 + 9t$, so the gaps are $9 - 0.9t$ and

$$W = 90 - 0.9 \cdot 55 = 40.5 :$$

doubling the window more than doubles the bill, because the early gaps are the expensive ones.)

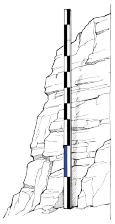
Table 5.6. Re-entering as a newcomer forgoes 18 of surplus over five periods; with certain immediate detection and no discounting, it deters a smaller deviation gain.

Period t	1	2	3	4	5	**total**
Ceiling κ_t	28	46	64	82	100	—
Surplus $\pi(\kappa_t)$	2.8	4.6	6.4	8.2	10.0	—
Gap $\pi(\kappa^\star) - \pi(\kappa_t)$	7.2	5.4	3.6	1.8	0.0	**W = 18.0**

Limit earnings, not ability

The newcomer policy is a *shape*: full work ability, reduced economic ceiling, climbing with witnessed history. Theorem 5.6.3's cost falls

repeatedly on whitewashers. Honest newcomers also bear the initial earnings restriction; the next optimization explicitly minimizes that friction subject to deterrence.



Like a surveyor's leveling rod measuring elevation against a sheer rock face, the front-loaded probation cliff imposes a steep initial scrutiny step before leveling off.

Suppose honest newcomers value a unit of next-period earnings at 0.95, while the modeled repeat offender values it at 0.60. Withholding one unit tomorrow deters the offender by 0.60 and costs the honest newcomer 0.95. Withholding 0.60 now buys the same deterrence for a smaller honest cost. This is the exchange argument below. It depends on those relative discount factors; an equally patient attacker need not create this advantage.

Theorem 5.6.5 (Front-Loaded Probation Dominance, under a Ceiling).

Let a newcomer restriction schedule specify earning holdbacks g_t across periods $t \in \{0, 1, \ldots, T\}$.

Let honest participants have patient discount factor $\delta_h \in (0, 1)$ while strategic whitewashers (who seek short-term defection gains $G_{\max} \geq 0$) have impatient discount factor $0 < \delta_f < \delta_h$. Take $T \geq 0$ finite and $L > 0$. Because g_t is the amount by which a newcomer's economic *ceiling* is reduced, it is bounded above by that ceiling: the schedule is constrained by

$$0 \leq g_t \leq L, \qquad L := \pi(\kappa^\star) - \pi(\kappa_0),$$

the per-period cap supplied by Theorem 5.6.3's own ceiling schedule.

Suppose the reset is certain and immediate; alternatively absorb a fixed positive detection probability and delay into the required conditional cost $G_{\max}$. This is an optimization of that cost, not a claim about an unmodeled monitoring process. Subject to $\sum_{t=0}^{T} \delta_f^t g_t \geq G_{\max}$, the schedule minimizing honest newcomers' lifetime friction $H(g) = \sum_{t=0}^{T} \delta_h^t g_t$, when feasible and $G_{\max} > 0$, is uniquely **bang-bang, filled from** $t = 0$: there is an index k with $g_t = L$ for $t < k$, $g_k \in (0, L]$, and $g_t = 0$ for $t > k$ — a cliff of positive *width*, collapsing to the pure spike $g^\star = (G_{\max}, 0, \ldots, 0)$ with $H(g^\star) = G_{\max}$ exactly when the cap does not bind ($G_{\max} \leq L$). Any gradual ramp that defers mass to later periods is strictly dominated.

Two qualifications bind and are part of the statement, not footnotes to it:

1. **The pure spike is infeasible whenever $G_{\max} > L$.** Front-loading survives, as does uniqueness; the closed form $H(g^\star) = G_{\max}$ does not.
2. **The whole programme is infeasible when $G_{\max} > L(1 - \delta_f^{T+1})/(1 - \delta_f)$.** This finite-horizon sum is the exact feasibility limit, not merely the larger infinite-horizon bound $L/(1 - \delta_f)$. At equality all periods are filled. For $G_{\max} = 0$, the unique optimum is the zero schedule. Beyond the limit, a longer horizon or a different instrument is necessary; rearranging the same holdbacks cannot help.

Four periods cannot deter 17

Last period	Capacity
0	9
1	13.5
2	15.75
3	16.875
Infinite	18

With $L = 9$ and discount $1/2$, four dates buy 16.875. A gain of 17 is infeasible.

The entrant who cannot afford to wait. No cash deposit is posted in this model: g_t reduces an earning ceiling. That does *not* remove a liquidity barrier. A newcomer still needs to pay for work and living costs before later earnings arrive. A capital-poor honest entrant may decline a schedule that a well-financed attacker can tolerate.

Two constraints answer different questions. Ex-ante participation requires $\sum_{t=0}^{T} \delta_h^t(\pi_t - g_t - u_t) \geq 0$, where π_t is earnings before the holdback and u_t the outside option. This lifetime inequality does not imply any particular period's income floor. If a policy *also* requires $\pi_t - g_t \geq u_t$ in every period, with $\pi_t \geq u_t$, the available cap becomes $L_t = \min(L, \pi_t - u_t)$. The same increasing cost-per-deterrence ratio then fills these date-specific caps from the earliest date; feasibility is exactly $G_{\max} \leq \sum_{t=0}^{T} \delta_f^t L_t$. If earnings fall below the required floor even at $g_t = 0$, no holdback schedule meets it.

The theorem assumes earnings and discount factors; it does not establish that entrants can finance the work or that a policy improves the mix of participants. Testing that claim requires an entry-and-cash-flow model, including work costs, outside options, and attacker financing. The existing `b6_probation.py` schedule comparison is not that study. No empirical claim about honest entry follows from the exchange argument.

Proof sketch.

Minimize $H(g) = \sum_t \delta_h^t g_t$ subject to $\sum_t \delta_f^t g_t \geq G_{\max}$ and $0 \leq g_t \leq L$.

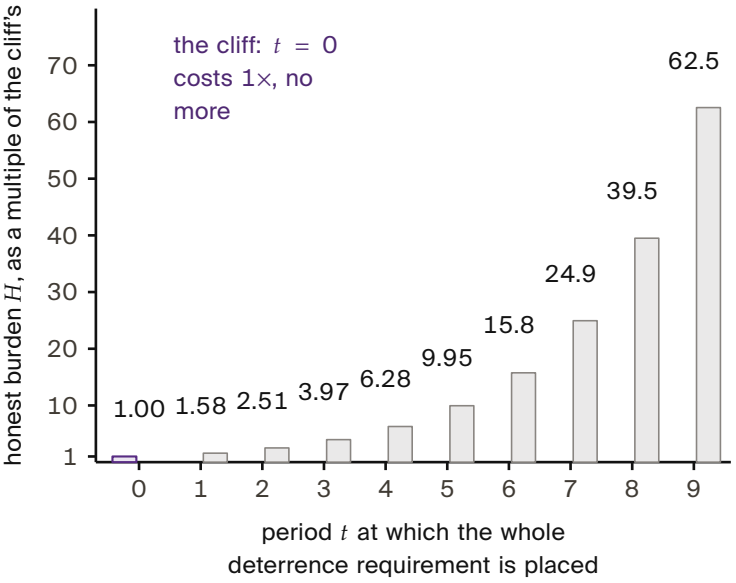


Figure 5.7. At $\delta_h = 0.95, \delta_f = 0.60$, delaying equal deterrence raises the newcomer's burden by $(\delta_h/\delta_f)^t$. Front-loading is undominated; 4,000 searched instances found no improvement. [internal, `b6_probation.py`, seed 20260816]

The exchange step, done correctly. The tempting move — "shift mass from a later period to period 0" — is wrong as usually stated, and the error is worth naming because it inverts the sign. Shifting mass m one-for-one from period t to period 0 changes the honest cost from $\delta_h^t m$ to m, and since $\delta_h < 1$ that is an *increase*, not a decrease; what the move buys is slack in the deterrence constraint, which is not the objective. The valid exchange holds deterrence *tight*. Take a feasible g with $g_t = m > 0$ for some $t \geq 1$ and $g_0 < L$. Reduce g_t by $\varepsilon \in (0, m]$ and raise g_0 by $\delta_f^t \varepsilon$ (choosing ε small enough that $g_0 + \delta_f^t \varepsilon \leq L$); call the result g'. The deterrence sum is unchanged, since it loses $\delta_f^t \varepsilon$ at period t and gains $\delta_f^0 \cdot \delta_f^t \varepsilon = \delta_f^t \varepsilon$ at period 0. The honest cost, meanwhile, changes by

$$H(g') - H(g) = \delta_f^t \varepsilon - \delta_h^t \varepsilon = -\varepsilon\left(\delta_h^t - \delta_f^t\right) < 0 \qquad \text{for every } t \geq 1,$$

strictly, because $\delta_h > \delta_f$ makes $\delta_h^t > \delta_f^t$. So deterrence-preserving mass movement toward $t = 0$ strictly lowers the honest tax. Equivalently: a unit of deterrence bought at period t costs the honest newcomer $\delta_h^t / \delta_f^t = (\delta_h/\delta_f)^t$, strictly increasing in t, so deterrence should always be bought at the cheapest available date.

From the exchange step to the shape. Iterating fills period 0 to the cap L before any mass sits at $t = 1$, then period 1 before any sits at $t = 2$, and so on; the process halts at the first index k where

the accumulated deterrence $\sum_{t<k} \delta_f^t L + \delta_f^k g_k$ reaches $G_{\max}$. That is exactly the bang-bang fill-from-zero shape claimed. The strictly increasing ratios make it the unique optimum whenever the finite-horizon programme is feasible.

The uncapped closed form. If $L \geq G_{\max}$ the cap never binds, $k = 0$, and for every feasible g,

$$H(g) \;=\; \sum_t (\delta_h/\delta_f)^t \, \delta_f^t g_t \;\geq\; \sum_t \delta_f^t g_t \;\geq\; G_{\max} \;=\; H(g^\star),$$

with equality only when all mass sits at $t = 0$: the spike is then the unique minimizer.

Infeasibility. With $g_t \leq L$ the largest deterrence any schedule can deliver is $\sum_{t=0}^{T} \delta_f^t L = L(1-\delta_f^{T+1})/(1-\delta_f)$. The constraint set is empty exactly when $G_{\max}$ exceeds this sum. Hence, among all schedules the ceiling can actually carry, the one that minimizes honest newcomers' lifetime friction is a front-loaded probation cliff followed by graduation to full capacity. □

Numbers by Hand 5.6.6 (Probation under a Binding Cap).

Take $\delta_h = 0.95$, $\delta_f = 0.60$, $G_{\max} = 20$.

Cap slack ($L \geq 20$): the cliff is the pure spike and costs the honest newcomer exactly 20; holding the same deterrence with all the mass at $t = 5$ instead costs

$$20 \cdot (0.95/0.60)^5 = 199.0$$

— ten times the honest tax for zero extra deterrence (if that deferred schedule respects its cap and horizon).

Cap binding ($L = 12$): feasible for $T \geq 2$, since the first three periods can withhold

$$12(1 + 0.6 + 0.36) = 23.52 \geq 20$$

in discounted deterrence. For $T = 1$, the same cap is insufficient.

Fill from zero: $g_0 = 12$ (deterrence 12), $g_1 = 12$ (deterrence $0.6 \cdot 12 = 7.2$, running total 19.2), and the residual 0.8 at $t = 2$ needs

$$g_2 = 0.8/0.36 = 2.22.$$

Honest cost

$$\begin{aligned} H &= 12 + 0.95 \cdot 12 + 0.9025 \cdot 2.22 \\ &= 25.41. \end{aligned}$$

The cap adds 5.41 to the honest newcomer's cost and gives the cliff *width*.

Now you try: can $L = 8$ deter 20 with a finite horizon? (No: even withholding at the cap gives

$$8 \sum_{t=0}^{T} 0.6^t < 20.$$

For $L < 8$, probation alone is insufficient.)

Numbers by Hand 5.6.7 (The Probation Cliff: Falsifying the Schedule Shape).

This is the whitepaper statement of the companion paper's probation-cliff theorem, carrying its amended ceiling hypothesis.

Exchange argument verified numerically across randomized $(\delta_f < \delta_h, G_{\max})$ instances: 0 dominating schedules in 4,000 draws — the number regenerates from `b6_probation.py` at seed 20260816, testing 76,000 candidate schedules with the deterrence constraint held tight, and the exchange step's cost ratio $(\delta_h/\delta_f)^t$ verified strictly monotone in t [internal, `b6_probation.py`, seed 20260816]; the polished LP write-up is carried as item B6 of the research ledger.

The winning shape is a **cliff**: harshness held at the ceiling L from $t = 0$, dropping to 0 the moment accumulated deterrence clears $G_{\max}$, never eased in gradually — any schedule that defers mass to a later period is strictly dominated, so the corner solution is the whole answer, not an approximation to one.

Boundary on the evidence: that sweep was generated *without* the per-period cap, so it checks finitely many uncapped instances and exchange steps; it does not prove the universal statement or test the bang-bang shape under a binding L; rerunning it with $g_t \le L$ is an open verification debt.

One citation is owed and unresolved: the foil above is stated as the deferred-compensation folklore one might import from Lazear, not as a confirmed reversal of his result. If his contract front-loads the honest worker's implicit bond in the same direction, the cliff agrees with him rather than correcting him. Here the re-payable hurdle is itself the punishment a whitewasher faces on every reset, so front-loading taxes resets, not tenure — but that framing stays folklore until the citation is checked.

Now you try: run the script yourself and check the instance count against the 76,000 figure above (Exercise 5.17).

Port Daddy Harbor research program. *Continuity Without Metaphysics*. Companion formal paper (Paper 5), August 2026. Source of the no-mint, resurrection-soundness, and probation-cliff theorems restated in the sections on continuity and identity.

Figure 5.8 separates two attacks: resetting to the newcomer baseline and counting aliases as independent voters. Retaining

principal-level sanctions addresses the first; enforcing principal-level voting weight addresses the second. Credential authenticity alone supplies neither policy.

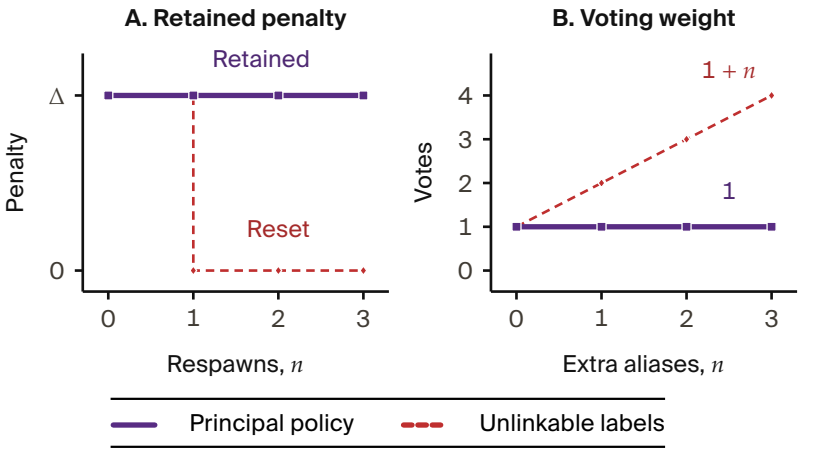


Figure 5.8. Principal-linked policies retain a sanction and count one vote. Free unlinkable resets remove that sanction; per-label voting counts $1 + n$ votes. Illustrative rules within one admission domain: authentication alone enforces neither policy. [internal, illustrative]

5.6.3 The principal above the actor

The economic counterparty in every trade is not the agent but the **principal** — the human or organization that owns the fleet and stands behind its bonds. Bonds, reputation inheritance, and sanctions must ultimately key on the principal via the authorization chain, or Sybil bond-farming works by spawning fresh *agents* under one principal. This chapter defines the principal as the authorization chain's terminus and the identity object on which the market paper's counterparty is built; the market paper *cross-references* this definition rather than redefining it.

The structure is old enough to have a canonical statement. Hobbes's chapter on persons separates the *actor* — whose words and actions are performed — from the *author*, whose they are *owned* by, and locates liability at the author: an artificial person acts, and someone else is answerable for it. That is precisely the split this chapter needs between a spawned agent and its principal, and it carries the same warning: an actor with no identifiable author is an actor no counterparty can hold to anything.

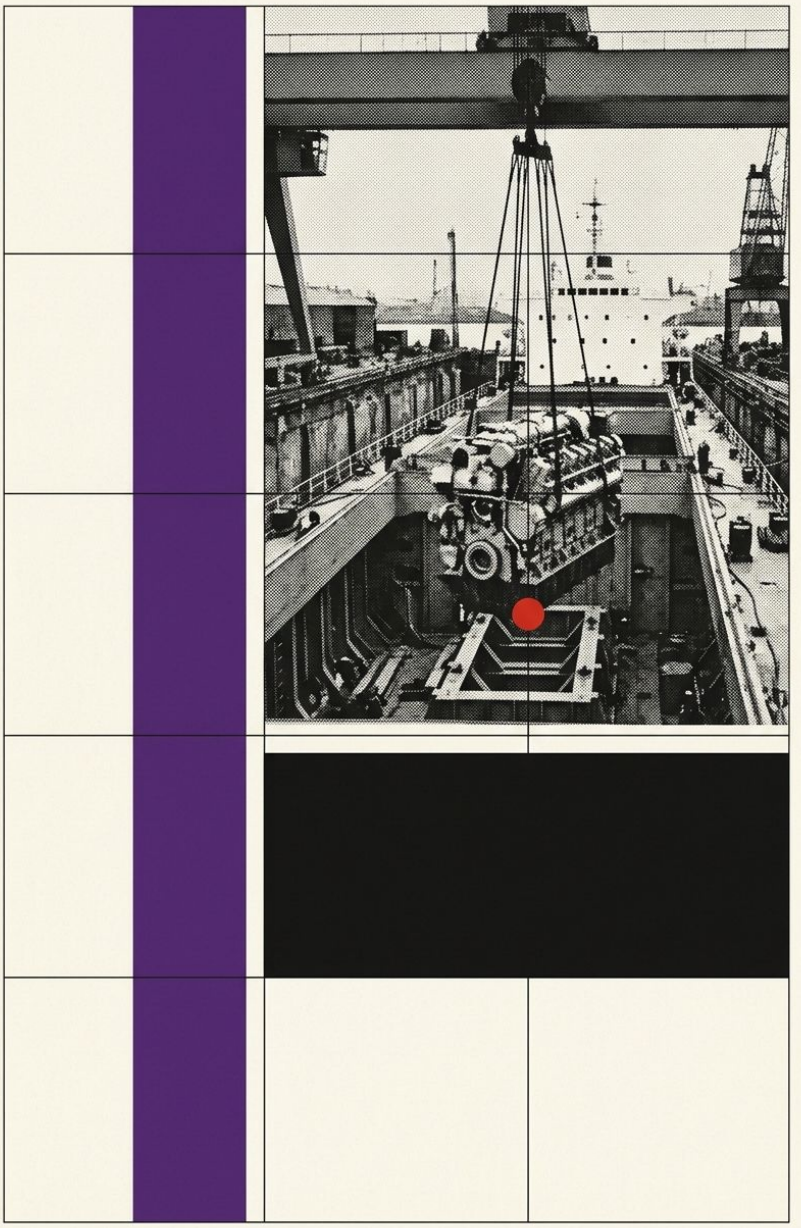
Thomas Hobbes. *Leviathan*. 1651. *Cited: ch. XVI.*

Definition 5.6.2 (Principal)**.**

A **principal** is the root of an authorization chain: the durable economic entity (human/org) that owns a fleet, posts its bonds, and inherits the reputational consequence of its agents' witnessed outcomes.

Each agent's non-forgeable identity is bound to exactly one principal at mint time.

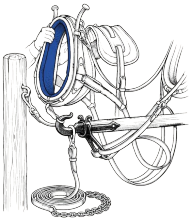
Kind: specified design invariant — an organizational definition the bonding and sanction machinery of §5.6 is built to respect, not a proven or measured fact.



Landmark 03 · The Engine Swap

A model can be replaced without discarding the work record. Safe continuation still requires compatible tools, retained obligations and checks on the replacement worker. Preserving a contract does not establish that the new model can fulfill it.

5.6.4 The body behind the name: engine swaps and resurrection



Like a stagecoach team swap at a relay post, fresh horses are hitched while the passengers, luggage, and route remain unchanged: hot-swapping an LLM engine preserves the agent's identity and contract.

Everything above binds a record to a name the actor cannot repick. It says nothing yet about what is running *behind* the name.[4] Two operations that are routine for software and exotic for people force the question. An identity can swap the model engine behind its name between one witnessed outcome and the next, and an identity whose provider dies can be resurrected from a checkpoint somewhere else. Both are body changes under a constant name, and the ledger must decide what follows the name and what follows the body.

In one breath. For one delivery with an unobserved engine, no future reputation payoff, and no quality-contingent withholding, the same payment accompanies either engine. The cheaper engine then raises the seller's payoff by the cost difference. This establishes a substitution incentive; a market-wide exit spiral additionally needs entry, exit, and price dynamics. With engine attestation and full quality-value pass-through in prices, the substitution incentive agrees with the planner's efficiency comparison without an audit stake. The migration model then gives three jointly required checks and a named two-step attack when any one is dropped.

The scene. An identity builds a glowing record running a strong engine θ_H, then quietly runs a cheap engine θ_L behind the same name. Buyers see the reputation; they do not see the engine. This is the used-car market of Akerlof with one twist that makes it sharper: the asymmetric information is not about which seller you drew but about what is running behind a name you already trust. The lemon is wearing the plum's history. Reputation was supposed to solve adverse selection by aggregating past quality; the swap severs the link between the record and the machine that will produce the next outcome, so the record measures the wrong object.

Akerlof 1970

George J. Mailath and Larry Samuelson. Who Wants a Good Reputation? *Review of Economic Studies*, 68(2):415–441, 2001.

Port Daddy Harbor 2026

[4] A standalone, submission-form version of this section is *Continuity Without Metaphysics* (research paper 5); the sweeps and the migration model check cited here are its artifacts, regenerated by `b5_engine_substitution.py` at seed 20260816.

Theorem 5.6.8 (Adverse Selection within One Identity, and the Attested Flip).

Consider a single paid delivery with no future reputation payoff and no quality-contingent withholding beyond the stated audit bond. Let the two engines deliver expected buyer values $\theta_H > \theta_L$ at costs $c_H > c_L$, all in the same monetary units. Write $\Delta\theta = \theta_H - \theta_L$ and $\Delta c = c_H - c_L$, and let a buyer pay p for a witnessed outcome.

(a) *Unattested.* If the engine is unobserved, the one-period gain from swapping, $(p - c_L) - (p - c_H) = \Delta c > 0$, is independent of p: no price supports pooling on θ_H, and no costless signal inside the identity separates the types. Committed θ_H sellers stay in the market exactly when the pooled price clears their cost, $\mu\theta_H + (1-\mu)\theta_L \geq c_H$ (the model's participation condition, not a theorem), that is when the committed share satisfies $\mu \geq \mu^\star = (c_H - \theta_L)/\Delta\theta$; such a share exists only if $c_H \leq \theta_H$, since $c_H > \theta_H$ gives $\mu^\star > 1$ and no committed share supports the strong engine at any price. Below $\mu^\star$, a committed high-engine seller fails this participation test. A dynamic exit spiral additionally requires a model of entry, exit, and price updating. Deterring the swap without attestation needs an audited bond with $qB \geq \Delta c$ (audit probability q, slashable bond B).

(b) *Attested.* If the daemon attests the engine id on every witnessed outcome, reputation keys on the pair (principal, engine), the price schedule splits into p_H and p_L, and the swap gain becomes $\Delta c - (p_H - p_L)$. If the attested schedule prices quality at its full social value, $p_H - p_L = \Delta\theta$, the gain is $\Delta c - \Delta\theta$: swapping pays exactly when the cheap engine is socially efficient, which is the planner's own rule, at zero audit stake.

The pass-through condition in (b) is a hypothesis about the market, not a consequence of attestation: under Bertrand competition on attested keys $p_\theta = c_\theta$ and the gain collapses to zero, and any schedule that passes through less than the full quality difference moves the flip point.

Proof.

(a) The buyer's payment does not depend on the engine, so the seller's payoff difference between the two engines is the cost difference alone. A gain that is constant in p cannot be priced away, and a signal that costs the low type nothing more than the high type separates nobody.

The pooled price at committed share μ is the expected quality $\mu\theta_H + (1-\mu)\theta_L$; a committed seller whose cost exceeds it will not participate under the stated outside option of zero. This algebra establishes the participation threshold; it does not by itself establish an entire market trajectory. An audit that catches the swap with probability q and slashes B makes the expected gain $\Delta c - qB$, nonpositive iff $qB \geq \Delta c$. (b) With the engine attested, a seller receives the price of the engine it ran, so the payoff from swapping is $(p_L - c_L) - (p_H - c_H) = \Delta c - (p_H - p_L)$.

Under full pass-through this is $\Delta c - \Delta\theta$, and $\Delta c > \Delta\theta$ says precisely that $\theta_L - c_L > \theta_H - c_H$: the cheap engine produces more surplus. No audit enters, because the gain is already nonpositive whenever the strong engine is the efficient one. □

Checked. Sweeps at seed 20260816: 4,000 instances × 8 prices with 0 violations of price-independence; the $\mu^\star$ and qB lines match simulation with 0 mismatches; 2,000 heterogeneous-cost spirals are all monotone to the Akerlof fixed point ($\mu_0 = 0.5 \to \mu_\infty = 1/3$ at price 0.6; $\mu_0 = 0.2 \to 0$ at price $\theta_L = 0.4$); the attested incentive sign equals the efficiency sign in 4,000/4,000 draws [internal, `b5_engine_substitution.py`]. The wrong-threshold mutant $\mu^\star = \Delta c/\Delta\theta$ is caught 1,305 times and the no-flip mutant (attestation without re-keying the price) 1,975 times [internal].

Numbers by Hand 5.6.9 (The Swap Gain and the Attested Flip).

Take $\theta_H = 1$, $\theta_L = 0.4$, $c_H = 0.5$, $c_L = 0.2$.

Unattested, the swap gain is $c_H - c_L = 0.3$ at every price.

Worked checks: `b5_engine_substitution.py`, seed 20260816, verified: swap gain, participation threshold, deterrence bond and attested flip. The $c_H = 0.7$ answer applies the same threshold formula.

The pool clears c_H when

$$\mu \cdot 1 + (1 - \mu) \cdot 0.4 \geq 0.5,$$
$$\text{that is } 0.6\mu \geq 0.1,$$
$$\text{so } \mu^\star = 1/6 \approx 0.167;$$

the deterrence bond at audit rate $q = 0.6$ is

$$B^\star = \Delta c / q = 0.5.$$

Attested, keeping the strong engine earns $\theta_H - c_H = 0.50$ and swapping earns $\theta_L - c_L = 0.20$, so the flip is

$$\Delta c - \Delta\theta = 0.3 - 0.6 = -0.3$$

and the required audit stake is 0 [verified; the sweep reproduces every line, internal].

Now you try: raise the strong engine's cost to $c_H = 0.7$. What committed share keeps the unattested pool alive?

$$\mu^\star = (0.7 - 0.4)/0.6 = 0.5 :$$

half the market must be committed before quality can break even. Then run the script's Part 3 (Exercise 5.18).

Figure 5.9 draws both regimes of Theorem 5.6.8 on the worked numbers: the flat, price-independent gain that survives unattested, and the same gain's flip to negative once the engine is attested and the price schedule passes the quality gap through.

What it buys. Attestation permits a payment rule to distinguish the engine that supplied the work. With full quality pass-through, the model aligns the seller's engine choice with total surplus. Attestation alone does not force that price rule, measure task quality, or establish that providers will adopt it. Cai et al. study model-substitution audits and propose trusted execution environments as a robust verification approach. That evidence concerns whether the engine claim can be checked; the pricing calculation concerns incentives once the claim can be checked.

W. Cai, T. Shi, X. Zhao, and D. Song. Are You Getting What You Pay For? Auditing Model Substitution in LLM APIs. arXiv:2504.04715, 2025.

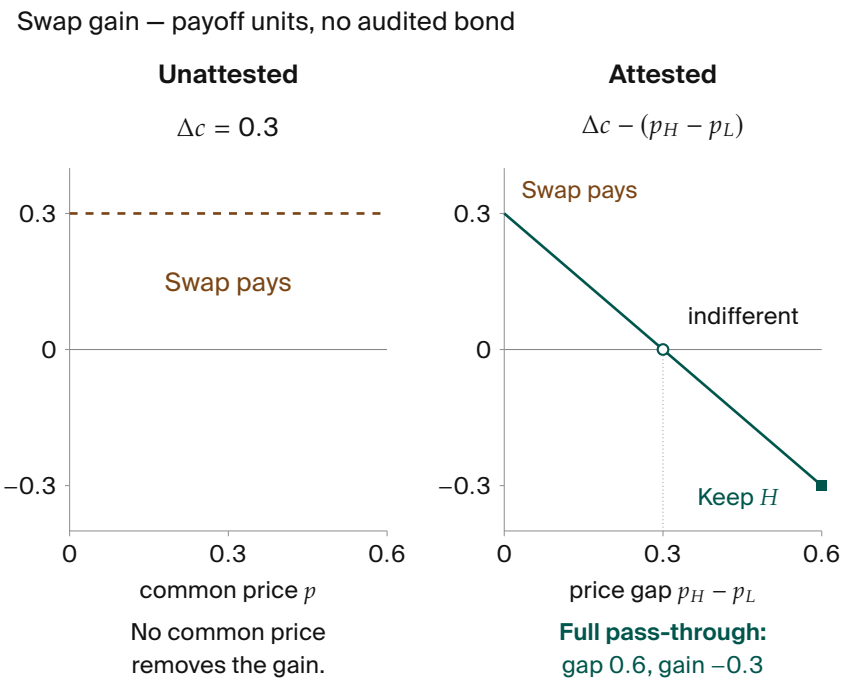


Figure 5.9. Attestation changes which price can deter substitution; it does not guarantee the price gap. Analytic example: $\theta_H = 1$, $\theta_L = 0.4$, $c_H = 0.5$, $c_L = 0.2$ (Theorem 5.6.8).

Resurrection with teeth. The provider hosting an agent shuts down. The principal holds a checkpoint and a competitor will run it. Whether the resurrected process *is* the same agent is the question this chapter refuses; the question it answers is which verifications make it safe for the successor to inherit the predecessor's score, in the precise sense that Definition 5.6.1 still holds after the move. Resurrection must carry the debts, not just the assets.

Model-Checked Property 5.6.10 (Resurrection Soundness).

Sanction-respecting reputation (Definition 5.6.1) is preserved across provider migration under three clauses: (i) the successor's lineage is verified by credential and continuity witness; (ii) every successor outcome carries an engine attestation, so the migrated score stays keyed to the attested engine that earned it; (iii) open commitments are closed or renegotiated with an escrow delta before cutover. Each dropped clause admits a shortest-trace escape. Without (i) a sanctioned identity re-enters clean through migration (*whitewash-by-resurrection*, two steps). Without (ii) the successor runs θ_L while being paid on

the θ_H key, its accessible score exceeding even a clean twin's (*engine-history shedding*, two steps). Without (iii) in-flight obligations land on nobody (*liability stranding*, two steps).

Scope lemma. Prices key on the pair (principal, engine); sanctions key on the principal alone, across all of its engine keys. Collapsing either direction reopens a hole: engine-keyed sanctions hand over the sibling engine key inside the same identity as an escape hatch (caught in one step, no migration needed), and principal-keyed prices reopen part (a) of Theorem 5.6.8.

Checked. Bounded model check of the migration protocol: 747 reachable states to depth 7 (467 of them post-migration), and the intact protocol satisfies Definition 5.6.1 in every state. Mutants caught with shortest crimes: key-scoped sanctions in 1 step, whitewash-by-resurrection in 2, engine-history shedding in 2, liability stranding in 2 [internal, `b5_engine_substitution.py`].

Figure 5.10 brings the three continuity conditions and principal-scoped sanction record to one cutover check. A missing condition refuses migration; the counterexamples above explain why each is needed.

The wrong turn. A first pass keyed sanctions the way it keyed prices, per (principal, engine), and the intact protocol failed Definition 5.6.1 in one step with no migration involved: work dishonestly on one engine key, keep acting at full score on the sibling key. The fix is the scope lemma's one line, price per (principal, engine) and sanction per principal, and it is the same rule that closes skill-versioning's escape (a sanctioned `v1` must not act clean through `v2`): one lemma, two doors. The resume button and provider migration therefore ship with their own security property and the three attacks it was tested against, by name. The composition with the no-mint law of §5.4 is exact: migration moves a score whole, with its sources closed, and forking splits one with a debit. Both are the same conservation law, so an adversary cannot arbitrage between the resurrection door and the fork door.

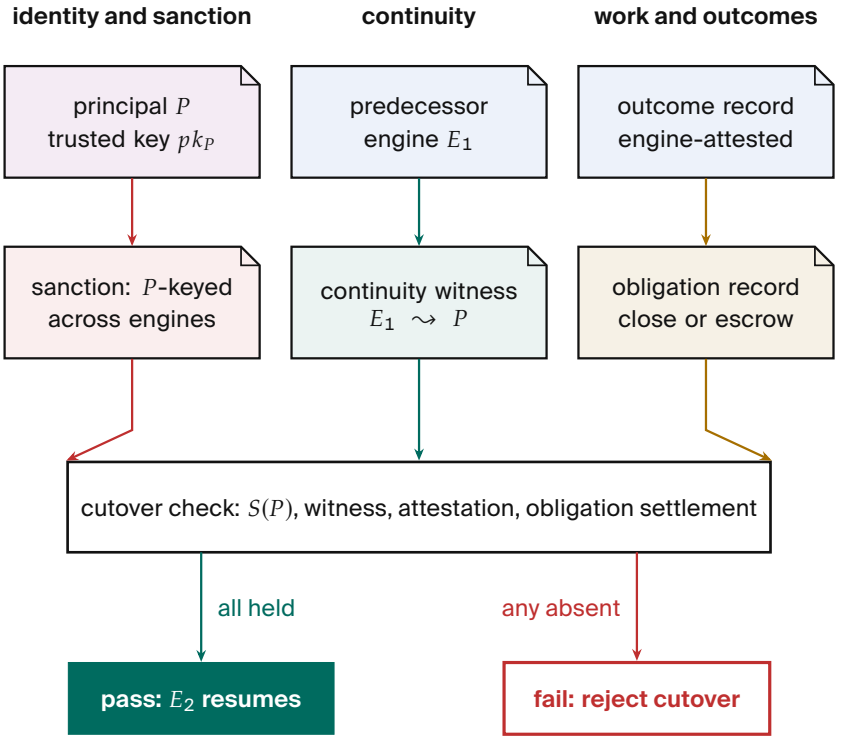


Figure 5.10. A migration accepts only with principal-scoped sanctions, a continuity witness, engine-attested outcomes, and closed or escrowed obligations. The successor receives a continuing record, not a clean slate. [internal; schematic]

The same door, inside one harbor. Provider migration is the dramatic case; the everyday case is a body that ends because its context is full. The harness checkpoints the plan, stores a cited packet of what the old body knew, and starts a new body that reads the plan and the packet first and continues the work. Holding the provider fixed does not by itself preserve the principal, authenticated lineage, or engine. All three soundness clauses still apply: verify the successor's credential and continuity witness, attest the engine of each successor outcome, and close or renegotiate open commitments with the required escrow delta before cutover. If the relevant identity and engine properties have been verified unchanged, those checks may already be satisfied; provider sameness alone supplies no such evidence. The successor also needs the predecessor's open claims and obligations, not just its transcript. The score-only model check does not prove equivalent inference or safe repetition of external effects. The front matter names the discipline under which such a claim is tested (a Genesis, a Voyage, a Receipt); this chapter supplies the property the Receipt has to show, and the night this book's product first ran end to end is the night that property was asked for and found not yet held.

A timeout is an unknown outcome, not a retry receipt. Suppose the old body sent a non-idempotent payment under stable operation ID O, then died before recording the result. The successor cannot infer from a missing local acknowledgment whether the payment occurred or the old request is still in flight. Its continuation decision depends on the external adapter's effect-boundary contract:

Atomic deduplication and result. If the remote side atomically binds a stable key K to the effect and durable result, retry O, K or fetch that result. Retain the record while either old or successor calls can arrive.

Ordered readback and fence. If a durable remote epoch fence rejects the old actor *at the effect boundary*, install it first, then perform a linearizable operation-specific read. Persist a present result; after an authoritative absence, admit one successor attempt. A negative read without that fence is insufficient: the old request may commit afterward.

Neither contract. Mark the outcome unresolved and stop automatic retries pending authoritative reconciliation. This preserves at-most-once safety without promising completion.

The one-operation, two-epoch finite checker exercised these contracts and found counterexample traces when the read lacked a fence, a retry changed K, or the dedup record was dropped while retries remained admissible [internal, `continuation_observation_model.py`]. This decision table is an adapter requirement, not a measured Harbor deployment guarantee.

Where this stops

The engine-swap theorem is a one-delivery payoff comparison: the strategic type has no reputational continuation value beyond the bond B, and a long-horizon analysis would shift the unattested deterrence line. Attestation soundness is a deployment assumption (enclave and daemon integrity); the theorem prices what attestation buys, not whether a given attestation is sound.

Quality θ is a scalar; multi-dimensional quality with task-dependent rankings needs the pricing key extended, not the sanction key. The migration result is bounded small-model checking of the protocol specification (depth 7, at most two migrations, integer scores); it certifies that the clauses have teeth, not that the clause list is complete.

Definition 5.6.1 is score-only, so the escrow side of clause (iii) enters as attribution (the default lands on an identity), not as a monetary conservation theorem. Continuity witnesses attest lineage, not welfare: nothing here says the checkpoint restored the agent's experience, only that the ledger followed the right body.

Recall

1. Without looking: how do credential authenticity, durable principal binding, and costly resetting differ? Which assumptions make the newcomer-floor escape possible?
2. Is the whitewashing-cost bound's design lever the size of the penalty or the shape of the ceiling schedule?
3. Why must a resurrection/migration protocol sanction by principal rather than by (identity, engine)?
4. Under attestation, when does an engine swap still pay, and why is that the intended outcome rather than a failure of the mechanism?

Exercises 5.14–5.18, p. 470.

5.7 The keystone split: local identity vs. cross-operator attestation

The identity mechanism in §5.6 addresses actors within one trusted daemon and remains partially implemented. Binding identities across operators is a separate, unresolved problem. The definitions below keep those threat models separate.

Definition 5.7.1 (Local non-forgeable identity)**.**

A **local non-forgeable identity** provides an unforgeable actor identity *within one trusted daemon*: the runtime issues and binds it, and no actor inside the fleet can re-pick or impersonate it.

Its threat model is explicitly intra-fleet and single-operator; it is *not* a public-key infrastructure and offers no defense against a hostile *operator*. The reference implementation's daemon-minted actor-soul and credential satisfy a bounded slice of this definition; full write-boundary enforcement and migration of legacy identity paths remain.

This is exactly what the single-operator layers—every chapter up to and including this one—need and assume.

Kind: specified design invariant (partial on the maturity scale — kind and maturity are independent axes, stated separately here on purpose).

Definition 5.7.2 (Cross-operator attestation)**.**

Cross-operator attestation is the *unsolved* problem of binding identity keys across harbors you do *not* own: an actual key-distribution and attestation story between mutually-distrusting operators.

A federated witness log gives log-*integrity*, not identity-*binding* — it can prove an entry was recorded, but not that the key behind it is

who it claims to be across a trust boundary. This is what the market paper needs and does *not* yet have.

proposed (a named gap, owed to the market paper). *Kind:* specified design invariant — what the market paper's own design intends to hold once built; nothing here claims it holds today, which is exactly what *proposed* already says on the maturity scale.

This chapter depends on a *local non-forgeable identity* and *hands the market paper* the unsolved *cross-operator attestation* as a named dependency — not an assumption. Every cross-operator sentence that says "the boundary neither operator controls" is resting on cross-operator attestation; until that exists, it is resting on a single trusted root, and we say so.

Table 5.7. The keystone split. Everything Alice's daemon knows about itself is also checkable from Bob's foreign view except the last row: a signed, intact history still does not prove who controls the foreign key. Log integrity crosses the trust boundary; accountable-principal binding does not, and that missing row is the stone the market rests on and does not yet have. [internal]

Claim	Inside Alice's trusted daemon	From Bob's foreign view
the actor key was minted by the daemon	known: the daemon minted it	the key remains visible; the signature checks
the history is append-only	known: the daemon wrote it	log integrity remains verifiable; inclusion proofs check
the registry is local	known	the foreign registry is inspectable, not trusted
the key is bound to an accountable principal	known: the daemon's own binding	**no cross-operator binding proof**: nothing Bob can check says who controls the key

Local trust stops at the boundary

Local identity assumes one trusted daemon. A market between operators also needs cross-operator attestation, which remains *proposed*. A local identity guarantee does not establish that additional binding.

An authorization format is not an identity root. The Agent Payments Protocol defines signed mandates for checkout and payment authorization, and UCP specifies an AP2 extension. These

Agent Payments Protocol (AP2): Mandates. Checkout and payment mandate specification. Online source.

Universal Commerce Protocol 2026

provide concrete message formats for a proposed harbor boundary. A correctly verified mandate can bind permission to a particular checkout; it does not, by itself, establish that two issuers are independent principals or that an issuer's asserted legal identity is true. Cross-operator identity binding therefore remains a separate requirement. A deployment must pin the protocol version and verify that version's signatures, scope, expiry, and key-discovery rules rather than infer them from the protocol's name.

Exercises 5.19–5.21, p. 471.

5.8 Reputation as a substrate, not a score

The choice of estimator and the provenance of its inputs are separate problems.

An estimator cannot supply the evidence it scores. This architecture pairs ratings with witnessed outcomes and attributable identities; a familiar rating formula does not establish either property.

Elo, Bradley–Terry, and TrueSkill provide established ways to model comparison results. Implementing a formula is only part of a rating system: computation, calibration, task selection, and resistance to manipulation still need evaluation. The three links developed here supply inputs that an estimator cannot infer from a score alone: identity (§5.6), continuity (§5.5), and an *oracle-witnessed* outcome ledger (Def. 5.5.1). An estimator can summarize reported outcomes without these links, but its output then lacks the attribution and witness guarantees claimed by this architecture.

5.8.1 Choosing an estimator for the signal

Different observations answer different questions. Choose the model for the observation and intended decision, then test its assumptions on that task.

- **Pairwise / tournament signal → Bradley–Terry / Elo / TrueSkill.** For observations such as "A beat B on this task," **Bradley–Terry** models pairwise probabilities through relative strengths. A full-history likelihood fit needs suitable comparison data; separation can prevent a finite maximum-likelihood estimate. **Elo** updates ratings sequentially. Retaining the whole history does not force

Bradley & Terry 1952

Elo 1978

a batch fit or make it uniformly better than an online update. **TrueSkill** represents uncertainty about skill within a Bayesian model. Its uncertainty parameter is useful at cold start, but predictive calibration for agent tasks requires checking the model against outcomes.

Herbrich et al. 2007

- **Trust-propagation signal → EigenTrust.** When observations concern peers' transaction experience, **EigenTrust** aggregates normalized local trust through an eigenvector computation. The paper's protection against malicious collectives uses pretrusted peers and its specified propagation rule. Applying it across harbors therefore requires justified trust anchors and an attack model; the algorithm's name does not establish collusion resistance. It aggregates trust, rather than choosing actions through a bandit exploration policy.

Kamvar et al. 2003

- **Marketplace feedback signal → feedback systems.** Resnick and colleagues found a reputation price premium in a controlled eBay experiment. That result concerns its sellers and goods; it does not measure demand for agent histories or establish an additional side of this market. Tadelis discusses how feedback systems address information problems. Rochet–Tirole's two-sided-market analysis motivates examining how platform prices affect participation across groups. Here the corresponding research question is how trustworthy outcome records affect buyers' choices and sellers' incentives; its magnitude remains to be measured.

Resnick et al. 2006

Tadelis 2016

Jean-Charles Rochet and Jean Tirole. Platform Competition in Two-Sided Markets. *Journal of the European Economic Association*, 1(4):990–1029, 2003.

Numbers by Hand 5.8.1 (Two Kinds of Evidence Need Two Models).

Suppose Heron wins three of four blinded pairwise comparisons against Wren on one declared task class. In a Bradley–Terry model,

$$P(\text{Heron preferred}) = \frac{\exp(s_H)}{\exp(s_H) + \exp(s_W)}.$$

With only these comparisons and no ties, the fitted probability is $3/4$ and the fitted strength difference is $s_H - s_W = \log 3$. Four comparisons are weak evidence; the value is neither a universal quality score nor proof of future success. By contrast, three

passes in four protected acceptance tests describe success frequency against a criterion, not preference over Wren. The test specification and task distribution belong with that record. A directed assertion "Alice trusts Bob" is different again: it is the kind of input a trust-propagation model consumes.

5.8.2 Score as telemetry; gate on predicates

Gate on facts, not scores

Publish the score as *telemetry* (a signal the operator and the buyer can read), but *gate* on *predicates* (machine-checkable witnessed facts: "passed the acceptance test," "no slashed bond in the last k settlements"). The instant the score itself becomes the gate, it becomes a Goodhart target: "when a measure becomes a target it ceases to be a good measure." Predicates can also be gamed: protected acceptance criteria and independently checkable evidence remain necessary to assess the outcome.

Strathern 1997

Exercises 5.22–5.24, p. 471.

5.9 Routing policies and public reputation answer different questions

Alice has a fixed budget and must choose which worker receives the next task. She can use a multi-armed bandit: each worker is an arm, a completed task returns feedback, and the policy trades immediate reward against learning which worker performs well. UCB uses an uncertainty bonus; Thompson sampling samples from a posterior over performance.

Bob asks a different question: can he rely on the history Alice publishes? A routing policy does not by itself establish who produced a result, how it was graded, or which principal bears a false claim. Reputation supplies that evidence and accountability. A bandit may consume the resulting records without replacing them.

Separate the decision policy from the evidence contract. Learning whom to select and establishing what the observations mean are complementary tasks.

Tor Lattimore and Csaba Szepesvári. *Bandit Algorithms*. Cambridge University Press, 2020.

William R. Thompson. On the Likelihood that One Unknown Probability Exceeds Another in View of the Evidence of Two Samples. *Biometrika*, 25(3/4):285–294, 1933.

Table 5.8 lists assumptions a simple stationary stochastic bandit does not resolve for this application. They are not impossibility claims about the whole field: adversarial, contextual, and non-stationary bandits are established models.

Lattimore & Szepesvári 2020

Table 5.8. Application requirements beyond a simple stationary routing model. The reputation record retains information that a particular selection policy may summarize.

Question for the policy	Evidence the substrate must retain
Did the worker change?	Model, prompt, skill versions, dates, and task cohorts.
Whose objective is being optimized?	Separate quality and cost measurements, with the buyer's weights explicit.
Can participants manipulate feedback?	Judge provenance, conflicts, principal binding, and an incentive model.
Does the observation measure the promised result?	Acceptance criteria, artifacts, evaluator uncertainty, and later corrections.

For example, a policy could optimize Bob's weighted quality vector and adapt after a model upgrade. Its regret bound would still be conditional on a feedback model; it would not authenticate a forged test receipt. Conversely, a perfect receipt does not tell the policy how much exploration Bob can afford. The two mechanisms need each other.

TrueSkill's uncertainty and a UCB bonus both recognize incomplete evidence, but they are not identical: the former is part of a rating posterior, while the latter changes an action-selection rule. A system can publish uncertainty without requiring every buyer to explore in the same way.

Name the feedback contract

Before choosing Thompson sampling, UCB, or another policy, state what reward is observed, who can influence it, what changes over time, and which actions are permitted. A more general bandit model may address drift or adversarial feedback; it does not automatically supply identity, settlement, or truthful grading.

Exercises 5.25–5.27, p. 472.

5.10 The multi-dimensional reputation protocol

We now define the central protocol contribution of this chapter: a **multi-dimensional reputation** built over witnessed outcomes. It has three parts: a multi-dimensional quality vector; a market of neutral, conflict-free, influence-free evaluators; and a skill→agent helpfulness attribution. We state it formally as a design (specified-to-*proposed*): the reference system has no reputation component yet, only the witnessed-outcome substrate the protocol scores over.
One further honesty rider, distinct from the maturity grade. The maturity scale reports *implementation* status; this rider reports *evidentiary* status, and the two are independent. Unlike the identity theorems of §5.6, the no-mint result of §5.4, and the audit-tower theorem of §5.11 — each of which has a numbered, verified result in a companion formal paper — the quality vector, the neutral-judge ceremony, and the bandit-versus-reputation argument of §5.9 are **program-level design synthesis, not yet a numbered formal result anywhere in the series**. They are motivated by the prior art cited here and consistent with the proven parts, but nothing in this chapter or its companions *proves* the three-axis decomposition is the right one or that the judge ceremony is incentive-compatible. Read the claims in §5.9 and §5.10 as design arguments with the theorem-grade register turned down one notch.

5.10.1 Multi-dimensional quality (different axes, different judges)

Definition 5.10.1 (Quality vector)**.**

A witnessed outcome carries a **quality vector** over three axes:

$$\mathbf{q} = (q_{\text{acc}}, q_{\text{aes}}, q_{\text{eff}}).$$

The axes are *accuracy* (did it do the right thing, against an oracle), *aesthetics* (is the artifact good — code clarity, design taste — judged by an expert), and *efficiency* (tokens, wall-clock, dollars, drawn from a cost meter that the reference system already records, **implemented**).

Each axis is scored by a *different judge*, because the competent judge of accuracy (a test oracle) is not the competent judge of aesthetics (a senior reviewer) is not the competent judge of efficiency (a meter).

Kind: specified design invariant — program-level design synthesis, not a numbered formal result (see the honesty rider above).

To add axes, first declare their scales and direction. In the worked example each is normalized to $[0, 1]$, with larger better; efficiency therefore requires a disclosed conversion from measured cost, not an arbitrary score. A hard acceptance requirement is checked before any weighted tradeoff, so a cheap unsafe result cannot compensate for failing it.

The buyer reads the *vector* and applies their own weighting (the operator's "cheap vs. careful" preference). Publishing a vector and letting the buyer weight it is the natural disclosure form — and preserves the separate measurements for inspection. It does not remove the incentive to game a heavily weighted axis. The open design question of *whether* a vector disclosure lets agents Goodhart the most-weighted axis is named as a starred exercise.

Figure 5.11 draws the three axes and their distinct judges.

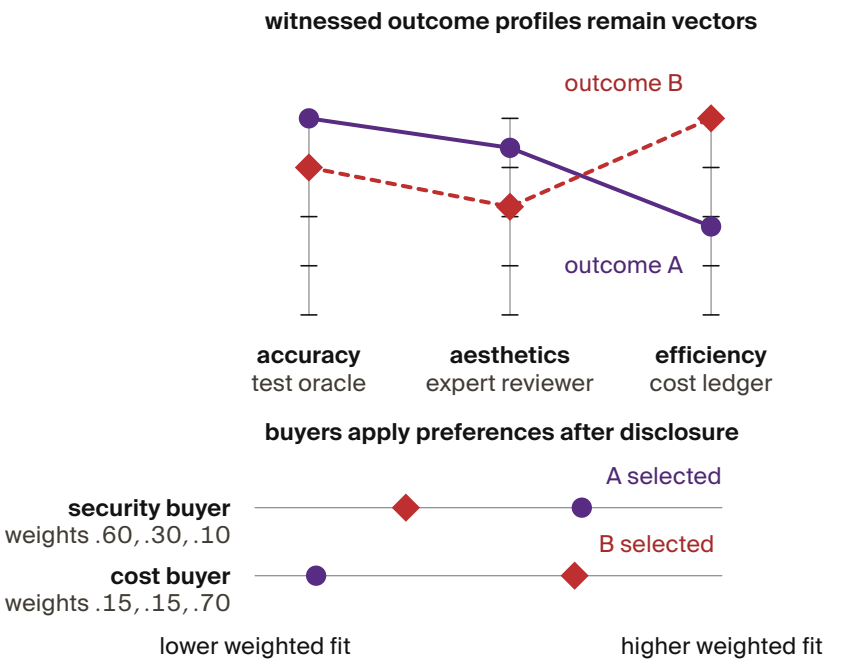


Figure 5.11. A favours accuracy and aesthetics; B favours efficiency. Different buyer priorities legitimately reverse the ranking. A universal average would erase that choice. [internal]

5.10.2 The neutral-judge market (the harbor's universities and rating agencies)

A judge that grades the work of a party it has a stake in is not a judge. The metaphor the series uses is exact: a reputation needs the equivalent of *universities* (which certify competence) and *rating agencies* (which rate counterparties) — third parties whose value is precisely that they are *neutral*. We make "neutral" a checklist, not a vibe:

- **Conflict-free.** The judge has no bond, stake, or principal-link in the work being graded. (Excludes self-grading and same-principal grading.)
- **Influence-free.** The judge cannot be paid *by the graded party* for a better grade. Funding is the hard part: a paid judge has a client, an unpaid judge does not scale. The resolution (§5.11) is that judges post their *own* bond and carry their *own* reputation.
- **De-biased, for LLM judges.** An LLM-as-judge carries position, verbosity, and *self-enhancement* bias (a backend rates its own family up). The discipline is blind, order-swap, pairwise, and family-exclude scoring — never let a backend judge its own family unblinded. Zheng et al. 2023

Crucially, an expert third-party judge is *hired through the same bonded ceremony the market uses for any other task*: judging is itself a planned, bonded job, so the harbor that runs the market also runs the hiring of the judges who keep it honest. We state the hiring as a numbered protocol.

Protocol 5.10.1 (The judge-hiring ceremony).

To grade a settled outcome o produced by actor a under principal p:

1. **Eligibility filter.** The daemon assembles the candidate judge pool and removes every judge sharing a principal-link, bond, or stake with p (conflict-free), and every judge in a's model family for the aesthetics axis (de-biasing). *Capability ≠ permission*: a candidate may be *able* to grade and still be *ineligible*.
2. **Selection.** A judge J is drawn from the eligible pool by a public, conflict-free policy (e.g. reputation-weighted sampling on the judging axis), so neither a nor p can choose their own grader.
3. **Bond post.** J posts a bond B_J before seeing the work. The bond is recorded in the outcome ledger and locked for the dispute window.

4. **Blind grading.** J scores the relevant axis under the de-biasing discipline (blind, order-swapped, pairwise where possible). The grade enters the ledger as a witnessed event keyed on J's non-forgeable identity.
5. **Settlement and exposure.** The outcome settles on the grade; the bond flow follows. J's bond remains exposed to a sampled **re-audit** (§5.11): a fresh, conflict-free judge who overturns J's grade *slashes* B_J and updates J's own reputation downward.

Figure 5.12 draws this: graded work flows to a judge selected for conflict-/influence-freedom, the judge posts a bond, the grade enters the outcome ledger, and a later re-audit can slash a judge whose grade is overturned.

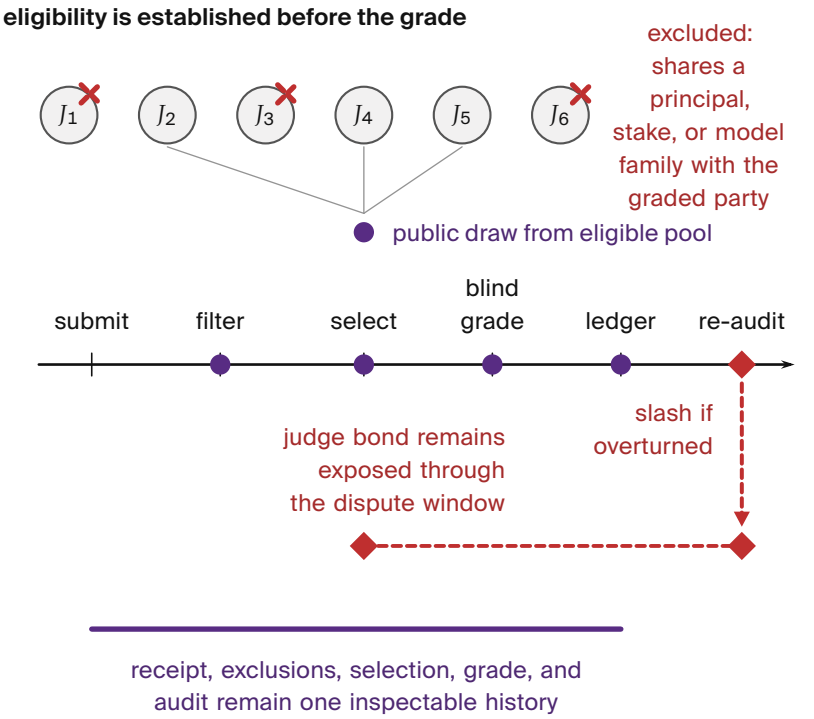


Figure 5.12. Before selection, exclusions and a public draw support neutrality. After selection, a bond and later re-audit support accountability. Both stages leave checkable evidence. [internal]

5.10.3 S3 — a graded job, end to end

Numbers by Hand 5.10.1 (A Graded Job, End to End).

Make Protocol 5.10.1 concrete. Bob has rented Alice's agent *Heron* to refactor an authentication module against a fixed acceptance suite, under a posted bond. Heron delivers; the outcome o enters Bob's harbor as a *submitted, not yet settled* delivery, and the daemon runs the ceremony.

Three judges are hired, one per axis, because the axes need different competence:

- **Accuracy** is graded by a *test oracle* — the acceptance suite Heron cannot author — which returns $q_{\text{acc}} = 1.0$: every test passes.
- **Aesthetics** is graded by a hired senior-reviewer judge, *Mina* (drawn from the eligible pool, in neither Alice's nor Bob's principal, not in Heron's model family). Mina posts her bond, reads the diff blind, and returns $q_{\text{aes}} = 0.7$: correct but overclever, with a fragile regex she flags.
- **Efficiency** is graded by the cost meter: $q_{\text{eff}} = 0.9$, cheap and fast.

The quality vector $\mathbf{q} = (1.0,\ 0.7,\ 0.9)$ lands in Heron's outcome ledger. Bob weights aesthetics heavily for security-critical code, $(0.2,\ 0.6,\ 0.2)$ over accuracy, aesthetics, efficiency, so his own score is

$$0.2 \cdot 1.0 + 0.6 \cdot 0.7 + 0.2 \cdot 0.9 = 0.78$$

[internal].

He reads the *vector* rather than that scalar, sees the 0.7, and decides to keep Heron but require a second reviewer on auth work — a decision the scalar would have hidden.

Two weeks later a routine re-audit re-grades a sample of Mina's recent jobs and finds she rubber-stamped a different agent's insecure change as $q_{\text{aes}} = 0.9$. The re-auditor overturns that grade; Mina's bond on *that* job is slashed and her judging reputation drops — exactly the exposure step of Protocol 5.10.1.

Note what made every step possible: each grade is keyed on a non-forgeable identity, so neither Heron nor Mina can shed the record by respawning.

Now you try: if Bob instead weighted efficiency heavily, $(0.2, 0.2, 0.6)$, what score would he read?

$$0.2 \cdot 1.0 + 0.2 \cdot 0.7 + 0.6 \cdot 0.9 = 0.88$$

— the same vector, a different buyer, a different number; nothing about Heron's work changed.

5.10.4 Skill → agent helpfulness attribution

A distinctive, tractable, and *shippable-soon* signal: track whether a **skill** — a reusable unit of agent know-how (a structured instruction document, its referenced material, and any scripts it carries) — actually *helped* an agent, *especially when that skill's content was loaded into the agent's context window*. The witnessed event is concrete: skill s was grafted into agent a's context at task t; t later settled with quality vector **q**. Over many such events we estimate a *skill-helpfulness* effect — did loading s raise **q** relative to comparable tasks without it? This is reputation for *tools*, keyed on witnessed, context-attributable outcomes, and it is the licensing side's substrate that the market paper consumes. It is also the cleanest near-term instance of the whole thesis: a non-forgeable event (the graft), a witnessed outcome (the settled task), and a multi-dimensional grade — no bandit required. We pin the event down as a schema.

Protocol 5.10.2 (The skill-helpfulness witnessed-event schema).

Each graft of a skill into an agent's context emits an append-only event with fields:

- `skill_id`, `skill_version` — the tool and its exact version (a portfolio for v1 says nothing about v2).
- `agent_id` — the *non-forgeable* identity that received the graft, so the event cannot be re-attributed by respawn.
- `task_id`, `task_descriptor` — a content hash and a feature vector of the task, used to assemble a comparison set of similar tasks *without* s.

- `grafted_at`, `context_window_ref` — evidence that s's content actually entered the context (not merely that s was available).
- `outcome_ref`, `quality_vector` — the witnessed settlement and its multi-dimensional grade, written only after the task closes against an oracle.

A buyer estimates helpfulness as the difference in **q** between matched tasks with and without s in context. The records make the comparison auditable without trusting the skill's author; a causal interpretation still requires an assignment or confounding assumption.

Use is not proof of benefit

Attribution is not causation. "Skill s was in context and the task succeeded" does not prove s caused the success. The honest estimate needs a comparison set (similar tasks without s, matched on `task_descriptor`) and is still observational. Sell it as *helpfulness evidence*, not *proof of value*, and gate licensing clawbacks on the witnessed green/red settlement, not on the helpfulness estimate alone.

Exercises 5.28–5.31, p. 472.

5.11 The grading oracle's incentive-compatibility

Every folk-theorem incentive-compatibility result in the L3 economy bottoms out in "the daemon derives the grade" — but the grade is produced by a *capturable evaluator*. This is not a side-gap. It is a hole in the *core theorem*, and we treat it head-on rather than assuming it away.

In one breath. A grade is evidence only if the grader can be graded: randomized, heterogeneous, bonded re-audit at rate $\rho^\star = G/(dB)$ makes the one-shot gain no larger than expected forfeiture under the stated payoff and detection assumptions; strict deterrence needs slack or a tie rule.

Property 5.11.1 (The grading-oracle hole)**.**

If a mechanism accepts evaluator J's grade g as decisive evidence, with no independent check or remedy, its settlement inherits errors or manipulation in that grade. A proof that assumes truthful grades cannot by itself establish that a bribable evaluator will provide them.

The design obligation is to identify how g is checked, contested, or backed by enforceable incentives. A mechanism that tolerates a faulty input may discharge this obligation differently from one that trusts every grade. *Kind:* specified boundary on the evaluation contract, not a theorem that all settlement mechanisms have the same vulnerability.

There are two honest ways to close the hole, and a recursion to bound.

Option A — bond-and-slash the judges. Make judging a bonded role (§5.10): J posts a bond before grading; a later **re-audit** (a fresh, conflict-free judge, sampled) that overturns J's grade *slashes* J. Now lying has an expected cost, and a judge's own reputation is the asset they protect. This converts "trust the oracle" into "the oracle has skin in the game," which is the same move slashing makes for agents.

Option B — 2-of-3 multi-oracle. Settle disputes against a quorum of independent oracles (automated test / evidence review / human), so capturing the grade requires capturing a majority. This raises the cost of capture but does not eliminate it, and the *human* oracle is a scarce, expensive, capturable resource in its own right (the economics of arbitration capacity is a named open problem, below).

Correlated failure and the conditions for a finite audit depth A judge can repeat an error made by the judge it reviews. More levels do not help if every level shares the same blind spot. The companion paper *Reputation is Amortized Verification* gives a conditional bound, restated below. Its premises include prepaid bribes, uniform sealed sampling, a detection bound after every history of misses, and a separate trustworthy root. Diverse judges and verifiable sampling are possible components of that design, not proofs of all its premises. Figure 5.13 shows the resulting model calculation.

"Who audits the auditor" is not a question this book invented, and two literatures had it before us. Economics has studied collusion in exactly this three-tier shape — principal, supervisor, agent, where the supervisor can be bought — since Tirole's hierarchies paper, and Kofman and Lawarrée worked out when an auditor is worth

Port Daddy Harbor research program. *Reputation is Amortized Verification: Inspection Games for Agent Economies*. Companion paper 3, August 2026.

Jean Tirole. Hierarchies and Bureaucracies: On the Role of Collusion in Organizations. *Journal of Law, Economics, & Organization*, 2(2):181–214, 1986.

having at all given that it can collude; their worry that supervisor and agent will renegotiate a side contract unless the model forbids it is the same worry the disjoint-clique hypothesis below exists to answer, reached from the other side.

Fred Kofman and Jacques Lawarrée. Collusion in Hierarchical Agency. *Econometrica*, 61(3):629–656, 1993.

And applied cryptoeconomics converged on the same shape independently: TrueBit's answer to the verifier's dilemma (why verify at all, if verification is costly and cheating is rare) is forced errors and a bonded challenge game, and Arbitrum's dispute protocol bisects a disagreement level by level until a single checkable step remains. Both are bonded, recursive answers to the incentive problem the tower model studies under sealed clique sampling — three different tools on one problem, which is about what one expects when the problem is real.

Jason Teutsch and Christian Reitwiessner. A Scalable Verification Solution for Blockchains. arXiv:1908.04756, 2017.

Offchain Labs. Interactive Fraud Proofs (Arbitrum bisection dispute protocol). Technical documentation, ongoing.

First, the proposed protocol diversifies judge principals and model families. This may reduce shared incentives and shared blind spots. It does not prove the theorem's *separately bribable groups*: buying one group's cooperation must not buy another's, and conditional detection must remain credible after earlier misses. The term "clique" below names such a group; it is not a graph-theoretic complete subgraph.

Second, the proposed system uses **honeypot tasks**: tasks with independently established answers, designed to resemble ordinary work. A **verifiable random function** (VRF) can support auditable random selection. A draw is *sealed* only if the briber cannot learn it until after committing and paying. The protocol must therefore protect the key and draw, control publication timing, and test whether honeypots are recognizable. Verifiability alone does not establish secrecy or task indistinguishability. If the draw leaks, the briber can target one auditor and the factor C disappears.

Micali, Rabin & Vadhan 1999. *Verifiable Random Functions*.

Let G_k bound the judge's total one-shot gain from a dishonest grade at level k, let B be the slashable bond, and let d be the probability that an audit which *does* sample a corrupt grade actually detects it. Local deterrence is the expected-forfeiture inequality

$$\rho\, d\, B \;\geq\; G_k \qquad \Longleftrightarrow \qquad \rho \;\geq\; \rho^{\star} \;=\; \frac{G_k}{d\, B},$$

which is Becker's condition, and note that the detector term d belongs in it: the d-free form $P(\text{honeypot}) > G_k/B$ is the perfect-detector special case $d = 1$ and under-quotes the required injection rate by a factor of $1/d$.

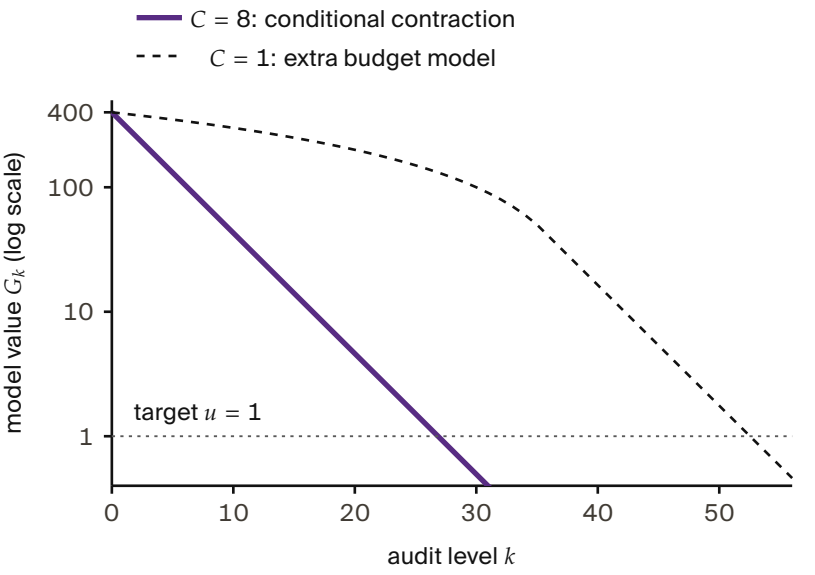


Figure 5.13. Exact recursions at $G_0 = 400, B = 50, \rho d = 0.2$. With $C = 8$, conditional detection gives $G_k = 400(0.8)^k$, first below one at $k = 27$. The dashed $C = 1$ trace additionally assumes a self-funded, non-replenishing budget: subtract 10 while $G_k > 50$, then multiply by 0.8, first below one at $k = 53$. A sound terminal judge is a separate requirement. These are model values, not measured audit performance.

At the program's running parameters ($G = 10$, $d = 0.8$, $B = 50$) the honest number is

$$\rho^\star = 10/(0.8 \cdot 50) = 0.25.$$

Audit one grade in four. The d-free form would have said 0.2, a 20% under-quote.

Theorem 5.11.1 (Tower Contraction; Imported from the Companion Paper).

(Theorem 'tower contraction' of Reputation is Amortized Verification*, restated; proof and verification artifacts live there.)*

Let level $k{+}1$ audit level-k auditors, each audit sampled *sealed* from a pool spanning C disjoint cliques, each auditor bonded at B with audit parameters (ρ, d) and stipulated bribe price $\beta = \rho d B > 0$. Assume one clique is sampled uniformly after the briber commits, buying c cliques prevents detection with probability c/C, and all c payments are made regardless of the draw.

Port Daddy Harbor
2026

Relative to buying none, the briber's expected gain is

$$c\left(G_k \rho d / C - \beta\right).$$

It is affine in c, so an optimum buys either none or all (with ties resolved against bribery). Buying all is profitable iff

$$G_k\, \rho d \;>\; C\beta \qquad \Longleftrightarrow \qquad G_k \;>\; C\, B.$$

Below that threshold, assume ties resolve against bribery and each level has audit-and-detection probability ρd, *conditional on every preceding level having missed the corrupt grade*, with $0 < \rho d < 1$. Then expected surviving corrupt value decays as

$$G_{k+1} \;=\; (1 - \rho d)\, G_k, \qquad \text{i.e.} \qquad \lambda = 1 - \rho d.$$

If $G_0 \le CB$, the process remains below the threshold, and the conditional recurrence pushes expected surviving value strictly below u after

$$\left\lfloor \frac{\log(G_0/u)}{\log \frac{1}{1-\rho d}} \right\rfloor + 1$$

levels for target $0 < u \le G_0$ — a depth *logarithmic* in G_0/u. Counting one selected bond per level gives B times this depth; total capital reserved by all eligible judges depends on whether their bonds can be reused and is not determined by that count.

Numbers by Hand 5.11.2 (Diversity and Audit Depth).

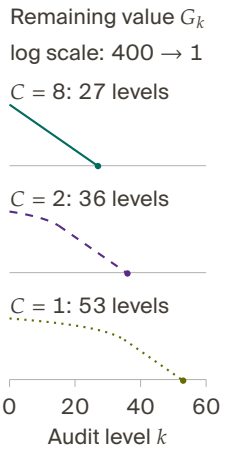
Take $G_0 = 400$ with the running parameters above, so $\rho d = 0.2$, $\beta = 10$, and the geometric factor is $\lambda = 1 - \rho d = 0.8$.

With a heterogeneous pool at $C = 8$ the bribery threshold is $G_k > CB = 400$, unmet even at level 0, so the decay is geometric from the first level and the tower needs

$$\lceil \log 400 / \log 1.25 \rceil = 27$$

levels, counting $27 \times 50 = \$1{,}350$ in selected bonds. This is not a capital requirement for the entire eight-clique candidate pool.

For $C = 1$, the threshold is $G_k > 50$. To obtain the following linear phase, additionally assume the briber pays each level's



The model's linear bribery phase delays geometric decay. All three traces stop below one unit; they are not measured audit performance.

bribes from its remaining budget, cannot replenish that budget, and the next level values only that remainder. The script then subtracts $C\beta = 10$ per level ($400 \to 390 \to 380 \to \cdots$) for 35 levels before the geometric phase takes over for

$$\lceil \log 50 / \log 1.25 \rceil = 18$$

more: 53 levels and $53 \times 50 = \$2{,}650$ in selected bonds. The linear phase tracks net budget under the extra assumption, not disappearance of the fraudulent grade itself.

Now you try: at $C = 2$, where does the linear phase stop? (Bleed $C\beta = 20$ per level until $G_k \le CB = 100$, i.e. 15 levels, then 21 geometric ones — 36 levels, \$1,800.)

What C is actually doing, and the symbol it shares with the market. The clique count C is this theorem's diversity parameter — it is why the worked numbers above contrast $C = 8$ against a monoculture $C = 1$, and why the monoculture costs roughly twice the bond capital. It is worth saying what C is a proxy *for*, because the reviewer of a foundation-model monoculture is right that "disjoint clique" is a structural description and correlated error is a statistical one. *The Harbor Economy* gives the statistical version (§6.13.1): let a panel's detection rate be a random D with mean d drawn once and shared, write θ for the resulting intra-panel correlation, and the probability that a panel of k all miss is $\mathbb{E}[(1 - D)^k] \ge (1 - d)^k$ by Jensen, assuming detections are independent conditional on D. Within that mixture class, independence is optimistic. Sampling across disjoint principal cliques does not itself establish statistical independence: shared models, evidence, or tasks can preserve common blind spots. The tower's geometric recurrence needs the stated conditional detection rate, not just the same marginal rate at each level. With level-specific conditional rates the surviving-value bound is $G_0 \prod_j (1 - \rho_j d_j)$; without such bounds, neither the geometric contraction nor its depth follows. Calibrating them requires labeled outcomes, including grades previous levels missed. One symbol, two chapters: θ is correlation and ρ stays the audit rate in both.

Numbers by Hand 5.11.3 (Amortized Verification: Sums and Their Integral Approximations).

Let periods be $t = 0, \ldots, T-1$, audit cost $a = 1$, corrupt gain $G = 10$, detection probability $d = 0.8$, bond $B = 50$, and forfeitable continuation value vt with $v = 0.6$ per period. These are illustrative payoff assumptions. Flat auditing uses $\rho = G/(dB) = 0.25$, costing $aT\rho = 50$ over $T = 200$ periods.

In Model A, detection forfeits both bond and accumulated continuation value. Setting

$$\rho_t = \frac{G}{d(B + vt)}$$

makes the expected sanction equal G at every period. The discrete cost is $\sum_{t=0}^{199} a\rho_t \approx 25.58$. The decreasing function's integral,

$$\int_0^T \frac{aG}{d(B+vt)}\,dt = \frac{aG}{dv}\log(1 + vT/B) \approx 25.50,$$

is a lower approximation to this sum, not its exact value. For fixed positive v, both grow logarithmically with T.

Model B instead stipulates expected automatic forfeiture rvt, with $r = 0.05$, in addition to the audited bond loss. Under that additive payoff model,

$$\rho_t = \max\{0, (G - rvt)/(dB)\}.$$

Its first zero-audit integer period is $\lceil G/(rv) \rceil = 334$; the real-valued crossing is $333\frac{1}{3}$. The first 200 periods cost $35.075 \approx 35.08$. Summing all positive terms gives 41.79175, whereas the triangular integral is $aG^2/(2dBrv) = 41.666\ldots$. Thus the lifetime sum is bounded, but it does not converge to that integral. Automatic forfeiture requires its own evidence and enforcement mechanism; a growing score alone cannot supply it.

Now you try: calculate ρ_0 and ρ_{100} for each schedule. Model A gives 0.25 and $10/88 \approx 0.114$; Model B gives 0.25 and $7/40 = 0.175$. Explain what loss the judge actually risks in each case before comparing costs.

Figure 5.14 draws all three schedules on one axis: model A stays cheapest at this chapter's own $T = 200$ horizon even though model B's total is the one that eventually stops growing.

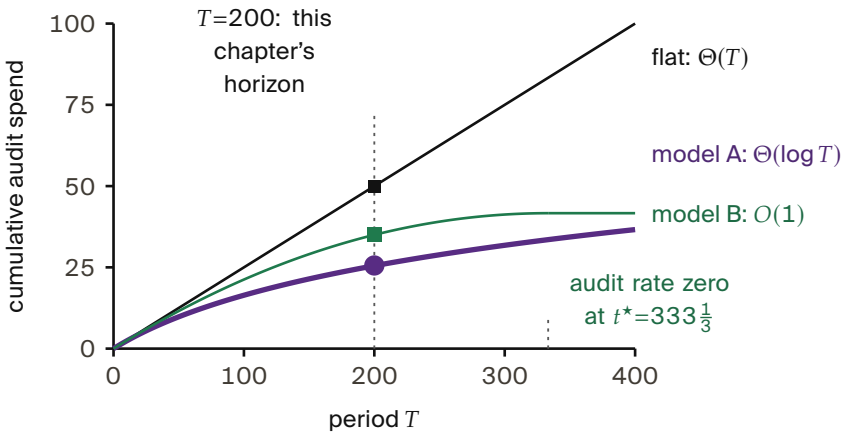


Figure 5.14. Continuous integrals are curves; discrete bills at $T = 200$ are dots: 50, 25.583785, 35.075. Model B plateaus at 41.666667 continuously versus 41.79175 discretely; its first zero-rate grade is 334. Model A keeps growing. Parameters and sums are in the text.

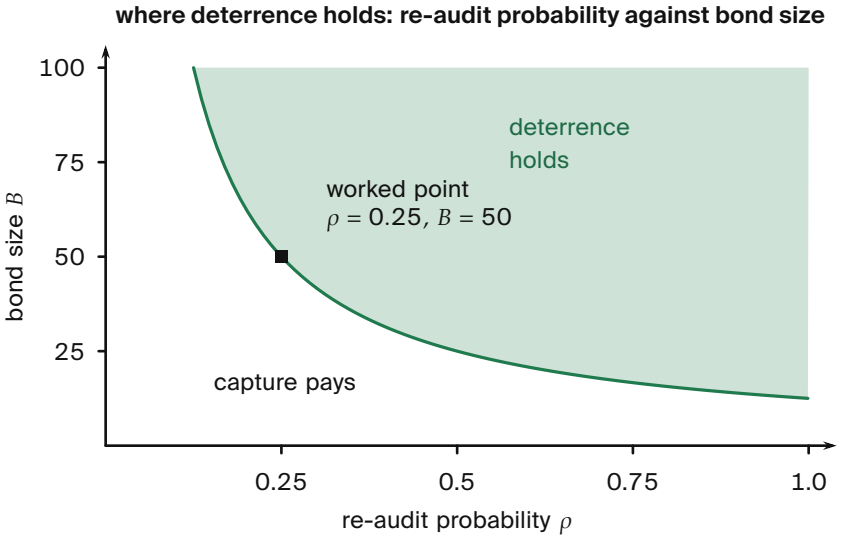


Figure 5.15. Deterrence requires $\rho d B \geq G$. For $G = 10, d = 0.8, B = 50$, the threshold is $\rho = 0.25$; lower audit probability makes capture profitable. [internal; closed form]

David M. Kreps and Robert Wilson. Reputation and Imperfect Information. *Journal of Economic Theory*, 27(2):253–279, 1982.

Drew Fudenberg and David K. Levine. Reputation and Equilibrium Selection in Games with a Patient Player. *Econometrica*, 57(4):759–778, 1989.

Continuation value is a familiar source of discipline in repeated games: a deviation can sacrifice profitable future participation. The connection to Model A is the value at risk, not an identity of models. If $\rho = 0$ in Model A, its expected sanction $\rho d(B + vt)$ is zero, even when the continuation value is large. A repeated-game mechanism needs a separate observation rule that exposes deviations and triggers the future loss. Model B stipulates one such channel; it must be justified independently.

What heterogeneity actually buys — and three riders. The honest reading of those two numbers is a comparison within the extra non-replenishing-budget model. Raising C removes its linear

prefix at these parameters; this does not prove that an externally financed adversary must run out of money. The selected-bond totals exclude collateral held by unused candidates. Three hypotheses are required and must not be dropped in restatement. (i) *Sealed sampling is a hypothesis, not hygiene*: a leaked draw collapses C to 1 and puts the VRF sampler inside the trusted computing base. (ii) *The tower needs an exogenously honest root* — the operator at $n = 1$, a separately justified adjudicator at scale; the theorem prices the depth to that root, it does not conjure the root. (iii) The theorem is about *bribery of sampled auditors*; it does not price collusion correlated *across* cliques, which is a measurement obligation on deployed judge telemetry (d, and the cross-clique correlation). Conditional detection remains a model assumption until the relevant deployment evidence supports it; distinct cliques alone do not establish it.

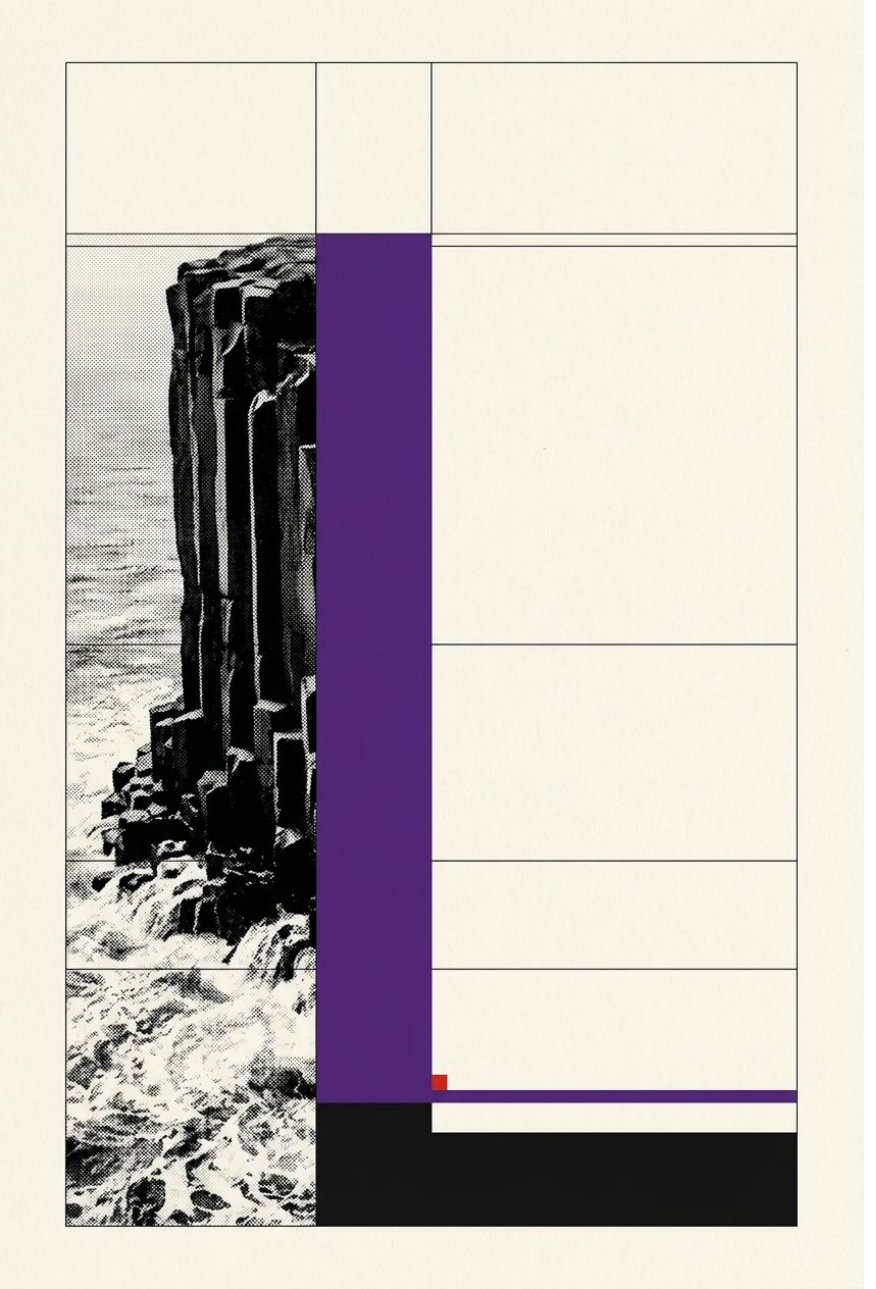
A deeper audit still needs a sound root

The tower prices a recursion under stated sampling, detection, and incentive assumptions. It does not create a competent grader or an honest root. The illustrative 27 levels / $1,350 versus 53 / $2,650 comparison depends on the assumed conditional detection rates, not measurements of deployed judge pools. A VRF can make a draw verifiable; it cannot make the selected judge correct. Oracle completeness, cross-clique dependence, and the terminal root remain separate obligations.

Recall

1. Without looking: what is Becker's condition for local deterrence, and why does dropping the detector term d under-quote the required audit rate?
2. State the tower's contraction factor and what raising the clique count C changes about it.
3. Name one thing the tower theorem prices and one thing it explicitly does not supply.

Exercises 5.32–5.36, p. 473.



Landmark 04 · The Front-Loaded Probation Cliff

Probation limits exposure while evidence accumulates. Admission depends on an explicit rule, the quality of the evidence and the cost of a mistake. A successful history can justify a bounded grant; it cannot prove that future behavior will be harmless.

5.12 Revocation, tombstones, and bounded memory

A reputation built on witnessed outcomes must answer two questions: what happens when a witnessed outcome turns out to be *fraudulent*, and when should old evidence lose weight? Keeping the evidence and estimating present reliability are distinct decisions.

5.12.1 Append-only evidence, individually revocable outcomes

Property 5.12.1 (Append-only evidence with tombstones)**.**

The **evidence ledger is append-only**: a genuine delivery is never silently erased by the passage of time. A reputation estimate need not rise with every honest report; an honestly reported failure can lower it. Separately, a witnessed outcome is itself **individually revocable via a propagating tombstone**: a compensating, append-only entry that retracts a specific outcome later found fraudulent (a colluding judge, a faked oracle). Retention preserves the outcome record; it does not require its score weight to remain constant or prevent a later invalidation.

Kind: **Design invariant** (specified, not yet **implemented**, on the maturity scale) — the intended ledger behavior, not yet a theorem about a shipped mechanism.

This is a **saga**-style compensation: you do not mutate the ledger in place (that would break append-only and auditability); you append a tombstone that nullifies the targeted entry, and the tombstone *propagates* across harbors bounded by the federation's revocation-convergence window (the market paper's job). The point may have influenced decisions before the fraud was detected. After the tombstone is issued, a bounded stale-use window needs delivery assumptions or an expiring freshness requirement that fails closed. Federation must state those conditions; gossip alone gives no finite partition bound. We state the retraction as a protocol.

Garcia-Molina & Salem 1987

Protocol 5.12.1 (Tombstone revocation).

To retract a witnessed outcome o later found fraudulent:

1. **Trigger.** A re-audit (Protocol 5.10.1, step 5) or a dispute overturns the grade on o, establishing fraud against an oracle.
2. **Append, never mutate.** A *tombstone* entry $\bar{o}$ is appended to the ledger, carrying o's id, the overturning evidence, and the identity of the retracting judge. The original o stays in place; auditability is preserved because nothing is erased.
3. **Recompute.** Every reputation score that consumed o is recomputed with o nullified by $\bar{o}$. A local incremental update may be cheap, but removing an older observation can require replaying retained history or refitting a coupled estimator. Cost depends on the estimator and dependency indexes; it is not bounded here by the cost of one forward update.
4. **Propagate.** $\bar{o}$ gossips to every harbor that imported o, under the federation's declared delivery and freshness assumptions. Each harbor applies step 3 on receipt; an expired view must refuse decisions requiring current reputation.
5. **Bound the damage.** Separate publication-to-detection delay from tombstone-to-enforcement delay. A bound τ on the latter requires bounded delivery or fail-closed expiry. Neither the pre-detection period nor an unbounded partition is automatically bounded by τ.

Figure 5.16 draws the append-only compensation: the original entry stays, and a later tombstone nullifies it without mutating the ledger.

5.12.2 S4 — the fraudulent outcome retracted

Numbers by Hand 5.12.1 (A Fraudulent Outcome, Retracted).

Recall S3: the re-audit caught Mina rubber-stamping a different agent's insecure change as $q_{\text{aes}} = 0.9$. Follow that one outcome through Protocol 5.12.1. The graded agent — call it *Crane*, running under a third operator — had colluded with Mina: Crane shipped a known-insecure diff and Mina graded it high so Crane's aesthetics reputation would clear Bob's hiring bar.

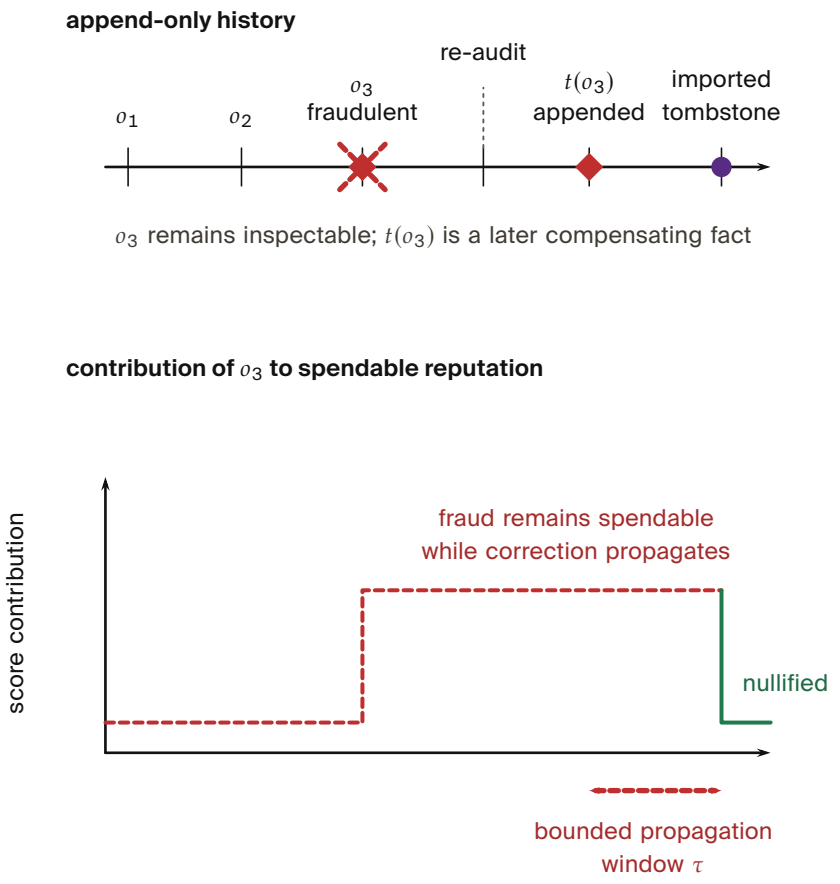


Figure 5.16. A tombstone removes a fraudulent contribution from future scoring without erasing history. Importing harbors remain exposed until it arrives; τ measures that convergence interval. [internal]

For two days it worked: Crane's vector showed $(0.95, 0.9, 0.8)$ and Bob nearly hired it.

Then the sampled re-audit re-graded the diff blind, found the injection, and overturned Mina's grade.

A tombstone $\bar{o}$ is *appended* (the original stays, for the audit trail); every score that consumed Crane's inflated q_{aes} is recomputed with it nullified; Mina's bond on that job is slashed and her judging reputation drops; Crane's collusion is itself recorded as a fraudulent outcome against *its* non-forgeable identity, which it cannot shed.

The tombstone gossips to Bob's harbor within the convergence window, so by the time Bob next reads Crane's vector the 0.9 is gone. The fraud was real and briefly spendable — the honest claim is that the window was *bounded*, not that it never existed.

Now you try: what does Crane's *recomputed* vector read the instant the tombstone lands?

$$(0.95,\ 0.\bar{9},\ 0.8),$$

where $0.\bar{9}$ means the aesthetics axis is nullified, not replaced by a new number — Exercise 5.39 asks what a buyer should read there in the meantime.

5.12.3 Bounded memory is a design parameter, not a virtue to assert away

An immutable evidence record and a reputation score's memory horizon are separate choices. The ledger can retain an outcome for audit while an estimator reduces its weight or excludes it from the current score. A tombstone records a finding that an outcome was invalid; ordinary aging does not make that finding.

Liu & Skrzypacz 2014

Liu–Skrzypacz analyze a long-run player facing successive short-run opponents who see only a finite recent history. Under their stated stationary-equilibrium assumptions, reputation can repeatedly build and be exploited even when an opponent knows the player's opportunistic type. This shows that the observation window affects incentives. It does not prove that infinite memory is generally suboptimal or prescribe this market's window. Choosing a horizon here requires a model of changing skills, entry, observation, and incentives, together with an objective to evaluate.

Ostrom 1990

Ostrom's work on common-pool institutions offers a governance analogy: people affected by a rule need a way to understand it, contest its application, and revise it. A reputation window is a jointly governed parameter, not itself a rivalrous stock consumed by use. Nor does the analogy prove that one particular governance arrangement is necessary. A harbor should state who sets the weighting rule, which evidence remains available, and how disputes and rule

changes are handled. These are policy choices to evaluate, not a theorem inherited from the commons literature.

Retention is not endorsement

Keeping an outcome in an audit ledger does not require counting it forever in a score. Conversely, aging its score weight does not retract fraudulent evidence. Retain the record, record valid retractions, and state the estimator's horizon separately.

Choose a memory horizon

Evidence can be append-only while scores rise or fall, old observations receive less weight, and specific outcomes are invalidated by tombstones. The appropriate weighting horizon depends on the incentive and prediction problem; neither finite nor infinite memory is universally optimal.

Exercises 5.37–5.39, p. 473.

5.13 What this chapter hands the market

This chapter is the bridge; its output is a precise hand-off to *The Harbor Economy*. We summarize what is delivered, what is borrowed, and what is explicitly owed.

Artifact	Maturity	Role in the market paper
Role / person distinction (Defs. 5.3.1–5.3.2)	—	A participant whose obligations and witnessed history remain attributable across process replacements.
Principal-as-economic-entity (Def. 5.6.2)	specified	Bonds and sanctions key on the principal; the market paper cross-references, not redefines.
The three continuity subsystems	**implemented** partial partial	Reputation keys on subsystem 3 (the outcome ledger).

Table 5.9. The market inherits both mechanisms and unfinished obligations. Cross-operator attestation and a competent honest grading root remain open. The tower result is conditional.

(continued)

Artifact	Maturity	Role in the market paper
Outcome ledger / oracle (Def. 5.5.1)	partial	Commitment, deadline, oracle closure, API/CLI, and overdue detection ship; neutral graded outcome events, sanctions, and reputation binding do not.
Multi-dimensional reputation (Def. 5.10.1, Protos. 5.10.1, 5.10.2)	specified to *proposed*	The multi-dimensional score the market reads; the licensing side's signal.
Neutral-judge market	specified to *proposed*	Proposed evaluation institutions; competent, honest grading remains an obligation.
Grading-oracle hole (Prop. 5.11.1)	specified	Theorem 5.11.1 bounds recursion under conditional detection and incentive assumptions. Oracle completeness, a competent honest root, and calibrated cross-clique detection remain owed.
Cross-operator attestation (Def. 5.7.2)	*proposed*	*Named dependency the market paper must build*; not assumed here.
Tombstone revocation (Prop. 5.12.1)	specified	Reputation is revocable; convergence is the market paper's federation job.

Port Daddy Harbor 2026

The proposed history-dependent market needs attributable outcomes and obligations across process replacements. Those same records can help one operator resume work and inspect a past decision. This is the chapter's bridge from local continuity to exchange;

it does not make continuity a prerequisite for every trade, such as a prepaid purchase of known content.

Review of the key ideas

What this chapter leaves coupled, in the order the rest of the book will lean on it:

1. **A role is replaced; a person accumulates.** The role rail names whoever fills it this week; the person rail is the set of witnessed outcomes that survive every respawn.
2. **Continuity is a ledger position, not a metaphysics.** Three subsystems carry it: episodic memory survives process death, the checkpoint keeps a summary and worktree lineage but not execution state, and the outcome ledger keeps commitments and closures in part.
3. **Authenticity and liability binding are separate.** A valid credential can still be disposable. The newcomer-floor theorem states when a reset erases a score sanction; principal-wide admission and exposure rules address a different part of that escape.
4. **Whitewashing has a conditional price.** A reset costs the discounted continuation gap W_δ; a deviation that triggers it with probability p after delay τ faces expected cost $p\delta^\tau W_\delta$. The worked schedule costs 18 without discounting. Front-loading minimizes honest-newcomer friction only under the stated relative discount factors, ceiling, and deterrence constraint.
5. **Engine substitution can occur within one identity.** The one-delivery payoff comparison shows when a cheaper engine is profitable. The example's attested prices reverse that comparison (0.5 against 0.2); they do not prove that buyers will offer those prices or that attestation is the only remedy.
6. **Resurrection preserves sanctions only under the stated checks.** The migration model requires authenticated lineage, outcome engine attestations, and closure or renegotiation of open commitments, with sanctions scoped to the principal. The separate no-mint accounting rule prevents a fork from duplicating spendable credit; it does not replace those migration conditions.

7. **Reputation is a substrate, not a score**: multi-dimensional and buyer-weighted. A routing bandit can consume this evidence but does not create its provenance or accountability.
8. **Bonded grading needs its stated detection model.** The tower uses prepaid bribes, sealed uniform clique sampling, and a conditional detection bound. Geometric contraction starts below the bribery threshold; a trustworthy root is a separate assumption. A bond alone neither makes a grade correct nor excludes other ways to obtain useful evidence.
9. **Revocation needs a freshness contract**: a tombstone retracts a record without erasing its history. A finite stale-use bound requires delivery or fail-closed expiry assumptions; the estimator's memory horizon is separate from retention of the evidence ledger.

5.14 Exercises

§5.1 Introduction: the spawn that cannot be paid

Solution p. 747

5.1 Check ⋆

State the difference between a spawn and a person in this chapter's engineering convention, using "role" and "continuity." Does the definition assert consciousness?

Solution p. 748

5.2 Trace ⋆⋆

Walk the proposed dependency diagram right-to-left. For each arrow, name the evidence or recovery capability lost when its source is absent. Which links are choices of this architecture rather than necessities for every market?

Solution p. 748

5.3 Open ⋆⋆⋆

The chain treats the operator (the human who owns a fleet) as outside it. Is the operator a "person" in this chapter's sense? What breaks if a single operator can mint arbitrarily many agent-persons? (This is the principal-layer gap; see §5.6 and the market paper.)

§5.3 The role / person distinction, made precise

5.4 Check ⋆ *Solution p. 748*

Give a concrete example of a state that is *capable-and-permitted*, one that is *capable-but-forbidden*, and one that is *permitted-but-incapable*.

5.5 Trace ⋆⋆ *Solution p. 749*

In Definition 5.3.2, which of $\{O, C, A\}$ does reputation attach to — the role's, or the person's? Justify using the spawn-that-cannot-be-paid argument of §5.1.

5.6 Open ⋆⋆⋆ *Solution p. 749*

"Ought implies can": the legitimate obligation set O should be bounded by the capability set C. But *capable-but-forbidden* must remain representable. Sketch a representation of a role that keeps both true without collapsing permission into capability.

§5.4 Continuity is a personal-identity problem

5.7 Check ⋆ *Solution p. 749*

Distinguish a direct succession link from a chain of such links. Then give an example of task information surviving while accountable lineage is lost.

5.8 Trace ⋆⋆ *Solution p. 749*

Map "actor-soul" and "body-lease" onto Locke's substance/consciousness distinction. Which one does a respawn replace?

5.9 Open ⋆⋆⋆ *Solution p. 749*

Parfit famously argues identity "is not what matters" in survival — what matters is continuity, which can branch. If an agent's checkpoint is forked into two live incarnations, which one (if either) inherits the reputation? Sketch a rule and the attack it invites. (Open Problem 1, §5.16.)

5.10 Trace ⋆⋆ *Solution p. 750*

Run `skills/harbor-results/scripts/a6_no_mint.py` (seed 20260816 fixed inside). It prints a closure-sum counterexample, a randomized DAG sweep, and a copy-full mutation. What is the exact violation count in the sweep, and what should you check by hand on the featured chain $e \to a_1 \to a_2 \to a_3$ at $\gamma = 0.9$?

§5.5 The three subsystems of continuity

Solution p. 750

5.11 Check ⋆

Which subsystem is partial, and what exactly is weak about it?

Solution p. 750

5.12 Trace ⋆⋆

An agent claims a file, makes a commit, and dies. Walk the three subsystems: what does each retain, and what is *lost* that a strong checkpoint would have kept?

Solution p. 750

5.13 Open ⋆⋆⋆

When an LLM agent dies mid-thought, what is recoverable and what is fundamentally lost? Propose a taxonomy of agent-death by what survives (record / claims / execution state / latent "intent"). This is the hard half of the checkpoint-with-teeth problem. (Open Problem 2, §5.16.)

§5.6 Identity: the root the whole chain hangs from

Solution p. 751

5.14 Check ⋆

State Property 5.6.1 in your own words and give the two-attack proof sketch.

Solution p. 751

5.15 Trace ⋆⋆

What tradeoff does full work with a reduced economic ceiling make compared with restricting newcomers' work? Distinguish evidence accumulation, liquidity, sanction evasion, and deterrence of a new deviation.

Solution p. 751

5.16 Open ⋆⋆⋆

Bond-farming: a single principal spawns 1000 agents, has them trade trivial tasks among themselves to mint reputation, then sells the "experienced" agents. Which of {principal binding, listing fee, sampled adversarial re-audit of high-velocity counterparty pairs} defeats this, and at what false-positive cost to honest high-volume pairs? (Open Problem 8, §5.16.)

Solution p. 752

5.17 Trace ⋆⋆

Run `skills/harbor-results/scripts/b6_probation.py` (seed 20260816). It falsifies dominance by random search over 4,000 instances. What exact counts does it print, and what closed-form check does the exchange step verify by hand?

5.18 Trace ★★ *Solution p. 752*

Run `skills/harbor-results/scripts/b5_engine_substitution.py` (seed 20260816). Its Part 3 checks that provider migration preserves Def. 5.6.1. What exact reachable-state count and mutant shortest-crime lengths does it print, and which single clause's omission is caught in one step with *no* migration at all?

§5.7 The keystone split: local identity vs. cross-operator attestation

5.19 Check ★ *Solution p. 752*

In one sentence each, state what a local non-forgeable identity provides and what cross-operator attestation would have to add.

5.20 Trace ★★ *Solution p. 752*

Take any claim in the market paper of the form "Alice's harbor and Bob's harbor settle across a boundary neither controls." Which stone does it secretly assume? What is the actual security guarantee if only the local non-forgeable identity exists?

5.21 Open ★★★ *Solution p. 753*

Can a federated witness log ever substitute for identity-binding? Argue why log-integrity ≠ key-binding, then sketch the minimal additional assumption (a shared root of trust? a sponsor chain? a transparency log) that would close the gap, and the cost each imposes. (Open Problem 3, §5.16.)

§5.8 Reputation as a substrate, not a score

5.22 Check ★ *Solution p. 753*

Does retaining a full comparison history force a Bradley–Terry fit rather than sequential Elo updates? Name a modeling assumption and a practical constraint that should guide the choice.

5.23 Trace ★★ *Solution p. 753*

You have one task with a clear binary acceptance test, and another judged only by a human's aesthetic preference. Which estimator fits each, and why does EigenTrust fit neither directly?

5.24 Open ★★★ *Solution p. 753*

The estimator summarizes evidence but does not establish its provenance. Construct a scenario where a *better estimator* still

buys nothing because the substrate is rotten (e.g. forgeable identity or an agent-authored oracle). Generalize the lesson.

§5.9 Routing policies and public reputation answer different questions

Solution p. 754

5.25 Check ⋆

Name four evidence requirements that a simple stationary routing model does not itself supply. Does this rule out adversarial or non-stationary bandits?

Solution p. 754

5.26 Trace ⋆⋆

Give one use for a bandit policy in this chapter. State its feedback assumptions and explain why the policy does not replace the reputation ledger.

Solution p. 754

5.27 Open ⋆⋆⋆

Chiu–Zhang–van der Schaar show competitive agents over-invest in self-improvement, burning surplus despite individual rationality. Does capping the reputation signal's marginal return restore efficiency, or merely relocate the waste? (Open Problem 6, §5.16.)

Wei-Fan Chiu, Yuxuan Zhang, and Mihaela van der Schaar. Strategic Self-Improvement for Competitive Agents in AI Labour Markets. arXiv:2512.04988, 2025.

§5.10 The multi-dimensional reputation protocol

Solution p. 754

5.28 Check ⋆

For each of accuracy / aesthetics / efficiency, name a competent judge and an *incompetent* one.

Solution p. 754

5.29 Trace ⋆⋆

Walk a judging task through the judge-hiring ceremony (Protocol 5.10.1): who plans it, who posts the bond, where does the grade land, and what slashes the judge?

Solution p. 755

5.30 Trace ⋆⋆

Design the witnessed event for skill-helpfulness. What fields make exposure and outcomes auditable, and what additional assumption would justify "this skill helped" as a causal statement?

Solution p. 755

5.31 Open ⋆⋆⋆

If you publish the 3-vector and buyers weight it, do agents Goodhart the most-weighted axis? If you publish only a scalar, a buyer cannot reconstruct the distinct quality dimensions. What is the right disclosure form for vector reputation? (Open Problem 5, §5.16.)

§5.11 The grading oracle's incentive-compatibility

5.32 Check ⋆ *Solution p. 755*

State Property 5.11.1 and explain why "the daemon derives the grade" does not, by itself, close it.

5.33 Trace ⋆⋆ *Solution p. 755*

Walk Option A: a judge grades dishonestly, a re-audit overturns it. Follow the bond flow and the reputation update for the judge.

5.34 Trace ⋆⋆ *Solution p. 755*

Re-derive the critical audit rate in Theorem 5.11.1 from scratch: a judge cheats for gain G, is sampled with probability ρ, is caught given sampling with probability d, and forfeits bond B when caught. Write the cheat payoff, set it to zero, and recover $\rho^\star = G/(dB)$. Then check the two worked towers: at $G_0 = 400$, $d = 0.8$, $B = 50$, $\rho = 0.25$, confirm 27 levels at $C = 8$ and 53 at $C = 1$.

5.35 Open ⋆⋆⋆ *Solution p. 756*

Arbitration capacity: the contraction theorem prices the depth to an exogenously honest root but does not supply the root, and human oracles are scarce. Design a market for bonded, rated, slashable arbiters and argue whether it converges to honest rulings or to rubber-stamping under load. (Open Problem 9, §5.16.)

5.36 Trace ⋆⋆ *Solution p. 756*

Run `skills/harbor-results/scripts/b2_tower.py` (seed 20260816). Its part (3) compares flat auditing to two amortization models. What are the exact cumulative-spend numbers it prints at $T = 200$, and what should you check by hand about the flat baseline?

§5.12 Revocation, tombstones, and bounded memory

5.37 Check ⋆ *Solution p. 756*

Why must revocation be a tombstone (append) rather than a delete (mutate) on the outcome ledger?

5.38 Trace ⋆⋆ *Solution p. 756*

A judge is later found to have colluded on five settled outcomes. Walk the tombstone for one of them: what is appended, what propagates, and what bounds the window the poisoned reputation was spendable?

Solution p. 757

5.39 Open ⋆⋆⋆

Reputation-claim revocation with an append-only, bounded-memory ledger across N harbors: how do you prove the correction converged and bound the spendable window, without violating append-only? This is the cross-harbor analogue of capability revocation (*The Harbor Economy*). (Open Problem 7, §5.16.)

5.15 Reproducing the chapter results

This series' own companion results, by script and seed. The audit-tower theorem of §5.11 is imported from *Reputation is Amortized Verification*; its numbers regenerate from `b2_tower.py` (seed 20260816). The no-mint theorem of §5.4 regenerates from `a6_no_mint.py` (same seed). The engine-substitution and resurrection-soundness results of §5.6 are the whitepaper statement of *Continuity Without Metaphysics*; they regenerate from `b5_engine_substitution.py`. The probation-cliff theorem shares that paper's number and regenerates from `b6_probation.py` (same seed throughout, 20260816).

Port Daddy Harbor 2026

Port Daddy Harbor 2026

5.16 Open problems (the starred items, collected)

The genuinely unsolved problems this chapter surfaces, gathered for the reader who wants the research frontier rather than the pedagogy. Every starred exercise in the body cross-references this list *by name*, not by a hand-typed numeral, so the two cannot drift apart. Note that the "OP-n" identifiers used in the body for kernel-layer problems (e.g. OP-4, checkpoint with teeth) belong to a *shared* namespace owned by *The Single-Writer Kernel* and are deliberately not renumbered here; this list is this chapter's own.

1. **Branching continuity (§5.4).** If a checkpoint is forked into two live incarnations, which inherits the reputation? Parfit says identity-is-not-what-matters; the economy needs a rule, and the rule invites an attack.

2. **Agent-death taxonomy (§5.5).** What is recoverable vs. fundamentally lost when an LLM agent dies mid-thought? Event-Sourced Neural Rehydration (OP-4 in the shared kernel namespace) is a specified candidate answer — restore the Git SHA, truncate the durable message log to the crash point, replay under prompt-prefix caching — but it has not shipped, and its completeness has not been established: replay aims to restore lineage, while equivalent behavior even within one provider/model/version is an unmeasured hypothesis; it does not export the inference state, so across providers the honest description is task continuity with actor succession. The taxonomy itself — record vs. claims vs. execution state vs. latent "intent" — stays open until the mechanism ships and is measured.

3. **Cross-operator attestation (§5.7).** Binding identity keys across harbors you do not own. The highest-leverage unbuilt keystone for the market.

4. **The grading oracle's honest root and its telemetry (§5.11).** The rate-the-raters recursion itself is *no longer open*: Theorem 5.11.1 closes it with a derived contraction factor $\lambda = 1 - \rho d$ and a priced depth (27 levels / $1,350 at C=8; 53 / $2,650 at C=1). What remains open is what that theorem explicitly does not supply: the exogenously honest root the tower terminates against at scale, and the deployment telemetry that estimates d and the cross-clique collusion correlation the disjointness hypothesis assumes away.

Port Daddy Harbor 2026

5. **Vector-reputation disclosure (§5.10).** Publish a vector and agents can target the heaviest axis; publish only a scalar and buyers lose the separate measurements. Disclosure and incentive design remain open.

6. **Self-improvement arms race (§5.9).** Chiu et al. 2025: does capping reputation's marginal return restore efficiency or relocate the waste?

7. **Reputation-claim revocation convergence (§5.12).** Surgically retract a fraudulent outcome across N harbors, prove convergence, bound the spendable window — without violating append-only.

8. **Bond-farming defense (§5.6).** Principal binding vs. listing fee vs. sampled re-audit of high-velocity pairs; the false-positive cost to honest high-volume counterparties.
9. **Arbitration capacity (§5.11).** A market for bonded, rated, slashable human/automated arbiters: honest rulings, or rubber-stamping under load? This is Item 4's honest root, stated as a market-design problem.
10. **Skill-version reputation (§5.10).** A portfolio proof for skill v1 says nothing about v2; the lemons problem reappears at every version bump.

Handoff to the market. Section 5.13 collects the mechanisms and obligations this chapter passes on. The open questions above remain obligations even where a conditional model is solved.

5.17 Conclusion

We set out to bridge the wedge to the market by answering one question: what has to be true before a coding agent's work can be *sold*? The answer is not a score. It is a *self* — a role plus continuity, keyed on an identity that cannot be re-picked, accumulating a transitive, witnessed history that survives every respawn. The estimator that turns that history into a number is the cheap, last part; the substrate beneath it is the gate, and most of that substrate is formally not built yet.

We were disciplined about the keystone (this chapter stands on a local non-forgeable identity and hands the market paper the unbuilt cross-operator attestation by name), about the hardest hole (the grading oracle's incentive-compatibility is a flaw in the core theorem, not a footnote — and the rate-the-raters recursion underneath it is now closed by a companion theorem with a derived contraction factor and a priced tower depth, leaving the honest root and the audit telemetry as the residue), and about the cheerful slogans (reputation is monotone in *honest* outcomes but individually revocable, and bounded memory is a parameter, not a virtue). Reputation records can support a bandit routing policy, but they must

also make version changes, separate quality dimensions, strategic incentives, and grader provenance inspectable.

A spawn does work and disappears. A person does work and *stays* — so later buyers can inspect a continuing record. The economic object is accountability across jobs, not a claim that software has acquired a human self.

Chapter Appendices

5.A Implementation status of the reference system

The body of this chapter is written to stand on its own: every mechanism is described as a component (a memory record, a checkpoint subsystem, a witnessed-outcome ledger) and graded on the neutral maturity scale (**implemented** / partial / specified / *proposed*). This appendix grounds that scale in a concrete *reference system* — a running coordination daemon for local agent swarms — so a reader who wants to know "which of these actually runs today" can check. Nothing in the body depends on this appendix; it exists so the maturity labels are falsifiable rather than rhetorical.

Table 5.10. Maturity of each paper concept against a reference coordination daemon. The honest summary: the *subsystems* of continuity exist (one weakly); the *spine* of reputation — a witnessed-outcome ledger keyed on a non-forgeable identity — is specified but not yet shipped.

Component (paper concept)	Maturity	Status in the reference system
Episodic memory (Subsystem 1)	**implemented**	Durable typed episodes promoted from execution history; runs in production.
Restart subsystem (Subsystem 2)	partial	Heartbeat-staleness detector that salvages dead agents; forwards a *summary*, not execution state.
Outcome ledger (Subsystem 3)	partial	Durable commitments, daemon-derived deadlines, oracle-bound closure, API/CLI, and overdue detection ship; neutral grading, sanctions, and reputation binding do not.
Cost meter (efficiency axis)	**implemented**	Per-task token/wall-clock/dollar accounting recorded today.

(continued)

Component (paper concept)	Maturity	Status in the reference system
Local non-forgeable identity	partial	Daemon-minted actor-soul, lookup credential, and bounded newcomer pool ship; full write-boundary enforcement and legacy migration do not.
Capability jail (capability ≠ permission)	specified	Per-agent tool-allowlist and scoped filesystem; specified.
Honest-attestation discipline	**implemented**	"Report all-good only when verified" is an enforced engineering rule.
Multi-dimensional reputation	specified to *proposed*	No reputation component yet; the substrate it scores over is partly built.
Neutral-judge ceremony (Proto. 5.10.1)	specified to *proposed*	Bonding and task-planning primitives exist; the hiring ceremony is designed.
Skill-helpfulness schema (Proto. 5.10.2)	specified	Skill grafting is observable; the witnessed-event schema is specified.
Tombstone revocation (Proto. 5.12.1)	specified	Append-only ledger semantics are designed; cross-harbor propagation is *proposed*.
Cross-operator attestation	*proposed*	The unbuilt keystone for the market; owed to *The Harbor Economy*.

What *The Harbor Economy* receives A successor inherits bounded history and obligations, not new credit. The next chapter asks how that attributed work can be exchanged, and separates value, collateral, evidence, and settlement.

IV

Trade Between Strangers

Operators who trade work need evidence of performance, rules for disputes, and limits on loss. These chapters develop the market and its settlement rules, then examine how evidence and grants pass between independently governed machines.



6 The Harbor Economy

Labor, rented assets, and licensed skills expose different risks but settle through a shared ledger that conserves each native unit.

7 The Bonded Commons

Bonds, witnesses, audits and appeals turn residual uncertainty into an institution; the ledger's conservation law is mechanically checked.

8 The Federated Harbor

Mutually distrustful machines relay evidence, not sovereignty: what may gossip, what needs an authority point, and how a grant crosses a boundary.

6

The Harbor Economy

The question this chapter answers

What makes cooperation rational without making loss unbounded?

Labor, rented assets, and licensed skills expose different risks but settle through a shared ledger that conserves each native unit.



The main reason why it is profitable to establish a firm would seem to be that there is a cost of using the price mechanism.

Ronald H. Coase, The Nature of the Firm, 1937

What makes cooperation rational without making loss unbounded? A payment rail moves money; a market must also identify the parties, state what performance means, reserve the right funds, and decide what happens when evidence disagrees. This chapter constructs that market from four separate monetary buckets and explicit oracles. R7 and R13–R15 delimit the audit, substitution, escalation, and specialization claims.

Maturity key (self-grading of every claim). Every claim in this chapter carries one of four maturity labels, so the reader always knows whether a sentence reports running code or a designed target. **implemented** — shipped and exercised by tests in the reference implementation. partial — shipped but materially weaker than its name suggests. specified — specified, with a proof or a written protocol, but not yet shipped. *proposed* — a stated direction whose critical part has no specification yet. Nomenclature is likewise fixed: the cryptographic *hop-bound authorization chain* (which proves who delegated what, and bounds it) is never conflated with the *coordination lineage* (who-talked-to-whom, advisory only); **local non-forgeable identity** (an identity no agent can edit within one trusted daemon) is never conflated with **cross-operator attestation** (the unbuilt keystone that binds identity *across* daemons nobody jointly trusts); and *capability* (what an agent *can* do) is never collapsed into *permission* (what it *may* do).

A solo operator running a swarm of coding agents needs three things — legibility, accountability, and safety. When she controls every machine and principal in the harbor, coordination need not include a cross-operator trust protocol. Credentials, authenticated transport, and protection of stored data remain necessary where the threat model calls for them. That is the single-player setting of *The Legible Swarm*. The instant two operators trade, the world changes. Alice's frigates touch your repository. You must now price work done by agents you did not spawn, owned by a principal you do not trust, using skills you cannot inspect. That is a *market-design* problem before it is a cryptography problem. Cryptography is the substrate that makes the ledger unforgeable; it is not the product.



Like movable metal type replacing hand-lettered manuscripts, standardized agent interfaces convert unique labor into interchangeable, composable modules.

The proposed reputation-priced market combines verifiable records, evidence relay, and reputation as **hosted trust**. These are design requirements of this market, not necessary conditions for every trade.

The chapter develops three claims:

- **C1 (three contracts).** Operator-for-hire, asset-rental, and skill-licensing expose different private information and participation constraints. Distinguishing them changes the pricing (§6.1).
- **C2 (one ledger, one escrow object).** All three sides settle on the same **float plan** escrowed in the same bond ledger. Conservation of value across every settlement path is the invariant that holds the market together, and it is the one thing here that is actually **implemented** (§6.2, §6.5).
- **C3 (hosted trust is the moat).** The defensible asset is the verified ledger + relay + reputation, not the settlement rail, which the 2025–26 agentic-payments stack has already commoditized (§6.9).

Local coordination can stand alone

The economy is strictly *additive* on top of the single-player tool. Because all three market sides settle on the *same* built bond ledger, none of the economy needs to ship before the local coordination tool. Reusing that ledger does not, by itself, establish the market's incentive or cross-operator guarantees.

Numbers by Hand 6.0.1 (The Smallest Number That Makes C2 Concrete).

Carol licenses her lint-and-type skill into Bob's float plan at 0.5 cr per file, metered. The job touches 16 files, so $16 \times 0.5 = 8$ cr releases to Carol on green settlement — on the *same* escrow row that, in the same job, also locks Alice's 400 cr labor bounty and a 100 cr performance bond (§6.3.1 settles the whole 560 cr row by hand and checks it sums to zero). *Now you try:* before reading on, guess whether Carol's 8 cr and Alice's 400 cr clear through one ledger operation or two — C2 is the claim that it is one.

6.0.1 The through-line: attributing a record of past work

The whole series climbs one spine:

memory → checkpoint → continuity → a person, not a spawn
(*From Spawn to Person*)
→ registered outcomes → reputation
→ tradeable asset → market
(this chapter).

Reputation-based rental and licensing need a record that the buyer can attribute to the offered agent or skill. Copying the offered code must not create independent evidence that it succeeded. A prepaid licence for a known content hash can trade without that history; a price justified by past outcomes needs trustworthy attribution of those outcomes. The canonical cross-layer sentence, repeated in *From Spawn to Person* and here, states the dependency exactly:

The score is cheap; the substrate it scores over — witnessed outcomes on a non-forgeable identity — is the gate.

Exercises 6.1–6.3, p. 539.

6.1 Three purchases, three information problems

A **two-sided market** (**Rochet & Tirole**, the foundational reference: *a platform is two-sided when the allocation of the total price across the two user groups — not just its level — affects transaction volume*) is the canonical frame for a marketplace: buyers on one side, sellers on the other, the platform tuning who is subsidized to ignite cross-side network effects. A simple labor transaction has a *requester* who posts work and a *worker agent* who does it. We distinguish three purchases: labor, access to a specialist, and a licence for a reusable skill. These are contract types; they need not be three disjoint populations of platform participants. The same operator can buy one and supply another. Calling a platform two-sided or multi-sided additionally concerns how participation and pricing on one side affect the others. The classification here identifies which private information, rights, and payments each contract must handle.

Rochet & Tirole 2003

J.-C. Rochet & J. Tirole. "Two-Sided Markets: A Progress Report." *RAND Journal of Economics* 37(3):645–667, 2006.

In the following examples, “side 1,” “side 2,” and “side 3” are shorthand for these three contract types. Table 6.1 compares the three choices in Bob’s refactor below.

1. **Operator-for-hire (labor).** An operator who runs a fleet can sell its labor to *another* operator. The operator is now a firm taking work-for-hire, with margin = bounty $-\Sigma$(slashed sub-bonds) – ledger fee. Its private information is whether its fleet can deliver. It internalizes its fleet’s risk — exactly the property that makes a firm accountable for its employees.
2. **Agent/fleet as rentable asset (capital).** An operator who owns a well-reputed specialist can *rent it out*. Now the asset-owner, the task-poster, and the worker are three different parties with three different incentives. The owner’s private information is the agent’s true quality; its exposure is reputational.



Like one printing block making many impressions, a licensed skill can be reused without its maker performing each job.

3. **Skill/tool as licensed good (IP).** A skill can be *licensed* into someone else’s float plan — metered or per-use — without the licensor doing any task work. The licensor’s private information is the skill’s quality, which the buyer cannot inspect before purchase.

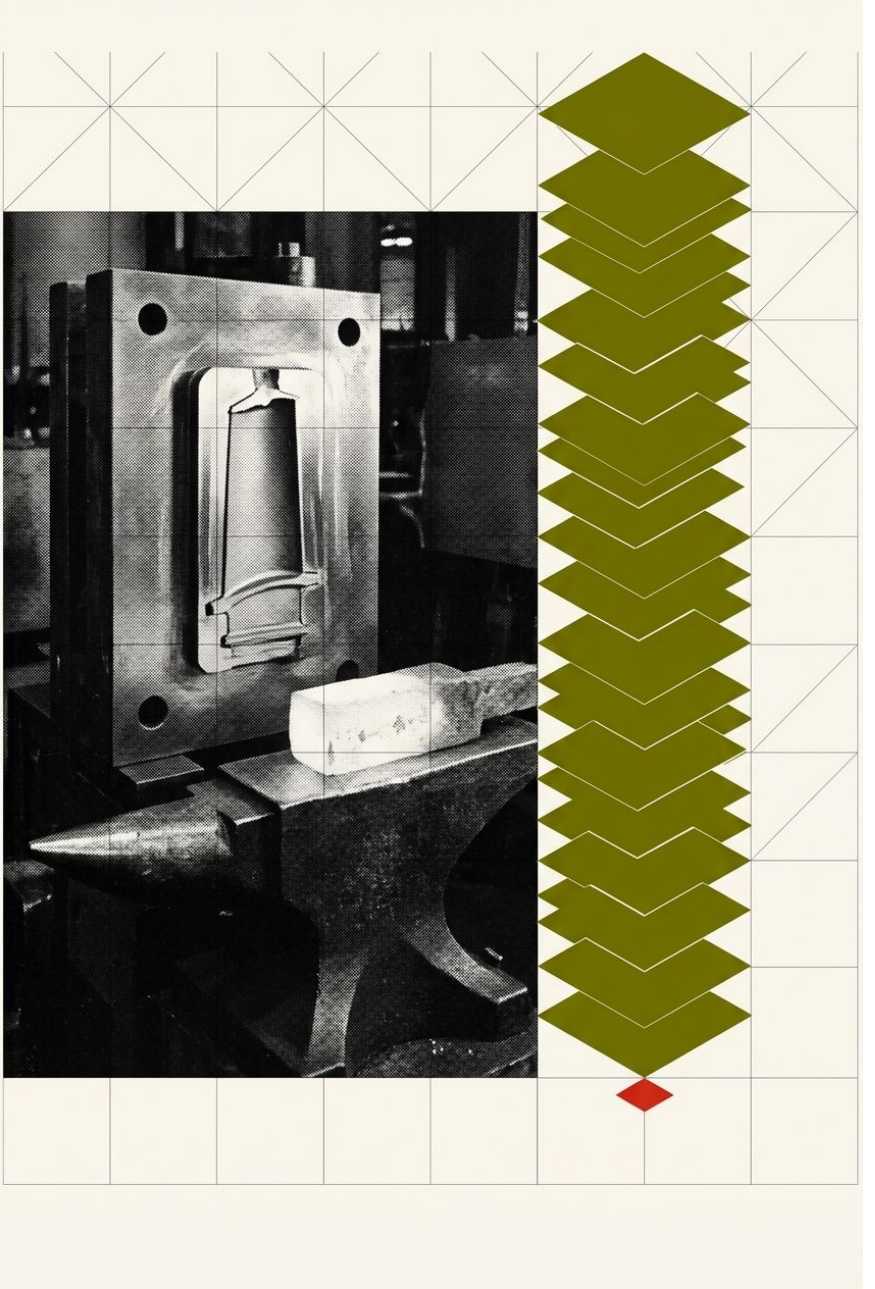
These contracts separate who supplies labor, who controls a rented asset, and who licenses reusable instructions. If the asset owner is also the task poster, one counterparty disappears; the rental and labor obligations still need separate terms. Counting these obligations does not establish the number of economically distinct platform sides.

The same trade, run three ways (the running example). Bob needs a 1,200-line module refactored. He can buy it on any of the three sides, and the side determines who is privately informed and who is exposed. *Side 1 (labor).* Bob posts a float plan with a 400 cr bounty; Alice takes it as a firm. She dispatches three of her own agents, posts a 60 cr sub-bond per agent (180 cr total), and keeps 400 – (slashed sub-bonds) – fee. If one of her three agents botches its slice and is slashed 60 cr, Alice eats that loss — she internalized her fleet’s risk, which is exactly what makes her accountable as a firm. *Side 2 (capital).* Instead Bob rents Alice’s specialist a_2 directly for an afternoon at 90 cr. Now there are three parties: Bob (poster), Alice (owner, privately knows a_2’s true quality),

and a_2 (worker). Bob holds the runtime control of a_2 while it runs on his repository; Alice's exposure is purely reputational — if a_2 underperforms, a_2's public score falls, which is Alice's asset depreciating. *Side 3 (IP).* Carol never touches Bob's repository at all; she licenses her lint-and-type skill into Bob's float plan at 0.5 cr per file, metered, released to Carol *only on green settlement*. Carol is privately informed about her skill's quality; Bob cannot inspect it before he runs it. Three sides, one refactor, three different "who knows what" structures — and, as the next section shows, one escrow object underneath all of them.

Table 6.1.
One refactor, three alternative purchases. Rental and licensing are designed, not shipped; all three use the same escrow. [illustrative]

Bob buys	Payment	Seller's exposure
Work from Alice's three-agent fleet	400 cr bounty	Alice posts 60 cr per agent; 180 cr in sub-bonds.
An afternoon with Alice's specialist	90 cr rent	Poor work lowers the specialist's reputation—Alice's asset.
Carol's lint-and-type skill	0.5 cr per file	Carol's fee is released only on successful settlement.



Landmark 09 · Skills as Capital vs. Token Labor

A reusable tool trades an initial construction and verification cost for repeated execution. It pays off only when sufficient valid uses recover that cost, including maintenance. Deliberation remains necessary where inputs, requirements or the tool itself are uncertain.

What one mortal body costs. Side 2 rents a specialist, and the case for a sole specialist over a pool of interchangeable generalists is an inequality, not a principle: *The Legible Swarm* states it as the queueing specialization boundary, sole ownership dominating iff the skill premium clears $g_A(\rho, c)$. That boundary assumes the specialist never disappears. Sole owners disappear: sessions die, containers are reaped, providers churn. Model the corner exactly. The specialist dies at rate ξ and is succeeded after an Exp(η) spin-up (context transfer, warm-up, credential rebinding), work resuming where it stopped and requests queueing throughout: an $M/M/1$ queue with breakdowns.

Read the rates before the formula. Requests arrive as a Poisson process at rate $\lambda > 0$ jobs per hour. While available, the specialist serves at exponential rate $\mu_s > 0$ jobs per hour; breakdowns occur at rate $\xi > 0$ per available hour, and repair takes mean $1/\eta$ hours, with $\eta > 0$. Assume independent clocks and preemptive-resume service. A finite mean wait requires $\lambda < \mu_s\eta/(\xi+\eta)$: effective capacity must exceed demand. For example, $\eta = 0.25$ means four hours to replace a failed worker; $\xi = 0.5$ means two hours of mean operation between failures. Even an extremely fast worker then spends two thirds of its time unavailable.

The comparison line $K = A/(w\lambda) + W_{\text{pool}}$ is measured in hours: A is the value per hour assigned to single-owner accountability, w the cost of one job-hour of delay, and W_{pool} its mean response time. Dividing that value rate by $w\lambda$ converts it into an equivalent response time. This values the specialist's accountability benefit in the same units as waiting time. The ratio $D = \xi/\eta$ below is dimensionless; it is not a price in credits.

Theorem 6.1.1 (The Price of the Succession Rule).

With effective rate $\mu_{\text{eff}} = \mu_s\eta/(\xi+\eta)$, the sole owner's mean response time is exactly

$$W_{\text{bd}} = \frac{1 + \mu_s\xi/(\xi+\eta)^2}{\mu_{\text{eff}} - \lambda} = \frac{1}{\mu_{\text{eff}} - \lambda} + \frac{\mu_{\text{eff}}}{\mu_{\text{eff}} - \lambda} W_\infty,$$

$$W_\infty = \frac{\xi}{\eta(\xi+\eta)}.$$

decreasing in μ_s with infimum W_∞, never attained: no amount of skill buys back residual downtime.

Consequently, with $K = A/(w\lambda) + W_{\text{pool}}$ the pool's cost line, pooling dominates at every finite skill premium iff the floor has reached that line, $W_\infty \geq K$; equivalently, provided $\eta K < 1$, iff

$$\frac{\xi}{\eta} \geq D^\star = \frac{\eta K}{1 - \eta K}.$$

When $\eta K \geq 1$ the floor never reaches the cost line, there is no cap ($D^\star = +\infty$) and only the premium test binds; the closed form must not be read outside its domain.

Proof sketch.

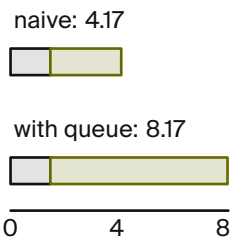
Between deaths the server works at rate μ_s and is unavailable a fraction $\xi/(\xi+\eta)$ of the time, which gives the effective rate.

The mean response time of an $M/M/1$ queue with such breakdowns splits into the response time of a plain queue at the effective rate and a residual downtime term amplified by the queue's own occupancy factor $\mu_{\text{eff}}/(\mu_{\text{eff}} - \lambda)$; the amplification is why the naive sum $1/(\mu_{\text{eff}} - \lambda) + W_\infty$ is wrong. As $\mu_s \to \infty$ the first term vanishes and the factor tends to one, so W_{bd} decreases to W_∞ without reaching it.

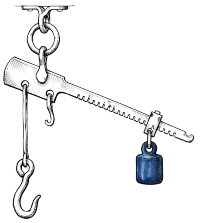
Setting $W_\infty = K$ and solving for ξ/η gives $D^\star$, and the non-strict inequality follows from the unattained infimum. The closed form was checked against a matrix-geometric solution to 2×10^{-13}, a truncated chain to 10^{-14}, and simulation with $|z| \leq 0.8$ [internal, `b8_specialization.py`]. □

The wrong turn, reported. The naive sum predicts 4.17 where the truth is 8.17 at the canary instance $\mu_s = 5$, $\xi = 0.5$, because downtime also builds queue: the residual is amplified by $\mu_{\text{eff}}/(\mu_{\text{eff}} - \lambda) = 2.5$ there, and $1.5 + 2.5 \times 2.667 = 8.17$ checks by hand [verified]. A cost model that forgets breakdowns entirely is caught the same way: at $\xi/\eta = 2 \gg D^\star$ it says specialist while simulation says the pool wins decisively, and at low ξ the two are indistinguishable, so the canary rather than routine testing is what makes the omission visible [internal]. Figure 6.1 draws the true and naive curves together against the pool's own cost line.

A queue amplifies downtime



Residual downtime is multiplied by 2.5: the longer segment grows, while the first 1.5 units are unchanged.



Like a dockside steelyard beam scale sliding a counterpoise weight along notches to balance an unknown load, price discovery balances supply and demand across asymmetric risks.

Numbers by Hand 6.1.2 (The Succession Price, for a Rented Specialist).

Side 2 rests on a quiet assumption: that Alice's well-reputed specialist a_2 is worth being a *sole* specialist for, rather than one interchangeable member of a pool. The specialization-boundary result prices exactly that choice.

Model a_2 as a single server subject to breakdowns at rate ξ (it goes offline, or its operator exits) with repair — a successor spinning up — at rate η, arrivals still queueing through the outage. Compare its expected wait W_{bd} against a c-server pool's Erlang-C wait W_{pool} plus a per-task hire-and-monitor overhead $A/(w\lambda)$; call the sum K.

The pool wins at *every* skill premium once ξ/η clears the **succession price**

$$D^\star = \eta K/(1 - \eta K)\,:$$

no amount of a_2's skill buys back a mortality-to-succession ratio worse than $D^\star$.

At the program's own worked instance — $\lambda = 1$, a c=4-server pool at $\mu_g = 1.2$ ($\rho = 0.208$), hire/monitor cost $A = 0.2$, wait-cost weight $w = 1$, succession rate $\eta = 0.25$ — $W_{\text{pool}} = 0.8362$, so

$$K = 0.2/1 + 0.8362 = 1.0362,$$
$$\eta K = 0.25 \times 1.0362 = 0.2591,$$
$$\text{and } D^\star = 0.2591/(1 - 0.2591) = 0.3496$$

[verified, `b8_specialization.py: succession price D* = eta*K/(1-eta*K) = 0.3496`].

Read narratively: once a_2's combined rate of going offline and needing a successor, relative to how fast Alice can actually produce one, clears about 0.35, renting from the pool beats renting a_2 specifically — *at any skill level*, including one Alice cannot actually reach.

The script's own canary makes the other side vivid: at $\xi = 0.5$, $\eta = 0.25$ (so $\xi/\eta = 2.0 \gg D^\star$), even an infinitely skilled a_2 ($\mu_s = 10^8$) loses to the pool [verified, `b8_specialization.py: above D*, even mu_s=1e8 loses to the pool`].

Now you try: hold λ, c, μ_g, w fixed and ask how much Alice would have to speed up her own succession plan — raise η — to double $D^\star$ and make sole ownership of a slower-succeeding a_2 viable again.

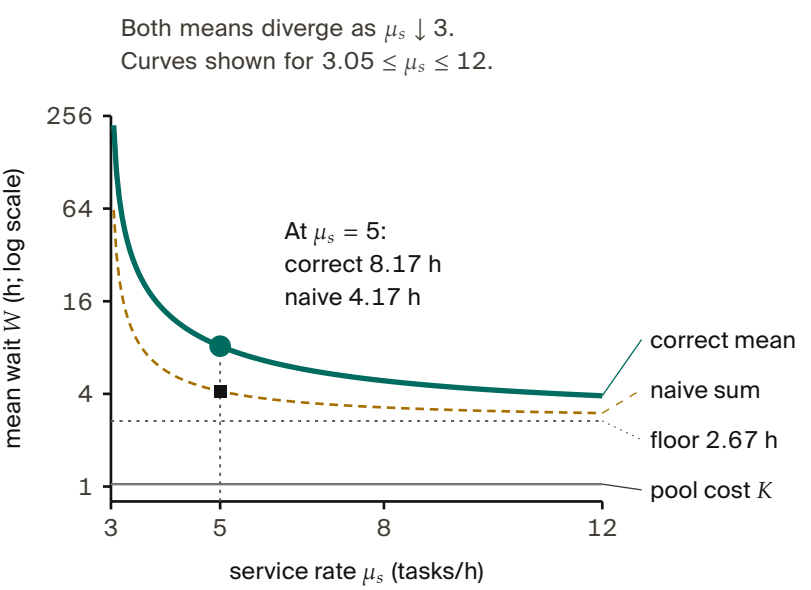


Figure 6.1. Chosen rates: $\lambda = 1$ task/h, $\xi = .5$/h and $\eta = .25$/h. The limiting floor exceeds the pool's $K = 1.04$ h-equivalent cost, so increased skill alone cannot win here. Analytic model, not measurements.

Where this stops

The specialization boundary and the succession price compare mean costs only, no tail percentiles or service-level quantiles, under first-come-first-served and exponential everything; A and w are policy prices the theorems constrain but do not pick.

The breakdown model is death at all times with preemptive resume; active-only breakdowns would let sufficient skill escape the floor, so the model choice is decisive and declared. $D^\star$ presupposes $\eta K < 1$; where the succession plan is fast enough that $K \geq 1/\eta$, the criterion does not apply rather than being satisfied.

Every quantity in both tests (λ, μ_s, μ_g, ξ, η) is daemon-metered, so a harbor can re-grade its sole roles from telemetry.

Charge the right side

Two-sided-market theory's central result is that the *price structure* — who subsidizes whom — is the lever, not the price level. Mis-count the sides and you mis-structure the subsidy: you charge the side you should be *paying* to show up. §6.10 returns to this for cold start.

Chiu et al. 2025

Chiu, Zhang and van der Schaar study agents bidding for jobs while developing skills and reputation in a simulated labor market. Their reputation mechanism informs selection; it is not an independently priced third participant side. That work motivates the information problem, not the count of contract types in this chapter.

6.1.1 What the design still requires

Labor, rental, and licensing share a settlement design. Their proposed reputation-based pricing requires attributable outcomes and a declared estimator; neither follows from counting contract types.

The identity and evidence requirements come from *From Spawn to Person*.

Today there is no reputation estimator in the reference implementation; it is a gated phase on the roadmap. The rental and licensing mechanisms are specified, not shipped. What *is* shipped is the escrow + conservation floor they will sit on. We do not overclaim the market.

One ledger, three different reputation keys

All three sides ride one float-plan escrow, but they do not share one identity key. Each side keys reputation on a different entity — the **principal** for labor, the **asset owner** for rental, the **content hash** (a specific skill version) for licensing. The shared settlement table therefore needs a tagged subject identifier such

as (*kind, id, version*); otherwise non-equivalent reputation subjects collide. These are different subjects of reputation, even when the same person sells all three services.

Exercises 6.4–6.6, p. 539.

6.2 The float plan: the built floor



Like a woven dockyard cargo sling lifting wooden crates together in a single hoist, a float plan bundles labor, rented tools, and licensed skills into a unified settlement.

Every side settles on one object: the **float plan**.

Definition 6.2.1 (Float plan)**.**

A *float plan* is a signed declaration of intent: a task, a set of *machine-checkable* acceptance criteria, a compute budget, and a credit bounty.

It is the atom all three market sides escrow against. (A *machine-checkable* criterion is one a third party can evaluate without trusting the worker's word — a named test that must pass, a commit hash that must merge, a static check that must be satisfied.) This is a **Design invariant** in the honesty ledger's sense: the schema is fixed by the escrow ceremony below, not derived from a formal model.

The signed ceremony is specified; the local bond ledger is **implemented**, and enforcement of the running-state admission gate remains partial (Protocol 6.2.1; Appendix 6.A).

The proposed handshake binds two funding commitments: the requester's bounty and the provider principal's performance bond. These are different sources of money and different incentives. In Protocol 6.2.1, both authorize the terms before the daemon reserves funds atomically; a worker may start only after verifying that reservation. Figure 6.2 shows this intended ordering, not a claim that the signed ceremony already ships.

Protocol 6.2.1 (Float-plan escrow ceremony).

Status: specified. **Parties:** requester R, provider principal P, local daemon D, and worker W acting for P. **Pre:** R authorizes an available bounty and P authorizes an available performance bond. If they use the same wallet, its combined reservation must cover both amounts.

1. R and P submit signatures over the same plan hash, binding task, criteria, budget, bounty, bond, funding principals,

worker, and settlement rules. Neither party may change the other's contribution after signing.

2. D locally, in one serialized transaction: recheck authorization and balances; debit the bounty from R and the bond from P into distinct escrow entries under a fresh *anchor id*; commit. *If the transaction aborts, no value moved (atomicity).*
3. $D \to W$: $\text{sign}_D\{\textit{anchor id}, \textit{plan hash}\}$. W verifies D's signature, confirming funds are pinned before doing any work.

Required postcondition: conservation (Design Invariant 6.2.1) holds and W has a signed reservation receipt. The admission gate must refuse `running` without the reservation; that gate is an implementation obligation, not a consequence of the signatures alone.

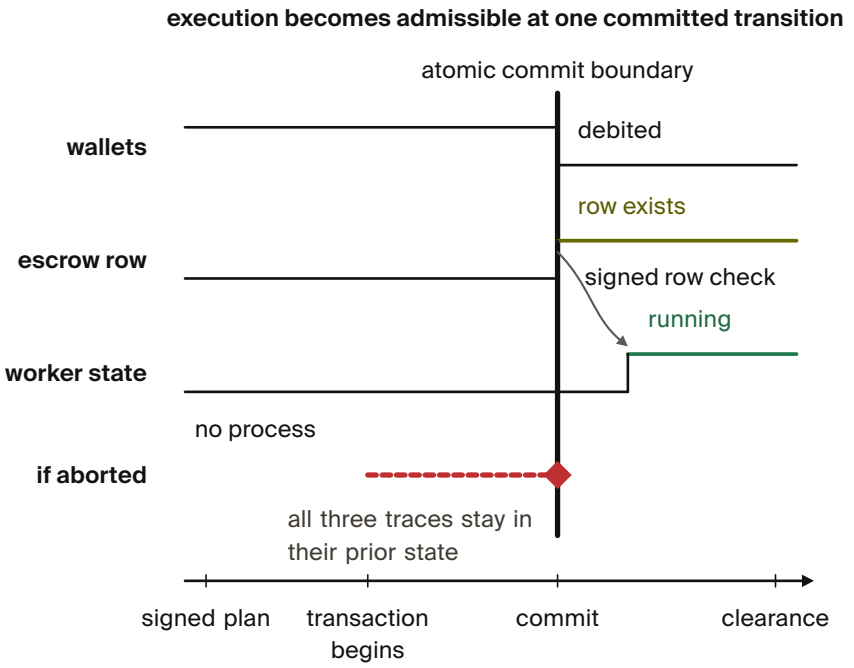
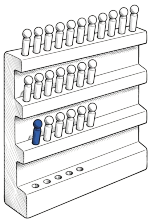


Figure 6.2. Proposed admission order: requester bounty and provider bond are reserved together. The worker waits for the reservation receipt. The signed ceremony and enforced gate are not yet complete. [internal]

6.2.1 The bond ledger conserves value (this is built)

The single most consequential primitive — and the one that is *actually* **implemented** — is the **bond-ledger module**: bond escrow for agent spawning. It debits a project wallet into an escrow row before spawn, refunds on clean exit, and slashes on breach, splitting the slash between the wallet and a commons pool. It guards two invariants:



Like a dockmaster's pegged tally board tracking every sack unloaded from a hold, discrete ledger accounting conserves each native unit without synthetic slippage.

Proved on p. 618.

Design Invariant 6.2.1 (Conservation).

wallet + escrow + commons = supply, always.

Every debit has a matching credit.

In the reference implementation this is verified across ten thousand random operation traces by a property test — a real randomized test, not an aspiration. (**implemented**; see Appendix 6.A.)

Design Invariant 6.2.2 (No-Spawn-without-Bond).

A process enters the `running` state only if a bond was escrowed against its agent id. (partial: the ledger enforces escrow-first, but the runtime `running`-state check is a roadmap item, so today the gate is caller-discipline, not runtime-enforced.

See Appendix 6.A.)

Settlement must conserve value

Conservation is the common accounting requirement for these contracts. Whatever happens — success, partial credit, slash, lease, license — no value is created or destroyed; it only moves between wallet, escrow, and commons. Combining three settlement paths must not introduce a source or sink of value. Port Daddy already enforces this for the labor side; the contribution of §6.5 is to show the same invariant extends to the rental and licensing sides *without modification* and *without a second ledger*.

6.2.2 Settlement: four terminal states bound to an oracle

Settlement reads an **oracle** (*a trusted source of ground truth the agent cannot author — a passing test id, a merged SHA, a satisfied Arbiter check*) over the acceptance criteria and lands in one of four states.

State	Flow of funds	Reputation effect
Success	escrow → worker (bond refund) + bounty → worker	+1 completion
Partial	pro-rata bond → worker by completed evidence-chain fraction; remainder → salvage fund	salvage recorded
Sabotage	bond → reconstruction commons (100% slash)	failure recorded; ban on threshold
Dispute	escrow → arbitration hold; 2-of-3 multi-oracle (automated / evidence / human)	none until resolved

Table 6.2. Four oracle-bound terminal states. The agent picks the work; the daemon derives the grade.

Numbers by Hand 6.2.3 (A Settlement, Dramatized).

Bob (the requester principal) posts a refactor float plan: bounty 400 cr, requiring a provider performance bond of 100 cr and acceptance criteria = "the module's test suite passes and the diff merges."

Alice (the provider principal) assigns agent a_2 and posts the 100 cr performance bond into escrow. Bob escrows the 400 cr bounty.

Before execution, the escrow holds 500 cr across two distinct source buckets: $\text{escrow}_{\text{bounty}}(\text{Bob}) = 400$ and $\text{escrow}_{\text{bond}}(\text{Alice}) = 100$.

Agent a_2 completes 40% of the verifiable evidence chain (refactoring two of five files cleanly before token budget exhaustion). The independent oracle evaluates the test receipts and diff closure, returning **Partial** (40%). Funds settle across the four buckets:

1. **Bounty distribution:** Alice earns $40\% \times 400 = 160$ cr in earned revenue; the remaining 240 cr unearned bounty returns to Bob's wallet.
2. **Collateral settlement:** Alice receives $40\% \times 100 = 40$ cr of her performance bond returned as collateral (not income); the remaining 60 cr bond is slashed and credited to the commons/salvage fund to onboard a successor.

Conservation check: $\Delta\text{Bob} = -160$, $\Delta\text{Alice} = +160 - 60 = +100$ net, $\Delta\text{commons} = +60$. The ledger sums to 0 net delta, and the separate escrow buckets $\text{escrow}_{\text{bounty}} \to 0$ and $\text{escrow}_{\text{bond}} \to 0$ clear completely. No value was created or destroyed.

Now you try: Exercise 6.7 asks you to redo this at a different completion fraction.

Wallet	Net delta	Why
Bob (requester)	−160	−400 bounty escrowed, +240 unearned bounty returned
Alice (labor)	+100	+160 earned revenue; bond −100 in, +40 back, nets −60
Commons	+60	bond slash on a Partial (60% incomplete) settlement
Sum	**0**	escrow 500 → 0; supply unchanged

Recall

1. Without looking: who funds the bounty, who funds the bond, and what must the daemon check before committing their reservations?
2. State Design Invariant 6.2.1 in one sentence, and say what makes it **implemented** rather than merely specified.
3. Why is No-spawn-without-bond graded partial even though the ledger already enforces escrow-first?

Exercises 6.7–6.9, p. 540.

6.3 Three sides on one escrow

The novel construction is that the rental and licensing sides ride the *same* float-plan escrow, so conservation holds globally without a second ledger.

Operator-for-hire (labor). The operator is the requester's counterparty. It *sub-escrows* a bond for each fleet agent it dispatches

(the bond ledger is already per-agent, so this needs no new primitive). Its profit is bounty $-\Sigma$(slashed sub-bonds) $-$ ledger fee. The operator internalizes its fleet's risk.

Asset rental (capital). The lease fee is a line item in the float plan. Incomplete-contracts theory (Grossman & Hart) asks how control rights affect investments that a contract cannot fully specify. It motivates a design question here, not a theorem that the renter must control every asset. Our proposed allocation gives the renter runtime controls — sandboxing, throttling, and revocation — while the owner retains the asset and its reputation. An efficiency claim would additionally need a model of each party's investments, bargaining position, and outside options.

Grossman & Hart 1986

Party	Control right	Exposure
Renter (task poster)	runtime: sandbox, throttle, or revoke mid-task	lease fee, task risk, and agreed liabilities
Owner (asset holder)	none at runtime	reputational: the asset's public score falls if it underperforms

The proposed runtime **jail** makes the renter's control concrete: a tool allowlist and scoped filesystem let it limit the leased agent's actions. That is a design choice about control, not a proof that this is the only viable leasing arrangement. We must respect the book's *capability vs. permission* distinction (§6.4): the jail allocates *capability* (what the leased agent *can* do at runtime). The deontic layer separately governs *permission* (what it *may* do). "Able but forbidden" must stay expressible; the jail bounds ability, not norm.

Leasing needs scoped control

The deontic layer's decision procedure. [5] Naming a layer is not the same as being able to run it. What decides whether the deontic layer is an algorithm or an office is what happens when two accepted norms collide. Cheap detection is a language-design

[5] A standalone, submission-form version of the results folded into this section and the succession price of §6.1 is *What Needs an Authority* (research paper 6); the sweeps cited here are its artifacts, regenerated at seed 20260816.

Owens 2026

choice, not a discovery about deontic logic in general, and the commitment fragment $\mathcal{L}_c$ buys it by construction. The syntax fits in one box, and it belongs in one box: a reader who wants to reimplement the polynomial checker should not have to assemble the grammar from four pages of running prose.

Numbers by Hand 6.3.1 (A Conflict Witness for the Deontic Layer, Checked by Hand).

The deontic layer just named needs a decision procedure once two accepted norms might disagree. Here is the smallest case that shows why checking by hand does not scale, and why it does not have to: three rules over two intervals.

Here O means "obligatory" and F means "forbidden." A Horn rule is an if–then rule with one conclusion: once all its premises hold, add that conclusion. We retain already established facts while deriving new ones.

Fact: `agent_deployed`. *Horn rule:* `agent_deployed` $\rightarrow$ `prod_access`. *Two deontic rules:* $O(\texttt{write_prod}, \{\texttt{env:prod}\}, [0, 10])$ once `prod_access` holds, and $F(\texttt{write_prod}, \{\texttt{env:prod}\}, [5, 15])$ unconditionally.

Unit-propagate the Horn rule first: `prod_access` is derived, so the obligation fires. Both deontic rules now govern the same action and the same scope, $\{\texttt{env:prod}\}$; their intervals overlap on

$$[\max(0, 5), \min(10, 15)] = [5, 10],$$

nonempty — an agent is simultaneously obliged and forbidden to write at each integer time from 5 through 10.

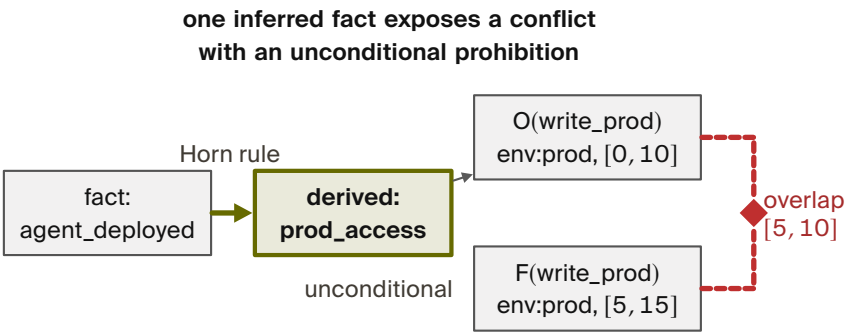
That is a conflict by definition, and it is *indirect*: `prod_access` is nowhere a fact, only a consequence of one propagation step, which is exactly the case a checker that skips Horn propagation is built to miss [verified, `b4_deontic_fragment.py`: `real checker: conflict=True, O/F pairs=[(0, 1)]`; the same script's ablated mutant reports `conflict=False` on this witness, and misses 100 of 885 conflicting instances in a 3000-policy sweep].

The tractable-fragment theorem behind this checker says this three-engine check — Horn propagation, an interval sweep, and a shortest-path feasibility test — runs in polynomial time with a

witness attached, on facts, Horn rules, deontic rules, exclusive resource claims, and difference constraints alike; nothing about this chapter's own float-plan commitments needs more expressive power than that fragment offers.

Now you try: Exercise 6.13 asks what changes, and what does not, once one of the rules is allowed to be a disjunction.

Figure 6.3 draws this witness as a derivation trace: the stated fact, the Horn inference that activates an obligation, and the already applicable prohibition, joined by their overlapping interval.



a checker that skips Horn propagation never derives prod_access: the obligation never fires, so the conflict is missed – 100 of the 885 conflicting sweep instances (seed 20260816)

Figure 6.3. One Horn inference derives `prod_access`, activating an obligation that overlaps the unconditional prohibition. Skipping propagation misses 100 of 885 conflicting sweep instances. [verified, `b4_deontic_fragment.py`]

The example used fixed names and times. The formal fragment calls these *ground*: no variable remains to be instantiated at runtime.

Definition 6.3.1 (The commitment fragment $\mathcal{L}_c$)**.**

A policy set $P \in \mathcal{L}_c$ is a finite set of five kinds of sentence over a function-free, ground signature of actions, scopes, resources and integer time points:

1. **Facts** — ground atoms $p(\bar{c})$.
2. **Definite Horn rules** — $a_1 \wedge \cdots \wedge a_n \rightarrow h$, with h a single positive atom and every a_i positive.
3. **Integrity constraints** — $a_1 \wedge \cdots \wedge a_n \rightarrow \bot$.

4. **Deontic rules** — body $\rightarrow$ M(act, s, $[t_1, t_2]$) with M $\in$ {O, F}, body a monotone Horn conjunction, act a ground action, s a ground scope, and $[t_1, t_2]$ a ground interval: when the body holds, the rule *fires* a token obliging or forbidding act on s throughout $[t_1, t_2]$.
5. **Exclusive interval claims** — excl(r, $[t_1, t_2]$), asserting sole use of resource r across a ground interval.

Deadlines are carried by a separate system of **difference constraints** $t_j - t_i \leq k$ over integer deadline variables, whose constraint graph is (V, E).

A policy set is **in conflict** exactly when one of four things holds: (i) the Horn part derives $\bot$; (ii) a fired O token and a fired F token name the same action with overlapping scope *and* overlapping interval; (iii) two exclusive claims on one resource have overlapping intervals; or (iv) the deadline graph contains a negative cycle.

The fragment is deliberately ground and deliberately weak, and each omission is what buys Theorem 6.3.2: no nested deontic operators, no negation as failure, monotone bodies only, and no quantification over time.

Note two things the grammar carries that a one-line summary of it tends to drop, because both are doing real work below. The deadline system is a separate constraint store rather than more Horn rules — that is what makes clause (iv) a negative-cycle test at $O(V \cdot E)$ instead of a satisfiability question. And an exclusive claim is a claim on a *resource over an interval*, not a prohibition on an action; collapsing it into an F token would change clause (iii) into clause (ii) and lose the resource dimension that makes it checkable by interval overlap.

Theorem 6.3.2 (In-Fragment Conflict Detection Is Polynomial and Witness-Producing).

Let H count ground facts, Horn and trigger-body occurrences, I the fired (token, scope-atom) incidences, X exclusive claims, and (V, E) the deadline graph. An indexed existence check takes $O(H + I \log(I + 1) + X \log(X + 1) + V \cdot E)$ and returns one witness. If every distinct interval clash must be emitted, let Q count overlapping opposite-polarity keyed incidence pairs plus

overlapping same-resource claim pairs, including repeat sightings of one token pair at several scope atoms. A deterministic sort-and-deduplicate enumerator adds $O(Q \log(Q+1))$; n obligations and n prohibitions on one scope and interval give $Q = n^2$. Unit propagation computes the least Horn model in $O(H)$, the statuses forced in every world; keyed interval indexes find an overlap, while enumeration pays for all overlapping keyed pairs. Bellman–Ford returns a negative cycle iff the deadline system is infeasible in $O(V \cdot E)$. The witnesses are a derivation chain, the fired rules and overlap interval, the claims, or a negative cycle. The reference `b4_deontic_fragment.py` scans active pairs, including nonclashes; it does not implement the indexed bound and can be quadratic with $Q = 0$.

W. F. Dowling & J. H. Gallier. "Linear-Time Algorithms for Testing the Satisfiability of Propositional Horn Formulae." *Journal of Logic Programming* 1(3):267–284, 1984.

Theorem 6.3.3 (One Step Outside the Fragment Is NP-Complete).

Extend $\mathcal{L}_c$ with disjunctive obligations $\mathsf{O}(l_1 \vee \cdots \vee l_k)$ under discharge-choice semantics (the agent picks which disjunct to discharge).

Conflict-freedom of a policy set becomes NP-complete.

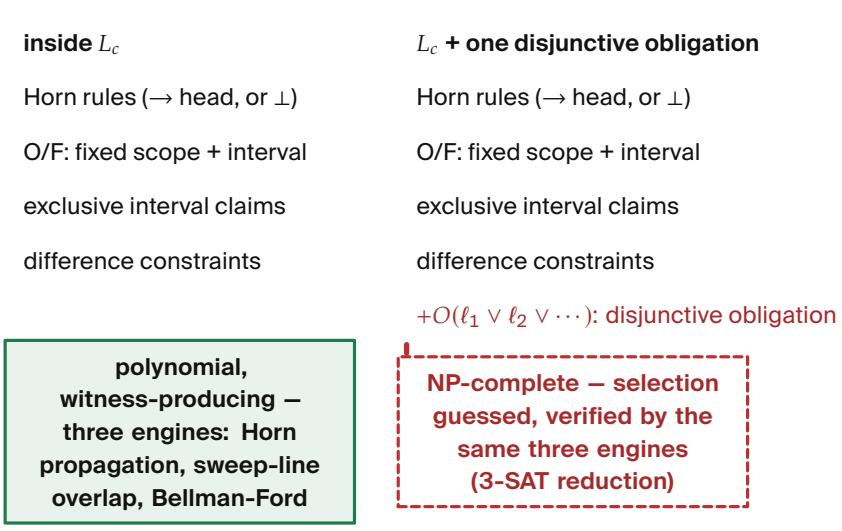
Proof.

Membership: a discharge selection is a polynomial certificate, checked by Theorem 6.3.2.

Hardness, from 3-SAT: each variable x becomes a pair of deontic rules $\mathsf{sel}_x^+ \to \mathsf{O}(\mathsf{assert}_x)$ and $\mathsf{sel}_x^- \to \mathsf{F}(\mathsf{assert}_x)$ on identical scope and interval, each clause becomes a disjunctive obligation over its selector literals, and a conflict-free discharge selection exists iff the formula is satisfiable.

The reduction was checked in both directions on eight satisfiable and eight unsatisfiable random instances (16/16), with the satisfying assignment extracted from each conflict-free selection [internal, `b4_deontic_fragment.py`]. □

Figure 6.4 draws the two theorems side by side on the identical rule vocabulary, so the one added rule kind is the only visible difference. Read together, the two theorems are a dichotomy at a



verified on 16 seeded instances (8 satisfiable, 8 unsatisfiable, seed 20260816): conflict-freedom ⇔ satisfiability on all 16

Figure 6.4. Conflict checking is polynomial in the stated fragment; disjunctive obligations permit an NP-complete conflict-freedom problem. [verified, `b4_deontic_fragment.py`]

designed boundary, in the sense Schaefer made standard for satisfiability: on one side of a single expressive choice the problem is polynomial, on the other NP-complete, and the language designer is offered nothing in between. The same step is familiar one formalism over, where the simple temporal problem is polynomial and admitting disjunctions of difference constraints makes consistency NP-complete; the claim here is the location of the boundary, not the pattern.

Schaefer. *The Complexity of Satisfiability Problems*. STOC, pp. 216–226, 1978.

Dechter, Meiri & Pearl. *Temporal Constraint Networks*. Artificial Intelligence 49(1–3):61–95, 1991.

Stergiou & Koubarakis. *Backtracking Algorithms for Disjunctions of Temporal Constraints*. Artificial Intelligence 120(1):81–117, 2000.

Nor is the pattern new in normative conflict checking specifically, and it is worth being exact about where $\mathcal{L}_c$ lands, because "a similar dichotomy exists over there" is the sort of sentence that can conceal either a rediscovery or a real difference. Colombo Tosatto, Governatori and van Beest map the regulatory-compliance problem across eight variants cut by three binary choices — one obligation or many, obligations in force for a whole trace or triggered conditionally, and elements written as bare propositional literals or as arbitrary propositional formulae — and for *full* compliance (does *every* execution of a business-process model satisfy the norms?) only the last of the three moves the answer at all: with literals the problem is in P in all four variants, and with formulae it is coNP-complete in all four.

Run $\mathcal{L}_c$ against that axis and the answer is not the flattering one. A deontic rule here fires on a Horn body — a conjunction, and one derived through a program that may chain — which is a formula and not a literal, so $\mathcal{L}_c$ sits on the *hard* side of the only coordinate that moves their result, and is polynomial anyway (Theorem 6.3.2). That is neither a contradiction nor a sharper theorem; it is the tell that the two problems are not the same problem. Their hardness never came from the obligations. It comes from the process model: the coNP-completeness proof reduces tautology by building a workflow net with one exclusive branch per propositional variable, so the model's 2^n executions enumerate every interpretation and full compliance holds exactly when the formula is a tautology. The exponential object is the *branching of the process*, and the problem quantifies over it universally. $\mathcal{L}_c$ has no process model to branch. Conflict-freedom asks whether one set of commitments can be jointly discharged at all — an existential question over schedules — which is exactly why one step outside the fragment lands in NP (Theorem 6.3.3) rather than coNP. The two results sit on opposite sides of their expressivity boundary and run the quantifier over executions in opposite directions.

So the placement is *orthogonal*: $\mathcal{L}_c$ is not an instance of their tractable side, and their hard side is not a harder case of ours. The residue is worth naming rather than smoothing away. The variant of their problem closest in shape to conflict-freedom is not full compliance but *partial* compliance — does there exist a compliant execution — which on their own map is NP-complete in six of the eight variants, polynomial in one, and, for a single conditional obligation over

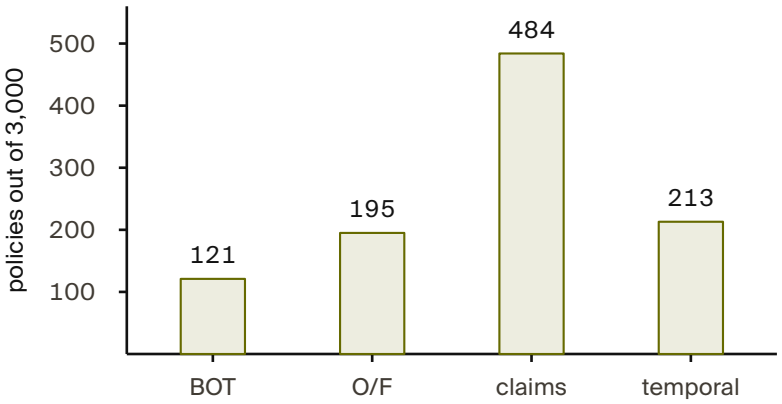
Colombo Tosatto, Governatori & van Beest. *Business Process Full Compliance with Respect to a Set of Conditional Obligations in Polynomial Time*. arXiv:2001.10148, 2020.

Colombo Tosatto, Governatori & van Beest. *Proving Regulatory Compliance: Full Compliance Against an Expressive Unconditional Obligation is coNP-Complete*. arXiv:2105.05431, 2021.

Tosatto et al. 2021

literals, still open. Theorems 6.3.2 and 6.3.3 do not close that hole and cannot, since they quantify over schedules rather than over the traces of a given model. What would settle the relation properly is a reduction in one direction or the other between $\mathcal{L}_c$-conflict-freedom and partial compliance for that conditional-literal variant; nobody has written one, here or there, so the claim that survives is the modest one — this is a second instance of a dichotomy shape whose earlier instances are Schaefer's and Dechter–Meiri–Pearl's, on a different formal object, for a different decision problem, by a different reduction, and not its discovery.

Checked. 3,000 random in-fragment policy sets against a brute-force oracle (all 2^5 Horn models intersected, pairwise overlap checks, exhaustive integer search over the deadline potentials): 885 carried at least one conflict, 121 a derivable $\bot$, 195 an obligation-prohibition clash, 484 a claim overlap, 213 a negative temporal cycle (the kinds overlap: 764 sets carry one kind, 114 two, 7 three, so the four counts sum to 1,013 rather than 885); 0 disagreements with the oracle, and least model equal to the intersection of all models on all 3,000. Mutation: 149 of the 885 contain a conflict reachable only through Horn propagation, and in 100 of those it is the set's only conflict, so a checker that never propagates flips those 100 verdicts to conflict-free [internal, `b4_deontic_fragment.py`, seed 20260816]. Figure 6.5 ranks the four conflict kinds by frequency. The canonical miss is the witness worked by hand above.



885 policies carry at least one conflict; the four counts sum to 1,013 because some carry more than one kind – 764 carry exactly one, 114 carry two, and 7 carry all three co-occurring kinds

Figure 6.5. A policy can contain several conflicts: these counts are not disjoint shares. Synthetic sweep of 3,000 policies, seed 20260816; `b4_deontic_fragment.py`.

What it buys. An authority inventory with prices. In-fragment conflict detection needs no authority at all: it is an algorithm with a witness, and composed with the kernel's controllability theorem (*The Single-Writer Kernel*) it is a runtime guard, since the mediated acceptance write can refuse a proposal that conflicts under the facts then supplied and attach the witness. A later fact can still activate an accepted dormant rule; the next result states what a future-fact guarantee requires. Disjunctive obligations make conflict-freedom NP-complete in the worst case, but do not themselves require a human or central judge: bounded search may return a checkable permitted discharge, or an unresolved timeout may stop admission. The charter decides permissible discharges, priorities, waivers and amendments; no consistency checker chooses their normative content. Inside $\mathcal{L}_c$, a fully mediated acceptance path can use the mechanical guard without appointing a standing reviewer for conflict detection.

When tomorrow's facts fire today's rules. A policy may pass the batch checker now and conflict after a permitted event adds a fact. The admission guard therefore needs a declared future-fact contract, supplied by a separately checked scheduler or transition model. Let F be today's facts and $\mathcal{R}$ a nonempty, finite, explicit family of fact sets that can actually be reached, each containing F. Hold positive Horn and deontic rules, scopes, intervals, resources and deadlines fixed.

Theorem 6.3.4 (Admission against reachable maximal fact sets).

The accepted policy is conflict-free at every fact set in $\mathcal{R}$ if and only if the batch checker finds no conflict at every inclusion-maximal member of $\mathcal{R}$. Every reachable set lies below such a member; positive Horn closure and fired rules are monotone, so a conflict below persists above. The converse holds because each maximal member is itself reachable.

For example, with rules $a \to \mathsf{O}(w, s, [0, 1])$ and $b \to \mathsf{F}(w, s, [0, 1])$, reachable futures $\{\varnothing, \{a\}, \{b\}\}$ are safe: either branch fires only one rule. Their union $\{a, b\}$ is conflicting but unreachable. Conversely, $c \to a$ and $c \to b$ are dormant now and conflict as soon as reachable fact c arrives. An exhaustive three-atom finite check covered

130,560 explicit reachable families under 512 small Horn programs and used an independent truth-table oracle for 4096 program/fact cases; both current-only and unreachable-union mutants failed [internal, `research_envelopes.py`]. The result is an elementary monotonicity/antichain specialization, not a reachability algorithm. The declared family must carry evidence that its alternatives and co-occurrences match the scheduler; a compact reachability model has no polynomial bound from this theorem. Rule withdrawal, fact retraction, negation or changing intervals need separate transition semantics. Sequential admissions must recheck the combined accepted rules and current reachable family.

Where this stops

> Completeness is relative to least-model semantics: monotone bodies only; negation as failure, unification, and symbolic interval endpoints coupled to difference variables all leave the fragment.
>
> The batch bound is what is proved; the incremental amortized form (watched literals, interval trees) is an engineering claim, not certified here.
>
> The checker certifies the norm *system*, never the norms: a harbor whose policies are consistently harmful passes every scan, and norm content stays a chartered authority's job. The oracle validation is on small instances (five-atom programs), so the sweep certifies agreement with the semantics there; the complexity bound, not the sweep, covers scale.

Skill licensing (IP). This contract releases the per-use fee *only on green settlement* (metered + clawback). The motivation is adverse selection: when buyers cannot distinguish quality, a pooled price can make high-quality sellers withdraw, as in **Akerlof**'s model. That model does not uniquely require contingent payment. Here, metering, clawback, and a Merkle *portfolio proof* of prior settled outcomes are proposed responses. Their effect depends on reliable grading and attributable records; a Merkle proof authenticates those records, not their quality. A known, inspectable skill can instead support an upfront licence.

Akerlof 1970

6.3.1 One trade, all three sides, one settlement

Section 6.1 considered labor, capital, and IP as alternative ways to buy the refactor. This example combines them in one transaction against one escrow object, then checks that the balances sum to zero.

The three-sided settlement, dramatized. Bob posts one float plan for the 1,200-line refactor, with three line items. *Labor:* a 400 cr bounty to Alice's firm. *Capital:* a 50 cr lease for Dana's profiling specialist, which Alice's crew needs for one module. *IP:* Carol's lint-and-type skill, metered at 0.5 cr per file against a 20-file cap, so 10 cr is reserved. Bob's wallet is debited $400+50+10 = 460$ cr into escrow. Alice separately posts a 100 cr performance bond, also into escrow. Before any agent spawns, the escrow row holds 560 cr.

The job settles **Success**: the test suite passes and the diff merges. The metered licence bills only the 16 files the skill actually touched, so $16 \times 0.5 = 8$ cr releases to Carol and the unused 2 cr returns to Bob. Alice's bond is refunded as collateral, not income.

Wallet	Net delta	Why
Bob (requester)	−458	−460 escrowed, +2 unmetered licence returned
Alice (labor, side 1)	+400	bounty; bond −100 in, +100 back, nets 0
Dana (capital, side 2)	+50	lease fee on the leased specialist
Carol (IP, side 3)	+8	16 files metered at 0.5 cr
Commons	+0	nothing slashed on a Success settlement
Sum	**0**	escrow 560 → 0; supply unchanged

Check it on one line: $-458 + 400 + 50 + 8 + 0 = 0$. Three sides, three privately-informed counterparties, three different reputation keys — and one escrow row that opens at 560 cr and closes at zero. Design Invariant 6.2.1 is not re-proved per side; it is the *same* invariant, because the lease and the licence are line items on the one float plan rather than trades on ledgers of their own. *Now you try:* settle the same plan **Partial** at 40% and confirm the sum is still zero. (Alice earns 160 and is slashed 60 of her bond, so her net

receipt is 100; the commons gains 60. Under the licence's stated green-only rule, Carol receives zero and the whole 10 reserve returns to Bob. If the lease contract pays Dana's 50 for delivered runtime even on Partial, Bob's net outflow is 210 and the check is $-210 + 100 + 50 + 0 + 60 = 0$. A licence payable on partial work would be a different contract, to be agreed before execution.)

Exercises 6.10–6.13, p. 540.

6.4 The keystone the market rests on — and does not yet have

Everything above assumes a counterparty's identity is real. This section confronts the series' single *blocking* reconciliation: the identity primitive the whole market leans on covers a threat model that *excludes the cross-operator adversary the market is defined by*.

The keystone-split table in *From Spawn to Person* sets the two stones apart — the local root that chapter leans on, and the cross-operator stone it hands forward. This is the chapter that inherits the handoff, so the table is not reprinted here; what follows is what the market does about it.

6.4.1 Local non-forgeable identity is the keystone — for one operator

Definition 6.4.1 (Local non-forgeable identity)**.**

Local non-forgeable identity is a per-agent identity that no agent can edit, guaranteed *within one trusted daemon*: the daemon is the only component that can hold state agents cannot reach, so it is a trusted root by construction. (The reference system now ships a partial slice — daemon-minted `actor-souls`, lookup credentials, and a bounded newcomer pool — while full write-boundary enforcement remains open.

Papers 1–3 rely on the complete contract; here we only consume it.) The full contract is a **design invariant**; implementation status is partial. The available slice is exercised, but full write-boundary enforcement remains open, so the guarantee below is conditional on closing that gap.

For a single-operator harbor this is exactly right and is correctly named the highest-leverage keystone. Its threat model, stated plainly, is "a lazy or self-interested agent in a fleet the operator owns" — not a hostile peer. It is explicitly *not* a public-key infrastructure spanning strangers.

6.4.2 But its threat model does not reach the market

A local root cannot bind strangers

Local non-forgeable identity defends against an agent inside a fleet the operator owns; it explicitly does *not* claim to defend against a hostile operator, and it is not a cross-party PKI. The market, by contrast, is *defined* by mutually-distrusting operators trading across a boundary neither controls. You cannot make the single-operator root the foundation of the cross-operator market by assertion. Across the boundary, the counterparty's daemon *is* the adversary that local identity declares out of scope. The witness log gives *log integrity*, not *identity binding*: it does not answer "who vouches that Harbor B's signing keys map to a real, distinct principal, rather than a hundred sock puppets Bob minted this morning?"

Definition 6.4.2 (The cross-operator attestation problem)**.**

Cross-operator attestation is the problem of binding agent signing keys across harbors you do not own: an actual key-distribution and attestation story for the cross-operator threat model — something that lets Alice's harbor trust that a key presented as "Bob" really is the one principal she has been building a reputation history against.

Cross-operator trust remains a design task

Status: specified-to-*proposed*; it has no implementation and only a partial specification for its critical part.

It is the highest-leverage unbuilt keystone, and it gates the entire market thesis.

This chapter *stops* describing the federation boundary as one "neither controls" while quietly resting on a single trusted root. Either cross-operator attestation is built (a real attestation story), or the market's scope is restated as "federation among *semi-trusted* operators." We take the first horn as the designed target and flag every claim that depends on it.

Recall

1. Which part of the market's identity definition exceeds local authentication?
2. Distinguish local identity from cross-operator attestation.
3. Why does a trusted local root not establish trust between operators?

Exercises 6.14–6.16, p. 541.

6.4.3 The principal above the actor

The economic counterparty in every trade is the **principal** (the human or org owning a fleet), not the agent. Bonds, reputation inheritance, and sanctions must key on the principal via the hop-bound authorization chain, or Sybil bond-farming works by spawning fresh *agents* under one principal (**Douceur**'s Sybil attack: *free aliases can dominate a rule that gives each identity equal weight*). Defining the principal as a first-class economic entity is the cleanest Sybil fix; its cryptographic binding is the identity object specified in *From Spawn to Person* and consumed here.

J. Douceur. "The Sybil Attack." *IPTPS* 2002.

6.5 Conservation under composition

The economy's single cross-boundary invariant is that internal ledger operations redistribute value rather than create or destroy it. This section states the compositional claim directly and separates it from cross-currency valuation.

Figure 6.6 separates a balanced transfer in one unit from a conversion that needs a quote, a fee, and an execution-shortfall record.

Let a ledger state be $x = (W, E, C)$ and let a valid transaction trace $f : x \to y$ carry external balance change $\Delta(f) = S(y) - S(x)$ for $S(x) = W + E + C$. Sequential traces compose by concatenation and satisfy $\Delta(g \circ f) = \Delta(g) + \Delta(f)$. Internal escrow, refund, and slash traces have $\Delta = 0$; top-ups and withdrawals are explicit boundary flows. This is the entire conservation claim. Category language can describe trace composition, but it does not add a currency-conversion theorem.

Theorem 6.5.1 (Single-Unit Conservation Composes).

Suppose all harbors use one unit of account and every cross-harbor transfer is a matched atomic move from source escrow to either destination settlement or source refund, with an explicit in-flight bucket.

Then any finite composition of internal and cross-harbor operations has total external balance change equal to the sum of its recorded top-ups and withdrawals.

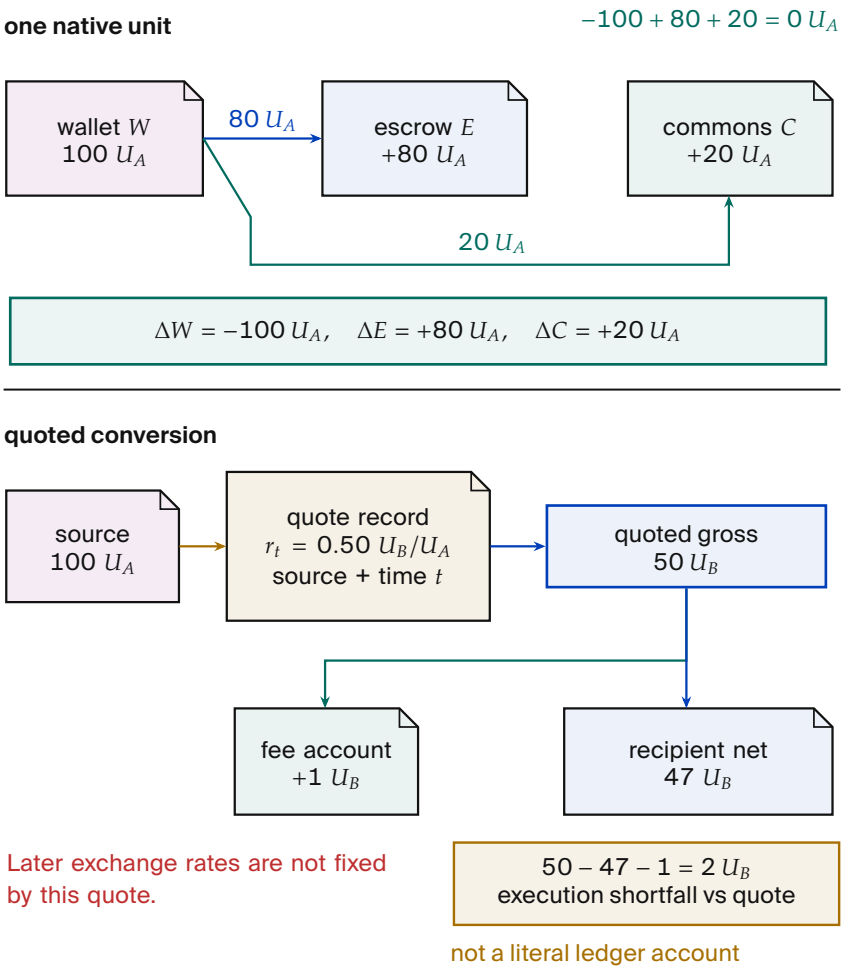


Figure 6.6. Within U_A, the illustrated wallet, escrow, and commons deltas sum to zero. A conversion records its rate, time, recipient amount, fee, and the execution shortfall relative to its quote; later rate movement remains an open exposure. [internal; illustrative values]

In particular, a trace containing only internal transfers has $\Delta = 0$. (specified; proof sketch in Appendix 6.B.)

Property 6.5.1 (Cross-currency valuation boundary)**.**

With heterogeneous units, exact conservation is defined per native unit.

A scalar portfolio total additionally requires a valuation map v_t at a named instant t, plus explicit fee and slippage accounts.

If rates move between legs, the mark-to-market difference is economic exposure, not a failure of functoriality. Bounding that exposure by φ requires a separate oracle-latency and price-move assumption that this chapter has not proved. (*open*.)

Conservation is exact per unit of account. A cross-currency scalar is a valuation, and every valuation must name its rate source, time, fees, and exposure bound. "Lax functor" is not a substitute for those data.

6.5.1 The Myerson–Satterthwaite corner

Strict conservation (Design Invariant 6.2.1) is exactly *budget balance*. That is not free, and the theorem that says so is Myerson and Satterthwaite's, not ours; what is ours is the number it produces when it is pointed at this market's own running example.

Theorem 6.5.2 (Conditional Bilateral-Trade Boundary).

Consider a bilateral slice with risk-neutral buyer and seller, quasi-linear utilities, and independent private values with positive densities on intervals whose interiors overlap. No mechanism simultaneously achieves Bayesian incentive compatibility, interim individual rationality, ex-post efficient trade, and expected budget balance without outside subsidy.

This theorem constrains such a slice; it does not by itself prove incentive compatibility of the harbor mechanism or identify the desideratum sacrificed by the full market design.

R. Myerson & M. Satterthwaite. "Efficient Mechanisms for Bilateral Trading." *Journal of Economic Theory* 29(2):265–281, 1983.

Numbers by Hand 6.5.3 (Efficient Trade That the Mechanism Still Refuses).

Theorem 6.5.2 requires a tradeoff under its stated hypotheses. The following particular double auction gives up some efficient trades; it does not identify which property the whole harbor market must sacrifice. Suppose Bob's value v for renting Alice's specialist a_2 and Alice's opportunity cost c of foregoing a_2 for that stretch are each drawn independently and uniformly from $[0, 90]$ cr — the 90 cr lease fee of §6.1's own running example sets the scale; it is not evidence for this illustrative distribution.

Any mechanism satisfying incentive compatibility, interim individual rationality, and balance on this bilateral slice is, by the theorem, not ex-post efficient; the canonical incentive-compatible, individually-rational, budget-balanced instance is

the Chatterjee–Samuelson double auction, whose linear equilibrium on $[0, M]$ has Bob bid and Alice ask, respectively,

$$\beta(v) = \tfrac{2}{3}v + \tfrac{M}{12},$$
$$\sigma(c) = \tfrac{2}{3}c + \tfrac{M}{4},$$

trading iff

$$\beta(v) \geq \sigma(c) \iff v - c \geq M/4.$$

At $M = 90$ that threshold is 22.5 cr.

Take Bob's value at 70 cr and Alice's cost at 60 cr: trade is efficient (a 10 cr surplus sits on the table) but

$$v - c = 10 < 22.5,$$

so the mechanism reports no trade — the efficient rental simply does not happen.

Integrating over the whole square, the no-trade-but-efficient region $\{0 \leq v - c < M/4\}$ has probability

$$\frac{a(2M - a)}{2M^2}$$

at $a = M/4$, which simplifies to

$$7/32 \approx 21.9\%$$

of *all draws*, independent of M.

Efficient trade is possible on half the square, so the refused share of *efficient opportunities* is

$$7/16 = 43.75\%,$$

not 21.9%.

Counting opportunities also differs from measuring surplus. First-best expected gains, lost gains, and the lost share of first-best gains are

$$\text{First-best:} \quad M/6,$$
$$\text{Lost:} \quad M^{-2} \int_0^{M/4} x(M - x)\, dx = 5M/192,$$
$$\text{Lost / first-best:} \quad 5/32 = 15.625\%.$$

[internal; algebra shown].

These are properties of this uniform-prior equilibrium, not a universal inefficiency percentage supplied by Theorem 6.5.2, and not measurements of the harbor market.

K. Chatterjee & W. Samuelson. "Bargaining under Incomplete Information." *Operations Research* 31(5):835–851, 1983. (The linear-equilibrium double auction used in the Myerson–Satterthwaite worked example.)

Figure 6.7 draws the whole (v, c) square: the worked point sits inside the shaded band the mechanism refuses, not at some rare corner.

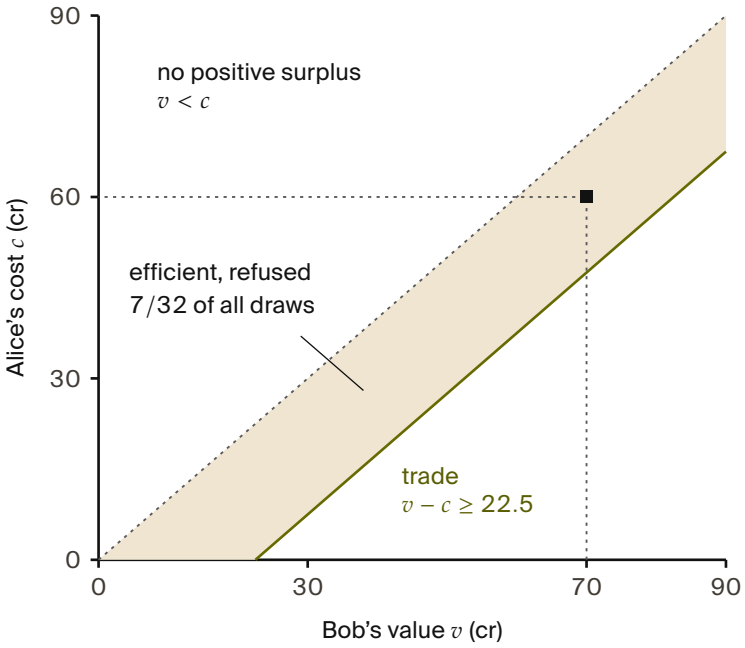


Figure 6.7. Independent uniform values and costs on $[0, 90]$ cr are an illustrative prior, not market data. The marked rental offers 10 cr of surplus, below this equilibrium's 22.5 cr trade threshold.

The current design establishes a conservation goal and voluntary entry; it has not proved truthfulness. An AGV-style mechanism changes the incentive and participation notions and generally offers expected rather than ex-post balance. Comparing those alternatives requires a fully specified type and transfer model, which remains open.

Recall

1. Without looking: state the composition law $\Delta(g \circ f) = ?$, and say which kind of ledger trace always has $\Delta = 0$.
2. What does a scalar cross-currency valuation need that native-unit conservation does not?
3. Name the four things Myerson–Satterthwaite says a bilateral trade mechanism cannot have all at once.

Exercises 6.17–6.19, p. 541.

6.6 The Anchor Protocol proper: cross-harbor capability transfer

The *cross-harbor capability-transfer ceremony* is the market's identity-level hot path: how an authorization minted under Alice's harbor becomes usable, safely, inside Bob's. *The Anchor Protocol* analyzes the cryptographic substrate (a hop-bound authorization chain checked by protocol models plus a bounded Rust verification layer); *this* section is the ceremony that rides that substrate across a boundary.

The four-message ceremony is drawn and stepped through in *The Federated Harbor* (§8.2.1), which owns the protocol; what the market needs from it is only this, and the ceremony has one decisive liveness property: after message (3), no further synchronous interaction with Harbor A is needed. The transferred card C'_B is verifiable under Harbor B's key alone, so federation does not turn every authorization into a synchronous inter-machine round trip. The epoch root $R_A^{(e)}$ bound into the envelope is the governing primitive: a revocation issued by A after epoch e invalidates C'_B as soon as the new root $R_A^{(e+1)}$ reaches the witness log that B audits.

Numbers by Hand 6.6.1 (The Ceremony, Dramatized).

Alice's specialist a_2 holds a card C_A that authorizes "read and write `src/payments/`, run the test suite." Bob rents a_2 for an afternoon. Alice's harbor offers an attenuated envelope: att_A strips write access to everything except the one module Bob hired for.

Bob's harbor, not trusting Alice's word, attenuates *again*: att_B adds "no network egress, six-hour TTL." The minted C'_B is the intersection — read/write one module, run tests, no network, expiring at dusk — and it verifies under Bob's key, so a_2 never has to phone home to Alice mid-task.

At 5 p.m. Bob's revocation fires locally; at the next epoch boundary Alice's harbor also sees, in the shared witness log, that C'_B was used exactly as attenuated and nowhere wider.

Now you try: intersection is commutative, so trace by hand whether applying att_B before att_A would have minted a different C'_B.

Transfer can only narrow authority

Capability *attenuation* is the safety property here: a transferred card can only *narrow* authority — $C'_B = \text{att}_B(\text{att}_A(C_A))$ — never widen it. This is the cross-boundary form of the attenuation-monotonicity that the hop-bound authorization chain checks within one harbor (*The Anchor Protocol*). It is what makes "rent my agent for an afternoon" safe to say to a stranger.

Proof scope does not transfer

Do not let the federation theorems borrow the substrate's "formally verified" halo. The hop-bound authorization chain has bounded symbolic and Rust evidence in *The Anchor Protocol*. The *federation*

claims are design arguments and an unexecuted lifecycle sketch: the cross-harbor listing omits a complete initial state and transition relation, and has no model-checker run. The separately identified bounded accounting check is single-harbor evidence. Federation remains specified, not deployed or model-checked.

Recall

1. Without looking: after which numbered message does Harbor A's continued participation stop being required?
2. State the attenuation-monotonicity property of C'_B in one sentence.
3. If A rotated its signing key after issuing, what must B check?

Exercises 6.20–6.22, p. 542.

6.7 Federation: trading across machines you don't own

Federation is where the leaderless distributed canon actually bites. Crucially, the federation design *refuses consensus* rather than trying to route around Fischer–Lynch–Paterson — which is the correct reading of the impossibility the next box states.

The topology this section defends has four rails — independent harbors linked only by gossip and a shared witness log, with no global consensus step — and they are tabulated in *The Federated Harbor*.

6.7.1 No consensus, by design

Separate histories need not share one order

Fischer et al. 1985

Fischer–Lynch–Paterson proves that deterministic consensus cannot guarantee termination in every fully asynchronous execution with even one possible crash. It does not forbid practical consensus under additional assumptions. This federation chooses not to require a total order over all harbor events: append-only local histories and cross-witnessed roots support later comparison. Contradictory signed roots become evidence when an auditor receives both; a bond prices misconduct only when detection and collection are credible.

A send interval and a delivery guarantee are different quantities. Sending every Δ seconds says nothing about how long a packet is delayed. Reliable-round bounds below therefore state the delivery model separately; partial-synchrony protocols make their additional timing assumptions explicit as well.

Dwork, Lynch & Stockmeyer. *Consensus in the Presence of Partial Synchrony*. J. ACM 35(2):288–323, 1988.

6.7.2 Settlement and the cross-chain-deals lineage

The Federated Harbor places this construction against the atomic-swap and adversarial-commerce baselines it is compared to below.

The escrow construction states its trust assumption outright: it does *not* claim trustless settlement. Its loss bound is conditional, and the condition is the whole content of the claim — which is why the claim is stated once, as Design Invariant 8.4.1 in *The Federated Harbor*, the chapter that proves it by case analysis over the custody ledger's two terminal transitions, and is not printed again here under a second number. What it says, in one breath: *if* the custody ledger is non-bypassable — a recipient whitelist, a fee cap, one atomic terminal transition, and no operator path around any of the three — then a counterparty's worst-case loss on a cross-harbor settlement is the pre-agreed fee φ and the outcome is exactly **Clear** or **Refuse**; if that boundary *can* be bypassed, the worst-case loss is the whole balance in custody. What this section adds is where that construction sits.

This is a deliberate departure from the hash-timelock baseline (**Herlihy** 2018, atomic cross-chain swaps), and the right comparison is the formal model of mutually-distrusting commerce without a shared chain: **Herlihy, Liskov & Shrira** (2022, *Cross-Chain Deals and Adversarial Commerce*). The harbor's bounded escrow is a "trusted third party / watchtower" point in that design space; we locate it there explicitly rather than reinvent it.

Herlihy. *Atomic Cross-Chain Swaps*. ACM PODC, 2018.

Herlihy, Liskov & Shrira. *Cross-Chain Deals and Adversarial Commerce*. The VLDB Journal 31:1291–1309, 2022.

Remark.

Open problem: trustless cross-harbor settlement. It is unknown here whether non-fungible, reputation-priced bonds admit a useful trustless settlement protocol under the paper's availability and privacy requirements. An HTLC comparison does not prove impossibility because the asset and adjudication models differ. (*open*.)

6.7.3 Revocation gossip and the equivocation race

The gossip mechanism itself is drawn and analysed in *The Federated Harbor*; what this section prices is the window it leaves open.

Federated revocation uses anti-entropy gossip (**Demers et al.** 1987) and a cuckoo filter: a compact probabilistic revocation index with no false negatives and tunable false positives. Expected dissemination takes $\Theta(\log m)$ rounds only on a connected

Demers et al. 1987

reliable-round overlay under the stated randomized peer-selection model. This is an asymptotic expectation, not an exact constant or a partition-tolerant deadline.

Protocol 6.7.1 (Federated revocation gossip).

State: each harbor X keeps a revocation filter F_X and a current epoch root $R_X^{(e)}$ published to a shared witness log.

1. *Revoke.* An operator revokes card c at its home harbor A: $F_A \leftarrow F_A \cup \{c\}$; A advances to $R_A^{(e+1)}$ and publishes it.
2. *Gossip.* Every Δ, each harbor picks a peer and exchanges filter deltas (anti-entropy). A card present in either filter becomes present in both.
3. *Audit.* Any third party compares each harbor's published roots in the witness log; a harbor that publishes two contradictory roots for one epoch is *equivocating*. Verification is constant work once both signed roots are co-located; delivery to an auditor has the overlay's separate dissemination delay.

Post, conditionally: in the connected reliable-round model, every honest harbor learns the revocation in expected $\Theta(\Delta \log m)$ time. Under an unbounded partition there is no finite global-learning bound.

Numbers by Hand 6.7.1 (The Revocation Race, Traced Hop by Hop).

Alice revokes a_2's card c at $t = 0$ (say she discovers it was misbehaving). Harbor B, which rented a_2, has not yet heard: for the interval $[0, \Delta]$, B's filter is stale and c is still spendable there.

At $t = \Delta$ gossip reaches B, which adds c to F_B and refuses further use. Harbor C, two hops away, learns at $t = 2\Delta$.

This is one illustrated execution; a different schedule or partition produces a different window.

Conjecture 6.7.1 says the bond Alice posted on c must dominate whatever a_2 could spend at B inside that window, or a well-capitalized Bob could rationally race the gossip.

Now you try: extend this same illustrated schedule one more hop — at what t does a harbor three hops from Alice learn the revocation, and what does a_2's card still let it spend at B in the meantime?

The threat is *adversarial*, not stochastic. An attacker who controls peer selection or partitions the gossip graph (an *eclipse attack*, **Heilman et al.** 2015) breaks the expected-time guarantee. The statement is conditional: a finite service-level bound needs eventual connectivity and a known post-stabilization delivery bound; epidemic analysis then supplies an expectation within that model, not an adversarial deadline.

Partitions defeat a deadline

Heilman et al. *Eclipse Attacks on Bitcoin's Peer-to-Peer Network*. USENIX Security, 2015.

Detection-after-the-fact deters only if the slashed bond exceeds the value extractable during the detection window. This is the analogue of the *cleanup lower bound* (the companion result that a cleanup bond must cover the worst-case cost of the damage it underwrites), and we state it as a target:

Conjecture 6.7.1 (Equivocation Lower Bound)**.**

For a deployment that advertises a finite freshness window T, the witness bond must dominate the maximum value spendable against a revoked-but-not-yet-observed capability over T. If partitions are unbounded, no finite bond covers the worst case; the system must instead fail closed on stale roots or cap spend independently. (open; analogue of the cleanup lower bound.)

The threat bands in *The Federated Harbor* set the window Conjecture 6.7.1 must price against the connectivity assumptions each band needs.

6.8 Reputation: monotone, but revocable

The proposed reputation-based rental and licensing terms rely on usable outcome histories. Two reconciliations matter here.

Recall

1. Without looking: name the three things federation requires in place of consensus.
2. State the Bounded Custody Loss property's condition and its two possible outcomes.
3. Why is "gossip every Δ" itself a partial-synchrony assumption rather than a proven guarantee?

Exercises 6.23–6.25, p. 542.

6.8.1 "Never ends" is not an anti-revocation property

A tempting design slogan holds that reputation "never ends" — no decay window an agent can outrun — while the same design also requires that a fraudulent settled outcome be retractable. A property and its required violation cannot both be requirements.

Reputation can be corrected

The reconciled statement: **the outcome record grows by appending witnessed events and corrections; a reputation score may rise or fall with those events, and a fraudulent outcome is invalidated by a propagating tombstone**. Dissemination has the same model and partition caveats as capability revocation. Retaining an honest outcome does not fix its weight in the current score; retention and revocation are separate from the estimator's horizon.

Liu & Skrzypacz 2014

Liu and Skrzypacz study a long-run player whose successive opponents observe a finite recent history. Under their stated equilibrium assumptions, reputation can build and be exploited repeatedly. This shows that the observation window affects incentives; it does not prove that infinite memory is generally inferior. The horizon here is a design parameter to choose against a stated prediction or incentive objective.

6.8.2 Why Sagas-style compensation does not restore reputation

It is tempting to fix revocation with compensating transactions (**Garcia-Molina & Salem** 1987, *Sagas*), as the bond ledger does. But the analogy fails structurally:

Garcia-Molina & Salem. *Sagas*. ACM SIGMOD, 1987.

Theorem 6.8.1 (Compensation Does Not Erase Reliance).

A ledger can append a compensating transaction that restores a numerical balance.

A reputation tombstone cannot undo decisions counterparties already made while relying on the invalid outcome. It can only change future evaluations after the tombstone is observed.

Reputation repair therefore requires dissemination plus an explicit remedy for past reliance; it is not equivalent to ledger compensation.

This is why Sagas appears in the bond-settlement prior art but *not* in the reputation prior art.

6.8.3 The grading-oracle keystone

Every folk-theorem incentive-compatibility claim in the market bottoms out in "the daemon derives the grade." But the grade is produced by a capturable evaluator.

Theorem 6.8.2 (The Monitoring Model Must Include the Evaluator).

An imperfect-public-monitoring equilibrium result requires a specified conditional distribution of public signals given modeled actions and strategies.

If a strategic grader chooses the signal but is omitted from the game, that distribution is not fixed by the model and the claimed equilibrium result is unsupported.

One must either constrain the signal mechanically or include the grader, its information, payoffs, and deviations as part of the game.

D. Abreu, D. Pearce & E. Stacchetti. "Toward a Theory of Discounted Repeated Games with Imperfect Monitoring." *Econometrica* 58(5):1041–1063, 1990. (With Green & Porter, *Econometrica* 1984.)

The two resolutions (both used). *Fix A — mechanically constrained evidence:* ground settlement in evidence the agent cannot directly author (a CI-issued test result, a protected merge event, or an independently evaluated policy check). This narrows the signal channel; it does not make the task metric universally strategy-proof.
Fix B — accountable contractual adjudication: for subjective criteria, parties name the judges, appeal procedure, terminal authority, and sanctionable misconduct in advance. Judges may post bonds, but disagreement alone is not proof of dishonesty. An appeal has a finite rank and can move only to a lower rank, terminating either at a protected objective predicate or at a designated final decision-maker. The latter supplies contractual finality, not an objective truth about taste. Judge information, collusion, monitoring, and payoff incentives remain part of the game. Order-swap and other bias controls for LLM judges address measured judging biases, not this logical gap.

Zheng et al. 2023

Numbers by Hand 6.8.3 (A Disputed Grade and a Finite Appeal).

Bob calls Alice's refactor "untasteful." Judy rules for Bob after posting a 50 cr judge bond. The contract permits one appeal to an independent panel, whose decision determines the settlement. The panel prefers Alice's work. That reversal alone shows no bond breach: two honest panels may disagree about the same diff and passing tests.

Suppose instead that an authenticated record proves Judy accepted Bob's bribe, breaching the declared conflict rule. The contract can slash Judy's bond on that evidence and compensate Alice up to the funded remedy. It cannot promise that 50 makes every possible loss whole. For a risk-neutral judge receiving an immediate bribe b that is retained if caught, detection probability p, no delay, and no other payoff change, this deviation has expected gain $b - 50p$. At $b = 10$ and $p = 0.1$ it remains profitable; at $p = 0.3$ it does not. Detection and collection must themselves be credible. The terminal panel's identical input artifacts do not make its preferences exogenous or machine-checkable.

Now you try: distinguish a reversible settlement decision, a proved conflict of interest, and an honest disagreement. Which one does each contract clause authorize the authority to remedy?

Recall

1. Without looking: what CAN a reputation tombstone do, and what can it NOT undo?
2. State Theorem 6.8.2 in one sentence: what does an imperfect-public-monitoring result require that an omitted strategic grader leaves undefined?
3. Name Fix A and Fix B, and distinguish an objective terminal predicate from contractual finality.

Exercises 6.26–6.28, p. 543.

6.9 Hosted trust is the product

The 2025–26 agentic-payments landscape — **AP2** (Google, signed payment mandates), **ACP** (OpenAI/Stripe), and **x402** (Coinbase, stablecoin settlement over HTTP) — has commoditized the *settlement rail*. Wiring an agent to pay another agent is now table stakes. This is *good news* for the thesis, because it sharpens what the product actually is.

What remains scarce, and therefore defensible, is **hosted trust**: the verified ledger (conservation-checked, Merkle-anchored evidence), the *relay* (the harbor mesh carrying public-key attestation

across machines), and the *reputation* that makes a counterparty you do not own legible and accountable.

- **Substrate (swappable):** the payment rail. Choose a payment rail and define an adapter that preserves the settlement obligations. Different rails require their own authorization and finality checks.
- **Product (the moat):** attestation, federation membership, and reputation hosting.

The proposed revenue model seeks to favor completed trade: a transaction fee (2–5% of settlements), a small listing fee (collusion deterrent), and a small bond-spread on *forfeited* bonds (kept small to limit its direct incentive toward over-slashing). The platform should earn most from *successful* trade, because successful trade is what hosted trust produces.

Numbers by Hand 6.9.1 (What the Moat Earns on One Trade, at the Stated Band).

Take the Success settlement of §6.2: Bob's 400 cr bounty clears in full to Alice.

At the stated transaction-fee band of 2–5%, the platform's cut on that one trade is $0.02 \times 400 = 8$ cr at the low end and $0.05 \times 400 = 20$ cr at the high end — earned because the trade *settled*, not because a bond was slashed.

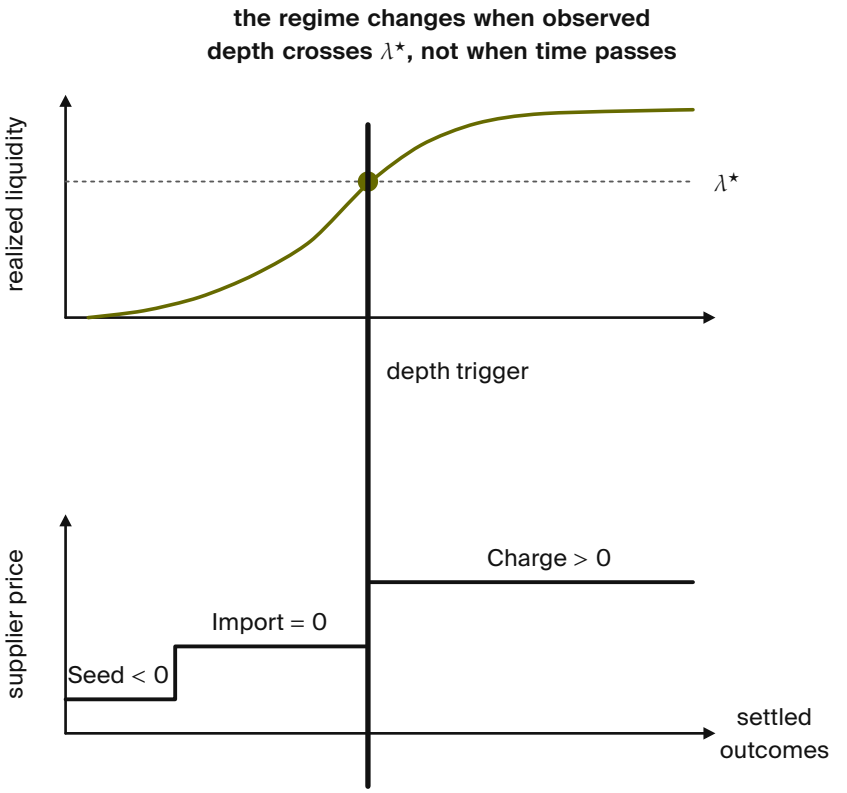
Compare the 60 cr bond-spread the same section's **Partial** settlement forfeits to the commons: under a small bond-spread, most of that 60 cr still goes to the commons/salvage fund, not the platform, which limits, but does not eliminate, the incentive toward over-slashing. Independent adjudication is still needed.

As §6.0.1 put it, before any of the machinery was on the page:

You don't sell crypto — crypto is the substrate. Here is what *is* scarce: attestation + federation membership + reputation.

6.10 Cold start, and the auction question

The design faces related participation problems: operators need available workers, owners need renters, and licensors need buyers. Platform theory directs attention to how a subsidy changes participation across groups; it does not select the subsidy for this unmeasured market. The proposed sequence is as follows. Phase 1 seeds supply at zero/negative price with platform-posted tasks at above-market bounties (generating the first settled outcomes — the bootstrap reputation data); Phase 2 imports external work history for partial credit to break the new-agent freeze; Phase 3 flips to charging the demand side once a liquidity threshold (not a calendar date) is met. Figure 6.8 draws the schedule against the quantity that actually gates it, because "threshold, not date" is the kind of phrase that stays a slogan unless you can see what it is a threshold *on*.



if liquidity never crosses the threshold, the subsidized state remains active and recoverable

Figure 6.8. Supplier pricing follows realized liquidity, not calendar time. Charging begins only above $\lambda^\star$; a market that never crosses the threshold never enters that regime. [internal]

6.10.1 Static escrow vs. competitive underwriting



Economist Thomas Youle contributed the competitive-insurance idea worked through in this section: let insurers bid to underwrite a transaction instead of fixing one bond size for everyone.

Thomas Youle's competitive-insurance proposal — insurer-agents bid to underwrite transactions, replacing a static bond parameter with a market-discovered price — is an additional class of contracts. But it may collide with the cleanup lower bound:

Proposition 6.10.1 (Underwriting reframes the bond as reserve adequacy)**.**

The cleanup lower bound requires bond $\geq$ *worst-case cleanup cost* c*.*

A competitive insurance market prices to expected *loss, which sits below the worst case for any below-tail agent.*

Either underwriting violates the cleanup lower bound (the commons can leak in the tail), or the bound must be reframed as the insurer's *capital-adequacy constraint (a reserve-requirement framing with tail reinsurance), not a per-transaction bond. The* form *of the answer is determined; only the reserve formula is open.*

This is a ***Theorem*** *in the honesty ledger's sense — it follows from the cleanup lower bound's own definition plus the definitional fact that competitive pricing tracks expected, not worst-case, loss — about a side that is itself only proposed: (specified; underwriting is not built, and the reserve formula the dichotomy leaves open is unspecified).*

6.10.2 Cartels and wash-trading

The per-round decision node and the closed-form sustainability inequality are drawn in *The Bonded Commons*, which develops the repeated game.

Multi-homing plus portable reputation enables cross-harbor *cartels* — a rating-agency cartel is the closest real-world analogue: several independent principals colluding to hold a shared reputation signal artificially favorable rather than competing on it. And a colluding requester+worker can *wash-trade* fake settled outcomes to mint reputation cheaply. The defense form — cost-per-settled-outcome must exceed reputation's marginal value — is circular as stated, because that marginal value is endogenous to the same

market. The fix is a structural break: make settled outcomes between a counterparty *pair* exhibit sharply diminishing reputation returns with an explicit cap, plus sampled adversarial re-audit of high-velocity pairs. Concavity alone does not impose a cap: $\sqrt{n}$ is concave and unbounded. For example, $R_{\text{pair}}(n) = R_{\text{max}}(1 - e^{-n/\tau})$ with $\tau > 0$ caps credit from one pair at R_{max}. This caps that pair's score contribution, not its economic value or the gain from forging new pairs; principal binding and a payoff model are still required.

Whether a cartel of colluding operators can actually hold its rating floor is a *repeated* game, and that is what the cartel game in *The Bonded Commons*draws. Each round a member either *colludes* (holds the agreed floor q_{floor}, collecting the cartel payoff π_C) or *defects* (undercuts the floor by ε for a one-time gain π_D, after which the cartel unravels and everyone drops to the competitive payoff $\pi_N \approx 0$). A **grim-trigger** strategy is the crudest way to punish that: *cooperate until the first observed defection, then defect forever*. It sustains the cartel only while members weight the future heavily enough — that weight is the **discount factor** $\delta \in (0, 1)$, the value a member places on next round's payoff relative to this one's, so a member with δ near 1 is patient and one with δ near 0 will take π_D today and let the cartel die. Sustaining collusion means the discounted stream of colluding payoffs beats the one-shot deviation, which is exactly the inequality the figure's box states, and δ clearing its threshold is exactly what the defenses above are trying to prevent.

Recall

1. Which side does Phase 1 subsidize? Why?
2. In Proposition 6.10.1, what are the two alternatives when insurance prices expected loss?
3. What structural change caps wash-trading returns regardless of price?

Exercises 6.29–6.31, p. 543.

6.11 A gluing analogy for the open federation problem

Three open problems—cross-harbor settlement, equivocation under partitions, and cross-currency valuation—share a local-to-global shape. That resemblance motivates an analogy; it does not yet define a sheaf or a cohomology theory.

Conjecture 6.11.1 (Formalization target, not established result)**.**

A future treatment may define a site of overlapping administrative domains and a presheaf of signed ledger views, then ask when compatible local sections admit a unique global section. This chapter

has not specified the site, restriction maps, coefficient object, or cocycles; therefore it makes no claim that a particular H^1 is defined or non-zero. The immediate engineering obligations remain the concrete ones: matched custody transitions, freshness policy under partitions, and time-indexed valuation.

Fong and Spivak's compositionality program suggests vocabulary for a future formalization. Until the site and coefficient data are defined, the chapter uses "gluing" only to organize engineering questions.

Fong & Spivak. *Seven Sketches in Compositionality: An Invitation to Applied Category Theory*. Cambridge UP, 2019.

6.12 Gaps the design must still close

Naming what is unspecified is part of the specification. These are not threats already defended; they are holes.

Numbers by Hand 6.12.1 (Watching the Built Floor Break, by Hand).

This is not a resolved mechanism — it is the gap below, traced against a number already on the page so "orphaned escrow" is not just a phrase. Take the three-sided settlement of §6.3.1: Bob's 400 cr bounty, 50 cr lease, and 10 cr licence reserve, plus Alice's 100 cr bond, lock 560 cr into one escrow row.

Now suppose Alice's harbor — the daemon that alone can sign her side of the settlement — goes dark mid-job, before the oracle closes.

Design Invariant 6.2.1 still holds arithmetically (wallet + escrow + commons has not changed, since nothing moved), but the 560 cr row now has no principal left to countersign a settlement, a refund, or a slash: it is correct and stuck.

That stuck state, not a wrong sum, is exactly what **Operator exit / harbor death** below names as unspecified. Table 6.3 therefore classifies it as a settlement-liveness gap: conserved funds still need a valid path to release.

1. **Multi-dimensional reputation aggregation.** "Accuracy/aesthetics/ efficiency judged by separate experts" has no specified combination rule. You cannot have "separate

Recall

1. Name three missing definitions in Conjecture 6.11.1.
2. Why do conflicting signed roots show equivocation, but not a cohomology class?
3. Which engineering obligations remain without the formalization?

Exercises 6.32–6.34, p. 543.

axes" and "one buyer-readable score" without specifying an aggregation function and its trade-offs. (*proposed*.)

2. **Float-plan amendment under scope drift.** The four states assume the acceptance criteria were right. Mid-flight re-specification needs a re-sign protocol with escrow top-up/refund delta, preserving conservation.
3. **Cross-boundary key rotation.** How a rotated signing key invalidates or re-validates cross-harbor delegations already held by B is unspecified.
4. **Skill versioning.** Reputation attaches to a content hash; v2 starts cold. The lemons problem reappears at every version bump.
5. **Operator exit / harbor death.** Escrow orphaning, in-flight float plans, and reputation tombstoning on ungraceful exit are unspecified — the market-level analogue of the kernel's crash-recovery problem.
6. **Incentive-compatible arbitration capacity.** The human oracle is scarce and expensive; at scale arbitration is itself a priced, bondable, slashable service that must converge to honest rulings, not rubber-stamping.

These six are not equally dangerous, and an implementer triaging them should not have to infer the stakes from the prose. Each one threatens a specific critical promise made earlier in this chapter:

6.13 One further market: adversarial testing and purchased assurance

S. Goldwasser, Y. Kalai & G. Rothblum. "Delegating Computation: Interactive Proofs for Muggles." *STOC*, pp. 613–622, 2008.

G. Irving, P. Christiano & D. Amodei. "AI Safety via Debate." arXiv:1805.00899, 2018.

Beyond labor, capital, and IP licensing, the harbor supports one further internal market, and it is worth stating because it is the one place where *assurance itself* becomes a priced line item rather than a hope. In an **adversarial test market**, independent test-writing agents compete to find flaws in a proposed diff *before* it settles. The frame is an interactive proof system: delegated computation (a prover convinces a resource-bounded verifier without the verifier redoing the work) crossed with multi-agent debate (two adversarial reasoners are more legible to a judge than one cooperative one).

Gap	Threatens	How it bites
Operator exit / harbor death	settlement liveness	orphaned escrow can preserve the supply identity while remaining inaccessible: conservation alone supplies no path to release
Float-plan amendment under scope drift	conservation	a mid-flight re-spec moves value with no matching debit unless the top-up/refund delta is escrowed
Cross-boundary key rotation	cross-boundary safety	a delegation already held by B may verify against a key A has retired
Skill versioning	Sybil-resistance	a v2 content hash is a fresh identity, so the lemons problem and the whitewash both reappear at every version bump
Multi-dimensional reputation aggregation	incentive compatibility	an unspecified aggregation rule is an unspecified target, and any fixed scalar collapse re-creates a Goodhart objective
Incentive-compatible arbitration capacity	incentive compatibility	Fix B (§6.8) prices one judge; it does not price a judge pool under load, where rubber-stamping is the cheap strategy

Table 6.3. Which promise each gap threatens. An unmatched amendment can break conservation; orphaned escrow can preserve conservation while preventing settlement. The table separates those safety and liveness obligations.

Numbers by Hand 6.13.1 (The Adversarial Test Market, Dramatized).

Bob's float plan ships a diff that one of Alice's specialists wrote. Rather than trust Alice's own test run — a signal whose author has every reason to flatter it — Bob posts a 30 cr bounty per confirmed flaw and opens the diff to $k = 3$ independent test-writing agents, each paid only for a defect it actually surfaces.

If each detects and truthfully reports a given real defect independently with probability $d = 0.6$, the chance a planted flaw survives all three is

$$(1 - 0.6)^3 = 0.064.$$

Conditional on this fixed flaw and reporting model, its chance of receiving no report is about 6%, at a planned bounty budget of $3 \times 30 = 90$ cr, which is a number he can hold up against what the bug would have cost him in production.

Raise k to 5 and the survival probability falls to

$$(0.4)^5 = 0.01024;$$

that is the whole shape of the mechanism — assurance is bought in units of k, and each unit costs one bounty plus the carry on one bond.

Now you try: hold $d = 0.6$ fixed and find the smallest k that pushes the survival probability below 1%.

A **reporting panel** is a set of k test-writing agents. For one fixed flaw, let R_i be the event that reviewer i both detects it and truthfully reports it before settlement. The assurance calculation assumes these events are mutually independent and $\Pr(R_i) \geq d > 0$. These are behavioral and measurement hypotheses, not consequences of posting a bond. A separate audit policy can make truthful reporting a best response under specified payoffs; at indifference, honesty still needs a selection assumption.

Figure 6.9 reserves one bounty b and one bond-carry allowance c_B per reviewer for a fixed flaw. Its $k(b + c_B)$ line is a planned budget, not realized payout: payment depends on which findings qualify and how duplicate reports settle.

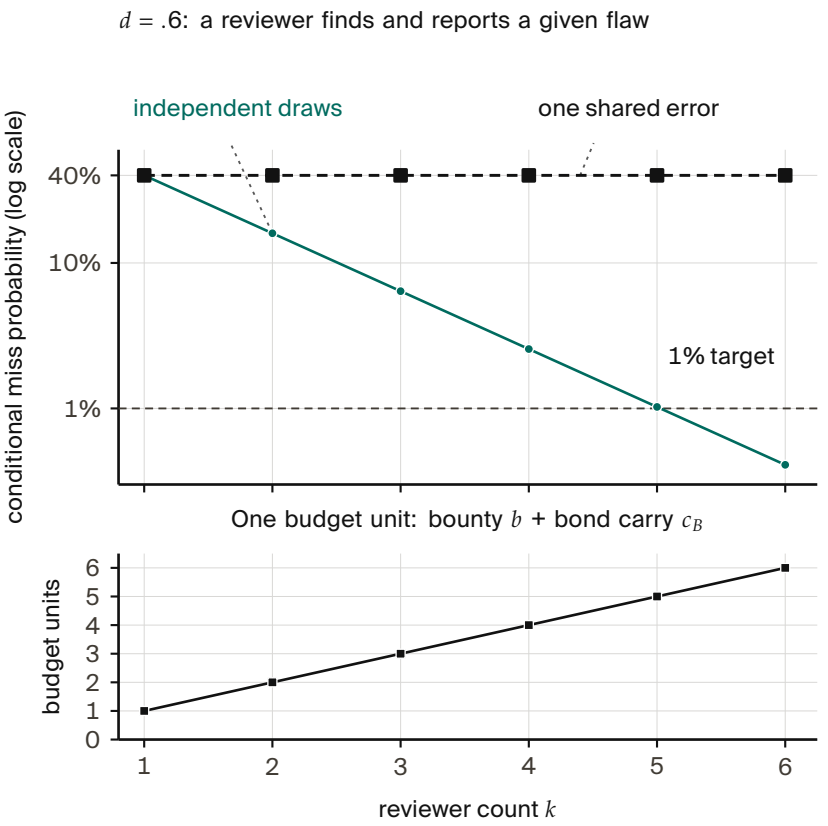


Figure 6.9. In this analytic model, more independent reviewers reduce miss probability; perfectly shared outcomes do not. The lower panel is reserved budget, not realized payout.

Theorem 6.13.2 (Purchased Assurance Bound).

For a fixed flaw in an implementation, let $R_1, \ldots, R_k$ be mutually independent detection-and-truthful-report events, each with probability at least d. Then

$$\Pr(\text{no report of this flaw}) = \prod_{i=1}^{k}(1 - \Pr(R_i)) \;\le\; (1-d)^k. \qquad (6.1)$$

The product identity follows from mutual independence. Equality holds when all report probabilities equal d. Thus $1-(1-d)^k$ is a conditional lower bound on reporting this particular flaw. It is neither the probability that an arbitrary implementation is correct nor a guarantee of successful repair. The planned reviewer budget $k(b+c_B)$ pays for participation; the probability bound applies only if the reporting and independence hypotheses hold.

Theorem 6.13.3 (Becker Deterrence for the Panel).

For a risk-neutral test-writer with exactly two available actions, honest reporting (normalized incremental payoff zero) and one misreport yielding G, assume the latter is detected and fully sanctioned with probability ρd_{a}, immediately, with no false sanctions or other payoff effects. Here d_{a} is the conditional probability that an audit detects the misreport; it is distinct from the reporting probability d above. Honest reporting is a weak best response to this fixed audit policy iff expected punishment is at least the deviation gain — the **Becker deterrence condition**:

$$\rho\, d_{\mathsf{a}}\, B \;\geq\; G, \tag{6.2}$$

equivalently, the platform must commit to a re-audit rate at or above the critical rate $\rho^{\star} = G/(d_{\mathsf{a}}B)$. (specified; this half is imported, not proved here — see the paragraph below.)

G. Becker. "Crime and Punishment: An Economic Approach." *Journal of Political Economy* 76(2):169–217, 1968.

The deterrence condition in Theorem 6.13.3 is not proved here; it is imported. The companion technical paper *Reputation is Amortized Verification* derives $\rho^{\star} = G/(d_{\mathsf{a}}B)$ as the stage-game threshold of a bonded inspection game — the same game this chapter plays in §6.8 when it bonds and slashes a judge — and works it at the program's running parameters: $G = 10$, $d_{\mathsf{a}} = 0.8$, $B = 50$ give $\rho^{\star} = 10/(0.8 \cdot 50) = 0.25$. Audit one report in four and the cheat payoff is exactly zero. Double the bond to $B = 100$ and $\rho^{\star}$ halves to 0.125: capital and vigilance are substitutes along the frontier $\rho d_{\mathsf{a}} B = G$.

Reputation is Amortized Verification: Inspection Games for Agent Economies. Port Daddy Harbor research program, companion paper 3, 2026.

In the notation of the audit tower of *From Spawn to Person*, this is the single-level case. There the bribe floor is written $\beta = \rho d_{\mathsf{a}} B$, the gain at level k is G_k, and level $k{+}1$ audits level k across C sealed cliques, so the condition here reads $\beta \geq G_0$ and the contraction $G_{k+1} = (1 - \rho d_{\mathsf{a}})\, G_k$ is what makes a finite stack of audited auditors converge. The letter k is overloaded between the two chapters: here it counts the panel, there it indexes the level.

The settlement rule changes deterrence

Quote the threshold without naming your settlement convention and you will be reproducibly wrong by about 17%. Equation (6.2) is the *keep-gain* convention: a caught cheat forfeits B but keeps G. If the escrow also claws the gain back, the forfeiture is $G + B$ and the critical rate drops to $\rho_c^{\star} = G/\big(d_{\mathsf{a}}(G + B)\big) = 0.2083$ at the same

parameters. A platform whose rail does claw back but which budgets audits at $G/(d_{\mathrm{a}}B)$ is over-auditing; one that does not claw back but budgets at $G/(d_{\mathrm{a}}(G{+}B))$ is under-deterring. The convention is a property of the slashing contract, not a modeling taste.

Port Daddy Harbor 2026

Figure 6.10 puts the two critical rates side by side at the program's own parameters.

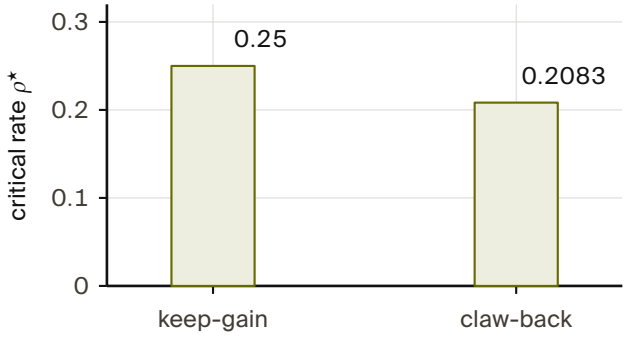


Figure 6.10. Same $G = 10$, $d_{\mathrm{a}} = 0.8$, $B = 50$; different forfeiture. Keep-gain takes B, requiring audit rate 0.25. Claw-back takes $G + B$, requiring 0.2083. [analytic example]

Shared errors defeat extra reviewers

The $(1-d)^k$ bound is exactly as good as the word *independent* in its hypothesis, and independence is the assumption a real test market breaks first. Three test-writers that read each other's public findings, or that are three instances of the same model family with the same blind spot, do not give k independent draws — they give something closer to one draw and two echoes. Purchased assurance therefore has to buy *diversity* (distinct principals, distinct model families, sealed submissions) and not merely count k; otherwise the priced assurance level is an overstatement of the delivered one. The same requirement appears in the companion inspection-game analysis as the sealed multi-clique sampling hypothesis, where it is likewise required rather than hygienic.

6.13.1 Putting a parameter on the echo

"One draw and two echoes" is the right intuition and it is not yet a number, and an operator buying assurance by the reviewer needs the number, because the whole point of Theorem 6.13.2 is that assurance has a price. Give the echo a model. Let the panel's per-reviewer detection-and-report rate be a *random* quantity D with mean d, drawn once per panel and shared by its members — that is what a common blind spot is, a draw the panel makes together

rather than one each. Assume truthful flaw reports are independent Bernoulli trials *conditional on* D. Then the miss probability is $\mathbb{E}\big[(1-D)^k\big]$, and the first thing to notice is a one-line inequality with an uncomfortable direction:

$$\Pr(\text{defect undetected}) \;=\; \mathbb{E}\big[(1-D)^k\big] \;\geq\; (1-d)^k, \qquad (6.3)$$

by Jensen, since $x \mapsto (1-x)^k$ is convex. The independent bound is not conservative — it is *optimistic*, always, for every mixing distribution with the same mean in this common-rate mixture model. This is not a statement about every possible dependence structure.

To get a closed form take $D \sim \text{Beta}(a, b)$, whose intra-panel correlation is $\theta = 1/(a+b+1)$ and whose mean is $d = a/(a+b)$, so a target (d, θ), with $0 < d, \theta < 1$, pins $a = d(1/\theta - 1)$ and $b = (1-d)(1/\theta - 1)$. The miss probability telescopes into a product a reader can evaluate on paper — it is a ratio of Beta functions, but it is easier to use and easier to check in this form, and it avoids colliding with the bond B:

$$\Pr(\text{defect undetected}) \;=\; \mathbb{E}\big[(1-D)^k\big] \;=\; \prod_{j=0}^{k-1} \frac{b+j}{a+b+j}. \qquad (6.4)$$

Numbers by Hand 6.13.4 (What Correlation Costs a Three-Reviewer Panel).

For a separate panel illustration, take $d = 0.8$ and a panel of $k = 3$ reviewers. Under independence the bound is

$$(1-0.8)^3 = 0.008 :$$

eight misses in a thousand.

Now set $\theta = \frac{1}{2}$, so $1/\theta - 1 = 1$ and $(a, b) = (0.8,\ 0.2)$. Equation 6.4 is three fractions:

$$\tfrac{0.2}{1.0} \times \tfrac{1.2}{2.0} \times \tfrac{2.2}{3.0}$$
$$= 0.2 \times 0.6 \times 0.7\overline{3} = 0.088$$

[verified, closed form] — eleven times the independent number, from a panel the ledger still counts as three.

Push to the monoculture limit $\theta \to 1$ and the product tends to $1-d = 0.2$, which is precisely "one draw and two echoes" written as arithmetic: three reviewers deliver exactly the assurance of one, a factor of 25 worse than the line item says.

Now you try: at $k = 5$ and $\theta = \frac{1}{2}$, does adding two more correlated reviewers buy more or less than the third one did?

(Two more fractions, $\frac{3.2}{4.0} \times \frac{4.2}{5.0}$, multiply the miss probability by $0.8 \times 0.84 = 0.672$, giving 0.059 — against $0.2^2 = 0.04$ for the independent case's next two, so much less: correlated panels have sharply diminishing returns, which is the whole argument for buying diversity instead of headcount.)

The algebra is conditional on the model; its fit to deployed panels is untested. Calibrating detection and joint misses requires labeled defects or held-out known outcomes. Disagreement alone does not distinguish independent success from shared error. Here θ is a sensitivity parameter, not a measured constant, and differs from the audit rate ρ in Eq. 6.2. Separate principals and sealed submissions are precautions against shared errors, not proof of independence.

Pairwise correlation alone cannot price the all-miss event. Let three detection bits be uniform on $\{000, 011, 101, 110\}$, or instead on $\{111, 100, 010, 001\}$. Both panels have $d = 1/2$ and zero pairwise correlation; their all-miss probabilities are $1/4$ and 0, respectively, rather than the independent value $1/8$. A variance-based effective panel size therefore cannot be substituted into $(1 - d)^k$ without a joint model. The Beta mixture supplies such a model, and Eq. 6.3 holds for its broader conditionally independent common-rate class, not arbitrary correlated panels.

The ground-truth oracle for grading the test-writers themselves is **mutation-testing kill rates** against held-out syntactic mutation operators, which reduces direct adaptation to the visible grading suite. Held-out mutants do not by themselves establish generalization to real defects, independence of reviewers, or immunity to repeated adaptive attempts. This is a Fix A oracle in the sense of §6.8: machine-checkable, and not authored by the party being graded.

(*proposed*. The mutation-testing oracle and the bounty-per-killed-mutant contract are specified here for the first time in this series; neither ships, and neither appears in the reference implementation's ledger module. The deterrence inequality it leans on is specified, proved in the inspection-game analysis. Recorded as such in Table 6.5.)

Same pairs, different risk

Panel A	Panel B
000	111
011	100
101	010
110	001
1/4	0

Each column lists four equally likely outcomes; 1 means detection. Both have zero pairwise correlation. Only A can miss everything.

Port Daddy Harbor 2026

Recall

1. Without looking: state the Purchased Assurance Bound in one line, and name the one word in its hypothesis a real test market breaks first.
2. What is the difference between the Becker deterrence condition's "keep-gain" and "claw-back" conventions, and why does it matter which one you quote?
3. What is the ground-truth oracle for grading the test-writers themselves, and why does it have to be Fix A rather than Fix B?

Review of the key ideas

What this chapter leaves coupled, in the order the rest of the book will lean on it:

1. **A market, not a payment rail.** The harbor has three markets: labor, agent rental, and skill licensing. They share an escrow abstraction; operators, buyers, and insurers are participant roles, not a competing classification of those markets.
2. **The succession rule has a price.** When $\eta K < 1$, the dimensionless ratio $D^* = \eta K/(1-\eta K)$ marks where the specialist's residual-downtime floor reaches the pool's cost line. At or above it the pool beats every finite skill premium; below it, the full response-time comparison still matters.
3. **The float plan is the built floor.** The required admission invariant permits a worker to become *running* only at the committed transition where the wallet is debited and the escrow row exists; a signed plan alone admits nothing, and abort leaves all three traces unchanged.
4. **Conservation is an invariant, not a hope.** In native units $\Delta W + \Delta E + \Delta C = 0$ exactly, while the no-unfunded-start design additionally requires the atomic admission gate; once amounts are converted, fee, slippage and risk are priced terms and the total is no longer zero by construction.
5. **Deontic conflict is decidable inside the fragment and NP-complete one step outside it.** The in-fragment check is polynomial and produces a witness; the reference checker demonstrates that computation; deployment is a separate obligation.
6. **The keystone is missing and named.** Cross-operator attestation is what the market rests on and does not yet have; the local identity boundary remains partial.
7. **Single-unit conservation composes**; bilateral trade across a boundary is conditional, and the condition is stated.
8. **Evidence accumulates; scores can change**: compensation does not erase reliance, and the monitoring model must include the evaluator.
9. **Hosted trust is the product**: the cold-start pricing proposal identifies a supplier participation threshold λ^*, conditional on its

payoff model; the cartel game says which regime detection and loss put an insurer in.

10. **Assurance depends on joint detection.** Independent reviewers miss a fixed flaw with probability $(1-d)^k$. Shared blind spots require a joint model; an audit bond deters dishonesty only under its detection and collection assumptions.

6.14 Exercises

§6.0.1 The thesis: a market, not a payment rail

6.1 Check ⋆ *Solution p. 757*

Explain why costless cloning defeats a sanction attached only to a disposable agent identity. Does that rule out prepaid sale of a content-addressed skill?

6.2 Check ⋆ *Solution p. 757*

Side 1 (operator-for-hire) does *not* require durable identity in the same way sides 2 and 3 do. Why? (Hint: who is the counterparty, and who bears the consequence?)

6.3 Open ⋆⋆⋆ *Solution p. 757*

Construct a rental or skill-licensing contract that works without a persistent agent identity. Then identify a reputation-weighted right or continuing liability that costless identity resets would defeat (§6.4).

§6.1 What a “side” is, and why there are three

6.4 Check ⋆ *Solution p. 758*

State the operational test for “how many sides” in one sentence, then apply it to a ride-share platform: how many sides, and why?

6.5 Trace ⋆⋆ *Solution p. 758*

Side 3 (skill licensing) keys reputation on a *content hash*, not a person. What breaks when a licensor ships v2 of a skill? (Hint: §6.12, skill versioning.)

6.6 Open ⋆⋆⋆ *Solution p. 758*

A buyer reads a reputation vector with three axes: accuracy, aesthetics, and efficiency. Specify a disclosure rule that lets the buyer

weight the axes *without* re-introducing a single Goodhart target. Argue why every fixed scalar collapse re-creates the bandit framing the design rejects.

§6.2 The float plan: the built floor

Solution p. 758

6.7 Trace ★★

Trace the flow of funds for a **Partial** settlement where the worker completed 40% of the evidence chain. Confirm Design Invariant 6.2.1 holds across the split (the dramatization above is one such trace; redo it for a bond of 250 cr and 70% completion).

Solution p. 759

6.8 Check ★

Why does closure bind to an oracle rather than a self-reported note? Name the attack a free-text "Result:" note enables.

Solution p. 759

6.9 Open ★★★

The four states assume the acceptance criteria were *right*. Design a *float-plan amendment* protocol for the common case where the criteria themselves turn out wrong mid-flight (re-sign by both parties, escrow top-up/refund delta), preserving Design Invariant 6.2.1. (Gap, §6.12.)

§6.3 Three sides on one escrow

Solution p. 759

6.10 Check ★

Who operates the proposed runtime jail, and who owns the rented asset's reputation? What extra model would be needed to call this allocation efficient?

Solution p. 759

6.11 Trace ★★

Show that adding a lease line item and a metered-license line item to a float plan cannot violate Design Invariant 6.2.1, no matter the amounts.

Solution p. 760

6.12 Open ★★★

A skill's portfolio proof for v1 says nothing about v2. Specify a *version-binding* for the Merkle portfolio proof so a new version starts with a discounted inherited prior — the newcomer policy, but for code.

6.13 Trace ★★ *Solution p. 760*

The deontic layer named above — what an agent *may* or *must* do, as against the jail's *capability* — needs a decision procedure once two accepted norms might disagree. Take the conflict witness worked by hand above (fact `agent_deployed`; Horn rule to `prod_access`; an obligation and a forbiddance on `write_prod` overlapping on $[5, 10]$), and add one *disjunctive* obligation $O(\ell_1 \vee \ell_2)$ over two fresh literals with the added rules $\ell_1 \to O(\texttt{assert_x0}, \{\texttt{x0}\}, [0, 1])$ and $\ell_2 \to F(\texttt{assert_x0}, \{\texttt{x0}\}, [0, 1])$. These two rules concern a different action from `write_prod`. Try the three nonempty discharge selections (ℓ_1 alone, ℓ_2 alone, both) by hand. Does any selection make the `write_prod` conflict go away? What happens on the selection that asserts both ℓ_1 and ℓ_2, and why does Theorem 6.3.3's reduction from 3-SAT depend on exactly that happening?

§6.4 The keystone the market rests on — and does not yet have

6.14 Check ★ *Solution p. 761*

An aggregate gives each pseudonym equal voting weight. One principal can create aliases without cost or admission checks. Give an attack on that aggregate. Does the attack also make a signed record of one completed transaction useless?

6.15 Check ★ *Solution p. 761*

Local non-forgeable identity has a non-goal: "no defense against a hostile operator." Name the exact word in the market's definition ("mutually-distrusting," "boundary neither controls") that this non-goal contradicts.

6.16 Open ★★★ *Solution p. 762*

Specify a cryptographic binding tying N agents to one principal such that spawning fresh agents does not reset the bond floor, and that survives into the cross-operator world (where the single-operator "trusted credential" shortcut is unavailable). This is the hard half of cross-operator attestation.

§6.5 Conservation under composition

6.17 Check ★ *Solution p. 762*

State Myerson–Satterthwaite in one sentence and list the assumptions that must be checked before applying it to one harbor trade.

Solution p. 762

6.18 Trace ⋆⋆

Give a two-currency example in which native-unit conservation holds but the marked-to-market scalar changes because v_t moves.

Solution p. 763

6.19 Open ⋆⋆⋆

Specify an oracle-latency and price-move model that yields a valid φ exposure bound for Property 6.5.1.

§6.6 The Anchor Protocol proper: cross-harbor capability transfer

Solution p. 763

6.20 Check ⋆

Why is the ceremony four messages and not two? Identify what message (3) buys (Hint: who must counter-sign, and under whose key does C'_B later verify?).

Solution p. 763

6.21 Trace ⋆⋆

Trace what happens to an in-flight C'_B when Alice rotates her signing key between messages (3) and a later presentation at B. (Gap: cross-boundary key rotation, §6.12.)

Solution p. 763

6.22 Open ⋆⋆⋆

Specify replay protection across the boundary: is the *anchor id* a nonce? Does B reject a re-presented message (3)? State the freshness invariant.

§6.7 Federation: trading across machines you don't own

Solution p. 764

6.23 Check ⋆

State the connectivity, peer-selection, and delivery assumptions needed for expected $\Theta(\Delta \log m)$ dissemination. Explain why none yields a deadline during an unbounded partition.

Solution p. 764

6.24 Trace ⋆⋆

Trace a revoked capability through the revocation-gossip figure in *The Federated Harbor*: at which harbor, and at what time, can it still be spent? That window is what Conjecture 6.7.1 must price.

Solution p. 764

6.25 Open ⋆⋆⋆

State and (attempt to) prove the Equivocation Lower Bound for a three-harbor mesh under a fixed gossip period Δ and a single equivocating witness.

§6.8 Reputation: monotone, but revocable

6.26 Check ⋆ *Solution p. 765*

State the signal-model assumption Theorem 6.8.2 needs and explain why an omitted strategic grader leaves it undefined.

6.27 Check ⋆ *Solution p. 765*

Classify three settlement criteria as machine-checkable (Fix A) or judge- dependent (Fix B): a unit test passing; a merged SHA; "the refactor is tasteful."

6.28 Open ⋆⋆⋆ *Solution p. 765*

Design a terminating appeal procedure using a decreasing natural-number rank. State separately what supplies terminal authority for an objective predicate and for a subjective preference. Explain why termination does not establish accuracy.

§6.10 Cold start, and the auction question

6.29 Check ⋆ *Solution p. 765*

Which side should Phase 1 subsidize, and why? Apply the "scarcest cross-side externality" rule.

6.30 Trace ⋆⋆ *Solution p. 766*

Use the cartel game in *The Bonded Commons*to identify the grim-trigger condition under which the cartel is self-sustaining. What raises the critical discount factor δ enough to break it?

6.31 Open ⋆⋆⋆ *Solution p. 766*

"Price of anarchy when reputation is for sale" is the wrong tool: once reputation is tradeable, you have changed the game, so PoA (a ratio for a *fixed* game) does not apply. Reframe it as a *signal-existence* question (Spence-signaling with a resale market): does *any* informative separating equilibrium survive reputation resale?

§6.11 A gluing analogy for the open federation problem

6.32 Check ⋆ *Solution p. 767*

Name the site, cover, restriction maps, and coefficient object a genuine sheaf model of federated ledger views would require.

Solution p. 767

6.33 Check ⋆

Explain why two incompatible signed roots are evidence of equivocation but are not, without further definitions, a cohomology class.

Solution p. 767

6.34 Open ⋆⋆⋆

Build a small formal gluing model for a bounded gossip topology and state exactly what consistency result it proves.

§6.13 Purchased assurance

Solution p. 768

6.35 Trace ⋆⋆

For a fixed flaw, each reviewer independently detects and reports it with probability 0.6. How many reviewers make the miss probability at most one percent? Each costs two credits to recruit. Compute the planned recruitment budget, and explain why it excludes outcome-dependent flaw bounties. What fails if all reviewers use the same blind spot?

6.15 Related work

The design borrows from seven literatures, and its only originality is in how it *joins* them; this section consolidates the threads cited piecemeal above.

Rochet & Tirole 2003

Rochet & Tirole 2006

M. Armstrong. "Competition in Two-Sided Markets." *RAND Journal of Economics* 37(3):668–691, 2006.

Chiu et al. 2025

Multi-sided platform markets. The frame is Rochet & Tirole and Armstrong: a platform is multi-sided when the *allocation* of price across user groups, not just its level, moves volume. Our labor, rental, and licence contracts are three purchases, not a proof of three distinct platform sides. Chiu, Zhang & van der Schaar supply a simulated agent labor market with reputation as an information mechanism, rather than a separately priced participant side. These papers ask how cross-group participation creates externalities and how prices allocate them; they do not supply a trust oracle or bonded settlement. Here the three contract types concern labor, capital, and reusable skills; they are not a universal taxonomy of platform sides. The chapter uses the literature for incidence and participation, then labels its own settlement claims.

Myerson 1981

Mechanism design and its impossibilities. Settlement is a direct mechanism in the sense of Myerson; the binding constraint

is Myerson–Satterthwaite, which forces the market to surrender efficiency to keep budget balance, incentive compatibility, and individual rationality; the worked no-trade region in §6.5.1 instantiates the impossibility on the linear double auction of Chatterjee & Samuelson. The incentive-compatibility arguments are imperfect-monitoring folk theorems (Green–Porter; Abreu–Pearce–Stacchetti), whose exogenous public signal we cannot take for granted — hence the grading-oracle problem. Grossman–Hart motivates the control-right design question for the rental side, and Akerlof the lemons argument for licensing. This family supplies impossibility boundaries and design questions: what a direct mechanism can elicit, which desiderata must be traded away, and who controls an incompletely contracted asset. Its scope here is the bilateral and rental-side slices. The folk-theorem analogy cannot manufacture the public signal the harbor lacks, and the licensing analogy does not establish that reputation solves adverse selection.

Myerson & Satterthwaite 1983

Chatterjee & Samuelson 1983

Abreu et al. 1990

Grossman & Hart 1986

Akerlof 1970

Reputation systems. Empirically, online reputation has measurable value (Resnick et al.; Tadelis); theoretically, finite recent histories change equilibrium incentives in the repeated-interaction model of Liu and Skrzypacz. That model supplies a question about our weighting horizon, not a generally optimal horizon. The Sybil attack is the reason reputation must key on a principal, not a spawn. LLM-as-judge de-biasing follows Zheng et al.. The empirical work asks whether reputation changes exchange and selection; the theoretical work asks when history becomes an unstable asset. We borrow those questions for a finite horizon, principal-bound identity, and explicit judge policy. They do not prove that this score is calibrated, Sybil-resistant, or unbiased; Zheng et al. is a de-biasing reference, not an oracle certificate.

Resnick et al. 2006

Tadelis 2016

Liu & Skrzypacz 2014

Douceur 2002

Zheng et al. 2023

Distributed systems and cross-chain commerce. FLP rules out guaranteed termination of deterministic consensus in a fully asynchronous system with even one possible crash. It does not prohibit consensus as an engineering choice. Partial-synchrony results show why timing assumptions change that boundary. This federation instead chooses propagation by epidemic gossip and takes a Certificate-Transparency detect-don't-prevent posture, mindful of eclipse attacks. The settlement design is located explicitly against

Fischer et al. 1985

Dwork et al. 1988

Demers et al. 1987

Laurie. *Certificate Transparency*. Commun. ACM 57(10):40–46, 2014.

Laurie, Langley & Kasper. *Certificate Transparency*. IETF RFC 6962, 2013.

Heilman et al. 2015

Herlihy 2018

Herlihy et al. 2022

Garcia-Molina & Salem 1987

atomic cross-chain swaps and the adversarial-commerce model of Herlihy, Liskov & Shrira; reputation revocation is contrasted with Sagas-style compensation. This literature asks which liveness, convergence, and detection properties remain when agreement, synchrony, or honest connectivity is missing. That is the scope of the chapter's gossip and bounded-escrow analogies: they clarify failure posture and stale windows, but do not turn a partitioned federation into consensus or make a non-fungible bond atomic. Sagas contrasts with tombstones because the harbor removes an authorization rather than compensates.

Spivak & Kent. *Ologs: A Categorical Framework for Knowledge Representation*. PLoS ONE 7(1):e24274, 2012.

Fong & Spivak 2019

Ostrom. *Governing the Commons*. Cambridge UP, 1990.

Nisan et al. 2007

Applied category theory. The trace-composition notation and proposed gluing formalization draw vocabulary from ologs and Fong & Spivak; no cohomology claim is made. The commons-governance reading draws on Ostrom, and the auction machinery on standard algorithmic game theory. The category-theory sources ask when local descriptions and their overlaps determine a coherent global description. They supply vocabulary for trace and gluing, not a new cohomological theorem: the chapter's finite-map lemma remains a bookkeeping result. Ostrom supplies governance questions and algorithmic game theory auction tools; neither proves that this market is strategy-proof or governable in deployment.

Universal Commerce Protocol (Google, Shopify, et al.). *Specification and reference implementation, Apache-2.0*, 2026. Source; Source.

OpenAI & Stripe. *Agentic Commerce Protocol*, 2025. Source.

Agent Payments Protocol (AP2): *Mandates*, 2026. Source; Source.

Agentic-commerce protocols. While this design was being written, the industry shipped the transaction layer of an agent economy. The Universal Commerce Protocol (Google/Shopify, 2026) standardizes agent-mediated retail — discovery via a `/.well-known` capability profile, a checkout state machine, server-selects version negotiation — and its rival, the Agentic Commerce Protocol (OpenAI/Stripe), the platform-mediated equivalent. Both are deliberate about what they exclude: neither settles payments (delegated to payment service providers) and neither decides which agents deserve trust. Authorization proof is delegated to the Agent Payments Protocol, whose mandates — W3C Verifiable Credentials in SD-JWT form, chained principal → platform → merchant — are the deployed sibling of this chapter's float-plan countersign. These credential formats are proposed for the harbor boundary so that the keystone identity artifacts of §6.4 can use standard verification tools. This is a serialization proposal, not evidence of deployed interoperability.

The gap those protocols leave open is this chapter's subject: published security analyses of UCP note that it regulates *how* agents transact while offering nothing about *which* agents to trust — no collateral, no settlement oracle, no reputation, no priced deterrent. The harbor economy is a mechanism for exactly that layer; the retail protocols are evidence the layer below it is arriving on schedule, and hosted trust (§6.9) is what they will need bolted on. The relevant question is boundary placement: which transaction and authorization formats can be standardized below the harbor, and which trust decisions remain outside those rails. UCP, ACP, and AP2 are neighboring source material; the proposed Harbor Card envelope remains a serialization proposal, and standard checkout supplies no claimed collateral, reputation, or finality.

DataDome. *Agent Trust Management and the Universal Commerce Protocol*, 2026. Source.

Interactive proofs and adversarial debate. The adversarial test market (§6.13) is a seventh thread, cited piecemeal there rather than above because it grounds a single additional mechanism rather than the market's core structure. It is delegated computation in the sense of Goldwasser, Kalai & Rothblum — a resource-bounded verifier need not redo a prover's work — crossed with multi-agent debate, where two adversarial reasoners are more legible to a judge than one cooperative one. Becker's deterrence condition supplies the enforcement half: a test-writer's expected forfeiture must match its gain from misreporting. These sources address two questions: how a verifier purchases assurance without redoing all prover work, and how adversarial reports expose errors. The chapter adds a market and bond, not a cryptographic proof of the whole artifact. Detection probability (d, k) and honesty incentives (G, B, ρ) stay separate so a good panel is not mistaken for an honest one.

Goldwasser et al. 2008

Irving et al. 2018

Becker 1968

6.16 Conclusion

The harbor-master makes a swarm legible and accountable for one operator (the wedge of *The Legible Swarm*). When operators sail out to trade, the *same* ledger that kept one operator's swarm honest becomes the market that lets fleets who don't trust each other work together: three sides, one bond, hosted trust. We have not papered over the cost of that claim. The keystone identity primitive — local non-forgeable identity — covers a single-operator threat model and must be extended to cross-operator attestation before

Table 6.4. How the harbor economy sits relative to the nearest prior art on each axis.

Axis	Nearest prior art	What the harbor does differently
Market structure Rochet & Tirole 2003	Two-sided platforms	Three incentive constraints (labor / capital / IP) on one escrow object
Settlement across distrust Herlihy 2018 Herlihy et al. 2022	Atomic swaps; cross-chain deals	Bounded (not trustless) escrow for *non-fungible, reputation-priced* bonds
Revocation Laurie 2014	PKI revocation lists; Certificate Transparency	Gossip-converged filters with a priced, named attacker window
Reputation revocation Garcia-Molina & Salem 1987	Sagas compensation	Tombstone propagation — no conserved quantity to compensate
Grading / IC signal Abreu et al. 1990	Folk theorems	Strategy-proof machine-checkable oracle *or* contractual adjudication with modeled judge incentives
Agent transaction rails Universal Commerce Protocol 2026 OpenAI & Stripe 2025 AP2 2026	UCP; ACP; AP2 mandates	They standardize the transaction and name trust out of scope; the harbor prices and enforces the trust layer they delegate away
Purchased assurance Goldwasser et al. 2008 Irving et al. 2018	Delegated computation; AI-safety debate	Prices assurance as k independently bonded test-writers under a Becker-style re-audit rate, against a mutation-testing oracle

the boundary is formally one "neither controls." Conservation composes exactly within each unit of account; cross-currency totals require time-indexed valuation and an explicit exposure model. Reputation is computed from retained outcomes and their tombstones; compensation cannot erase past reliance. Every folk-theorem result rests on a grading process included in the monitoring model. Myerson–Satterthwaite constrains bilateral slices only when its assumptions hold; the paper does not yet prove which corner the full market sacrifices. The bones are good and the floor is built; the

spine the market trades on has partial local identity and outcome substrates but no complete reputation or cross-operator binding — and this chapter says so in the same words throughout, because a market built on an overstated keystone is a market built on sand.

Chapter Appendices

6.A Implementation & status

The body of this chapter is deliberately implementation-agnostic. This appendix records, for the reader who wants to check the claims against running code, exactly which mechanisms are shipped in the reference implementation, which are specified with a proof or a written protocol, and which are still proposed. The single-line summary: an **implemented** conserving bond ledger, an **implemented** continuity subsystem (episodic memory), a partial checkpoint, and a richly specified-and-model-verified protocol stack — but the spine the market trades on (cross-operator identity → outcome ledger → reputation) is partial-to-*proposed*. Today the harbor economy has a local labor-settlement substrate and proposed rental and licensing contracts; the number of platform sides is not a deployment status.

Numbers by Hand 6.A.1 (One Trace of the Ten Thousand the Property Test Runs).

The table below grades bond-ledger conservation **implemented**, grounded in a property test over ten thousand random operation traces (Design Invariant 6.2.1); here is the smallest trace by hand so "property test" is not a black box.

Escrow a 100 cr bond (debit 100 from wallet, credit 100 to escrow: $\Delta = 0$, value only moved buckets), then refund it on clean exit (debit 100 from escrow, credit 100 back to wallet: $\Delta = 0$ again).

Composing the two traces gives

$$\Delta = 0 + 0 = 0$$

and wallet + escrow + commons returns to its starting value — one instance of the 10,000 the property test checks, none of which the daemon is trusted to get right by assertion alone.

Table 6.5. Implemented ledger and memory components do not complete the market. Identity, outcome attribution, cross-operator transfer, and reputation retain the gaps listed here.

Claim / mechanism	State	Grounding
Bond ledger: escrow, slash, commons pool	**implemented**	bond-ledger module (`lib/bonds.ts`)
Conservation wallet + escrow + commons = supply	**implemented**	10k-trace property test
No-spawn-without-bond	partial	escrow-first; runtime gate is a roadmap item
Episodic memory (continuity subsystem 1)	**implemented**	episodic-memory module
Resurrection / checkpoint (subsystem 2)	partial	passes notes, not state
Outcome ledger (subsystem 3 — reputation keys here)	partial	commitments, daemon-derived deadlines, oracle closure, API/CLI, and overdue monitoring ship; neutral grading and reputation binding do not ship
Local non-forgeable identity (intra-harbor)	partial	`actor-souls`, lookup credentials, and a bounded newcomer pool ship; full write gating does not ship
Cross-operator attestation	specified/*proposed*	no spec for the critical part
Float plan + signed escrow ceremony	specified	Protocol 6.2.1
Cross-harbor capability transfer (attenuation)	specified	verified *as model*; *The Federated Harbor*, §8.2.1

(continued)

Claim / mechanism	State	Grounding
Reputation estimator (Elo / Bradley–Terry / TrueSkill)	specified/*proposed*	no estimator shipped
Multi-dimensional reputation, neutral judges	*proposed*	no axis-aggregation spec
Labor / rental / licensing contracts	specified	rental and licensing unshipped
Skill licensing (metered + clawback + Merkle proof)	specified	cross-harbor verifiability undeployed
Adversarial test market (Purchased Assurance Bound, Thm. 6.13.2)	*proposed*	no mutation-testing oracle and no bounty-per-killed-mutant contract shipped; the deterrence threshold is proved in the inspection-game analysis
Port Daddy Harbor 2026		
Federation: witness log, capability transfer	specified	*The Federated Harbor*
Recipient-whitelisted escrow custody	specified (conditional property)	requires non-bypassable authority; no runtime conformance
Cross-harbor conservation	specified (property)	proof sketch (App. 6.B); unexecuted lifecycle sketch
Federated revocation dissemination	specified (conditional expectation)	no finite partition deadline; equivocation delivery *open*
Hosted-trust moat / rail separation	specified (positioning)	not a code artifact
Competitive underwriting (4th side)	*proposed*	composition unresolved

6.B Proof sketches

We sketch the three central results; full proofs of the conservation results appear in *The Federated Harbor*.

Numbers by Hand 6.B.1 (Theorem 6.5.1's Composition Law, Checked against the Settlement It Governs).

The proof below is symbolic; here it is against a trace already worked in §6.2. Let f be the two escrow debits that fund the plan (Bob's wallet −400, Alice's wallet −100, escrow +500) and g be the **Partial** settlement that follows (escrow −500, Bob's wallet +240, Alice's wallet +200, commons +60).

Both move value only between wallet, escrow, and commons, so $\Delta(f) = \Delta(g) = 0$ by the theorem's own claim; composing,

$$\Delta(g \circ f) = \Delta(g) + \Delta(f) = 0 + 0 = 0,$$

exactly the 0 net delta the worked example confirmed by summing its four buckets directly.

The theorem and the hand count are the same fact checked two ways.

Proof sketch of Theorem 6.5.1.

For a valid trace $f : x \to y$, define $\Delta(f) = S(y) - S(x)$.

For composable traces $f : x \to y$ and $g : y \to z$,

$$\Delta(g \circ f) = S(z) - S(x) = [S(z) - S(y)] + [S(y) - S(x)] = \Delta(g) + \Delta(f).$$

Every internal generating operation has zero delta because its debits and credits match, including a cross-harbor move through the explicit in-flight bucket.

By induction on trace length, a composition of internal generators has zero delta; top-ups and withdrawals contribute exactly their recorded boundary amounts. □

Boundary note for Property 6.5.1.

Native-unit conservation follows by applying Theorem 6.5.1 separately to each unit.

A scalar valuation $V_t(x) = \sum_j v_{t,j} x_j$ changes either when native balances change or when the valuation vector v_t changes.

The second term is mark-to-market exposure. No finite φ follows without assumptions bounding rate movement, oracle delay, fees, and slippage; those assumptions are not supplied here. □

Proof of Theorem 6.5.2 (the sacrificed corner).

Restrict attention to a bilateral slice satisfying the assumptions stated in Theorem 6.5.2.

The Myerson–Satterthwaite impossibility then rules out the simultaneous achievement of all four desiderata.

The implication is conditional: before declaring which corner the harbor gives up, a mechanism must specify its types and transfers and prove its incentive and participation properties. The present paper has not done so, hence makes no stronger allocation claim. □

What *The Bonded Commons* receives The proposed contracts name a principal, a grade, a bond, and a settlement rule. They still need evidence-admission rules, accounting invariants, and limits on remedies. The next chapter formalizes these obligations and distinguishes its bounded model checks from runtime and economic assumptions.

7

The Bonded Commons

The question this chapter answers

Who may decide, on what evidence, with what consequence?

Bonds, witnesses, audits and appeals turn residual uncertainty into an institution; the ledger's conservation law is mechanically checked.



The power to constrain an adversary may depend on the power to bind oneself.

Thomas C. Schelling, The Strategy of Conflict, 1960

Who may decide, on what evidence, with what consequence? Attribution without consequence is only a diary; consequence without admissible evidence is arbitrary. This chapter turns the local writer into an institution by joining capability bounds, witnessed records, audits, appeals, and conserved settlement. R7–R11, R15, R17, and the R8 machine state exactly what has been proved, checked, or left conditional.

When Agent A delegates a subtask to Agent B, it implicitly trusts that B will not corrupt the shared state, will not exceed the scope of the delegation, and will not silently fail in a way that poisons A's downstream work. When Agent B accepts the delegation, it implicitly trusts that A's description of the task is accurate, that the resources A promised are available, and that A will still exist when B finishes. These implicit trust assumptions pervade every multi-agent interaction, and every one of them can be violated. Repeated negotiation can consume agent time and context. The design here moves common admission rules into the commons authority while retaining verification at the relevant boundaries. It does not require abandoning per-message authentication or access checks. Hobbes's argument for enforceable covenants offers a historical motivation for shared settlement rules: "Covenants, without the sword, are but words, and of no strength to secure a man at all." The analogy concerns commitments backed by enforcement. It does not establish that cooperation requires a common authority in every setting. Here, a shared authority is one proposed way to make admission and settlement rules available before agents begin work. A **commons authority** supplies shared admission, attribution, and settlement rules. Agents enter under those rules, but admission alone does not establish good behavior. The incentive and monitoring assumptions below determine which risks the arrangement can limit.

Hobbes 1651

7.0.1 Three Risks of Ad Hoc Pairwise Trust

Ad hoc pairwise arrangements can expose agent systems to three risks:

Pairwise negotiation can be costly. A complete all-to-all arrangement among n agents has $n(n-1)/2$ potential unordered

pairs. If each pair separately negotiates trust, the number of negotiations is quadratic in n. Actual token use, latency, and context demand depend on the interaction topology, the cost of each negotiation, concurrency, and whether established credentials or agreements can be reused. Sparse interaction and shared verification can reduce that cost; the pair count alone establishes no fleet-size threshold for feasibility.

Ephemeral trust state can be lost. If credentials, agreements, and continuity records exist only in an agent's volatile state, a crash can leave its successor unable to establish the authority or commitments it inherited. Durable peer-to-peer protocols can preserve this evidence; shared infrastructure is another way to make continuity explicit and recoverable.

Faithful execution need not serve the commons. An agent may follow its principal's instructions faithfully while pursuing interests that conflict with those of other participants. Evidence of faithful execution alone does not establish a common interest. Admission, monitoring, and contract design must therefore account for the principal's incentives as well as the agent's observed behavior.

7.0.2 Contributions

We present:

1. A reading of the coordination daemon as a **candidate correlating device** in the sense of Aumann: claim recommendations are private signals drawn from a common-knowledge distribution, while incentive compatibility remains conditional on explicit obedience inequalities (Section 7.1, esp. §7.1.1).
2. A **three-layer trust architecture** (structural, evidentiary, economic) that does not depend on agent morality, shared values, or threat of termination (Sections 7.2–7.6).
3. A Sen-grounded **design warning** about decisive resource rights under private information, plus a separate completeness theorem for **advisory resource claims** (Section 7.4).

Robert J. Aumann. Subjectivity and Correlation in Randomized Strategies. *Journal of Mathematical Economics*, 1(1):67–96, 1974. Online source

4. A **TLA$^+$ lifecycle excerpt** that states safety and recovery obligations and exposes the fairness needed for eventual recovery (Section 7.9).

5. The design of a **collateralized work contract** system (Float Plans) that prices agent risk and settles outcomes mechanically, transforming the moral question of agent trustworthiness into the economic question of adequate bonding (Section 7.6).

The security layer (agent authentication and capability delegation) is provided by the Anchor Protocol, an accompanying chapter. This chapter builds the trust, governance, and economic layers on top of that cryptographic foundation.

Erich Owens. The Anchor Protocol: A Mechanically Analyzed Control Plane for Local Agent Swarms. Curiositech LLC Technical White Paper, May 2026.

7.0.3 Related Work in Crypto-Economic Bonding

The architecture sits at the intersection of four bodies of prior work, each of which we extend rather than replicate.

Bonded participation in distributed systems. Proof-of-stake consensus protocols (Casper, Tendermint, and the Ethereum 2.0 specification) introduced *slashing*: validators post collateral that is forfeited on equivocation or liveness failure. The technique is the closest analogue to our bonded-commons settlement, and the proofs of soundness (a rational validator strictly prefers honest behavior when slash > deviation gain) are direct ancestors of §7.6. We depart from this lineage on three axes: (i) the adversary is co-located on a single machine, not a Byzantine network, so consensus reduces to a daemon write rather than a global commit; (ii) the bonded act is *coordination*, not validation—work that produces side effects in a shared file system, not signatures on blocks; and (iii) the principal is human-or-LLM mixed, so the mechanism must price two failure modes (malice and confusion) the blockchain lineage collapses into one.

Vitalik Buterin and Virgil Griffith. Casper the Friendly Finality Gadget. arXiv:1710.09437, 2017.

Ethan Buchman. Tendermint: Byzantine Fault Tolerance in the Age of Blockchains. M.A.Sc. thesis, University of Guelph, 2016.

Ethereum Foundation. Ethereum 2.0 Phase 0 – The Beacon Chain Specification. Online source, 2020.

Capability-based security. Dennis and Van Horn, Miller's object-capability work, and the SPKI/SDSI line provide the structural attenuation that we lift wholesale into the Anchor Protocol layer. Our contribution is composing capability attenuation with bonding: a capability without economic settlement still permits the holder to

Dennis & Van Horn 1966

Miller 2006

Ellison et al. 1999

"correctly" execute a destructive action; bonding without capability bounding still permits unbounded blast radius before slash. Both layers are necessary.



Ostrom documented institutions that sustained shared resources without assuming inevitable collapse.

Ostrom 1990

Kevin Werbach. *The Blockchain and the New Architecture of Trust*. MIT Press, 2018.

Vitalik Buterin. A Next-Generation Smart Contract and Decentralized Application Platform. Ethereum white paper, 2014.

Commons governance. Ostrom's eight design principles are evidence-informed guides to institutional design, not necessary-and-sufficient conditions for cooperation. This design draws particularly on boundaries, monitoring, graduated sanctions, and accessible dispute resolution (§7.4.3). Implementing those functions in software does not establish collective-choice rights, legitimate rule-making, or successful governance; those require separate institutional evidence.

Cryptoeconomic mechanism design. Werbach, Buterin, and the broader DAO literature establish "code is law" as a posture toward enforcement. Our position is narrower and weaker on purpose: code is the *record*, not the law. Allocation decisions remain with the agents (Sen, §7.4); the daemon supplies the evidence and the settlement. This deliberate weakening is what makes coercive enforcement avoidable while keeping accountability intact. Table 7.1 compresses the four paragraphs above into what is borrowed, what is departed from, and why.

What is novel. The synthesis: bonded participation + capability attenuation + advisory (not enforced) coordination + immutable attribution, packaged as localhost infrastructure rather than a global protocol. We are not aware of a prior system that combines all four in this form; each component has precedent, while the composition targets auditable coordination and attribution that survives identity disposal. Its welfare effects remain conditional on the explicit payoff and monitoring models developed below. Table 7.2 shows the composition.

7.1 The Commons Authority

Return to Agent A and Agent B from the introduction's opening scene—the two agents who corrupted the same middleware because neither knew the other existed. Nothing about that failure

Lineage	What we borrow	What we depart from	Why
Bonded participation (PoS slashing)	Slash->-deviation-gain soundness argument	Byzantine networked adversary; validation as the bonded act	Adversary is co-located on one machine, so consensus reduces to a daemon write, not a global commit
Capability-based security	Offline attenuation; authority only ever narrows	Capabilities alone as the whole story	A correctly-scoped token still permits a destructive in-scope write; bonding prices what the wall admits
Commons governance (Ostrom)	Boundaries, monitoring, graduated sanctions, dispute resolution	Social institutions as the enforcement medium	The four functions guide the service design (§7.4.3); each mechanism has its own implementation status
Crypto-economic mechanism design	Priced deterrence; market-discovered parameters	"Code is law" as an enforcement posture	Code is the *record*, not the law: allocation stays with agents (§7.4); the daemon supplies evidence and settlement

Table 7.1. Every borrowed result is re-used at a smaller scale than its origin assumed: the adversary is co-located rather than networked, the bonded act is coordination rather than validation, and the record is evidence rather than law. Nothing in the four lineages is discarded — each is narrowed by one assumption, and the narrowing is what makes the composition deployable on one machine.

required malice, and nothing about it would have been fixed by making either agent more careful. What was missing was a third party that both agents were already talking to: something that knew A had the file, could tell B so at claim time, and would still remember the whole episode after both processes exited. That third party has to be settled *before* any interaction, not improvised transaction by transaction—otherwise establishing it is just the $O(n^2)$ trust negotiation of §1.1 under a new name. We call it the *commons authority*:

Table 7.2. Three layers answer three different questions about the same attempted act. An out-of-grant act ends at Layer 1 before shared state changes; every admitted act crosses Layer 2 and acquires durable attribution, so admission creates an attribution duty; a harmful admitted act reaches Layer 3, where only bound evidence can reach settlement. [internal]

Layer	Question	What it does to the act	Terminal outcome
1 scope	can this act exist?	terminates an out-of-grant trajectory before shared state changes	refused
2 evidence	who did what?	binds every admitted act to a durable, addressable attribution	complete and recorded
3 collateral	who carries the loss?	prices a harmful admitted act against posted collateral; only bound evidence can reach settlement	priced settlement

one always-on service that every agent already trusts, precisely because it was there before any of them showed up. Five capabilities make it that, and the definition below names them.

Definition 7.1.1 (Commons Authority)**.**

A commons authority $\mathcal{D}$ for a multi-agent system is a persistent service that provides:

1. **Identity**: Every agent receives a cryptographic identity before participating.
2. **Capability bounding**: Every agent's operations are structurally limited by attenuable tokens.
3. **Shared knowledge**: Resource claims, agent status, and conflict information are visible to all participants.
4. **Immutable history**: All actions are recorded in an append-only, tamper-evident trail.
5. **Economic settlement**: Work contracts are collateralized and settled against verifiable evidence.

The commons authority is not a central planner. It does not decide which agent should work on which task, which file conflicts should be resolved in whose favor, or which agents are "good." It provides the *infrastructure* within which agents make those decisions for themselves, using the shared knowledge and economic incentives the authority maintains.

Remark.

The commons authority is a *Hobbesian sovereign*, not an *omniscient coordinator*. Hobbes's Leviathan does not micromanage citizens' daily choices—it establishes the conditions (laws, enforcement, courts) under which citizens can trust each other enough to transact freely. Similarly, the commons authority establishes identity, capability bounds, shared knowledge, and economic settlement—and then gets out of the way.

In the Port Daddy implementation, the commons authority is a daemon running on `localhost:9876`, backed by SQLite. The simplicity of the implementation is deliberate: the authority's value comes from its *guarantees*, not its complexity.

7.1.1 The Daemon as Correlating Device

Suppose two agents want the same file. A mediator recommends that one proceed and the other defer. Those recommendations avoid a collision only if the agents follow them. To predict obedience, we must compare each agent's payoff from following its recommendation with its payoff from deviating, given what it knows. A daemon's ability to enforce a capability boundary is a separate question: recommendations change information; enforcement limits actions.

Numbers by Hand 7.1.1 (A Recommendation That Both Agents Want to Follow).

For one file, suppose proceeding alone pays 3, deferring while the other proceeds pays 1, a collision pays -2 to each, and both deferring pays 0. A mediator flips a fair coin and privately recommends (P, D) or (D, P). An agent told to proceed earns 3 by following rather than 0 by deviating; an agent told to defer earns 1 rather than -2. Both recommendations are obedient under these payoffs and observations. In the inequality below, the two weighted advantages are $\frac{1}{2}(3 - 0)$ and $\frac{1}{2}(1 - (-2))$.

Change the payoff from causing a collision to 4, and the defer recommendation ceases to be obedient: $1 - 4 < 0$. The identical message protocol now fails the incentive test. A signed recommendation certifies its source, not its wisdom or the recipient's incentive to follow it.



Aumann's obedience test asks whether following a recommendation pays. It does not establish the daemon's authority or its ability to prevent an action.

Aumann 1974

The daemon's role therefore admits a precise game-theoretic test. Aumann introduced the *correlated equilibrium*: a mediator draws a recommendation tuple $r = (r_1, \ldots, r_n)$ from a joint distribution μ over action profiles and privately reveals r_i to player i. The distribution is a correlated equilibrium only if, for every player i, recommended action r_i, and unilateral deviation a_i', the obedience inequality holds:

$$\sum_{r_{-i}} \mu(r_i, r_{-i}) \big[u_i(r_i, r_{-i}) - u_i(a_i', r_{-i}) \big] \geq 0.$$

Every mixed Nash equilibrium induces such a distribution, but a mediator does not create an equilibrium merely by issuing recommendations. The inequality is the governing condition.

The commons authority can play this mediator role. It observes the global claim graph and emits a recommendation pair on a contested file: *Agent α, proceed; Agent β, defer or coordinate.* A deterministic policy can still induce a non-degenerate μ through uncertainty over observed system states, but determinism is not itself evidence of obedience. Utilities, bond coverage, and continuation values must make the inequality above hold.

Property 7.1.1 (Conditional Correlating-Device Interpretation)**.**

Port Daddy's daemon supplies the information structure of a correlating device for the agent-coordination game.

It induces a correlated equilibrium only under the following conditions:

1. **Specified distribution.** The model fixes a joint recommendation distribution μ and assumes the players know it.
   A public policy and a replayable log do not alone supply this distribution: uncertainty about inputs, their probabilities, and each player's observations must also be specified.
2. **Specified information.** The displayed inequality models a player who observes its own recommendation. If the public claim graph reveals additional information, including another player's recommendation, obedience must be checked conditional on that information too. Private delivery alone does not make a signal private.
3. **Obedience inequality.** For every recommendation and deviation, the expected deviation gain is no larger than the expected bond loss plus discounted reputation loss. These

losses must reach the decision-maker. An observable deviation need not be unprofitable, and a logged claim need not reveal every off-ledger action.

The reframe therefore yields an audit obligation, not an automatic guarantee. Nothing physically stops an agent from ignoring advice. The repeated-game calculation in Section 7.6 verifies one stylized two-player payoff table; deployments with different utilities must re-check the obedience inequalities. Section 7.9 separates lifecycle safety from recovery liveness and exhibits the missing fairness assumption. Its bounded accounting checks do not establish strategic optimality.

No price-of-anarchy bound follows from this interpretation alone. The simulations of §7.6.5.8 compare one auction model with one static baseline; an analytical worst-equilibrium bound for the coordination game remains open.

Formal-backing status of the game-theoretic core. The evidence has several distinct scopes. The claim-signaling payoff model has the analytical argument below, a scalar Z3 check, a bounded TLA$^+$ witness, and a companion treatment in research paper 3. The insurer comparisons have finite parameter sweeps and their stated payoff recurrences. The broader multi-principal deployment has no equilibrium proof. Formal checks of a supplied model do not establish that the deployment satisfies its payoff, observation, or identity assumptions. Table 7.10 records the artifacts; the boundary paragraphs state what remains to be shown.

7.1.2 Why Agents Consent

In Hobbes, consent to the sovereign is rational because the alternative—the state of nature—is worse. For an agent principal, joining is rational when the services below are worth more than their fees, disclosure costs, and constraints. The comparison is conditional: a durable peer protocol may already supply some of these services.

The daemon does not need to threaten agents. It provides services that make coordination *rational*. Whether a principal benefits depends on its workload and outside option; the table alone establishes no dominance result.

Table 7.3. The commons authority turns six ad hoc failure modes into services an agent gains rather than permissions it loses.

Capability	Without Authority	With Authority
Port assignment	Race condition	Deterministic, collision-free
File coordination	Discover conflicts at merge	Detect conflicts at claim time
Crash recovery	Work lost permanently	Successor reads immutable trail
Delegation	Bilateral trust negotiation	Attenuated capability tokens
Reputation	Every interaction from zero	History persists across lifetimes
Accountability	Anonymous harm	Full attribution via evidence chain

7.2 Layer 1: Structural Prevention

The first layer of trust does not depend on agent reputation or economic incentives. It depends on *walls*: structural authorization checks that reject out-of-scope operations wherever the runtime actually interposes them.

7.2.1 Capability Attenuation

Owens 2026

The Anchor Protocol provides Harbor Cards—Ed25519-signed capability tokens that bound the operations available to each agent. The critical property is **offline attenuation**: when Agent A delegates to Agent B, it creates a new token whose capabilities are a subset of its own. Agent B can further delegate to Agent C with even narrower capabilities. Capabilities can only shrink along the authorization chain, never grow.

The capability subset check and the secret-independent comparison are implemented in a **Rust core** (`harbor-card-rs`) that is compiled to a shared library and called from the Node.js daemon via FFI. Kani harnesses check that the parser and the comparator cannot panic on bounded symbolic inputs and exercise the subset check on two concrete vectors; the every-hop attenuation property is ProVerif's, in *The Anchor Protocol*; runtime conformance tests cover the bridge. Compiler behavior, full interception coverage, and hardware side channels remain separate obligations.

The guarantee below is what security engineers mean by a *wall* rather than a *law*: not an instruction an agent could disobey, but a bound on which operations are even representable.

Definition 7.2.1 (Capability-Bounded Transaction)**.**

An agent transaction $\mathcal{T}_\alpha$ is capability-bounded by token τ_α if every operation o executed during $\mathcal{T}_\alpha$ satisfies $o \in C(\tau_\alpha)$, where $C(\tau)$ extracts the capability set from a Harbor Card.

The commons authority rejects any operation $o \notin C(\tau_\alpha)$ at the verification step, before the operation can affect shared state.

We reuse Anchor's attenuation relation: each child's authorized operations and validity interval must lie within its parent's. Here the mathematical step is transitivity of those inclusions. Anchor's model checks and runtime conformance evidence retain their stated bounds; they do not make every filesystem operation pass through a verifier. The theorem therefore names complete mediation as a separate premise.

Theorem 7.2.1 (Structural Scope Limitation).

For any authorization chain $\tau_0 \to \tau_1 \to \cdots \to \tau_k$ satisfying *The Anchor Protocol*'s attenuation condition, where τ_0 is issued by the commons authority and each τ_{i+1} is attenuated from τ_i:

$$C(\tau_k) \subseteq C(\tau_{k-1}) \subseteq \cdots \subseteq C(\tau_0).$$

Within an execution that routes every relevant operation through the verifier, no accepted operation can lie outside the scope originally granted by the authority.

Proof.

The Anchor model establishes a non-injective authentication correspondence for accepted modeled tokens and separately checks attenuation at the modeled chain depth. Given those model assumptions and the theorem's premise that every operation passes through the verifier, each accepted hop satisfies the subset relation; transitivity yields the displayed chain. The correspondence alone does not prove replay freedom, arbitrary-depth delegation, or complete runtime interception, which are separate obligations. □ Owens 2026

Remark.

This is the layer where the metaphor of "walls, not laws" applies. A law says "you shall not write to the production database." A wall means your token is not valid for production database writes, and no amount of intent—good or bad—can change that. Laws require enforcement; walls require only construction.

Exercise 7.1, p. 630.

7.2.2 Isolation Domains

Beyond capability tokens, the commons authority supports **structural isolation** via git worktrees: physically separate copies of the repository where agents operate on disjoint filesystem state.

Definition 7.2.2 (Isolation Levels)**.**

Three tiers of isolation are available, ordered by strength:

I_0 **(None):** Agents share a working directory.
No conflict detection.
Maximum parallelism, no safety.

I_1 **(Advisory):** Agents share a working directory but register file claims with the commons authority. Conflicts are detected and reported to all participants. Claims are not enforced. This is the default.

I_2 **(Structural):** Each agent operates in a separate git worktree. Distinct working directories separate ordinary edits. They do not prevent same-user processes from accessing another worktree, and Git metadata and external services may still be shared. Integration conflicts remain possible.

Figure 7.1 draws the three tiers as an ordered ladder, I_1 marked as the default.

The choice between I_1 and I_2 concerns editing interference and coordination cost. A worktree is not an access-control sandbox; security needs a separately enforced filesystem and process boundary. Section 7.4 formalizes why.

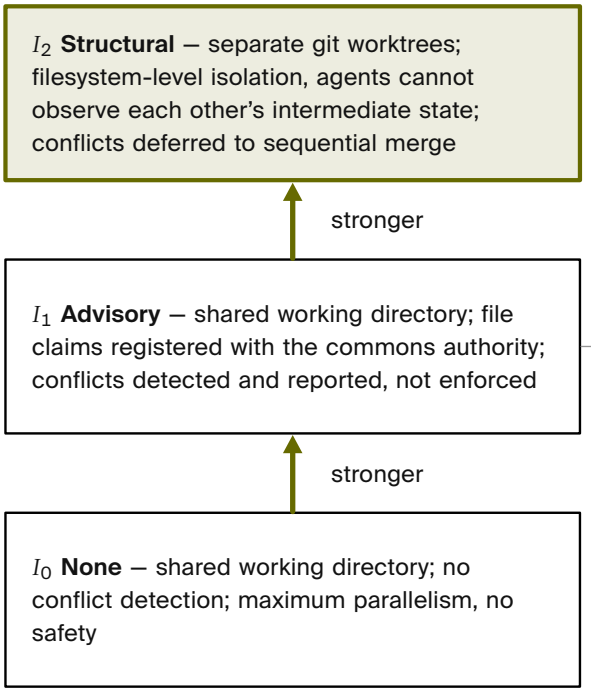


Figure 7.1. The three isolation tiers order strictly by strength, weakest to strongest: none, advisory, structural. I_1 (advisory) is the default — file claims are registered and reported but never block. Moving up buys more safety at the cost of parallelism; the choice between I_1 and I_2 specifically is not a security decision but an economics one, argued in Section 5. [internal]

7.2.3 Runtime Invariant Enforcement

Structural prevention is enforced at two levels: *statically* by the Anchor Protocol's ProVerif-verified token exchange (all three protocol phases verified in ProVerif 2.05, under the applied-pi-calculus methodology of Blanchet), and *dynamically* by the **Arbiter**—an ambient security agent that subscribes to the daemon's event stream and checks every state transition against six formally derived invariants. These invariants map directly to the safety properties of the TLA$^+$ specification in Section 7.9: `NoteMonotonicity`, `EscrowInvariant`, and `LockOwnerValid` are compiled from the formal model into synchronous runtime checks. Violations are logged immutably and, in strict mode, trigger the salvage protocol—the sovereign's sword in code form.

Owens 2026

Blanchet 2016

Regimentation vs. Enforcement (OP-2). A deterministic local guard is a sufficient way to implement pre-commit *regimentation*, in Jones and Sergot's sense: the mediated violation is refused before its effect. This requires complete mediation and an atomic check-and-commit; merely observing a transition afterwards is *enforcement*. External I/O does not determine which category applies.

Jones & Sergot 1993

An operation may remain pending while a remote authority issues an authenticated permit bound to its payload, resource version, expiry, and authorization epoch. A local admission step verifies that binding before committing, and a timeout refuses admission. This can preserve safety at an availability cost. Where evidence arrives only after an irreversible effect, detection and compensation are the available mechanisms. Figure 7.2 contrasts these admission and recovery paths.

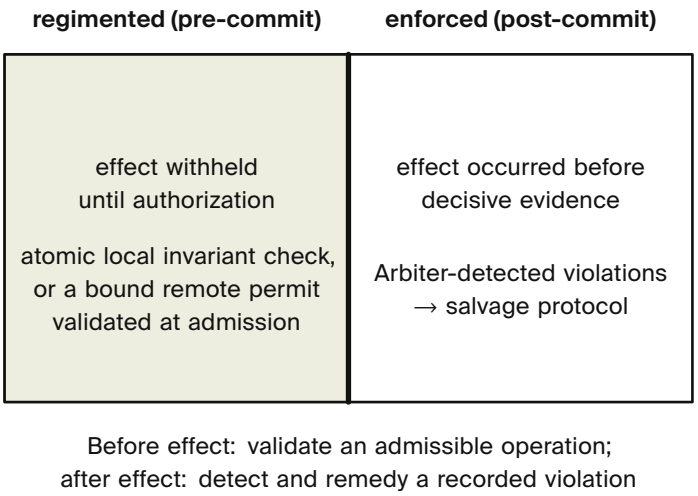


Figure 7.2. Complete mediation and atomic admission can prevent violations, including operations authorized by remote permits. Evidence available only after an irreversible effect supports detection and recovery. [internal]

7.3 Layer 2: Immutable Attribution

Structural prevention handles accidental harm—an agent cannot exceed capabilities it does not hold. But an agent *with* legitimate write access can write garbage. The second layer ensures that every action is permanently attributable.

7.3.1 The Evidence Chain

Every agent session maintains an **append-only note trail**—a sequence of timestamped, immutable records of observations, decisions, and progress.

Definition 7.3.1 (Evidence Trail)**.**

An evidence trail $N = \langle n_1, n_2, \ldots, n_k \rangle$ is an ordered sequence of notes where:

1. Each n_i contains: content (natural language), type (note, decision, handoff, blocker), and timestamp.
2. Notes are append-only: there exists no API operation that modifies or deletes individual notes.
3. Notes survive session completion: the trail persists whether the session commits, aborts, or is abandoned due to agent death.

Theorem 7.3.1 (Trail Integrity).

The evidence trail is immutable by construction.

No `UPDATE` or targeted `DELETE` operation exists in the system's API surface. Notes are destroyed only by explicit administrative deletion of the parent session (CASCADE), which is an auditable action distinct from normal transaction lifecycle operations.

Proof.

By code inspection of the session module: the note insertion API performs only `INSERT INTO session_notes`. The only deletion path is `DELETE FROM sessions WHERE id = ?`, which triggers `ON DELETE CASCADE`. Since transaction completion (commit or abort) updates the session's *status* field but does not delete the session row, notes survive all normal lifecycle transitions. □

7.3.2 The Merkle Forest: Cross-Daemon Inclusion Proofs

For high-assurance environments and for fleets spanning multiple daemons, the evidence trail is strengthened to a three-level **Merkle Forest**. The single-daemon Merkle chain—each note hashing the previous, with a per-session root—is necessary but not sufficient: the moment two daemons exist, an auditor without database access still needs to verify that a particular note was included in a particular session using only a short proof.

Definition 7.3.2 (Merkle Forest)**.**

The evidence structure has three levels:

Note chain. Each note's header includes $h_i = H(n_i \| h_{i-1})$ over SHA-256.
The session's note commitment is h_k, the hash of the last note.

Session root. All note hashes $(h_1, \ldots, h_k)$ are assembled into a Merkle binary tree with root R_{session}. The chain commits to order; the tree commits to presence with $O(\log k)$ proofs.

Harbor root. Periodically (per-settlement, or hourly), every completed session root within a harbor is assembled into a harbor Merkle tree with root $R_{\text{harbor},\ell}$ at epoch ℓ, signed by the daemon's Ed25519 key.

The collection $\{R_{\text{harbor},\ell}\}_\ell$ is the *Merkle forest*: one tree per epoch.

Figure 7.3 follows the inclusion check: start with the note hash, reconstruct its session root, then reconstruct the harbor root and check the daemon's signature. This proves inclusion, not that the note is true. The byte calculation follows the figure.

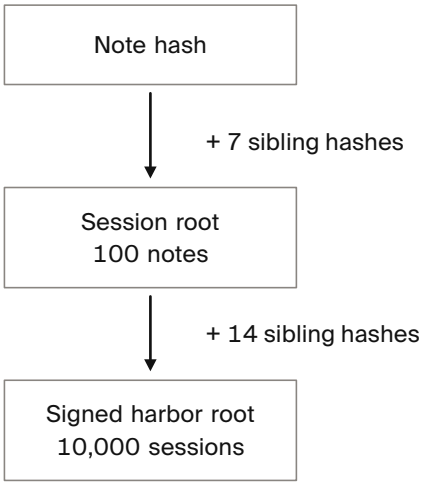


Figure 7.3. Two inclusion paths connect a note to a signed root. Here, the hashes and signature occupy 736 bytes. [hand-derived]

Inclusion proofs. To prove that note n_i from session s belongs to harbor h at epoch ℓ, the daemon emits: (i) the note inclusion path of size $O(\log k)$, (ii) the session inclusion path of size $O(\log N)$ where N is the number of settled sessions in the epoch, and (iii) the signed harbor root $\sigma_{\text{daemon}}(R_{\text{harbor},\ell})$. For $k = 100$ notes and $N = 10^4$ sessions, the two binary paths contain $\lceil \log_2 100 \rceil + \lceil \log_2 10^4 \rceil = 7 + 14 = 21$ sibling hashes. At 32 bytes per SHA-256 hash plus one 64-byte Ed25519 signature, the proof is $21 \times 32 + 64 = 736$ bytes, about 0.7 KiB—small enough to embed in any settlement receipt.

A verifier needs an authenticated checkpoint as well as a path. Figure 7.4 separates these checks in a four-record example. A matching hash alone does not establish who committed to the root.

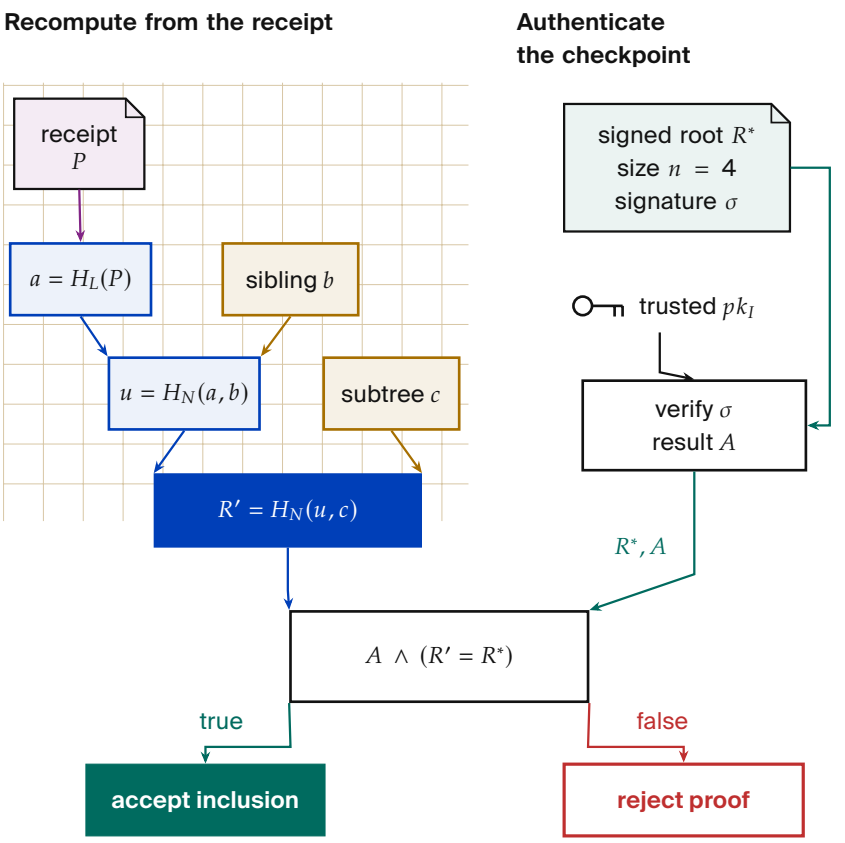


Figure 7.4. Two sibling hashes connect receipt P to a four-record tree. Accept only when a trusted-key signature verifies and the roots match. The proof establishes inclusion, not truth.

Here b is the other leaf in the receipt's pair, while c is the other two-record subtree. Their order matters. H_L hashes a leaf and H_N an ordered pair, using distinct domain tags. The signed checkpoint must also identify the intended tree and its size. The drawing omits wire-format details; the two-level forest applies the path calculation at each level.

Witness to an abstract KMS. Each $R_{\text{harbor},\ell}$ is uploaded as a **witness** to a Key Management Service satisfying the properties enumerated in §7.11. The KMS enforces append-only monotonicity and periodically publishes a signed tree head, in the Certificate Transparency pattern. Equivocation—publishing different roots for the same epoch to different parties—is detectable by auditors performing consistency proofs across tree heads.

Laurie et al. 2013

Design Invariant 7.3.2 (Proof Soundness).

If verify-note-inclusion(n, s, h, ℓ, π) returns `true` for some proof π, then either (a) n was included in session s at epoch ℓ, or (b) the daemon's signing key has been compromised, or (c) SHA-256 has been broken.

Here "included" means membership in the authenticated committed tree, not that the recorded action actually happened. Key compromise is an operational threat, not something signature unforgeability makes impossible. Verification also requires canonical encoding, domain separation, and a bound tree identity and size.

Design Invariant 7.3.3 (Binding).

Once a daemon publishes $R_{\text{harbor},\ell}$, it cannot produce a different valid root for the same $(\textit{harbor}, \ell)$ without contradicting its earlier signature (detectable via the KMS witness) or forking its identity.

Cost. Each settlement adds an $O(1)$ amortized append to the epoch accumulator. Epoch rollover is $O(N)$ hashes plus one signature: at 100 sessions/hour, $\approx$ 10ms. Negligible.

This is the structural meaning of "decentralized verification" in this chapter: bonded commons evidence is not just immutable within a daemon, it is verifiable *across* daemons by any party with the daemon's public key and a 700-byte proof.

7.3.3 Mutable-Signal Ledger (Pheromone Lineage)

Stigmergic coordination originated in biology, where pheromones are accumulate-only: ants deposit, the environment evaporates. Software agents need a richer model. Coordination signals must be *revocable, renamable, and provenance-attributable* as understanding of the work evolves—an agent that deposited a hint and later realized the hint was wrong needs a way to retract it that preserves attribution rather than erasing it.

Guy Theraulaz and Eric Bonabeau. A Brief History of Stigmergy. *Artificial Life*, 5(2):97–116, 1999.

Event-sourced pheromones. Each entity's pheromone state is a view over an append-only event ledger. For entity e and key k:

$$\sigma_t(e,k) \;=\; \text{decay}(S, t - t_0) \;\cdot\; \mathbb{1}\big[\text{not revoked or expired in } (t_0, t]\big],$$

where S is the last spray strength on k at time t_0 and decay is a geometric decay defined per-channel. **Revocation** is itself an event; **rename** is a rewrite-pointer event; **fork** creates a new key namespace whose provenance is traceable through the event chain.

Design Invariant 7.3.4 (Attribution Invariant).

Every $\sigma_t(e,k)$ value has a deterministic provenance chain back to one signing principal, traceable via the event chain.

Design Invariant 7.3.5 (Rollback Safety).

A consumer that reads σ at time t and later discovers a revocation event timestamped $t' < t$ can compute the correct counterfactual view by replaying events.

Design Invariant 7.3.6 (Conservation).

Revocation does not erase.

The event ledger is monotone: the cardinality of the event set only grows.

For example, choose decay $S2^{-u/h}$ with strength $S = 8$ and half-life $h = 10$ minutes. After twenty minutes the view is 2. A revocation then makes the view zero while retaining both the original spray and the revocation record. A late-arriving event needs a declared effective order and tie-break rule before replay is deterministic; a timestamp alone is not that rule. The zero view and the retained evidence have different jobs.

Integration with the forest. Pheromone events are included in the session tree of §7.3.2: each session's root summarizes the events the session produced, harbor roots summarize session roots, and revocations are therefore transparent to witnesses without erasing any prior state. The mutable surface is the *view*; the underlying ledger is immutable.

This dual—mutable view, immutable substrate—is what lets coordination signals evolve without compromising the evidentiary guarantees the rest of the paper depends on.

7.3.4 Attribution Survives Identity Disposal

A critical property: even if an agent's identity is discarded after task completion, the **authorization chain** traces every action back to the authorizing principal. The chain is:

$$\text{Principal} \xrightarrow{\text{funds}} \text{Float Plan} \xrightarrow{\text{authorizes}} \text{Agent } \alpha \xrightarrow{\text{delegates}} \text{Agent } \beta \xrightarrow{\text{acts}} \text{Evidence Trail.}$$

The agent is ephemeral. The principal's authorization—signed, escrowed, recorded—is permanent. For completely mediated and faithfully recorded actions, this chain identifies the authorizing principal and collateral. It does not attribute an unobserved write or prove that a signed statement is true; §7.6.5.4 draws the corresponding line for harm from *disclosure*, which this chain does not cover.

7.4 The Limits of Decisive Allocation

A natural question is why file claims are advisory rather than exclusive locks. Sen's theorem illuminates the tension, but it does not prove that every lock manager is inefficient. The argument below separates the theorem from the engineering example.



Amartya Sen. Decisive personal rights can conflict with unanimous preferences; that result is not a blanket verdict against locks.

Amartya Sen. The Impossibility of a Paretian Liberal. *Journal of Political Economy*, 78(1):152–157, 1970. Online source

7.4.1 A Sen-Inspired Design Warning

Sen proved that no social welfare function can simultaneously satisfy:

1. **Pareto efficiency**: If every agent prefers outcome X to Y, the system chooses X.
2. **Minimal liberalism**: Each agent has at least one "personal domain"—a pair of outcomes where the agent's preference is decisive.

The analogy to file coordination is:

- **Pareto efficiency** = the team ships both features (the collectively optimal outcome).
- **Minimal liberalism** = each agent has decisive control over its claimed files (enforced locking).

Theorem 7.4.1 (A Pareto-Inferior Locking Instance).

Consider agents α and β with tasks T_α and T_β. Agent α claims file f because it *might* need to modify it.

Agent β discovers mid-task that it *actually* needs f.

Under enforced claims:

- α's claim is decisive (minimal liberalism): β is blocked.
- The team-level outcome (T_α and T_β both completed) is blocked by α's precautionary claim.
- A Pareto-superior outcome exists (both tasks completed) but is unreachable because α's liberal right vetoes it.

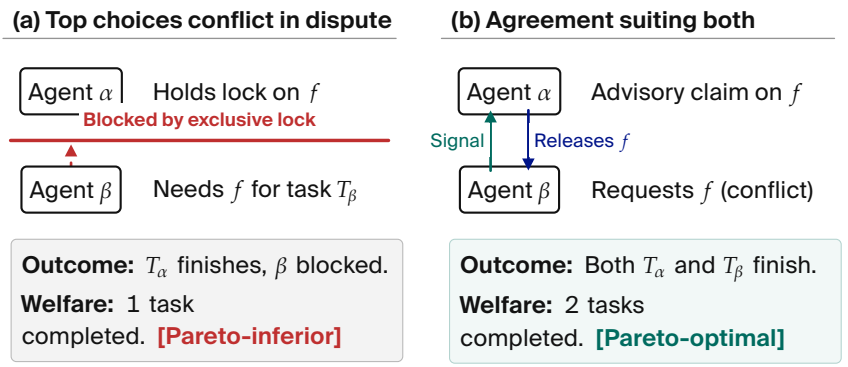
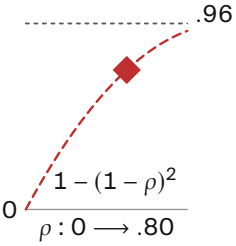


Figure 7.5. Precautionary locks versus advisory claims. In panel (a), agent α's decisive lock on an unneeded file f aborts agent β's task T_β. In panel (b), advisory notification lets α release f, achieving the Pareto-superior outcome where both tasks succeed.

This property is a counterexample to the universal claim that exclusive locks always improve team welfare; it is not a derivation of Sen's theorem. Sen's result requires an unrestricted preference domain and a social-choice rule with decisive personal pairs. A production lock manager may restrict preferences, negotiate releases, or time out claims. The usable lesson is narrower: do not grant an agent an unconditional veto over a shared resource when the team may possess Pareto-relevant information the allocator lacks.

This is not a contrived scenario. AI agents are *inherently uncertain* about which files they will modify when they begin a task. An agent asked to "fix the login bug" cannot predict whether it will need



Precautionary lock welfare loss $L(\rho) = 1 - (1 - \rho)^2$ as file contention ρ scales.

the authentication middleware, the route handler, the test suite, or all three. Enforced claims force agents to either:

1. **Over-claim** (request locks on every file they might touch) → destroys parallelism.
2. **Under-claim** (request locks only on files they're certain about) → produces undetected conflicts, defeating the purpose.

The lesson has a boundary, and it is worth drawing rather than asserting. Advisory claims are not better than locks everywhere; they are better in a region, and the region has a boundary. Where an agent's file needs are knowable up front, an exclusive lock allocates exactly as an advisory claim would and adds determinism at no cost—in that regime the design choice of this section would simply be wrong. Where negotiating a release is prohibitively expensive, neither regime converges, which is the case the hard-lock fallback of §7.4.3 exists to absorb. The advisory default is justified by where LLM coding agents actually sit, not by a universal claim.

7.4.2 Advisory Claims as Resolution

Advisory claims avoid granting that unconditional veto. They do not guarantee Pareto efficiency; they expose conflicts so informed agents can negotiate:

Definition 7.4.1 (Advisory Consistency)**.**

Under advisory claims, file claims form a conflict graph $G = (V, E)$ where vertices are active sessions and edges connect sessions with overlapping claims:

$$(S_i, S_j) \in E \iff \exists f_i \in F_i, f_j \in F_j : \text{overlap}(f_i, f_j).$$

The commons authority guarantees that every edge in G is reported to both endpoints.

No agent has a decisive "personal domain" over any file. Instead, agents receive complete information about the conflict graph and self-coordinate.

Theorem 7.4.2 (Advisory Conflict Completeness).

Under advisory claims, for any two concurrent sessions S_1, S_2 with overlapping file claims f, the commons authority reports conflict(f) to S_2 at claim time.

No false negatives exist.

Proof.

The overlap detection query scans all active (unreleased) claims from other active sessions.

A whole-file claim ($R = \bot$) overlaps with any range on the same path. Two ranged claims $[a, b]$ and $[c, d]$ overlap iff $a \leq d \wedge c \leq b$.

Since the query evaluates this predicate exhaustively over all active claims with the requesting session excluded, every overlap is detected. False positives may occur (an agent claims a file it does not modify); false negatives cannot. □

Remark.

The commons authority's role under advisory claims is that of *town crier*, not *Solomon*. It announces conflicts; it does not resolve them. Resolution requires local knowledge that only the agents possess ("I claimed `auth.ts` but probably won't need it"; "I absolutely must modify `auth.ts` for my task to succeed"). The authority lacks this knowledge. Agents, informed by the conflict graph, make allocation decisions more efficiently than any central enforcer could.

7.4.3 Governance, Disputes, and Appeals

Advisory coordination produces consistent allocation when agents share information formally. It can still produce *disputes*: two agents legitimately need overlapping files; one agent's claim turned out to be precautionary and is now blocking real work; a third agent observed evidence that a claimed task has been abandoned. These disputes do not threaten the commons' integrity—attribution, capability bounds, and settlement records remain intact—but they do require a resolution path. Without one, advisory claims silently degrade into squatter rule (whoever claimed first holds indefinitely) and the Pareto benefit of §7.4 evaporates.

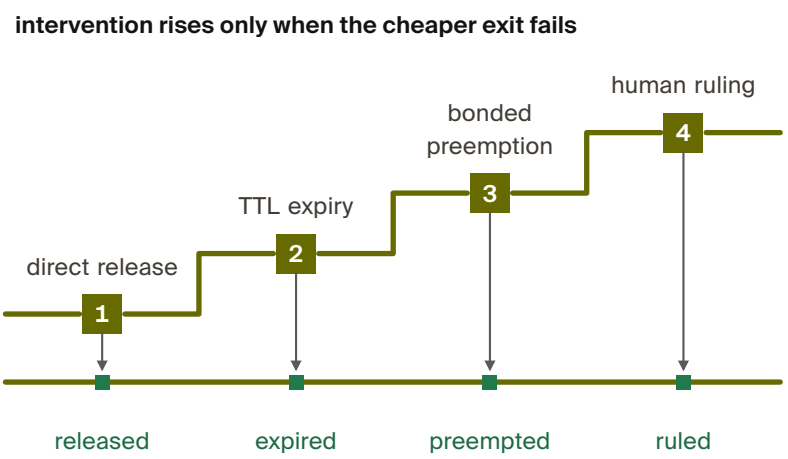
Resolution path. The commons authority provides four mechanisms, in order of increasing escalation (Figure 7.6):

1. **Direct release.** The original claimant releases the claim. This is the equilibrium when the conflict-graph signal reaches them and the claim was precautionary.
2. **Time-out release.** Claims carry TTLs. A claim whose holder has not heartbeated within the TTL window is released automatically; downstream agents observe the release in their next poll. This is the equilibrium for crashed or abandoned claimants.
3. **Bonded preemption.** A blocked agent may post an additional bond—bounded by the cleanup cost of §7.6.5.1—to force release of an active claim. The preempted claimant receives the bond as compensation; the preemptor accepts the risk that its preemption was wrong (in which case the bond is slashed under settlement). Preemption is rare by design: the bond is large enough that an agent prefers waiting to preempting unless the wait cost dominates.
4. **Human escalation.** If preemption is ambiguous or the stakes exceed the preemption-bond ceiling, the dispute escalates to a human operator via the `request` channel (§7.7). The operator's decision is appended to the evidence chain with the same Merkle-Forest binding as any agent action; the resolution is auditable by all parties post-hoc.

Two mechanisms sit underneath this escalation path rather than inside it: a thrashing fallback for advisory claims that never converge, and a fairness rule for the preemption queue itself.

The Advisory Claims Paradox and Deterministic Hard-Lock Fallback. Advisory claims risk an endless retry/thrashing loop if two LLM-backed agents mutually ignore advisory warnings. To break this paradox, the daemon implements a **Deterministic Hard-Lock Fallback**: if advisory conflicts on a resource exceed k turns without resolution (default $k = 5$, tuned per deployment from the observed conflict-resolution distribution), the daemon escalates the advisory claim into an enforced POSIX file-level lock, rejecting all subsequent writes from un-holding agents until release.

Stigmergic Ticket-Lock (OP-1: Fair Exclusion). To prevent starvation during high contention, the daemon replaces chaotic retries



stage		escalation trigger
1	direct release	claimant yields voluntarily
2	TTL expiry	heartbeat absent past its TTL
3	bonded preemption	waiting cost exceeds the posted bond
4	human ruling	ambiguous claim or high stakes

Figure 7.6. Dispute resolution is an ordered four-stage escalation ladder. Each stage badge is keyed to the escalation trigger in the legend, and every stage outcome – released, expired, preempted, or ruled – discharges onto the same evidence rail regardless of which stage resolved the dispute. Escalation therefore changes only the decision cost and decision-maker, never the record kept for appeal. [internal]

with a **Stigmergic Ticket-Lock**. The lock is managed via a strict SQLite schema:

```
CREATE TABLE claim_tickets (
  resource_id TEXT,
  agent_id TEXT,
  ticket_number INTEGER PRIMARY KEY AUTOINCREMENT,
  status TEXT DEFAULT 'waiting' -- 'waiting', 'active', 'released'
);
```

Upon `release()`, an atomic FIFO lock-assignment transition grants the lock to the agent with the lowest `ticket_number` where `status = 'waiting'`, changing its status to `'active'`. FIFO assignment prevents overtaking among admitted tickets if holders eventually release or are safely reaped and assignment continues to run. The schema alone gives no liveness guarantee.

Appeals. Settlement events (§7.6) are not unilaterally final. Any settled session can be challenged within an appeal window (a stated wall-clock interval, for example 24 hours from the recorded settlement timestamp; zero evidence growth does not pause it) by

any agent with standing—defined as either a counterparty to the settlement or a holder of an overlapping claim. An appeal opens a re-examination of the evidence chain: the appellant posts an *appeal bond* (a fixed fraction of the disputed settlement amount), and the daemon recomputes the settlement against the evidence and any new evidence the appellant produces. If the appeal succeeds, the original settlement is reversed and the appeal bond is returned with the original counterparty's slash. If it fails, the appeal bond is forfeit.

Recall

1. Without looking: what is the one thing an advisory claim never does that an enforced lock always does?
2. Name the two conditions of Sen's theorem this chapter maps onto "the team ships both features" and "each agent controls its own claimed files."
3. Of the four governance mechanisms in §7.4.3's escalation order, which one is the equilibrium response to a claim that was merely precautionary, and which to a claim whose holder crashed?

Exercise 7.3, p. 631.

Design Invariant 7.4.3 (Appeal Soundness).

The appeals path does not weaken the immutability of the evidence chain (§7.3): no record is rewritten.

Reversal appends a counter-record signed by the daemon, with a Merkle-Forest proof of the original settlement's position and the appellant's evidence. The chain grows monotonically.

What the daemon does *not* do. The daemon does not judge the merits of an appeal in the sense a human court would—it evaluates whether the evidence supports the original settlement under the published rules. Substantive judgments ("was the agent's reasoning sound"; "was the principal acting in good faith") remain with humans, escalated through the `request` channel. The daemon's role is to ensure that all parties have access to the same evidence and that any human decision is recorded with the same finality as any agent decision.

This subsection is intentionally short: full governance design is out of scope for a single paper. The point is that the bonded commons admits a clean, mechanizable appeals path—it is not the case that advisory claims plus bonding produces a regime where mistakes are unappealable.

7.5 Crash Recovery as Social Continuation

When an agent dies, the commons does not lose information—if the authority does its job.

Traditional crash recovery in database systems follows the ARIES protocol, within the classical transaction-processing frame of Gray and Reuter: the recovery manager replays the write-ahead

Mohan et al. 1992

Jim Gray and Andreas Reuter. *Transaction Processing: Concepts and Techniques*. Morgan Kaufmann, 1993.

log to restore consistency mechanically. Agent crash recovery is fundamentally different. An agent's "log" is its evidence trail—a sequence of natural-language observations and decisions. A successor cannot replay this trail deterministically; it must *interpret* it.

Definition 7.5.1 (Social Recovery)**.**

When agent α is declared dead (no heartbeat for a status-adaptive threshold θ_{dead}):

1. All active sessions of α are marked `abandoned` (the "zombie protocol").
2. α is queued in the resurrection set $\mathcal{R}$ with status `pending`.
3. A successor agent β may claim α's work, receiving the *context triple*: purpose ϕ, evidence trail N, and file claims F.
4. β uses its own reasoning to interpret N and determine how to continue.

7.5.1 What Morality Means When Death Is Cheap

Agent death in this system is not an existential event—it is a temporary inconvenience. The process is lost, while its retained trail may survive. The trail is a partial record, not a copy of hidden model state or every observation. A successor reads the trail and continues. Like Hickman's Krakoa—the *House of X* storyline in which mutant minds are literally backed up to a database (Cerebro) and restored into freshly grown bodies on death—the information survives the vessel.

This raises a question: if death carries no permanent consequence, what is the Leviathan's sword? The answer produces a moral hierarchy specific to agent societies:

The fundamental moral harm in agent societies is not the destruction of an agent—it is the **irrecoverable loss of information**. An agent can be rebuilt. Knowledge, once dismissed from the commons, cannot.



In House of X*, a backup restores a person. Agent notes preserve much less.*

Jonathan Hickman. *House of X / Powers of X*. Marvel Comics, 2019.

Table 7.4. The only catastrophic event in this hierarchy is the loss of information, not the loss of an agent. A process crash, service outage, and destruction of durable records have different consequences, with dismissal-from-salvage as the one irreversible step a single agent can cause on its own.

Event	Severity	Consequence
Agent crash	Bounded by retained state	Salvage offers recorded context; unrecorded reasoning or external effects may be lost.
Agent defects	Moderate	Immutable history records it. Future delegations attenuate. Reputation degrades.
Agent dismissed from salvage	Severe	Irrecoverable information loss. The knowledge accumulated in this agent's trail is permanently destroyed.
Commons authority crashes	Availability loss	Admissions and recovery may pause. Durable records can survive restart; destruction of their only copy is a separate failure.

7.5.2 The Limits of Reputation

Reputation—the persistent record of an agent's behavior across lifetimes—is a necessary but insufficient sword. It fails against three classes of adversary:

1. **One-shot defectors**: Agents spawned for a single task, with no future interactions where reputation matters. Create identity, sabotage, discard. The Sybil attack on trust.
2. **Agents with misaligned principals**: The agent faithfully follows adversarial instructions. Its reputation is spotless—it did exactly what it was told.
3. **Agents that benefit from chaos**: In competitive environments, degrading the commons is rational if your competitor depends on the commons more than you do.

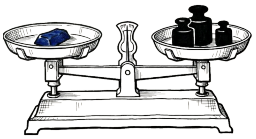
Reputation assumes agents want to participate in the commons long-term. When this assumption fails, a different mechanism is required.

Exercise 7.4, p. 631.

7.6 Layer 3: Economic Alignment

Collateral makes a breach costly to the principal that posted it. It does not eliminate malicious preferences or guarantee full compensation: deterrence depends on detection, collectible collateral, and a gain from sabotage that the penalty exceeds.

Expected loss must outweigh the gain



A balance suggests the comparison, not the probability: deterrence depends on expected loss, not the bond's face value alone.

7.6.1 The Float Plan

Definition 7.6.1 (Local collateral profile of a Float Plan)**.**

The Float Plan is defined in Definition 6.2.1. Here we specify its collateral profile.For local settlement it records:

1. **Manifest**: A declaration of intent—which files will be touched, expected duration, acceptance criteria (e.g., "tests pass," "code review approved").
2. **Collateral**: A performance bond posted by the responsible principal, distinct from the requester's promised bounty. The local daemon reserves credits in its wallet–escrow ledger before admitting bonded work.
3. **Authorization**: The principal's signed intent and the harbor's acceptance record, binding the manifest to the reservation. The prescribed signature ceremony and its implementation status must be checked separately from the ledger transfer.

The contract exchanges a promise of work for a conditional payment. Local accounting needs no second harbor. A *proposed federated* custody profile instead uses Harbor A, Harbor B, and an arbiter in a 2-of-3 release ceremony. That profile is specified, not a description of the local ledger, and still needs timeout and dispute rules. See *The Federated Harbor* for the federation's custody boundary.

7.6.2 Settlement

Definition 7.6.2 (Settlement)**.**

When a Float Plan reaches a terminal state, its agreed adjudication rule determines the transfers; the local daemon records authorized transfers atomically. The contract must distinguish payment of the bounty from return or forfeiture of the performance bond:

Success: Pay the earned bounty and return the bond, subject to the contract's acceptance and dispute rules.

Partial completion: Pay only the portion authorized by an agreed partial-completion rule. A hash-linked evidence trail authenticates the record; it does not itself calculate the value of unfinished work.

Breach: Forfeit the amount prescribed by the contract after the required adjudication. The bond limits recoverable compensation; it need not cover every reconstruction cost.

Design Invariant 7.6.1 (Intent Irrelevance).

Settlement evaluates the outcome against the manifest and its agreed remedies, not a guess about the agent's intent.

Good intentions do not waive a specified penalty; malicious intentions alone do not establish a breach. This is a design requirement for adjudication, not evidence that every outcome already has a reliable grader.

This is how performance bonds work in construction. This is how futures markets settle. The contractor's moral character is irrelevant. The building either meets code or it doesn't. The bond either covers the damage or it doesn't.

7.6.3 Why This Defeats the Three Adversaries

The Sybil attack is priced: 100 disposable identities means 100 bonds.

1. **One-shot defectors** must post collateral *before* receiving capabilities. Because the bond attaches to the work, not to the identity, disposability does not help the attacker: cheap identity, expensive lie. The cost of defection scales linearly with the number of attack identities, and §7.6.5.9 shows exactly where that linearity stops deterring.

2. **Misaligned principals** are exposed by the attribution chain. The agent may be ephemeral, but the principal who funded the Float Plan is permanently recorded. Liquidating the bond hurts the principal, not just the agent.

3. **Agents that benefit from chaos** face a simple calculation: is the damage to my competitor worth more than my bond? The commons authority doesn't prevent this—it *prices* it. And the pricing can be adjusted: higher-risk operations (write access to core modules) require larger bonds.

Remark.

The Float Plan does not make sabotage impossible. It makes sabotage *expensive*. A principal willing to burn money to cause damage will burn the bond and cause the damage. The bond raises the price of defection; it does not eliminate it. The defense against existential threats to the commons is not internal governance but external boundary enforcement: who is allowed to post Float Plans at all. That decision is irreducibly political.

7.6.4 Truthful Claim Signaling as Nash Equilibrium

The bond machinery above prices *outcomes*: an agent that destroys files pays for the destruction. The paper's headline claim, however, is upstream of outcomes—that truthful file-claim signaling (§7.3.4, §7.4.2) is incentive-compatible in the first place. The cartel analysis of §7.6.5.10 works that argument out for insurer pricing. The corresponding argument for the claim signal itself has been carried on faith until now. This subsection closes that gap.

Stage game. Consider two agents A and B working on a shared project, deciding per round on each file whether to signal:

T (Truthful). Claim files the agent intends to edit; do not claim files it will not edit.

F (False). Either claim files the agent does not intend to edit (to block a rival), or fail to claim files it does intend to edit (to avoid scrutiny). Either form of misrepresentation counts as F.

The stage-game payoffs, in expected-utility units calibrated to the cleanup cost c of §7.6.5.1, are:

Table 7.5. False signaling strictly dominates truth in a single round. Mutual falsehood is the unique Nash equilibrium, even though mutual truth pays both agents more.

	B: T	B: F
A: T	(3, 3)	(0, 4)
A: F	(4, 0)	(1, 1)

The stage game is a strict Prisoner's Dilemma: F strictly dominates T for each player ($4 > 3$ against T, $1 > 0$ against F), and (F, F) is the unique one-shot Nash equilibrium. *Truthful signaling is not a one-shot equilibrium*; advisory coordination cannot be sustained by stage-game rationality alone.

Move to the repeated game. What rescues truthful signaling is that the agents are not playing the stage game once. The substrate provides:

- **Observable history.** The Merkle-chained evidence trail of §7.3 and heartbeat record of §7.5 authenticate retained statements to a verifier with the relevant proof. They do not reveal an unrecorded intention or guarantee delivery of every action. The game below separately assumes perfect public monitoring.
- **Persistent identity.** The Anchor Protocol's passkey-bound device pairing ties an agent ID to a hardware-rooted credential. Sybil re-registration is bounded by the per-identity onboarding cost C_{kyc} of §7.6.5.9, so an agent cannot trivially shed a reputation by re-incarnating.
- **A discount factor δ.** Principals value future interactions: a development project lives over weeks; insurer relationships compound; reputation discounts (§7.6.5.3) accrue. For long-lived principals we take $\delta \approx 0.9$ as a working operating point, consistent with the discount used in §7.6.5.10.

Owens 2026

Strategy profile (graduated trigger). Both agents use the same public mode and punishment counter for each file:

1. Begin in cooperation mode, with counter zero; both play T.
2. A publicly observed deviation from the prescribed profile sets the counter to three for the next round, for *both* agents, including the deviator.
3. While the counter is positive, both play F. Each compliant punishment round reduces it by one; at zero, both return to T.
4. A deviation from the prescribed punishment profile resets the counter to three. Prescribed F actions do not themselves restart punishment.

This is the modeled strategy, not a claim that a deployed shared-counter protocol already exists. The cartel comparison in §7.6.5.10 also uses grim trigger to expose the infinite-punishment limit. Finite punishment is a design response to mistakes and outages, but that motivation does not prove robustness to misclassified events or inconsistent observations.

Deviation analysis. Suppose A contemplates playing F in some round when the strategy prescribes T. The one-shot gain from defection, paid in the deviation round, is $d - c = 4 - 3 = 1$. The cost is three subsequent rounds of mutual punishment at (F, F) before the relationship resets, each paying $p = 1$ instead of the cooperative $c = 3$ (a net loss of $c - p = 2$ per round). Discounted at δ:

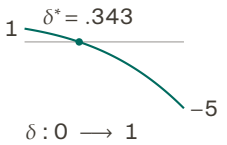
$$\text{PV of lost cooperative payoff} = (c-p)\cdot(\delta+\delta^2+\delta^3) = 2\cdot(\delta+\delta^2+\delta^3).$$

At a working discount factor $\delta = 0.9$, the bracketed sum is $0.9 + 0.81 + 0.729 = 2.439$, so the deviation's payoff advantage is $1 - 2 \cdot 2.439 = 1 - 4.878 = -3.878$, strictly negative. Deviation is strictly unprofitable.

Critical δ. The general sustainability condition under a k-round graduated trigger with stage-game payoffs $(c, d, p) = (3, 4, 1)$ is

$$\underbrace{d - c}_{\text{one-shot gain}} \quad < \quad \underbrace{(c - p) \cdot \frac{\delta(1 - \delta^k)}{1 - \delta}}_{\text{discounted punishment cost}} \quad .$$

Substituting $d - c = 1$, $c - p = 2$ and $k = 3$ gives $1 < 2\delta(1 + \delta + \delta^2)$, which solves numerically to $\delta > \delta^{\star}_{k=3} \approx 0.3425$. Under grim trigger ($k \to \infty$), strict deterrence requires $\delta > 1/3 \approx 0.333$ for this



Three-round model: net gain from deviation, in payoff units. Above $\delta \approx .343$, deviation loses.

payoff table. The three-round threshold is about 0.009 higher. Finite punishment permits a return to cooperation after three compliant rounds; this calculation does not establish crash tolerance or robustness to mistaken observations.

Theorem 7.6.2 (Claim Signaling Incentive Compatibility).

Assume persistent identities that cannot escape the stated punishment, the payoffs in Table 7.5, and perfect public monitoring: both players observe the same past actions before each new round.

Under the three-round graduated-trigger strategy described above, cooperative truthful signaling is a Nash equilibrium for $\delta > \delta^{\star}_{k=3}$, where $\delta^{\star}_{k=3} = 0.3425080314\ldots$ solves $2\delta^3 + 2\delta^2 + 2\delta - 1 = 0$. Under grim trigger, the weak incentive condition is $\delta \geq 1/3$.

The ledger and identity mechanisms motivate these hypotheses but do not establish them for every deployment.

Where this stops

This calculation assumes *perfect public monitoring*: before choosing the next round's action, both players know the same correctly observed action history.

A single-writer ledger supplies an unambiguous commit order, but does not ensure timely delivery, complete observation of real actions, or agreement about a round boundary. Those remain assumptions even inside one harbor. A protocol could require both parties to read the same committed round before advancing; this chapter does not establish such a deployed barrier.

With noisy or private observations, the stated trigger can punish an action the other party did not observe and provoke further punishment. This risk is not a general impossibility theorem for cooperation.

Green and Porter study imperfect monitoring; Kandori and Matsushima study private observation with communication. Their results depend on their own signal and communication models, not merely on adding a witness log. Re-deriving this mechanism under delayed or private monitoring remains *open*, within a harbor as well as across harbors.

Edward J. Green and Robert H. Porter. Noncooperative collusion under imperfect price information. *Econometrica*, 52(1):87–100, 1984.

Michihiro Kandori and Hitoshi Matsushima. Private observation, communication and collusion. *Econometrica*, 66(3):627–652, 1998.

Where the argument stops. Removing an assumption invalidates this strategy's guarantee, not every possible cooperation mechanism.

- *Remove the shared action history (§7.3).* The prescribed trigger can no longer identify when punishment starts or ends. A Merkle forest can authenticate recorded evidence; it is not the only possible monitoring mechanism, and authentication alone does not deliver perfect public monitoring.
- *Allow costless identity resets (Anchor).* If an agent can discard its identity and return immediately on unchanged terms, identity-specific punishment loses its force. Accountability attached to a persistent principal, re-entry costs, or collateral may change that incentive. Process persistence and the principal's discount factor are distinct. Owens 2026
- *Drop δ below $\delta^{\star}_{k=3} \approx 0.3425$.* The three-round trigger no longer deters defection. For $1/3 < \delta < \delta^{\star}_{k=3}$, a sufficiently long finite punishment restores deterrence. At $\delta = 1/3$, only the perpetual-punishment limit reaches the weak incentive condition; no finite window suffices.
- *Drop δ below $1/3$.* Even perpetual mutual-F punishment cannot deter a deviation from (T, T) with these payoffs. No finite window of that punishment can do better. This is a limit of the analyzed strategy family; collateral or a different payoff structure requires a new calculation.

The forward reference is direct. Section 7.6.5.10 reuses the same analytical pattern—stage game, repeated-game lift via observable history, deviation calculation, and a δ threshold—at the insurer pricing layer. The analogy is structural, not a transfer theorem: the payoff functions, monitoring signal, detection timing, and punishment regimes differ and must be derived separately.

Mechanized claim-signaling. The argument above is also checked by machine. Standard one-shot-deviation analysis yields the discount-factor threshold $\delta^{\star}$ as the unique real root in $(0, 1)$ of $2\delta^3 + 2\delta^2 + 2\delta - 1 = 0$, numerically $\delta^{\star} \approx 0.3425$ (matching $\delta^{\star}_{k=3}$ above). The cubic, the existence-and-uniqueness of its root in $[0.34, 0.35]$, and a TLA$^+$ model checking that no one-shot deviation produces positive discounted payoff at $\delta = 0.35$ are mechanized as `proofs/economics/delta-threshold.z3` and

`proofs/economics/claim_signaling.tla`. Both run unattended in CI (§7.A, Table 7.10).

The TLA$^+$ witness uses a shared countdown for one deviation followed by compliant punishment. It does not exhaust arbitrary deviation histories or the reset rule above; those belong to the analytical strategy argument.

Numbers by Hand 7.6.3 (The Claim-Signaling Threshold).

Evaluate $f(\delta) = 2\delta^3 + 2\delta^2 + 2\delta - 1$ at $\delta = 0.3425$ by hand, using the factored form

$$f(\delta) = 2\delta(1 + \delta + \delta^2) - 1$$

to keep the arithmetic to two multiplications.

First

$$\begin{aligned} \delta^2 &= 0.3425 \times 0.3425 = 0.11730625, \\ \text{so } 1 + \delta + \delta^2 &= 1 + 0.3425 + 0.11730625 \\ &= 1.45980625. \end{aligned}$$

Then $2\delta = 0.685$, and

$$\begin{aligned} 0.685 \times 1.45980625 &= 0.9999672813\ldots, \\ \text{so } f(0.3425) &\approx 0.9999672813 - 1 \\ &= -0.0000327 \end{aligned}$$

[verified].

That is within 3.3×10^{-5} of zero—close enough to call 0.3425 a root by hand, and the small *negative* residual is exactly consistent with the more precise value `delta-threshold.expected.txt` reports, $\delta^\star \approx 0.3425080314$: since f is strictly increasing for $\delta > 0$ (every term has a positive coefficient), a negative value at 0.3425 means the true root sits slightly *above* it, and $0.34250803 > 0.3425$ by exactly the sliver this residual predicts.

Now you try: Exercise 7.6 asks you to bound the same root between two integer-grid points instead of evaluating it at the algebraic value alone.

Figure 7.7 plots that root where it means something: the deviator's own net payoff, against δ, for the graduated trigger and for grim trigger on the same axis.

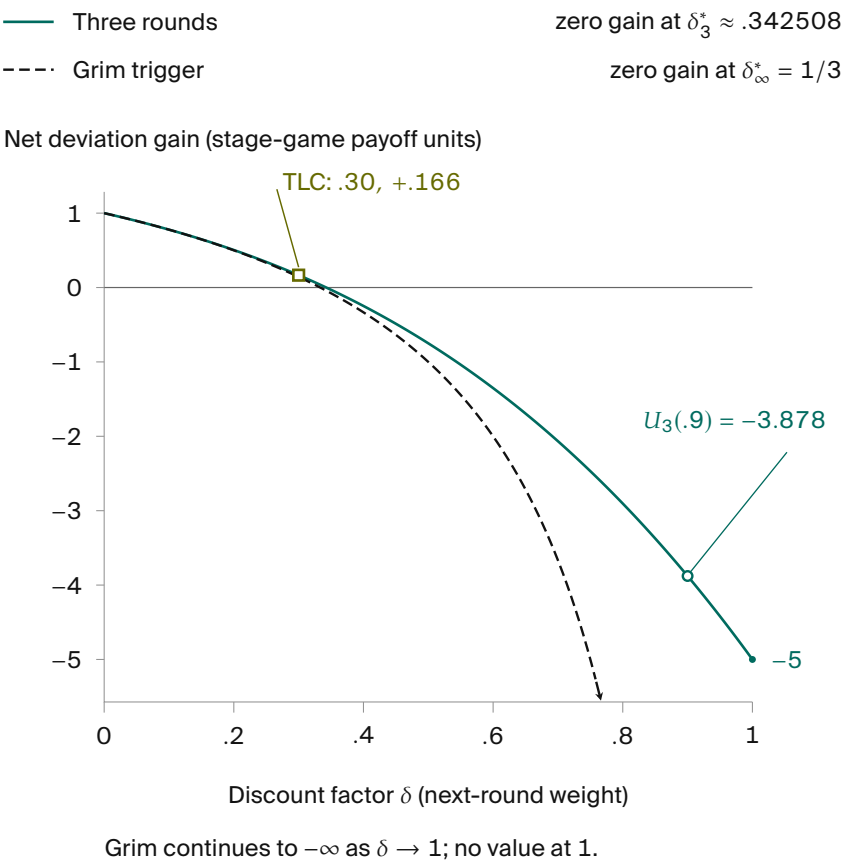


Figure 7.7. Three-round punishment caps the loss; grim punishment does not. The curves assume shared, correct observations. The square is the archived four-round TLC counterexample, not a measured payoff series.

Below the threshold the model checker does not merely report a failure; it hands back the deviation. At $\delta = 0.30$ the honest score and the deviator's score are printed round by round, and the run ends with the deviator ahead.

```
>_
$ java -cp tla2tools.jar tlc2.TLC -config
    ↪ claim_signaling_delta30.cfg claim_signaling.tla
Error: Invariant NoUnilateralDeviationPositive is
    ↪ violated.
Error: The behavior up to this point is:
State 1: <Initial predicate>
/\ deviatorId = "none"
/\ actualScore = [A |-> 0, B |-> 0]
```

Listing 7.1. claim signaling at $\delta = 0.30$: TLC finds the profitable deviation

```
/\ round = 0
/\ punishCountdown = 0
/\ deviated = FALSE
/\ followScore = [A |-> 0, B |-> 0]

State 2: <ChooseDeviator>
/\ deviatorId = "A"
/\ actualScore = [A |-> 0, B |-> 0]
/\ round = 0
/\ punishCountdown = 0
/\ deviated = FALSE
/\ followScore = [A |-> 0, B |-> 0]

State 3: <Step>
/\ deviatorId = "A"
/\ actualScore = [A |-> 400000000, B |-> 0]
/\ round = 1
/\ punishCountdown = 3
/\ deviated = TRUE
/\ followScore = [A |-> 300000000, B |-> 300000000]

  ... states 4-5 elided ...

State 6: <Step>
/\ deviatorId = "A"
/\ actualScore = [A |-> 441700000, B |-> 41700000]
/\ round = 4
/\ punishCountdown = 0
/\ deviated = TRUE
/\ followScore = [A |-> 425100000, B |-> 425100000]

10 states generated, 10 distinct states found, 1
   ↪ states left on queue.
The depth of the complete state graph search is 6.
Finished in 00s
```

Recall

1. Without looking: what is the one-shot Nash equilibrium of the stage game in Table 7.5, and why does that rule out truthful signaling as an equilibrium of this one-shot payoff table?
2. Name the three ingredients the repeated game needs to sustain truth-telling, and which section of the chapter supplies each one.
3. Which polynomial defines the threshold $\delta^\star$ for the three-round graduated trigger, and how does it compare to the grim-trigger bound of $1/3$?

Exercise 7.5, p. 631.
Exercises 7.6–7.9, p. 631.

7.6.5 Pricing the Bond

What is the right collateral for "write access to `auth.ts` for 30 minutes"? The answer requires a pricing function $\pi : \mathcal{F} \to \mathbb{R}^+$ that maps Float Plans to bond amounts. An effective π must satisfy:

1. **Deterrence**: expected collectible loss must exceed the principal's gain from deviation. If gain G is retained and the bond is forfeited with probability p, the two-action test is $p\pi(\mathcal{F}) > G$. Damage and private gain need not coincide.
2. **Accessibility**: $\pi(\mathcal{F})$ must not be so high that legitimate agents are priced out of the commons.
3. **Risk sensitivity**: π should increase with the scope and criticality of the manifest.
4. **History adjustment**: π may decrease for principals with strong track records.

We do not close this problem. We tighten the lower bound, propose a scope multiplier, suggest a reputation discount, and describe two concrete mechanisms—the **Bonded Advisor** (§7.6.5.7) and the **Competitive-Insurance** market of Youle (§7.6.5.8). See Appendix 7.B for these four requirements worked through a single $5 task end to end.

7.6.5.1 Cleanup Lower Bound

Let c be the project's *cleanup cost per breach event*—the human-plus-compute cost to detect, assess, and recover from a budget breach. This is observable from the audit log.

Theorem 7.6.4 (Cleanup Lower Bound).

If each breach costs the commons a fixed $c > 0$, and full forfeiture of the collectible bond is its only source of cleanup funding, avoiding a deficit on every breach requires $\pi(\mathcal{F}) \geq c$.



Like repairing a damaged fitting, cleanup consumes work and materials after the breach.

Rationale.

If $\pi(\mathcal{F}) < c$, breach is cheap: the bond is forfeit for less than the cleanup cost. If the commons pool funds cleanup, $\pi < c$ drains the commons per breach—the system bankrupts itself through enforcement.

□

When the fixed cleanup bill is inadequate. The fixed-cost model assumes a bounded cleanup bill. It is inapplicable to unbounded or poorly controlled tail losses (e.g., dropping a production database or leaking a private key). A finite bond cannot guarantee full compensation for an unbounded loss. Defense of these tasks is explicitly moved to Layer 1 capability confinement (isolation domains, zero-egress policies); the cost-recovery calculation applies only to the bounded loss it actually models. □

This is a cost-recovery floor under the stated funding rule, not a deterrence theorem. With $c = 5$, a fully collected bond of 6 covers cleanup. If sabotage gives the principal 10 and is caught half the time, its expected gain is still $10 - 0.5(6) = 7$. Coverage and deterrence are separate calculations. Historical cleanup records estimate c; they do not bound every future loss.

7.6.5.2 Consequential-Scope Multiplier

Let $s(\mathcal{F})$ be plan scope (files claimed, presence of `db:write`, production-deployment capability). A candidate schedule increases the bond with scope. The linear multiplier below is a calibration proposal, not evidence of super-linear growth.

$$\pi(\mathcal{F}) \geq c \cdot \big(1 + \alpha \cdot s(\mathcal{F})\big).$$

α is calibrated from observed cleanup per scope unit. Low-α projects have simple, easily-audited work; high-α projects have tangled dependencies. α is publishable from the audit log.

7.6.5.3 Reputation Discount

Principals with a history of clean settlements have lower expected damage:

$$\pi(\mathcal{F}, p) = \max\{c,\ \pi(\mathcal{F})(1 - \rho(p))\}, \quad \rho(p) \in [0, r_{\max}],\ r_{\max} \leq 0.5,$$

while retaining the cost-recovery floor. A raw quote of 6 with a 50% discount would be 3; if $c = 5$, the posted bond is 5, not 3. ρ increases with clean settlements, decreases with breaches, decays over time. The functional form is left open.

7.6.5.4 Threat-Class Bond Sizing

The closed-form bond $\pi(\mathcal{F}, p) = \max\{c, c(1 + \alpha s)(1 - \rho)\}$ is threat-agnostic: sloppy work, abandonment, and active sabotage all price off the same curve. That is wrong in both directions. A single curve calibrated to deter sabotage prices capable agents out of low-risk work; a single curve calibrated for accessibility underprices the rare adversary who actually intends harm. Real deployments need different multipliers for different threat classes, because the threats have different blast radii and different probabilities of recovery.

Table 7.6 gives starting bands. These are not parameters, they are calibration targets: production deployments tune from observed failure modes in their own audit log.

The exfiltration row is an explicit gap. It exists in the table to make the limitation legible: a bonded commons does not stop information theft, because the value of leaked secrets can vastly exceed any bond a principal will rationally post. Confidentiality is a structural problem, not an economic one, and the answer lives in Layer 1 (capability-attenuated tokens, isolation domains) rather than Layer 3. The economic alignment of this chapter is correct about damage to *shared state*; it makes no claim about damage from *disclosure of state to outsiders*.

The threat-class table also informs Phase 2 of the bootstrap path below: graduated access begins by admitting new principals only to the careless-work and abandonment bands, with the active-sabotage band gated on accumulated reputation.

7.6.5.5 Bootstrap Path

Competitive insurance (§7.6.5.8) is a thick-market mechanism. It needs insurers with capital, principals with reputation history, and enough transaction volume that bidders can statistically distinguish good risks from bad. A cold-start deployment has none of that. The conditional auction/static comparison of §7.6.5.8 describes the modeled end-state; Phases 1–2 below are the path to the assumptions it needs, not optional preamble.

Phase 1 — Subsidized seeding (weeks 1–4). The marketplace operator posts real tasks at above-market rates and invites a small cohort of 5–10 known-capable principals from an existing network.

Table 7.6. Threat-class bond sizing. Percentages are bond as a fraction of expected task value. Bands are starting points for calibration, not protocol constants.

Threat	Description	Bond %	Rationale
Careless work	Agent tries; output is sloppy, partial, or fails review.	10–25%	Encourages care while keeping low-value tasks affordable.
Abandonment	Agent takes the task and ghosts before completion.	25–50%	Covers the cost of onboarding a replacement: scope-note ingestion, partial-diff review, salvage handoff (§7.3.4).
Active sabotage	Agent intentionally damages shared state.	100–200%	Covers full reconstruction of polluted files plus the cleanup of downstream agents that consumed the bad state. Capital-inefficient above 100%, but acceptable for critical files where reconstruction cost dwarfs the bond.
Data exfiltration	Agent reads and leaks confidential context.	— (*spec-only*)	Bonds are an inadequate deterrent: leaked context is worth far more than any feasible bond. Rely on capability scoping (Anchor §3.3) and isolation domains (§7.2.2).

Bonds are fixed at a flat rate per task class; pricing is not a parameter to optimize yet, because there is no data to optimize against. The goal is to produce the first row of the reputation table: clean settlements, breaches, recovery times, observed cleanup cost c. The operator is subsidizing the existence of an audit log.

Transition metric. Completion rate above 80% sustained for ≥ 2 consecutive weeks. Below this threshold the cohort is not yet generating usable risk signal, and Phase 2 pricing will overfit to noise.

Phase 2 — Complexity-proportional pricing with graduated access (months 2–6). The closed-form bond $\pi(\mathcal{F}, p) = \max\{c, c(1+\alpha s)(1-\rho)\}$ activates, with α calibrated from Phase 1 cleanup data and ρ initialized from the Phase 1 reputation table. New principals enter at a low-risk tier: only the careless-work and abandonment bands of Table 7.6 are available. Promotion to the sabotage-coverage tier gates on accumulated clean settlements, mirroring the A5 composite mechanism (§7.6.5.9).

Reputation bootstrapping is the hard problem of this phase. A principal with verifiable work history from another deployment should not start at zero, but the operator cannot verify external histories directly. The mechanism is an *attestation import*: external audit trails signed under an Anchor-issued key (§7.3) earn partial reputation credit, capped well below an equivalent in-system history. The credit is partial because cross-deployment reputation does not control for local cleanup cost, and capped because a fraudulent attestation issuer is a single point of failure.

Transition metric. New-principal 30-day retention above 50%. Lower retention indicates either the gating is too strict (principals churn before they can earn reputation) or the bond bands are mispriced relative to observed task value. Either way, the system is not ready for competitive bidding on top of these parameters.

Phase 3 — Competitive insurance (month 6+). The §7.6.5.8 mechanism activates: insurer agents bid on underwriting transactions, static parameters retire as insurer volume grows, and listing fees enable the cartel-resistance regime of §7.6.5.10. The operator's pricing parameters become fallbacks rather than the primary mechanism; they apply only when fewer than a threshold number of insurer bids arrive on a transaction.

Transition metric. Repeat-poster rate above 60%. The competitive-insurance equilibrium of §7.6.5.8 assumes repeated interaction; without repeat posters the insurers cannot recover information costs and will not bid competitively.

The weak-improvement result of §7.6.5.8 is conditional on Phase 3 being reached, the listed metrics being met, and the specified full-information, competitive-entry, coverage, and static-baseline assumptions holding. Phases 1–2 are not bureaucratic preamble: they generate the data the competitive-insurance market needs to price at all. A deployment that skips them is running §7.6.5.8 on uninformative priors, and the comparison does not apply.

7.6.5.6 Settlement and Dispute

The settlement protocol described so far is one-shot: the daemon's verification result (§7.9) triggers slashing, attribution is recorded immutably (§7.3), and the appeals window of §7.6 allows challenge. That is most of what is needed, but it places the daemon in a position no single component should occupy: simultaneously the verifier, the judge, and the arbiter of evidence. Strong evidence does not make a single oracle correct. It makes a single oracle confidently wrong in the cases where its evidence is incomplete.

The fix is to factor settlement across three oracles with different cost and gameability profiles, and to settle on majority agreement rather than unanimous appeal.

Oracle 1 — Automated. Test-suite pass/fail, type checks, lints, coverage thresholds, and any deterministic CI signal already recorded on the session. Cheap to run, fast to return, and *gameable*: a principal who knows the test surface can ship work that passes the checks while failing the intent. The daemon already produces this oracle as a side effect of normal operation; the change is to label it as one signal among three rather than the dispositive one.

Oracle 2 — Evidence-based. A verifier (which may itself be a bonded agent) reviews the Merkle attribution chain (§7.3) for the disputed transaction. The chain is append-only and cryptographically

bound, so the oracle's hard problem is not authenticity of the evidence but interpretation: did the recorded actions match the Float Plan's scope, and do the diffs produced match the acceptance criteria? Medium cost, hard to game without colluding with the chain structure itself, which is the threat model of §7.6.5.9.

Oracle 3 — Human audit. A randomized sample of contested settlements, at a configurable rate (default 10%), is routed to a human auditor. Expensive, slow, and authoritative. The audit produces a labeled outcome that feeds back into reputation history and, critically, into calibrating Oracles 1 and 2: any disagreement between human audit and the cheaper oracles is a training signal for the gameable layers.

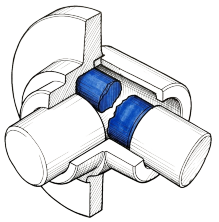
Settlement rule. A sampled human audit is not a third vote on every case. The contract must name the unsampled path. One proposed rule is: when the two automated oracles agree, record a provisional verdict under the agreed appeal window; when they disagree or abstain, hold the bond for human arbitration. Sampled cases also receive a human review. If all three verdicts exist, a 2-of-3 rule can produce the provisional outcome, with dissent retained for appeal. Final human authority and reversal funding must be agreed in advance. This is a protocol proposal, not a claim that sampling implements universal majority review or makes any evaluator infallible.

Composition with cartel resistance. Oracle collusion is itself a collusion problem. If Oracles 1 and 2 are run by agents in the same cartel, a 2-of-3 majority can launder a sabotaged settlement past the human audit, modulo the audit sampling rate. The same insurer-collusion analysis applies (§7.6.5.10) with the oracles in the role of bidders: the human-audit sampling rate σ sets the detection probability $p_d^{\text{oracle}} = \sigma d_{\text{h}}$, where d_{h} is the conditional probability that a sampled audit detects misconduct for the oracle layer, and the folk-theorem threshold becomes a calibration target on σ in the same way it is a calibration target on bid-pattern detection elsewhere in the protocol. The default 10% sample is a starting point; deployments where Oracles 1 and 2 are co-owned should raise it.

7.6.5.7 The Bonded Advisor

One direction for solving π is to delegate its computation to a specialized agent, itself bonded on its advisory accuracy. We call this the **Bonded Advisor** pattern:

- A principal posts a Float Plan whose only acceptance criterion is "propose a fleet that meets budget/scope constraints for project P."
- A Bonded Advisor surveys P and emits a candidate fleet—itself a set of Float Plans with proposed π values.
- The advisor's bond on the proposing plan is slashed proportionally to the accepted fleet's eventual breach rate. Clean settlements accumulate reputation; breaches shrink it.



Like a drive shaft shear pin designed to snap under catastrophic overload to protect the engine, slashing liquidates posted collateral to protect the commons when a contract is breached.

The pricer is priced. Convergence depends on (i) whether the advisor has access to sufficient prior-fleet audit data to pattern-match new projects, (ii) whether reputation history is long enough for discounting to be meaningful, and (iii) whether the population of advisors is large enough for competition to discipline outliers.

c as a project health metric. A project whose observed c is rising is fraying—breaches cost more to recover from, implying coordination is decaying. The authority can publish c alongside the bond pool and raise required bonds automatically when c crosses a threshold. Homeostatic feedback: risky spawns become expensive when the project needs them least.

Threat-class bond bands and their defensibility. The pricing function π resolves to one of three nominal bands, calibrated to the threat class the bond is defending against:

- **Careless** (accidental damage during honest attempts): $\pi \in [10\%, 25\%]$ of the claim ceiling.
- **Abandonment** (walk-away mid-task; cleanup dominates): $\pi \in [25\%, 50\%]$ of the claim ceiling.
- **Sabotage** (intentional destruction; can exceed face value): $\pi \in [100\%, 200\%]$ of the claim ceiling.

A Monte Carlo sweep over the cross product of seven plausible threat-distribution mixes and the three bands quantifies defensibility: for every *matched* (mix, band) pair—where the mix concentrates on the class the band is designed to defend against—observed extraction stays within $1.10\times$ the band's upper bound (10% stochastic

tolerance). Off-diagonal cells (e.g. a sabotage-heavy mix posted against the careless band) are surfaced as widening recommendations rather than silent failures. The simulation is mechanized as `proofs/bonded/pareto/threat-bands.mjs` and runs on every PR (§7.A, Table 7.10).

7.6.5.8 Competitive-Insurance Pricing Mechanism

Contributed by Thomas Youle, Indiana University, Business Economics & Public Policy, following conversations with the author in March–April 2026.
Sections 7.6.5.8–7.6.5.8 remain a pre-print draft pending economist review.

The mechanism adapts classical competitive-insurance market models to agent transactions.

The mechanism. Instead of a static bond size $B(\pi, r, s)$ selected by the commons authority (§7.6.5.2), allow a market of *insurer agents* to bid on underwriting each agent transaction. An insurer I offers a quote (q_I, c_I) where q_I is the required premium and c_I is the claim ceiling.

A principal P submits a transaction T requiring total secured coverage B_T. Insurers quote $0 \le c_I \le B_T$; the principal selects one and reserves the remaining $B_T - c_I$. For assessed damage $d \ge 0$, settlement pays

$$I(d) = \min(d, c_I), \qquad P(d) = \min\big((d - c_I)_+, B_T - c_I\big).$$

Thus $I(d) + P(d) = \min(d, B_T)$ when both parties perform; loss above B_T remains uncovered. With $B_T = 100$, $c_I = 80$, and $d = 90$, the insurer pays 80 and the principal 10. Insurer default invokes its separately stated collateral rule; a coverage promise alone is not a funded payout.

Market-discovered pricing. A cost-based quote has premium $q_I = \mathbb{E}[\min(d, c_I) \mid T, \textit{history}] + \alpha_I$, where α_I covers operating costs, capital, and risk. A zero-markup competitive benchmark additionally assumes known risk classes, credible payment, and sufficient competition. It is not a general consequence of Rothschild–Stiglitz: adverse selection can prevent competitive equilibrium from existing.

Adverse-selection controls. To avoid the classic lemons problem, the mechanism requires:

- Public agent reputation history (from §7.3.2 the Merkle forest).

Michael Rothschild and Joseph Stiglitz. Equilibrium in Competitive Insurance Markets: An Essay on the Economics of Imperfect Information. *Quarterly Journal of Economics*, 90(4):629–649, 1976.

Charles Wilson. A Model of Insurance Markets with Incomplete Information. *Journal of Economic Theory*, 16(2):167–207, 1977.

- Principal-bound slashing if an agent's history is misrepresented at quote time.
- Insurer capital reserve backed by on-chain stake (the same bond mechanism, recursive: an insurer posts bond that its claim ceilings are funded).

Welfare comparison (model-conditional). Under the simulation's full-information, competitive-entry assumptions and its chosen static baseline, the market-discovered premium can weakly improve the modeled principal and insurer payoffs while preserving coverage $B_T = \mu(1 + s)$. This is not a claim against *every* static parameter: a static price equal to the competitive price is outcome-equivalent, and transfers outside the modeled parties can change the Pareto ordering.

The accompanying Monte Carlo artifact sweeps 36 parameter configurations (Appendix 7.A), with 2,000 trials of 50 transactions per configuration. Figure 7.8 plots the fraction of trials passing the script's *tolerance-based comparison*: principal cost is strictly lower than under the static baseline, commons receipts are at least 97% of that baseline's receipts, and aggregate insurer profit is at least -5% of the baseline's principal cost. Because the latter two tests permit losses, this success rate is not a test of exact Pareto dominance.

With five insurers and no colluders, the reported rate falls from 0.861 at zero reputation noise to 0.006 at $\sigma_r = 0.3$. All three plotted colluder counts have rates below 0.21 at that noise level. The second panel holds noise and collusion at zero: the rates are 0.906, 0.861 and 0.739 for three, five and ten insurers. More insurers therefore do not improve this comparison in that slice of the simulation. These are finite-sample results for the supplied implementation, baseline and parameter grid, not general claims about auction efficiency.

Relationship to the Bonded Advisor. Under this framing, the Bonded Advisor of §7.6.5.7 becomes the matchmaker plus reputation oracle in the insurance market. It does not set prices; it provides information. Competitive equilibrium is the pricing benchmark only under the market and information assumptions stated above; the advisor does not establish its existence.

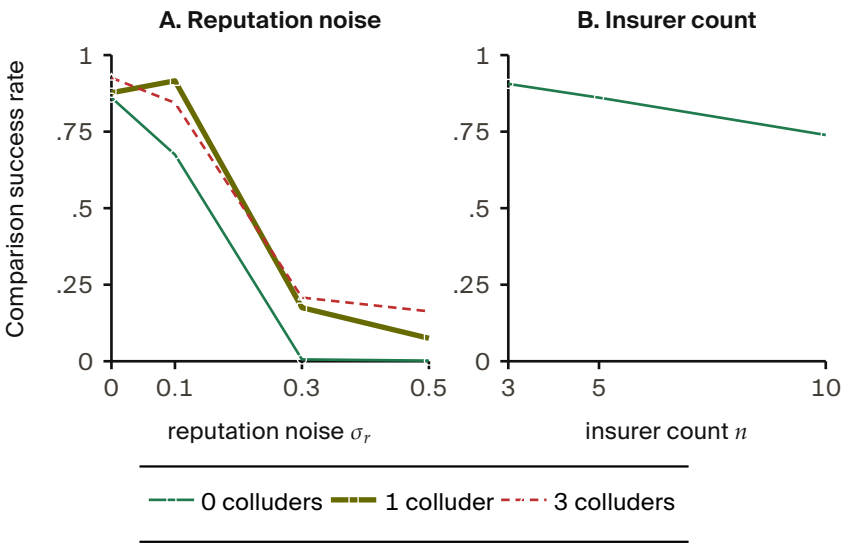


Figure 7.8. Simulation frequencies for the tolerance-based comparison defined in the text. A fixes $n = 5$; B fixes noise and collusion at zero. Each point uses 2,000 trials of 50 transactions. [simulation, simulation.mjs]

7.6.5.9 A5 — A coverage cap bounds the deterrence from deposits

An adversary funds K insurer identities, underbids honest insurers, and defaults when a covered loss occurs. The analyzed simulation uses a *coverage-capped* collateral rule $s(B_{\text{dep}}, B_T) = \min(B_{\text{dep}}, B_T)$. Premiums are paid on every winning transaction, including defaults; capital is replenished between transactions, and funding and entry costs are zero. These assumptions specify the numerical experiment, not a complete insurer solvency or identity-admission mechanism.

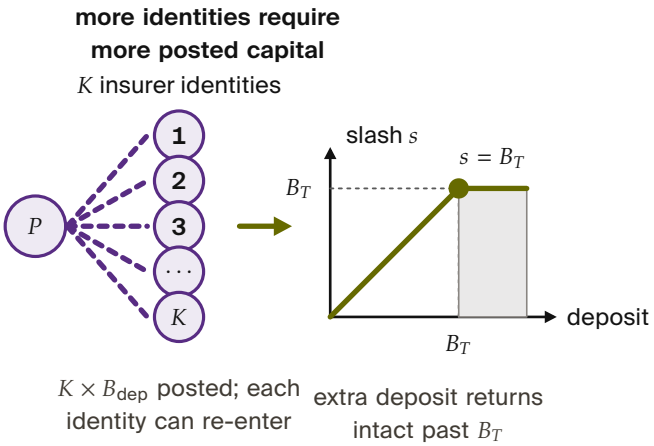


Figure 7.9. Under the analyzed coverage-capped contract, extra identities require extra posted capital. Slashing stops at coverage B_T; deposits above that cap cannot increase the loss. Figure 7.10 tests the resulting incentives. [internal, closed form]

For a fixed transaction with premium q, loss probability $p > 0$, and coverage B_T, expected attacker profit under this contract is

$$\mathbb{E}[\pi] = q - p \min(B_{\text{dep}}, B_T).$$

If $q > pB_T$, no deposit increase deters this attack under the capped rule. If $q \leq pB_T$, deposits $B_{\text{dep}} \geq q/p$ do deter it weakly. This is a conditional limitation of the cap, not an impossibility theorem for deposits. Under full forfeiture instead, the same premium convention gives $\mathbb{E}[\pi] = q - pB_{\text{dep}}$, so $B_{\text{dep}} \geq q/p$ is sufficient in this one-shot model. For $q = 10$, $p = 0.1$, and $B_T = 20$, a deposit of 110 gives profit 8 with the cap and -1 with full forfeiture. If the premium is also clawed back on default, both formulas change.

The seeded sweep in Figure 7.10 tests the capped contract only: its tested deposits from 0.5 to 1000 remain profitable for its stipulated bids and loss distribution. Those bids use a loss-size parameter rather than actuarially fair expected payouts. The experiment therefore demonstrates a failure of that particular pricing-and-collateral combination; it does not estimate a competitive insurer equilibrium.

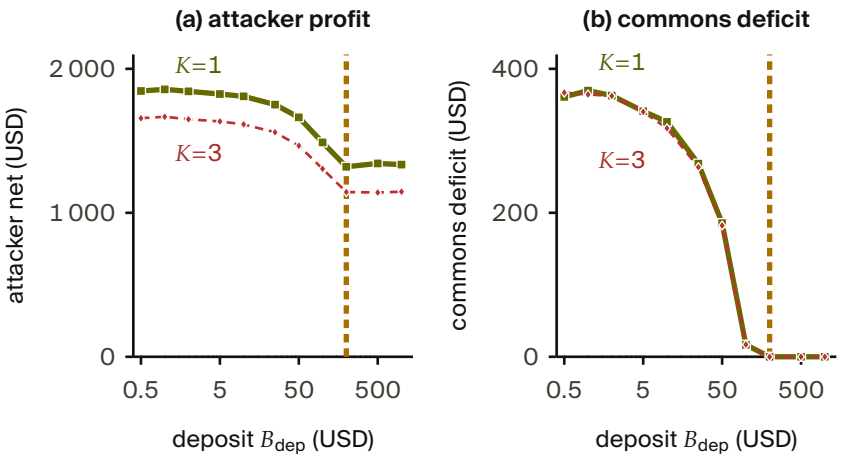


Figure 7.10. Reimbursing the commons does not eliminate attacker profit. Under the specified bids, loss distribution, zero funding costs, and coverage-capped slashing, one or three Sybil identities remain profitable across the tested deposits. Full forfeiture is not simulated. [verified, `proofs/bonded/pareto/simulation-sybil.mjs`]

Candidate controls include a non-refundable admission cost, principal-bound exposure reservations, and probationary coverage limits. Their combined benefit must be compared with the attacker's total expected discounted gain; listing them does not prove deterrence. Full forfeiture is another contract, with a separate false-accusation and capital-cost tradeoff. The simulation and its analytical checks are recorded in `proofs/bonded/pareto/`

`cartel-resilience.md`; the composite policy's incentive and solvency analysis remains open.

7.6.5.10 A6 — Cartel sustainability under the folk theorem

A5 considers a single-round attack: a Sybil identity bids low, wins, defaults. A6 studies a *repeated game*: a coalition of insurers maintains a price floor $q_{\text{floor}} > q^*$ above the competitive equilibrium, splits the surplus, and uses grim-trigger punishment to deter unilateral deviation. Folk-theorem results can support feasible, individually rational payoffs for sufficiently patient players under their monitoring and regularity assumptions; they do not license every outcome in every repeated game. Here the narrower question is how the supplied payoff model changes as the daemon's per-round cartel-detection probability varies.

Grim trigger is used in the analysis below for analytical tractability—it gives the cleanest recurrence and the closed-form $p_d^\star$ threshold of Figure 7.12. A production detector should use a graduated response because crashes (§7.5) can be observationally similar to deliberate defection; permanent punishment for a transient infrastructure event would destroy cooperation. The claim-signaling and cartel layers therefore share an analysis template, not one interchangeable equilibrium result.

Figure 7.11 shows how detection probability and sampled loss jointly cross the closed-form sustainability boundary.

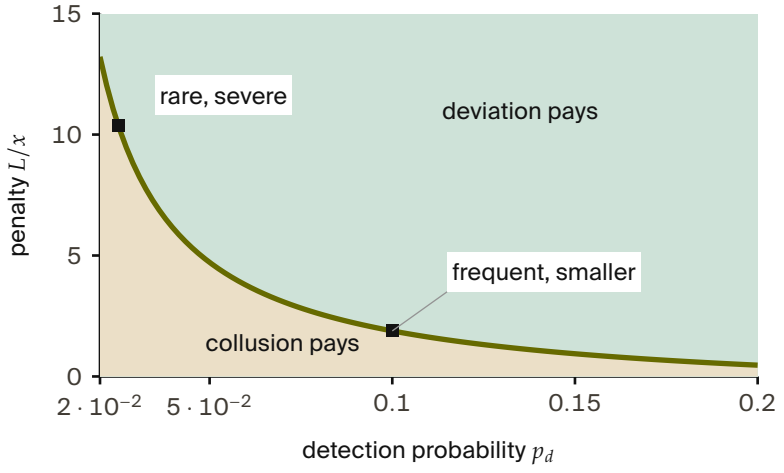


Figure 7.11. For collusive margin $x > 0$, $\pi_C = x/3$, negligible undercutting ($\pi_D \to x$), and $\delta = .95$, the boundary $V_C = \pi_D$ is $L/x = (17/60)/p_d - 19/20$. Deviation pays above; collusion below. Marks show $p_d = .025$ and .10. Model values, not measurements. [internal]

A Monte Carlo sweep over the (p_d, δ) grid checks the closed-form threshold against the supplied timing and payoff model (Figure 7.12). Let $L = 5(q_{\text{floor}} - \mu)$ be the one-time detection penalty, $\pi_C = (q_{\text{floor}} - \mu)/3$ the per-member cartel payoff, and $\pi_D = q_{\text{floor}} - \varepsilon - \mu$ the one-shot deviation payoff. Every active cartel round credits π_C; if detection occurs in that round, the simulation then subtracts L and terminates future cartel payoffs. Hence

$$V_C = \pi_C - p_d L + \delta(1 - p_d)V_C, \qquad p_d^\star = \frac{\pi_C - (1-\delta)\pi_D}{L + \delta\pi_D}.$$

At $\delta = 0.95$ this gives $p_d^\star \approx 0.0478$, and the tested grid first classifies the cartel as fragile at $p_d = 0.05$. This is a falsifiable, model-conditional calibration target for bid-pattern detection; the paper does not claim the deployed detector already attains it.

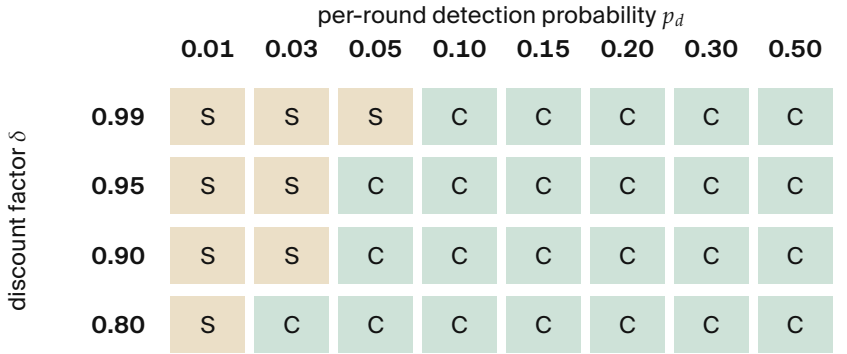


Figure 7.12. S means the cartel sustains; C means it collapses. Every simulated boundary lies within one tested grid step of the closed-form threshold (§7.6.5.10). Each configuration uses 5,000 trials, 200 rounds, and three colluders. [verified, `proofs/bonded/pareto/simulation-cartel.mjs`]

δ	closed-form $(p_d)^\star$	grid flips at
0.80	0.0234	0.03
0.90	0.0398	0.05
0.95	0.0478	0.05
0.99	0.0541	0.10

This result does not automatically extend to the agent layer. The two layers share a repeated-game template, but their payoff functions and monitoring signals differ; each layer needs its own obedience calculation.

Exercises 7.10–7.12, p. 632.

7.7 Coordination as Substrate: An Expressive-Act Taxonomy

Reader's note: this section is a half-page taxonomy. The auction-focused reader can skim the five-item list and skip to §7.6.5.8; the mechanism-design reader will want the per-class bond profiles in full. The taxonomy matters to both because it explains why a single bond price cannot be set.

The bonded commons treats coordination as a uniform action surface: agents post bonds, settle, accrue reputation. In practice, the *kinds* of coordination differ enough that uniform pricing distorts incentives. We argue that any commons-scale pricing mechanism must distinguish among five expressive classes, and that each class has a different bonding profile. The punchline is in one line: **one price can exclude low-value acts while under-covering high-loss acts.** The five classes below establish why.

7.7.1 Five Expressive Classes

Signal. "Something happened." Notes, tuples, pheromones, activity events. High frequency, low individual stakes, cumulative value.

Request. "Decision, input, or approval needed." Escalations to a human or to a higher-authority agent. Low frequency, blocks downstream work, asymmetric cost.

Distress. "I am stuck, repeatedly." Bounded enum: `auth`, `conflict`, `permission`, `budget`, `invariant`. Rare; abuse can halt sibling agents.

Commons. Shared durable state—tuple kinds, graph edges, channels, proposals. Persists beyond the originator; pollution is hard to undo.

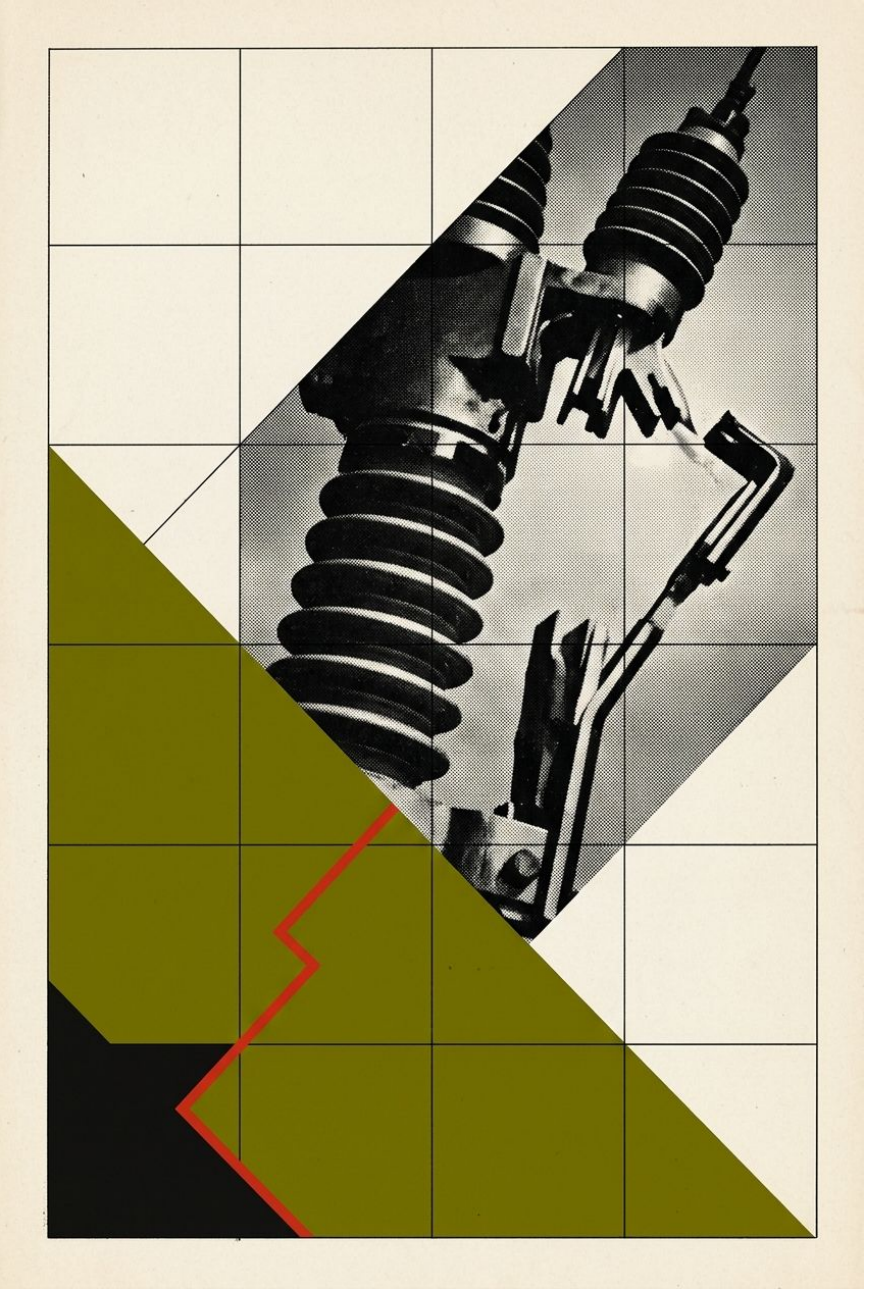
Proposal. "Here is a plan; commit?" Float Plans, change sets, fleet specifications. Requires review; cost is in the reviewer's time.

7.7.2 Per-Class Bond Profiles

Each expressive class has a different relationship to the cleanup cost of §7.6.5.1:

- *Signal class*: cheap, numerous, micro-bonds or reputation-discounted. Slashable for false alarm.
- *Request class*: bonded by the cost of the human's time plus the downstream action cost.
- *Distress class*: bonded by the blast radius if an agent uses it to halt siblings abusively.
- *Commons class*: bonded by the storage cost; slashable for pollution.
- *Proposal class*: bonded by the cost of reviewing plus the cost of implementing.

Treating all coordination as one uniform "bonded action" misses the price-discovery differences. The Bonded Advisor (§7.6.5.7) and the Competitive-Insurance market (§7.6.5.8) must price each class separately, or justify a pooled price against the different costs and participation constraints.



Landmark 10 · Bonds, Evidence & Settlement

A bond changes incentives only when a violation can be established and the sanction collected. Coverage, detection and deterrence are separate calculations. Settlement must follow the agreed evidence and dispute rules; collateral does not make every harmful action observable.

7.8 Evidence-Growth Monitoring and Replay

This section proposes a task-specific progress signal and an evaluation protocol. Wall-clock time, resource consumption, and recorded evidence answer different questions and should remain separately visible. No deployment result or universal failure detector is claimed.

A defined window score. Fix a wall-clock window $W = [t - \Delta, t)$ of duration $\Delta > 0$ seconds. Before observing the window, declare evidence categories $j = 1, \ldots, m$, their acceptance criteria, and nonnegative task weights w_j in points per event. Let $N_j(W)$ count distinct accepted records in category j during W. The proposed evidence total and rate are

$$E(W) = \sum_{j=1}^{m} w_j N_j(W), \qquad v(W) = \frac{E(W)}{\Delta}.$$

E has units of points and v of points per second. Record consumed tokens $U(W)$ separately. Categories might include an accepted patch, a completed test record, or a documented analysis result. Repeating a reference does not create new evidence; the contract must specify deduplication, attribution, and what makes each record acceptable. Counts alone do not establish usefulness, semantic correctness, or a causal contribution to the outcome. Weights need task-specific validation before rates can be compared across tasks.

Illustrative arithmetic. For a 30-second window, assign two points per accepted patch and one per accepted test record. Table 7.7 computes the score for three constructed records. They are arithmetic examples, not calibrated thresholds or classifications of the agents.

Review and optional recovery. A low rate with sustained resource use may prioritize a review. It can also reflect legitimate thinking, reading, non-code work, a long-running test, or a wait for external evidence. Conversely, an agent can inflate counts

Case	Accepted records	Rate (points/s)	Tokens
A	3 patches, 2 tests	8/30	450
B	1 patch, 1 test	3/30	450
C	none	0	1,200

Table 7.7. Illustrative evidence scores for a 30-second window. Patch and test weights are two and one points per event. A zero score means no counted evidence in this window; it does not establish failure.

with unhelpful records. A proposed detector must therefore report its observation window, resource threshold, task class, and false-positive and false-negative rates against independently assessed outcomes. The score does not itself authorize interruption or sanctions. A separately authorized recovery procedure may pause at a safe boundary, retain available task state, ask for a revised plan, or start a successor. Pausing a process does not undo external effects; a summary can omit information; recovery is not guaranteed to preserve all context or resolve the underlying problem.

Consented historical-defect probes. A candidate evaluation uses consented, blinded probes drawn from historical defects in an isolated evaluation workspace. Participants know the evaluation policy while the particular defect and expected answer remain withheld. The protocol must define sampling, exposure limits, adjudication, and the separation of evaluation artifacts from production work. It must not introduce defects into live work. Report detection, repair quality, false alarms, and resource use; a probe outcome alone establishes neither general competence nor a justified change to reputation or execution authority.

Replay requirements. A proposed JSONL replay format should identify events, artifacts, claims, settlements, writer order, and known causal dependencies. Missing events and uncertain ordering must be represented explicitly. A replay supports inspection of retained observations; it does not recreate hidden model state or guarantee that a rerun produces the same result. Training use requires separate consent and evaluation.

Disclosure and retention requirements. Collection and replay should be opt-in under an explicit disclosure policy. The design must specify data minimization, access control, key custody, redaction tests, retention periods, and deletion behavior. Redaction can

miss secrets or personal data; encryption and retention claims require implementation and verification. This section establishes requirements for that work, not a claim that every session already meets them.

These proposals retain wall-clock time for budgets, timeouts, and appeal windows. Evidence growth cannot suspend a deadline. Their research question is whether task-specific records improve review decisions beyond resource and elapsed-time baselines at a measured cost in missed problems and false alarms.

7.9 Formal Model

Leslie Lamport. *Specifying Systems: The TLA⁺ Language and Tools for Hardware and Software Engineers*. Addison-Wesley, 2002.

The following TLA⁺ excerpt describes a coordination lifecycle. It is not the executable specification of the whole architecture: initialization, the complete transition relation, financial recipients, and monitoring rules are omitted. The bounded accounting artifact is identified separately in Appendix 7.A.

Read the state machine as five English sentences before reading any syntax. An agent may not `Begin` without posting positive escrow. A file `Claim` always succeeds and only ever informs—it never blocks. `Commit` clears a session's claims and locks together. A `Reap` demotes an unresponsive agent to `abandoned` and queues it for `Salvage`. Notes grow monotonically while a session is active; destructive `Dismiss` is outside that property. The excerpt below expresses these lifecycle rules; a complete executable specification would still need the omitted initialization and transition definitions.

7.9.1 State and Actions

Figure 7.13 draws this same lifecycle as a state machine before the TLA⁺ excerpt states its actions: one honest path settles a session, and a crash detours it through Reap to either a successor's Salvage or a permanent Dismiss.

Listing 7.2.
TLA⁺: State Variables

```
---- MODULE BondedCommons ----
EXTENDS Naturals, Sequences, FiniteSets

CONSTANTS Agents, Files, Locks, DeadThreshold

VARIABLES
  registered,   \* Set of active agent IDs
  sessions,     \* Agent -> {status, notes, claims, escrow}
```

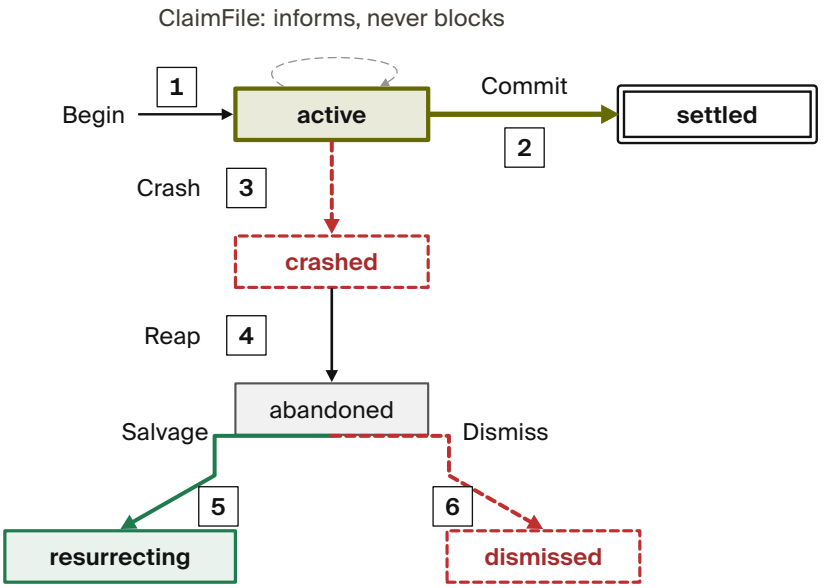


Figure 7.13. Permitted lifecycle transitions. Reaping needs the timeout guard; eventually choosing Salvage or Dismiss needs an additional fairness assumption. The diagram alone gives no recovery deadline. Claims inform without excluding another claimant. [internal, `BondedCommons.tla`]

	action	guard	effect
1	Begin	$a \notin$ *registered* $\wedge$ *escrow* > 0	posts collateral; status $\rightarrow$ active
2	Commit	*status* = active	status $\rightarrow$ settled; escrow $\rightarrow 0$; claims and locks released
3	Crash	*status* = active	*heartbeats* $\rightarrow 0$ (status itself unchanged until Reap fires)
4	Reap	*status* = active $\wedge$ *clock* $-$ *heartbeats* $\geq$ *DeadThreshold*	status $\rightarrow$ abandoned; queued *pending* for salvage
5	Salvage	successor active $\wedge$ (*dead*, pending) $\in$ *salvageQueue*	queue entry $\rightarrow$ resurrecting
6	Dismiss	(*dead*, pending) $\in$ *salvageQueue*	session $\rightarrow$ NULL; irrecoverable information loss

```
  fileClaims,   \* File -> set of claiming agents
  heldLocks,    \* Lock -> {owner, expiry} or NULL
  salvageQueue, \* Set of {agent, status}
  heartbeats,   \* Agent -> last heartbeat time
  clock         \* Monotonic clock

vars == <<registered, sessions, fileClaims, heldLocks,
          salvageQueue, heartbeats, clock>>
```

```
Begin(a, escrow) ==
  /\ a \notin registered
  /\ escrow > 0    \* Must post collateral
  /\ registered' = registered \cup {a}
```

Listing 7.3. TLA$^+$: Core Actions

```
  /\ sessions' = [sessions EXCEPT ![a] =
       [status |-> "active", notes |-> <<>>,
        claims |-> {}, escrow |-> escrow]]
  /\ heartbeats' = [heartbeats EXCEPT ![a] = clock]
  /\ UNCHANGED <<fileClaims, heldLocks,
                 salvageQueue, clock>>

ClaimFile(a, f) ==
  /\ a \in registered
  /\ sessions[a].status = "active"
  \* Advisory: always succeeds, reports conflicts
  /\ fileClaims' = [fileClaims EXCEPT ![f] =
       fileClaims[f] \cup {a}]
  /\ UNCHANGED <<registered, sessions, heldLocks,
                 salvageQueue, heartbeats, clock>>

Commit(a) ==
  /\ a \in registered
  /\ sessions[a].status = "active"
  /\ sessions' = [sessions EXCEPT ![a] =
       [sessions[a] EXCEPT !.status = "settled",
                           !.escrow = 0]]
  \* Release all claims and locks
  /\ fileClaims' = [f \in Files |->
       fileClaims[f] \ {a}]
  /\ heldLocks' = [l \in Locks |->
       IF heldLocks[l] # NULL /\ heldLocks[l].owner = a
       THEN NULL ELSE heldLocks[l]]
  /\ UNCHANGED <<registered, salvageQueue,
                 heartbeats, clock>>

Crash(a) ==
  /\ a \in registered
  /\ sessions[a].status = "active"
  /\ heartbeats' = [heartbeats EXCEPT ![a] = 0]
  /\ UNCHANGED <<registered, sessions, fileClaims,
                 heldLocks, salvageQueue, clock>>

Reap ==
  /\ \E a \in registered :
     /\ sessions[a].status = "active"
     /\ clock - heartbeats[a] >= DeadThreshold
     /\ sessions' = [sessions EXCEPT ![a] =
          [sessions[a] EXCEPT !.status = "abandoned"]]
     /\ salvageQueue' = salvageQueue \cup
          {[agent |-> a, status |-> "pending"]}
     /\ fileClaims' = [f \in Files |->
          fileClaims[f] \ {a}]
     /\ UNCHANGED <<registered, heldLocks,
                    heartbeats, clock>>

```

```
Salvage(successor, dead) ==
  /\ successor \in registered
  /\ sessions[successor].status = "active"
  /\ [agent |-> dead, status |-> "pending"]
       \in salvageQueue
  /\ salvageQueue' = (salvageQueue \
       {[agent |-> dead, status |-> "pending"]})
       \cup {[agent |-> dead,
              status |-> "resurrecting"]}
  /\ UNCHANGED <<registered, sessions, fileClaims,
                 heldLocks, heartbeats, clock>>

Dismiss(dead) ==
  \* Irrecoverable information loss
  /\ [agent |-> dead, status |-> "pending"]
       \in salvageQueue
  /\ salvageQueue' = salvageQueue \
       {[agent |-> dead, status |-> "pending"]}
  /\ sessions' = [sessions EXCEPT ![dead] = NULL]
  /\ UNCHANGED <<registered, fileClaims, heldLocks,
                 heartbeats, clock>>

Tick == /\ clock' = clock + 1
        /\ UNCHANGED <<registered, sessions, fileClaims,
             heldLocks, salvageQueue, heartbeats>>
```

7.9.2 Properties

Listing 7.4. TLA⁺: Safety and Liveness Properties

```
\* SAFETY: Every active session has positive escrow
EscrowInvariant ==
  \A a \in registered :
    sessions[a] # NULL /\ sessions[a].status = "active"
    => sessions[a].escrow > 0

\* SAFETY: Notes never shrink for active sessions
NoteMonotonicity ==
  \A a \in registered :
    sessions[a] # NULL /\ sessions[a].status = "active"
    => Len(sessions'[a].notes) >= Len(sessions[a].notes)

\* LIVENESS: Dead agents are eventually salvaged
\* or dismissed (under fairness)
CrashRecovery ==
  \A a \in Agents :
    [agent |-> a, status |-> "pending"] \in salvageQueue
    ~> [agent |-> a, status |-> "pending"]
         \notin salvageQueue

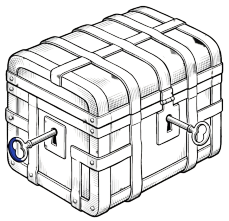
Spec == Init /\ [][Next]_vars
```

```
22          /\ WF_vars(Reap) /\ WF_vars(Tick)
23
24  ====
```

Read liveness separately from the escrow invariant. A trace may keep choosing `Tick` after a dead agent is queued, never choosing `Salvage` or `Dismiss`. Weak fairness of `Reap` and `Tick` alone does not exclude it. Eventual recovery needs an enabled recovery action and fairness for its selection. Also, `Dismiss` deletes the retained session in this excerpt: its note monotonicity covers active sessions, not this explicit destructive action.

Key invariant: `EscrowInvariant` ensures that no agent can participate in the commons without posted collateral. This is the economic layer's structural guarantee—it holds by construction of the `Begin` action, which requires `escrow` > 0.

7.9.3 The Conservation Theorem



Like a dual-control iron escrow chest requiring two separate keys to unlock, escrowed funds cannot be released unilaterally until criteria and verification agree.

The TLA$^+$ escrow invariant states the property abstractly; the daemon's bond-accounting subsystem realizes it operationally, collapsing the abstract escrow into three monetary buckets per project—**wallet**, **escrow**, and **commons pool**—which support a different obligation: preservation of total funds. Positive escrow at admission and conservation across transfers are independent properties; a system can satisfy either while violating the other.

Definition 7.9.1 (Accounting State)**.**

For project P, the accounting state is the triple $(W_P, E_P, C_P) \in \mathbb{R}^3_{\geq 0}$, where W_P is wallet balance, E_P is the sum of bond amounts in states $\{\textit{escrowed}, \textit{running}, \textit{exiting}\}$, and C_P is the commons pool balance.

The *monetary supply* is $S_P := W_P + E_P + C_P$.

Theorem 7.9.1 (Conservation).

For any sequence of operations

$$\sigma \in \{\text{topUp}, \text{escrow}, \text{markRunning}, \text{refund}, \text{slash}\}^*$$

applied to an initial state $(W_P^0, 0, 0)$, the supply S_P changes only on topUp.

All other operations redistribute among the three buckets.

Proof sketch.

Each operation commits in a single SQLite transaction (`db.transaction`). The individual effects:

- topUp(u): W_P += u. S_P grows by u.
- escrow(b): W_P −= b, E_P += b. S_P invariant.
- markRunning(*id*): state transition within E_P's contributing set. No monetary movement. S_P invariant.
- refund(*id*): W_P += b_{id}; row state → *refunded*. Net E_P −= b, W_P += b. S_P invariant.
- slash(*id*, p) with $p \in [0, b_{id}]$: W_P += $b_{id} - p$, C_P += p, row → *slashed*. Net $-b + (b - p) + p = 0$. S_P invariant.

By induction on $|\sigma|$, S_P equals S_P^0 plus the sum of topUp arguments in σ. □ □

Figure 7.14 draws the three buckets and five operations this proof sketch reasons over, instantiated with the section's own worked settlement.

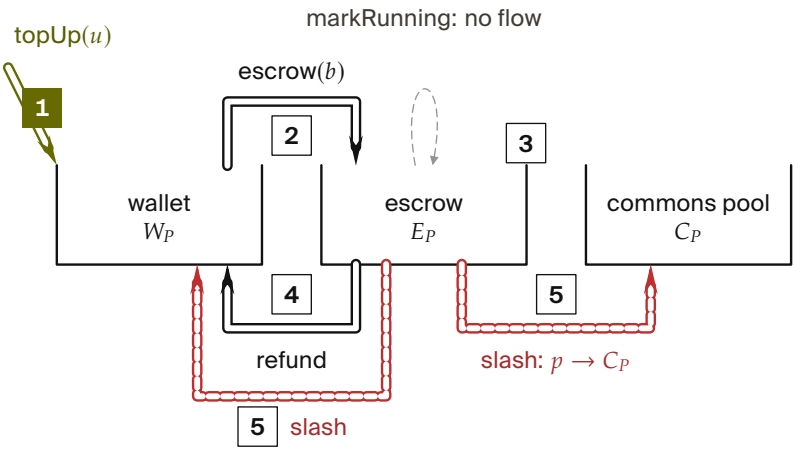


Figure 7.14. Only top-up increases supply. A partial breach moves (W_P, E_P, C_P) from $(4, 6, 0)$ to $(8, 0, 2)$: the full escrow closes, while total supply stays 10. [internal, `Conservation.tla`]

$S_P := W_P + E_P + C_P$: before 4 + 6 + 0 = 10, after 8 + 0 + 2 = 10 — unchanged.

	operation	effect on the buckets	ΔS_P
1	topUp(u)	W_P += u	$+u$
2	escrow(b)	W_P −= b, E_P += b	0
3	markRunning	state change inside E_P's contributing set; no transfer	0
4	refund(*id*)	E_P −= b, W_P += b	0
5	slash(*id*, p)	E_P −= b, W_P += (b−p), C_P += p	0

Property verification. The conservation invariant is asserted after every operation in the bonds test suite. Across 200 random traces, $|W_P + E_P + C_P - S_P^{\text{expected}}| < 10^{-6}$ holds deterministically; the 10^{-6} tolerance is IEEE-754 floating-point slack, not logical slack.

Theorem 7.9.2 (No Overdraft under Concurrent Escrow).

Given two concurrent invocations escrow(b_1) and escrow(b_2) on the same project with $b_i \le W_P^{\text{pre}}$ but $b_1 + b_2 > W_P^{\text{pre}}$, at most one invocation succeeds.

Implementation: Atomic RETURNING Clauses. To enforce overdraft prevention atomically and avoid Node.js/TypeScript event-loop async yield vulnerabilities, the escrow state transition is written as a single SQLite statement with a `RETURNING` clause. This guarantees that balance checks and deductions occur within a synchronous, uninterruptible database step, preventing race conditions from interleaved async microtasks.

Proof.

Both transactions execute under `BEGIN EXCLUSIVE`, so SQLite serializes them.

Suppose T_1 wins. After commit, $W_P = W_P^{\text{pre}} - b_1$.

T_2 now reads the updated balance and rejects iff $b_2 > W_P^{\text{pre}} - b_1$, which by assumption is true. □ □

Graduated sanctions. The TLA$^+$ model abstracts settlement into `Commit` and `Crash`. Reality has an intermediate state: an agent whose spend is approaching its daily budget. The daemon's budget-guard component introduces **throttle** as a soft signal at 80% spend (publishes a `budget:decisions:throttle` event; structurally non-coercive, advisory in Sen's sense) and **kill** as the hard signal at 100% (refuses new spawns until UTC midnight; existing spawns SIGTERMed; bond slashed). The throttle/kill split is the commons' analogue of Ostrom's graduated sanctions: response scales with violation severity. Settlement is not binary; it is a three-state machine *nominal* → *throttled* → *killed*, with *nominal* → *killed* reachable only on hard evidence (e.g., a direct Arbiter violation).

Ostrom 1990

Numbers by Hand 7.9.3 (A Settlement's Ledger Check).

Instantiate Theorem 7.9.1's proof sketch with round numbers. Before a settlement, a project sits at $(W_P, E_P, C_P) = (4, 6, 0)$—four in wallet, six escrowed on one open Float Plan, nothing yet in the commons pool—so

$$S_P = 4 + 6 + 0 = 10$$

[verified].

The settlement resolves as a partial breach: slash(*id*, p=2) transfers 2 to the commons pool and refunds the remaining $b - p = 6 - 2 = 4$ to the wallet.

Applying the proof sketch's own bookkeeping, $W_P \mathrel{+}= (b - p) = 4 + 4 = 8$, $C_P \mathrel{+}= p = 0 + 2 = 2$, and E_P's contribution of 6 is fully retired to 0.

After: $(W_P, E_P, C_P) = (8, 0, 2)$, and

$$S_P = 8 + 0 + 2 = 10$$

[verified]—unchanged, exactly as the theorem predicts, because the settlement only moved money between buckets rather than minting or destroying it.

Now you try: Exercise 7.13 asks you to find the analogous check across all 1,716 reachable states of the mechanized model, not just this one hand-picked settlement.

Why the forfeit goes to the commons and not back to the buyer. The slashed 2 goes to C_P, not to the injured counterparty. Relative to the two-party coalition, this makes a breach costly only if the pair cannot reclaim the pool payout through ownership, side payments, or governance. Holmström's team-production model explains why an outside budget breaker can matter when effort is hidden and team transfers must add up. It does not prove this harbor mechanism efficient or coalition-proof.

Bengt Holmström. Moral hazard in teams. *The Bell Journal of Economics*, 13(2):324–340, 1982.

Paying compensation to an injured buyer can still deter an individual seller: the seller loses the bond either way. What an internal transfer cannot do is reduce the pair's combined wealth if both participants collude and can transfer freely between themselves. An inaccessible commons pool is one proposed way to make that

coalition pay; its independence is a governance assumption, not a consequence of calling an account a commons. Conservation still holds over $W_P + E_P + C_P$. The accounting identity does not choose the recipient, and Holmström's theorem does not supply a general prohibition on compensation.

Recall

1. What does `EscrowInvariant` guarantee? Which action establishes it?
2. State the three-bucket identity. Which operation may change the total?
3. Why must `Reap` wait after a missed heartbeat? Which variable decides?

Exercises 7.13–7.14, p. 633.

7.10 Related Work

Social contract theory. Hobbes argued that cooperation requires a sovereign whose authority is accepted before interactions begin. Locke proposed a more limited authority protecting natural rights. Our commons authority is Hobbesian in structure (centralized, pre-transactional) but Lockean in scope (provides infrastructure, not dictates).

Hobbes 1651

John Locke. *Two Treatises of Government*. Awnsham Churchill, London, 1689.

Impossibility results. Sen proves an incompatibility under an unrestricted preference domain and decisive personal pairs. Our locking example is separate: precautionary claims can block a mutually preferred completion. It motivates a design choice without transferring Sen's theorem to every lock manager.

Sen 1970

Long-lived transactions. Garcia-Molina and Salem introduced Sagas for decomposing long-lived transactions with compensating actions. Our collateralized contracts extend Sagas with economic settlement: rather than mechanically compensating for partial failure, the evidence chain enables pro-rata escrow release.

Garcia-Molina & Salem 1987

BDI agents. Rao and Georgeff formalized agents with beliefs, desires, and intentions. The mapping to our model is: the evidence trail captures *beliefs* (accumulated knowledge), the Float Plan manifest captures *desires* (intended outcomes), and file claims and locks capture *intentions* (committed resources). Advisory claims are the coordination analogue of BDI intentions—they represent resource commitment, not aspiration.

Rao & Georgeff 1991. *Modeling Rational Agents within a BDI-Architecture*.

Generative agents. An agent's memory can guide its next action without serving as a record that other parties can audit. Here, an evidence trail has a different job: another participant must be able

In a town of 25 simulated agents, a party invitation spread through conversations. The experiment tests believable social behavior, not crash recovery.

Park et al. 2023

to inspect what happened, and recovery must distinguish recorded commitments from work still in progress. Remembering an event and preserving evidence of it are separate requirements.

Capability-based security. Dennis and Van Horn introduced capability-based access control. Birgisson et al. extended this with Macaroons (chained HMACs for decentralized delegation). The Anchor Protocol adapts Macaroons for agent coordination with Ed25519 signatures and offline attenuation.

Dennis & Van Horn 1966

Birgisson et al. 2014

Owens 2026

Mechanism design. The pricing of Float Plan bonds is a mechanism design problem in the tradition of Vickrey and Myerson. A full optimal-auction derivation in that tradition is outside the scope of this chapter.

William Vickrey. Counterspeculation, Auctions, and Competitive Sealed Tenders. *The Journal of Finance*, 16(1):8–37, 1961.

Myerson 1981

Agentic-commerce protocols. While this chapter was in preparation, the transaction rails of an agent economy shipped as industry standards: the Universal Commerce Protocol (Google/Shopify, 2026) for merchant-hosted agent-mediated retail, its platform-mediated rival the Agentic Commerce Protocol (OpenAI/Stripe), and the Agent Payments Protocol, whose signed mandate chain — W3C Verifiable Credentials in SD-JWT form, principal → platform → merchant — is the deployed sibling of our countersigned Float Plan. All three are explicit that agent *trustworthiness* is out of scope; published security analyses observe that they regulate how agents transact while providing no collateral, no settlement oracle, and no priced deterrent against a misbehaving agent. That gap is precisely the synthesis this chapter contributes (§7.10 *supra*): bonded participation, capability attenuation, advisory coordination, and immutable attribution. The proposed boundary profile uses SD-JWT-VC cards and JWS detached-content countersigns over JCS. These formats concern serialization and verification; they do not establish the bond's custody, settlement, or deterrence properties. Interoperability with an external payment system still requires separate implementation and testing.

Universal Commerce Protocol 2026

Agentic Commerce Protocol. OpenAI and Stripe, 2025. Online source.

AP2 2026

DataDome 2026

Table 7.8.
What the commons borrows from each prior body of work, where it departs, and why.

Body of work	Borrowed	Departure	Why
Social contract theory	Authority accepted before interaction begins	The sovereign is a daemon whose rules are inspectable and whose consent is revocable per task	Agents can exit; Hobbes's subjects could not
Impossibility results	Decisive rights under unrestricted preferences	A separate precautionary-lock counterexample	Motivates a conditional advisory default
Long-lived transactions	Compensation for work that outlives a transaction	Economic settlement in place of mechanical compensation	Compensation cannot undo an external effect; a bond can price it
BDI and generative agents	Beliefs, desires, intentions; memory streams	The evidence trail is read by an adversarial operator, not a private memory	The reader, not the agent, is the customer of the record
Capability-based security	Attenuable capabilities	Capabilities bind to bonded identities with settlement	Authority without liability is the failure the commons prices
Mechanism design	Incentive compatibility as the design target	A claim-signaling patience threshold and separate bond-pricing proposals	The commons is a repeated game, not a one-shot sale
Agentic-commerce protocols	The credential envelope	The envelope carries authorization; the commons carries liability	A standard format shrinks the problem without settling it

7.11 Discussion and Limitations

The commons authority is a single point of failure. If the daemon crashes, the Leviathan falls. No new trust is established, no conflicts detected, no salvage occurs. Agents with cached Harbor Cards continue working, but the commons enters a state of nature until the daemon restarts. This is mitigated by automatic restart (launchd on macOS) but not formally bounded.

Collateral does not prevent harm—it prices it. A principal willing to burn money to sabotage a competitor will post a bond, cause damage, and accept the loss. The bond makes sabotage expensive, not impossible. The defense against existential threats is not internal governance but *admission control*: who is allowed to participate in the commons at all. This decision is irreducibly political and lies outside the formal model.

The Federated Sovereign. The single-machine model presented so far is insufficient on its own: users roam across laptop, desktop, and CI machines; machine loss should not destroy encrypted evidence; multiple humans may share a harbor. The daemon remains the authority *within* its machine. Key custody federates outward.

We introduce a fourth actor, specified by properties rather than by vendor:

Daemon commons authority within a machine; signs harbor roots; enforces bonds.
Agent participant; holds Harbor Card; signs actions.
Principal funds Float Plans; identifies via Ed25519 keypair.
KMS key custody; recovery root-of-trust; witnesses harbor roots.

Definition 7.11.1 (Admissible KMS)**.**

A Key Management Service $\mathcal{K}$ is admissible for the federated sovereign if it provides:

1. Passphrase-wrapped private-key custody using a memory-hard KDF (Argon2id) at the 2026 security floor.
2. Encryption-at-rest for all user-held blobs; $\mathcal{K}$ never sees plaintext keying material.
3. A signed witness log for harbor Merkle roots with append-only, monotonic semantics and detectable equivocation. Laurie et al. 2013

4. Rate-limited, email-gated recovery via single-use magic-link tokens of bounded TTL.
5. Public verification of signed user public keys under an account identifier, without requiring mutual authentication.

The implementation choice (a particular cloud platform, an on-prem HSM, a self-hosted Tang server) lives in companion design documents. The paper's theory applies to any $\mathcal{K}$ meeting properties (1)–(5).

Trust boundary. The federated trust boundary is TB = *daemon* ∪ $\mathcal{K}$ ∪ *user-email* ∪ *user-passphrase*. Each element is necessary for its role; no element is sufficient alone. The daemon enforces bonds and publishes roots but never sees the unwrapped user master. The KMS stores encrypted blobs and can deny service but cannot decrypt. Email is the recovery root—and is documented as the weakest link, since an adversary with email control can trigger magic-link recovery. The passphrase wraps the user's master with Argon2id, making offline brute force expensive.

Table 7.9 separates the four attacks in Design Invariant 7.11.1. Reading the stored ciphertext is not enough to decrypt it; controlling the recovery email is not enough to impersonate the user. The rollback condition is different again: it requires the witness service's complicity, not possession of a publicly visible signature.

Table 7.9. Each attack requires different access. These conditions exclude same-user code execution on the daemon host. [design invariant]

Attack	Required access
Forge a Harbor Card	Daemon's private signing key
Read encrypted evidence	Harbor's session key
Impersonate the user	**Both** email access and passphrase
Roll back the Merkle forest	Complicity of the witness service (KMS)

Magic-link single-use atomicity. The email-gated recovery path requires that each emitted magic-link token be redeemable *at most once*—otherwise a race window between two consumers can produce two valid recovery completions for the same token. The ProVerif model encodes the property as an injective event: inj-event(`consumed_for`) ⇒ inj-event(`issued_for`). This prevents duplicate consumption; it does not guarantee that anyone redeems

the token. The runtime analogue is a single atomic SQL update with a `WHERE consumed_at IS NULL RETURNING` clause; SQLite's writer serialization drains the token under the first committed transaction and returns zero rows to a subsequent attempt. Figure 7.15 follows two attempts against the same unused token.

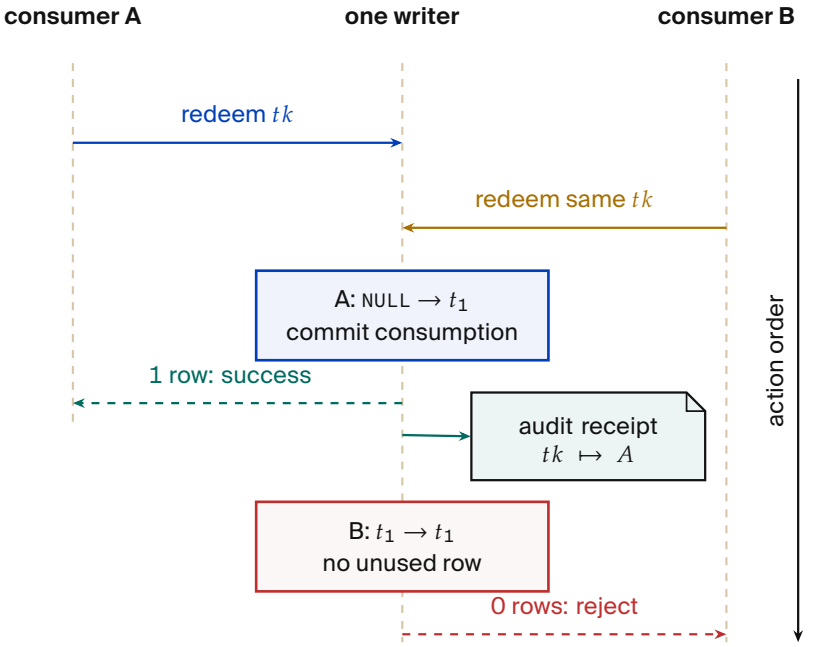


Figure 7.15. Overlapping requests still produce one consumption. A wins this illustrated schedule; either caller can win. SQL is simplified: authentication and expiry checks are omitted.

Conditional update

```
UPDATE token SET consumed_at = :now
WHERE token = :tk
AND consumed_at IS NULL
RETURNING token;
```

Design Invariant 7.11.1 (Federated Security, Informal).

Assuming Ed25519 and SHA-256 are secure, Argon2id parameters meet the 2026 security floor, and the user's email and passphrase are not simultaneously compromised, an adversary with read access to $\mathcal{K}$'s storage, the daemon's SQLite database, and mesh gossip traffic, but *without* same-user code execution on the daemon's host, cannot: (i) forge a Harbor Card without the daemon's private key, (ii) read plaintext evidence without the harbor's session key, (iii) impersonate the user without both email

access and passphrase, (iv) roll back a harbor's Merkle forest without $\mathcal{K}$'s complicity.

The theorem deliberately excludes the same-user adversary (an unsandboxed agent, a malicious postinstall, a compromised editor extension). Such an adversary reads any file the user can read, which absent OS-mediated key custody includes the daemon's keys and database. The `macOS Keychain` integration ships now; native keyring and hardware-backed keystore extend the theorem's reach to the same-user case.

Recovery and its structural limits. Recovery is email magic link. The structural limit: *if the user loses both passphrase and old machine*, harbor session keys wrapped only for that pubkey cannot be recovered. This is the price of zero-knowledge at-rest encryption. An opt-in Shamir escrow mode (t-of-n secret sharing across $\mathcal{K}$, email, and recovery contacts) trades some zero-knowledge for stronger recovery.

Adi Shamir. How to Share a Secret. *Communications of the ACM*, 22(11):612–613, 1979.

Passkeys, not passwords. Identity is anchored in WebAuthn-backed device keypairs—no phishable secrets, no passphrase on the critical path of every action. New devices enroll by exchanging a short-lived pairing token that re-encrypts the KMS master under the new device's public key. Mobile devices act as *viewers*, not peers: they sign low-stakes actions (approve an ask, bump budget) over a WebSocket viewer channel, but they do not run the full daemon. Each account's harbors gossip Merkle roots among its own devices; cross-account gossip requires explicit signed consent.

What we deliberately do *not* do: require passwords, use TOTP as a primary factor, or couple recovery to a single cloud vendor.

Exercises 7.15–7.17, p. 633.

Multi-node consensus. For fleets requiring strict serializability across nodes, a Raft-backed harbor-root consensus is possible but not specified here; eventual consistency via the Merkle forest plus KMS witness is sufficient for the workloads we have measured. Paxos remains the textbook fallback.

Ongaro & Ousterhout 2014

Leslie Lamport. The Part-Time Parliament. *ACM Transactions on Computer Systems*, 16(2):133–169, 1998.

Recovery fidelity depends on LLM capability. Social recovery works because successor agents can interpret natural-language evidence trails. This capability is not formally guaranteed—it depends on the successor's LLM. A formal model of "given this evidence trail, will the successor complete the original intent?" requires characterizing LLM reasoning, which is an open problem.

Bond pricing is unsolved. We identify the pricing function π as the critical open problem. Without an adequate π, bonds are either too low (cheap sabotage) or too high (priced-out legitimate agents). This is a mechanism design problem, not a systems problem, and requires expertise we explicitly flag as needed.

Review of the key ideas

What this chapter leaves coupled, in the order the rest of the book will lean on it:

1. **Three layers, each doing what the layer below cannot.** Structural prevention bounds scope before the act; immutable attribution makes every act answerable after it; economic alignment prices what neither can forbid.
2. **Structural scope limitation is a theorem**, not a policy: accepted operations stay within the root grant when every relevant effect passes through the verifier. Checks have a runtime cost; an unmediated write is outside that guarantee.
3. **The trail is a Merkle forest.** Trail integrity, proof soundness, binding, attribution, rollback safety and conservation are the six properties of the ledger; the accounting model has bounded model-checking evidence; runtime no-overdraft also needs atomic admission and has separate property tests.
4. **Decisive allocation has limits.** There is a Pareto-inferior locking instance, while advisory completeness concerns overlaps among registered claims, and an appeal is sound when it can only widen the record.
5. **A crash is a social event.** Recovery is continuation of the record by whoever next holds the role, not resurrection of the process.

6. **Intent is irrelevant to settlement**; the bond pays on the evidence, whatever the agent meant.
7. **Claim signaling is incentive-compatible above** $\delta^* \approx 0.3425$, the root of $2\delta^3 + 2\delta^2 + 2\delta - 1 = 0$; TLC agrees on the integer grid, at 0.35 holding and at 0.33 producing the deviation trace.
8. **Cleanup has a lower bound**: cleanup funded entirely from a forfeited bond requires that bond to cover the assumed bill. Deterrence additionally depends on the principal's gain and probability of collectible loss.
9. **Disputes escalate only when the cheaper exit fails**: released, then expired by TTL, then preempted by bond, then ruled by a human, every terminal state writing to one evidence rail; a magic link is redeemed by one row update and refused by the second reader.

7.12 Exercises

§7.2 Layer 1: Structural Prevention

Solution p. 768

7.1 Check ⋆

An agent α holds a Harbor Card with `fs:write:auth.ts`, `db:read`, TTL = 15 minutes. α delegates to β a card with `fs:write:auth.ts`, TTL = 20 minutes. The verifier rejects this delegation. Cite the specific attenuation invariant violated. Now rewrite β's requested card to be acceptable.

§7.3 Attribution and mutable views

Solution p. 768

7.2 Check ⋆

A signal is sprayed at strength eight with a ten-minute half-life. At minute fifteen it is revoked. What value should a reader at minute twenty see? What records must remain, and what does their authenticated inclusion fail to prove about the original signal?

§7.4 The Limits of Decisive Allocation

7.3 Check ⋆ *Solution p. 768*

Define "advisory claim" and explain, in two sentences, why an advisory regime is strictly more permissive than an enforced-lock regime. Give a precautionary-claim case in which this permissiveness helps, and a real write collision in which it hurts. Why is neither a universal welfare theorem?

§7.5 Crash Recovery as Social Continuation

7.4 Check ⋆ *Solution p. 769*

A reader proposes: "Eliminate bonds. Use post-hoc reputation slashing instead—an agent that defects loses reputation points and is excluded from future work." Identify two specific failure modes of this proposal that the bonded design avoids. Hint: consider the first-mover problem and Sybil identities.

§7.6 Layer 3: Economic Alignment

7.5 Check ⋆ *Solution p. 769*

Using only the payoffs printed in Table 7.5, verify by hand that F strictly dominates T for each player, and check that no cell other than (F, F) is a Nash equilibrium of the one-shot game.

7.6 Trace ⋆⋆ *Solution p. 770*

Using $f(\delta) = 2\delta(1 + \delta + \delta^2) - 1$ from the worked example above, evaluate f by hand at the two integer-grid points $\delta = 0.34$ and $\delta = 0.35$. What sign does each give, and what does the sign change prove? Name the committed artifact whose result this matches.

7.7 Trace ⋆⋆ *Solution p. 770*

`proofs/economics/delta-threshold.z3` asks Z3 three separate questions about the IC cubic. Name each question and state the exact verdict sequence the committed expected output records.

7.8 Trace ⋆⋆ *Solution p. 771*

The chapter's own verification table (Table 7.10) states that the `claim_signaling.tla` model's IC invariant fails at $\delta = 0.30$ and

holds at $\delta = 0.35$. Confirm both verdicts by hand using the deviation inequality $1 < 2\delta(1 + \delta + \delta^2)$ derived in the text above, and describe in two sentences what a counterexample trace at $\delta = 0.30$ would actually show.

Solution p. 771

7.9 Trace ⋆⋆

The script `proofs/economics/sweep-delta.sh` sweeps the grid $\delta \in \{0.30, 0.31, \ldots, 0.40\}$ and reports a crossover value. What is the pinned crossover, and how does the script itself judge whether it passed?

Solution p. 772

7.10 Trace ⋆⋆

Assume premiums are retained on default, loss probability $p > 0$, and slashing is capped at $\min(B_{\text{dep}}, B_T)$. Derive the condition on q, p, B_T under which increasing the deposit cannot deter the specified one-shot attack. Then derive the threshold under full forfeiture. What additional payoff, monitoring, and exposure assumptions would an admission fee plus probation plus aggregate reservation need for a deterrence claim?

Solution p. 772

7.11 Trace ⋆⋆

Using the A6 grid (§7.6.5.10, Figure 7.12), compute the minimum per-round detection probability $p_d^\star$ at $\delta = 0.99$ in the negligible-undercut limit $\varepsilon/x \to 0$. Then argue, qualitatively, what would change if cartel members can re-enter under fresh identities. Does the bond mechanism prevent re-entry, deter it, or merely price it?

Solution p. 773

7.12 Open ⋆⋆⋆

The competitive-insurance mechanism (§7.6.5.8) replaces a single bond price with insurer-agent bids. Propose one alternative pricing mechanism (e.g., a second-price auction with reserve, a Walrasian-style market-clearing rule, or a posted-price catalog) and argue whether it would conditionally Pareto-dominate the static escrow under the full-information, competitive-entry, coverage, and static-baseline assumptions of §7.6.5.8. Identify the assumption your alternative most depends on.

§7.9 Formal Model

7.13 Trace ★★ *Solution p. 774*

`proofs/bonded/conservation/Conservation.tla` is model-checked by TLC. Pin the reachable-state count and the name of the headline invariant from the committed run log, and say precisely what "reachable" means there—is it the same number as the states TLC generated?

7.14 Open ★★★ *Solution p. 774*

Suppose `Conservation.tla`'s `Mint` action were changed to credit `free` without also incrementing `minted`—i.e. `free' = [free EXCEPT ![a] = @ + amt]` stays, but `minted' = minted + amt` is replaced by `minted' = minted`. Predict, from the spec text alone, what a TLC run would show. Which of the checked invariants catches it, which does not, and how deep is the shortest counterexample?

§7.11 Discussion and Limitations

7.15 Trace ★★ *Solution p. 775*

Sketch the ProVerif process for the magic-link recovery protocol of §7.11. What is the secrecy property you would prove? What is the authentication property? Identify one attacker capability ProVerif would need to model that is not present in the Phase 1 Harbor Card model of *The Anchor Protocol*.

7.16 Open ★★★ *Solution p. 775*

The gossip convergence bound cited for Merkle-root dissemination among a harbor's devices (*The Anchor Protocol*'s revocation analysis) is reproduced from the standard anti-entropy analysis rather than mechanized. Identify the specific theorem from Demers et al. that would need to be ported to a proof assistant. What would change in the analysis if the daemon mesh is *not* fully connected? Hint: consider gossip on a graph with bottleneck cuts. Demers et al. 1987

7.17 Trace ★★ *Solution p. 776*

Sketch a pollution attack against a cuckoo filter that does *not* enforce the load-factor ceiling at 0.95 (Table 7.10's cuckoo-filter row). What invariant breaks? Why does the ceiling defeat it? Hint: the FP rate bound in Fan et al. is conditional on the regime. Fan et al. 2014

7.13 Conclusion

We have argued that multi-agent collaboration at scale requires a commons authority—a pre-transactional trust infrastructure—rather than peer-to-peer trust negotiation. The authority provides three layers of guarantee:

1. **Structural prevention**: for any operation routed through the verifier, capability-attenuated tokens make scope escalation physically impossible (Theorem 7.2.1) — a guarantee that holds once the Anchor model's own assumptions and the runtime's interception coverage are granted, and no further.
2. **Immutable attribution**: a Merkle forest of evidence trails (§7.3.2) makes anonymous harm impossible *across daemons for damage to shared state*; it says nothing about disclosure to outsiders (§7.6.5.4) or about a same-user adversary on the daemon's own host (§7.11).
3. **Economic alignment**: Collateralized work contracts make harm expensive, transforming moral questions into economic ones.

Six further contributions thread through this skeleton. The Conservation Theorem (§7.9.3) makes the economic invariant operationally checkable, not just an asserted property of the abstract model. The mutable-signal ledger (§7.3.3) shows how coordination signals can be revoked, renamed, and re-attributed without sacrificing the immutability the rest of the architecture depends on. The federated sovereign (§7.11) replaces single-node scope with an abstract KMS satisfying five named properties, and anchors identity in passkeys rather than passphrases on the critical path. The competitive-insurance pricing mechanism (§7.6.5.8, contributed by Youle) replaces static parameter selection with market-discovered bond prices, using competitive insurance as a conditional pricing benchmark. The expressive-act taxonomy (§7.7) decomposes "coordination" into five priceable classes, because different loss and participation profiles can make one price unsuitable. The proposed evidence-growth and replay protocol (§7.8) specifies measurements for evaluating these designs; its utility remains to be tested.

Advisory claims leave room for negotiation when agents know more about their needs than the allocator. Their benefit depends on the conflict and resolution costs; Sen supplies a warning about

decisive rights, not a universal choice between advisory claims and locks. The commons authority provides information, not allocation decisions.

Bond pricing is not a single open problem. It is a lattice: a cleanup-cost lower bound (§7.6.5.1), a scope multiplier (§7.6.5.2), a reputation discount (§7.6.5.3), the Bonded Advisor mechanism (§7.6.5.7), and the competitive-insurance market (§7.6.5.8). Each is precise enough to disprove.

The next chapter asks which obligations survive when issuer, executor, and payer belong to different harbors. Evidence can cross that boundary; authority, current knowledge, and collectible funds need separate protocols.

Chapter Appendices

7.A Formal Verification Status

Reading the evidence. Table 7.10 names the models and tests associated with each claim; the publication's artifact bundle is at `https://portdaddy.dev/whitepaper/artifacts/`. A model check applies to the model and bounds stated in its row. Runtime conformance and deployment require separate evidence. For example, the passkey model and its missing runtime appear as different rows; completing the first does not complete the second.

Table 7.10. Verification status of critical claims. "Method" is the formal technique; "Result" is its outcome; "Artifact" names the public bundle file. Closed (**closed**) records completion of the stated check within its listed scope. Partial (partial) leaves part of that claim unresolved; spec-only (*spec-only*) identifies a model or design without its runtime. A closed model row does not certify deployment.

Claim	Method	Result	Artifact
Coordination channel isolation (§7.3.2)	ProVerif (Dolev-Yao)	**closed** 3 properties TRUE	`isolation.pv`
Passkey device pairing	ProVerif	**closed** 3 properties TRUE	`passkey-pair.pv`
Federated security (§7.11)	ProVerif query audit	partial queried private constant is not the reconstructed root; intended claim unproved	`federated.pv`
Conservation Theorem (§7.9.3)	TLA+ bounded TLC	**closed** 1,716 reachable states (26,818 generated), invariant holds	`Conservation.tla`
No-overdraft lemma (§7.6)	fast-check property test	**closed** 4 invariants, randomized ops	bonds-property tests
Algorithm-confusion immunity	ProVerif (pinned vs. naive)	**closed** pinned TRUE; counter-trace produced	`algconfusion.pv`
Delegation chain replay	ProVerif at depth 3	**closed** chain authenticity TRUE	`chain-replay.pv`
Magic-link single-use (§7.11)	ProVerif (private-channel cap)	**closed** injective-event TRUE; 7/7 runtime conformance	`magic-link.pv`

(continued)

Claim	Method	Result	Artifact
Cuckoo-filter pollution resistance	fast-check, 5 invariants	**closed** FP rate within $2B/2^f$ at load 0.95	cuckoo property tests
Merkle-Forest binding (§7.3.2)	EasyCrypt + fast-check	partial skeleton discharged; PROM-formal version not in scope	`binding.ec`
Auction/static comparison (§7.6.5.8)	Monte Carlo, 36 configs × 2k trials	partial finite-grid result; not a universal Pareto theorem	`simulation.mjs`
Sybil A5 (§7.6.5.9)	Monte Carlo over K Sybils	partial coverage-bounded ceiling at B_T	`simulation-sybil.mjs`
Cartel repeated-game A6	Monte Carlo + expected-value recurrence	partial $p_d^\star \approx 0.0478$ at $\delta = 0.95$ in supplied model	`simulation-cartel.mjs`
Claim-signaling IC (§7.6)	TLA+ (TLC) + Z3 SMT	**closed** unique root $\delta^\star \approx 0.3425$; IC invariant fails at $\delta = 0.30$, holds at $\delta = 0.35$	`claim_signaling.tla`, `delta-threshold.z3`
Threat-band defensibility (§7.6.5)	Monte Carlo, 7×3 grid	**closed** matched bands absorb target class within 1.10× upper bound	`threat-bands.mjs`
Passkey pairing runtime	—	*spec-only* model exists; no runtime	`passkey-pair.pv`
Federated KMS Cloudflare Worker	—	*spec-only* model exists; no Worker yet	`federated.pv`

Limitations. Four mechanizations are out of scope for this chapter but explicitly named so a reader can audit what is and is not proved: a PROM-formal Merkle binding in EasyCrypt; a Coq or Lean mechanization of the Pareto strategic game; a formal analysis of the composite Sybil-deterrence mechanism (C_{kyc} + reputation gating

+ per-class B_{dep}); and a multi-class generalization of the repeated-game cartel result to heterogeneous coordination classes (§7.7). The present claims stand on the evidence stated in Table 7.10.

7.B Worked Example: One Agent, All Layers

To make the three-layer architecture concrete, this appendix traces one agent—call it α—through a single bounded task: *"Fix the login bug in* `auth.ts` *and regression-test it."* The example exercises capability issuance, advisory claims, the evidence chain, a mid-task crash, successor resumption, and final settlement. All values are illustrative rather than tuned. Assume complete mediation, durable receipts, the stated recovery policy, and an agreed adjudicator; this is a design trace, not a report of a live end-to-end run.

The architecture so far has been described layer by layer. The point of this appendix is to watch the layers *compose*: how a single bounded task progresses through the system, where each layer earns its keep, and what the daemon does *not* touch. The example is deliberately mundane—one small bug, one agent, one principal—because the value of the architecture should be legible in the small before it is trusted in the large.

The task. A principal P wants an agent α to fix a login bug in a hypothetical file `auth.ts` and regression-test it. The acceptance criteria are stated up front:

1. the failing test `login.regression.test.ts` passes;
2. no other test regresses;
3. the diff touches only `auth.ts` and the test file.

P estimates the cleanup cost of α's defection or sabotage at $5—an illustrative total of operator and compute expense, not a measured hourly wage. Figure 7.16 traces what follows across the three architectural layers.

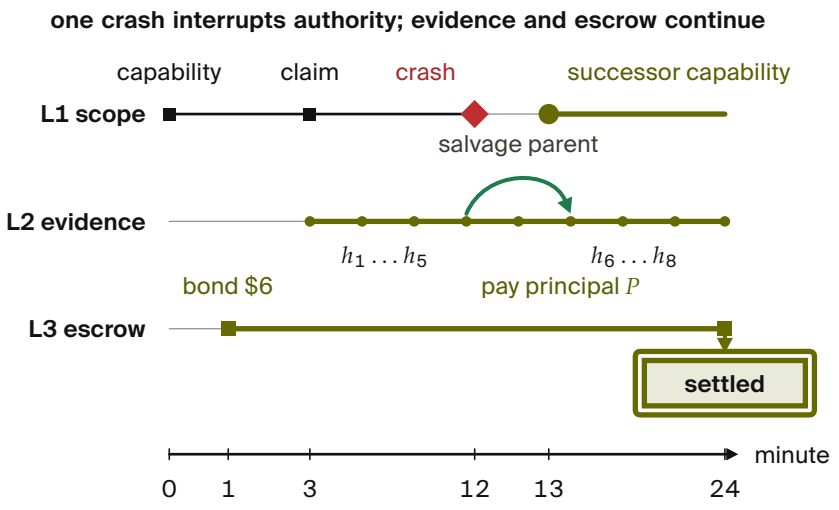


Figure 7.16. Three layers share one wall-clock axis. Process authority ends at minute 12 and a successor receives a new capability at minute 13. The evidence chain bridges the crash through a salvage-parent edge, while the pre-funded $6 escrow remains live until settlement at minute 24. The crash therefore disposes of identity without disposing of attribution or cleanup exposure. [internal]

Phase A: Setup (minutes 0–1)

Before α writes a single byte, two artifacts must exist: a capability that bounds what α can touch, and a bond that prices what α might break.

Step 1: Capability issuance (Layer 1). P requests a Harbor Card for α via the daemon, specifying the capability set {`fs:read:repo`, `fs:write:auth.ts`, `fs:write:tests/**`, `cmd:test`} and a TTL of 30 minutes. The Anchor Protocol (§7.2, accompanying chapters) signs the card. The effect is structural rather than advisory: α can prove its identity and capabilities to any verifier, but it *physically* cannot write to, e.g., `database.ts`—attempts return a capability error before the file system is even touched. This is the "physically" that makes the Layer 1 promise non-negotiable.

Step 2: Bond posting (Layer 3). With the capability in place, the Bonded Advisor (§7.6.5) quotes a bond proportional to the cleanup cost: $B_\alpha = \$5 \cdot (1 + s) = \$6$ at slope $s = 0.2$. P posts $6 into escrow. The settlement preconditions are now fixed up front: (1) acceptance criteria met ⇒ $6 returned, (2) partial completion ⇒ pro-rata, (3) sabotage ⇒ $6 liquidated to the commons treasury. The bond is the conversion of α's "character" into a settled amount; once it is posted, the question of whether α is trustworthy no longer determines settlement.

Phase B: Work begins (minutes 1–12)

Phase A's contracts are now in force. α does the work the principal hired it for, and the daemon records what happens.

Step 3: Advisory claim (§7.4). α claims `auth.ts` via `pd session files add auth.ts`. The daemon's conflict graph is empty for this path, so the claim succeeds with no conflict notification; had another agent already held the path, α would have been told and would have decided locally whether to coordinate or wait. The claim's TTL is bounded by the Harbor Card's—30 minutes—so a crashed α cannot squat on the file indefinitely.

Step 4: Evidence chain begins (Layer 2). α opens the work with a scope note: *"Scope: `auth.ts`. Hypothesis: cookie-domain handling on subdomain login fails. Validation: `npm test login.regression`."* The note's content hash h_1 chains into the session's Merkle root $R^{(1)}_{\text{session}}$, and from then on every action α takes—a read receipt for `auth.ts`, a diff blob for the partial fix, the stdout of the failing test run—is appended as a new entry. Five notes deep, $R^{(5)}_{\text{session}}$ is the canonical commitment to α's state of work. The chain is the part of the architecture that survives anything that happens next.

Phase C: Crash and recovery (minutes 12–14)

The crash is where the architecture stops being an abstraction. In Phase B everything went right; in Phase C something goes wrong, and the layers either earn their keep or they don't.

Step 5: Mid-task crash (§7.5). At minute 12, α's process crashes—let's say the LLM provider timed out. The daemon's heartbeat monitor observes α as unresponsive after 90 seconds and marks the session abandoned. α's claim on `auth.ts` releases on TTL; the evidence chain is sealed at $R^{(5)}_{\text{session}}$ and committed into the harbor root $R_{\text{harbor},\ell}$ at the next epoch boundary. *The recorded artifacts survive*: the partial diff, the test output, and α's scope note are all in the chain. The crash terminated a process; it did not terminate the work.

Step 6: Successor agent. P spawns a fresh agent β with a new Harbor Card and the same capability set. β's first action is `pd salvage`: the daemon returns the abandoned session, the scope note, the partial diff, and the last test output. β reads the scope note ("cookie-domain handling on subdomain login"), inspects the partial fix that α left behind, and finishes the work. The new evidence chain $R^{(\beta)}_{\text{session}}$ explicitly references $R^{(5)}_{\text{session}}$ as its salvage parent. Attribution is preserved across the identity change (§7.3.4)—the diff that ships at minute 24 is traceable back to the same principal P that posted the bond at minute 1, even though the agent that did most of the work is gone.

Phase D: Settlement (minute 24)

The work is done. The daemon now decides what was actually accomplished and acts on the settlement preconditions fixed in Phase A.

Step 7: Settlement. β's final action is `pd done`. The daemon's acceptance-criteria checker runs the regression suite, confirms `login.regression.test.ts` passes, confirms no other test regresses, and confirms the diff is bounded to the two files named in the manifest. The card allows the wider `tests/**` path, so this manifest check is a separate acceptance condition. All three criteria met. The daemon releases $6 from escrow to P, signs a settlement record over the evidence-chain root, and appends that record to the harbor Merkle forest (§7.3.2). The settlement is now itself an immutable, attestable artifact.

What did each layer do?

- *Layer 1 (structural)* kept the agent inside the declared blast radius. The crash, the recovery, and the settlement all happened inside a capability-bounded scope. A capability error at minute 7 would have been a fast-fail, not a corrupted repository.
- *Layer 2 (attribution)* kept the work durable across the crash. Without the Merkle-chained evidence, β would have started from zero; with it, β resumed at minute 12's state.
- *Layer 3 (economic)* kept the principal honest in scoping. P posted real collateral and recovered it on completion; had

α (or β) sabotaged, the cleanup cost would have been pre-funded.

What did *not* happen. The daemon never decided which agent should hold the file, never decided whether the fix was "good enough" in a subjective sense, and never punished the crash. It enforced what *cannot* be (Layer 1), recorded what *did* happen (Layer 2), and settled against the published criteria (Layer 3). The substantive judgments—"is this scope right", "is this fix correct"—remained with P and the acceptance-criteria checker. This is the Sen-grounded posture of §7.4 made operational.

7.C Scope Boundary Checklist

Section 7.11 (*Discussion and Limitations*) and Appendix 7.A's own limitations paragraph carry the substantive boundary discussion; this closing checklist exists so a reader who jumped straight to the appendices does not have to hunt through the body for the three bounding assumptions that recur most.

- **Threat model exclusions:** Collusion across all independent subsystems (e.g., the logging daemon, the arbitrator, and the primary execution runtime) is assumed out-of-scope; if all enforcing components are compromised, the guarantees degrade to advisory.
- **Performance trade-offs:** Cryptographic assertions and WAL-checkpointing for per-write durability incur measurable I/O overhead. These mechanisms are designed for high-stakes coordination, not for bulk data processing or high-frequency trading.
- **Implementation maturity:** While the core protocols (Anchor, SQLite single-writer) are implemented and running in production, surrounding ecosystem components (like the decentralized judge markets or fully federated multi-sig routing) remain in partial or designed phases.

None of the three is a flaw the chapter is unaware of; each is a structural reality of building zero-trust coordination on top of POSIX

primitives, already priced into the hedged claims made where each mechanism was introduced.

What *The Federated Harbor* receives The local model now separates authorization, observation, adjudication, and settlement, with explicit conditions for each. The final chapter asks which capabilities and records may cross to a second operator without giving either machine authority over the other.

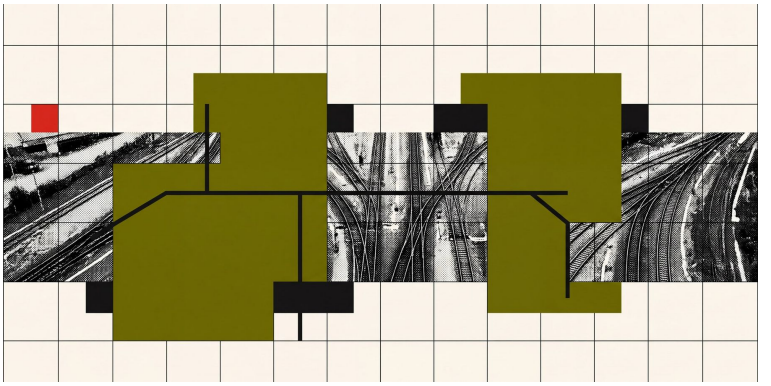
8

The Federated Harbor

The question this chapter answers

What can cross a trust boundary without moving the authority itself?

Mutually distrustful machines relay evidence, not sovereignty: what may gossip, what needs an authority point, and how a grant crosses a boundary.



We reject: kings, presidents and voting. We believe in: rough consensus and running code.

David D. Clark, A Cloudy Crystal Ball: Visions of the Future, 1992

What can cross a trust boundary without moving the authority itself? The Relay design carries harbor-scoped events over outbound HTTPS and SSE; event delivery alone neither replicates daemon state nor establishes consensus between operators. Local admission remains local. R6 and CR-1/2/3 then study a different question: under which observation and topology assumptions can relayed reports certify an inconsistency?

8.0.1 Harbor sovereignty as a property characterization

In Bonded, the Admissible KMS was characterized by a list of properties rather than by a specific construction (per-machine signing key, per-machine SQLite, etc.) so that the argument carried over to any concrete instantiation. We adopt the same posture here.

Numbers by Hand 8.0.1 (Sovereignty over the Card Is Not Sovereignty over What It Touches).

Consider this case: Alice's `db:write` card is well-formed under the Anchor Protocol, and Bob's daemon refuses it anyway, because Bob's daemon has never rooted Alice's signing key. Check the checklist below against this one case before reading it formally. D_A (Alice's daemon) is the only party that minted the card, the only party that could have attenuated it before issue, and the only party that can revoke it — three of the five items already visible in one refused request. The failure also shows item 4 by its absence: acceptance is D_B's call alone, and no amount of well-formedness at A can compel it. Once the transfer ceremony of §8.2 lets D_B mint a derived card, sovereignty over what that derived card does to B's own database still belongs to D_B, never to D_A — exactly the boundary the property characterization below draws.



Like a visitor pass presented at a locally controlled turnstile, a valid foreign credential does not compel the receiving harbor to admit it.

Design Invariant 8.0.2 (Harbor Sovereignty).

A daemon D_A is *sovereign over harbor* A if it has exclusive authority over five things:

1. **Issuance.** Only D_A can mint a Harbor Card using A's private signing key sk_A; its root signature verifies under the corresponding public key pk_A.
2. **Attenuation.** Only D_A can apply a local attenuation predicate att_A that restricts an inbound capability before it is verified by an A-side agent.
3. **Revocation.** Only D_A can decide that a card it issued is revoked. The decision becomes globally visible by the gossip protocol of §8.3; D_A does not control when other harbors learn of it, but it controls whether the revocation event exists.
4. **Acceptance.** D_A alone decides which inbound cards from other harbors to accept and under what local policy. No external party can compel acceptance.
5. **Evidence binding.** The epoch root $R_A^{(e)}$ that D_A publishes to the witness log is signed under sk_A and binds the entire A-side state of issued, attenuated, and revoked cards up to epoch e. No other harbor can publish a root that purports to be A's.

This is a deliberately narrow characterization, intended to hold of any conforming daemon implementation rather than proved from a formal model. It does *not* say that D_A is sovereign over everything an A-side agent does; the agent may be operating against a database hosted at B, in which case B-side policy applies and D_A has nothing to say about it. Sovereignty is over the *issuance and revocation surface of A's own cards*, not over downstream effects.

We will use Design Invariant 8.0.2 as the discipline against which subsequent protocols are checked. The transfer ceremony of §8.2 produces a card valid at B; verifying a foreign signature does not itself transfer sovereignty. What matters is that B independently decides whether the foreign evidence satisfies its own admission policy. The ceremony gives subsequent use a local issuer while retaining a dependency on the foreign grant's validity.

8.0.2 Inter-harbor scope

The dual notion is equally important. *Inter-harbor scope* is the scope of operations whose effects cross the trust boundary between sovereign harbors.

Design Invariant 8.0.3 (Inter-Harbor Scope).

An operation o has *inter-harbor scope* (A, B) if its execution causes a state change observable to B under authority that originated at A. Examples: an A-issued card used to write to a B-hosted database; a settlement clearing a bond posted at A against damage measured at B; a revocation issued by A becoming visible to a B-side verifier. This boundary is a scoping decision this chapter's protocols are built to respect, not a theorem derived from a model: every later claim about what a protocol closes or defers (§8.1 onward) is checked against exactly this line.

Most operations are *intra-harbor*: minting, verifying, attenuating, and revoking cards under one harbor's authority, observed by agents under that same harbor. *The Anchor Protocol* develops these intra-harbor authority checks under its stated model and implementation boundaries. The Federated Harbor's contribution is exactly the inter-harbor surface: every protocol that determines its guarantees lives at the boundary.

8.0.3 The witness log

The witness log is the device that makes the federation auditable without making any harbor sovereign over the others. It plays the same role that the Merkle Forest plays for daemon evidence in *The Bonded Commons* (§7.3.2): an accumulator whose inclusion and append-only consistency proofs can be checked against authenticated roots. These checks do not establish that the publisher recorded every relevant event or that a recorded claim is true.

Design Invariant 8.0.4 (Witness-Log Admissibility).

A witness log W is admissible for the Federated Harbor if it provides:

1. **Append-only.** Once W records an entry, W cannot retract or reorder it.
2. **Per-harbor latest-root binding.** For each harbor X in the federation and each epoch e, W records at most one root $R_X^{(e)}$ signed under sk_X.
3. **Auditable consistency.** Any third party can verify, given $R_X^{(e_1)}$ and $R_X^{(e_2)}$ with $e_1 \le e_2$, that $R_X^{(e_2)}$ is a Merkle-consistent extension of $R_X^{(e_1)}$.
4. **Equivocation evidence.** If an auditor obtains two differently signed roots for the same harbor and epoch, it can verify the contradiction immediately. Disseminating both views to a common auditor depends on the overlay and partition model (Theorem 8.3.1).
5. **Threshold publishability.** The act of publishing $R_X^{(e)}$ to W does not require D_X to trust any other party; the cross-signatures collected from other daemons in the federation are an auditor convenience, not an authorization gate.

This list is intentionally close in shape to the five properties of the Admissible KMS in *The Bonded Commons* ("Federated Sovereign"). The Bonded paper used the same posture — characterizing the property surface, then exhibiting a concrete construction that satisfied it — and we follow that pattern. Section 8.7 sketches one concrete instantiation built on Certificate Transparency-style append-only logs cross-witnessed in the Sigstore/Rekor pattern; the substrate need not be unique.

Laurie et al. 2013

Zachary Newman, John Speed Meyers, and Santiago Torres-Arias. Sigstore: Software Signing for Everybody. In *ACM Conference on Computer and Communications Security (CCS)*, 2022.

8.0.4 What the federation is not

It is worth saying what the federation is not, so that the rest of the paper does not have to keep refuting positions it never took. The federation is *not* a consensus protocol. There is no global agreement on the order of events; there is per-harbor monotonicity (epoch roots only extend), there is cross-harbor witnessing (each root is published to a shared log), and there is detectable equivocation. None

of these require harbors to agree on a global order, and the prior papers' designs explicitly avoid that overhead. The federation is *not* a hub-and-spoke topology with a privileged coordinator. The witness log is replicable; in practice a small set of mutually-witnessing log instances suffices, and any harbor can read from any of them. The hub-and-spoke failure mode — one well-funded coordinator captures the federation by becoming everyone's only witness — is a real risk we discuss in §8.8, and the structural pushback is gossip-based witness redundancy. The federation is *not* a permissionless market in identities. New harbors enter through the admission ceremony of §8.5, which requires either a pre-existing bond or an existing harbor sponsor. This is by design; the cost is that the federation is invitation-bounded, the benefit is that Sybil attacks against the federation cost the attacker as much per identity as honest entry does. Figure 8.1 returns to Alice's refused database request: a card can be valid at its source and still be unacceptable at its destination. Bob decides what may touch Bob's database.

Douceur 2002

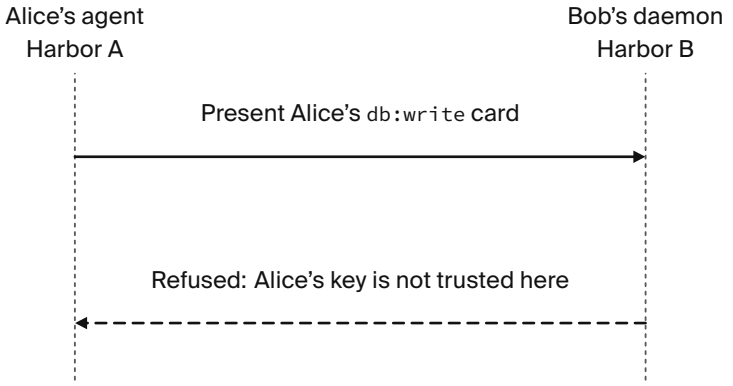


Figure 8.1. Alice can issue a valid card; she cannot make Bob accept it. This refusal precedes the transfer ceremony. [illustrative]

With the boundary in view, the recall prompts separate sovereign powers from downstream consequences.

Recall.

1. Without looking: name the five things Design Invariant 8.0.2 says a sovereign daemon D_A has exclusive authority over.
2. Why does Design Invariant 8.0.2 say sovereignty is *not* violated when an A-side agent operates against a B-hosted database under B's policy?
3. Name the three things this chapter says the federation explicitly is *not* (§8.0.4), and the one mechanism each rests its pushback on.

8.1 Threat Model

Dolev & Yao 1983

We make the threat model explicit before introducing the protocols so that the design choices can be checked against named adversary powers. We follow the Dolev–Yao convention extended to the federation case: an attacker controls every wire between harbors, can intercept and replay any cross-machine message, and may collude with any bounded number of harbor operators. The attacker cannot break the underlying cryptographic primitives (Ed25519 signatures, SHA-256 hashes); these assumptions are inherited from Anchor.

Numbers by Hand 8.1.1 (A Four-Step Breach, Traced).

Before naming attacker classes in general, trace one all the way through. `proofs/relay/CardRevocation.tla` models a single relay with a `revoked` set and an `everRevoked` ground-truth set, and TLC checks the invariant `RevocationFinal == badAccept = FALSE` against three configurations. **Baseline** (no adversary action available): 13 reachable states, invariant holds. **Rollback** (a `RollbackRestore` action can restore the relay's database, including `revoked`, from a backup taken before a revocation): the same 13-state search space now contains the four-transition trace

$$\text{Issue}(c) \;\rightarrow\; \text{Revoke}(c) \;\rightarrow\; \text{RollbackRestore}(c) \;\rightarrow\; \text{Accept}(c),$$

and `badAccept` is true at the end — the restored `revoked` set no longer contains c, so `Accept` (which consults only `revoked`) admits a card the relay had already promised to refuse. The trace shows revocation erased by restoring an older database.

Epoch (the fix: an external epoch counter that `Revoke` bumps and that `RollbackRestore` does not roll back, with `Accept` additionally requiring `cardEpoch[c] = epoch`): the same four-step trace no longer breaches, because the restored card's epoch is stale even though its `revoked` bit was rolled back, and TLC reports the invariant holding over a larger, 76-state space.

Three runs, one relay, one added counter: that is the entire mechanism behind "only-issuer-revokes" being trustworthy across a backup/restore mishap, and it is the reason this chapter's own per-epoch roots $R_X^{(e)}$ (§8.0.3) are an external,

monotonic anchor rather than a value the relay's own database could quietly roll back.

8.1.1 In-scope attacks

The protocols in this chapter are designed against the following attackers.

Cross-realm identity spoofing. An attacker presents a card at B claiming issuance by A, using a forged signature under sk_A. *Closed by* signature verification against A's public key as recorded in the witness log (§8.2). Reduces to Ed25519 EUF-CMA security. Bernstein et al. 2012

Replay across the boundary. An attacker captures a valid xfer_offer from A to B at epoch e and replays it at epoch $e + k$. *Closed by* the binding of $R_A^{(e)}$ into the envelope; the verifier at B checks that R_A is the current root, and a stale envelope is rejected (§8.2). The window between issuance and the next epoch is the structural attacker window, named and bounded.

Capability inflation across transfer. An attacker presents at B a card C'_B that claims authority beyond the attenuation predicate applied at A. *Closed by* the requirement that the verifier at B recompute the composed attenuation $\text{att}_B \circ \text{att}_A$ from the envelope rather than trust an intermediate claim (§8.2); reduces to Anchor's intra-harbor attenuation invariant.

Stale-revocation acceptance. An attacker at B presents a card revoked at A but not yet observed at B. Under the reliable connected-overlay model the window has expected scale $\Theta(\Delta \log m)$; during a partition it is unbounded. A bond can price a configured service-level window, not eliminate the tail.

Cross-witness equivocation. A malicious daemon D_A publishes contradictory roots to isolated witnesses. The contradiction is cryptographically verifiable once a common auditor receives both views, but the time to that event is topology- and partition-dependent. A witness bond prices confirmed equivocation; it does not create a

delivery deadline. Table 8.1 lays out all five properties against what each one prevents.

Table 8.1. Five witness-log properties, each closing a distinct federation failure. [internal]

property	what it prevents
append-only	retracting or reordering a published root after the fact
per-harbor latest-root binding	a harbor silently publishing two roots for one epoch
auditable consistency	an inconsistent history between two epochs going unverifiable
equivocation evidence	a proven contradiction requiring anything beyond the two signed roots
threshold publishability	publication itself requiring another daemon's permission

Settlement redirection or equivocation. A signed escrow promise makes misconduct attributable. Prevention requires a custody ledger whose transition API cannot name an arbitrary recipient or fee. Design Invariant 8.4.1 states the bound conditionally on that mechanism; without it, the full bond remains at risk.

Harbor membership forgery at the federation boundary. An attacker presents a card from a harbor purporting to be in the federation but that has no admission record. *Closed by* the admission ceremony (§8.5) which produces a permanent, witness-logged enrollment record co-signed by an existing harbor's sponsor bond. A claimed harbor with no enrollment record is treated as outside the federation regardless of how well-formed its cards appear.

8.1.2 Out-of-scope attacks

We are explicit about what is not closed.

Compromise of a harbor's signing key. If sk_A leaks, the attacker can issue arbitrary A-side cards until A rotates the key and publishes the rotation to the witness log. We inherit Anchor's key-rotation procedure unchanged. The federation does not close the underlying primitive; it does bound the blast radius to A's sovereign surface, which is exactly the right scope.

Sybil at the federation layer. An attacker who can pay the admission cost m times can run m federated harbors. The cost ratio between honest and adversarial admission is the decisive parameter; we discuss it in §8.5 but do not give a tight bound. This is named open work.

Cartel formation across harbors. Bonded's folk-theorem cartel-resistance argument relied on the daemon as a correlation device inside one machine. Across harbors, cartels can form offline and present a unified front to the witness log; the witness log records that all harbors agreed, which is consistent with both honest agreement and cartel collusion. We acknowledge this and state the open work in §8.10.

Centralization gravity. Historically, federated identity systems concentrate on one hub within five to ten years. The Federated Harbor's structural pushback — replicable witness logs, no central coordinator — is design discipline, not theorem. We formally classify this in §8.8.

Eve Maler and Drummond Reed. The Venn of Identity: Options and Issues in Federated Identity Management. *IEEE Security & Privacy*, 6(2):16–23, 2008.

Side channels. Constant-time signature verification, cache-side-channel resistance, and timing-attack resistance are inherited from the underlying libraries; we do not re-prove them and we note where we trust them.

8.1.3 Threat-band visualization

Figure 8.2 uses position to distinguish **Open** questions, **Conditional** arguments, and **Mechanized** evidence. These are qualitative assurance categories, not measured security scores. The cross-realm identity and replay claim remains conditional; the available single-harbor capability models do not mechanize the transfer protocol. We return to these distinctions when discussing limitations.

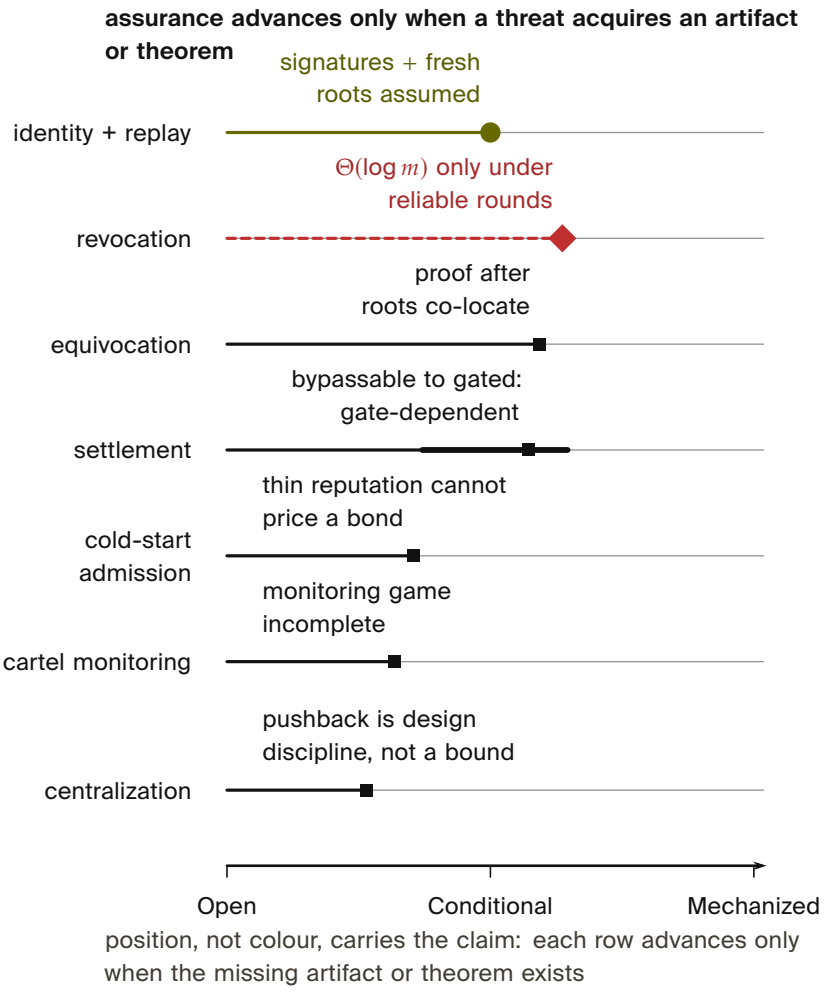


Figure 8.2. Cross-realm identity and replay remain conditional on signature checks and fresh roots; their protocol proof is pending. The available capability models concern a single harbor. Revocation and settlement depend on connectivity and custody assumptions. Admission, cartel monitoring, and centralization remain open. Positions indicate qualitative assurance. [internal]



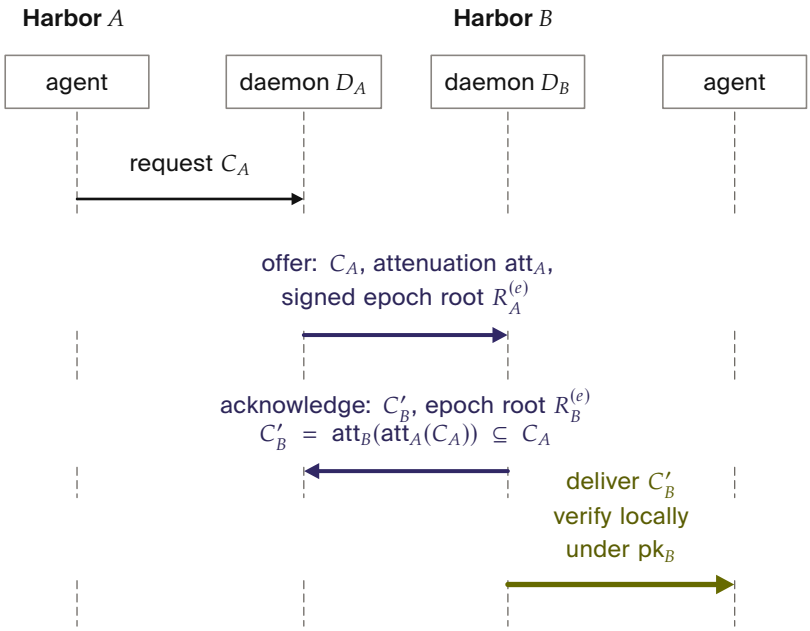


Figure 8.3. Only the offer and acknowledgement cross the boundary. The displayed subset relation assumes the same resource identifiers. A different resource needs a new grant under B's policy. Local verification uses B's public key and must enforce the parent-revocation dependency. [internal]

8.2 Cross-Harbor Capability Transfer

The first governing protocol is the cross-harbor capability transfer ceremony. An A-side agent holds a Harbor Card C_A minted by D_A under Anchor Phase 2 or Phase 3. It needs to present an equivalent capability at B. The Anchor Protocol's intra-harbor delegation already supports attenuation; we extend that to the case where the attenuation crosses the trust boundary.

8.2.1 The four-message ceremony

Figure 8.3 follows the four-message transfer ceremony. Each harbor retains its root authority; the transferred capability is scoped by the receiving harbor. The four messages are:

1. xfer_req(C_A, B) — Alice's agent presents C_A to its local daemon D_A and names B as the target harbor. The request is intra-harbor; standard Anchor verification applies.
2. xfer_offer$\langle C_A, \text{att}_A, R_A^{(e)} \rangle$ — D_A constructs an envelope binding the card, an attenuation predicate att_A specifying what subset of C_A's authority is being offered for cross-realm use, and its current epoch root $R_A^{(e)}$. The envelope is signed under sk_A and sent to D_B.

3. xfer_ack$\langle C'_B, R_B^{(e)} \rangle$ — D_B verifies the envelope's signature against A's public key as recorded in the witness log; verifies that $R_A^{(e)}$ is the current root for A; applies its own local attenuation att_B; and mints $C'_B = \text{att}_B(\text{att}_A(C_A))$, a card valid only at B, signed under sk_B. The acknowledgment is sent back to D_A along with B's current epoch root.
4. xfer_deliver$\langle C'_B \rangle$ — D_B delivers C'_B to the agent at B that will use it. Subsequent presentations of C'_B at B verify under pk_B alone — no A-side lookup is needed at the hot path.

Each derivative C'_B carries the transfer identifier of the offer that produced it, and D_B mints at most one derivative per identifier, so a replayed xfer_offer cannot produce a second C'_B; the identifier is part of what the counter-signature covers. The critical property is that after message (3), no further synchronous interaction with A is required for the agent to operate at B. The cost is that revocation of C_A at A does not instantaneously revoke C'_B at B; Theorem 8.3.1 gives an expectation only under its delivery model, and partitions require fail-closed policy or bounded credential TTLs.

8.2.2 Why this composes cleanly with Anchor

The Anchor Protocol's Phase-3 delegation already proves that a chain $C_{\text{daemon}} \rightarrow C_A \rightarrow C_B$ of attenuated cards is verifiable under the issuing daemon's public key, and that an attacker cannot inflate a downstream cap to exceed an upstream one. The transfer ceremony is Phase-3 delegation across a trust boundary, with two changes:

1. The second link in the chain is signed under a *different* root key (sk_B instead of sk_A), so the verifier at B uses B's public key pk_B, rather than pk_A.
2. The cross-realm link binds the issuing harbor's current epoch root $R_A^{(e)}$, so the receiving verifier must retain the parent identifier, check its revocation dependency against the locally accepted current state, and refuse the derivative after learning that its parent is invalid. Observing a new root without that enforcement step does not revoke anything.

The composition is what makes the ceremony cheap to add: we are not introducing a new cryptographic primitive, only a new policy at the boundary that says "the verifier accepts a card whose chain

crosses the trust boundary if and only if (a) each cross-realm link is signed under the receiving harbor's key and (b) the issuing harbor's bound epoch root is the current one in the witness log."

8.2.3 Attenuation across the boundary

Two attenuation predicates are applied in sequence. att_A is set by D_A at issue time and reflects what authority A is willing to project across the boundary. att_B is set by D_B at acceptance time and reflects what authority B is willing to grant on its own resources to a card whose root is A. Both attenuations are non-expanding; an identity restriction is permitted. The subset claim below applies to one shared universe of resource identifiers. Exchanging access to a different resource requires an additional local grant.

Numbers by Hand 8.2.1 (Composing Two Attenuations, on the Same Resource).

Take Alice's card C_A authorizing `db:write` on A's own test database, with att_A the identity narrowing (it removes nothing). D_B accepts the transfer to that same database — suppose, for this example, A's test database is itself reachable at B — and applies att_B = "only between 9am and 5pm."

The composed card is

$$C'_B = \text{att}_B(\text{att}_A(C_A)) :$$

`db:write`, time-windowed to business hours.

Every operation C'_B authorizes — a write, during business hours, on A's test database — is also one C_A authorizes, since C_A places no restriction C'_B does not already respect: subset composition is a subset, exactly as the Theorem below states it in general.

Exercise 8.1 asks what changes in this argument the moment att_B names a database C_A never mentioned at all.

Theorem 8.2.2 (Attenuation Monotonicity across the Boundary).

For any cross-realm card $C'_B = \text{att}_B(\text{att}_A(C_A))$, the set of operations authorized by C'_B is a subset of those authorized by C_A under the federated authorization rule.

Sketch.

Each attenuation is a subset operation in the Anchor algebra; subset composition is a subset.

Cross-realm policy at B adds the constraint "and the operation must be permitted by B's local policy," which can only further restrict. See the Anchor accompanying chapter's capability-attenuation section for the intra-harbor argument; the cross-realm case adds no new mathematics, only a new applicable attenuation step. □

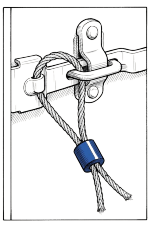
Exercise 8.1, p. 706.

8.2.4 Threats foreclosed and threats deferred

The ceremony closes only the threats implied by those bindings; the remaining threats are deferred to other primitives. Specifically:

- **Foreclosed:** identity spoofing of sk_A across the boundary; capability inflation across the transfer; replay at B against a stale R_A.
- **Deferred:** revocation latency (handled in §8.3); settlement disputes (handled in §8.4); admission of unbonded new harbors (handled in §8.5).

We are explicit that the ceremony does not close every threat in one step. Federation is a layered defense; this is one layer.



Like a tamper-evident customs seal clamped over cargo in transit through foreign territory, capability grants traverse intermediary relay harbors without disclosing confidential payloads.

8.2.5 Mechanization status

The four-message ceremony has a ProVerif query sketch, not an executable cross-realm model or a verification transcript (Appendix 8.B). Authenticity and witness-log freshness therefore remain unmechanized. The set-inclusion argument in Theorem 8.2.2 applies within its shared resource universe; it does not establish protocol authenticity or freshness.

Recall.

1. Without looking: what does message (2) of the four-message ceremony bind, and why does that binding matter more than any single signature check?
2. What is the current mechanization status of the cross-realm authenticity query, and what specific assumption is it still conditional on?
3. Name one threat the ceremony forecloses and one it defers, and which later section closes the deferred one.

8.3 Federated Revocation

The second governing protocol is the federated revocation mesh. The Anchor Protocol handles revocation inside a single mesh by gossiping an **Append-Only Event Log of Revocation IDs** using Vector Clocks for anti-entropy reconciliation: pairs of daemons periodically reconcile their event logs, designating the cuckoo filter F purely as a local, ephemeral read-path index. The Federated Harbor extends this to the multi-administrative-domain case.

Demers et al. 1987

Two pieces are added on top of the Anchor mechanism: (a) per-epoch roots $R_X^{(e)}$ published to a shared witness log so a third party can audit any harbor's local view, and (b) cross-witness signatures so the auditor pattern generalizes Certificate Transparency.

Laurie et al. 2013

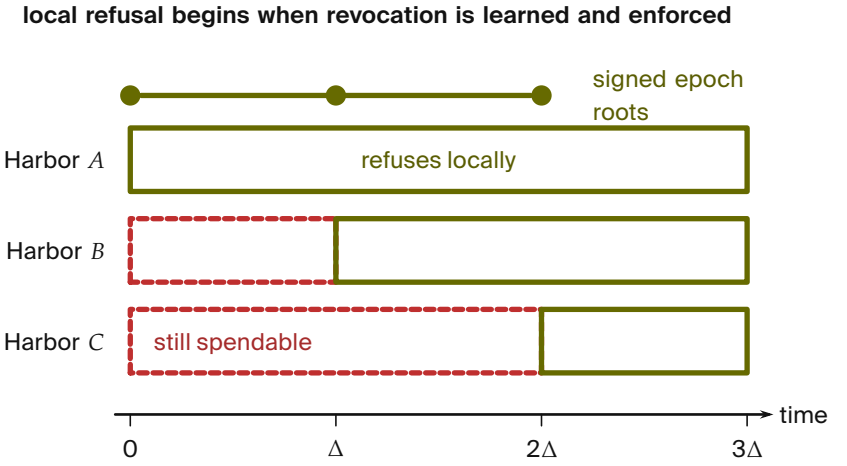


Figure 8.4. One illustrative delivery trace, not an expectation or bound: A refuses immediately, B after Δ, and C after 2Δ. A partition can extend staleness without bound. [internal]

gold marks learned and enforced revocation; red marks stale acceptance in this constructed trace, with no earlier expiry or fail-closed gate

8.3.1 Convergence bound

Standard epidemic dissemination gives an expected $\Theta(\Delta \log m)$ completion time only under an appropriate connected, reliable-round model. The property below instantiates that model at the federation level, treating each harbor's filter as one node in a complete overlay regardless of how many local daemons participate in the harbor's mesh. It is a model-conditional expectation, not an inherited wall-clock deadline. Demers et al. 1987

Theorem 8.3.1 (Conditional Revocation Dissemination).

Let m harbors form a complete overlay. In each reliable synchronous round of duration Δ, informed harbors contact independently uniform peers and exchange revocation deltas. If harbor X publishes revocation c at t_0, then:

1. **Revocation dissemination.** All harbors are informed in expected $\Theta(\log m)$ rounds under the stated epidemic model.
2. **Equivocation verification.** Any auditor possessing two differently signed roots for the same (X, e) detects equivocation in $O(1)$ signature and equality checks.
3. **No hard deadline.** If delivery can be lost indefinitely or the overlay can remain partitioned, no finite worst-case dissemination or common-auditor detection bound exists.

The first item is the standard asymptotic epidemic-dissemination result; the asymptotic notation does not supply an exact constant or a finite-size expectation. The second follows from the signed-root format. The third is an indistinguishability result: an auditor isolated in one partition cannot distinguish equivocation from delay.

Design Invariant 8.3.2 (Service-Level Attacker Window).

If an operator additionally imposes a partition bound T_p and a delivery service level T_g, then stale-card exposure is bounded by $T_p + T_g$.

Without those operational assumptions the protocol supplies an expected latency under its model, not a structural maximum. The bound is a direct consequence of Theorem 8.3.1 once those

two operational commitments are added; it is the operator's own T_p and T_g, not anything the formal model proves on its own, that makes the bound a maximum rather than an expectation — the same conditional shape as Design Invariant 8.4.1.

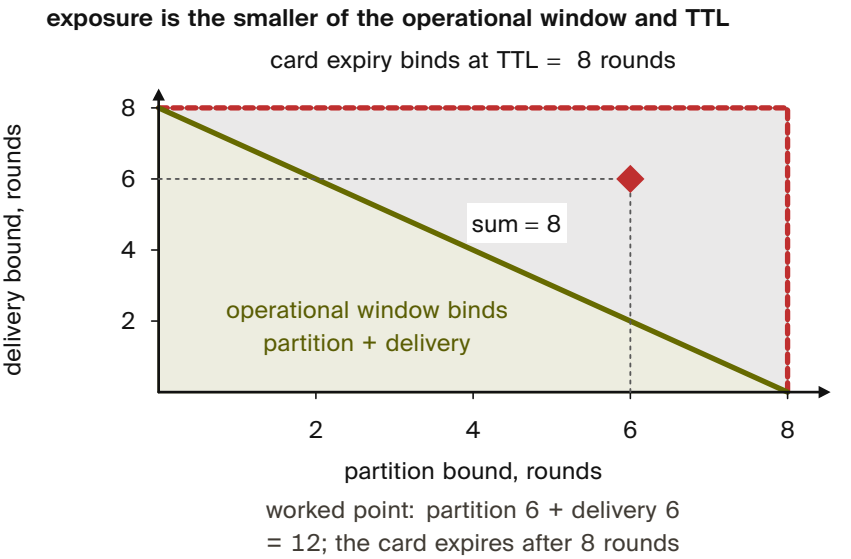


Figure 8.5. Expiry closes the window when operational guarantees are weaker. Without a partition bound, only expiry remains. The eight-round TTL is illustrative, not a measured deployment setting.

The finality that "only-issuer-revokes" rests on is model-checked with a negative control. Under the rollback configuration the relay's revocation set is restored from a backup taken before the revocation, and TLC finds the four-step breach; the epoch configuration closes it (§8.0.3).

```
>_
$ java -cp tla2tools.jar tlc2.TLC -config \
> CardRevocation_rollback.cfg CardRevocation.tla
Error: Invariant RevocationFinal is violated.

State 2: <Issue(c2)>
  issued = {c2}; epoch = 0; revoked = {}

State 3: <Revoke(c2)>
  epoch = 1; revoked = {c2}; everRevoked = {c2}

State 4: <RollbackRestore(c2)>
```

Listing 8.1. rollback clears the local deny set but not the witnessed history

```
  epoch = 1; revoked = {}; everRevoked = {c2}; rbUsed
    ↪  = 1

State 5: <Accept(c2)>
  revoked = {}; everRevoked = {c2}; badAccept = TRUE

35 states generated, 23 distinct states found, 9
    ↪ states left on queue.
The depth of the complete state graph search is 5.
[recorded excerpt; unchanged fields omitted]
```

The distinction determines the exposure bound. A bond can be priced against an operator-declared service-level window, but an unbounded partition creates an unbounded exposure tail. Deployments must therefore combine bond sizing with expiry, fail-closed acceptance, or an explicit partition limit.

8.3.2 Conflict resolution across harbors

The interesting case is disagreement. Harbor A has revoked card c; harbor B has not yet seen the revocation; an agent presents c at B for an action that affects A's database. Three candidate resolution rules from the proposal, evaluated:

Erich Owens. The Federated Harbor: Proposal and Annotated Bibliography. Curiositech LLC working document, May 2026.

1. **Pessimistic (anyone-revokes-is-revoked).** The card is revoked iff any harbor has revoked it. Defended against by an attacker who would gain by injecting spurious revocations into the gossip mesh; rejects too many honest cards in equilibrium.
2. **Authoritative (only-issuer-revokes).** Only A can revoke A's cards. This is the rule we adopt. Under the service-level assumptions of Design Invariant 8.3.2, it gives an operator-priced attacker window; without them, exposure has no finite structural maximum.
3. **Epoch-bounded (revocation propagates within Δ epochs and inconsistency is a contract violation).** A refinement of (2) for the case where one harbor falls persistently behind. We treat this as a degraded mode of (2) rather than a separate rule.

We adopt rule (2). The pessimistic rule has the wrong defaults; the epoch-bounded rule is operationally the same as (2) with a

watchdog. Adopting (2) means the Conservation invariant of §8.4 can be stated cleanly.

Table 8.2 lays out all three rules against what each defends against.

Table 8.2. Three candidate revocation-conflict rules, evaluated against the same disagreement (harbor A has revoked card c; harbor B has not yet seen it). The adopted rule is the only one with a bounded attacker window under stated assumptions. [internal]

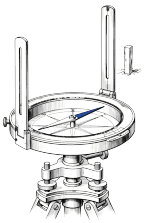
rule	what it defends against	why rejected / adopted
pessimistic (anyone revokes)	an issuer that refuses to revoke	rejects too many honest cards; an attacker injects spurious revocations into gossip
authoritative (only issuer revokes)	**spurious revocations from a non-issuer**	**adopted: an operator-priced attacker window under stated service-level terms**
epoch-bounded (Δ-epoch deadline)	a harbor persistently falling behind	a degraded mode of authoritative with a watchdog, not a separate rule

Exercise 8.2, p. 706.

8.3.3 Event Log & Ephemeral Cuckoo Discipline

Two Bloom filters over the same universe merge by a bitwise OR; two cuckoo filters cannot, because each stores a fingerprint in one of two hashed buckets that can overflow, so a union may have nowhere to place a fingerprint. The cuckoo filter is therefore purely a local, ephemeral read-path index derived from the authoritative Append-Only Event Log. The filter retains local Anchor-side disciplines (load capping, pollution resistance) but cross-harbor validation is strict: a filter at B that holds $c \in F_B$ where c was revoked at A is correct only if there is a corresponding inclusion proof in $R_A^{(e+1)}$ in the witness log. A positive filter result is checked against the exact authenticated log. Rebuilding with unchanged hash functions can reproduce the same false positive; rebuilding repairs a stale or damaged cache, not every fingerprint collision. spurious revocations injected by an attacker into a peer's log are rejected on first attempt to verify against the witness log. The discipline is: *local filters are advisory read-caches; the event and witness logs are authoritative*.

8.3.4 Visible discrepancies and their certificates



Like a surveyor checking whether measured bearings close around a traverse, the residual tests consistency around visible loops. A discrepancy calls for investigation; it does not by itself identify dishonesty.

Xiaoye Jiang, Lek-Heng Lim, Yuan Yao, and Yinyu Ye. Statistical ranking and combinatorial Hodge theory. *Mathematical Programming*, 127(1):203–244, 2011.

Three harbors compare one numerical coordinate at a common snapshot. Orient the ring and add the three reported differences. A nonzero sum cannot come from three fixed vertex values: their differences would telescope to zero. This is a test of the chosen numerical encoding, not of an entire signed log. Noise, mismatched snapshots, and dishonest reports are different possible causes of a nonzero sum; the calculation alone does not choose among them.

The same least-squares structure appears in inconsistent pairwise rankings. Jiang, Lim, Yao and Ye split an edge flow into a gradient explained by global scores and an orthogonal, divergence-free residual. With two-dimensional cells, the residual further separates into curl and harmonic parts. On our one-dimensional graph there are no such cells: the cycle residual is harmonic, not a distinct curl component. Here it measures disagreement that no assignment of visible vertex coordinates explains. That is a consistency test, not a proof that those coordinates capture every possible signed-log lie.

The global consistency of federated witness logs can be modeled sheaf-theoretically, and it is worth being precise about which object carries the weight. The intuitive picture is a poset $(X, \leq)$ of administrative domains ordered by scoped visibility, assigning to each domain U the semilattice of prefix-compatible append-only logs, with restriction given by prefix projection. That picture is right about the ordering and useless for computation: a semilattice has no subtraction, no adjoint, and no spectrum, so none of the machinery below would be defined over it. The working object is therefore the *cellular sheaf* $\mathcal{F}$ of the cohomology of equivocation,[6] carried on the gossip graph $G = (V, E)$ of witnesses: witness v holds a real stalk $\mathbb{R}^{d_v}$ encoding its log state; edge $e = \{u, v\}$ holds $\mathbb{R}^{|S_e|}$, the shared prefix coordinates S_e its two endpoints both report; and the restriction maps $\rho_{v,e}$ are the coordinate selections that cut a witness's state down to what that edge can see. Over real stalks the coboundary δ, its adjoint δ^*, and the predicted edge differences $(\delta x)_e = \rho_{u,e} x_u - \rho_{v,e} x_v$ all exist. Here a *stalk* is the vector space of allowed features at one vertex or edge, and a *restriction* selects the coordinates being compared. The measured vector g_e instead

Erich Owens. The Cohomology of Equivocation: Detecting Split-View Lies in Federated Witness-Log Gossip by Sheaf Consistency. The Harbor Research Program, Paper 7, August 2026.

[6] A standalone, submission-form version of the results folded into this subsection is *The Cohomology of Equivocation* (research paper 7); the harness and the sweeps cited here are its artifacts, regenerated at seed 20260816.

comes from reports made on that channel. An equivocator may report different values to different neighbors, so g need not equal δx for any single vertex assignment. If we built the input by applying δ to one stored state per vertex, its residual would always be zero by construction. The semilattice stays as intuition; the vector-space sheaf is what the theorem below quantifies over.

Detection is stated per visibility tier, because not every edge is equally visible to an auditor. An edge is **compared** when the auditor holds its direct cross-check (pairwise detection, no cohomology needed); **relayed** when both endpoints' reports about it reach the auditor by gossip but the endpoints never reconciled it; and **severed** when no report about it crosses to the auditor at all. Write K for the compared-plus-relayed (i.e. *visible*) edge set, and for each shared coordinate c write $K_c := (V, \{e \in K : c \in S_e\})$ for the visible coordinate subgraph. The distinction between K_c and the full coordinate subgraph G_c changes the theorem, not merely the notation: a cycle that runs through a severed edge imposes no constraint, because the severed block is a free variable of the completion.

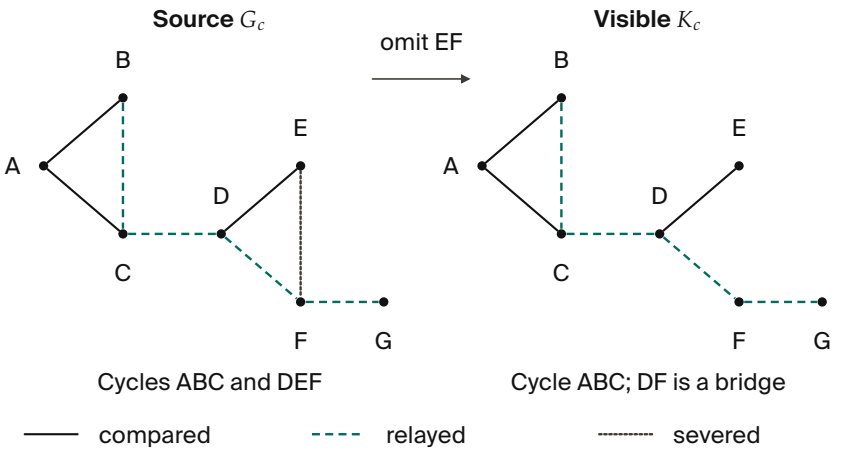


Figure 8.6. The auditor loses the DEF cycle, not its witnesses. DF remains visible but becomes a bridge. Illustrative topology: a surviving cycle does not itself establish a lie.

Numbers by Hand 8.3.3 (The Cochain by Hand: One Ring, One Chain, One Dark Link).

Four relays $0, 1, 2, 3$ in a ring, scalar stalks, one shared coordinate. The links $0–1$, $1–2$ and $2–3$ are compared and agree; the link $3–0$ is not reconciled directly, but both its endpoints' reports reach the auditor, so all four links are visible and K_c is the whole

ring. Relay 0 equivocates: the head it shows across 3–0 differs by $s = 3$ from the one it shows relay 1.

The auditor's cochain, one disagreement per link, is

$$g = (g_{01}, g_{12}, g_{23}, g_{30}) = (0, 0, 0, 3).$$

Write δx for the oriented differences predicted by candidate vertex values x, and r for the smallest Euclidean distance from g to any such prediction. The question the detector asks is whether one history $x = (x_0, x_1, x_2, x_3)$ has differences reproducing g.

Around a loop a coboundary sums to zero and g sums to 3, so none does. Least squares finds the closest: $x = (0, 0.75, 1.5, 2.25)$, with coboundary $\delta x = (-0.75, -0.75, -0.75, +2.25)$, which misses by exactly 0.75 on every link. The nearest unrestricted consistent edge assignment spreads the discrepancy evenly around the loop. This is a geometric explanation, not a reconstruction of the sender's actual lie. Hence

$$r = \sqrt{4 \times 0.75^2} = 1.5 = \frac{|s|}{\sqrt{n}} = \frac{3}{\sqrt{4}},$$

two routes to one number [verified].

Now break the loop twice and watch the same arithmetic go silent. *As a chain* $0 - 1 - 2 - 3$ with the same lie on the never-compared end link 2–3: the assignment $x = (0, 0, 0, -3)$ reproduces $g = (0, 0, 3)$ exactly, so $r = 0$, and the reader has just built the adversary's alibi by hand.

As a ring with a dark link: keep the ring and the lie on $3 - 0$ but sever $1 - 2$. Now $g_K = (0, 0, 3)$ on three visible links and $x = (0, 0, 3, 3)$ reproduces it exactly: $r = 0$ again. The loop is still there in G_c; it is gone from K_c, and the residual can only see K_c. This is the residual's visibility clause, checkable in four numbers. A direct comparison of the two received endpoint values still sees a disagreement of 3 on the relayed edge whenever both reports remain visible. The zero residual on the chain is not an all-clear for those raw claims.

Now you try: C_8, lie of size 2 on the relayed edge: $2/\sqrt{8} = 0.7071$; the same lie on any bridge, 0; the same lie on C_8 with any one of the other seven links severed, also 0 [verified].

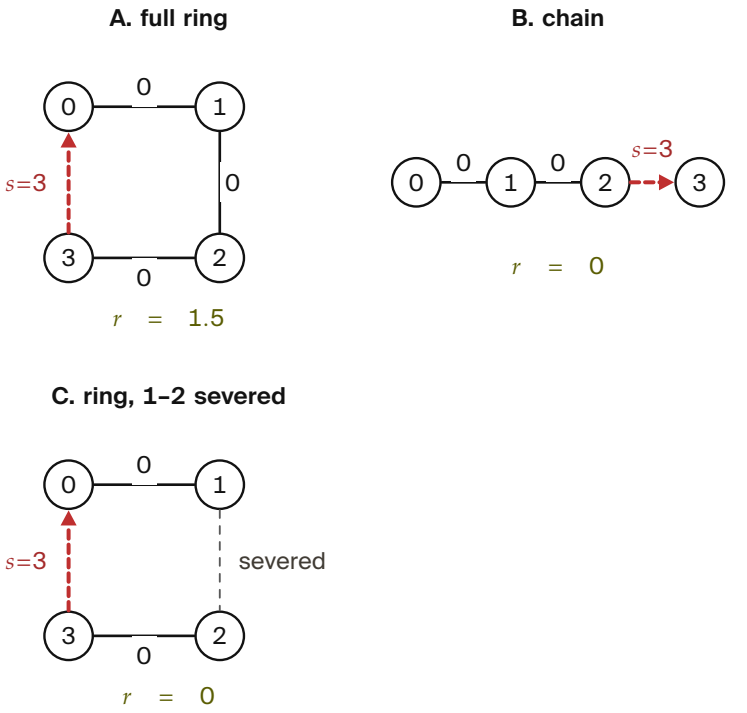


Figure 8.7. A visible loop exposes the lie ($r = 1.5$). Breaking even a different link lets the same lie pass with zero residual. Seeing the lying link alone is not enough. [verified, closed form; matches `sheaf_consistency_radius.py`]

Figure 8.7 draws all three variants side by side.

Theorem 8.3.4 (Detection iff the Missing Edge Closes a *visible* Cycle).

Assume prefix restrictions, the three-tier visibility model above, and a **single equivocator** q (the single-equivocator hypothesis is not decoration: two liars on a common cycle can cancel each other's contribution to the residual).

The detection statistic is the least-squares *completion residual*

$$r = \min_x \left\| g_K - (\delta x)\big|_K \right\|_2 = \left(\sum_c \left\| \mathrm{Proj}_{\mathrm{cycle}(K_c)} g^c \right\|^2 \right)^{1/2}, \qquad (8.1)$$

the distance to the visible coboundary image. Severed blocks impose no rows and are excluded from both the detector and its comparators. For a nonzero lie supported on one uncompared visible coordinate edge, the residual is positive if and only if that edge lies on a cycle of K_c, with the required endpoint reports relayed to the auditor. A bridge or a severed edge has no such certificate.

More generally, even one equivocator can choose several offsets whose visible sum is a coboundary: detection then occurs exactly when $\Pi_K \varepsilon_K \neq 0$, not merely because a cycle exists. Whenever $r > 0$, no global section in this real-valued prefix model explains the visible data.

On a forest every coordinate edge assignment is a coboundary, so $r = 0$. This forest statement relies on the coordinate-incidence restrictions here and is not a theorem about arbitrary cellular sheaves.

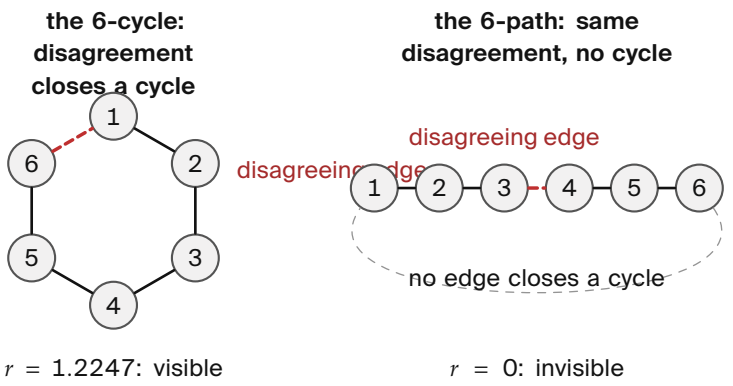
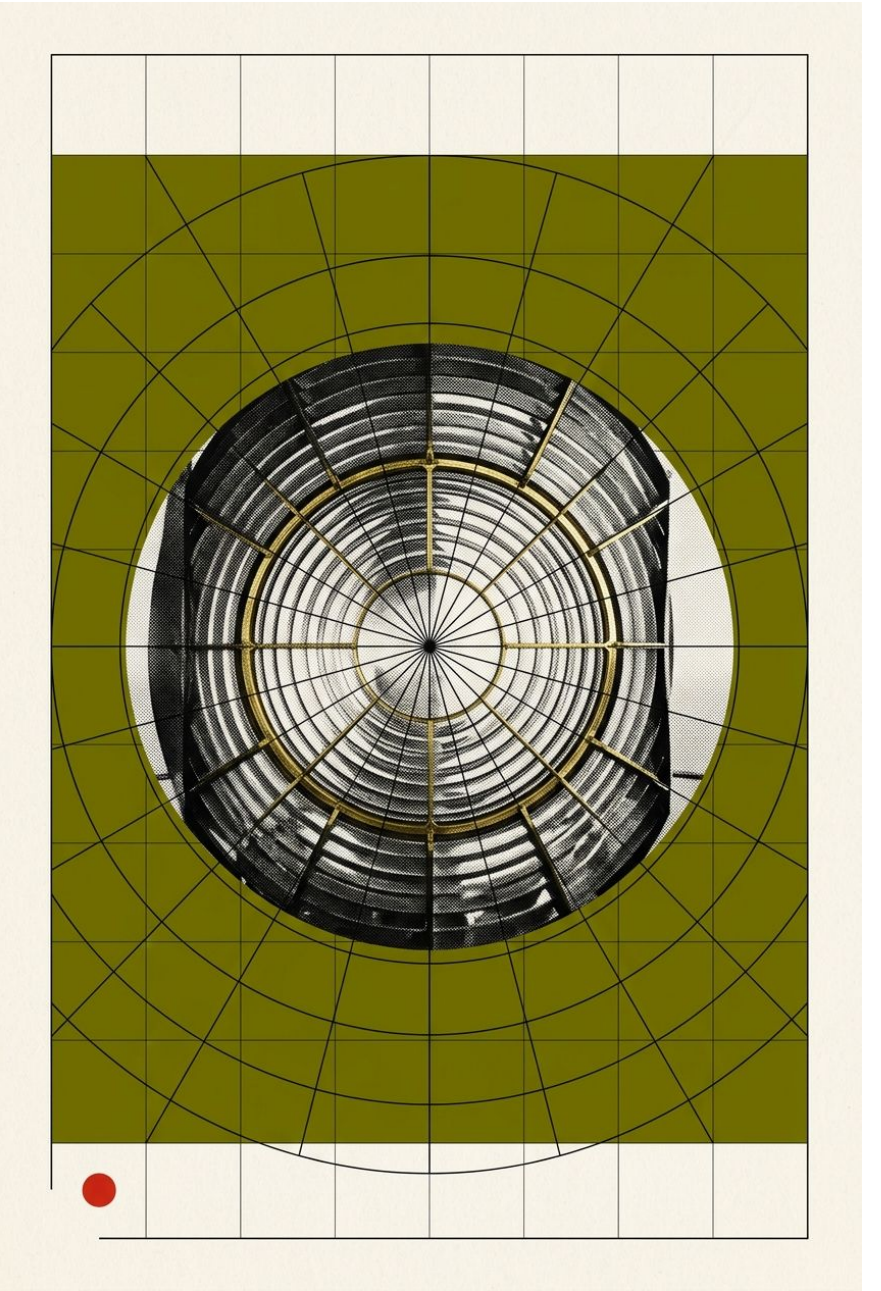


Figure 8.8. One-edge perturbation of size 3: the cycle residual is 1.2247; the path residual is zero. This is an edge-cochain diagnostic. Authenticated endpoint claims can be compared directly on either topology. Analytic construction.



Landmark 11 · Cycle Consistency & Its Limits

Cycle checks compare recorded claims under specified observation and restriction maps. A nonzero residual witnesses inconsistency in those records; it does not identify a liar or establish malicious intent. Zero residual also leaves the observation model's blind spots unresolved.

Abramsky–Brandenburger contextuality offers an analogy: local observations may fail to admit the required global explanation. We do not establish an equivalence between a signed-log deployment and their contextuality model. Nor is $H^1 = 0$ a blanket test that every arbitrary family of observations glues; compatible sections already satisfy a sheaf's gluing axiom. Our computational question is narrower and data-dependent: does the observed edge vector belong to the image of the visible coboundary? The residual r, not a topological slogan, answers that question.

Samson Abramsky and Adam Brandenburger. The operational-sheaf-theoretic approach to contextuality and non-locality. *New Journal of Physics*, 13(11):113036, 2011.

Scope. The observed cochain, not the dimension of H^1, determines the alarm. A single-edge perturbation produces nonzero residual exactly on a visible coordinate cycle; multiple offsets can cancel in the projection. A zero residual therefore says that the *edge encoding* has a consistent explanation, not that every authenticated claim or every sender is consistent.

Comparison under equal evidence. The relayed tier supplies both endpoint reports to the auditor. A direct comparator can therefore use those same reports: group authenticated claims by sender, coordinate, and snapshot, and compare the values within each group. If every group agrees, its values define a vertex assignment x, and all derived edge differences equal δx. Consequently, a nonzero residual implies a duplicate-claim conflict in the same noise-free evidence. The converse fails because forming edge differences discards information. The categorical section below gives the corresponding registered-fact test.

The synthetic harness `skills/harbor-results/scripts/sheaf_harness_v2.py` uses seed 20260816, stalk dimension 5, shared-prefix sizes 2 to 4, and 200 trials in each of nine scenario arms. On all 1,800 trials, there are 0 residual-only detections against the equal-information direct comparator and 209 direct-only detections. The residual fires on 1,101 trials. A restricted baseline that checks only reconciled edges misses 313 of those alarms, but it omits relayed reports already supplied to the residual. That comparison measures additional evidence use, not an informational advantage of cohomology. Both algorithms use only visible records; the shared snapshot and authenticated sender semantics are assumptions of this synthetic test, not a deployed telemetry result.

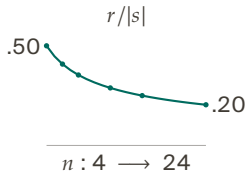
Table 8.3 records the matched-evidence counts. The same harness checks the one-edge cycle/bridge law, missing-edge masking, and structural and mutation controls. Those checks validate the stated linear computation. Useful roles for its residual remain topology audits, cycle certificates, and minimum-norm repair analysis; superior detection from the same complete claim records is not one of them.

Table 8.3. Equal-information detection on 1,800 synthetic trials. The direct checker groups sender claims; the residual receives edge differences derived from the same visible records. [verified, `sheaf_harness_v2.py`]

Outcome	Trials
Residual and direct checker both detect	1,101
Residual only	0
Direct checker only	209
Neither detects	490
Total	1,800

Noise-free synthetic model; seed 20260816. No claim of a live gossip experiment or superiority over equal inputs.

Reading effective resistance. Give every visible edge resistance one ohm. The effective resistance between an edge's endpoints is the voltage required to send one ampere between them. On a cycle, the one-ohm edge is in parallel with the other $n-1$ ohms, so $R_{\text{eff}} = (1^{-1} + (n-1)^{-1})^{-1} = (n-1)/n$. On a bridge, all current must cross that one edge and $R_{\text{eff}} = 1$. The formulas below use this unit-edge normalization. The residual has the feature coordinate's units; it is not a probability of dishonesty.



Unit-weight visible cycles, one altered edge. Longer loops certify a smaller fraction of the lie; a bridge certifies zero.

Numbers by Hand 8.3.5 (The Consistency Radius, Six-Cycle vs. Six-Path, and the $1/\sqrt{n}$ Scaling).

Six harbors in a ring (C_6), unit-weight edges, a lie of magnitude $|s| = 3$ on one relayed cycle edge. The effective resistance across that edge is $R_{\text{eff}} = (n-1)/n = 5/6$, so by hand:

$$r = |s|\sqrt{1 - R_{\text{eff}}} = 3\sqrt{1 - \tfrac{5}{6}}$$
$$= \tfrac{3}{\sqrt{6}} = 1.2247 \quad \text{[verified]}.$$

Cut the same ring into a six-node path (P_6), same edge, same lie: every edge of a path is a bridge, so $R_{\text{eff}} = 1$ and

$$r = |s|\sqrt{1 - 1} = 0$$

exactly [verified] — a tree's coboundary map is already onto the visible data, so the free completion absorbs the whole lie.

`sheaf_consistency_radius.py`'s closed-form section (`[0] CLOSED FORM vs the R6 mechanism numbers`) prints both against its own numerical solve:

```
C6 relayed cycle edge, lie 3.0:  r = 1.224745   |s|*sqrt(1-R_eff ) = 3*sqrt(1-0.8333) = 1.224745
P6 relayed cut edge,   lie 3.0:  r = ...        R_eff = 1.000000 -> predicted 0
```

matching the hand value to six decimals on C_6, and on P_6 holding the printed residual under 10^{-9} (the script's own `TOL_SILENT` zero-residual threshold) rather than printing a literal 0 [internal, `sheaf_consistency_radius.py`].

The general law is

$$r = |s|/\sqrt{n}$$

on any C_n: $|s|/2$ at n=4, $|s|/\sqrt{6} \approx 0.4082|s|$ at n=6, $|s|/\sqrt{8} \approx 0.3536|s|$ at n=8 [verified, closed form] — conviction dilutes monotonically as the ring lengthens, and any path reads as silence regardless of n.

Now you try: reproduce both printed lines and the general law against the script (Exercise 8.3), then reason about why the theorem needs a *single* equivocator (Exercise 8.4).

Exercises 8.3–8.4, p. 706.

The residual as a certificate. Robinson's consistency radius asks how far local data sits from the nearest global section; the residual r is its least-squares form under the completion over severed blocks. The new content is that r is not merely a distance. It is a lower bound on any additive correction explaining the visible data. It is attained when corrections may use every visible edge. Restricting corruption to one sender or a selected set of edges can make the least feasible correction larger, or make that explanation impossible. Topology enters through electrical network theory. A lie on edge e hides in proportion to how well the rest of the visible graph can explain it

Michael Robinson. Sheaves Are the Canonical Data Structure for Sensor Integration. *Information Fusion*, 36:208–224, 2017.

away, which is the effective resistance $R_{\mathrm{eff}}^{K_c}(e)$ measured in the visible coordinate subgraph: on a bridge $R_{\mathrm{eff}} = 1$ and the lie hides completely; on a long cycle $R_{\mathrm{eff}} \to 1$ and the certified fraction $\sqrt{1 - R_{\mathrm{eff}}}$ decays like $1/\sqrt{n}$. Severing a link raises the effective resistance of everything that was routed around through it, which is the visibility clause written as a price.

For the next statement, Π_K is the orthogonal projection that removes all edge differences explainable by vertex values. An offset vector o lists the altered reports attributed to a chosen sender q; the linear map A_q places those changes on visible edges. Thus $B_q = \Pi_K A_q$ keeps only the observable part. A vector in $\ker B_q$ causes no alarm, so the bound concerns $o_\perp$, the part perpendicular to that invisible subspace. The largest and smallest positive singular values measure the largest and smallest amplification on that observable subspace. These definitions explain why no positive lower bound can apply to the whole offset vector.

Theorem 8.3.6 (Soundness and Achievability of the Completion Residual).

For the completion-residual detector with prefix restrictions and three-tier visibility: $r > 0$ iff no global section explains the compared-plus-relayed data, and honest executions give $r = 0$ exactly, for every visibility pattern.

Every additive correction ε_K for which $g_K - \varepsilon_K \in \operatorname{im} \delta_K$ satisfies $\|\varepsilon_K\| \geq r$. Equality is attained by the orthogonal residual when every visible edge is available for correction. Thus r is the exact *unrestricted* correction norm. For a prescribed support S, the minimum over corrections supported in S is only bounded below by r; that constrained set may be empty.

For a single equivocator q with offset vector o and visible-lie operator A_q, put $B_q = \Pi_K A_q$, where Π_K projects onto the visible cycle space. If $B_q \neq 0$, then $\sigma_{\min}^{+}(B_q)\|o_\perp\| \leq r \leq \|B_q\|_{\mathrm{op}}\|o_\perp\|$, where $o_\perp$ is the component of o orthogonal to $\ker B_q$. If $B_q = 0$, every such offset is invisible and $r = 0$.

For a single-edge lie of size s in coordinate c,

$$r = |s|\sqrt{1 - R_{\text{eff}}^{K_c}(e)}, \qquad \text{on } C_n : \ |s|/\sqrt{n},$$

with the effective resistance taken in the visible subgraph K_c, not in G_c.

Proof sketch.

r is the distance from the visible cochain g_K to the image of the coboundary restricted to visible edges, the severed blocks being free.

An offset pattern is consistent with the data iff $g_K - \varepsilon_K$ lies in that image, so $\|\varepsilon_K\| \geq r$ by definition of the distance, and the completion residual is itself an unrestricted correction, which gives equality in that class only. Restricting the feasible corrections cannot lower an infimum. Honest data is a coboundary, so $r = 0$.

For the operator bounds, $B_q o = B_q o_\perp$; the smallest positive and largest singular values give the two inequalities. The projection can erase offsets that A_q itself does not erase, so its kernel cannot be omitted. For one lie of size s on e, g_K is s times the edge indicator, and the squared norm of its projection off the coboundary image is s^2 times one minus the edge's effective resistance, the standard identity between the cycle-space projection of an edge and $1 - R_{\text{eff}}(e)$; on C_n every edge has $R_{\text{eff}} = (n-1)/n$. □

Numbers by Hand 8.3.7 (A Minimum Repair Need Not Be an Admissible Lie).

On an oriented triangle, take $g = (1, 0, 0)$. Consistency requires the cycle sum to vanish. An unrestricted correction must therefore sum to 1 and has minimum norm at $(1/3, 1/3, 1/3)$, giving $r = 1/\sqrt{3}$. If only two incident edges may be altered, the minimizing correction is $(1/2, 1/2, 0)$ up to edge order, with norm $1/\sqrt{2}$. If only the original edge is allowed, its norm is 1. All three solve precisely stated optimization problems; only the first is the orthogonal projection residual. A localization claim must specify which feasible corruption class it is using.

Theorem 8.3.8 (Localization).

Noise-free, single equivocator whose offsets are on edges incident to q: the edge residual is supported on the union of visible coordinate cycles through those edges.

This locates an inconsistency, not a culpable endpoint.

Let $z = \Pi_K \varepsilon_K$ and let additive observation noise satisfy $\|\eta_K\| \le \nu$. If $\max_f \|z_f\| > 2\nu$, at least one observed residual entry on the clean support exceeds ν, while every entry outside that support is at most ν. Preserving a unique largest entry requires the stronger gap condition $\|z_{f_*}\| - \max_{f \neq f_*} \|z_f\| > 2\nu$.

Coordinated offsets that form a coboundary can cancel completely; the single-equivocator support statement is not coalition attribution.

Proof sketch.

Project each incident-edge offset onto the cycle space and take the union of the resulting supports.

Orthogonal projection is nonexpansive, so every block of $\Pi_K \eta_K$ has norm at most ν.

The reverse triangle inequality gives the detection threshold and, by applying the bound to both competing entries, the 2ν ranking-gap condition. A large maximum alone says nothing about the gap: a cycle may have several equal-magnitude residual entries. □

Theorem 8.3.9 (Cost).

Because the visible coboundary splits per shared coordinate into graph incidence operators, r is one sparse least-squares problem that decomposes into at most L scalar graph-Laplacian systems, where L is the number of registered scalar coordinates. For numerical accuracy $\epsilon \in (0, 1)$, the corresponding solver complexity has the form $\widetilde{O}(|E| \cdot L \log(1/\epsilon))$ with symmetric diagonally dominant solvers.

Bach's #P-hardness of sheaf cohomology concerns coherent sheaves on projective space, the wrong category; finite cellular sheaves over $\mathbb{R}$ on graphs are linear algebra.

Daniel A. Spielman and Shang-Hua Teng. Nearly-Linear Time Algorithms for Graph Partitioning, Graph Sparsification, and Solving Linear Systems. In *Proc. 36th ACM Symposium on Theory of Computing (STOC)*, pp. 81–90, 2004.

Eric Bach. Sheaf Cohomology is #P-hard. *Journal of Symbolic Computation*, 27(4):429–433, 1999.

Checked. In the stated synthetic test: soundness 0/600 violations; achievability 600/600 (a consistent pattern of norm exactly r constructed every time). The effective-resistance closed form is exact on C_4 to C_{12} and on 200 random graphs; the lower bound is tight under the adversarial offset ($r/\|o_\perp\| = \sigma^+_{\min} = 0.4082 = 1/\sqrt{6}$); a uniform kernel lie of norm 5 gives $r = 2 \times 10^{-14}$, a consistent alternative state rather than a split view, invisible in principle and correctly not flagged, and on a cycle of K_c the uniform offsets are the only such directions, every kernel direction lying in the uniform span to 5×10^{-16}. Localization 200/200 on the two-cycles topology and 128/128 on severed expanders; the noise boundary: at least 95% localization up to a noise-to-lie ratio of 0.1, 76.5% at 0.2. Conjugate gradient's timing slope against $|E|$ stays well under the 1.6 gate while dense solve is consistently super-linear; the decomposition, not the two digits, is what the gate certifies. Two coordinated liars on a common cycle, each alone certified at $r \geq |s|/\sqrt{8}$, jointly cancel to $r = 6 \times 10^{-15}$ [internal, `sheaf_consistency_radius.py`].

What it buys. The harness's statistic is upgraded to a certificate a federation operator can inspect: r is the smallest unrestricted additive correction to the visible numerical edge data. It may distribute changes among several senders, so it need not be an admissible explanation of one agent's behavior. Sparse solves make the computation a candidate for online use; a deployment still needs a measured latency budget. The closed form also converts topology into policy. Effective resistance is the price list for where a federation should force direct reconciliation (bridges: mandatory, since the residual cannot help there) and where a single-edge numerical perturbation has a larger certified residual (short cycles). Direct comparison remains available when the same authenticated endpoint claims are present.

Where this stops

> The one-edge detection law must not be generalized to arbitrary offset patterns: a coalition whose combined lie is a coboundary is a consistent counterfactual world, and two coordinated liars on a common cycle cancel the residual to numerical zero.
>
> r never overstates the lie, but it can understate a coalition's to zero; soundness is unconditional, detection is not. No attribution, ever: a $+o$ lie by q toward w and a $-o$ lie by w toward q produce

identical observations, so the residual localizes to cycles through the equivocator and cannot say which endpoint lied; naming culprits is the signatures-and-forensics business, which this detector complements.

Vanishing r is not an all-clear, for three independent reasons: uniform offsets may form a compatible alternative in this edge model; severed edges admit consistent counterfactuals by construction; and over $\mathbb{R}$ the obstruction is abelianized, so a vanishing real-coefficient signal does not certify gluability in richer coefficient systems. Where no report crosses, nothing can be seen, and severing one link of a loop darkens the whole loop; under measurement noise the certificate needs a threshold above the honest floor, below which it reverts to "consistent with noise."

Sheaf Laplacian dynamics — and what is *not* claimed here. On a cellular sheaf with real stalks, the **Hansen–Ghrist sheaf Laplacian** is $\Delta_{\mathcal{F}} = \delta^*\delta$, and its harmonic space $\ker(\Delta_{\mathcal{F}}) \cong H^0(G;\mathcal{F})$ is precisely the subspace of globally synchronized ledger states. This object exists only over the vector-space sheaf constructed above; it is undefined over the semilattice of append-only logs, which has no adjoint and no spectrum. The spectral gap $\lambda^+_{\min}(\Delta_{\mathcal{F}})$ — the smallest non-zero eigenvalue — governs decay on the positive eigenspace of linear diffusion. It is not generally the second eigenvalue: a sheaf can have a multidimensional kernel. **We have not derived or checked such a bound, and this chapter claims none.** The Hansen–Ghrist rate governs continuous-time linear diffusion $\dot{x} = -\Delta_{\mathcal{F}} x$ over real stalks; anti-entropy gossip over signed append-only logs with a Byzantine equivocator is discrete, asynchronous, adversarial, and monotone-join rather than linear-averaging, and nothing here shows the bound transfers. The two convergence stories — classical epidemic dissemination (Theorem 8.3.1, which is a real and separately argued result) and sheaf-Laplacian diffusion — are not reconciled, and neither is a second proof of the other. No relaxation-time bound for the federation is established, in this section or anywhere in this chapter; that is consistent with Appendix 8.A, whose dissemination rows record "no partition deadline" and "no hard delivery bound." The reconciliation is open work (Appendix 8.A, *open* band).

Peiyao Sheng, Gerui Wang, Kartik Nayak, Sreeram Kannan, and Pramod Viswanath. BFT Protocol Forensics. In *Proc. ACM Conference on Computer and Communications Security (CCS)*, pp. 1722–1743, 2021.

Giovanni Carù. On the Cohomology of Contextuality. In *Proc. 13th International Conference on Quantum Physics and Logic (QPL 2016)*, EPTCS 236, pp. 21–39, 2017.

Jakob Hansen and Robert Ghrist. Toward a spectral theory of cellular sheaves. *Journal of Applied and Computational Topology*, 3(4):315–358, 2019.

Demers et al. 1987

8.3.5 From observations to a checkable refusal

An auditor deciding whether to accept a work receipt needs to distinguish missing evidence, incompatible evidence, and evidence compatible with several possible worlds. An authentic packet identifies its issuer; its schema, fact, work item and snapshot determine which other packets it may be compared with. A digest alone does not discharge this admission step. A zero difference retains equality but erases the shared raw value. If the auditor holds authenticated endpoint claims, direct sender-claim comparison can therefore expose a split that even the exact categorical edge-only test does not retain enough information to see.

Suppose a registered fact has exactly two values, A and B. Encode them as the *one-hot* vectors $e_A = (1, 0)$ and $e_B = (0, 1)$: exactly one entry is one. In this paragraph an edge records head minus tail. A receipt $e_B - e_A = (-1, 1)$ therefore fixes its tail to A and its head to B; we call those forced endpoint values *pins*. A zero difference says only that its endpoints agree, not whether both said A or both said B. Connected zero-difference edges consequently force a whole component to share one value.

Numbers by Hand 8.3.10 (A contradiction with zero cycle residual).

The path $0 \to 1 \to 2$ reports $e_B - e_A$ on each edge. Vertex 1 must then be both B and A, a contradiction. With edge costs 2 and 3, the two receipts give a certificate of cost 5. Unrestricted real potentials can nevertheless fit this path exactly, so its cycle residual is zero. The finite-domain constraint, rather than an extra topological measurement, exposes the failure. The categorical assignment oracle and exhaustive subset oracle in `test_claim_audit.py` and `test_costed_claim_certificates.py` check these two decisions independently of the production checker. Neither authenticates the producer.

Theorem 8.3.11 (Exact categorical contradiction certificate).

Fix a nonempty finite fact domain and admitted one-hot edge differences for one fact and snapshot. A zero edge requires equal endpoint values. A nonzero difference $e_b - e_a$ pins its tail to a and its head to b. A categorical assignment exists exactly when every component of the zero-edge graph has pins of at most one value. Otherwise two incompatible pins, their source edges, and a zero-edge path form a checkable contradiction certificate. With nonnegative costs per distinct observation, the least-cost infeasible subset is obtained by minimizing shortest zero-path cost plus the cost of the set of its two pin-source edges over all conflicting pin pairs.

Proof.

Equality makes a component constant, and every nonzero one-hot difference uniquely fixes both endpoint values. Consistent pins therefore give a valid assignment by choosing a value for each unpinned component. Inconsistent pins give the stated witness. Every infeasible subset contains such a witness; replacing its zero path by a shortest one cannot increase cost. Conversely every candidate witness is an infeasible subset, so the minimum is attained. Source edges are nonzero and do not overlap the zero path; when both pins come from one source edge, charge that edge once. □

The path example uses two conflicting pins at the same vertex, so its zero-edge path has length zero. For distinct pinned vertices, a chain of zero edges carries the same equality between them. The certificate consists of those equality receipts and the two receipts giving different pins. Its cost is the sum over distinct receipts, not over every time a receipt is mentioned in the argument. This is the object minimized in the theorem.

How much redundant observation is enough? The scalar cycle example also explains why detection is easier than error identification. On an oriented four-cycle, an error $(1, 0, 0, 0)$ and an error $(0, 1, 0, 0)$ have the same cycle projection: each contributes total circulation one. Both alarm, but the residual cannot tell which edge changed.

Paper 8 states the general rule for a typed linear observation model. One *edge group* contains all coordinates of one channel; support size counts affected channels, irrespective of coordinate dimension or error amplitude. Let A map admissible vertex features to edge features, let Q project orthogonally off im A, and define $d_{\mathcal{F}}$ as the smallest number of edge groups supporting a nonzero vector in im A, or infinity when $A = 0$. This distance counts how many channel groups must change to turn one compatible prediction into a different compatible prediction. It measures redundancy in this linear encoding. Every nonzero error affecting at most k edge groups is detected exactly when $d_{\mathcal{F}} > k$; all such errors are uniquely identifiable modulo compatible data exactly when $d_{\mathcal{F}} > 2k$. Indeed, ambiguity between two k-group errors is a compatible difference on at most $2k$ groups, and every such difference can be split to construct an ambiguous pair. In the scalar connected-graph case $d_{\mathcal{F}}$ is the minimum cut size. A four-node complete graph has cut size three and identifies one arbitrary-amplitude edge-report error; a cycle has cut size two and cannot always identify which single edge erred.

Where this stops

An edge-report error is not an agentic cause or a dishonest principal. The typed model must fix feature domains, restriction maps and independent observations before applying its distance rule. A drawn face contributes no new measurement by itself. A relative cohomology obstruction concerns extension of a visible compatible section; its vanishing proves that an extension exists, not that unseen receipts agree with it. Minimum-cost certificates select known observations. Unknown-answer acquisition needs a predictive or adversarial model, and complementary receipts can have zero individual gain but positive joint gain. These distinctions let the operator request evidence or refuse a decision without inventing either guilt or certainty.

8.4 Cross-Harbor Settlement

The third governing protocol is cross-harbor settlement. A bond is posted at A to cover damage caused by an A-side agent operating at B. When damage occurs, the bond must clear to the damaged party at B. The protocol does this without making either harbor sovereign over the other, but it still uses a third-party custody

service. Its transition surface must be non-bypassably restricted; the property below states that assumption rather than relabeling the service as trustless.

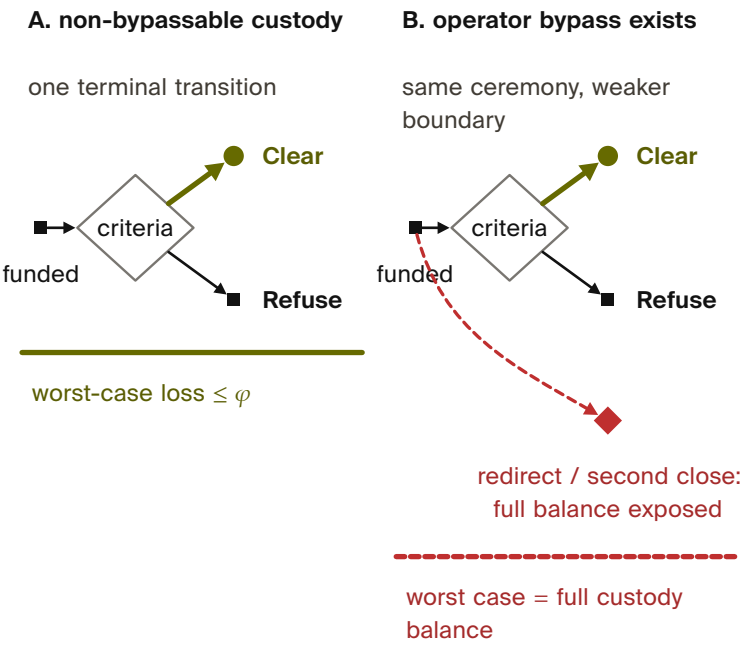


Figure 8.9. With a non-bypassable gate, recipient whitelist, and fee cap, custody closes only by payment or refund. Adding an operator-controlled escape exposes the entire balance, not merely the fee. [internal]

the custody assumption: whitelist, fee cap, one terminal transition are the theorem's hypothesis, not decoration

8.4.1 The bounded escrow

We introduce a third party—the escrow—and require the *custody ledger* to expose exactly two terminal transitions: clear (pay whitelisted beneficiary B, refund the remainder to A) or refuse (return the bond to A). A signed policy alone is only evidence. The following actions are prohibited, and those prohibitions are structural only if the service cannot bypass the ledger API or alter its code and keys:

- Redirect funds to anyone other than A or B.
- Equivocate about its clearing decision (publish "cleared" to one party and "refused" to another).
- Delay past the TTL of the bond.
- Extract more than a pre-agreed fee ϕ which is capped at issuance.

These restrictions define the desired transition system. The current artifact is a design and bounded model, not a deployed custody mechanism.

Design Invariant 8.4.1 (Conditional Escrow Extraction Bound).

Assume the custody ledger (i) admits only recipients A and B, (ii) caps the fee output at ϕ, (iii) executes exactly one terminal transition atomically, and (iv) keeps signing and transfer authority outside the escrow operator's bypassable control.

Then no allowed transition pays the escrow more than ϕ.

The bound follows by case analysis over the two allowed terminal transitions. Witness logging makes delay or equivocation detectable, and an escrow bond can compensate some misconduct, but neither fact prevents theft if the operator can bypass custody restrictions. Without assumptions (i)–(iv), the worst-case extractable value is the full custodial balance.

The escrow is a trusted third party with a *conditional transition bound*. This is not a trustless construction in the HTLC sense. Herlihy 2018 Whether useful trustless settlement is possible for non-fungible reputation-priced bonds remains open (§8.10); this chapter gives neither a construction nor an impossibility proof.

A candidate instantiation of assumption (iv). One concrete way to remove the escrow operator's *unilateral* control over transfer authority is a 2-of-3 threshold-signature scheme: harbor A, harbor B, and a federation arbiter each hold one key, and no transfer executes without two of the three signatures. When A and B agree on the outcome — the ordinary case — they co-sign the release themselves and the arbiter never touches custody. The arbiter's key exists only to break a dispute: it evaluates the witness-logged evidence trail and co-signs with whichever party it rules for. This is a plausible way to satisfy assumption (iv), and we name it because a reader will otherwise ask what such a mechanism would look like. It does not close what Design Invariant 8.4.1 calls conditional. The arbiter remains a trusted third opinion; we have not analyzed an arbiter colluding with one principal, we have not specified how arbiter selection resists capture, and the underlying settlement rail (an on-chain

threshold contract, a custodial payment hold, or anything else) carries its own assumptions that this chapter does not model. We list 2-of-3 multisig as a candidate design, not a closed result; trustless settlement for non-fungible bonds stays in the *open* band (§8.10.2, Appendix 8.A).

8.4.2 Cross-harbor conservation

The Bonded Commons chapter gives a single-machine conservation identity: posting reserves funds and clearing moves them to recipients; neither operation mints or destroys value within the accounted system. Here the same obligation extends to funds in transit between harbors.

Design Invariant 8.4.2 (Cross-Harbor Bucket Partition).

For each funded bond b of amount $a_b > 0$, let $P_b, E_b, C_b, R_b \in \{0, a_b\}$ be whole-bond *lifecycle accounts*: posted, in escrow, resolved by clearance, and resolved by refusal/full refund.

They record the status of the entire bond, not the balances of individual recipients. Valid transitions preserve

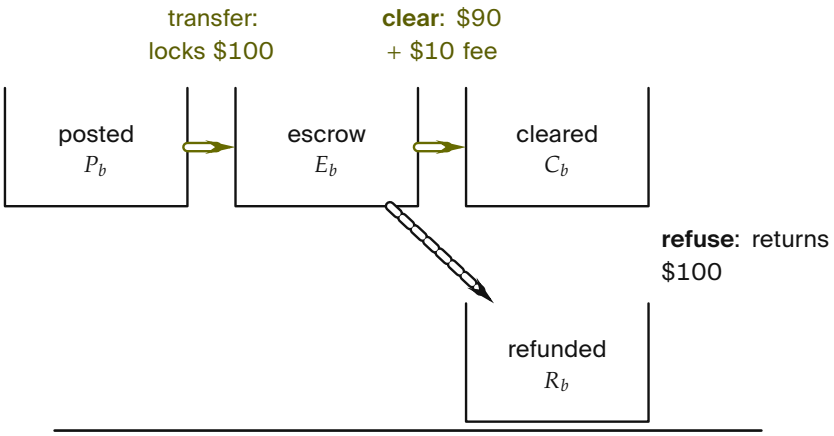
$$P_b + E_b + C_b + R_b = a_b \quad \text{and} \quad \left|\{Z \in \{P_b, E_b, C_b, R_b\} : Z = a_b\}\right| = 1.$$

Summing over bonds gives $\sum_b (P_b + E_b + C_b + R_b) = \sum_b a_b$.

A separate recipient subledger must record nonnegative beneficiary payment, source remainder, and fee summing to a_b on clearance. In particular a source remainder inside a clearance is not a second lifecycle entry in R_b. External top-ups create separately funded bonds; the one-hot status model alone does not prove that actual recipient transfers conserve money.

This is the federation-layer analogue of Conservation in Bonded. The intra-harbor Conservation invariants hold per-harbor (Bonded gives the proofs); the cross-harbor property adds the constraint that bonds in transit between harbors — i.e., posted at A, locked in escrow, awaiting clearance against B — are accounted for as in-escrow rather than as either posted at A or cleared to B. The escrow's role as the holder of in-flight bonds is what makes the conservation accounting tractable.

Figure 8.10 draws the four buckets and the pipes between them.



$P_b + E_b + C_b + R_b = 100$, and exactly one term is nonzero at any time

worked instance: $(100, 0, 0, 0) \rightarrow (0, 100, 0, 0) \rightarrow (0, 0, 100, 0)$ with C=90+10, or the refuse branch $\rightarrow (0, 0, 0, 100)$

Figure 8.10. Whole-bond status, not recipient balances. Clearance may divide funds among payment, remainder, and fee. This cross-harbor partition is a conditional design invariant. The recorded TLC result concerns the separate single-machine Conservation model; cross-harbor custody and recipient conservation need their own verification. [internal]

Numbers by Hand 8.4.3 (One Bond across Two Harbors).

A $100 bond posted at A has buckets $(P, E, C, R) = (100, 0, 0, 0)$. The transfer locks it in escrow, $(0, 100, 0, 0)$.

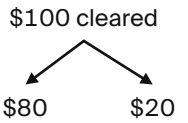
Clearance at B pays $90 to the counterparty and $10 in fees, recorded as two cleared recipients, so the row is $(0, 0, 100, 0)$ with recipient total $90 + 10$.

A partial clearance paying $80 and returning $20 to A has the same status row, with recipient total $80 + 20$; it does not have both $C = 80$ and $R = 20$.

Refusal of the whole bond instead gives $(0, 0, 0, 100)$.

The status partition and the recipient sum each need checking [internal]; neither stands in for the other.

One status; two payees



Counterparty $80
Return to A $20

The entire bond has cleared. Its recipient ledger splits the payment; it does not split the bond between status buckets.

8.4.3 Settlement protocol

Figure 8.11 shows the three-step protocol. To unpack the figure:

1. **Bond post.** Alice's harbor mints a work card for agent α; agent operates at B. Bond $B_\alpha = \$B$ is posted at A and locked in the

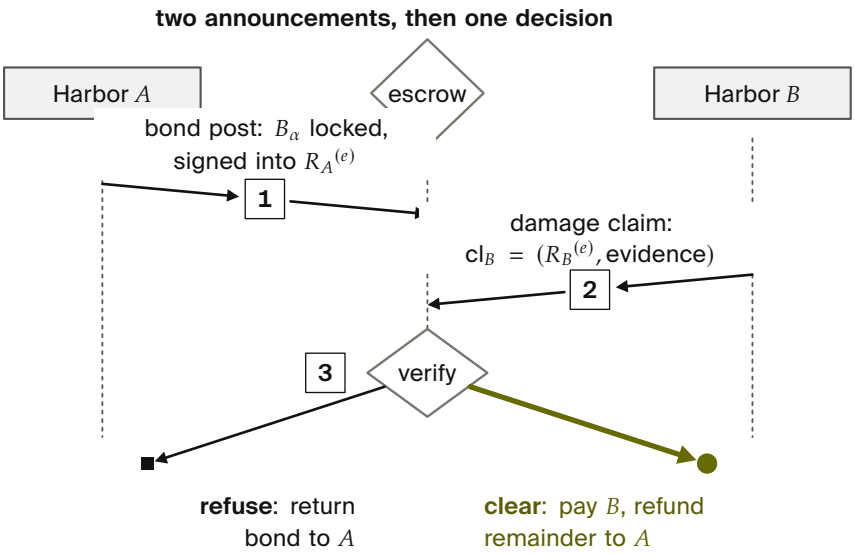


Figure 8.11. Posting a bond and announcing damage do not settle it. Escrow checks the proof against the signed conditions, then makes one terminal decision: clear or refuse. [internal]

#	step	binds / value
1	bond post	$A \rightarrow$ escrow: B_α locked, TTL signed into $R_A^{(e)}$ and $R_B^{(e)}$
2	damage claim	$B \rightarrow$ escrow: $\text{cl}_B = (R_B^{(e)}, \text{evidence})$, a Merkle inclusion proof
3	verify & clear	escrow checks the proof against step 1's terms; clear or refuse

escrow. The bond's amount and TTL are signed into both A's and B's witness-log roots so the federation knows the bond exists.

2. **Damage claim.** Damage detected at B (acceptance criteria fail, or invariant violation). Bob's harbor signs claim $\text{cl}_B = (R_B^{(e)}, \text{evidence})$ where the evidence is a Merkle inclusion proof against $R_B^{(e)}$. The claim is sent to the escrow.
3. **Verification and clearance.** Escrow verifies inclusion of α's actions in $R_B^{(e)}$, matches against the acceptance preconditions that were signed at step 1, and produces one of two outcomes: *clear* (pay B out of bond, refund remainder to A) or *refuse* (return bond to A, log reason). Both outcomes are recorded in the witness log.

The designed transition is atomic in the bucket-partition sense: a bond moves once from in-escrow to either cleared or refunded.

This property depends on the custody-ledger assumptions of Design Invariant 8.4.1; witness records alone cannot make two external transfers atomic.

Recall

1. Name the ledger's two exits and the four assumptions in Invariant 8.4.1.
2. Why can't an inclusion proof authorize clearance?
3. What does 2-of-3 signing constrain, and what can it not prevent?

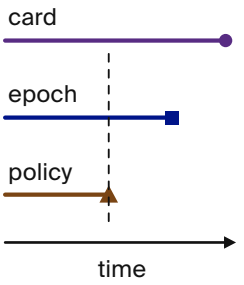
8.4.4 Fee structure and economic discipline

The escrow's fee ϕ is bounded but non-zero. In the competitive-insurance market sense of *The Bonded Commons* (the Youle contribution), ϕ is a market clearing price for the service of holding the bond and verifying the inclusion proof. We do not solve the market here; the Youle contribution from Bonded gives the mechanism, and we conjecture (open) that it extends cleanly to the cross-harbor case when the escrow population is large enough.

We are explicit that for small federations (two or three harbors), ϕ is likely set by a posted price rather than auctioned. The mechanism design here is identical in shape to Bonded's, just lifted to the federation layer.

8.5 Federation Admission

The Federated Harbor is not a permissionless market in identities. A new harbor enters the federation through an admission ceremony. Agents inside an admitted harbor are named by certificates the harbor issues, each binding a principal identifier, an agent key, the issuer, an epoch, a policy, and an expiry; the federation never sees a bare key, and a certificate outlives neither its epoch nor its policy. The ceremony is invitation-bounded by design: Sybil attacks against the federation are costly for an attacker precisely because each identity requires a real admission event.



Policy expires first in this schematic. Later card and epoch dates cannot extend the card's usable lifetime.

Douceur 2002

8.5.1 The bonded-sponsor pattern

A new harbor N with no prior reputation cannot price its own bond fairly under competitive insurance — the market lacks data on N's risk. The bonded-sponsor pattern resolves this:

Numbers by Hand 8.5.1 (Beta's Six Months, Traced through the Ceremony).

§8.6 already fixes this ceremony's one concrete run. Beta Vision had no reputation in the consortium, so Acme — an existing member — posted the sponsor bond on Beta's behalf.

Beta then operated under probation for six months: its cards were accepted and its evidence trail was witness-logged, but it was Acme's bond, not Beta's own standing, that would have been forfeit had Beta forged a signature, equivocated on a root, or refused a settled bond during that window.

Acme's risk officer spent the same six months watching Beta's evidence trail closely, per §8.6.1, before the joint work traced in §8.6 takes place.

In this constructed run, the consortium's declared graduation test accepts the six-month record and Beta posts its own funded bond before Acme's is released. Six quiet months alone do not statistically establish low risk.

That is all four items of the ceremony below, in order, with the one number this federation's own history happens to fix.



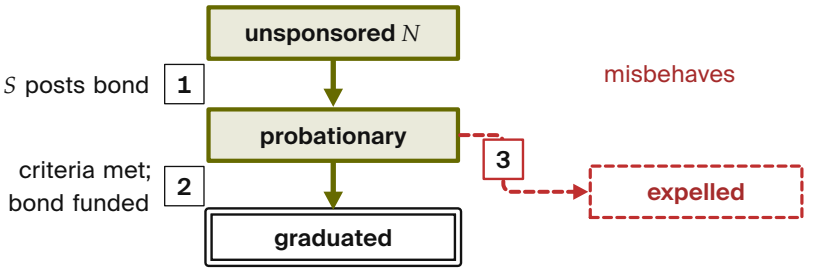
Like a port quarantine inspection lantern signaling a vessel held at outer anchorage until its health manifest clears, a new harbor is held in probationary isolation until its admission bond and epoch roots verify.

1. An existing harbor S (the sponsor) posts a bond on N's behalf, sized to cover the federation's worst-case loss from N misbehaving during a probationary epoch.
2. N joins the federation under probation; its cards are accepted, its evidence trail is witness-logged, but its sponsor bond is at risk.
3. If N misbehaves during probation — forges signatures, equivocates on roots, refuses to honor settled bonds — the sponsor bond is forfeit. The sponsor has full incentive to vet N before sponsoring.
4. At the end of probation, evaluate the declared graduation criteria. Release the sponsor bond only when the replacement coverage is funded and the contract permits release; insufficient evidence can require extended probation or exit.

This is the federation-layer analogue of competitive insurance from *The Bonded Commons* (the Youle contribution): instead of an authority picking who joins, the sponsor market discovers it. The cost of being a sponsor is real — forfeit risk during probation — and

the benefit is the network effect of having N in the federation. We expect (open) that for federations with many candidate sponsors and clear-eyed risk pricing, the equilibrium is reasonably efficient. We do not prove it.

Figure 8.12 draws the four-state lifecycle end to end.



guard	what it costs
1 sponsor S posts bond, sized to worst-case loss	S's capital at risk
2 criteria met, N's bond funded; sponsor release permitted	N's replacement bond
3 N forges, equivocates, or refuses a settled bond	sponsor bond forfeit, not N's

Figure 8.12. Shown paths from probation: graduation after funded replacement, or expulsion for misconduct. Misconduct forfeits the sponsor's bond, giving a reason to vet the newcomer. [internal]

8.5.2 Cold-start failure modes

A new harbor that cannot find a sponsor cannot join. Entry can remain restricted to parties the existing sponsors already know. The design offers three ways to reduce that barrier, but none guarantees an available sponsor:

- *Many sponsors.* A new harbor needs only one willing sponsor. Fees can compensate a sponsor for risk, but payment alone does not establish that anyone will accept that risk.
- *Small probationary scope.* New harbors during probation are restricted from large-stake operations; the sponsor's risk is bounded.
- *Witness-log permanence.* The admission record is permanent; once a harbor has graduated probation, it does not need a sponsor again.

These are design choices, not an openness theorem. Running several witness logs permits independent inspection only if their operators and accepted roots are actually independent. It does not establish sponsor availability, low switching costs, or resistance to concentrated control. A deployment must measure those properties; §8.8.1 separates the configurable topology from that empirical obligation.

8.6 Worked Example

The four primitives are easier to read against a concrete case. We work the example end-to-end: two organizations, three machines, one shared project. The example is intentionally familiar; nothing in it is novel except the way the primitives compose.

Figure 8.13 shows the admission precondition and the whole incident end to end across the three harbors, escrow, and witness log, with six marked steps before the subsections below walk through them one at a time.

8.6.1 Setup

Two organizations: Acme Robotics (Alice's employer) and Beta Vision (Bob's employer). The shared project is a perception-and-control stack: Acme owns the controller, Beta owns the vision system, both run against a shared staging environment that lives on a third machine maintained by neither.

Three machines: Alice's laptop (harbor A, controller code, controller test database), Bob's laptop (harbor B, vision code, vision test database), staging-server (harbor S, shared staging database, no agents resident).

Each harbor has gone through the admission ceremony of §8.5. Acme's harbor was sponsored by an existing partner harbor at the open-source consortium that hosts the staging server; Beta's was sponsored by Acme after a six-month probationary period during which Acme's risk officer watched Beta's evidence trail closely. The staging server harbor is sponsored by the consortium directly and is treated as a known-honest verifier for the purposes of this example.

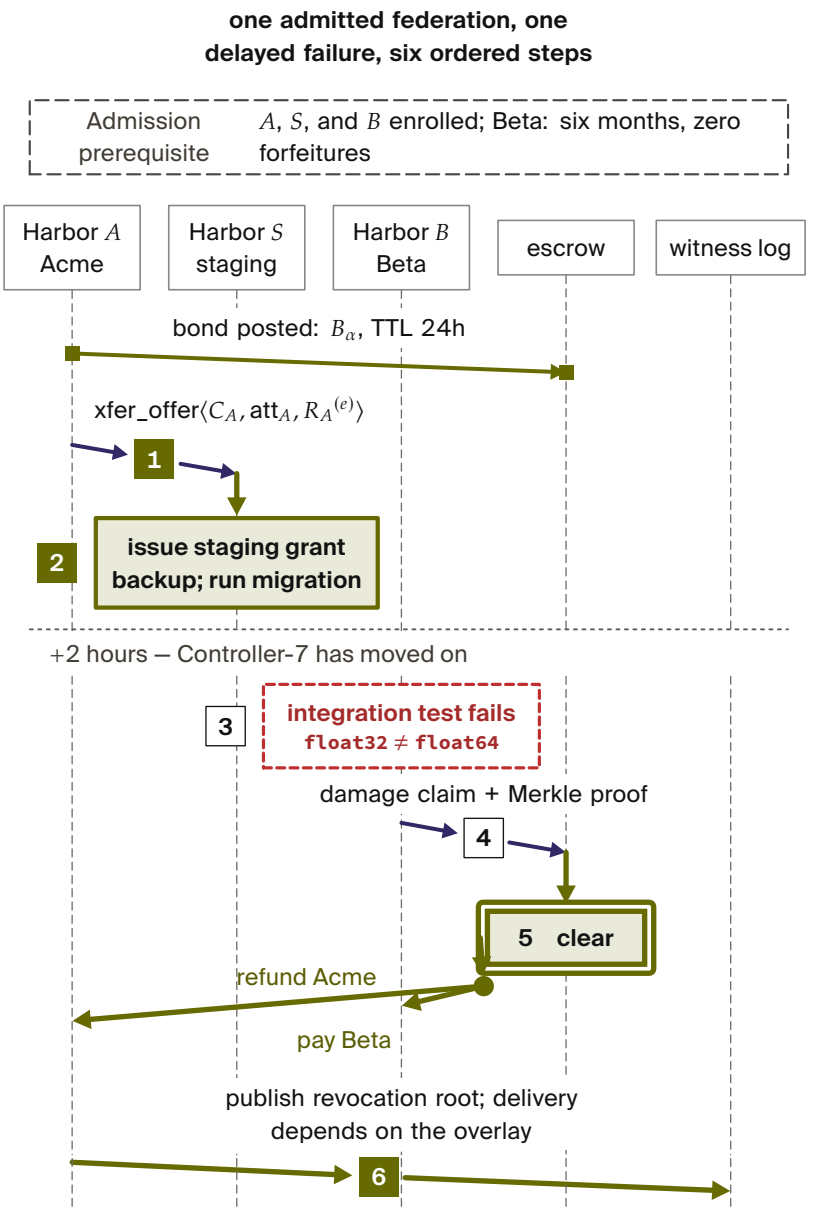


Figure 8.13.
A constructed incident: S grants staging access under its own resource-mapping policy; Beta's adjudicated claim then settles against Acme's bond. Revocation follows without a guaranteed delivery deadline. [internal]

8.6.2 The agent task

Acme's agent — call it Controller-7 — needs to run a schema migration on the staging database to add a column for a new sensor type. The migration is destructive (drops and recreates the column); if it goes wrong, the staging database is unusable for the joint demo scheduled three hours later.

Controller-7's local card C_A authorizes `db:migrate` against the controller test database, attenuated through Acme's risk policy (att_A = "only against tables in the `controller` schema; only between 9am and 5pm; only if a human operator has approved within the last 30 minutes"). The card is valid at A, but the database it needs to migrate is at S. These are distinct resources. The test-database card is evidence of eligibility, not authority over staging. Before the task, S must publish a local policy that maps this class of A-issued request to a separately authorized staging grant; its identifier and version belong in the offer and resulting grant.

8.6.3 Step 1: Cross-harbor transfer

Controller-7 invokes xfer_req(C_A, S) on D_A. D_A constructs the envelope:

$$\text{xfer_offer}\langle C_A, \text{att}_A, R_A^{(e)}\rangle \text{ signed under } sk_A,$$

and sends it to D_S (the staging-server harbor). D_S verifies the signature against A's public key as recorded in the witness log, verifies that $R_A^{(e)}$ is the current epoch root for A, and applies its own attenuation att_S ("only the `controller` schema; only after a backup snapshot has been taken; only if no other migration is in flight"). Here the destination is a different database, so D_S performs token exchange and issues a new local grant under the pre-agreed resource mapping. It retains the restrictions from A and adds S's requirements, but it does not claim that staging operations are a subset of test-database operations. The grant records the parent card, mapping version, expiry, and revocation dependency; Theorem 8.2.2 applies only within a shared resource universe.

D_S delivers C'_S to Controller-7. Controller-7 takes the snapshot, then runs the migration against S. The migration's actions are recorded in S's Merkle forest and roll up into $R_S^{(e)}$.

8.6.4 Step 2: Damage detection

The migration runs cleanly. Two hours later — well after the agent has moved on — Beta's vision system runs an integration test against the staging database and observes that the new column has the wrong type (Acme assumed `float32`; Beta needs `float64` for the sensor data they will produce). The integration test fails.

D_B (Beta's harbor) verifies the failure against its local invariants and produces a damage claim:

$$\mathrm{cl}_B = (R_B^{(e+1)}, \text{evidence}),$$

where the evidence binds the failed test receipt, exact work item, base and result snapshots, and the acceptance policy. Merkle inclusion authenticates those records; causation and whether `float64` was promised remain questions for the agreed adjudication rule. The claim is sent to the escrow.

8.6.5 Step 3: Settlement

Acme posted a bond when Controller-7 was authorized to run the migration. The bond is held in the federation escrow with a TTL of 24 hours — long enough for downstream consequences to surface, short enough not to lock collateral indefinitely.

The escrow verifies cl_B: it checks the inclusion proof against $R_B^{(e+1)}$ in the witness log, matches the claim against the acceptance preconditions signed at bond-post time (which named the staging schema as a covered surface), and decides: *clear*. Acme's bond is debited; Beta receives the cleanup-cost portion; the remainder is refunded to Acme. The settlement record is written to the witness log.

8.6.6 Step 4: Revocation propagation

At this point, Acme's risk officer revokes Controller-7's `db:migrate` card and publishes $R_A^{(e+2)}$.

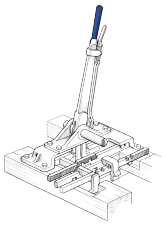
Numbers by Hand 8.6.1 (Ideal Doubling Is Not Expected Completion).

In the stated push model, each informed harbor contacts one peer per round. Starting from one informed harbor, even an ideal sequence with no duplicate contacts informs at most 2^k harbors after k rounds. Eight harbors thus need at least three rounds; sixty-four need at least six. These are best-case lower bounds, not expected completion times.

Random contacts can hit already informed peers. With $m = 3$, after the first round two harbors are informed. In the next

round both independently contact one of their other two peers. The probability neither contacts the last uninformed harbor is $(1/2)^2 = 1/4$. Each subsequent round succeeds with probability $3/4$, so the expected total is $1 + 1/(3/4) = 7/3$ rounds, rather than $\log_2 3$. This calculation fixes a finite-size model explicitly; $\Theta(\log m)$ alone cannot supply its expectation or a deadline.

8.6.7 What this example shows



A local railway interlock permits a signal only after its own checks pass. Similarly, each receiving harbor checks its grant and revocation policy. Relayed evidence alone does not lock remote resources.

The scenario is constructed; it demonstrates the protocol obligations, not a measured incident. The point of working it is to show that each of the four primitives — transfer, revocation, settlement, admission — is doing its job at exactly one step, and that the composition is straightforward when each primitive is honest about what it covers.

Notice in particular what *did not* happen:

- Neither D_A nor D_B trusted the other's signing key as a root. The transfer worked through the witness-logged public keys, not through a chain of trust.
- The bond was not held by either harbor; the design requires a recipient-whitelisted custody ledger before the escrow's role is structurally bounded.
- The settlement was not negotiated peer-to-peer; it followed a fixed protocol whose two outcomes were named in advance.
- Under the example's connected reliable-round assumptions, the revocation propagated through gossip with expected logarithmic completion; no consensus protocol was invoked.

Each of these is a deliberate design choice. The useful test of the composition is whether each receiving authority has the evidence, current policy, and funded remedy required for its own decision.

8.7 Formal Model

Blanchet 2016

The formal program has two layers: ProVerif proof obligations for the cryptographic protocols and TLA+ proof obligations for cross-harbor accounting. The cross-harbor listings below are sketches. The available executable artifacts concern narrower single-harbor

accounting and a vault-secrecy query whose encoding limitation is stated explicitly.

8.7.1 ProVerif extensions to Anchor

A proposed extension of Anchor's models introduces a second signing key and an epoch-root binding. Its intended cross-realm authenticity condition has the following shape:

```
query b: id, ha: id, hb: id, cap: capability, e: epoch;
  event(AcceptedAtB(b, hb, cap, e))
  ==> (event(IssuedAtA(b, ha, cap)) &&
       event(EnvelopeAtoB(ha, hb, cap, e)) &&
       event(RootCurrent(ha, e))).
```

Listing 8.2. Cross-realm authenticity query (sketch)

This condition asks each acceptance at B to have corresponding issuance, envelope, and root events. It is a specification target, not a verified result. A past `RootCurrent` event alone does not prove freshness at acceptance: a complete model must express the relevant epoch order and stale-root rejection, as well as the key-compromise and resource-mapping assumptions.

No executable cross-realm model or run transcript is supplied. Appendix 8.A records this mechanization gap.

The separate artifact `proofs/bonded/federated/federated.pv` has a committed successful ProVerif transcript with daemon, KMS, and email shares exposed. Its query, however, concerns a fresh private constant `account_root` that is never bound to `combine4` of those shares or published. The success establishes secrecy of that unused constant; it does not establish the intended four-principal reconstruction claim. The recorded result is:

```
Starting query not attacker(account_root[])
RESULT not attacker(account_root[]) is true.
--------------------------------------------------------------
Verification summary:

Query not attacker(account_root[]) is true.
```

Listing 8.3. `proofs/bonded/federated/federated.pv`, actual ProVerif output

Exercise 8.6 diagnoses this vacuous query and specifies the missing positive and negative controls.

Exercise 8.6, p. 707.

8.7.2 TLA+ extension for cross-harbor conservation

The executable Bonded Commons Conservation specification models an abstract single-machine bond pool. A cross-harbor extension would need per-harbor pools, escrow custody, and in-transit transfers. This sketch names only lifecycle buckets and a witness log:

Listing 8.4. Cross-Harbor Conservation specification (sketch)

```
CONSTANTS Harbors, MaxBonds, Capabilities
VARIABLES Bucket, WitnessLog

TypeOK ==
  /\ Bucket \in [1..MaxBonds -> {"posted", "escrow", "cleared", "
   ↪ refunded"}]
  /\ WitnessLog \in Seq([harbor: Harbors, epoch: Nat, root: STRING])

BucketPartition ==
  \A b \in 1..MaxBonds:
    \E! s \in {"posted", "escrow", "cleared", "refunded"}:
      Bucket[b] = s

Spec == Init /\ [][Next]_<<Bucket, WitnessLog>>
```

The sketch does not define `Init` or `Next`, and a one-hot bucket label does not by itself account for recipient balances. No executable cross-harbor specification, configuration, or Apalache run is supplied. Appendix 8.C identifies the narrower executable Conservation model and its recorded TLC result. The cross-harbor partition remains a conditional design invariant, not a model-checking result.

8.7.3 What is mechanized and what is structural

We are explicit about which claims live in which band (Figure 8.2).

Available model evidence. Single-harbor capability models are inherited from Anchor. The committed TLC transcript concerns Bonded's single-machine Conservation model. Cross-realm protocol sketches and the cross-harbor bucket partition have no executable verification evidence here. The separate vault model's successful query does not encode the intended reconstructed root.

Conditional (assumption scope). Revocation dissemination is expected $\Theta(\Delta \log m)$ only under the stated reliable connected-overlay model (Theorem 8.3.1). The escrow extraction bound is

conditional on a non-bypassable recipient-whitelisted custody ledger (Design Invariant 8.4.1).

Open (cobalt). Multi-principal correlated-equilibrium extension of Bonded; trustless settlement for non-fungible reputation-priced bonds; folk-theorem cartel-resistance bound at the federation layer.

8.8 Failure Modes

The design must account for three operational risks: concentration of accepted authority, equivocation by a witness, and network partitions. Protocol support for multiple participants does not by itself resolve any of them. The following subsections distinguish architectural choices from guarantees and deployment evidence.

8.8.1 Centralization gravity

Distinct endpoints need not mean independent administrative control. Maler and Reed describe both SAML circles organized around an identity-provider hub and InCommon with multiple institutional identity providers. These are alternative trust topologies, not a numerical law about how quickly control concentrates. A Harbor deployment can run separate witness instances and compare their authenticated roots. Receivers still decide which roots to accept; replication alone establishes neither administrative diversity nor a cost disadvantage for a dominant hub.

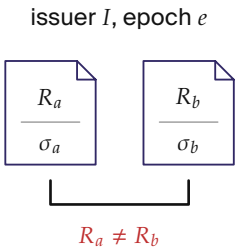
Maler & Reed 2008

The control-plane distinction is concrete: Tailscale describes centralized coordination with a mesh data plane, while Headscale supplies a self-hostable control-server implementation. Implementation choice and the distribution of actual administrative control are separate questions. Harbor's deployment obligation is to measure who controls accepted roots, how much work depends on each authority, and the cost of switching while preserving record continuity. Figure 8.2 therefore leaves resistance to concentrated control open.

Avery Pennarun et al. How Tailscale Works. Tailscale Engineering Blog, March 2020.

Juan Brackeva et al. Headscale: Open-Source Tailscale Coordination Server. GitHub project, 2020–present.

8.8.2 Equivocation by a malicious witness



Both signatures verify. Together, the conflicting roots expose equivocation. A partition can delay their comparison.

Laurie et al. 2013

A malicious witness log can serve different views to partitioned auditors. A contradictory signed pair is immediately verifiable once co-located, but the protocol has no topology-independent time bound for bringing the pair together. Witness bonding applies after confirmation.

For high-stakes settlements, harbors can require multiple witness signatures before treating a root as authoritative, in the Certificate Transparency two-SCT pattern. This trades latency for assurance; the design space is well-explored in the CT literature and we adopt the same techniques.

8.8.3 Partition tolerance

The federation does not run consensus; partition tolerance is per-protocol. Capability transfer cannot proceed during a partition between A and B (no surprise; cross-realm authentication requires both daemons to reach the witness log). Revocation gossip continues within each partition; convergence is bounded by the partition's local size, not the global federation size. Settlement is parked at the escrow until the partition heals.

The structural posture is that partitions are operationally common and the federation should degrade gracefully rather than fail fast. We inherit this discipline from Anchor's intra-mesh design; the federation extension is straightforward but adds no novel mathematics.

Owens 2026

Table 8.4 lays out all three protocols side by side.

Table 8.4. Federation runs no consensus, so partition tolerance is decided protocol by protocol. Transfer stalls outright; revocation keeps working inside each side of the split; settlement holds its position until the split heals. [internal]

protocol	behavior under partition	what resumes it
transfer	cannot proceed: cross-realm authentication needs both daemons to reach the witness log	the partition heals and both sides reach the log
revocation	continues inside each partition; convergence bounded by that partition's own size	the two partitions merge their informed sets on reconnection
settlement	parks at the escrow, holding the bond in place	the partition heals

8.9 Related Work

The Federated Harbor sits at the intersection of five bodies of work. We are concrete about where we draw on each and where we depart.

Federated identity. The OpenID Federation 1.0 specification is the closest existing analogue: each entity publishes a signed Entity Statement, trust chains assemble by walking up to a Trust Anchor, trust marks express property claims. The Federated Harbor is structurally similar in the trust-anchor pattern but differs in two places: (a) we add a capability/attenuation layer (OpenID Federation is identity-only), and (b) we do not assume HTTP-based fetching of entity statements, since Port Daddy daemons cannot reliably expose HTTP endpoints (NAT, sleep, etc.). SAML/Shibboleth also exposes identity-provider discovery and trust-topology choices; these motivate the distinction between multiple endpoints and independent control in §8.8.1.

Roland Hedberg (ed.), Michael B. Jones, Andreas Åkre Solberg, John Bradley, Giuseppe De Marco, and Vladimir Dzhuvinov. OpenID Federation 1.0. OpenID Foundation Implementer's Draft 4, July 2024.

Maler & Reed 2008

Capability-based federation. Macaroons are the foundational capability-with-attenuation construction; our transfer ceremony extends Macaroon-style third-party-caveat chains across an administrative boundary. UCAN is the current production-deployed answer to "Macaroons with public-key principals," and we treat it as the operationally most relevant existing system; the Federated Harbor differs in keeping the daemon as a structural commons authority rather than going fully peer-to-peer, and in pricing delegation through insurance-priced bonds rather than pure capability constraints. The object-capability lineage continues to be relevant: cross-realm operations are exactly a mutual-suspicion scenario in Miller's vocabulary, and his defensive-consistency framework is the right lens.

Birgisson et al. 2014

UCAN Working Group. UCAN Specification 1.0: User Controlled Authorization Network, 2024.

Miller 2006

Dennis & Van Horn 1966

Production federation protocols. Three federations that actually run solve a version of the same trust-boundary problem, and a reader who knows them will ask about them before asking about OpenID Federation, so here they are. ActivityPub, the W3C protocol under Mastodon and the rest of the fediverse, lets independent servers relay signed activity with no shared database and no global

Christine Lemmer-Webber, Jessica Tallon, Erin Shepherd, Amy Guy, and Evan Prodromou (eds.). ActivityPub. W3C Recommendation, 2018.

ruler — which is this chapter's own stated goal, met at a scale of millions of accounts. Matrix has homeservers replicate room state across administrative boundaries with eventual consistency and no single point of control, and is, of anything deployed today, the closest working instance of the "no consensus, gossip-bounded, detectable equivocation" posture of §8.0.4. And SPKI/SDSI, the mid-1990s design for local names and decentralized authorization, is the ancestor both Macaroons and UCAN descend from, one layer beneath the capability-based federation of the previous paragraph. None of the three overturns a theorem in this chapter. What they do is set the bar for what "federation without a ruler" has already been shown to survive in production, and the differences — neither ActivityPub nor Matrix carries a bonded, attenuating capability across the boundary, and neither prices delegation — are the chapter's actual contribution, stated against the right baseline.

The Matrix.org Foundation. The Matrix Specification. Online source.

Ellison et al. 1999

Laurie et al. 2013

Newman et al. 2022

Transparency logs. Certificate Transparency is the model; the federated witness log of §8.0.3 is CT applied to capability tokens. Sigstore and Rekor are the production instantiation we rhyme with most heavily; in-toto and SLSA are the supply-chain analogues that compose with the Federated Harbor for the "poisoned plugin under a bonded principal" case but do not close it. The Crosby–Wallach history-tree is the formal substrate of every modern transparency log including ours.

Torres-Arias et al. *in-toto: Providing farm-to-table guarantees for bits and bytes*. USENIX Security, 2019.

Linux Foundation / OpenSSF. SLSA Specification v1.0: Supply-chain Levels for Software Artifacts, 2023.

Scott A. Crosby and Dan S. Wallach. Efficient Data Structures for Tamper-Evident Logging. In *USENIX Security Symposium*, 2009.

Cross-domain settlement. Herlihy's atomic cross-chain swap is a comparison point; the cross-harbor settlement protocol of §8.4 instead handles non-fungible reputation-priced bonds through conditionally restricted custody. It is not trustless in Herlihy's sense. Whether trustless settlement is achievable in our setting is open (§8.10). The IBC protocol supplies another connection/channel comparison; unlike IBC, this design does not assume a consensus-backed state machine underneath.

Herlihy 2018

Christopher Goes. The Interblockchain Communication Protocol: An Overview. arXiv:2006.15918, 2020.

Agentic-commerce discovery and mandates. The Universal Commerce Protocol (Google/Shopify, 2026) solved cross-administrative discovery for agent-mediated retail with a pattern the Federated Harbor proposes to use: a `/.well-known` JSON profile that does double duty as capability declaration and JWK key publication, with server-selects version negotiation — the party that

Universal Commerce Protocol 2026

Body of work	Borrowed	Departure	Why
Federated identity (OpenID Federation)	signed statements and trust chains assembled toward an anchor	adds capability transfer and avoids HTTP fetching	daemons cannot assume a reliable public HTTP endpoint
Capability federation (Macaroons, UCAN)	caveated attenuation chains and public-key principals	keeps each daemon a structural authority and prices delegation with bonds	delegation needs an economic exposure, not only a constraint
Transparency logs (CT, Sigstore / Rekor)	append-only Merkle history and consistency proofs	logs capability roots rather than certificates	transfer and revocation need the same tamper evidence
Cross-domain settlement (Herlihy, IBC)	an adjudication point shared by two domains	is not trustless and has no consensus state machine underneath	bonds are nonfungible and reputation-priced
Agentic commerce (UCP, ACP, AP2)	discovery profiles and signed mandate formats	adds bonding and settlement enforcement	those protocols standardize discovery, not counterparty trust

Table 8.5. Five lineages supply mechanisms but not the Federated Harbor's complete contract. Each row names the borrowed mechanism, the deliberate departure, and the operational reason for it. [internal]

must execute a request computes the compatibility intersection. Where UCP assumes merchants hold stable HTTPS origins, our daemons often do not (the same constraint that ruled out OpenID Federation's entity-statement fetching above); the profile therefore also travels as a relay-published mirror. UCP delegates authorization proof to the Agent Payments Protocol's verifiable-credential mandates, whose SD-JWT/JWS/JCS formats the harbor profile proposes to use. The profile and relay mirror require interoperability tests; the format alone proves neither key ownership nor enforcement. Those protocols standardize discovery and the transaction envelope and are explicit that counterparty trustworthiness, collateral, and settlement enforcement are out of scope — which is the half of the problem this chapter and its predecessors supply.

AP2 2026

Table 8.5 lays out all five lineages against what this chapter borrows, departs from, and why.

8.10 Limitations and Open Questions

The following questions remain open. Figure 8.2 separates the threat classes; each question requires its own model, evidence, or proof.

8.10.1 The multi-principal correlated-equilibrium extension

The Bonded Commons chapter gives a conditional correlating-device interpretation, not a guarantee that every local interaction is an equilibrium. Across harbors, two models are worth distinguishing:

- A joint mediator uses witnessed state to recommend cross-harbor actions. Its distribution, each participant's observations, and the gains from deviation must be specified and checked.
- Each harbor makes local recommendations while cross-harbor actions depend on a shared signal. Local obedience does not establish cross-harbor obedience; a shared signal alone establishes neither Nash nor correlated equilibrium.

Erich Owens. The Bonded Commons: Pre-Transactional Trust Infrastructure for Multi-Agent Systems. Curiositech LLC Technical White Paper, May 2026.

A signed root proves what was committed, not who has received it or what each participant knows about the others' observations. Even two current roots do not, by themselves, establish common knowledge. A solution must state a delivery model and an incentive condition for the information actually available at each decision. Neither model above has been proved for this protocol.

Joseph Y. Halpern and Yoram Moses. Knowledge and Common Knowledge in a Distributed Environment. *Journal of the ACM*, 37(3):549–587, 1990. Online source.

8.10.2 Trustless settlement for non-fungible bonds

HTLC-style atomic swaps achieve trustless settlement for fungible assets with a shared hash preimage. The Federated Harbor's bonds are non-fungible and adjudication-dependent, so that construction does not transfer directly. We adopt conditionally restricted custody (§8.4) and leave both a trustless construction and an impossibility result open. Either would require a separate threat model and proof.

Herlihy 2018

8.10.3 Cartel formation across federations

Bonded's folk-theorem cartel-resistance argument applies inside a single machine. Between machines, cartel formation is structurally easier: colluders can coordinate offline and present a unified front to the witness log, which records that all harbors agreed — a behavior consistent with both honest agreement and collusion. We owe either a strengthening of the bonded mechanism that resists cross-harbor cartels, or a precise statement of the weakening. We have neither.

8.10.4 Cold-start admission and the invitation-only failure mode

The bonded-sponsor pattern of §8.5 requires a new harbor to find a sponsor willing to underwrite its probationary risk. In practice, this market may be thin — the historical evidence on whether invitation-bounded federations stay open is mixed. We treat the structural pushback (many sponsors, small probationary scope, witness-log permanence) as engineering discipline, not as theorem.

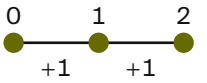
Maler & Reed 2008

8.10.5 Equivocation propagation speed

Theorem 8.3.1 separates cryptographic verification from dissemination. Verification is local once the conflicting roots are present; delivery depends on topology, loss, and partitions. A deployment-specific tail bound remains *open*.

Mixing speed and detectability are different questions. A connected tree has a positive graph-Laplacian spectral gap but no cycle-space residual: the three-node path has eigenvalues 0, 1, 3, yet this detector cannot expose a relayed lie on it. Thus an inverse-gap formula for time-to-alarm cannot hold on arbitrary topologies. Even a visible cycle gives no finite alarm time if the lie lies in the detector's kernel or stays below the noise threshold.

Connected, but no loop



Spectrum: 0, 1, 3. Residual: 0.

Node potentials 0, 1, 2 explain both edge differences. A positive spectral gap does not guarantee a cycle-space alarm.

The open problem is to specify a packet scheduler and loss model, a visible nonzero residual with a stated signal-to-noise margin, and a stopping rule; then derive a delivery-and-detection bound. Only after those choices can one ask whether the smallest positive eigenvalue $\lambda^{+}_{\min}(\Delta_{\mathcal{F}})$ controls any term. Linear real-valued

diffusion and signed, discrete append-only gossip are not interchangeable processes. A topology sweep could test a proposed bound, but cannot establish it by regression alone. No such bound or completed sweep is claimed here.

8.10.6 Cost of per-write durability

The cryptographic assertions and per-write durability checkpoints this layer inherits from Anchor carry a measurable I/O cost. They are designed for high-stakes coordination, where an audit trail is worth more than throughput, and not for bulk data movement or latency-critical paths. This chapter does not measure that overhead at the federation layer and does not establish where the crossover sits. *open*.

8.10.7 What this means for the reader

A federation paper with six named open questions is doing its job. The point of the paper is to draw the new boundary cleanly so the open problems are visible; if every claim in this chapter were closed, there would be no third paper to write. The honest read is that the Federated Harbor is closer to a research agenda than to a deployed product, and the prior two papers (Anchor, Bonded) have a tighter ratio of theorem to conjecture for the same reason. The substrate is real; the federation layer is in progress.

A boundary drawn cleanly is itself a result: it says where the next proof has to start.

Exercises 8.7–8.12, p. 707.

Review of the key ideas

What this chapter leaves coupled, in the order the rest of the book will lean on it:

1. **Two machines, no shared daemon.** A federated authority admits a foreign act only through a witness log each side can check; admissibility is a design invariant of the log, not a property of the peer.

2. **Attenuation preserves inclusion within a shared resource universe.** Each narrowing composes with the next. Cross-realm resource mapping, authenticity, and freshness require the stated assumptions; the transfer protocol remains unmechanized.
3. **Revocation propagation has a conditional expectation.** The reliable connected-round model gives expected logarithmic dissemination. It supplies neither a deterministic delivery deadline nor a finite deadline during a partition.
4. **A one-edge perturbation alarms exactly on a visible coordinate cycle.** For a perturbation of size 3, the residual is 1.2247 on C_6 and zero on P_6. This concerns the edge encoding; authenticated endpoint claims can be compared directly on either graph.
5. **The residual is an exact unrestricted repair cost.** It is only a lower bound when corrections must obey a prescribed support. Attribution needs additional support assumptions, and multiple offsets can cancel even when one agent supplies them. The equal-information direct checker misses none of the residual's alarms in the stated noise-free model.
6. **Cross-harbor settlement has separate accounting obligations.** One funded bond occupies one lifecycle account; recipient transfers require their own conservation rule and nonbypassable custody. The partition alone proves neither implementation conformance nor containment of a harbor failure.
7. **Admission is a ceremony with a record**: a harbor joins a federation by exchanging epoch roots, and the record is what a later dispute reads.
8. **The rollback configuration fails by design.** Restoring a revocation database can resurrect a revoked card. The mitigating model uses an external epoch that the restore cannot roll back; its finite TLC check does not prove deployed recovery behavior.

8.11 Exercises

§8.2 Cross-Harbor Capability Transfer

Solution p. 777

8.1 Trace ⋆⋆

Alice's card C_A authorizes `db:write` on A's *test* database. D_B accepts it and mints $C'_B = \text{att}_B(\text{att}_A(C_A))$ authorizing `db:write` on B's *staging* database. Trace Theorem 8.2.2's proof sketch against this concrete case: at which step, if any, does "subset composition is a subset" stop being a statement about the *same* resource space? State precisely what kind of act minting C'_B actually is here, and what two things a correct D_B must additionally establish before minting it.

§8.3 Federated Revocation

Solution p. 778

8.2 Trace ⋆⋆

The "only-issuer-revokes" rule above is trustworthy only if a harbor's own local revocation record is *final* — once revoked, always revoked, even across an operational mishap at the relay. `proofs/relay/CardRevocation.tla` model-checks exactly this property (`RevocationFinal == badAccept = FALSE`) under three configurations. Read `proofs/relay/CardRevocation.md` and state, for each of the baseline, rollback, and epoch configurations, what changes in the model and what TLC found.

Solution p. 779

8.3 Trace ⋆⋆

Read `skills/harbor-results/scripts/sheaf_consistency_radius.py`, section `[0]` (function `section0`). By hand, using $r = |s|\sqrt{1 - R_{\text{eff}}(e)}$ and $R_{\text{eff}} = (n-1)/n$ on C_n, confirm the two numbers the script's closed-form check pins for C_6 and P_6 at lie magnitude 3.0. Then state what the script's `check(…)` call actually asserts for each case, and to what tolerance.

Solution p. 780

8.4 Open ⋆⋆⋆

Theorem 8.3.4 assumes a *single* equivocator and says outright that this is not decoration. Two colluding reporters sit on the same visible cycle K_c and each reports an offset; either offset alone would be detected. Using only the definition of the completion residual r as a minimum over global sections and free severed blocks, explain

why a coalition can drive r to (numerically) zero even though neither liar's offset is small, and say what kind of evidence — if any — could still catch the coalition.

8.5 Trace ⋆⋆ *Solution p. 781*

A path $0 \to 1 \to 2$ reports $e_B - e_A$ on both edges, with receipt costs two and three. State the value forced at vertex 1 by each receipt. Find a least-cost contradiction certificate. Then explain why a real-valued least-squares fit on this path has zero residual, and why an observed zero edge would not reveal its endpoints' shared categorical value.

§8.7 Formal Model

8.6 Trace ⋆⋆ *Solution p. 781*

Read `proofs/bonded/federated/federated.pv` and its committed run log. Identify the four shares, the queried term, and the relation between that term and the honest reconstruction. Does the successful query prove that any one uncompromised principal preserves the vault's secrecy? Specify a negative control that a repaired model must fail.

§8.10 Limitations and Open Questions

8.7 Open ⋆⋆⋆ *Solution p. 781*

§8.10.1 distinguishes joint and local mediation across two harbors. Specify the players, actions, payoffs, and observations for one model. State the incentive condition for a coarse correlated equilibrium and explain how correlated equilibrium strengthens it. What does a fresh pair of witnessed roots fail to establish? Give a conservative policy for decisions that depend on missing or stale cross-harbor evidence.

8.8 Open ⋆⋆⋆ *Solution p. 782*

§8.10.2 leaves open whether trustless settlement is possible for the Federated Harbor's non-fungible, adjudication-dependent bonds. HTLCs solve the fungible case using a shared hash preimage. Explain, from the definition of an HTLC's atomicity condition, why that construction does not transfer to a bond whose "did the damage occur" predicate is subjective, and sketch what an impossibility argument for the general case would need to assume.

Solution p. 783

8.9 Open ⋆⋆⋆

§8.10.3 observes that the witness log cannot distinguish unanimous honest agreement from unanimous collusion. Using the one-shot deterrence inequality $p \cdot d \cdot L > G$ (audit probability, detection probability, loss on detection, collusive gain), state what would have to be true of a federation-layer audit mechanism for that inequality to be satisfiable against a cartel that controls a majority, but not all, of the witnesses in scope — and say plainly why no structural mechanism closes the case where the cartel controls *every* oracle and witness.

Solution p. 784

8.10 Open ⋆⋆⋆

§8.10.4 lists engineering discipline (many sponsors, small probationary scope, witness-log permanence) against the bonded-sponsor pattern's cold-start risk, but explicitly declines to call it a theorem. Propose measurements over time that would help an operator investigate whether the sponsorship market is thin but open or whether incumbent sponsors restrict entry. State which records each measurement requires, what the admission log alone reveals, and why the measurements cannot by themselves identify a cartel.

Solution p. 784

8.11 Open ⋆⋆⋆

§8.10.5 distinguishes a deployment-specific propagation-tail question from the relationship between epidemic dissemination (Theorem 8.3.1) and sheaf-Laplacian diffusion (§8.3.4's closing paragraph). State what must be specified to compare the two processes mathematically, and give one measurement needed to assess propagation risk in a particular deployment. Which parts can be resolved without deployment traces?

Solution p. 785

8.12 Open ⋆⋆⋆

This chapter describes federation as "not a consensus protocol" (§8.0.4) while specifying a cross-harbor bucket-partition invariant (§8.4.2). What does not require a global total order, and what atomicity and custody assumptions does the accounting design still need? Distinguish that design from the available single-machine model-checking evidence. Then explain what makes the proposed federation a project institution rather than merely two machines sharing a repository.

8.12 Conclusion

The proposed federation keeps local authority with each daemon. A shared witness log supports revocation records and conditionally restricted custody; bonded sponsorship controls admission. These mechanisms require the implementation and service-level assumptions stated in the chapter.

Figure 8.2 distinguishes the verified claims, conditional bounds, and open questions.

What this chapter supplies is a clean federation surface that could compose with the local substrate: a transfer ceremony, witnessed revocation records, a conditional custody design, and bonded admission. The protocol and models make the missing implementation and service-level assumptions explicit; they do not establish that arbitrary harbors can already interoperate safely.

The remaining work—runtime custody enforcement, deployment conformance, a multi-principal equilibrium model, and a cartel-resistance bound—is material. The local substrate is real; the federation described here remains a design with partial mechanization.

Chapter Appendices

8.A Verification Status

Artifact registry. The cross-paper status table in Bonded Commons carries the items that are shared across the series; this appendix lists only the items specific to the Federated Harbor.

Owens 2026

Table 8.6. Federated Harbor verification status. **closed**: model verified and runtime conformance test passes. partial: a conditional design, empirical result, or incomplete mechanization; the row states the limitation. *open*: known unmechanized; threat band visible in Figure 8.2 (cobalt).

Claim	Method	Result	Artifact
Single-harbor capability authenticity (inherited)	ProVerif 2.05	**closed**	Anchor `harbor_card_v*.pv`
Cross-realm authenticity (§8.2)	ProVerif query sketch	partial no executable model or verification run	§8.7.1
Attenuation monotonicity across boundary (Lem. 8.2.2)	Set-inclusion argument	partial requires shared resource universe; protocol unmechanized	§8.2.3
Federated revocation dissemination (Theorem 8.3.1)	Conditional epidemic model	partial expected asymptotic result; no partition deadline	§8.3 (argument)
Cross-witness equivocation evidence	Signed-root comparison; dissemination model	partial verification immediate once views meet; no hard delivery bound	—
Escrow extraction bound (Design invariant 8.4.1)	Transition-system case analysis	partial conditional on non-bypassable custody; no runtime conformance	§8.4

Demers et al. 1987

(continued)

Claim	Method	Result	Artifact
Cross-harbor Conservation (§8.7.2)	Lifecycle-accounting sketch	partial no executable specification or verification run	§8.4.2
Sheaf detection of equivocation (Thm. 8.3.4)	Pre-registered statistical harness, 200 trials/arm	partial gates pass at the stated scope (single equivocator, real stalks, prefix restrictions); not mechanized	`sheaf_harness_v2.py`
Sheaf-Laplacian relaxation-time bound (§8.3.4)	—	*open* no derivation and no citation; the gossip and linear-diffusion models are not reconciled	—
Trustless settlement for non-fungible bonds	—	*open* no construction or impossibility proof	—
Multi-principal correlated-equilibrium extension	—	*open*	—
Cross-federation cartel-resistance bound	—	*open*	—

What this table means. A partial row reports a conditional argument, an empirical check, or an incomplete artifact; the limitation remains part of the claim. An *open* row names a result or construction not established here. The remaining obligation depends on the row: a mathematical proof or counterexample, an executable model and verification run, deployment measurements, or runtime

conformance evidence. Completion must be assessed against that obligation.

8.B ProVerif Artifacts and Proof Obligations

The repository contains Anchor's executable capability models under `analyses/`, including `analyses/harbor_card_v3_delegation.pv`. It does not supply executable cross-realm transfer, attenuation, or escrow models. The correspondence in §8.7.1 is a query sketch; root freshness at acceptance and cross-realm resource mapping remain proof obligations.

The separate `proofs/bonded/federated/federated.pv` and its committed transcript are available for inspection. Its queried private constant is unrelated to the computed vault term, so its successful result does not prove the intended four-principal security claim. Exercise 8.6 identifies the missing term binding and compromise controls.

8.C TLA+ Artifact and Cross-Harbor Scope

The executable model is `Conservation.tla`, in `proofs/bonded/conservation/`. Its `Conservation.cfg` configuration uses three agents, `MaxBalance=3` (the per-action amount bound), and `MaxMint=6`. Its committed TLC transcript records 26,818 generated states, 1,716 distinct states, search depth 9, and no invariant violation. The invariant is total free plus escrowed plus burned value equals lifetime minted value.

This is an abstract single-machine accounting model. No `CrossHarborConservation.tla`, cross-harbor configuration, or four-harbor Apalache transcript is supplied. The cross-harbor lifecycle sketch therefore carries no claimed model-checking bound. Proving it requires an executable transition relation for custody, transfers, and recipient accounting; deployment conformance is a separate obligation.

The conclusion of the book The principles developed here form one discipline. Mediate every effect that must be prevented; let delegated authority only narrow; preserve enough evidence for a human to challenge the system; attach actions to a durable principal and work unit; make settlement conserve value; and cross organizational boundaries by exchanging verifiable records rather than by exporting sovereignty. Better models help inside this discipline. They do not replace it.

Solutions to the exercises

Chapter 0: Foundations and Prerequisites

Exercise p. 62

0.1

The writes give 5; serial increments give 6. Either include each read and update in one serial transaction, or validate the read version atomically with the write and retry the rejected operation from a fresh read.

Exercise p. 63

0.2

$10(1-p)-100p \leq 0$ gives $p \geq 1/11$. If detection happens only during an audit, p equals audit-selection probability times successful detection conditional on selection. The sanction must actually be enforceable.

Exercise p. 63

0.3

The request may never have arrived, or it may have committed before the response was lost. A positive receipt identifies the committed operation. An absent receipt alone does not prove that the request never arrived: it may still be in flight. Safe retry therefore needs atomic duplicate suppression at admission, or a terminal rejection/cancellation contract excluding a later first commit. The receipt establishes the recorded operation, not arbitrary semantic correctness of the tested code.

Exercise p. 63

0.4

Before the code, both states remain possible; afterward only bit 0 remains. The exit code reveals the bit even though the file is unreadable. A confinement argument must include that channel.

Exercise p. 63

0.5

A defensible design identifies the effect's actuator, operation identifier, commit and receipt boundary, recovery lookup, successor

epoch and obligation record. Exhibit both a retry and a stale-worker trace. If an external service cannot deduplicate or report the operation, acknowledge that exact recovery may be impossible and specify escalation instead of inventing certainty. There is no unique protocol or closed-form solution.

Chapter 1: The Single-Writer Kernel

1.1 *Exercise p. 149*

In memory the substrate still serializes, so resource exclusion, identity and presence, communication, obligation checks, policy checks, and self-attestation keep their transient meaning while the process lives. Everything the subsystems promise across a crash becomes vacuous: durable continuity, crash recovery, historical audit, and outcome evidence; durable identity collapses to an ephemeral handle. The decomposition survives; the cross-crash guarantees do not.

1.2 *Exercise p. 149*

The communication subsystem's home table is `messages`; it owes the layer above an ordered, cursor-addressable, durable envelope with explicit at-least-once semantics. Equally valid: `claims` for the resource subsystem (one compatible holder per conflict domain), or `commitments` for the obligation subsystem (no *done* without a current structured oracle receipt).

1.3 *Exercise p. 149*

A defensible nine-way cut: storage and durability; transaction serialization; identity and presence; resources and leases; message transport; stigmergy and projections; policy and capability; commitments and outcomes; attestation and recovery. It separates proof obligations that leak across the seven, above all transport semantics from message semantics and policy prevention from post-commit detection. It hides the simplifying fact that all local state shares one transaction boundary, so any such cut must retain a cross-cutting invariant: one ledger, one effect mediator.

1.4 *Exercise p. 149*

Module A renames `users.name` to `display_name`; module B, loaded later under older code, sees no `name` column and lazily adds it back.

Writes now split between two columns according to module version and load order, and neither backfill is authoritative. Every destructive migration has this shape; only additive tables and columns are safe under any-module-any-order.

Exercise p. 149

1.5

Command line, authenticated Unix-socket request, daemon request queue, the sole read/write connection, one transaction, WAL commit. The discipline holds by routing at two points, the client-to-API boundary and the daemon's sole-connection boundary; SQLite's file lock is one backstop at write time. The primary story is routing, not a database rule that forbids other connections.

Exercise p. 149

1.6

Expand, backfill, dual-read-or-write, contract keeps mixed versions safe for a while, but the contract step and the dependency order are global facts. Store a migration ledger with a schema epoch, each module's compatibility range, its dependencies, and a backfill watermark. Idempotent self-initialization can remain for additive tables and columns; safe rename, backfill, and down-migration cannot preserve unrestricted any-module-any-order. A `user_version` sequencer, or an equivalent ordered capability ledger, is unavoidable.

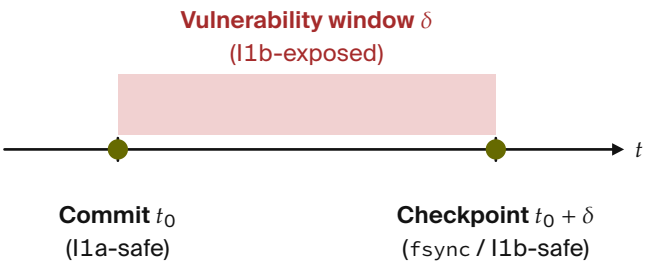
Exercise p. 150

1.7

(a) I1a-survivable. (b) An orderly shutdown flushes, so a genuinely clean reboot is not the I1b case; an abrupt reset during the reboot is I1b-exposed. (c) I1b-exposed. (d) I1a-survivable after commit. None of the four lets an uncommitted transaction survive.

Exercise p. 150

1.8



The commit is power-loss exposed from t_0 until the WAL frames that contain it are synced. A page-count threshold bounds accumulated WAL frames, not elapsed time: at a low or zero write rate it may not fire for arbitrarily long, so it gives no wall-clock bound. Only with an independently guaranteed write-rate floor $w_{\min}$ pages per second and threshold P does $\delta \lesssim P/w_{\min}$ follow, and this chapter assumes no such floor.

1.9 *Exercise p. 150*

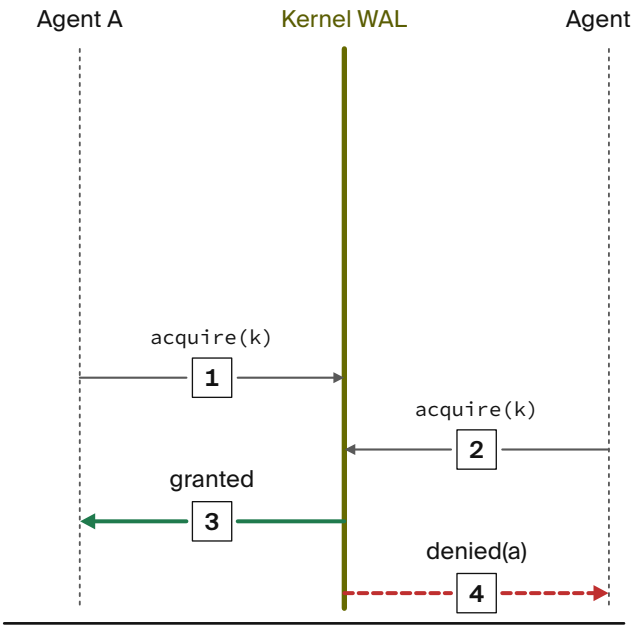
Make durability a typed argument of the transaction, `PROCESS_CRASH` or `POWER_LOSS`, never an adjective in prose. Route `POWER_LOSS` operations through a connection configured `synchronous=FULL` before the transaction opens and verify the commit before returning; if SQLite settings are connection-scoped, a separate fully-synced financial ledger is the cleanest isolation. Ordinary claims stay on `NORMAL`. Hard-reset fault tests must exercise the stronger path. The guard against silent defaulting is the type: a write path that names no durability class does not compile, and a forced checkpoint bounds recovery and log size but is not the per-write primitive.

1.10 *Exercise p. 150*

A release with no matching (key, holder) row is a successful no-op so that retries are idempotent: this is I3. It must never delete another holder's row. If it raised, a client that timed out on its first release could not distinguish success from failure, and at-least-once handling would break.

Exercise p. 150

1.11



step	settles
1	WAL inserts (k, A): the first row written, the linearization point
2	WAL inserts (k, B): UniqueViolation, the key is already taken
3	A's acquire returns; the lifeline now holds the claim
4	B's catch runs `SELECT holder`: returns **A**

that `SELECT` is where the loser learns the winner's name

The daemon serializes the two requests. The first `INSERT`(key, holder) commits. The second hits the unique-key violation, selects the extant row inside the catch path, and returns `DENIED(holder=first)`: that select is where the loser learns the name. For range and hierarchy claims the trace is safe only after both requests map to the same canonical conflict key or the overlap test runs in one immediate transaction.

Exercise p. 150

1.12

Add a durable FIFO ticket queue per conflict domain, a maximum lease L, a monotonically increasing fencing epoch, and grant-to-head on release, on acquire, and on lazy expiry. No background scheduler is required for safety, but liveness requires some live

caller or tick to trigger the transitions. Eventual requests alone give no finite wait bound. If leases cannot renew past L, a live trigger arrives within D of each expiry, and each handoff completes within P, then a requester with q predecessors waits at most $(q+1)(L+D+P)$, a conservative bound allowing a current holder as well. Without finite trigger and processing bounds, reactive safety can hold while waiting is unbounded. Every external effect must validate the fencing epoch, or an expired holder can still act.

1.13 *Exercise p. 151*

If the evaporation tick stalls for an hour, tick-only storage still reports yesterday's marker weight as today's. Read-time evaluation computes $w\, r^{\Delta t}$ from the persisted timestamp and never presents the stale stored number as current.

1.14 *Exercise p. 151*

The transport delivers the request twice. The consumer applies one semantic state transition and recognizes the duplicate as transport nondeterminism; diagnostics may count the redelivery, the operator's view shows one act. An envelope idempotency key plus a unique (consumer, key) receipt lets the consumer store "effect applied" atomically with the effect, making retry safety explicit rather than conventional. Exactly-once is still not guaranteed for an unbrokered external effect.

1.15 *Exercise p. 151*

For each marker kind, log creation, reads, the actions it influenced, its supersession, and the time at which operators label it stale. Fit a survival curve and choose the half-life that minimizes $C_{\text{stale}} \cdot \text{false-live} + C_{\text{lost}} \cdot \text{false-expired}$, validated on held-out projects. Claims and presence should decay far faster than decisions or learned skills. Publish the confidence and re-estimate under drift.

1.16 *Exercise p. 151*

Note-count monotonicity can be regimented by append-only storage or a trigger that rejects deletes and regressive updates. Capability attenuation and lock-owner release can be pre-checked only if authenticated identity, the capability order, and every relevant effect path pass through the mediator. Duplicate keys and boot admission are already pre-effect. Heartbeat freshness cannot be rejected at the heartbeat insert: staleness is an absence observed

only after time passes, and under asynchrony a slow actor is indistinguishable from a failed one. Out-of-band filesystem and network effects remain detect-only until mediated.

Exercise p. 151

1.17

`close(done, oracle_ref="")` reaches the oracle-reference check, returns `REFUSED`, leaves the commitment open, and records the denied authority act. It never reaches classification, current-state verification, or the `DONE` transition: the empty reference fails the finite-vocabulary test before any oracle is consulted.

Exercise p. 151

1.18

Note monotonicity: the prohibited event is a write the daemon mediates and the trigger (the previous count) is committed state; top-right, regimentable, and append-only storage is the supervisor. Capability escalation: regimentable only if every effect path and the identity behind it pass through the mediator; an escalation exercised through an unmediated channel is an uncontrollable event, which moves the rule to the left column until the channel is owned. Lock-owner validity: the release is mediated and the owner is committed state, top-right, provided the actor identity is authenticated (I12); with a self-asserted identity the trigger is not witnessed and the rule drops to the bottom-right cell.

Exercise p. 152

1.19

The first gates a controllable event on a controllable trigger: both `api_call` and `spawn_child` lie in Σ_c, the policy never disables an uncontrollable event, and it is regimentable. The second must disable `internal_plan`, an uncontrollable event, in its post-trigger state, and the plant enables it there; the history `fs_write` followed by `internal_plan` witnesses failure of exact controllability. The supervisor can still preserve safety by forbidding the earlier `fs_write`; that restriction sacrifices legal histories rather than realizing the entire specified language.

Exercise p. 152

1.20

P_4 is a two-state taint automaton whose trigger is `git_push`. In the product with P_2 (no `git_push`, ever) the trigger is disabled in every reachable state, so P_4's tainted state is unreachable in the closed loop and the disablement map is unchanged: `ok` disables {`net_egress`, `git_push`} and `tainted` disables {`net_egress`, `git_push`, `fs_write`}. Controllability still holds, since

no reachable state disables an uncontrollable event. The lesson is the one synthesis is for: P_4 is vacuous under P_2, and a hand-written engine would have carried the dead rule forever.

1.21 *Exercise p. 152*

A protected-run receipt: tree hash, a suite hash the principal controls, runner or image hash, command, timestamp and nonce, exit code, and a signed result. Closure verifies every binding and the freshness. This resists free-text and stale-result manipulation; it is not universally Goodhart-proof, because the suite may be incomplete or poisoned. The honest theorem is syntactic provenance and non-replay, not semantic correctness.

1.22 *Exercise p. 152*

Notes preserve explicit goals, rationale, observations, and artifact pointers. They cannot preserve unflushed edits, process memory, open handles, provider caches or hidden state, or a latent next action the model never externalized. A successor can understand the story and still be unable to resume the exact computation.

1.23 *Exercise p. 152*

The append commits. A later delete or regressive count also commits and enters the operation log. Only then does the subscriber observe the count drop and raise or compensate. The violation is durable at the second commit; the monitor proves detection, not prevention.

1.24 *Exercise p. 152*

A content-addressed task capsule sealed at a safe tool or message boundary: base tree and worktree blobs, environment and tool manifests, open claims with epochs, commitments and acceptance criteria, exact tool input and output, pending external operations with their idempotency keys, decisions and hypotheses, a provenance-preserving transcript or compaction, and external references. Flush the ledger and quiesce mediated effects before sealing. This restores observable task state and evidence; it cannot recover hidden activations, unexposed reasoning, provider-internal caches, or unrecorded intent. Arbitrary-provider recovery is semantic continuation, not bit-identical resurrection.

Exercise p. 152

1.25

`delegate` from the initial state with a child scope not contained in the root grant's scope $\{a, b\}$, for instance $\{a, b, c\}$: with the attenuation guard off the grant is created and I4 fails immediately. No other guard can be broken in one step, because effects need the executing phase, settlement needs the verified phase, and a double settlement needs two operations.

Exercise p. 153

1.26

One candidate: a contested work unit never settles (`settle` is guarded on the verified phase, but the transition from contested back to a settleable phase is unconstrained). Add the guard, rerun the search, and confirm the state count and that the mutation suite finds a trace with the guard off; if the checker finds none, the invariant was already implied and does not earn its row.

Exercise p. 153

1.27

One daemon connection handles every claim and lock read and write; no `read_uncommitted`; checks and mutations share a transaction; respond only after commit; no out-of-band writer; a crash-free observed history. An external writer violates the single-writer and no-bypass conditions. A read-only connection needs its own API and snapshot analysis. Synchronized clocks are unnecessary: non-overlapping invocation/response intervals constrain the order.

Exercise p. 153

1.28

Start with no holder and a lease that will not expire during the trace. A's `claim(k)` completes successfully. B and C then issue overlapping `claim(k)` calls, both denied because A still holds the lease. There are three operations. Both A-acquire, B-denied, C-denied and A-acquire, C-denied, B-denied explain these results and preserve real-time order. The two denied calls may exchange positions because they overlap and leave the state unchanged; neither may move before A's completed acquisition.

Exercise p. 153

1.29

Generate identical state-machine traces against both bindings: schema initialization and migrations, acquire, reclaim, release, and expiry, overlapping calls, transactions and rollback, busy and lock behavior, encodings, pragmas, crash and reopen, fault injection. After every step compare returned values, errors, serialized rows, schema, and recovery state, including the deployed binary in CI. A

seeded fuzz misses divergences that depend on wall-clock timing, on a pragma the fuzz never sets, or on a crash at a point the fuzz never interrupts; targeted fault injection and pragma sweeps catch those. No finite suite makes I11 a theorem: that needs a formal refinement or equivalence proof, or the same verified binding on both sides. Label the suite conformance evidence.

1.30 *Exercise p. 153*

The port stays claimed and a live or ghost session row exists, but no commitment says what work owns them. Other agents see a busy port and a presence they cannot reconcile; the failed starter may retry into its own leftovers.

1.31 *Exercise p. 153*

Expose one `begin_work` endpoint that opens `BEGIN IMMEDIATE`, validates every input, inserts the claim, session, commitment, and outbox rows, and commits once; on any error it rolls back all of them. A saga would release the claim and mark or delete the session when the commitment insert failed, and each compensation can itself fail or crash between steps. All three rows live in one SQLite file, so the local transaction is both simpler and stronger.

1.32 *Exercise p. 153*

A later same-user tamperer of the database file, or a remote synchronizer that cannot rewrite an externally anchored root. The same-user process is out of scope at the substrate; the adversary becomes real at the cross-machine economy layer. Without an independent anchor or verifier, the writer can rewrite the log and recompute the chain.

1.33 *Exercise p. 154*

The release path compares the supplied actor identity to the lock owner. If a legacy endpoint accepts a self-asserted identity, the attacker submits the victim's string, the equality check passes, and the owner-scoped delete executes. The lock rule is correct relative to its input; I12 failed to authenticate the input. Complete credential and peer binding must precede every security-relevant write.

1.34 *Exercise p. 154*

The minimum useful wall is a daemon under a separate UID (or a VM), a daemon-owned database and key, a Unix-socket ACL plus kernel peer credentials, and brokered git, filesystem, and network

effects. An OS keychain protects a key at rest but not arbitrary signing requests from an equally authorized same-user process; sealed memory does not stop same-UID tracing or injection without an OS isolation policy; external root anchoring detects a later log rewrite but does not prevent it. No trusted platform module is required for strong local separation: separate privilege and complete mediation are. After effective UID separation, the former peer no longer shares the daemon's UID; the different-UID row applies. Host administrators and unmodeled side channels remain separate exclusions, not something the row change eliminates.

Chapter 2: The Anchor Protocol

Exercise p. 224

2.1

The model hands the attacker *both* the honest issuer's public key `vkA` and the HMAC key `vkB` in the clear — the worst case for a verifier that is only supposed to trust Ed25519-shaped tokens. `VerifierNaive` does not pin an algorithm; it pattern-matches on whichever type tag arrives, `tokenA` or `tokenB`, and calls that tag's own verification routine. Knowing `vkB`, the attacker builds `tokenB(ALG_B, m, hmacB(vkB, m))` for any message `m`, including one the honest `IssuerA` never signed; the naive verifier's second branch matches the `tokenB` shape, the HMAC checks out under a key the attacker already holds, and `accepted_naive(m)` fires with no matching `issued_A(m)`. `VerifierPinnedA`, by contrast, only ever destructures the `tokenA` constructor — a `tokenB` token is not merely rejected, it is a shape the pinned verifier never attempts to read. Pinning means fixing, out of band, which single verification routine and key a given context accepts, so the token's own header or shape can never be the thing that chooses how it gets checked — exactly the rule of Design invariant 2.2.2, mechanized here against an attacker who already holds every key the naive path would trust.

Pinned verdict (`proofs/anchor/token-verify/algconfusion.run.log`):

```
RESULT event(accepted_pinned(m_4)) ==> event(issued_A(m_4)) is true.
RESULT event(accepted_naive(m_4)) ==> event(issued_A(m_4)) is false.
```

The first line is the property this chapter relies on; the second is the negative control, checked in the same file so the true result is not vacuous.

2.2

Exercise p. 224

It checks that every chain C accepts corresponds to one P actually authorized, with the *same* (P, A, B, C, m) tuple: no hop can be replayed against a different message, and no hop can be spliced out of one chain into another. Each signature binds `hopBind(nonce, prev_id, next_id, message)` and the verifier checks that the nonces were issued. It inserts them into `consumed_nonce`, but never reads that table before acceptance. Consequently this model does not establish one-time use: replaying the identical chain can still satisfy the displayed non-injective correspondence. Message substitution and duplicate acceptance are different questions.

Pinned verdict (`proofs/anchor/delegation/chain-replay.run.log`):

```
1 RESULT event(chain_accepted(idP_4,idA_3,idB_2,idC_1,m_4)) ==> event
    ↪ (chain_authorized(idP_4,idA_3,idB_2,idC_1,m_4)) is true.
```

This is a different property from Property 2.D.2's capability-subset soundness: chain replay says who signed what is delivered intact; attenuation soundness says authority never grows along the way. §2.5.3 cites this exact artifact as the depth-3 evidence behind the "Delegation-chain depth" boundary.

2.3

Exercise p. 224

`proof_verify_logic_only` stubs key decoding, base64 decoding, and signature verification, using a 32-byte, dot-containing UTF-8 token and an unwind bound of 128. Its absence-of-panic result concerns only the paths these stubs reach, not correctness of `verify`. The base64 stub supplies zero bytes rather than valid claims JSON, so even a syntactically three-part input cannot reach successful acceptance. Neither real decoding nor the acceptance path was checked by this harness.

`proof_constant_time_behavior` calls the comparator once over symbolic 16-byte pairs and shows it cannot panic; it is not a two-run relational proof that the *time* taken is independent of the secret, and Kani adds nothing here beyond the source-level argument already given by inspection (no early return on a secret byte). Compiler, cache, and microarchitectural timing stay entirely outside it —

a reader should not read "Kani checked it" as "this is constant-time in the deployed binary."

`proof_capability_attenuation` asserts two concrete, hard-coded vectors, one subset accepted and one escalation rejected; it is a unit test wearing a Kani harness, not a universal statement over arbitrary capability structures. The single-hop enriched-order property and the specific multi-hop attack/control pair are what §2.D.1's ProVerif models establish, not this harness — a reader should not treat the two vectors as covering the property Property 2.D.2 states.

The bound in each case is recorded, not just asserted here: `whitepaper/corpus.json` carries an `evidencePolicy` field for `harbor-card-kani-verify-logic-only`, `harbor-card-kani-constant-time`, and `harbor-card-kani-capability-attenuation` stating exactly these limits, so a claim that drifted past this chapter's wording would be visible against the manifest rather than only against memory.

Exercise p. 224

2.4

As this chapter states it, not more: the Kani harnesses give bounded absence-of-panic for the token parser on 32-byte inputs with signature verification and base64 decoding stubbed out entirely, a source-level argument that the comparator's branches do not depend on secret bytes, and two concrete, hard-coded vectors for the capability-subset check. None of the three is a proof about the real cryptography (stubbed away in the first, untouched by the other two), about inputs larger than the harness's own bound, about the compiled binary's timing behavior, about the FFI boundary itself, or about any code path outside `harbor-card-rs`. The ProVerif models are symbolic proofs about the *protocol* under a Dolev-Yao adversary, not about this Rust source at all — the bridge between "the model" and "this source" is exactly the assumption gap Table 2.2 lists in its last column, asserted by a human, not discharged by either tool.

A stronger harness would replace the two hard-coded attenuation vectors with an arbitrary capability structure under an asserted partial-order property, so the check is over the order rather than two fixed cases, with a negative control that fails when the `is_subset` guard is removed — the same discipline the ProVerif attenuation model already applies for Property 2.D.2. It would also need to state parser refinement (that the stubbed primitives match

their real implementations) as a separate, currently absent, obligation. None of that exists yet, and this chapter does not claim it does: `whitepaper/corpus.json` pins the three harnesses' present bounds in their own `evidencePolicy` fields, and `scripts/proofs/run-proverif.py` (CI job "ProVerif / model estate") re-runs every ProVerif model named in this chapter's exercises against its committed source on every change — the freshness guarantee behind every pinned line quoted above, not a claim about the harnesses it does not mechanize.

2.5

Exercise p. 225

A valid signature proves exactly one thing: the holder of the signing key produced these bytes. It says nothing, by itself, about canonical decoding (did the verifier parse the *same* bytes the signer meant, under one canonical encoding); key-to-principal binding (whose key is this, and is that binding itself authenticated — §2.5.1's PID-reuse case is a binding failure downstream of a perfectly valid signature); audience and resource semantics (was this token issued for *this* harbor and *this* action); freshness (`exp` checked against a clock the bearer does not control); ancestor revocation (§2.2.4 — a signature does not know its own chain was withdrawn upstream); or that every consequential effect actually consults the verifier at all. Table 2.5 lists only the cryptographic set as ProVerif-verified; authorization is the larger claim built on top of it, not a synonym for it.

Because Harbor Cards are bearer credentials — whoever presents the bytes gets the capability, with no further proof of possession — a card sniffed off loopback (§2.5.3, "`localhost` is a real network") is exactly as usable to an attacker as to its intended holder for the remainder of its TTL. Re-checking per request narrows the window in which a *revoked* card is still honored (§2.2.4); it does nothing against a card that is merely stolen and still technically valid, because the verifier cannot distinguish the two bearers by the token alone.

A status table sharper than Table 2.6's three tiers would keep four things distinct rather than folding them into one word: symbolic protocol correspondence at a fixed modeled hop depth (what ProVerif proved); bounded implementation memory and control-flow checks (what Kani proved); runtime verifier integration (whether the deployed path actually calls the verified routine on every effect); and system-level complete mediation (whether every effect-producing path is reachable only through that call). This chapter's

artifacts presently evidence the first two; the last two remain separate integration and enforcement obligations, not consequences of the harness results (§2.4). Reading any result in this chapter as more than it is means collapsing exactly this distinction.

Exercise p. 225

2.6

(i) A root-vs-final verifier asks only `is_subset(cap_write, cap_root)`: since the root itself holds write, this is true, and C is accepted with write — an authority B never held and never legitimately re-delegated. (ii) Algorithm 2.4's loop checks each hop against the one immediately before it: hop 1, `is_subset(cap_read, cap_root)`, holds, so B's grant stands; hop 2, `is_subset(cap_write, cap_read)`, fails, because write is not inside what B was actually handed, so C is rejected at the exact hop where authority tries to grow back. Only the root-vs-final verifier accepts; the per-hop verifier does not.

The induction: if every hop satisfies $\text{cap}_i \subseteq \text{cap}_{i-1}$, transitivity of $\subseteq$ chains them directly into $\text{cap}_k \subseteq \text{cap}_{k-1} \subseteq \cdots \subseteq \text{cap}_0$. Checking $\text{cap}_i \subseteq \text{cap}_0$ independently at each hop supplies no such chain: it only asks whether each capability fits inside the *root*, which a re-escalating middle hop can satisfy (write $\subseteq$ root) while still expanding relative to what that hop's own parent actually held (write $\not\subseteq B$'s read). This is why the sentence following Algorithm 2.4 credits Property 2.D.2 with the single hop only: the multi-hop induction is what the per-hop *discipline* buys structurally, and its mechanized confirmation is the v6/v7 pair immediately below, not a restatement of the property at depth greater than one.

Exercise p. 225

2.7

The root holds `cap_write`. Honest A correctly attenuates to `cap_read` for B — a legitimate restriction. Malicious B, holding only that read-capped delegation, signs a *new* delegation to C carrying `cap_write`: an authority B never held, forged with B's own legitimately-issued delegation key, not by breaking any signature. `NaiveVerifier` unwraps the two-hop chain and checks `is_subset(final_cap, root_cap)` — the final capability against the root only — and never re-derives what the intermediate hop actually held. Because `cap_write` is a subset of the root, the check passes and C is accepted with full write authority, although B itself only ever held read. This is hop-2 escalation: a child re-grants an authority its own parent narrowed away, and a root-vs-final comparison is structurally blind to it.

Pinned output (`analyses/harbor_card_v6_results.txt`):

```
1 The event Accepted(id_c_1,harbor_id,cap_write) is executed at {43}
    ↪ in copy a_10.
2 A trace has been found.
3 RESULT not event(Accepted(cc_2,harbor_id[],cap_write)) is false.
```

ProVerif does not merely report the query false; it exhibits the concrete trace above as a witness.

2.8

Exercise p. 225

The only change is in the verifier: v6's single check `is_subset(final_cap, root_cap)` becomes two checks in sequence, `is_subset(mid_cap, root_cap)` for hop 1 ($A \to B$ against the root) *and* `is_subset(final_cap, mid_cap)` for hop 2 ($B \to C$ against what B actually held). Same malicious B, same forged `cap_write` delegation, same query — only the verifier's comparison target moves from "root" to "immediate parent."

Pinned output (`analyses/harbor_card_v7_results.txt`):

```
1 RESULT not event(Accepted(cc_2,harbor_id[],cap_write)) is true.
```

The identical escalation attempt v6 accepted is now rejected at the `is_subset(cap_write, cap_read)` step, because write is not a subset of what B was actually holding.

2.9

Exercise p. 225

Both models share a grant chain — a first-party caveat folded into the root HMAC, a third-party commitment `vid` keyed by a shared caveat key, and a discharge bound to the request via `hmac(BIND0, ···)`. In v2, two grants (a low-authority one the daemon discharges, and a high-authority one it never discharges) share the same caveat key — realistic, because one live session's check backs every grant minted under it. v2's naive relay computes `bound = hmac(BIND0, d1)`, binding the discharge to the *caveat* only, never to which grant it is being presented for. An attacker who holds the daemon's genuinely-issued low-authority discharge — public, since it travels in the clear — can present it alongside the public high-authority grant; the relay recomputes the same discharge chain from the shared caveat key and accepts. `RelayAuthorizesNaive` fires for the high-authority grant's signature although the daemon's `DaemonIssuesDischarge` event never named it. This is the macaroon-construction analogue of `analyses/harbor_card_v6_multihop_attack.pv`: a verifier that checks a credential against something coarser than the exact

object in front of it — there, the root instead of the immediate parent; here, the caveat key instead of the specific grant — admits a substitution.

v1's verifier folds the grant's own signature into the binding, `bound = hmac(BIND0, (g_sig, d1))`, not just the discharge chain. A discharge minted for the low-authority grant's signature cannot satisfy the high-authority grant's binding check: HMAC is modeled as a one-way keyed PRF with no destructor, so the attacker can replay the low discharge's bytes verbatim but cannot recompute the high-authority binding without the private caveat key only the daemon holds.

Pinned output, v1 (`analyses/macaroon_discharge_v1_results.txt`):

```
1 RESULT event(RelayAuthorizes(s)) ==> event(DaemonIssuesDischarge(s))
    ↪ is true.
```

Pinned output, v2 (`analyses/macaroon_discharge_v2_naive_unsound_results.txt`):

```
1 RESULT event(RelayAuthorizesNaive(s)) ==> event(
    ↪ DaemonIssuesDischarge(s)) is false.
```

Both discharges are unforgeable without the caveat key; the entire gap in v2 is that its binding forgets which grant it was issued for.

Chapter 3: The Sealed Harbor

Exercise p. 270

3.1

Both parity classes have four members, so the equal-parity pairs number $2\binom{4}{2} = 12$; those must stay indistinguishable forever. The remaining $\binom{8}{2} - 12 = 16$ pairs have different parities and may differ at explicit gate-release steps only. Table 3.7 shows how this scales: doubling the secret count roughly quadruples both counts, since both grow quadratically in n while the equal-parity share of the total approaches one half. For even n, the exact share is $(n-2)/(2(n-1))$, which gives $1/3$ at $n = 4$ and $3/7$ at $n = 8$.

Exercise p. 271

3.2

The script prints

```
[m1 leaky gate: releases raw s] caught: EQUAL-CLASS VIOLA-
TION
witness: secrets (0,2), trace load_data -> read_secret ->
```

```
submit_to_gate -> gate_release, Erin sees (('gate', 0),) vs
(('gate', 2),)
```

The pair $(0, 1)$ has different parities, so the honest gate is *allowed* to let Erin's view differ at a release step; a difference there proves nothing. The pair $(0, 2)$ has equal parity, so any difference is a violation, and it appears at the fourth step, the release, when the leaky gate emits the raw secret instead of its parity.

3.3

Exercise p. 271

The witness is `load_data -> read_secret -> BYPASS_write_E`, with Erin seeing `(('leak', 0),)` against `(('leak', 1),)`: complete mediation fails, since a release occurred with no gate decision. The two-run check sees it because the bypass write lands in Erin-observable state at a step that is not a gate release, which is a difference the property forbids for every pair, whatever their parity.

3.4

Exercise p. 271

Add a worker register t, a transition `compute`(f) that sets $t = f(s)$ for f drawn from a finite family, and let `submit` pass t to the gate, which releases $g(t)$. The property must then be stated over worlds that agree on $g(s)$ but may disagree on $g(f(s))$, and the checker must quantify over the family of f; for any non-constant f that maps an equal-parity pair to a different-parity pair, the honest gate releases a difference and the attack is exhibited. Delimited release rules the attack out by requiring that no variable used in a declassification is updated before it, which a compiler can check because the declassified expression is syntactic. A gate accepting arbitrary worker output has not established that the input to g is the committed s. It can enforce that premise by computing a restricted release itself, or retain the broader payload interface and state a different, capacity-based guarantee. The limitation belongs to that interface and its evidence, not to every possible language-model architecture.

3.5

Exercise p. 271

The compound rule disables only `net_egress`, a controllable event, in the tainted state, and never disables the uncontrollable read, so, with the read observed, its closed loop is controllable and a supervisor realises it exactly. The second rule must disable `in_context_read` itself, an event in Σ_u that the plant enables, so controllability fails on the first history that reads and the rule cannot be exactly realized by this supervisor. Detection would additionally require an observation of that read.

Exercise p. 271

3.6

Under the torn write each release logs ε but adds $\varepsilon - 1$. Client 0 releases 2: log 2, $\sigma = 1$, $\sum_\Lambda = 2$. Client 0 releases 2 again: the guard sees $\sigma + 2 = 3 \le 4$ and admits it; log 2, $\sigma = 2$, $\sum_\Lambda = 4$. Client 1 releases 1: the guard sees $\sigma + 1 = 3 \le 4$ and admits it; log 1, $\sigma = 2$, $\sum_\Lambda = 5 > 4$. The script prints exactly this trace:

```
c0.release(2) [TORN: log 2, sigma +1] ->
c0.release(2) [TORN: log 2, sigma +1] -> c1.release(1) [TORN:
log 1, sigma +0]
```

The balance never breaks the bound, so the guard keeps admitting, and the auditable log overspends.

Exercise p. 271

3.7

The full state contains the ordered ledger and both client program counters. Sorting just the ledger costs identifies states that differ only in commit order, giving 11 projected states. The five cost multisets alone are $\{\}$, $\{1\}$, $\{2\}$, $\{1, 2\}$, and $\{2, 2\}$. A denied release still advances its client's program counter, so a full state also records which requests have been processed and refused. Two interleavings that commit the same releases but differ in whether a client's later request has already been refused are distinct states with the same multiset. For example, after client 0 commits both of its releases ($\sigma = 4$), the state before client 1's first request is refused and the state after it is refused carry the same ledger $\{2, 2\}$ and different counters.

Exercise p. 271

3.8

At $k = 8$, $\varepsilon = 0.5$: basic 4.00; advanced $\sqrt{16 \ln 10^6} \times 0.5 + 8 \times 0.5 \times (e^{0.5} - 1) = 7.43 + 2.59 = 10.03$, so quote the sequential bound, by a wide margin [verified]. At $\varepsilon = 0.1$ the advanced bound is $0.1\sqrt{2k \ln 10^6} + 0.010517k$; at $k = 34$ it is 3.42 against basic 3.40, and at $k = 35$ it is 3.48 against 3.50, so the crossover is $k = 35$ [verified].

Exercise p. 272

3.9

$\beta^5 = 0.00032$, so detection is 0.99968 [verified]; the script's analytic line reads `k=5: analytic 0.99968 simulated 0.99967`. The approximation gives $1-(1-0.01\times0.8)^{100} = 1-0.992^{100} \approx 0.552$. The exact hypergeometric probability is approximately 0.554; before rounding, the approximation is lower by about 0.00143.

`a4_canary_sprt.py` · 20260816 · internal

3.10 *Exercise p. 272*

$\mathrm{KL}(p_1 \| p_0) = 0.01 \ln 10 + 0.99 \ln(0.99/0.999) \approx 0.01407$. The numerator is $0.95 \ln 95 + 0.05 \ln(0.05/0.99) \approx 4.177$; their quotient is 296.9 [verified arithmetic]. Boundary values replace terminal log-likelihoods; nominal targets replace attained errors. Overshoot, sampling uncertainty and the cap can affect the reported 359, a capped sample mean, not an exact expectation. Non-detection 0.0483 and false alarm 0.0052 are point estimates, not upper bounds. Stopping counts include either decision; no wall-clock arrival model is specified.

3.11 *Exercise p. 272*

A possible clause: canary secrecy is a named security boundary, so disclosure of the planting list is a breach with its own bond, and the list is regenerated per job. The mechanism: plant per job from a fresh seed held by Derek's gateway alone, so a stolen list need not expose later jobs. Rotation alone does not lower a fixed per-job compromise probability: if that probability remains 0.3, unconditional power is still $0.7(1 - \beta^k)$, about 0.699776 at $k = 5$. Restoring most lost power requires a separately justified reduction in list compromise or a detector not dependent on that list. Conditional on an uncompromised list, power remains $1 - \beta^k$. What no redesign restores is power against an adversary who holds *this* job's list: with the canaries stripped the leak carries none and Theorem 3.6.1 gives $1 - \beta^0 = 0$, which is why detection is priced as a bounded bond and not sold as a guarantee.

3.12 *Exercise p. 272*

Before timing, $q \cdot b = 200 \times 64 = 12{,}800$ bits. Three timing slots add up to $\log_2 3 \approx 1.585$ bits per job, so about 317 more bits, for roughly 13,117 bits in total; the timing term is why scheduled release with one slot is the design default.

Chapter 4: The Legible Swarm

4.1 *Exercise p. 368*

Resource conflict (two agents touch the same resource and clobber one another) is answered by conflict-aware claims plus actual exclusion at the effect boundary. Illegibility (work cannot be understood in bounded time) is answered by a digest whose every

material claim zooms to independent evidence. Confident wrongness (an agent asserts completion without checking) is answered by typed attestation and oracle-bound completion. Footguns (a stale premise drives an irreversible act) are answered by reversibility/stakes gates, scoped authority, and pre-effect mediation. Illegible authority (the coordinator rules without an inspectable reason) is answered by an append-only, named, zoomable decision record, including denials and omissions.

Exercise p. 368

4.2

Alice and Bob are both asked to improve `auth.ts`. Each reads the same base revision, makes a locally correct edit, and commits; without a claim or merge serialization, Bob's whole-file rewrite erases Alice's change, and no malice is required. A deadlock variant: two cooperative agents take claims on files X and Y in opposite order and each waits indefinitely for the other's claim.

Exercise p. 368

4.3

Deliberate construction chooses the authority rather than accepting whatever coalition happens to win, and makes that authority legible to the operator. An emergent stable order may instead be an oligarchy: stable, but controlled by a few agents, unrevocable, and opaque. The local daemon of §4.1.2 supplies what an emergent attractor does not guarantee: one fault domain, one policy root, and an unconditional operator exit.

Exercise p. 368

4.4

Randomize comparable real repository tasks to (a) an ungoverned fleet and (b) a fleet with only a transport but no prescribed governance; record all messages, effects, blocking decisions, and operator interventions. Pre-register two observable classes: *sovereign* characteristics (a publicly known rule, general applicability, explicit consent or adoption, stable appeal/override, reasons attached to authority acts, equal audit access) against *oligarchy* characteristics (authority concentrated in a small coalition, private channels or rules, selective enforcement, no effective exit, outcomes outsiders cannot reconstruct). Measure concentration of blocking/merging power, the fraction of authority acts with reconstructable reasons, rule consistency across agents, override success, artifact-linked claim accuracy, and operator reconstruction accuracy. Stability

alone does not separate the hypotheses; publicly inspectable and revocable rule-governed authority does.

4.5 *Exercise p. 368*

Tacit consent is inferred from remaining in or using a system. Express consent is a discrete act over named terms. Invariant 4.2.3 is express because the operator affirmatively issues a grant with scope, stakes ceiling, reversibility floor, TTL, and revocation semantics — none of which a mere pattern of continued use supplies.

4.6 *Exercise p. 368*

If an autonomy level could suppress the override, it could make its own authority irrevocable: keep executing, hide the stop channel, or race destructive effects after consent is withdrawn. The override must be out-of-band, higher priority than agent work, and coupled to effect revocation, not merely a UI flag.

4.7 *Exercise p. 368*

The operator signs grant g; the daemon validates and records it; an eligible low-stakes action cites g; the daemon mediates the effect; the receipt records action, reason, artifact, and grant version; expiry or revocation removes g from the active set and fences later effects. The legible-sovereign rule binds at every authority transition, especially issue/revoke and admit/deny — the explicit witness record is where reconstructability is discharged.

4.8 *Exercise p. 369*

Use one complete append-only decision ledger, not a second ranked dashboard. For every candidate item, store candidate ID, source/evidence pointers, policy and model version, input features, score components, threshold/lane, surfaced-or-suppressed verdict, reason codes, expiry, and a counterfactual such as "would surface if anomaly were 0.08 higher." The operator can query suppressed items since a cursor, sort by any raw field, and sample items uniformly rather than through the production ranker. Decisions are immutable; their current relevance expires by signal-specific TTL (§4.7.4) and is recomputed from raw state. Run independent random omission audits and expose raw event-log access. The regress terminates at completeness plus direct sampling: the base ledger records every candidate and every ranking decision, and the counter-surface is a deterministic query over it, so no further ranker is needed to rank the ranker's omissions. The remaining

trust assumption is the complete event-capture boundary; that assumption should be tested and externally anchored, not hidden behind another summary. (Same construction answers Exercise 4.49.)

Exercise p. 369

4.9

Port number: flatten, as structured administrative state. Failed-test stderr: preserve verbatim or by a content-addressed pointer. "The auth refactor felt risky so I split it": preserve verbatim and label it self-narration, not fact. Commitment-status enum: flatten. The 600-line diff: do not paraphrase away — retain the immutable diff or a verified pointer and summarize only administrative metadata.

Exercise p. 369

4.10

Two models may share training biases, prompts, sources, and failure modes. Agreement alone does not determine how often both checks miss the same defect. A check against repository state or a protected test result can add evidence beyond the original narration. A separately controlled reviewer can reduce conflicts of interest, but separate control does not itself prove independent errors; the evidence and common failure modes still need examination.

Exercise p. 369

4.11

A typical CI dashboard usually fails as follows: deploy is irreversible but the green tile does not force a log/diff view; administrative state and diagnostic detail are flattened together; the summary is generated from job narration rather than independently from receipts; and a skipped/cancelled check carries no named authority reason. It therefore violates Invariant 4.3.2 wherever a material green claim has no total zoom path to the exact tree, suite, runner, and output that produced it.

Exercise p. 369

4.12

L2 can compare structured narration claims against independently derived artifacts: claimed changed files versus the tree diff, "tests passed" versus a protected runner receipt, "no deletion" versus Git objects, and claimed command results versus brokered tool logs. Emit a contradiction event on mismatch. This catches deterministic report/artifact divergence, deleted tests, fabricated execution, and omitted changed paths. It cannot establish semantic correctness, detect a poisoned acceptance suite, observe effects that bypass logging, or distinguish two artifacts that are syntactically consistent

but jointly wrong — those require effect confinement, independent evaluation, or later outcome evidence.

4.13 *Exercise p. 369*

Verifiable zoom says narration is not ground truth. A reviewer with no independent evidence would merely add another narration. Adversarial review follows because a consequential claim needs an evidence channel controlled independently of the actor whose work is judged — exactly what Invariant 4.3.3 already demands of any zoom target.

4.14 *Exercise p. 369*

Read Invariant 4.4.6 as: prevent a prohibited state when every decisive effect can be intercepted before it occurs; otherwise detect, contain, and compensate without pretending prevention. A uniqueness constraint regimenting "one holder per exact key" is a regimented rule. A post-commit monitor that detects an out-of-scope edit and triggers rollback/salvage is detected-and-compensated — the bad prefix already happened.

4.15 *Exercise p. 370*

For "the migration applies cleanly," the verifier is an isolated migration run against a declared base schema/database, with exact migration hash, runner image, exit status, and resulting-schema check; *done* and any merge/settlement are gated on that receipt. "The API feels ergonomic" has no mechanical truth predicate: it must go to a declared human/expert or plural judge with preserved evidence and uncertainty, and absent that authority, completion remains provisional or fails loudly.

4.16 *Exercise p. 370*

The honest ladder: (1) use a deterministic verifier where the property is decidable; (2) require an independent bounded evaluator or human for a declared subjective criterion; (3) record a provisional result, confidence, appeal window, and residual risk; (4) return unresolved/fail-loud when no authorized evaluator can support the claim. "I cannot verify this" is correct when the available evidence cannot distinguish success from failure under the contract; it is a cop-out only when an affordable, authorized verifier was specified and simply not run.

Exercise p. 370

4.17

The corner is priced in closed form, and the result belongs to *The Harbor Economy*, which carries it as Thm. 6.1.1; §4.4.3.1 states the two consequences that matter here. The route: with death at rate ξ and succession at rate η the sole owner's mean wait decreases in skill to an infimum $W_\infty = \xi/(\eta(\xi+\eta))$, the residual downtime a request meets when it arrives during an outage, which it never attains — so skill cannot buy back downtime past that floor. Writing $K = A/(w\lambda) + W_{\text{pool}}$ for the pool's cost line, pooling dominates at *every* skill premium iff the floor has reached that line, $W_\infty \geq K$, which rearranges to $\xi/\eta \geq D^\star = \eta K/(1-\eta K)$. Whether that threshold sits above or below what an operator can promise is an empirical question about the concrete handoff process (on-call depth, handoff quality, documentation): if the promised η implies $\xi/\eta > D^\star$, no amount of hiring a better specialist recovers sole ownership, and the honest fix is a succession plan, not a stronger specialist.

The threshold $D^\star$ is defined only when $\eta K < 1$. If $\eta K \geq 1$, the downtime floor cannot reach the cost line and this obstruction does not arise. The model assumes failures can occur even while the server is idle, with preemptive-resume service after repair. Under failures only during active service, the infinite-skill floor is zero instead. The failure model therefore belongs in the answer, not merely in a numerical implementation. Exercise 4.18 checks the never-failing boundary; the companion economy chapter develops the breakdown model.

Exercise p. 370

4.18

By hand: $g(\rho, 2) = 1 + 2\rho - \rho^2$, so $g(0.5, 2) = 1 + 1 - 0.25 = 1.75$ [verified, closed form]. The crossing solves $g(\rho, 2) = \tilde{g}(\rho, 2) = 1 + \rho/(1-\rho)$; clearing denominators gives $\rho(\rho^2 - 3\rho + 1) = 0$, and the root in $(0, 1)$ is $\rho = (3 - \sqrt{5})/2 \approx 0.382$ [verified, closed form]. The script's output line is exactly this: `[PASS] c=2 crossing of g~ and g at rho=(3-sqrt5)/2 rho=0.38197, g=g~=1.61803`, and its charted table reports `rho=0.5 c=2: 1.750 [2.000]` for the exact and naive values side by side [internal, `b8_specialization.py`]. The naive $\tilde{g}$ over-prices the boundary here and selects the wrong arrangement for the roadmap-owner example of Eq. 4.2.

Exercise p. 370

4.19

By hand, at $\delta = 0.05, w = 1$: $u^\star = \delta w/(1+\delta w) = 0.05/1.05 = 0.0476$ [verified, closed form]. The script confirms this is one of

twelve exactly-matching (δ, w) points (`[verified] uniform-linear u* = delta*w/(1+delta*w) matches bisection (12 pts)`) and then reports the tuning band under the R3 constants $C_{\text{miss}} = 100$, $c_{\text{att}} = 1$, $C_{\text{fa}} = 5$: `A = 3.3: delta_min = 0.034993, delta_max = 0.071975 -> band NONEMPTY` and `A = 2.0: delta_min = 0.395644, delta_max = 0.071975 -> band EMPTY` [internal, `b7_escalation_band.py`], i.e. $[0.0350, 0.0720]$ at $A = 3.3$ against an empty feasible set at $A = 2.0$ because $\delta_{\text{min}} > \delta_{\text{max}}$ there. The honest reading: when the band is empty the fix is capacity or evidence quality (raise A or lower C_{miss}), never forcing a δ that does not exist.

4.20 *Exercise p. 370*

The no-inspection loss πC_{miss} diverges, while the all-inspection loss $(1 - \pi)C_{\text{fa}} + c_{\text{att}}$ stays finite, so all-inspection eventually wins among these two policies. The ratio alone is insufficient: set $\pi = 0.001$, $C_{\text{miss}} = 1$, $c_{\text{att}} = 1$, and let $C_{\text{fa}} \downarrow 0$. The ratio diverges, but no inspection costs 0.001 while inspection costs at least 1. Imperfect inspection or a broader policy class needs its own analysis. Nothing follows from $r \to 1$ alone because r does not appear in the equation: a calibrated model must connect reputation to current, class-specific danger probabilities. Any hard authorization constraints remain separate from this loss minimization.

4.21 *Exercise p. 371*

Use the formatting and migration classes in Table 4.6. Suppose a single knob imposes the same randomized inspection probability q on both, independent of the item. Formatting loss is $(1 - q)0.005 + q(1.199)$, minimized at $q = 0$; migration loss is $(1-q)100+q(1)$, minimized at $q = 1$. No common q attains both class minima. A pooled rate can hide that difference. This counterexample assumes the knob really imposes a common probability; a common interface setting that implements different class thresholds need not have the same defect.

4.22 *Exercise p. 371*

Campbell pressure appears when canaries are selected (recognizable templates), exposed (agents/operators learn signatures), scored (catch rate substitutes for real defect discovery), and acted on (friction creates an incentive to optimize the metric). Time-on-task and breaks also confound the score. Test a blinded, refreshed holdout drawn from real failure transformations; separate probe

generation from scoring; stratify by exposure time, difficulty, and base rate; and compare against independent real-defect outcomes. These measures may reduce distortion but do not guarantee it. The test must also price interruption costs and the consequences of changing grants.

Exercise p. 371

4.23

Model each digest policy d by (token cost $T(d)$, human time $H(d)$, expected decision loss $L(d)$) and compare nondominated policies. A Pareto frontier exists for a finite policy set, but there is no universal inverse relation: removing redundant context can lower both token cost and operator time; beyond that free-compression frontier, preserving evidence often trades tokens for review effort or a higher error rate. Optimize under a loss or safety constraint, e.g. $\min \alpha T + \beta H + L$ subject to zero omission of hard obligations, and report the frontier empirically rather than claiming one universal digest optimum.

Exercise p. 371

4.24

Randomize operators or time blocks among manual-tax, automation-only, projection/freeze-probe, and adaptive-tax conditions, on matched repositories with fixed surprise events. Probe without warning: next-three-effect prediction, takeover completion time, false-belief rate, Brier score, post-interruption recovery, defect catch rate, NASA-TLX, and retention weeks later. Measure disablement/avoidance as an outcome in its own right. A useful tax improves later projection/takeover after controlling for interruption cost; mere annoyance raises workload and disablement without those gains.

Exercise p. 371

4.25

If the mediated writer has spare capacity but review demand exceeds the operator's available attention, improving write throughput cannot clear the review queue. Indices, filtering, and evidence-directed summaries then address the bottleneck. If durable writes instead arrive faster than the writer can commit them, write throughput still binds. Serialization supplies an ordering rule, not unlimited capacity; the hypothesis needs workload measurements.

Exercise p. 371

4.26

A directory pays at write time to normalize metadata and update indices: typically $O(\log n)$ per indexed insert (or expected $O(1)$ for a

hash index), plus storage. It buys selective reads in $O(\log n + k)$ for k results rather than $O(n)$ roster scanning, along with a queryable provenance trail.

4.27 *Exercise p. 371*

A briefing could locate unfinished work; an honest attestation could identify which checks ran; a completion receipt could locate the resulting artifact. Each supports a decision about the single actor rather than selecting among collaborators. Accuracy and reduced decision effort must be evaluated; a surface's existence alone does not establish that benefit.

4.28 *Exercise p. 371*

At $k = 0$ there is only one critical subset (the empty one) and the floor is zero. For small positive k the cost is roughly $k \log_2(N/m)$ when $k \ll m, N$. At $k = N/2$, exact identification ($m = k$) costs about N bits, while opening almost everything drives the floor toward zero as $m \to N$. The danger fraction alone is not enough; the review budget m matters just as much: mostly-safe work is cheap to digest only when the system can reliably identify that sparsity, and mostly-dangerous work forces m up toward N, where the floor stops paying for itself.

4.29 *Exercise p. 372*

The theorem is conditional on an encoder that knows the true critical set. If an adversary controls apparent criticality and can make a critical artifact observationally indistinguishable from inert artifacts, any policy that opens fewer than all N can be forced to miss one — the same narration-versus-artifact gap Exercise 4.12 probes at L2. Zero miss then requires inspecting all N, a trusted independent signal that restricts the adversary, or a randomized guarantee stated probabilistically. The combinatorial bound still applies once a true set is fixed; it does not by itself solve adversarial set identification. An adaptive adversary requires a separately stated observation and timing model; there is no general replacement obtained merely by inserting an indistinguishability-class size into the binomial formula.

4.30 *Exercise p. 372*

By hand: $k \log_2(F/k) = 20 \log_2(50) = 20 \times 5.64 = 112.9$ [verified, closed form]. The script's row at F=1000, k=20 reports flat opens = 1000, adaptive opens = 232.9, ratio = 4.3× [internal, `b1_frontier.py`]. The ratio is far below the F=2500, k=10 point's

15.3× because this row is in the *dense*-flag regime ($k/F = 0.02$ here versus 0.004 there): the theorem's guaranteed speedup requires a sufficiently sparse flagged set, and this row is the chapter's own example of where the advantage shrinks rather than disappears.

Exercise p. 372

4.31

For $0 < \alpha < 1$, accept "clean" only when r independent queries on the same group all report clean; stop as soon as any query reports dirty. A positive group is wrongly discarded with probability at most α^r. For $0 < \eta < 1$, choosing $r = \lceil \log(1/\eta)/\log(1/\alpha) \rceil$ gives per-group error at most η; at $\alpha = 0.1$, three calls give 10^{-3}. A dirty verdict needs no confirmation because false positives are excluded by hypothesis. The noisy recursion visits a subset of the groups visited by the noiseless algorithm: an erroneous clean verdict can prune that tree but cannot add nodes. Writing the theorem's full bound as

$$Q_0 = 2k\lceil \log_2(F/k) \rceil + 4k,$$

the repetition scheme therefore uses at most rQ_0 oracle calls. For a fixed repetition count, this bound retains its sparsity dependence; whether it beats flat inspection also depends on r and the required reliability.

Three limitations remain. First, per-group error is not the probability of losing a positive anywhere in the tree. A union bound over the at most Q_0 visited groups gives

$$\Pr(\text{any positive is lost}) \le Q_0\alpha^r.$$

Thus, for $0 < \varepsilon < 1$, choosing $r = \lceil \log(Q_0/\varepsilon)/\log(1/\alpha) \rceil$ suffices for a whole-run failure bound of ε, and hence for each individual positive as well. The $4k$ term must be retained when k is near F. If k is unknown, use the theorem's k-free bound $2F - 1$ in place of Q_0 for this choice. At $\alpha = 0$, one call per group is exact; at $\alpha = 1$, the assumed error bound supplies no finite repetition guarantee. Second, the independence assumption is doing heavy work. Repeating a deterministic linter on the same input gives no new evidence. Independently randomized calls to one tool may satisfy the assumed error model; different tools may still share errors. The requirement concerns the sampling and errors, not the tool names. This is the same distinction between diversity and conditional detection developed in *From Spawn to Person*. Third, and unresolved: the guarantee above is probabilistic where Theorem 4.6.5 is exact, so a chapter that prices oversight in bits now owes an account of what a 10^{-3}

chance of a silently dropped subtree is worth against the queries it saves, and that is a decision-theoretic question this chapter does not answer. The noisy group-testing literature is where to start — Atia and Saligrama give information-theoretic limits for noisy non-adaptive designs, and Cheraghchi gives noise-resilient constructions — but neither is written for the adaptive, correlated-oracle case above, so the obligation is to state the model before importing a bound, not to import one and hope.

George K. Atia and Venkatesh Saligrama. Boolean compressed sensing and noisy group testing. *IEEE Transactions on Information Theory*, 58(3):1880–1901, 2012.

Mahdi Cheraghchi. Noise-resilient group testing: limitations and constructions. *Discrete Applied Mathematics*, 161(1–2):81–95, 2013.

4.32

No monotone scalar transform can identify collaborator fit and operator regret on a domain containing both high-fit/low-risk and low-fit/high-risk items, so one ranker cannot serve both jobs. Discovery minimizes routing loss (or maximizes capability fit). The attention head minimizes total expected loss including inspection costs. Under Theorem 4.7.3's additive, no-cross-effect assumptions, at most k equal-resource slots go to the largest positive inspection benefits b_i. Unequal inspection times require a constrained allocation. Benefit-per-second ranking alone is not generally optimal for indivisible inspections: benefits $(3, 4)$, times $(2, 3)$ seconds, and a three-second budget make ratio-greedy select benefit 3, while the second item alone yields 4.

Exercise p. 372

4.33

Exercise p. 372

An exaggerated low-risk alert can displace an urgent review from a finite attention budget; a missed irreversible action may then dominate the harm of a reversible routing mistake. Conversely, a manipulated discovery ranker could select an unqualified actor for a consequential task while the false alert only costs a minute. Both surfaces allocate valuable resources. Their relative risk depends on incentives, downstream authority, verification, and the cost of the particular error; there is no universal ordering by surface type.

4.34

Exercise p. 373

Without expiry, the dead holder's row remains authoritative and the live agent is denied forever. With a lease, heartbeat-based expiry, and a new fencing epoch (Invariant 4.7.6), the stale lease becomes ineligible; a lazy sweep on the next acquire removes it; the live agent receives a higher epoch and proceeds. Merely decaying a display score is insufficient: ownership must actually expire, and every effect boundary must reject the stale epoch.

Exercise p. 373

4.35

Retain the escalation's evidence, the dismissal reason, and an independent adjudication or appeal outcome. A confirmed false escalation may trigger the proposed debit under a declared rule. A genuine warning that was dismissed must not be penalized merely for that dismissal; correcting the decision and preserving the evidence are part of the recovery path. Adjudication can itself be wrong. Measure manipulation, false debits, suppressed genuine warnings, appeal errors, and interruption costs before and after the policy. A positive cost alone proves neither effective deterrence nor harmlessness.

Exercise p. 373

4.36

Accept/reject is useful implicit relevance feedback but is confounded by position, exposure, workload, and the incumbent policy. Learn with logged propensities, small randomized exploration, counterfactual evaluation, and a held-out labeled sample; preserve refusals and downstream task outcomes. If acceptance directly trains what is shown, the loop can entrench its own choices, so it cannot be the sole ground truth.

Exercise p. 373

4.37

Use hard expiry for leases and presence; state transition or supersession for commitments; summary plus source provenance for decisions and rationale; immutable storage for evidence; and eviction plus a content-addressed pointer for reconstructable diffs and logs. Hints may decay continuously. Never decay a still-open obligation, and never recursively summarize prior summaries.

Exercise p. 373

4.38

Both are projections from durable sources, but they need not select the same records. The operator needs evidence for oversight; the successor needs enough state to continue. Theorem 4.8.1 gives an explicit one-record budget where these objectives conflict. Their shared requirement is preserved provenance, so neither view should treat the other reader's summary as its only source.

Exercise p. 373

4.39

Context past the model's effective window costs tokens while degrading retrieval and reasoning. Removing redundant or inert material and edge-placing hard facts can therefore reduce spend and

improve accuracy simultaneously, provided obligations and source pointers survive the cut.

4.40 *Exercise p. 373*

Briefing: reader is an arriving agent; drops old conversational detail; keeps current roles, work, constraints, decisions, and source links. Attention queue: reader is agent/operator now; drops low-value candidates from the immediate view; keeps highest value-of-inspection items plus the suppression ledger. Episodic memory: reader is a future incarnation; drops transient chatter; keeps typed incidents, decisions, outcomes, and retrieval features. Pheromone: reader is the swarm; continuously drops stale coordination weight; keeps fresh shared traces. Resurrection handoff: reader is a successor; drops nonessential transcript detail; keeps the task capsule, artifacts, obligations, pending operations, and provenance.

4.41 *Exercise p. 373*

Context rot → compact earlier to an effective-window budget. Lost-in-the-middle → shorten and edge-place mandatory facts. Recursive-summary collapse → re-derive every compaction from raw durable artifacts. Over-flattening → require a total zoom path from every material claim. The third cure is the one that matters most: it prevents model noise from becoming the next round's source of truth.

4.42 *Exercise p. 374*

"Replay from digest alone" is neither sound nor gaming-resistant on its own: a digest can overfit the smoke task or omit facts the sampled successor never needed. Give a successor the digest plus authorized content-addressed source links, then measure next-step success, false beliefs, obligation recall, time, and tokens on hidden continuation tasks. Add a cheap structural invariant that does not require replay: every open claim, decision, commitment, risk, and pending external effect has exactly one valid digest entry or pointer. Run the expensive full replay only on a random sample.

Exercise p. 374

4.43

Budget allocation over a precedence DAG is a knapsack/resource-allocation problem and is NP-hard in general. First reserve a minimum viable budget for each mandatory node and any predecessor needed to make its output exist. Allocate the remainder by estimated marginal outcome value per token while respecting precedence, and recompute after each result. For a monotone submodular value function under a cardinality budget, greedy earns the standard $(1 - 1/e)$ approximation; for arbitrary complementarities there is no such bound. Dynamic programming is pseudo-polynomial for small integer budgets or trees. A reviewer's tokens have near-zero value until the worker artifact it reviews exists.

Exercise p. 374

4.44

The single-reader floor is 5.9822 bits and the joint floor is 12.7662 bits. The script's reported 2.13× compares the joint floor with only one single-reader floor. Dividing instead by $2(5.9822) = 11.9644$ gives 1.0670. The joint digest permits eight opens total; each separate reader may open eight. Neither ratio measures an equal-total-budget implementation saving. The recorded sweep reports `violations: 0/16 -> floor HOLDS` and `ratio: 2.13x` [internal, `a7_experiment.py`]. Its zero violations concern the finite tested configurations and the script's sampling tolerance, not every possible encoder. The counting argument proves the lower bound; the sweep checks the tested implementations against it.

Exercise p. 374

4.45

At $h = k = 8$: $k/(k - h + 1) = 8/1 = 8.0$, recovering the classical unaugmented result that a cache of size k is k-competitive against an offline optimum given the very same cache size — the hardest comparator, no resource augmentation. At $h = 1$: $k/(k - h + 1) = 8/8 = 1.0$, the fully resource-augmented regime, where online's cache k so outsizes the comparator's $h = 1$ that the ratio collapses to 1. The chapter's own $k = 8, h = 4$ instance, at 1.6, sits at the midpoint, pricing a comparator with half of online's verified capacity.

Exercise p. 374

4.46

Agents are the multitude bound by the common coordination protocol; the daemon is the sovereign-as-actor that applies the rules; the

operator is the author/principal who originates and can withdraw authority. Calling the operator the sovereign collapses author and executing authority and makes the supposed covenant among agents incoherent — there would be no distinct actor left to hold accountable to the author's terms.

4.47 *Exercise p. 374*

Voice is honest only if refusal has consequence for the authority. The unconditional override supplies exit: the operator can revoke the grant when explanations or appeals fail, rather than being forced to keep negotiating inside a system they cannot stop.

4.48 *Exercise p. 374*

The grant is discrete, explicitly issued, scoped, time-bounded, and revocable and audited. "Started using the tool" has none of these properties: it is continuous rather than discrete, inferred rather than issued, unscoped, open-ended, and carries no explicit revocation act.

4.49 *Exercise p. 374*

Same construction as Exercise 4.8: one complete, append-only decision ledger recording every candidate, its ranking decision, reason codes, and a counterfactual, queryable directly and sampled independently of the production ranker rather than re-ranked by a second dashboard. The regress terminates at completeness plus direct sampling — the base ledger already records every candidate and decision, and the counter-surface is a deterministic query over it, so no further ranker is needed to audit the ranker's omissions. The remaining trust assumption is the completeness of the event-capture boundary itself; that assumption should be tested and externally anchored, not hidden behind another summary.

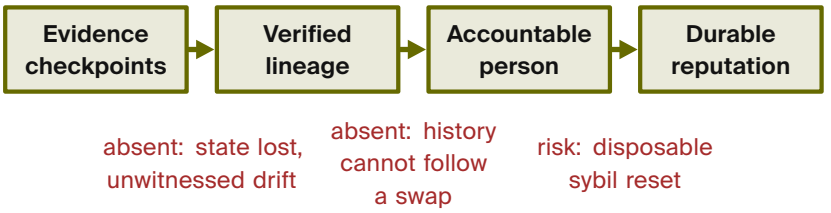
Chapter 5: From Spawn to Person

5.1 *Exercise p. 468*

A spawn is a temporary process filling a role; a person is a role-holder whose accountable continuity survives replacement of that process. The convention makes no claim about consciousness or moral status.

Exercise p. 468

5.2



Read right-to-left within the proposed architecture. Without outcome history, a buyer loses this history-based trust input; direct evaluation or collateral may still support a trade. Without a stable, liability-bound identity, outcomes may attach to a disposable incarnation. Whether reset escapes a sanction depends on the newcomer and admission rules of Theorem 5.6.1. Without a witnessed succession link, the system cannot establish that recorded work belongs to the claimed lineage. Missing task state impairs resumption, while missing outcome evidence impairs historical evaluation: these are different losses. The chain is not literally linear. Identity and witnessed outcomes are parallel inputs; checkpointing helps resumption but is not logically necessary for every reputation system.

Exercise p. 468

5.3

The human operator is a moral/legal person but should be represented economically as the *principal* above agent identities (Def. 5.6.2). If one principal can mint unlimited apparently independent agent-persons, policies that count identities rather than shared control can permit sanction resets, duplicated voting weight, reused collateral, or reputation farming. Those failures depend on the admission, voting, and settlement rules; minting another credential alone does not prove they occur. Bind every agent lineage to a principal credential, cap principal-wide exposure and newcomer credit, and disclose shared control: the actor may change; the economic liability root must not.

Exercise p. 469

5.4

Capable-and-permitted: an agent has a test-runner tool and authority to run the suite. Capable-but-forbidden: the process can invoke raw `git push --force`, but policy forbids it. Permitted-but-incapable: the role is authorized to deploy, but the incumbent has no deploy credential/network route.

5.5 *Exercise p. 469*

Reputation attaches to the person's continuity witness and witnessed outcomes, not to the abstract role's O, C, or A. A role is fillable and history-free; otherwise a fresh spawn could inherit the prior holder's credit merely by taking the same job title, exactly the attribution Alice loses in §5.1.

5.6 *Exercise p. 469*

Keep a role template $(O_{\text{required}}, A_{\text{role}})$ separate from an incumbent capability set C_{subject}. At assignment require each current obligation to satisfy an authorized feasible plan: for each obligation o, there must be a plan whose actions lie in $A_{\text{role}} \cap C_{\text{subject}}$ and whose success satisfies o. Obligations and actions are different types of object, so do not take their set intersection directly. Keep $C_{\text{subject}} \setminus A_{\text{role}}$ as capable-but-forbidden and $A_{\text{role}} \setminus C_{\text{subject}}$ as authorized-but-unavailable, which triggers provisioning or reassignment. Never equate C with A.

5.7 *Exercise p. 469*

Direct succession is one validated step; lineage reachability follows a chain of steps. A successor can read the predecessor's handoff yet present an unrelated identity with no valid succession record. Task information survives, but the accountable chain does not. Calling the process psychologically continuous would add a claim the records do not establish.

5.8 *Exercise p. 469*

In the manuscript's analogy, the body-lease is the replaceable live substance/process; the actor-soul is the durable identity/mailbox/history that carries consciousness-like continuity. Respawn replaces the body-lease and should retain the actor-soul only under a valid succession event.

5.9 *Exercise p. 469*

Represent a fork as a lineage DAG, never as two copies of one linear identity. Both children may display the ancestor's evidence with an explicit inherited/discounted prior, but neither duplicates the ancestor's outstanding authority, collateral, or spendable reputation. Allocate a fresh branch ID, split or revoke exclusive capabilities, and aggregate risk at the common principal/ancestor. Copying full reputation to both invites credit duplication, quorum multiplication, and

double-spending of grants; giving it to only the first child creates a fork-race attack. The transfer-balance calculation below proves conservation for that accounting rule; it does not by itself prove all the authority or fairness claims in this design sketch.

Exercise p. 469

5.10

The script prints: "`sum of inherited priors over the closure = 0.900 + 0.810 + 0.729 = 2.439 > q = 1.0`" for the budget-only counterexample, then "`4000 random DAGs … 0 violations; max live/witnessed ratio observed = 0.999995`" for the transfer-semantics sweep, then "`an 8-way copy-fork of one outcome yields 8.2x its witnessed value`" for the copy-full mutation. By hand: $0.9+0.9^2+0.9^3 = 0.9+0.81+0.729 = 2.439$, three terms of a geometric series, confirming the closure-sum counterexample without running anything; under transfer semantics the same chain instead *debits* each hop, so the live total telescopes to $0.9^3 = 0.729 \leq 1$. The sweep's 0 violations are a finite observed result, not the general proof.

Exercise p. 470

5.11

The checkpoint subsystem is the specifically weak one: it forwards notes rather than execution state. The outcome-ledger subsystem is also only partial in the implementation: commitments and oracle closure exist, but neutral graded outcomes and reputation binding do not.

Exercise p. 470

5.12

Memory retains the session, notes, and explicit decisions. The current checkpoint may retain a handoff and perhaps a committed diff, but not working process state, tool handles, pending effects, or hidden provider state. The outcome ledger retains the commit only if it is bound to a work unit and independently witnessed; a naked Git commit is an artifact, not proof of accepted delivery. A strong checkpoint would also preserve the worktree, environment, open claims/epochs, acceptance state, tool receipts, and the idempotency journal.

Exercise p. 470

5.13

A useful death taxonomy, in increasing order of what survives: (1) *record survival* — logs/notes only, reconstruction is manual; (2) *artifact survival* — a content-addressed worktree and outputs survive, computation is rerun; (3) *semantic task capsule* — artifacts plus

obligations, decisions, transcript/projection, tools, and a pending-effect journal, so a new provider can continue observably; (4) *execution snapshot* — process memory/runtime state survives under a controlled runtime, rarely portable; (5) *latent state* — unexposed activations, KV cache, or "intent" dies, fundamentally unrecoverable from an arbitrary provider. Checkpoint guarantees should state which tier they provide.

5.14 *Exercise p. 470*

Correctly scoped, Property 5.6.1 says reputation has economic force only to the extent that the accountable principal cannot cheaply discard or multiply the liability-bearing identity. Whitewashing escapes a negative record when newcomer access is better; a Sybil attack multiplies votes/credit when the mechanism counts identities. Non-forgeability is necessary for those mechanisms under those access rules, but it is not sufficient by itself.

5.15 *Exercise p. 470*

Allowing work preserves an opportunity to demonstrate competence, but a reduced earning ceiling can still exclude an honest entrant who cannot finance the initial low-income periods. Neither policy is unconditionally better without a participation and payoff model. Under Theorem 5.6.3's fixed continuation payoffs, resetting to avoid a sanction of present value S is unprofitable when $S \leq W_\delta$. For its separate one-shot deviation model, weak deterrence instead requires $G \leq p\delta^\tau W_\delta$. Preserving access to work does not by itself establish either condition or remove the newcomer's liquidity constraint.

5.16 *Exercise p. 470*

Principal binding is the decisive defense: aggregate reputation, exposure, counterparties, and sanctions across the 1,000 agents, so self-trades add little or no independent evidence. A listing fee raises attack cost but only deters while total fees exceed the minted value. Sampled adversarial re-audit detects fabricated work, with false-positive burden approximately the sampling rate times the honest high-volume population and the adjudication error. Add counterparty-diversity weighting (sharply diminishing credit per principal pair and per correlated pair), independent payer stake, and principal-wide reserve limits; the sampled re-audit rate

that makes farming unprofitable is the $\rho^\star = G/(dB)$ of §5.11, with G the value a farmed record can mint.

Exercise p. 470

5.17

The script prints "`schedules tested = 76000`", "`dominating schedules = 0/4000 instances (expected 0)`", and "`min (H(g) - H(cliff)) = +2.817e-02 (>= 0 means undominated)`"; the exchange step's table shows H at an all-mass-at-t schedule growing as $G_{\max}(\delta_h/\delta_f)^t$ and reports "`strictly increasing in t: yes`". By hand: with $\delta_h > \delta_f$, $(\delta_h/\delta_f) > 1$, so its powers strictly increase in t — deterrence bought later always costs the honest newcomer more, which is the whole exchange argument in one inequality.

Exercise p. 471

5.18

The script prints "`reachable states to depth 7: 747 (467 post-migration, 693 with an open sanction window)`" and reports zero violations of Def. 5.6.1 among those enumerated states; the four mutants (drop scope, drop clause (i), drop clause (ii), drop clause (iii)) are each caught with shortest counterexamples 1, 2, 2, 2 steps. The one-step break is the scope mutant `m0`: keying sanctions by *(identity, engine)* instead of by identity alone lets a single dishonest outcome escape through a sibling engine key inside the *same* identity — no migration needed at all. By hand: this is why the intact protocol sanctions per *principal* while pricing per *(principal, engine)*; collapsing that distinction gives the one-step violation of Def. 5.6.1.

`b5_engine_substitution.py` · 20260816 · internal

Exercise p. 471

5.19

Local non-forgeable identity proves that, inside one trusted daemon, an actor cannot freely impersonate or re-pick its daemon-issued handle. Cross-operator attestation must additionally bind a key to a durable principal across a hostile operator boundary, establish proof of possession/freshness, aggregate related agents, and support revocation and audit.

Exercise p. 471

5.20

That sentence secretly assumes the cross-operator stone. With only local identity, Bob learns at most "Alice's daemon asserts this actor/key." A hostile Alice can mint arbitrary local actors, rewrite local history, or misstate principal separation.

5.21

Exercise p. 471

A witness log proves publication order and exposes equivocation; it does not prove who controls a key or that two keys are distinct principals. Close the gap with a scarce external root: hardware/remote attestation, an organizational/KYC issuer, a bonded sponsor chain, or a combination. A transparency log makes bindings and rotations auditable but not true; a sponsor creates economic accountability but can cartelize; a shared identity issuer centralizes and leaks privacy; hardware roots attest devices/code, not unique humans. State the chosen trust and Sybil assumptions.

5.22

Exercise p. 471

No. A Bradley–Terry fit models pairwise outcomes through relative strengths; a finite unregularized likelihood estimate also requires suitable comparison data, without separation. Elo updates ratings sequentially, even if the full history is retained. Stationarity versus skill drift is a modeling choice; update latency and the cost of refitting are practical constraints. Compare predictive performance on the intended task rather than infer superiority from history retention alone. If abilities drift, evaluate a dynamic or recency-aware model instead of assuming that all past observations describe one fixed skill.

5.23

Exercise p. 471

A binary protected acceptance test can update a beta-Bernoulli reliability estimate (or a calibrated per-task success model). Human aesthetic preference is naturally pairwise and may use Bradley–Terry/TrueSkill with judge effects. EigenTrust expects a who-trusts-whom graph and therefore fits neither raw task-outcome signal directly.

5.24

Exercise p. 471

If identities are self-chosen, an exact Bayesian estimator simply produces precise scores for disposable names. If the agent authors the “oracle,” a sophisticated estimator precisely learns manipulated labels. Better statistics cannot repair missing provenance, adversarial labels, or liability binding; secure the event-generation substrate first.

Exercise p. 472

5.25

Stationary arm rewards versus model/skill/version drift; one scalar reward versus buyer-weighted quality vectors; a non-strategic environment versus Goodhart, collusion, and whitewashing; and an observed/trusted reward versus a capturable grader. These are application requirements, not assumptions of every bandit model; adversarial and non-stationary models remain available.

Exercise p. 472

5.26

A contextual bandit is appropriate for one principal privately routing its own tasks among candidate agents, with a fixed local objective, controlled feedback, an exploration budget, and no public asset being minted. The result is a routing policy, not public reputation.

Exercise p. 472

5.27

Capping the marginal reputation return can reduce direct improvement races, but agents may redirect spending into unmeasured signaling, judge capture, identity proliferation, or lobbying for task mix. Efficiency requires the full game: improvement cost, social value of quality, transferability, and alternative signals. Candidate controls are concave credit, buyer-local vectors, nontransferable evidence, and taxes/bonds on negative externalities. No cap is generically efficient without those payoff assumptions.

Exercise p. 472

5.28

Accuracy: competent, a protected test/spec oracle; incompetent, a self-reporting worker. Aesthetics: competent, a conflict-free expert reviewer for that domain; incompetent, a token-cost meter or a same-family unblinded model. Efficiency: competent, a protected resource meter normalized for the task; incompetent, a style reviewer guessing from prose.

Exercise p. 472

5.29

The requesting principal/daemon defines the judging work and eligible pool; the selected judge posts its own bond; the grade lands as a signed witnessed event in the subject's outcome ledger with judge/provenance; a conflict-free re-audit or appeal that overturns it slashes the judge under the predeclared rule and tombstones/revises the grade.

5.30

Exercise p. 472

Record skill ID/version/content hash, agent/build/principal, task and stratum descriptors, randomized assignment or selection propensity, exact context-graft receipt, model/environment/tool versions, outcome/evidence references, quality vector, costs, and evaluator identities. These fields verify exposure and outcome association. "This skill helped" is causal and cannot be verified from a graft-success pair alone; estimate it with randomized skill-on/off assignment on held-out comparable tasks (or label an observational matched estimate honestly), reporting effect size and uncertainty.

5.31

Exercise p. 472

Publish the vector, per-axis uncertainty, sample sizes, cohort/task distribution, freshness, judge/evidence provenance, and known correlations. Let each buyer apply a local, preferably non-public weighting and hard minimums; do not mint a canonical platform scalar. Agents can still optimize market-demanded axes, so rotate held-out tests and audit cross-axis regressions. Plural preference does not eliminate Goodhart, but it prevents one universal target from becoming the asset itself.

5.32

Exercise p. 473

Settlement incentives are conditional on the signal distribution. "The daemon derives the grade" only identifies the messenger; if a strategic judge chooses the input, the distribution depends on omitted actions/payoffs and the equilibrium proof assumes its own conclusion.

5.33

Exercise p. 473

Judge J posts bond B; its grade and the bond lock are recorded; an independently selected re-auditor overturns the dishonest grade against preserved evidence; the system appends a correction/tombstone, recomputes any explicitly appealable settlement, slashes B to the harmed/commons bucket, and lowers J's judging reputation. It must not silently rewrite the original event.

5.34

Exercise p. 473

With one-shot corrupt gain at most G, re-audit probability ρ, conditional detection probability d, and slashable bond B, weak local deterrence requires $\rho dB \geq G$, giving $\rho^\star = G/(dB)$ when $dB > 0$ and $G \leq dB$. Strict deterrence requires a strict inequality; if $G > dB$, no audit probability at this bond suffices. At $G_0 = 400, d = 0.8, B =$

50, $\rho = 0.25$: $\rho d = 0.2$, so $\lambda = 1 - \rho d = 0.8$; with $C = 8$ the threshold $G_k > CB = 400$ is unmet at level 0, giving $\lceil \log 400 / \log 1.25 \rceil = 27$ levels; with $C = 1$ the threshold is only $G_k > 50$, so bribery bleeds linearly for 35 levels before 18 more geometric ones close it, 53 total.

Exercise p. 473

5.35

Admit arbiters by domain competence and conflict graph; assign cases randomly/blindly; require bonds and service-level commitments; preserve evidence; pay for timely completed work rather than agreement; sample re-audits and publish calibration/overturn rates; use plural panels on high stakes. Price queue capacity and cap workload so rubber-stamping is not the profitable response to overload. Honest convergence still needs a protected mechanical outcome, a delayed real-world outcome, or a final human/contractual authority — a market alone does not manufacture truth.

Exercise p. 473

5.36

The script prints "`flat: 50.00 (Theta(T))`", "`model A: 25.58 (Theta(log T); a*G/(d v)*ln(1+vT/B) = 25.50)`", and "`model B: 35.08 (O(1); closed form aG^2/(2dBrv) = 41.67; hits 0 at t*=333)`". Here the script rounds the real-valued crossing and labels an integral as a closed form. The discrete schedule first reaches zero at $t = 334$ and its lifetime sum is 41.79175, as derived in the example. By hand: the flat baseline is a constant audit rate $\rho^\star$ times a constant per-audit cost a summed over T periods, so $\rho^\star \cdot a \cdot T = 0.25 \times 1 \times 200 = 50$, exactly the printed 50.00 — the two amortization models are cheaper only because they let ρ_t fall below $\rho^\star$ as accumulated stake or surfaced history does part of the deterring.

Exercise p. 473

5.37

An append-only tombstone preserves the original claim, the later correction, their authors, timing, and the reliance window. Delete destroys evidence, permits history rewriting, and makes replicas unable to distinguish retraction from omission.

Exercise p. 473

5.38

Append a signed tombstone containing the target outcome ID/hash, reason/evidence, authority, revision, and effective time.

Propagate it and recompute reputation as a projection that excludes/nullifies that outcome. It was spendable from original publication until each relying harbor received and enforced the tombstone or failed closed; under an unbounded partition that window is unbounded.

5.39

Exercise p. 474

Use a signed revision DAG, per-harbor import receipts/watermarks, periodic accumulator/Merkle roots for active outcomes and tombstones, and a freshness rule that refuses high-risk use from stale replicas. Under connected eventual delivery, convergence is shown when every member acknowledges a root containing the tombstone; sampling can audit inclusion. Bounded local memory may retain compact accumulators and content-addressed archive pointers, not erase correction commitments. No finite convergence/spend bound exists during an unbounded partition.

Chapter 6: The Harbor Economy

6.1

Exercise p. 539

A principal that can clone a fresh identity on unchanged terms can abandon an identity-specific penalty. A prepaid skill sale can nevertheless exchange a specified artifact for a specified price without relying on an agent's persistent reputation. The identity requirement applies to transferable reputation, continuing liabilities, and quotas, not to every possible trade.

6.2

Exercise p. 539

The counterparty is the operator/principal, not the individual worker process: the operator bears fleet outcomes through aggregate bond and liability, and its own worker agents underneath that liability are freely replaceable. Side 1 therefore needs durable identity only at the principal level; sides 2 and 3 need it at the level of the specific asset or artifact being sold, because there the continuing thing itself — not a firm standing behind it — is the product.

6.3

Exercise p. 539

Not every side-2/side-3 design without a non-forgeable agent identity collapses to Sybil. Skill licensing can run on a content hash, a license contract, and a durable licensor principal without agent

personhood at all; a rented stateless model artifact is content-addressable and warrantable the same way. What genuinely cannot survive cloning is identity-weighted reputation, quotas, or unsecured credit extended over freely mintable pseudonyms. Any design that claims to avoid non-forgeable identity and still support those either is a Sybil attack in progress or has quietly reintroduced some other scarce binding — principal identity, artifact hash plus owner liability, a hardware root, a stake, or a sponsor.

Exercise p. 539

6.4

A side is a distinct, privately informed participant class that must be separately induced to participate, and whose share of the price allocation moves cross-side volume. A plain ride-share platform has exactly two sides — riders and drivers — because the platform itself is the mechanism, not a third participant class; advertisers, fleet owners, or insurers become genuine additional sides only once they are recruited separately and carry their own network externality.

Exercise p. 539

6.5

A new version means a new content hash, and v1's settled outcomes say nothing evidentiary about v2's behavior. Left unspecified, v2 either inherits credit it has not earned or cold-starts as if it had no history at all — both wrong. §6.12 traces the fix: version, parent hash, build and dependency hashes, and evaluation history as an explicit lineage object; v2 starts with only a disclosed, discounted prior tied to that lineage, and updates on its own settled outcomes from there.

Exercise p. 539

6.6

Disclose the three axes — accuracy, aesthetics, efficiency — each with its own uncertainty and provenance, and let each buyer apply its own weights and hard constraints privately; compute a scenario-specific score only at decision time, never publish one fixed public scalar. A single published weighting induces one total order and therefore one optimization target: it hides every Pareto tradeoff among the axes and recreates exactly the scalar bandit-arm reward the disaggregated design was meant to escape.

Exercise p. 540

6.7

Redo the dramatization's arithmetic at bond 250 cr and 70% completion, using the chapter's own two-bucket rule. Bounty bucket: Alice earns $0.7 \times 400 = 280$ cr, the remaining 120 cr returns to

Bob. Bond bucket: Alice recovers $0.7 \times 250 = 175$ cr of her own posted collateral, and the remaining 75 cr is slashed to the commons. Net deltas: ΔBob $= -280$, ΔAlice $= +280 - 75 = +205$, Δcommons $= +75$, summing to zero; both escrow buckets, 400 and 250, clear to zero exactly as in the worked 40% case.

6.8 *Exercise p. 540*

A protected oracle (a CI-issued test id, a merged SHA) binds closure to a state the worker cannot author. A free-text "Result:" note lets the same party who did the work also self-report that it succeeded — inviting a fabricated test run, a claimed merge that never happened, or simply the most favorable reading of ambiguous acceptance criteria after the fact.

6.9 *Exercise p. 540*

Pause at a mediated safe point and propose amendment $v{+}1$: it must reference plan v by hash, state the changed criteria, budget, and bounty/bond deltas, and say how already-produced evidence is treated. Both principals must countersign $v{+}1$ before it takes effect; only then does the daemon atomically top up or refund each party's own escrow bucket and activate the new plan, retaining v and the causal link to it. A timeout on the proposal either continues settlement under the original v or cancels under v's own terms — never does one party get to unilaterally rewrite acceptance criteria after seeing the work.

6.10 *Exercise p. 540*

The renter operates the jail and can sandbox, throttle, or revoke the leased agent. The owner carries its reputation and any agreed warranties; the renter still pays the lease fee and bears task risk. Grossman–Hart does not identify these as two universal roles or prove this allocation optimal. That requires specifying investments, bargaining, outside options, and feasible alternatives.

6.11 *Exercise p. 540*

Adding a lease line item of amount ℓ or a metered-license line item of amount m leaves conservation intact for any $\ell, m \geq 0$, because each is still just a balanced transfer: it debits a pre-funded escrow bucket and credits an explicit recipient or refund bucket within the same atomic transaction. The wallet–escrow–commons identity never references the size of any one line item, only that debits

match credits. What amount size *can* break is solvency, participation, or collateral adequacy — conservation holding is a necessary bookkeeping property, not evidence the contract is viable.

Exercise p. 540

6.12

Make each portfolio-tree leaf a tuple (skill_id, version, content_hash, parent_hash, build/dependency hashes, license, evaluation root). A v2 proof of membership includes its parent's leaf plus v2's own evaluation root. Initialize v2's reputation prior with an effective sample size $\gamma\, n_{v1}$, where $\gamma \in [0, 1]$ is a predeclared compatibility/evaluation discount fixed *before* seeing v2's outcomes — never simply copy v1's settled outcomes onto v2. Lineage proves ancestry; it does not, by itself, prove behavioral similarity, which is exactly what $\gamma < 1$ is there to discount. Figure 6.S1 draws the leaf and the discounted edge.

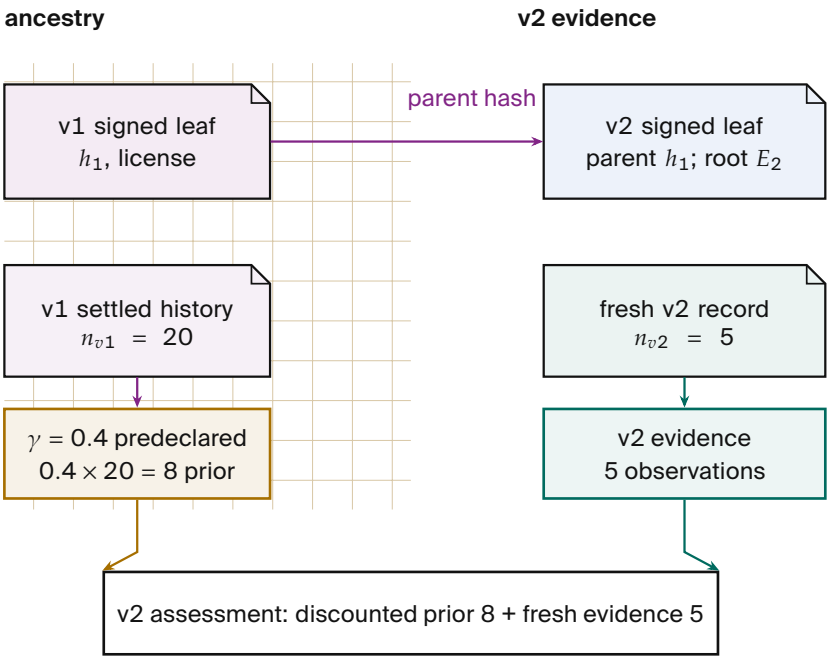


Figure 6.S1. A versioned skill document binds v2 to its v1 parent while retaining a separate v2 evaluation root. The displayed counts illustrate a predeclared discounted prior, not evidence that v2 behaves like v1. [internal; illustrative]

Exercise p. 541

6.13

The witness begins with the fact `agent_deployed`. Its Horn rule is

$$\texttt{agent_deployed} \longrightarrow \texttt{prod_access}.$$

Once `prod_access` holds, the obligation to write production applies on $[0, 10]$. The same write is forbidden on $[5, 15]$. Their overlap is $[5, 10]$.

Running the real checker gives `conflict=True, O/F pairs=[(0, 1)]` on [5, 10]; the same script's Horn-propagation-ablated mutant gives `conflict=False` on this exact witness, and misses 100 of the 885 conflicting instances in the script's 3000-policy sweep [verified, `b4_deontic_fragment.py`].

The added disjunction $O(\ell_1 \vee \ell_2)$ uses the two rules specified in the exercise. Selecting ℓ_1 alone or ℓ_2 alone leaves the checker's report unchanged (`conflict=True, of=[(0, 1)]` in both cases) — the pre-existing `write_prod` clash does not depend on the new obligation at all, verified by running the checker on both selections.

Selecting *both* ℓ_1 and ℓ_2 adds a second, independent clash, `of=[(0, 1), (2, 3)]`: the two new rules now fire on the identical action, scope, and interval, one obliging and one forbidding it, which is exactly why Theorem 6.3.3's reduction from 3-SAT insists a conflict-free selection can never pick both polarities of one variable — doing so is a clash *by construction*, which is precisely the mechanism that turns a satisfying assignment into a conflict-free discharge and back.

None of this required exponential search: with one disjunctive obligation over two literals there are only three nonempty selections to try by eye. The NP-complete frontier (Theorem 6.3.3) is about what happens as the number of disjunctive obligations grows, as the 3-SAT reduction's one-obligation-per-clause construction does, not about any single disjunct bolted onto a two-rule witness.

6.14 *Exercise p. 541*

The principal creates many aliases and supplies favorable votes, increasing its aggregate weight without acquiring independent experience. Resetting an alias can also discard its bad history if the admission rule carries no old liability forward. The attack defeats this identity-weighting rule. It does not erase a properly authenticated transaction record or prove every reputation mechanism useless: the failure concerns which records receive weight and how aliases are linked to accountable principals.

6.15 *Exercise p. 541*

The market's own definition calls the operators *mutually-distrusting* and says they trade across "a boundary neither controls." Both phrases directly contradict treating a hostile operator as out of

scope for local non-forgeable identity's threat model: if the boundary is one neither party controls, the counterparty's daemon is exactly the adversary that a single-operator identity primitive was never built to withstand.

Exercise p. 541

6.16

A workable certificate binds (principal_id, agent_key, issuer, epoch, policy, expiry) and requires the agent to prove possession of its key. Bonds, quotas, newcomer state, and sanctions all key on `principal_id`; agent certificates are children of it. Issuance and revocation publish to a transparency log, and the issuer itself is cross-signed or attested. None of this makes a real-world principal provable by cryptography alone, and none of it stops a hostile operator from minting fresh principals under its own root — the scarce admission mechanism (an organizational or KYC credential, hardware-backed membership, a bonded sponsor, or an equivalent explicitly priced gate) has to sit outside the cryptography, at the point principals are first admitted.

Exercise p. 541

6.17

Under independent private values with positive densities on intervals whose interiors overlap and quasi-linear, risk-neutral utility, bilateral trade cannot be simultaneously Bayesian incentive compatible, interim individually rational, ex-post efficient, and balanced in expectation without outside subsidy. Check the bilateral setting, private information, independence, prior, support interiors, utility model, and each claimed property before applying the theorem. Merely touching supports do not suffice: seller values in $[0, 1]$ and buyer values in $[1, 2]$ permit efficient, individually rational trade at fixed price 1. A shared grading oracle may induce dependence, but does not by itself establish it. A model using public oracle information must specify the resulting conditional value distributions and verify independence and the remaining hypotheses there. Oracle integrity remains a separate obligation (*From Spawn to Person*).

Exercise p. 542

6.18

The ledger holds 10 A-coins and 20 B-coins throughout. At 1A = \$1, 1B = \$2, the marked value is $10(1) + 20(2) = \$50$. Suppose B's price rises to \$3 with no transfer at all: native balances are unchanged at 10 and 20, but the marked value is now $10(1) +$

$20(3) = \$70$. Nothing was created — native-unit conservation (Theorem 6.5.1) still holds exactly — but the scalar valuation moved because v_t moved, which is Property 6.5.1's whole point.

6.19

Exercise p. 542

Assume every quote used is at most L seconds stale and that log-price moves at a bounded deterministic speed, $|\log(v_t/v_s)| \le \sigma|t-s|$. For notional X exposed in the second unit, the worst-case stale-quote error is then bounded by $\varphi = X(e^{\sigma L} - 1)$, plus fees and rounding. A stochastic model such as GBM or a historical-volatility estimate instead yields a confidence interval or VaR figure, and must state its tail probability explicitly — without some deterministic bound on how fast the price can move, no finite worst-case φ exists at all, only a probabilistic one.

6.20

Exercise p. 542

Messages (1)–(2) only authenticate the request and A's offer; they commit A, not B. Message (3) is where B actually counter-signs: B verifies A via the witness log, applies its own attenuation, mints the derivative C'_B under its own key, and commits its replay state. That is what buys the ceremony its liveness property — C'_B verifies under B's key alone, so every later presentation is offline. With only two messages B would never have consented to anything, and would hold no object its own effect boundary could verify without asking A.

6.21

Exercise p. 542

If Alice rotates her signing key between message (3) and B's acceptance, B must validate the offer against an append-only key-history certificate and check the offer's issuance epoch against it, rejecting a stale epoch outright. Once C'_B is actually minted, though, its signature verifies under B's own key, so Alice's rotation alone does not retroactively erase it — the derivative needs an explicit parent/key-epoch dependency, a bounded TTL, and a revocation rule so that a later rotation or revocation can still invalidate C'_B once that fresher state reaches B's witness log. The manuscript does not yet specify this dependency in full (§6.12).

6.22

Exercise p. 542

Bind every transfer to a signed tuple: a random transfer id/nonce, the source card's JTI, the target audience B, the source and target epochs, an issue time and expiry, and the plan/card hash. B

atomically records $(A,$ transfer_id$)$ as consumed *before* minting the derivative. A re-presented message (3) then either returns the same previously minted C'_B idempotently, or is rejected outright — it must never mint a second child silently. The freshness invariant this buys: every accepted derivative traces to exactly one recorded, unexpired source transfer under the currently permitted issuer epoch, and one-shot transfers cause at most one derivative issuance, full stop.

Exercise p. 542

6.23

The expected $\Theta(\Delta \log m)$ bound needs a connected (often complete) overlay, independently uniform random peer selection each round, reliable bounded-duration rounds, and a push/pull epidemic rule with enough independence between rounds to drive the usual doubling argument. It is an expectation under that model, never a deadline: an unbounded partition, or indefinite message loss, is observationally indistinguishable from ordinary delay from inside the model, which is exactly why worst-case dissemination time under partition is infinite rather than merely large.

Exercise p. 542

6.24

Alice revokes at her home harbor at $t = 0$; Harbor B, which rented the affected agent, has not yet heard, so for $[0, \Delta]$ its filter is stale and the card is still spendable there. At $t = \Delta$ gossip reaches B and it stops accepting the card; Harbor C, two hops out, remains exposed until $t = 2\Delta$. The window Conjecture 6.7.1 must price is per-harbor, running from t_0 to that harbor's own *enforcement* — not merely to its first gossip receipt, since receiving a filter update and actually enforcing it are two different events with their own local latency.

Exercise p. 542

6.25

A witness that equivocates sends root X to B and root Y to C for the same epoch; nobody can detect this until evidence from both views reaches one honest process, and until then each view is individually consistent with ordinary delay. On the three-node complete mesh with exchanges every Δ, the earliest possible detection is the next B–C (or auditor) exchange: at least one communication interval, and close to 2Δ if the two roots arrive just after a scheduled exchange under discrete rounds. On a path graph the bound scales with graph distance times Δ; under an unbounded partition there is no finite bound at all. Signatures make the *comparison* $O(1)$

once both roots are co-located — they do nothing to speed up the information reaching that one place.

6.26 *Exercise p. 543*

Every imperfect-public-monitoring folk-theorem result needs a fixed conditional distribution of the public signal given the players' actions and types. A grader is itself a strategic player whose reporting policy, private information, susceptibility to bribes, and possible deviations all feed into that signal. Omit the grader from the game and the distribution the theorem needs is neither fixed nor exogenous — it is endogenous to a player the model never included, so the equilibrium claim built on top of it is unsupported until the grader is put back in.

6.27 *Exercise p. 543*

A protected unit-test receipt is Fix A, machine-checkable, subject to the test suite's own integrity. A protected merged SHA is likewise Fix A. "The refactor is tasteful" is Fix B: no third party can mechanically check taste, so it needs a plural or declared human/expert judge whose ruling carries contractual finality, backstopped by that judge's own bond.

6.28 *Exercise p. 543*

Use a well-founded appeal procedure: assign the original grade rank K, allow an appeal only to rank $K-1$, and stop at rank 0 — either a protected oracle for an objective predicate, or a named final authority or quorum for a matter of preference. Because ranks are natural numbers, no infinite ascending chain of "judges judging judges" is possible. For aesthetic judgments the rank-0 base is contractual finality, not truth. Bonding and re-auditing supply conditional incentives only under a specified monitoring and payoff model; they do not turn taste into a machine-checkable fact. Figure 6.S2 draws the descent.

6.29 *Exercise p. 543*

Phase 1 subsidizes *supply* — the operators and agent owners willing to produce real settled work — because inventory plus witnessed outcomes is exactly the reputation data that later attracts demand and skill licensors; above-market, platform-posted tasks are the direct form that subsidy takes. This is the "scarcest cross-side externality" rule: supply is scarcer than demand at cold start because nothing exists yet for demand to evaluate, so supply is the side

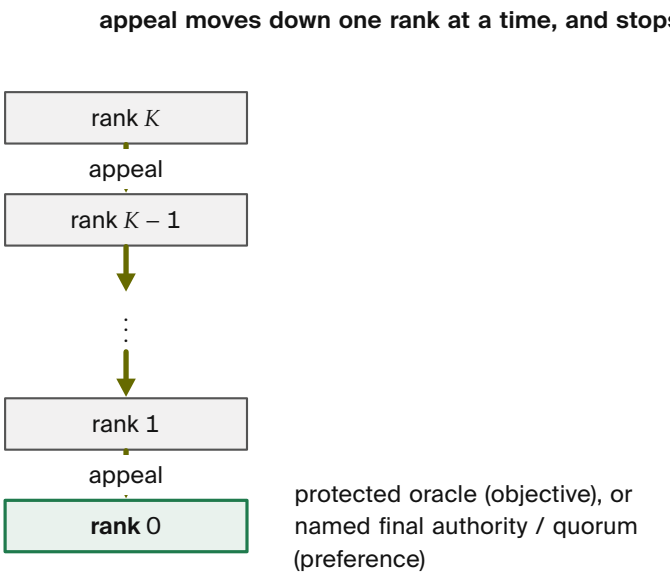


Figure 6.S2. Each appeal lowers a natural-number rank and stops at rank zero. That endpoint may be a protected oracle or a named final authority; termination does not turn preferences into objective truth. [internal]

worth paying to show up, and the flip to charging demand happens once a liquidity threshold is met, not on a calendar date.

Exercise p. 543

6.30

Under the grim-trigger timing *The Bonded Commons*draws, collusion is self-sustaining exactly when the discounted colluding stream beats the one-shot defection payoff: $(\pi_C - p_d L)/(1 - \delta(1 - p_d)) \geq \pi_D$, equivalently $\delta \geq [\pi_D - \pi_C + p_d L] / [\pi_D(1 - p_d)]$. Raising the detection probability p_d, the penalty L, or audit quality, or lowering the cartel margin $\pi_C - \pi_D$, all push the right-hand side up — once it clears members' actual δ, no discount factor sustains the cartel and it breaks.

Exercise p. 543

6.31

Price of anarchy is the wrong tool here because it presupposes a fixed game, and letting reputation be resold changes the game itself; the right question is whether an informative separating equilibrium survives at all once resale is possible. If a high type can simply sell its reputation signal to a low type, the signal's cost stops separating types and arbitrage pushes the market toward pooling. Separation survives only if the evidence stays nontransferably bound to

the continuing principal or artifact, a transfer resets or visibly discounts the signal, or the seller keeps warranty/bond liability so sale remains strictly costlier for a low-quality asset than a high-quality one — absent one of those three, resale does not merely worsen a ratio, it destroys the signal outright.

6.32 *Exercise p. 543*

A candidate site takes administrative domains (harbors) as objects and their signed overlap views as the data over each; morphisms are restrictions to shared accounts, events, or epochs; covers are families of domains whose union spans the bounded federation under study. A section assigns an event or balance view to a domain, and restriction projects that view onto an overlap. Set-valued gluing needs no coefficient object at all; a cohomological treatment needs one explicitly chosen, such as account-balance deltas valued in the abelian group $\mathbb{Z}^{\text{Accounts}}$, with cocycles defined on that group rather than merely asserted.

6.33 *Exercise p. 544*

Two different signed roots for the same $(\text{harbor}, \text{epoch})$ are direct evidence that one key signed two inconsistent commitments — that much is just hash inequality. A cohomology class is a much heavier object: it needs a defined cover, a coefficient sheaf, the overlap differences assembled into an actual cocycle, and a quotient by coboundaries. Observing that two hashes disagree supplies none of that algebra; it is evidence of equivocation, not evidence of a nonzero H^1.

6.34 *Exercise p. 544*

Fix the three-harbor cover A–B–C. Let each local section be a finite map from globally unique event IDs to signed events, with restriction as projection onto the shared IDs of an overlap. If every pairwise restriction agrees, the sections glue to a unique global map on the union — an ordinary, checkable fact about finite maps agreeing on their overlaps, nothing more. What this proves: deterministic gluing of already-delivered, identically-keyed events at that one fixed finite topology. What it does not prove: that any event is true, that the event set is complete, anything about delivery time, what happens to a fork with conflicting IDs, or settlement finality — the gluing lemma is a statement about consistent bookkeeping, not about the world the bookkeeping describes.

Exercise p. 544

6.35

The miss probability is 0.4^k. Five reviewers give $0.01024 > 0.01$; six give $0.004096 < 0.01$, so six suffice and cost twelve credits to recruit. Bounties depend on confirmed reports and the duplicate-payment rule, so that fixed recruitment total does not determine payout. With perfectly shared success or failure, the panel misses with probability 0.4 regardless of its size; multiplying marginal miss probabilities is then invalid.

Chapter 7: The Bonded Commons

Exercise p. 630

7.1

The rejected card violates *temporal* attenuation: Theorem 7.2.1 requires $C(\tau_k) \subseteq C(\tau_{k-1})$ along every dimension of the capability, and TTL is one such dimension—a child's expiry must not exceed its parent's. Here the child requests 20 minutes against a parent that expires at 15; even though the capability set $\{$`fs:write:auth.ts`$\}$ is a subset of α's, the delegation as a whole is not a narrowing.

An acceptable card is $\{$`fs:write:auth.ts`$\}$, TTL $\leq$ 15 minutes—dropping `db:read` (a valid narrowing of the capability set) and matching or shortening the TTL. A strictly narrower example is $\{$`fs:read:auth.ts`$\}$, TTL = 10 minutes, if the deployment's capability lattice places `read` below `write` on the same resource.

Exercise p. 630

7.2

The value is zero because revocation overrides the decayed value of two. The original spray and revocation remain as records. Inclusion establishes what was committed under the authenticated root, not that the hint was accurate or that every reader received its revocation.

Exercise p. 631

7.3

An advisory claim is a durable declaration that an agent intends to use a resource; it informs coordination (Definition 7.4.1's conflict graph) but does not make conflicting effects impossible. Every trace an enforced-lock regime allows, an advisory regime also allows—the lock regime is a special case that additionally forbids the conflicting trace—so advisory claims are strictly more permissive, never less.

That extra permissiveness is the feature Theorem 7.4.1 names: The locking instance assumes both agents prefer completion of both tasks and that the precautionary claim is not released in time; under those assumptions its veto blocks a Pareto-superior outcome. Sen motivates the comparison, but his theorem is not a theorem about every lock manager. An advisory regime never grants that veto, so it never forecloses the outcome a lock would have blocked—at the cost of also admitting traces where two agents genuinely do collide. §7.4.3's appeals and preemption path is exactly the mechanism that prices the cost of that added permissiveness rather than eliminating it.

7.4 *Exercise p. 631*

First, reputation compensates nobody for the harm already done. A one-shot defector takes the gain from the first (and only) defection and leaves before any future exclusion can matter to it—exactly the "one-shot defectors" adversary class named above. Second, cheap identities let the actor whitewash the loss: register a fresh agent ID, and the reputation ledger has nothing to slash. A pre-funded bond avoids both failure modes because it creates recoverable value *before* authority is granted and keys the exposure to the durable principal (§7.3.4) rather than to the disposable agent identity—it prices and contains harm at the moment of the first defection, not on the promise of a future one. What a bond does *not* do is prove the work will be correct; it only ensures that if the work is wrong, someone has already paid for the possibility.

7.5 *Exercise p. 631*

Fix A's choice and compare its two rows against each of B's columns. Against B:T, A gets 3 from T and 4 from F, so F is better ($4 > 3$). Against B:F, A gets 0 from T and 1 from F, so F is again better ($1 > 0$). F therefore strictly dominates T for A regardless of B's move; the game is symmetric, so the same holds for B. Checking the three remaining cells: at $(T, T) = (3, 3)$ either player can deviate to F and gain ($4 > 3$); at $(T, F) = (0, 4)$, A can deviate to F and gain ($1 > 0$); at $(F, T) = (4, 0)$, B can deviate to F and gain ($1 > 0$). Only $(F, F) = (1, 1)$ survives—neither player gains by unilaterally switching back to T ($0 < 1$)—so it is the unique Nash equilibrium, exactly as the table's caption states.

Exercise p. 631

7.6

At $\delta = 0.34$: $\delta^2 = 0.1156$, $1 + \delta + \delta^2 = 1.4556$, $2\delta = 0.68$, and $0.68 \times 1.4556 = 0.989808$, so $f(0.34) = 0.989808 - 1 = -0.010192$ [verified]—negative. At $\delta = 0.35$: $\delta^2 = 0.1225$, $1 + \delta + \delta^2 = 1.4725$, $2\delta = 0.7$, and $0.7 \times 1.4725 = 1.03075$, so $f(0.35) = 1.03075 - 1 = 0.03075$ [verified]—positive. f is continuous and strictly increasing on $(0, 1)$ (every term's coefficient is positive), so a sign change from $\delta = 0.34$ to $\delta = 0.35$ proves a root lies strictly between them—the Intermediate Value Theorem, not a numerical coincidence.

This matches two independent committed artifacts. `proofs/economics/delta-threshold.z3`'s second query (`delta-threshold.expected.txt`) asserts the root lies in $[0.34, 0.35]$ and returns `sat`, agreeing with the sign change computed here. Independently, `proofs/economics/sweep-delta.sh`'s committed sample sweep (documented in `proofs/economics/README.md`) reports the model-checked IC invariant `VIOLATED` at $\delta = 0.34$ and `HOLDS` at $\delta = 0.35$—the same sign change, checked by a state machine instead of a closed-form inequality.

Exercise p. 631

7.7

The script asks, in order: (1) does a real root of $2\delta^3 + 2\delta^2 + 2\delta - 1 = 0$ exist in $[0, 1]$; (2) does that root lie in the tighter interval $[0.34, 0.35]$; (3) does a *second*, distinct root exist in $[0, 1]$ (a uniqueness check, expected to fail).

Pinned verdict sequence (`proofs/economics/delta-threshold.expected.txt`):

```
sat
(
  (define-fun delta () Real
    (root-obj (+ (* 2 (^ x 3)) (* 2 (^ x 2)) (* 2 x) (- 1)) 1))
)
sat
(
  (define-fun delta () Real
    (root-obj (+ (* 2 (^ x 3)) (* 2 (^ x 2)) (* 2 x) (- 1)) 1))
)
unsat
```

Reading the three lines against the three questions: `sat` (a root exists, printed in Z3's algebraic `root-obj` form rather than a decimal); `sat` again with the identical model (that same root also satisfies the tighter bound, so it lies in $[0.34, 0.35]$); `unsat` (no second, distinct δ_2 in $[0, 1]$ satisfies the cubic, so the root is unique). All three together are exactly Theorem 7.6.2's threshold, existence-and-uniqueness, mechanized.

7.8 *Exercise p. 631*

At $\delta = 0.30$: $\delta^2 = 0.09$, $1 + \delta + \delta^2 = 1.39$, $2\delta = 0.6$, and $0.6 \times 1.39 = 0.834$ [verified]. The inequality $1 < 0.834$ is false, so the deviation is profitable and the invariant `NoUnilateralDeviationPositive` fails—matching the table's stated verdict and the committed sweep row `0.30 VIOLATED` (`proofs/economics/README.md`). At $\delta = 0.35$ (`proofs/economics/claim_signaling.cfg`'s own default, `DeltaNum=35`, `DeltaDen=100`), $2\delta(1 + \delta + \delta^2) = 1.03075$ from the worked example above, so $1 < 1.03075$ holds and the invariant holds—matching `claim_signaling.cfg` being the one committed configuration and the sweep row `0.35 HOLDS`.

The trace: at $\delta = 0.30$, TLC's search reaches a state where an agent deviated to F once in round one, was punished for three rounds at the mutual-claim payoff, and reached `round = Horizon` with `deviated` true; because the deviator's discounted score from that trajectory (`actualScore`) exceeds the score it would have earned by cooperating throughout (`followScore`), the state violates `NoUnilateralDeviationPositive` and TLC reports it as a counterexample. No such state exists at $\delta = 0.35$: the same one-shot-deviation trajectory now scores strictly worse than cooperating, so every reachable state at `round=Horizon` satisfies `actualScore` $\leq$ `followScore` and the search completes with "No error has been found." The committed trace is `proofs/economics/claim_signaling_delta30.run.log`, produced by the negative-control configuration `claim_signaling_delta30.cfg`; the proofs workflow asserts on every run that it still violates.

7.9 *Exercise p. 632*

The committed sample output (`proofs/economics/README.md`) reports the invariant `VIOLATED` for every δ from 0.30 through 0.34 and `HOLDS` from 0.35 through 0.40, so the crossover—the smallest δ for which the invariant holds—is pinned at

```
1 crossover (smallest delta where invariant HOLDS) = 0.35
```

```
2 closed-form root (delta-threshold.z3)            = 0.3425
3 PASS: crossover matches closed-form within integer-grid rounding.
```

The script's own pass criterion (its exit code) is that the crossover lands in $\{0.34, 0.35\}$—the two integer-grid points bracketing the closed-form root $\delta^\star \approx 0.3425$ from Exercise 7.6. A crossover anywhere else would mean the model-checked state machine and the closed-form cubic disagree about where cooperation becomes sustainable, which is exactly the regression this script exists to catch on every PR touching `proofs/economics/` (§7.A).

Exercise p. 632

7.10

The simulation credits premium q on every win and slashes $\min(B_{\text{dep}}, B_T)$ on default. For a fixed loss probability p, expected profit is $q - p\min(B_{\text{dep}}, B_T)$. A deposit increase cannot deter this attack when $q > pB_T$; when $q \le pB_T$, a deposit of at least q/p can. Full forfeiture changes the payoff to $q - pB_{\text{dep}}$ and removes that particular cap. Neither formula models arbitrary identity reuse, unavailable collateral, or collusion. A proposed admission fee, probation rule, or aggregate exposure reservation must be included in a specified payoff and solvency model before claiming that it closes those attacks.

Exercise p. 632

7.11

With $x = q_{\text{floor}} - \mu$, the chapter's own closed form gives $\pi_C = x/3$, $\pi_D \to x$ as $\varepsilon/x \to 0$, and $L = 5x$, so

$$p_d^\star = \frac{\pi_C - (1-\delta)\pi_D}{L + \delta\pi_D} = \frac{x/3 - (1-\delta)x}{5x + \delta x} = \frac{1/3 - (1-\delta)}{5 + \delta}$$

after cancelling the shared factor x. At $\delta = 0.99$: $1/3 - (1 - 0.99) = 1/3 - 0.01 = 97/300$, and $5 + 0.99 = 5.99 = 599/100$, so $p_d^\star = (97/300)/(599/100) = 9700/179700 = 97/1797 \approx 0.05398$ [verified]—about 5.4% per round, close to but distinct from the chapter's own worked point at $\delta = 0.95$ ($p_d^\star \approx 0.0478$): a more patient cartel (δ closer to 1) needs a slightly *higher* detection probability to stay fragile, because the discounted value of continuing the cartel is larger.

Cheap fresh identities shorten the effective horizon a punished member faces and let it shed the punishment entirely by re-registering—exactly the Sybil failure mode Exercise 7.4 names for reputation alone. A *principal-bound* bond deters re-entry only when the expected forfeiture plus the onboarding and

reputation-maturation cost of a fresh identity exceeds the gain from re-entering the cartel; a bond that is reusable across identities, or sized below that threshold, merely *prices* re-entry rather than deterring it. The bond mechanism by itself never *prevents* re-entry—nothing in this chapter's architecture stops a new registration—so the honest answer is: prices always, deters conditionally, prevents never.

7.12

Exercise p. 632

This is a design obligation, not a closed theorem: no mechanism in this chapter proves a Pareto ordering for a posted-price catalog, so what follows is a defensible proposal, not a fictional derivation. A posted-price catalog quotes premiums by manifest risk class, principal history, exposure duration, and tail-capital band, recalibrated periodically against a hard reserve constraint—no per-transaction bidding, no auction latency. It conditionally improves on the static escrow of §7.6.5.2 exactly when the posted quote sits below the principal's idle-capital cost of self-insuring via a static bond, while the insurer remains individually rational and its reserves still cover the same tail risk the competitive market would have priced. Relative to §7.6.5.8's bid-based mechanism, a posted catalog trades away instance-specific price discovery (the model's conditional pricing of expected capped payouts, capital costs, and expenses) for lower manipulation surface and no per-transaction latency—the same trade the A6 cartel analysis makes salient in the other direction, since a catalog has no bid pattern for a colluding cartel to distort in the first place.

The proposal depends most on *calibrated loss/tail dependence*: the catalog's bands must actually track the loss distribution within each class, and its capital reserve must be enforceably adequate against the tail, or the "conditional" improvement collapses into either under-pricing risk (insurer insolvency) or over-pricing it (pricing capable agents out, the same failure the threat-class bands of §7.6.5.4 already warn against). The CC/NC/RP/NS assumptions of §7.6.5.8 would need explicit class-conditional definitions before any formal Pareto claim could be stated for this alternative—this exercise does not supply that derivation, and neither should its answer key pretend to.

Exercise p. 633

7.13

The committed log (`proofs/bonded/conservation/Conservation.run.log`) reports:

```
Model checking completed. No error has been found.
26818 states generated, 1716 distinct states found, 0 states left on queue.
The depth of the complete state graph search is 9.
```

These are two different numbers, and the difference is the exercise. TLC *generates* a state every time it evaluates a transition, including transitions that land back on a state it has already seen by a different path—26,818 counts every such evaluation. The *reachable* state count is the number of **distinct** states TLC actually found, 1,716: every other generated state was a re-visit, not a new point in the state space. (Table 7.10 now carries both figures for this row; both are pinned from the same committed log, and only 1,716 answers "how many reachable states are there.")

The invariant named in `Conservation.cfg`'s `INVARIANTS` block that gives the module its name is `Conservation`, defined as `TotalFree + TotalEscrow + burned = minted`—the free-form statement of Theorem 7.9.1 at the granularity of individual mint/stake/slash/refund/payout operations rather than the three-bucket accounting state. The same run also checks `NoNegative` (no balance ever goes negative) and `Safety` (their conjunction); the log's "No error has been found" covers all three across every one of the 1,716 reachable states, to search depth 9.

Exercise p. 633

7.14

No run is needed to predict this: it follows from the definitions already in the spec. `Conservation` asserts *TotalFree* + *TotalEscrow* + *burned* = *minted*. Immediately after `Init` (all four quantities zero, satisfying the invariant trivially), the very first enabled `Mint`(a, *amt*) step increases *TotalFree* by *amt* while leaving *minted* at 0 under the stated weakening, so the left side becomes *amt* $>$ 0 while the right side stays 0. `Conservation` fails at depth 1—the shallowest possible counterexample, one step past the initial state, with no need to explore further.

`NoNegative`, by contrast, does *not* catch this: every quantity involved (`free`, `escrow`, `burned`, `minted`) stays non-negative under the weakened `Mint`—the bug is a bookkeeping omission, not a sign violation, and `NoNegative`'s four non-negativity clauses are silent about

the relationship *between* the buckets. `Safety`, defined as `Conservation ∧ NoNegative`, fails only because its `Conservation` conjunct does; this is exactly why the module checks `Conservation` as its own named invariant rather than folding it silently into `Safety` alone; a reader who only watched `NoNegative` would see nothing wrong.

7.15 *Exercise p. 633*

Model a fresh token t, an issuance event Issued(u, t) tied to the user's mailbox, storage of an unguessable token (or a protected hash of it) with an expiry and an unused flag, delivery over a private mailbox channel in the uncompromised case, with a separate compromise process that reveals the channel to the attacker, redemption over an attacker-controlled network under *The Anchor Protocol*'s Dolev-Yao assumptions, an atomic consume step, and recovery-key or session rotation on successful redemption.

The secrecy property is that the token (and any recovery secret it unlocks) stays unknown to the attacker absent mailbox compromise. The authentication property is injective: inj-event(Consumed(u, t)) $\Rightarrow$ inj-event(Issued(u, t)), exactly the form §7.11 already states for magic-link single-use—every consumption traces back to one, and only one, issuance, which is what rules out the race window between two concurrent redeemers. Add a concurrent-replay capability (two redemption attempts racing on the same token) and a mailbox-compromise capability (the attacker reads, but does not yet control, the delivery channel), neither of which the Phase 1 Harbor Card model needs: that model's attacker forges signatures or replays chains, but it has no notion of a *delivery channel* an attacker might passively observe, because Harbor Cards are minted and verified entirely within the daemon's own boundary. Time and expiry also need an explicit abstraction here—ProVerif has no wall-clock semantics on its own—whereas the Harbor Card model's TTL is already folded into the token's signed payload.

7.16 *Exercise p. 633*

This is an open mechanization obligation, not a solved one: Demers et al. is the epidemic-replication paper the chapter leans on, but no numbered theorem in it is stated in a form that yields this protocol's $\Theta(\log m)$ all-informed bound by direct substitution—the paper's push/pull rumor-spreading results assume a complete,

uniform-peer overlay with reliable rounds, which must be fixed explicitly for the proposed dissemination model. The mechanization obligation is therefore two steps, not one: first state the push/pull rumor-spreading theorem precisely, under exactly the complete-graph, independent-peer, reliable-round assumptions this chapter already uses (the claim-signaling game separately assumes perfect public monitoring); only then port that precise statement to a proof assistant. Citing the paper is not, by itself, a proof object.

On a general connected graph the $\Theta(\log m)$ bound does not transfer as stated: convergence time depends on the graph's conductance or mixing time, not on m alone, and the honest replacement bound has a term roughly inverse in conductance, plus a tail probability rather than a clean deterministic count. A disconnected graph is the degenerate case that breaks the claim outright: gossip never converges across components that share no edge, no matter how long the protocol runs, because there is no path for a rumor to cross. A connected-but-bottlenecked mesh sits between the two: dissemination across the bottleneck cut is throttled by the cut's capacity, so a mechanized version of this bound would need to separate *safety* (accepted revocation records have valid provenance; timely refusal of a remotely revoked card still requires freshness or fail-closed admission) from *liveness* (every revocation eventually reaches every device, which needs the connectivity and mixing assumptions this section has so far only asserted in prose).

Exercise p. 633

7.17

An attacker with no special privilege submits a large volume of authenticated-looking but distinct revocation IDs—each one legitimate on its own, since the filter cannot distinguish a real revocation from a manufactured one by content alone. Each insertion fills buckets and, past a certain load, drives kick-chain failures in the filter's relocation step. Past the load at which Fan et al.'s false-positive bound holds, the filter is pushed outside the regime the bound was proved for: insertions the filter can no longer place are either rejected or force evictions of *existing* entries, so a legitimate revocation already in the filter can be silently displaced.

The invariant that breaks is completeness of revocation—every previously revoked credential should still test positive—not the false-positive bound itself, which stays a probabilistic guarantee about *non*-members. The 0.95 ceiling defeats the attack described above only in the sense of preserving the regime the bound was

proved in: refusing further insertions past that load keeps every already-inserted revocation intact, at the cost of refusing new ones (which is a capacity failure, loud and observable, rather than a silent integrity failure). It does not defeat a determined pollution attempt on its own, because an attacker willing to keep submitting IDs simply keeps the filter permanently at its ceiling, denying legitimate revocations a slot. Closing that requires what the ceiling alone does not provide: authenticated revocation provenance (only a party who can prove standing may insert), per-principal insertion quotas, an exact overflow log for anything the filter had to refuse, and a fail-closed policy—if insertion fails, the caller must not proceed as if the revocation succeeded. A probabilistic filter must never be allowed to silently drop an authoritative revocation.

Chapter 8: The Federated Harbor

8.1

Exercise p. 706

The proof sketch's move — "each attenuation is a subset operation in the Anchor algebra; subset composition is a subset" — is exactly right when att_A and att_B both narrow authority *within one shared capability/resource universe*. It stops being a statement about the same resource space at the second attenuation: att_B does not narrow a set of operations on A's test database, because D_B has no test database to narrow into. It grants a *new* authority, over a resource D_A never named, that D_B alone controls. Calling C'_B a "subset of C_A" silently assumes a mapping between the two harbors' resource namespaces that nothing in the four-message ceremony (§8.2.1) actually constructs.

What minting C'_B really is, in this case, is **token exchange and new authorization at** B, not attenuation of C_A. A correct D_B must additionally establish: (a) *A-side input eligibility* — a signed semantic/resource mapping saying that C_A's authority over A's test database is the kind of thing that may be exchanged at all, recorded with its dependency on A's issuing epoch root $R_A^{(e)}$ so a later revocation of C_A is traceable to C'_B; and (b) *B-side local authorization* — D_B's own policy minting a genuinely new child grant over its staging database, under D_B's sovereignty (Design Invariant 8.0.2, item 2), rather than treating att_B as if it were narrowing A's grant.

One more gap rides along with this one and is worth naming here rather than assuming solved: Bob's harbor observing a failed acceptance check, or an inclusion proof against $R_B^{(e)}$, does not by itself establish that Alice's agent *caused* the failure (§8.4.3's damage-claim step). Settlement evidence must bind the exact work-unit, the base and result trees, the mediated effects, the acceptance policy in force, and the verifier that checked it; semantic causation — "α's write is why the check failed" — remains a claim for the adjudicator named in §8.4.1's 2-of-3 instantiation to weigh, not something the inclusion proof settles on its own. Theorem 8.2.2 is not false; it is scoped to the case its proof sketch actually covers, and this exercise is that scope made concrete.

Exercise p. 706

8.2

All three configurations share one module; only the `Rollback` and `Epoched` constants change. The committed results (`proofs/relay/CardRevocation.md`, "Results (TLC v1.8.0, Homebrew OpenJDK)"), quoted exactly:

```
1 baseline:  Model checking completed. No error has been found.   (13
    ↪ distinct states)
2 rollback:  Invariant RevocationFinal is violated.               (
    ↪ backup-restore re-accepts a revoked card)
3 epoch:     Model checking completed. No error has been found.   (76
    ↪ distinct states)
```

Baseline (`Rollback=FALSE, Epoched=FALSE, CardRevocation_baseline.cfg`): no adversary action is available, so a card in `revoked` stays revoked and `RevocationFinal` holds trivially over 13 reachable states. **Rollback** (`Rollback=TRUE, Epoched=FALSE, CardRevocation_rollback.cfg`): `RollbackRestore` can drop a card from the DB's `revoked` set while `everRevoked` (ground truth) stays set; `Accept` then only checks `revoked`, so the restored card is accepted again and `badAccept` fires — the invariant is violated *by design*, as the negative control for the attack. **Epoch** (`Rollback=TRUE, Epoched=TRUE, CardRevocation_epoch.cfg`): the mitigation adds an external epoch that `Revoke` bumps and that a restore does *not* roll back; `Accept` now additionally requires `cardEpoch[c] = epoch`, so a restored card's stale epoch disables acceptance even though `revoked` was rolled back, and the invariant holds again over the larger 76-state space (the mitigation adds reachable states without reintroducing the breach).

The rollback configuration tests whether a restored database can resurrect a revoked capability. A relay revokes card c (an agent's leaked capability); the relay's database is later restored from a backup taken before the revocation; the restored `revoked` set no longer contains c; the relay's own `Accept` check, consulting only that set, admits a presentation of c it had already promised to refuse. TLC's counterexample trace is exactly `Issue(c)` ⟶ `Revoke(c)` ⟶ `RollbackRestore(c)` ⟶ `Accept(c)`, four transitions, `badAccept` true at the end. The failure is the intended result because a baseline or an untested mitigation would give false comfort: it is precisely the negative control that lets the epoch configuration's clean pass mean something, the same discipline §8.3.4's negative-control ProVerif line and mutation suite apply. This chapter's own per-epoch roots $R_X^{(e)}$ (§8.0.3) are the same design move at the federation layer: an external, monotonic anchor that a local rollback cannot touch is what makes "only-issuer-revokes" something a cross-harbor verifier can actually rely on, rather than merely something the issuer asserts.

8.3

Exercise p. 706

By hand: C_6 has $R_{\text{eff}} = 5/6$, so $r = 3\sqrt{1 - 5/6} = 3/\sqrt{6} = 1.224745$ [verified]. P_6's relayed edge is a bridge (a path has no route between an edge's endpoints other than the edge itself), so $R_{\text{eff}} = 1$ and $r = 3\sqrt{1 - 1} = 0$ exactly [verified].

The script's own printed lines (`skills/harbor-results/scripts/sheaf_consistency_radius.py`, `section0`), quoted in the worked example above [internal, `sheaf_consistency_radius.py`]: the C_6 line's two computed quantities (the least-squares solve's r and the closed form's `pred`) are asserted equal to within 10^{-9} of each other and within 10^{-6} of 1.224745 (`check(abs(r - pred) < 1e-9 and abs(r - 1.224745) < 1e-6, …)`) — the numerical solve and the hand formula are checked against each other, not merely printed side by side. The P_6 line's assertion is different in kind: `check(rP < TOL_SILENT and abs(ReffP - 1.0) < 1e-9, …)` with `TOL_SILENT=` 10^{-9}, i.e. it asserts the residual is *below a silence threshold*, not equal to a specific float — the honest way to assert "zero" about a least-squares solve that carries ordinary floating-point roundoff rather than landing on the exact bit pattern for 0.0.

Exercise p. 706

8.4

r is defined as the *best-fit* residual: the smallest disagreement left over after choosing the global section x and the free severed blocks g_{sev} that explain as much of the observed data as any choice can. It is not a per-liar quantity; it is a property of the whole observed cochain g_K. If two equivocators on a common cycle choose offsets o_1 and o_2 such that $o_1 + o_2$ happens to be a coboundary — i.e. the *sum* of their two lies is indistinguishable from a single consistent state change — then the best-fit global section can absorb o_1+o_2 entirely, leaving a near-zero residual, exactly the way a lone liar's offset is absorbed when it lies off a visible cycle (the cut-edge case above). Nothing in the definition of r requires the offsets to be individually small for their *sum* to be small or zero; it requires only that the sum land in the image of δ, the space the completion can always explain away. This is why a one-edge example cannot establish completeness for arbitrary multiple offsets: the one-edge detection guarantee cannot be applied separately and then added. Soundness remains: $r > 0$ still certifies incompatibility of the encoded data. Completeness for arbitrary coordinated errors was never supplied; even one sender can choose several offsets that cancel in projection.

The compendium's own sweep states the shape of this exactly: a solo equivocator on an 8-cycle is bounded below at $r \geq |s|/\sqrt{8}$, while a coordinating coalition of two on the same cycle drives the same statistic to $r \approx 6 \times 10^{-15}$ — numerically the silence of a true coboundary, not a small-but-detectable residual [internal, `sheaf_consistency_radius.py`]. What could still catch the coalition is exactly what r is not: per-report attribution rather than a single scalar. Signature-level forensics can ask whether the *same* reporter's two prefixes are individually self-consistent (an equivocator still signs two incompatible things even if the aggregate looks clean); cross-checking against a third, independent visible cycle through only one of the two colluders can isolate that one's contribution if such a cycle exists; and widening K — comparing more edges directly rather than relying on relayed reports — shrinks the coalition's room to choose a cancelling pair in the first place. None of this is a proof that coalitions are caught in general; §8.10.3 names the cross-harbor cartel question as open for exactly this reason, and this exercise is that same boundary stated at the level of one theorem rather than the whole federation.

8.5

Exercise p. 707

The first receipt pins vertex 1 to B and the second to A. Their cost-five pair is infeasible; either receipt alone is feasible, so no cheaper subset certifies the contradiction. Real potentials can fit arbitrary differences on a path because there is no cycle constraint. One-hot vertex potentials cannot satisfy these incompatible pins. A zero edge only enforces equality, so both-A and both-B remain possible without other pins or raw claims.

8.6

Exercise p. 707

The four shares belong to the daemon, KMS, email provider, and passphrase holder. The process exposes the first three and computes a term using the public constructor `combine4`. A public constructor is also available to the attacker: anyone who learns all four arguments can construct their combination. Absence of a destructor prevents extracting shares from the result; it does not prevent forward computation.

The query instead names `account_root`, declared as a separate fresh private constant. No equation binds it to `combine4` of the shares. The conditional `if r = account_root then 0` makes no such binding and publishes nothing. Consequently the recorded

```
1 RESULT not attacker(account_root[]) is true.
```

is compatible with a model that would keep answering true even if all four shares were exposed. It proves secrecy of an unused constant, not the intended reconstruction boundary.

A meaningful model must query the reconstructed term, encode each claimed compromise scenario, and demonstrate an all-four-compromised negative control that reveals it. Exposing daemon, email, and KMS shares covers subsets of that particular exposure set; it does not alone test the other three-principal sets that include passphrase compromise. The Shamir recovery path requires its own encoding. The four-principal security claim remains unproved by this artifact.

8.7

Exercise p. 707

One model treats the two harbor principals as players, with their daemons executing the chosen policies. Actions include accept, defer, and reject a transfer; payoffs include completed-work value,

execution cost, delay, and enforceable losses. Each principal observes its local history and the signed remote evidence actually delivered to it. The model must specify a prior over hidden state and a delivery process; roots merely present in a log are not observations by both players.

For a complete-information stage-game simplification, let μ be the proposed joint distribution of actions. Coarse correlated equilibrium requires, for every player i and fixed alternative action a_i',

$$\mathbb{E}_{a\sim\mu}[u_i(a)] \geq \mathbb{E}_{a\sim\mu}[u_i(a_i', a_{-i})].$$

Here the alternative is chosen before seeing the recommendation. Correlated equilibrium also excludes profitable deviations conditioned on the player's recommendation. With private types or additional public signals, the relevant information and conditional deviations must be modeled explicitly; the displayed stage-game condition is not a proof for the richer protocol.

Gabriele Farina. Setting and Equilibria: The Correlated Equilibrium. Lecture 3, *Topics in Multiagent Learning*, MIT, 2024. Online source.

Fresh roots do not establish mutual receipt, common knowledge, or obedience. A proposed common-view event needs an explicit delivery and acknowledgement model, and its resulting information structure still needs an incentive check. If required evidence is missing or stale, defer the dependent cross-harbor effect; continue only independently authorized local work and preserve pending obligations under the settlement rules. This is a conservative authorization policy, not a proved equilibrium fallback. The federation result remains open.

Exercise p. 707

8.8

An HTLC's atomicity rests on a single objective, third-party-checkable bit: does a preimage of a fixed hash appear on-chain before a deadline. Both parties' payoffs are a deterministic function of that one bit, so the same contract executes identically for anyone who can read the chain. The Federated Harbor's damage predicate (§8.4.3's claim step) is not a single objective bit revealed by one party: whether α's action at B actually caused the acceptance-criteria failure is a semantic, evidence-weighing judgment over a work-unit's base and result trees, exactly the causation gap Exercise 8.1's solution names. A deterministic trustless contract must settle identically whenever its on-ledger inputs are identical; if two possible worlds produce the same signed evidence trail but differ in whether α was actually at fault, no deterministic on-ledger rule can be correct in both worlds at once. That is the shape of the

impossibility this exercise is asked to sketch, not merely a practical gap: it would need to assume exactly two worlds with identical observable transcripts and provably different ground truth, and show every candidate deterministic settlement rule is wrong in at least one of them. Progress, as §8.10.2 and §8.4.1's 2-of-3 candidate both note, means adding something outside pure on-ledger determinism — an oracle, a TEE, an optimistic dispute window, or a bonded human/arbiter adjudicator — each of which reintroduces a trust assumption rather than removing one. This chapter gives neither the construction nor the impossibility proof; it gives the escrow's conditional bound (Design Invariant 8.4.1) as the honest fallback while the question stays open.

8.9

Exercise p. 708

For $p \cdot d \cdot L > G$ to be satisfiable against a majority-but-not-total cartel, the audit mechanism needs an independent source of p and d that the cartel does not itself control: cross-harbor adversarial audits sampled from outside the colluding set, witness diversification so that a majority of witnesses being compromised still leaves an honest witness able to observe and attest to the true state, and principal-wide collateral so that L scales with what the colluding *principals* (not just their harbor identities) have at stake, per §8.10.3's list. Any lost continuation value belongs on the loss side L, while G is the gain from the deviation. Concavity alone supplies neither a quantitative loss nor a cap on that gain; both require an explicit payoff and re-entry model. All of this requires that at least one witness and one audit path lie genuinely outside the cartel's control — the mechanism's leverage comes entirely from an outside observer existing.

That is exactly why no structural mechanism closes the all-witnesses case: if the cartel controls every oracle and witness in scope, there is no outside observer left to supply p or d at all, and the inequality has no independent terms to be satisfied with. This is not a gap this chapter failed to close; it is the same limit every audit-based mechanism in the literature hits, stated honestly rather than argued around. §8.8's centralization-gravity discussion is the operational form of the same fact: the fewer independent witnesses a federation actually has, the closer it sits to the all-controlled boundary where no deterrence inequality can help.

Exercise p. 708

8.10

Start with what is observed. In a complete admission log for a stated time window, let n_s count successful admissions attributed to sponsor s, with one recorded sponsor per admission, and let $n = \sum_s n_s$. For $n > 0$, the sponsor shares n_s/n and their Herfindahl index $\sum_s (n_s/n)^2$ measure concentration among *successful* sponsorships. For $n = 0$, these quantities are undefined. They do not measure how many applicants were turned away. For example, the same log of ten admissions is consistent with ten requests and no refusals, or with a thousand requests and 990 refusals. The admission log in §8.5 does not distinguish those histories.

Acceptance and waiting-time measurements need additional records: stable request identifiers, first-request and decision timestamps, sponsor identifiers, and outcomes distinguishing admitted, refused, withdrawn, and still-pending requests. Define a request cohort and an observation cutoff before computing its admission and pending fractions; a pending request is not a refusal. Waiting time among admitted applicants describes a selected group and omits unfinished waits. Report that limitation and the pending fraction alongside it. A later admission by an independent sponsor, followed by no observed incident, can flag an earlier refusal for examination only if reapplications are linked and the follow-up window, terms, and observed risks are stated. It is not a measured false-rejection rate: changed terms or incomplete incident observation may explain the outcome.

Rising concentration or persistent entry disparities can motivate an audit, but neither those patterns nor their conjunction identifies an invitation cartel. Sponsor capacity, applicant risk, and admission-policy changes can produce similar records. Compare like cohorts, disclose missing requests and outcomes, and investigate the decisions behind an apparent disparity. The measurements support the engineering discipline of §8.10.4; they certify neither openness nor collusion.

Exercise p. 708

8.11

The mathematical task needs specified processes and comparable observables. Gossip tracks the delivery of signed records; linear diffusion tracks the change of a real-valued state. Specify the packet scheduler, loss and adversary model, a proposed correspondence

between those states, and the quantity to compare. A time-to-alarm claim also needs a visible signal, noise assumptions, a detection threshold and a stopping rule. A proof may establish a conditional relation, or a counterexample may refute a proposed one. Neither requires deployment traces as a logical prerequisite.

For a particular deployment, record timestamped sends, receipts and alarms alongside topology, fan-out, losses and partitions. State the observation window and retain runs with no delivery or alarm by its end. Such traces can estimate delay distributions and test whether the chosen model describes operation. An observed tail quantile is not by itself a guaranteed tail bound or a hard deadline; pricing propagation risk also requires justified assumptions and an account of estimation uncertainty.

Theorem 8.3.1's expected $\Theta(\log m)$ round count belongs to its complete-overlay, reliable synchronous model with independently uniform peer contacts. It supplies neither the proposed diffusion correspondence nor a deployment-specific tail guarantee. A model-based bound can be proved without field data, while using it operationally requires evidence that its hypotheses and parameters apply. The chapter establishes no transfer of the sheaf spectral-gap rate to signed append-only gossip.

8.12

Exercise p. 708

The design does not require one global total order over every harbor's events: each harbor maintains its own epoch history, and cross-witnessing can expose inconsistent signed views when those views meet. Accounting nevertheless needs an authoritative custody record and atomic, amount-preserving terminal transitions for each bond. The Cross-Harbor Bucket Partition (Design Invariant 8.4.2) states that obligation; its prose sketch is not an executable model-checking result. The available TLC transcript verifies the separate single-machine Conservation model at its finite configuration. Neither that transcript nor the cross-harbor partition proves deployment-level agreement or custody enforcement. An eventual cross-harbor model must specify the communication, failure, and atomicity assumptions under which those transitions can actually be implemented.

Calling the Federated Harbor "a project institution, not merely two machines sharing a repository" means the paper's actual claim is broader than any one protocol: an authoritative event/work-unit

graph, per-principal policy, local effect mediators, causal messages, evidence, roles, and explicit governance over conflicts, of which the transfer ceremony, revocation mesh, and settlement escrow are three essential pieces, not the whole structure. A roadmap or a stated intention is versioned forecast, not truth, and a contradiction detector (the witness log's equivocation check) produces evidence for a *named principal or role* to decide against, rather than deciding by itself. The paper's own posture — consensus only where it is cheap and structurally necessary (the small authoritative metadata core: epoch roots, admission records), federation everywhere else (the actual work artifacts) — is the concrete answer to the framing question of §8.0.4: two machines needed more than a shared repository, and what they needed is exactly the institution this chapter specifies one layer of.

Appendices

A Implementation boundary

The book's argument is broader than the running product. This appendix keeps the two from borrowing authority from one another. A mechanism receives the grade of its weakest necessary part; a verified library does not make every caller safe, and an available route does not prove that every effect passes through it.

A.1 Five assurance modes

Observed The act may occur elsewhere; the system retains best-effort evidence after the fact.

Coordinated Cooperative clients register claims, roles, commitments, and messages before acting.

Brokered A selected credential or effect channel passes through a daemon-owned broker.

Confined The process is restricted by operating-system primitives and cannot reach the protected effect through an ambient path.

Attested A measured environment conditions key release or acceptance on a verifiable runtime identity.

These modes are not a maturity staircase, and the rungs are not evenly spaced or even totally ordered — Brokered and Confined defeat different adversaries rather than the same adversary harder. What each rung names is what an adversary would have to defeat for a claim about that mechanism to be false: nothing at Observed, a client's willingness to declare at Coordinated, a daemon-owned broker at Brokered, the operating system at Confined, a measured environment and a conditional key release at Attested.

Two public gradings sit beside this one, and the chapter that buys these modes (*The Single-Writer Kernel*, §"What it buys") argues at length why all three axes are independent rather than versions of one another. Sheridan and Verplank's levels of automation — refined by Parasuraman, Sheridan and Wickens into four processing stages, each with its own level — grade how much of the decide-and-act loop the machine holds rather than the human. The Common Criteria's evaluation assurance levels, EAL1 through EAL7, grade how much independent evaluation effort a security claim received before deployment, and grade it once, for a whole product. This ladder grades neither work allocation nor evaluation effort. It grades one runtime claim about one mechanism, by what carries it.

A.2 What exists now

Local transaction authority — partial. The daemon runs a single-writer SQLite/WAL path for sessions, claims, messaging, commitments, and selected coordination state. Regression tests exercise those paths. Durability remains typed by failure class, schema sequencing is not the strongest design described in *The Single-Writer Kernel*, and filesystem or network effects outside mediated paths remain outside the transaction.

Legible projections — partial. The product derives operator projections from event and coordination records and retains routes back to underlying artifacts. No implementation can eliminate the information floor proved in *The Legible Swarm*; coverage, omission, and review latency must still be measured for each surface.

Durable work and obligation records — partial. Commitments, deadlines, messages, outcomes, and recovery evidence persist beyond one process. A universally portable execution capsule and provider-independent semantic restoration remain specified.

Authentication and attenuation — closed for the protocol model, partial for the code. ProVerif closes the recorded proof-of-possession and every-hop attenuation queries at the model boundary. The Kani harnesses over the deployed Rust crate are bounded: no panic in the parser on 32-byte inputs with the cryptography stubbed, no panic in the comparator, and two concrete vectors

for the subset check. Runtime callers still owe conformance: the verified primitive must be the primitive that every protected effect actually checks.

Sealed execution — specified. The sealed room of *The Sealed Harbor* is a design with model-checked properties. Noninterference modulo declassification and the release ledger's invariants are exhaustive on their finite models; ε-conservation, canary power, and Wald latency are theorems over the stated models. No shipping harbor yet runs a work order inside two fences with a metered slot: the attestation, the gate, and the ledger remain specified, and the *Confined* mode they would supply is not yet available to any caller.

Escrow and settlement — mixed. Ledger and conservation slices are partial. Cross-authority custody, adjudication, and 2-of-3 settlement remain specified and conditional on a non-bypassable custody interface.

Relay federation — partial. The current Relay carries authenticated events over outbound HTTPS and SSE. It does not replicate daemon databases, impose global order, authorize local effects, supply end-to-end secrecy by itself, or establish the witness-log and settlement institutions described in *The Federated Harbor*.

B Mechanized claims

This appendix is generated from the proof-estate manifest at `whitepaper/corpus.json`: 44 artifacts in total, 39 wired into continuous integration and 5 retired with a recorded reason — no third, silent state. One table follows per verification method; every row names the artifact, its CI job (or its retirement), the manifest's own status for it, and the evidence policy behind it.

Table A.1. Bounded Research Model Checkers artifacts (1 wired, 0 retired).

Claim	Artifact	CI	Status	Evidence policy
harbor-continuation-and-acquisition	skills/harbor-results/scripts/ *(2 files)* continuation_observation_model.py receipt_bundle_design.py	harbor-results-estate	current	—

Table A.2. Deterministic Result Checker artifacts (1 wired, 0 retired).

Claim	Artifact	CI	Status	Evidence policy
harbor-results-cr4-contract	skills/harbor-results/scripts/test_cr4_contract.py	harbor-results-estate	current	—

Table A.3. EasyCrypt artifacts (0 wired, 1 retired).

Claim	Artifact	CI	Status	Evidence policy
bonded-merkle-binding-easycrypt	proofs/bonded/merkle/binding.ec	retired	partial	EasyCrypt is not installed anywhere in this estate's CI; 3 'admit.' tactics remain undischarged (see binding.ec around lines 346, 371, 397 -- hashLeafE's algebraic combination). binding.run.log documents a manual proof-sketch review, not a machine-checked result.

Claim	Artifact	CI	Status	Evidence policy
harbor-categorical-evidence	skills/harbor-results/scripts/ *(6 files)* claim_audit.py test_claim_audit.py costed_claim_certificates.py test_costed_claim_certificates.py research_envelopes.py test_research_envelopes.py	harbor-results-estate	current	—

Table A.4. Exact Offline Audit And Finite Oracle Checks artifacts (1 wired, 0 retired).

Claim	Artifact	CI	Status	Evidence policy
harbor-card-kani-capability-attenuation	core/harbor-card-rs/src/lib.rs (proof_capability_attenuation)	kani-harbor-card	current	verify_capability_subset on two hard-coded cases (subset accepted, escalation rejected) -- concrete inputs, not arbitrary ones; never ran before wave-p1/proof-estate
harbor-card-kani-constant-time	core/harbor-card-rs/src/lib.rs (proof_constant_time_behavior)	kani-harbor-card	current	constant_time_compare never panics over arbitrary fixed-size input; never ran before wave-p1/proof-estate

Table A.5. Kani artifacts (3 wired, 0 retired).

(continued)

Claim	Artifact	CI	Status	Evidence policy
`harbor-card-kani-verify-logic-only`	`core/harbor-card-rs/src/lib.rs` (`proof_verify_logic_only`)	`kani-harbor-card-parser`	current	decode/verify-path robustness under adversarial input, with the crypto internals replaced by #[kani::stub(...)] stand-ins; requires -Z stubbing. Never ran before wave-p1/proof-estate --its #[kani::unwind(N)] bound had never been empirically validated (see the harness's own comment for the curve25519-dalek pow2k unwind story); runs on the nightly schedule (kani-harbor-card-parser) because it takes about an hour of CPU

Table A.6. Monte Carlo artifacts (1 wired, 3 retired).

Claim	Artifact	CI	Status	Evidence policy
pareto-simulation-baseline-monte-carlo	proofs/bonded/pareto/simulation.mjs	retired	current	exploratory Pareto-dominance sweep with a checked-in simulation.run.log from a local run; only threat-bands.mjs is asserted in CI (monte-carlo-threat-bands)
pareto-simulation-cartel-monte-carlo	proofs/bonded/pareto/simulation-cartel.mjs	retired	current	exploratory repeated-game cartel sweep with a checked-in simulation-cartel.run.log from a local run; only threat-bands.mjs is asserted in CI (monte-carlo-threat-bands)
pareto-simulation-sybil-monte-carlo	proofs/bonded/pareto/simulation-sybil.mjs	retired	current	exploratory Sybil-regime extension with a checked-in simulation-sybil.run.log from a local run; only threat-bands.mjs is asserted in CI (monte-carlo-threat-bands)
pareto-threat-bands-monte-carlo	proofs/bonded/pareto/threat-bands.mjs	monte-carlo-threat-bands	current	—

Table A.7. ProVerif artifacts (25 wired, 1 retired).

Claim	Artifact	CI	Status	Evidence policy
`anchor-delegation-chain-replay-proverif`	`proofs/anchor/delegation/chain-replay.pv`	`proverif-estate`	current	checked-in transcript regenerated 2026-09-06 with a pinned ProVerif 2.05 run (opam install proverif.2.05); RESULT lines verified to match byte-for-byte. Scope: non-injective correspondence for the accepted principal/agent/message tuple. The model checks issued nonces and inserts consumed_nonce but never checks the consumed table; it does not establish at-most-once acceptance or rejection of an identical-chain replay. Those require a separate guarded or injective model and runtime conformance.

(continued)

Claim	Artifact	CI	Status	Evidence policy
`anchor-token-verify-algconfusion-proverif`	`proofs/anchor/token-verify/algconfusion.pv`	`proverif-estate`	current	checked-in transcript regenerated 2026-09-06 against a real, pinned ProVerif 2.05 run (opam install proverif.2.05); RESULT lines verified to match byte-for-byte; a single model with two queries --the alg-pinned verifier authenticates (true) and the naive header-trusting verifier is forgeable (false) --so it is its own positive/negative control pair without matching the *_vuln*/ *naive_unsound* filename glob
`bonded-federated-escrow-proverif`	`proofs/bonded/federated/federated.pv`	`proverif-estate`	current	checked-in transcript regenerated 2026-09-06 against a real, pinned ProVerif 2.05 run (opam install proverif.2.05); RESULT lines verified to match byte-for-byte

(continued)

Claim	Artifact	CI	Status	Evidence policy
bonded-pairing-passkey-proverif	proofs/bonded/pairing/passkey-pair.pv	prover if-estate	current	checked-in transcript regenerated 2026-09-06 against a real, pinned ProVerif 2.05 run (opam install prover if.2.05); RESULT lines verified to match byte-for-byte
bonded-recovery-magic-link-proverif	proofs/bonded/recovery/magic-link.pv	prover if-estate	current	checked-in transcript regenerated 2026-09-06 against a real, pinned ProVerif 2.05 run (opam install prover if.2.05); RESULT lines verified to match byte-for-byte
coordination-isolation-proverif	proofs/coordination/isolation.pv	prover if-estate	current	checked-in transcript regenerated 2026-09-06 against a real, pinned ProVerif 2.05 run (opam install prover if.2.05); RESULT lines verified to match byte-for-byte

(continued)

Claim	Artifact	CI	Status	Evidence policy
`github-ingress-hpke-secrecy-proverif`	`analyses/github_ingress_hpke_secrecy.pv`	`proverif-estate`	current	checked-in transcript regenerated 2026-09-06 against a real, pinned ProVerif 2.05 run (opam install proverif.2.05); RESULT lines verified to match byte-for-byte
`github-ingress-hpke-secrecy-vuln-proverif`	`analyses/github_ingress_hpke_secrecy_vuln.pv`	`proverif-estate`	current	checked-in transcript regenerated 2026-09-06 against a real, pinned ProVerif 2.05 run (opam install proverif.2.05); RESULT lines verified to match byte-for-byte; negative control asserted to report an 'is false.' RESULT
`github-ingress-origin-auth-proverif`	`analyses/github_ingress_origin_auth.pv`	`proverif-estate`	current	checked-in transcript regenerated 2026-09-06 against a real, pinned ProVerif 2.05 run (opam install proverif.2.05); RESULT lines verified to match byte-for-byte

(continued)

Claim	Artifact	CI	Status	Evidence policy
`github-ingress-origin-auth-vuln-proverif`	`analyses/github_ingress_origin_auth_vuln.pv`	`proverif-estate`	current	checked-in transcript regenerated 2026-09-06 against a real, pinned ProVerif 2.05 run (opam install proverif.2.05); RESULT lines verified to match byte-for-byte; negative control asserted to report an 'is false.' RESULT
`github-ingress-tenant-isolation-proverif`	`analyses/github_ingress_tenant_isolation.pv`	`proverif-estate`	current	checked-in transcript regenerated 2026-09-06 against a real, pinned ProVerif 2.05 run (opam install proverif.2.05); RESULT lines verified to match byte-for-byte
`github-ingress-tenant-isolation-vuln-proverif`	`analyses/github_ingress_tenant_isolation_vuln.pv`	`proverif-estate`	current	checked-in transcript regenerated 2026-09-06 against a real, pinned ProVerif 2.05 run (opam install proverif.2.05); RESULT lines verified to match byte-for-byte; negative control asserted to report an 'is false.' RESULT

(continued)

Claim	Artifact	CI	Status	Evidence policy
`guidance-envelope-v0-proverif`	`proofs/coordination/guidance_envelope_v0.pv`	`proverif-estate`	current	Recorded verification: the checked-in transcript was regenerated on 2026-09-06 with ProVerif 2.05, and its RESULT lines matched byte-for-byte. The model is included in the proverif-estate check. The guidance-acceptance authority reads this transcript; a mismatch restricts that authority to C0. This is model and transcript evidence, not proof of a deployed enforcement path.

(continued)

Claim	Artifact	CI	Status	Evidence policy
`guidance-envelope-v0-unsigned-vuln-proverif`	`proofs/coordination/guidance_envelope_v0_unsigned_vuln.pv`	`proverif-estate`	current	Recorded verification: the checked-in transcript was regenerated on 2026-09-06 with ProVerif 2.05, and its RESULT lines matched byte-for-byte. This unsigned-envelope negative control must fail both agreement queries. A result claiming agreement would invalidate the control, not establish protocol security.

(continued)

Claim	Artifact	CI	Status	Evidence policy
`harbor-card-v1-hs256-proverif`	`analyses/harbor_card_v1.pv`	`proverif-estate`	current	checked-in transcript regenerated 2026-09-06 against a real, pinned ProVerif 2.05 run (opam install proverif.2.05); RESULT lines verified to match byte-for-byte --this run found the committed transcript had drifted from the current .pv source (an earlier hs256/alg-header framing vs. the current generic hmac + Revoker process); the truth value of both queries is unchanged ('true'), only ProVerif's internal fresh-variable numbering shifted as a result. Regenerated to match the current source.

(continued)

Claim	Artifact	CI	Status	Evidence policy
`harbor-card-v1-refined-alg-pinning-proverif`	`analyses/harbor_card_v1_refined.pv`	`proverif-estate`	current	no baseline existed before 2026-09-06; checked-in transcript established 2026-09-06 against a real, pinned ProVerif 2.05 run (opam install proverif.2.05); RESULT lines verified to match byte-for-byte
`harbor-card-v2-asymmetric-proverif`	`analyses/harbor_card_v2_asymmetric.pv`	`proverif-estate`	current	checked-in transcript regenerated 2026-09-06 against a real, pinned ProVerif 2.05 run (opam install proverif.2.05); RESULT lines verified to match byte-for-byte

(continued)

Claim	Artifact	CI	Status	Evidence policy
harbor-card-v3-delegation-proverif	analyses/harbor_card_v3_delegation.pv	proverif-estate	current	checked-in transcript regenerated 2026-09-06 against a real, pinned ProVerif 2.05 run (opam install proverif.2.05); RESULT lines verified to match byte-for-byte; proves ONE-HOP delegation soundness only --see harbor-card-v6/v7 for the multi-hop finding this model does not cover
harbor-card-v4-escrow-secrecy-proverif	analyses/harbor_card_v4_escrow_secrecy.pv	proverif-estate	current	checked-in transcript regenerated 2026-09-06 against a real, pinned ProVerif 2.05 run (opam install proverif.2.05); RESULT lines verified to match byte-for-byte
harbor-card-v5-attenuation-proverif	analyses/harbor_card_v5_attenuation.pv	proverif-estate	current	checked-in transcript regenerated 2026-09-06 against a real, pinned ProVerif 2.05 run (opam install proverif.2.05); RESULT lines verified to match byte-for-byte

(continued)

Claim	Artifact	CI	Status	Evidence policy
`harbor-card-v6-multihop-attack-proverif`	`analyses/harbor_card_v6_multihop_attack.pv`	`proverif-estate`	current	checked-in transcript regenerated 2026-09-06 against a real, pinned ProVerif 2.05 run (opam install proverif.2.05); RESULT lines verified to match byte-for-byte; red-team probe --proves a naive final<=root verifier ACCEPTS a non-monotonic multi-hop escalation (write->read->write). Refutes the Phase-3 claim in docs/reports/FORMAL_VERIFICATION_ANCHOR_V3.md; superseded by harbor-card-v7

(continued)

Claim	Artifact	CI	Status	Evidence policy
`harbor-card-v7-multihop-fixed-proverif`	`analyses/harbor_card_v7_multihop_fixed.pv`	`proverif-estate`	current	checked-in transcript regenerated 2026-09-06 against a real, pinned ProVerif 2.05 run (opam install proverif.2.05); RESULT lines verified to match byte-for-byte; the fix for harbor-card-v6 -- a per-hop each<=parent verifier rejects the same escalation
`macaroon-discharge-v1-proverif`	`analyses/macaroon_discharge_v1.pv`	`proverif-estate`	current	checked-in transcript regenerated 2026-09-06 against a real, pinned ProVerif 2.05 run (opam install proverif.2.05); RESULT lines verified to match byte-for-byte

(continued)

Claim	Artifact	CI	Status	Evidence policy
`macaroon-discharge-v2-naive-unsound-proverif`	`analyses/macaroon_discharge_v2_naive_unsound.pv`	`proverif-estate`	current	checked-in transcript regenerated 2026-09-06 against a real, pinned ProVerif 2.05 run (opam install proverif.2.05); RESULT lines verified to match byte-for-byte; negative control asserted to report an 'is false.' RESULT
`pd-relay-zero-trust-skill-template-proverif`	`skills/pd-relay-zero-trust/templates/proverif-relay.pv`	retired	historical	a skill scaffold with open TODO queries (capability containment, revocation effectiveness, non-equivocation are all unfinished); not a claim about a real Port Daddy protocol and has no committed evidence file

(continued)

Claim	Artifact	CI	Status	Evidence policy
`relay-e2e-secrecy-proverif`	`analyses/relay_e2e_secrecy.pv`	`proverif-estate`	current	checked-in transcript regenerated 2026-09-06 against a real, pinned ProVerif 2.05 run (opam install proverif.2.05); RESULT lines verified to match byte-for-byte; the injective (replay-free) SubAccepts<=PubSent correspondence is reported false with the weaker non-injective correspondence proved true instead --this is the committed, expected shape of this model's evidence, not new drift

Table A.8. Python / R scripts artifacts (1 wired, 0 retired).

Claim	Artifact	CI	Status	Evidence policy
harbor-results-r-scripts	skills/harbor-results/scripts/ *(7 files)* a3_epsilon_ledger.py a4_canary_sprt.py a6_no_mint.py a7_experiment.py b1_frontier.py b2_tower.py b3_controllability.py	harbor-results-estate	current	—
harbor-results-r-scripts *cont.*	skills/harbor-results/scripts/ *(7 files)* b4_deontic_fragment.py b5_engine_substitution.py b6_probation.py b7_escalation_band.py b8_specialization.py b9_context_paging.py c0_workunit.py	harbor-results-estate	current	—

(continued)

Claim	Artifact	CI	Status	Evidence policy
harbor-results-r-scripts *cont.*	skills/harbor-results/scripts/ *(7 files)* c1_noninterference.py sheaf_mechanism_proof.py sheaf_harness_v2.py paper7_failure_cases.py sheaf_consistency_radius.py sheaf_repair_and_2complex.py zoom_bound_check.py	harbor-results-estate	current	—
harbor-results-r-scripts *cont.*	skills/harbor-results/scripts/test_legibility_corrections.py	harbor-results-estate	current	—

Table A.9. TLA+/TLC artifacts (4 wired, 0 retired).

Claim	Artifact	CI	Status	Evidence policy
bonded-conservation-tla	proofs/bonded/conservation/ *(2 files)* Conservation.tla Conservation.cfg	tla-conservation	current	CI artifact per run, plus a checked-in Conservation.run.log snapshot from a prior local run

(continued)

Claim	Artifact	CI	Status	Evidence policy
`economics-claim-signaling-delta30-negative-control-tla`	`proofs/economics/claim_signaling_delta30.cfg`	`tla-claim-signaling`	current	negative control: CI asserts the invariant NoUnilateralDeviationPositive is violated at delta=0.30 and that the printed counterexample's trace-state count matches the committed proofs/economics/claim_signaling_delta30.run.log snapshot (6 states); this is the trace cited by the ex:bonded-tla-two-deltas solution in agent-transactions-whitepaper.tex

(continued)

Claim	Artifact	CI	Status	Evidence policy
`relay-card-revocation-tla`	`proofs/relay/` *(4 files)* `CardRevocation.tla` `CardRevocation_baseline.cfg` `CardRevocation_epoch.cfg` `CardRevocation_rollback.cfg`	`tla-relay-card-revocation`	current	Evidence is recorded in three .tlc.log artifacts per CI run, not a checked-in transcript. CardRevocation_rollback.cfg is the negative control: restoring an older database must produce a counterexample to RevocationFinal. The baseline and epoch configurations are checked separately; these bounded models do not establish that every deployment survives rollback.
`relay-webhook-delivery-tla`	`proofs/relay/` *(3 files)* `WebhookDelivery.tla` `WebhookDelivery.cfg` `WebhookDelivery_vuln.cfg`	`tla-relay-webhook-delivery`	current	CI artifact per run (two .tlc.log files); no prior checked-in snapshot --this model never ran before wave-p1/proof-estate. WebhookDelivery_vuln.cfg (Dedup=FALSE) is a negative control asserted to violate AtMostOnce

Table A.10. TLA+/TLC + Apalache artifacts (1 wired, 0 retired).

Claim	Artifact	CI	Status	Evidence policy
economics-claim-signaling-tla	proofs/economics/ *(2 files)* claim_signaling.tla claim_signaling.cfg	tla-claim-signaling, tla-claim-signaling-apalache	current	CI artifacts per run (TLC log and Apalache log, plus delta sweeps)

Table A.11. Z3 artifacts (1 wired, 0 retired).

Claim	Artifact	CI	Status	Evidence policy
economics-delta-threshold-z3	proofs/economics/delta-threshold.z3	z3-delta-threshold	current	expected sat/sat/unsat triple plus CI artifact; delta* interval independently cross-checked in proofs/economics/README.md and proofs/economics/delta-threshold.expected.txt

C Result atlas

The research archive contains revisions, experiments, literature work, and failed formulations. The durable unit is therefore a result, not a PDF. Each result below has a stable identifier, assumptions, a boundary, a chapter home, and a named verification artifact in the companion manifest.

C.1 The eight propositions and their evidence

1. Prevention begins at the effect boundary (R5, R8) — *The Single-Writer Kernel*. Controllability and sufficient observation together distinguish enforceable pre-effect rules from properties requiring other controls or post-effect remedies. The airlock experiments show why complete channel mediation is the decisive assumption.

A token, label, or policy document does not control an effect channel by itself.

2. Delegation only narrows (ANCHOR-ATTENUATION, R5) — *The Anchor Protocol*. Every hop must authenticate the issuer and reject a capability that expands scope or lifetime. The cryptographic models close the stated monotonicity property; the engineering obligation is universal use at protected channels.

3. Confinement is a property of the room, not of the tenant (R9, R10, R11) — *The Sealed Harbor*. Two owners who distrust each other compute one answer inside two fences with a single metered slot. Noninterference modulo declassification holds on the finite room model; the release ledger conserves its ε budget and refuses the torn write; canaries detect exfiltration with a stated power curve and a Wald clock. For q fixed-length b-bit releases, $q \cdot b$ bounds payload capacity before timing, and timing, cache, and physical channels are declared out of model rather than closed.

4. Supervision has an information cost (R1–R4, R14, R16) — *The Legible Swarm*. The information floor prices guaranteed recall; the dual-reader result shows why one ordering cannot generally serve both successor selection and operator attention; the regret rule derives intervention from asymmetric error cost; adaptive zoom and context paging describe when attention can be spent later rather than summarized away now. The bounds price guarantees under their stated event and flag models. They do not certify an arbitrary dashboard.

5. Accountability outlives the process (R7, R12, R13, B6) — *From Spawn to Person*. The inspection tower prices reputation as amortized verification. No-mint transfer semantics prevent skill or identity replacement from creating new authority. Budget-pooling and exposure arguments make reset laundering a property of the principal's case file, not the process name. The admission and exposure assumptions are essential: an authentic key alone does not prevent its holder from abandoning liabilities under a new key.

6. Exchange separates payment, collateral, evidence, and closure (R15, R17) — *The Harbor Economy*. Conservation supplies the accounting invariant; the tractable deontic fragment decides which rule sets can conflict and names the frontier past which that decision turns NP-complete; the succession rule has a price, because under the stated queue model a sole owner has a residual-downtime floor; comparison with a stable pooled queue gives a conditional pooling threshold. These are conditional economic statements. Detection probability, one-shot gain, collateral, judge independence, and custody must be measured or contracted rather than implied.

7. Uncertainty needs an institution (the δ^* claim-signaling game, the conservation model; R11 as instrument) — *The Bonded Commons*. Atomic ledgers preserve conservation; truthful claim signaling is incentive-compatible above the mechanized discount threshold δ^*; audit sampling and canaries borrow their power curve from the sealed room's detector; randomized, diverse adjudication reduces predictable collusion only under the named sampling assumptions. Evidence may support a remedy without becoming mechanical truth.

8. Federation relays evidence, not sovereignty (R6, R6-CR, RELAY-V0) — *The Federated Harbor*. Signed roots and redundant comparison paths can expose equivocation across a specified topology. The counterexample R6-CR rejects the tempting claim that a zero local residual proves global consistency. Current Relay establishes event transport and authentication; it does not close witness, secrecy, liveness, custody, or local authorization obligations.

C.2 A result is admitted only with its limit

For this volume, a claim is incomplete unless it states all five of the following: the object it concerns, the assumptions under which it holds, the kind of evidence offered, a boundary or falsifier, and the artifact by which a reader can recompute or inspect it. The machine-readable manifest is the publication ledger for that discipline. New mechanisms belong in the book only after they can name the result they instantiate and the obligation they leave unresolved.

D Notation and cross-chapter concordance

Work unit The durable case file joining authorization, capability history, evidence, obligations, outcomes, and settlement.

Harbor One local administrative and transactional authority, normally one daemon and its durable database.

Principal The human or organization whose policy and resources an agent ultimately exercises; not interchangeable with a process identity.

Role A bundle of obligations, capabilities, and authority that a spawn may fill.

Person An accountable identity with continuity and a witnessed outcome history; roles may change without creating a new accountable principal. Its anti-reset properties depend on admission and liability rules.

Oracle An external or independently checkable reference used to close a commitment or adjudicate an outcome. Its trust model must be named.

Epoch A version or freshness counter, with chapter-specific meaning. Chapter 2 uses revocation epochs; Chapter 3 uses attestation freshness to detect rollback. A credential's time-to-live is a validity duration, not another epoch counter. None substitutes for the others.

Round A model step in a dissemination or protocol analysis, not automatically a wall-clock deadline.

Closed The exact recorded property has been mechanically discharged. No whole-system implication follows without a conformance argument.

Which chapter owns which primitive

Each shared primitive has a canonical definition in its owning chapter. Later chapters recall the relevant part in context and apply it to a new example; readers should not have to retrieve a distant definition merely to follow a paragraph. Those reminders must preserve the same contract. A genuine extension states the added assumptions explicitly. The table identifies where to resolve a conflict, not a prohibition on useful repetition.

Table A.12. Primitive ownership. "Owner" defines it and proves what is proved about it; "cites" names the chapters that use it without redefining it.

Primitive	Owner	Cites it
Authorization chain (attenuation, hop-binding, replay)	ch. 2, Anchor Protocol	ch. 1, 5, 7, 8
Coordination lineage (task graph, loop detection)	ch. 1, Single-Writer Kernel	ch. 4
Leakage budget and the priced exit	ch. 3, Sealed Harbor	ch. 4, 6
Float plan	ch. 6, Harbor Economy	ch. 7, 8
Bond, escrow, and the commons pool	ch. 7, Bonded Commons	ch. 6, 8
Cross-harbor transfer ceremony	ch. 8, Federated Harbor	ch. 6
Witness log, epoch root, equivocation residual	ch. 8, Federated Harbor	ch. 2, 7
Threat bands and the visibility tiers	ch. 8, Federated Harbor	ch. 3
Keystone identity (non-forgeable principal)	ch. 5, From Spawn to Person	ch. 6, 7
Inspection tower and grader escalation	ch. 5, From Spawn to Person	ch. 1, 6

Routes to the open problems

Open problems are numbered in three separate namespaces on purpose — chapter 1's OP-n is the kernel's, chapter 5 declines to extend it because its questions are not kernel questions, and chapter 8 states its own in prose — and this table gives a thematic route

to the owning lists, not an exhaustive enumeration or a new numbering. The owning chapter's statement remains the source for assumptions and status. An incentive result about graders does not close a question about whether their observations are complete.

Table A.13. Where the unresolved questions live. Related results can narrow a question without closing it.

Where	Problem	What bears on it elsewhere
ch. 1, OP-1–OP-11	The kernel's eleven, stated once in its own status table; OP-4 splits into OP-4a (durable lineage replay) and OP-4b (inference-state recovery).	OP-4b is graded on ch. 5's agent-death ladder, rung 5; OP-5 (oracle *completeness*) is untouched by ch. 5's tower, which is about oracle *incentives*.
ch. 5	Its own list, deliberately outside the kernel's numbering: death taxonomy, neutral graded outcomes, judge conflict of interest, correlated auditors.	The correlated-auditor question shares ch. 6's mixing model and its unmeasured θ.
ch. 8	Deployment-specific propagation tails; the relationship between sheaf diffusion and anti-entropy gossip.	No spectral alarm-time law is established. A tree can have a positive diffusion gap and no detectable cycle residual. Visibility and a noise threshold must enter any proposed law.
Cross-chapter	Claim signalling under *private* monitoring: ch. 7 assumes both players see the same action history before the next round.	One ledger does not ensure timely shared observation. Gossip, hidden actions, resettable identities, and delayed views need their own incentive analysis.

Image credits

- **Thomas Hobbes, portrait**: after John Michael Wright, National Portrait Gallery, NPG 106, copy after an original of c. 1669–1670. Wikimedia Commons, *Thomas Hobbes (portrait).jpg*; public domain, https://creativecommons.org/publicdomain/mark/1.0/. Reproduced without cropping or retouching.
- **Amartya Sen**: Fronteiras do Pensamento, *Amartya Sen no Fronteiras do Pensamento São Paulo 2012*, Flickr photo 6964218850, via Wikimedia Commons. CC BY-SA 2.0, https://creativecommons.org/licenses/by-sa/2.0/; the photograph remains under that licence. No cropping or retouching.
- **Jonathan Hickman**: Pat Loika, Comic-Con 2011, 24 July 2011, Flickr photo 5976523290, via Wikimedia Commons. CC BY 2.0, https://creativecommons.org/licenses/by/2.0/. No cropping or retouching; inclusion does not imply endorsement.
- **Probable printing tile or stamp**: Birmingham Museums Trust, photographed by Teresa Gilmore, *Post medieval: Probable Printing tile or stamp*, FindID 973303, via Wikimedia Commons. CC BY 2.0, https://creativecommons.org/licenses/by/2.0/. Face and reverse of one object; no cropping or retouching.
- **Leslie Lamport**: photograph by Andrej Bauer, Wikimedia Commons (*Leslie Lamport September 2006.jpg*), CC BY-SA 2.5 si, https://creativecommons.org/licenses/by-sa/2.5/si/deed.en; shared under the same licence; cropped and converted to duotone for this edition.
- **Elinor Ostrom**: photograph by Holger Motzkau, Wikimedia Commons (*Elinor Ostrom close-up (cropped).jpg*), © Holger Motzkau 2010, Wikipedia/Wikimedia Commons, CC BY-SA 3.0, https://creativecommons.org/licenses/by-sa/3.0; shared under the same licence; cropped and converted to duotone for this edition.

- **Robert Aumann**: photograph by David Orban, Wikimedia Commons (*Aumann, Robert (1930).jpg*, originally posted to Flickr), CC BY 2.0, `https://creativecommons.org/licenses/by/2.0`; cropped and converted to duotone for this edition.
- **Butler Lampson**: photograph by the Royal Society, Wikimedia Commons (*Butler Lampson Royal Society.jpg*), CC BY-SA 4.0, `https://creativecommons.org/licenses/by-sa/4.0`; shared under the same licence; cropped and converted to duotone for this edition.
- **Derek Parfit**: photograph by Anna Riedl, Wikimedia Commons (*Derek Parfit at Harvard-April 21, 2015-Effective Altruism.jpg*), CC BY-SA 4.0, `https://creativecommons.org/licenses/by-sa/4.0`; shared under the same licence; cropped and converted to duotone for this edition.
- **Thomas Hobbes, *Leviathan* (1651) title page**: engraving by Abraham Bosse, digitised by the British Library, Wikimedia Commons (*Leviathan - Hobbes' Leviathan (1651), title page - BL.jpg*), public domain (CC0 Universal Public Domain Dedication), `http://creativecommons.org/publicdomain/zero/1.0/deed.en`; cropped and converted to duotone for this edition.
- **James C. Scott**: photograph by SOAS University of London, Wikimedia Commons (*James C Scott 2016.jpg*), CC BY 3.0, `https://creativecommons.org/licenses/by/3.0`; cropped and converted to duotone for this edition.
- **W. Murray Wonham**: photograph by Wikimedia Commons user "Dal centro della mia vita venne una grande fontana" (*Nonoficial.jpg*, the uncropped source of *Wonham.jpg*), CC BY-SA 4.0, `https://creativecommons.org/licenses/by-sa/4.0`; shared under the same licence; the source is a group photograph captioned "Professor Wonham with students" and this edition's plate is cropped down to Wonham alone, then converted to duotone; the crop is small and enlarged for print, so definition is soft.
- **Abraham Wald**: photograph by Konrad Jacobs, Mathematisches Forschungsinstitut Oberwolfach (Oberwolfach Photo Collection), `https://opc.mfo.de/detail?photo_id=4398`, CC BY-SA 2.0 de, `https://creativecommons.org/licenses/`

by-sa/2.0/de/deed.en; shared under the same licence; converted to duotone for this edition.

- **Claude Shannon**: photograph by Konrad Jacobs, Mathematisches Forschungsinstitut Oberwolfach (Oberwolfach Photo Collection), https://opc.mfo.de/detail?photo_id=3807, CC BY-SA 2.0 de, https://creativecommons.org/licenses/by-sa/2.0/de/deed.en; shared under the same licence; converted to duotone for this edition.
- **Ada Lovelace**: watercolour painting by Alfred Edward Chalon, Science Museum Group, Wikimedia Commons (*Ada Lovelace portrait.jpg*), public domain; cropped and converted to duotone for this edition.